

Kotlin

В ДЕЙСТВИИ

ВТОРОЕ ИЗДАНИЕ

Себастьян Айгнер
Роман Елизаров
Светлана Исакова
Дмитрий Жемеров



Kotlin in Action

SECOND EDITION

SEBASTIAN AIGNER, ROMAN ELIZAROV,
SVETLANA ISAKOVA, DMITRY JEMEROV



MANNING

SHELTER ISLAND

Kotlin в действии

ВТОРОЕ ИЗДАНИЕ

СЕБАСТЬЯН АЙГНЕР, РОМАН ЕЛИЗАРОВ,
СВЕТЛАНА ИСАКОВА, ДМИТРИЙ ЖЕМЕРОВ



Санкт-Петербург • Москва • Минск

2025

ББК 32.973.2-018.1
УДК 004.43
К73

**Айгнер Себастьян, Елизаров Роман, Исакова Светлана,
Жемеров Дмитрий**

K73 Kotlin в действии. 2-е изд. — СПб.: Питер, 2025. — 560 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-4198-2

Kotlin — простой и высокопроизводительный язык программирования, достаточно гибкий для работы с любыми веб-, мобильными, облачными и корпоративными приложениями. Разработчики приложений на Java по достоинству оценят простой синтаксис, интуитивно понятную систему типов, набор превосходных инструментов и поддержку функционального программирования. Кроме того, поскольку Kotlin работает на JVM, он легко интегрируется с существующим Java-кодом, библиотеками и фреймворками, включая Spring и Android.

Во второе издание бестселлера «Kotlin в действии» добавлено описание корутин, структурированного параллелизма и других новых возможностей языка. Это авторитетное руководство, написанное основными членами команды разработки языка Kotlin, представляет полезные методы использования стандартной библиотеки Kotlin, функционального программирования и расширенных возможностей, таких как обобщенное программирование и рефлексия. Проще говоря, это самая полная и точная книга по Kotlin из всех доступных.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018.1
УДК 004.43

Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1617299605 англ.

© Authorized translation of the English edition © 2023 Manning Publications.
This translation is published and sold by permission of Manning Publications,
the owner of all rights to publish and sell the same.

ISBN 978-5-4461-4198-2

© Перевод на русский язык ООО «Прогресс книга», 2025
© Издание на русском языке, оформление ООО «Прогресс книга», 2025
© Серия «Библиотека программиста», 2025

Краткое содержание

Часть I. Введение в Kotlin

Глава 1. Kotlin. Что это за язык и зачем он нужен	28
Глава 2. Основы языка Kotlin	52
Глава 3. Определение и вызов функций	90
Глава 4. Классы, объекты и интерфейсы	119
Глава 5. Программирование с использованием лямбда-выражений.	165
Глава 6. Работа с коллекциями и последовательностями	192
Глава 7. Работа с nullable-значениями	218
Глава 8. Базовые типы, коллекции и массивы	243

Часть II. Освоение Kotlin

Глава 9. Перегрузка операторов и другие соглашения	268
Глава 10. Функции высшего порядка: лямбды в качестве параметров и возвращаемых значений	301
Глава 11. Обобщения	328
Глава 12. Аннотации и рефлексия	367
Глава 13. Создание DSL	399

Часть III. Конкурентное программирование, корутины и потоки

Глава 14. Корутины	431
Глава 15. Структурированная конкурентность.	457
Глава 16. Потоки.	480
Глава 17. Операторы потока	506
Глава 18. Обработка ошибок и тестирование	521

Приложения

Приложение А. Создание проектов Kotlin	545
Приложение Б. Документирование кода Kotlin	549
Приложение В. Экосистема Kotlin	553

Оглавление

Предисловие	18
Благодарности	20
О книге.	21
Для кого эта книга	21
Структура издания	21
Условные обозначения	23
Другие интернет-ресурсы	24
От издательства	24
Об авторах.	25
Иллюстрация на обложке	26

Часть I. Введение в Kotlin

Глава 1. Kotlin. Что это за язык и зачем он нужен	28
1.1. Знакомство с Kotlin	29
1.2. Основные характеристики Kotlin	30
1.2.1. Варианты использования Kotlin: Android, серверная часть, везде, где работает Java, и не только	31
1.2.2. Статическая типизация повышает производительность, надежность и удобство сопровождения	31
1.2.3. Сочетание функционального и объектно-ориентированного программирования делает Kotlin безопасным и гибким	33
1.2.4. Конкурентный и асинхронный код становится естественным и структурированным при использовании корутин	35
1.2.5. Kotlin можно использовать для любых целей: он бесплатный и имеет открытый исходный код.	36
1.3. Области, в которых часто используется Kotlin	37
1.3.1. Обеспечение работы бэкенда — разработка серверной части ПО на Kotlin	37
1.3.2. Разработка мобильных приложений. Kotlin — первоочередной вариант для Android.	40
1.3.3. Мультиплатформенность — совместное использование бизнес-логики и минимизация дублирования работы на iOS, JVM, JS и не только	41
1.4. Философия Kotlin	42
1.4.1. Прагматичность.	42
1.4.2. Лаконичность	43
1.4.3. Безопасность.	44
1.4.4. Совместимость	45

1.5. Использование инструментов Kotlin	47
1.5.1. Настройка и запуск кода Kotlin	47
1.5.2. Компиляция кода Kotlin	48
Резюме.	50
Глава 2. Основы языка Kotlin	52
2.1. Основные элементы — функции и переменные.	53
2.1.1. Написание первой программы на языке Kotlin: «Hello, world!».	53
2.1.2. Объявление функций с параметрами и возвращаемыми значениями . .	54
2.1.3. Сокращенные определения функций с помощью тел выражений	56
2.1.4. Объявление переменных для хранения данных.	58
2.1.5. Переназначаемые переменные и переменные только для чтения	58
2.1.6. Упрощенное форматирование строк с помощью шаблонов	60
2.2. Инкапсуляция поведения и данных — классы и свойства	62
2.2.1. Ассоциирование данных с классом и обеспечение доступа к ним: свойства	63
2.2.2. Вычисление свойств вместо хранения их значений: переопределенные методы доступа	65
2.2.3. Компоновка исходного кода Kotlin: каталоги и пакеты	66
2.3. Представление и обработка вариантов выбора: перечисления и выражение when	68
2.3.1. Объявление классов и констант перечислений	68
2.3.2. Обработка классов перечислений с помощью выражения when.	70
2.3.3. Захват субъекта выражения when в переменную	71
2.3.4. Использование выражения when с произвольными объектами	72
2.3.5. Использование выражения when без аргумента.	73
2.3.6. Умное преобразование: комбинирование проверки и приведения типов	74
2.3.7. Рефакторинг: замена выражения if на when	77
2.3.8. Блоки как ветви конструкций if и when	78
2.4. Итерации над элементами: циклы while и for	79
2.4.1. Повторение кода, пока условие истинно: цикл while.	79
2.4.2. Итерация над числами: диапазоны и прогрессии	80
2.4.3. Итерации по картам	81
2.4.4. Оператор in для проверки коллекции и принадлежности к диапазону . .	83
2.5. Выбрасывание и перехват исключений в Kotlin	85
2.5.1. Обработка исключений и восстановление после ошибок: try, catch и finally	85
2.5.2. Использование блока try в качестве выражения	87
Резюме.	88
Глава 3. Определение и вызов функций	90
3.1. Создание коллекций в Kotlin	91
3.2. Упрощение вызова функций	92
3.2.1. Именованные аргументы	93
3.2.2. Значения параметров по умолчанию.	94
3.2.3. Избавление от статических служебных классов: функции верхнего уровня и свойства	96

3.3. Добавление методов к чужим классам: функции-расширения и свойства . . .	99
3.3.1. Импорт и функции-расширения	101
3.3.2. Вызов функций-расширений из Java.	101
3.3.3. Служебные функции в качестве расширений	102
3.3.4. Недопустимость переопределения для функций-расширений	103
3.3.5. Свойства-расширения	105
3.4. Работа с коллекциями: vararg, инфиксные вызовы и поддержка библиотек	106
3.4.1. Расширение API коллекций Java	106
3.4.2. Vararg — функции, принимающие произвольное количество аргументов	107
3.4.3. Работа с парами: инфиксные вызовы и объявления деструктуризации	108
3.5. Работа со строками и регулярными выражениями.	109
3.5.1. Разбиение строк.	109
3.5.2. Регулярные выражения и строки в тройных кавычках	110
3.5.3. Многострочные строковые значения с тройными кавычками	112
3.6. Аккуратный код: локальные функции и расширения	115
Резюме.	117
Глава 4. Классы, объекты и интерфейсы.	119
4.1. Определение иерархии классов.	120
4.1.1. Интерфейсы в Kotlin.	120
4.1.2. Модификаторы open, final и abstract: final по умолчанию.	123
4.1.3. Модификаторы видимости: public по умолчанию	126
4.1.4. Внутренние и вложенные классы. По умолчанию вложенные.	129
4.1.5. Запечатанные классы: определение ограниченных иерархий классов	131
4.2. Объявление класса с нетривиальными конструкторами или свойствами.	133
4.2.1. Инициализация классов: первичные конструкторы и блоки инициализатора	133
4.2.2. Вторичные конструкторы: инициализация суперкласса различными способами.	136
4.2.3. Реализация свойств, объявленных в интерфейсах	139
4.2.4. Доступ к теневому полю из геттера или сеттера.	141
4.2.5. Изменение видимости метода доступа	142
4.3. Методы, генерируемые компилятором. Классы данных и делегирование классов	144
4.3.1. Универсальные методы объектов.	144
4.3.2. Классы данных: автоматически генерируемые реализации универсальных методов	147
4.3.3. Делегирование класса: использование ключевого слова by.	149
4.4. Ключевое слово object: объявление класса и создание его экземпляра	151
4.4.1. Объявления объектов: объекты-одиночки — это просто	152
4.4.2. Объекты-компаньоны: место для фабричных методов и статических членов класса	154
4.4.3. Объекты-компаньоны в качестве обычных объектов	157

4.4.4. Объектные выражения: перефразирование анонимных внутренних классов	160
4.5. Дополнительная безопасность типов без дополнительных затрат: встроенные классы	161
Резюме.	164
Глава 5. Программирование с использованием лямбда-выражений.	165
5.1. Лямбда-выражения и ссылки на члены класса без вызова	166
5.1.1. Введение в лямбда-выражения: блоки кода как значения	166
5.1.2. Лямбды и коллекции	167
5.1.3. Синтаксис лямбда-выражений	169
5.1.4. Доступ к переменным в области видимости	174
5.1.5. Ссылки на члены класса без вызова	176
5.1.6. Связанные вызываемые ссылки.	178
5.2. Использование функциональных интерфейсов Java — Single Abstract Method (SAM)	179
5.2.1. Передача лямбды в качестве параметра в метод Java.	180
5.2.2. Конструкторы SAM: явное преобразование лямбд в функциональные интерфейсы	181
5.3. Определение функциональных интерфейсов в Kotlin: fun interface	183
5.4. Лямбды с приемниками: with, apply и also	185
5.4.1. Выполнение нескольких операций над одним объектом: функция with	185
5.4.2. Инициализация и конфигурирование объектов: функция apply	188
5.4.3. Выполнение дополнительных действий с объектом: функция also	190
Резюме.	191
Глава 6. Работа с коллекциями и последовательностями	192
6.1. Функциональные API для коллекций	192
6.1.1. Удаление и преобразование элементов: функции filter и map	193
6.1.2. Накопление значений для коллекций: функции reduce и fold	196
6.1.3. Применение предиката к коллекции: функции all, any, none, count и find	198
6.1.4. Разбиение одного списка на два: функция partition	201
6.1.5. Преобразование списка в карту групп: функция groupBy.	202
6.1.6. Преобразование коллекций в карты: функции associate, associateWith и associateBy	203
6.1.7. Замена элементов в изменяемых коллекциях: функции replaceAll и fill.	205
6.1.8. Обработка особых случаев для коллекций: функция isEmpty	205
6.1.9. Разбиение коллекций на части: функции chunked и windowed	206
6.1.10. Объединение коллекций: функция zip	208
6.1.11. Обработка элементов во вложенных коллекциях: функции flatMap и flatten	209
6.2. Отложенные операции коллекций: последовательности	211
6.2.1. Выполнение последовательных операций: промежуточные и терминальные операции	212
6.2.2. Создание последовательностей	216
Резюме.	217

Глава 7. Работа с nullable-значениями	218
7.1. Предотвращение <code>NullPointerException</code> и обработка отсутствия значений: null-безопасность	218
7.2. Явное определение допустимости null-переменных с помощью типов, допускающих значение null	219
7.3. Более пристальное изучение значения типов	221
7.4. Комбинирование проверок null и вызовов методов с помощью оператора безопасного вызова <code>?.</code>	223
7.5. Предоставление значений по умолчанию в null-случаях с помощью оператора <code>?:</code>	225
7.6. Безопасное приведение значений без выброса исключений: оператор <code>as?</code>	227
7.7. Обещания компилятору с помощью оператора утверждения «не null»: <code>!!</code>	228
7.8. Работа с выражениями, допускающими значение null: функция <code>let</code>	230
7.9. «Не null-типы» с поздней инициализацией: свойства с отложенной инициализацией	233
7.10. Расширение типов без оператора безопасного вызова: расширения для типов, допускающих значение null	235
7.11. Null-безопасность типовых параметров	237
7.12. Null-безопасность и Java	238
7.12.1. Платформенные типы	239
7.12.2. Наследование	241
Резюме	242
Глава 8. Базовые типы, коллекции и массивы	243
8.1. Прimitives и другие базовые типы	243
8.1.1. Представление целых чисел, чисел с плавающей запятой, символов и логических значений с помощью примитивных типов	244
8.1.2. Использование всего диапазона битов для представления положительных чисел: типы беззнаковых чисел	245
8.1.3. Примитивные типы, допускающие значение null: <code>Int?</code> , <code>Boolean?</code> и другие	246
8.1.4. В Kotlin преобразования чисел стали явными	248
8.1.5. Типы <code>Any</code> и <code>Any?</code> : корень иерархии типов Kotlin	250
8.1.6. Тип <code>Unit</code> — <code>void</code> для Kotlin	251
8.1.7. Тип <code>Nothing</code> : «Эта функция никогда не возвращает результат»	252
8.2. Коллекции и массивы	253
8.2.1. Коллекции nullable-значений и коллекции, допускающие значение null	253
8.2.2. Коллекции только для чтения и изменяемые коллекции	256
8.2.3. Коллекции Kotlin и Java глубоко связаны между собой	258
8.2.4. Объявленные в Java коллекции в Kotlin воспринимаются как платформенные типы	261
8.2.5. Создание массивов объектов и примитивные типы для обеспечения совместимости и производительности	263
Резюме	266

Часть II. Освоение Kotlin

Глава 9. Перегрузка операторов и другие соглашения	268
9.1. Перегрузка арифметических операторов делает операции для произвольных классов более удобными.	269
9.1.1. Функции plus, times, divide и другие: перегрузка операций двоичной арифметики	269
9.1.2. Применение операции и немедленное присвоение ее значения: перегрузка составных операторов присваивания.	273
9.1.3. Операторы с одним операндом: перегрузка унарных операторов.	274
9.2. Перегрузка операторов сравнения упрощает проверку отношений между объектами.	275
9.2.1. Операторы равенства: equals (==)	276
9.2.2. Упорядочение операторов: compareTo (<, >, ? и >=).	277
9.3. Соглашения для коллекций и диапазонов	278
9.3.1. Доступ к элементам по индексу: соглашения для get и set	279
9.3.2. Проверка принадлежности объекта к коллекции: соглашение in	280
9.3.3. Создание диапазонов из объектов: соглашения rangeTo и rangeUntil	281
9.3.4. Создание возможности циклического перехода по типам: соглашение iterator	283
9.4. Создание объявлений деструктуризации с помощью компонентных функций	284
9.4.1. Объявления деструктуризации и циклы	286
9.4.2. Игнорирование деструктурированных значений с помощью символа _	287
9.5. Повторное использование логики методов доступа к свойствам: делегированные свойства	289
9.5.1. Основной синтаксис и работа делегированных свойств.	289
9.5.2. Использование делегированных свойств: отложенная инициализация и выполняемая с помощью функции lazy().	290
9.5.3. Реализация собственных делегированных свойств.	292
9.5.4. Делегированные свойства переводятся в скрытые свойства с пользовательскими методами доступа	296
9.5.5. Доступ к динамическим атрибутам путем делегирования карт	297
9.5.6. Как использовать делегированные свойства во фреймворке.	298
Резюме.	300
Глава 10. Функции высшего порядка: лямбды в качестве параметров и возвращаемых значений.	301
10.1. Объявление функций, возвращающих или получающих другие функции: функции высшего порядка	302
10.1.1. Типы функций определяют типы параметров и возвращаемых значений лямбды.	302
10.1.2. Вызов функций, переданных в качестве аргументов	304
10.1.3. Лямбды Java автоматически преобразуются в типы функций Kotlin.	306
10.1.4. Параметры с типами функций могут задавать значения по умолчанию или допускать значение null	307

10.1.5. Возврат функций из функций	310
10.1.6. Повышение удобства использования кода за счет сокращения дублирования с помощью лямбд.	312
10.2. Устранение накладных расходов на лямбды с помощью встроенных функций	314
10.2.1. Встраивание означает подстановку тела функции в каждую точку вызова	315
10.2.2. Ограничения для встроенных функций	317
10.2.3. Встраивание операций с коллекциями.	318
10.2.4. Когда объявлять функции как встроенные	320
10.2.5. Использование встроенных лямбд для управления ресурсами с помощью функций withLock, use и useLines.	320
10.3. Возврат из лямбд. Поток управления в функциях высшего порядка	323
10.3.1. Выражения return в лямбдах: возврат из замкнутых функций.	323
10.3.2. Возврат из лямбд: оператор return с меткой	324
10.3.3. Анонимные функции: локальные операторы return по умолчанию.	326
Резюме	327
Глава 11. Обобщения	328
11.1. Создание типов с аргументами типа — типовые параметры обобщений	329
11.1.1. Функции и свойства, работающие с обобщенными типами.	330
11.1.2. Обобщенные классы объявляются с помощью синтаксиса угловых скобок.	332
11.1.3. Ограничение типа, который могут использовать обобщенный класс или функция: ограничения типовых параметров.	333
11.1.4. Исключение аргументов типа, допускающих значение null, путем явного обозначения типовых параметров как «не null»	335
11.2. Обобщения во время выполнения: стирание и параметры вещественного типа	337
11.2.1. Ограничения на поиск информации о типе обобщенного класса во время выполнения: проверка и приведение типов	338
11.2.2. Функции с параметрами вещественного типа могут ссылаться на фактические аргументы типа во время выполнения.	341
11.2.3. Исключение параметров класса java.lang.Class путем замены ссылок на классы параметрами вещественного типа	343
11.2.4. Объявление методов доступа с параметрами вещественного типа	345
11.2.5. У параметров вещественного типа есть ограничения.	345
11.3. Вариативность описывает отношения подтипизации между обобщенными аргументами	346
11.3.1. Вариативность определяет, безопасно ли передавать аргумент в функцию.	346
11.3.2. Изучение различий между классами, типами и подтипами.	347
11.3.3. Ковариантность сохраняет отношение подтипизации	350
11.3.4. Контравариантность меняет отношение подтипизации на противоположное	353
11.3.5. Определение вариативности встречающихся типов через вариативность точек использования	356

11.3.6. «Звездная» проекция: использование символа * для указания на недостаток информации об обобщенном аргументе	360
11.3.7. Псевдонимы типов	364
Резюме	366
Глава 12. Аннотации и рефлексия	367
12.1. Объявление и применение аннотаций	368
12.1.1. Применение аннотаций для меток объявлений	368
12.1.2. Указание точного объявления, на которое ссылается аннотация: цели аннотации	370
12.1.3. Использование аннотаций для настройки сериализации JSON	373
12.1.4. Создание собственных объявлений аннотаций	375
12.1.5. Метааннотации: управление обработкой аннотации	376
12.1.6. Передача классов в качестве параметров аннотации в целях дальнейшего управления поведением.	377
12.1.7. Обобщенные классы как параметры аннотаций	378
12.2. Рефлексия: интроспекция объектов Kotlin во время выполнения	379
12.2.1. API рефлексии Kotlin: KClass, KCallable, KFunction и KProperty. . .	380
12.2.2. Сериализация объектов с помощью рефлексии	384
12.2.3. Настройка сериализации с помощью аннотаций	386
12.2.4. Синтаксический анализ JSON и десериализация объектов	389
12.2.5. Последний этап десериализации: функция callBy() и создание объектов с помощью рефлексии	394
Резюме	398
Глава 13. Создание DSL	399
13.1. От API к DSL: создание выразительных пользовательских структур кода	400
13.1.1. Предметно-ориентированные языки	401
13.1.2. Внутренние DSL легко интегрируются в остальную часть программы	403
13.1.3. Структура DSL	404
13.1.4. Создание HTML-страницы с помощью внутреннего DSL	405
13.2. Создание структурированных API: лямбды с приемниками в DSL	406
13.2.1. Лямбды с приемниками и типы функций-расширений	407
13.2.2. Использование лямбд с приемниками в конструкторах HTML	410
13.2.3. Конструкторы Kotlin: обеспечение абстракции и повторного использования	416
13.3. Более гибкое вложение блоков с помощью соглашения invoke.	418
13.3.1. Соглашение invoke: объекты, вызываемые как функции	419
13.3.2. Соглашение invoke в DSL: объявление зависимостей в Gradle. . . .	420
13.4. DSL Kotlin на практике.	422
13.4.1. Цепочка инфиксных вызовов: функция should в тестовых фреймворках	422
13.4.2. Определение расширений для примитивных типов: работа с датами.	423
13.4.3. Функции-расширения членов класса: внутренний DSL для SQL. . .	424
Резюме	428

Часть III. Конкурентное программирование, корутины и потоки

Глава 14. Корутины	431
14.1. Конкурентность и параллелизм	432
14.2. Конкурентность Kotlin: функции приостановки и корутины	433
14.3. Сравнение потоков и корутин	433
14.4. Функции приостановки	436
14.4.1. Код на основе функций приостановки выглядит последовательным	436
14.5. Сравнение корутин с другими подходами	438
14.5.1. Вызов функции приостановки	440
14.6. Вход в мир корутин: конструкторы	441
14.6.1. Из обычного кода в область корутин: функция <code>runBlocking</code>	442
14.6.2. Создание корутин по сценарию «запустить и забыть»: функция <code>launch</code>	442
14.6.3. Вычисления с ожиданием: конструктор <code>async</code>	445
14.7. Решение о точке выполнения кода: диспетчеры	448
14.7.1. Выбор диспетчера	448
14.7.2. Передача диспетчера в конструктор корутин	451
14.7.3. Использование функции <code>withContext</code> для переключения диспетчера внутри корутины	451
14.7.4. Корутины и диспетчеры не станут волшебным средством для решения проблем безопасности потоков	452
14.8. Корутины содержат дополнительную информацию в своем контексте	454
Резюме	456
Глава 15. Структурированная конкурентность	457
15.1. Области видимости корутин устанавливают структуру между ними	458
15.1.1. Создание области видимости корутины — функция <code>coroutineScope</code>	459
15.1.2. Ассоциирование диапазонов корутин с компонентами: <code>CoroutineScope</code>	461
15.1.3. Опасность <code>GlobalScope</code>	463
15.1.4. Контексты корутин и структурированная конкурентность	464
15.2. Отмена	467
15.2.1. Инициирование отмены	467
15.2.2. Автоматическая отмена после превышения лимита времени	468
15.2.3. Каскад отмены распространяется на все дочерние объекты	469
15.2.4. Отмененные корутины выбрасывают <code>CancellationException</code> в специальных местах	470
15.2.5. Отмена — операция кооперативная	472
15.2.6. Проверка отмены корутины	473
15.2.7. Возможность перехватывать поток другими корутинами: функция <code>yield</code>	474
15.2.8. Помните об отмене при получении ресурсов	476
15.2.9. Фреймворки могут выполнять отмену автоматически	477
Резюме	479

Глава 16. Потоки.	480
16.1. Потоки моделируют последовательные потоки значений	480
16.1.1. Потоки позволяют работать с элементами по мере их создания	481
16.1.2. Различные типы потоков в Kotlin	482
16.2. Холодные потоки	483
16.2.1. Создание холодного потока с помощью функции-конструктора потока	483
16.2.2. Холодные потоки не выполняют никакой работы до тех пор, пока значения не будут собраны	484
16.2.3. Отмена сбора потока	487
16.2.4. Внутренняя работа холодных потоков	488
16.2.5. Конкурентные потоки на основе канальных потоков	489
16.3. Горячие потоки	492
16.3.1. Общие потоки транслируют значения абонентам	493
16.3.2. Отслеживание состояния в системе: поток состояний	498
16.3.3. Сравнение общих потоков и потоков состояний	502
16.3.4. Горячий, холодный, общий потоки, поток состояний: когда использовать тот или иной поток	504
Резюме	505
Глава 17. Операторы потока	506
17.1. Управление потоками с помощью операторов потока	506
17.2. Промежуточные операторы применяются к восходящему потоку и возвращают нисходящий	507
17.2.1. Выбросы произвольных значений для каждого элемента восходящего потока: функция transform	508
17.2.2. Семейство операторов take может отменить поток	509
17.2.3. Привязка к фазам потока с помощью функций onStart, onEach, onComplete и onEmpty	510
17.2.4. Буферизация элементов для последующих операторов и коллекторов: оператор buffer	511
17.2.5. Отбрасывание промежуточных значений: оператор conflate	514
17.2.6. Фильтрация значений по тайм-ауту: оператор debounce	515
17.2.7. Переключение контекста корутины, для которой выполняется поток: оператор flowOn	516
17.3. Создание пользовательских промежуточных операторов	517
17.4. Терминальные операторы выполняют восходящий поток и могут вычислять значение	518
17.4.1. Фреймворки предоставляют пользовательские операторы	519
Резюме	520
Глава 18. Обработка ошибок и тестирование	521
18.1. Обработка ошибок, возникающих внутри корутин	522
18.2. Распространение ошибок в корутинах Kotlin	524
18.2.1. Корутинны отменяют всех своих потомков, когда в одном из них возникает сбой	525
18.2.2. Структурированная конкурентность влияет только на исключения, выброшенные через границы корутин	527

18.2.3. Супервизоры не допускают отмены родительских и родственных корутин.	528
18.3. Последнее средство для обработки исключений — CoroutineExceptionHandler	530
18.3.1. Различия при использовании CoroutineExceptionHandler с функциями launch и async	533
18.4. Обработка ошибок в потоках	535
18.4.1. Обработка исключений восходящего потока с помощью оператора catch.	536
18.4.2. Повтор сбора потока, если предикат является истинным: оператор retry.	537
18.5. Тестирование корутин и потоков	538
18.5.1. Быстрое выполнение тестов с помощью корутин: виртуальное время и диспетчер тестов	539
18.5.2. Тестирование потоков с помощью библиотеки Turbine	542
Резюме.	543

Приложения

Приложение А. Создание проектов Kotlin	545
Сборка кода Kotlin с помощью Gradle	545
Создание проектов с обработкой аннотаций	547
Создание проектов Kotlin с помощью Maven	547
Приложение Б. Документирование кода Kotlin	549
Написание комментариев к документации Kotlin.	549
Создание документации по API	551
Приложение В. Экосистема Kotlin	553
Тестирование	554
Покрытие кода.	554
Бенчмаркинг	554
Документирование	554
Внедрение зависимостей	555
Сетевые приложения — серверы и клиенты	555
Сериализация	555
Доступ к базам данных.	556
Конкурентное программирование	556
Разработка пользовательского интерфейса.	556
Алфавитный указатель.	557

ОТЗЫВЫ О ПЕРВОМ ИЗДАНИИ

Как и во всех других замечательных книгах издательства Manning из серии «В действии», в этой вы найдете все, что вам нужно для того, чтобы быстро повысить свою продуктивность.

Кевин Опп, Sumus Solutions

С этой книгой изучать Kotlin легко и весело!

Филип Правица, Info.nl

Хорошо написанная и понятная книга со множеством подробностей.

Джейсон Лу, NetSuite

Kotlin — интересный, но прагматический язык. Каждому Java-программисту стоит его выучить, и для этого достаточно одной этой книги.

Тим Лейверс, старший инженер-программист, Pacific Knowledge Systems

Подробное руководство по концепциям и парадигмам языка программирования Kotlin.

Рональд Тишлиар, системный архитектор, WWK Insurance

Авторы продемонстрировали глубокие знания, и, очевидно, они смогут донести их до читателей.

Дилан Скотт, разработчик программного обеспечения, Shred Code

Эта уникальная книга отлично подходит для начала изучения языка Kotlin, а написали ее два замечательных разработчика из команды Kotlin в JetBrains.

Павел Гайда, Android-разработчик, EL Passion

Предисловие

Идея создания языка Kotlin зародилась в компании JetBrains в 2010 году. К тому моменту она уже стала авторитетным поставщиком инструментов разработки для таких языков, как Java, C#, JavaScript, Python, Ruby и PHP. Интегрированная среда разработки (IDE) для языка Java под названием IntelliJ IDEA была основным продуктом и содержала плагины для Groovy и Scala.

Благодаря опыту создания инструментария для такого широкого набора языков мы получили уникальные знания и смогли сформировать особый взгляд на пространство языкового дизайна в целом. Но среды разработки на платформе IntelliJ, в том числе IntelliJ IDEA, по-прежнему разрабатывались на Java.

Успех коллег из команды .NET, работающих на C# — современном, мощном и быстро развивающемся языке, вызывал у нас некоторую зависть. Но найти язык, который смог бы заменить Java, не представлялось возможным. Необходимо было сформировать перечень требований к нему.

Первым и самым очевидным требованием была статическая типизация. Мы не знаем другого способа, который позволил бы на протяжении нескольких лет без проблем разрабатывать кодовую базу объемом миллионы строк. Вторым требованием была полная совместимость с имеющимся кодом, написанным на Java. Существующая кодовая база — очень ценный актив для компании JetBrains, и мы не могли позволить себе потерять ее или обесценить из-за проблем с совместимостью. Третьим требованием стало отсутствие компромиссов в части качества инструментария разработки. Продуктивность разработчиков — самая важная ценность для JetBrains, и для достижения ее высоких показателей необходимы отличные инструменты. Наконец, нужен был легкий для изучения язык, обсуждение конструкций которого не вызывало бы затруднений.

Зная неудовлетворенную потребность нашей компании, мы могли сделать вывод, что и другие компании находятся в похожем положении. Следовательно, новое решение могло найти множество пользователей за пределами коллектива JetBrains. Исходя из этого, мы решили приступить к проекту создания нового языка — Kotlin.

Обстоятельства сложились так, что проект занял больше времени, чем мы ожидали, и релиз Kotlin 1.0 состоялся более чем через пять лет после первой фиксации нового кода в репозитории. С тех пор у языка сформировалась своя аудитория, он вырос в прекрасную активную экосистему.

Язык Kotlin назван в честь острова (Котлин), находящегося неподалеку от города Санкт-Петербург в России. На это решение повлиял прецедент, созданный разработчиками языков Java (остров Ява) и Seylon (остров Цейлон).

В процессе подготовки языка к релизу мы поняли, что было бы полезно написать о нем книгу. И ее авторами должны были стать люди, которые участвовали в принятии проектных решений для языка и могут объяснить, почему Kotlin устроен именно так. Эта книга представляет собой результат их работы, и мы надеемся, что она поможет вам изучить и понять язык Kotlin. Удачи вам, и пусть разработка всегда приносит удовольствие!

Благодарности

Прежде всего мы хотели бы поблагодарить Сергея Дмитриева и Макса Шафирова за то, что они поверили в идею нового языка и решили вложить в него ресурсы JetBrains. Без них не существовало бы ни языка, ни этой книги.

Мы хотели бы особо отметить Андрея Бреслава, ставшего главным вдохновителем создания языка, о котором приятно писать книгу (и на котором приятно писать код). Хотя Андрей возглавляет постоянно растущую команду Kotlin, он смог дать много полезных комментариев для первого издания, за что мы ему очень благодарны.

Мы благодарны команде издательства Manning, специалисты которой помогли нам сделать текст книги легкочитаемым и хорошо структурированным. В частности, хотим поблагодарить наших редакторов-разработчиков Дэна Махарри и Марину Майклс за самоотверженные попытки найти время для беседы, несмотря на плотные рабочие графики, а также Майкла Стивенса, Хелен Стергиус, Кевина Салливана, Тиффани Тейлор, Элизабет Мартин и Марию Тудор. Мы также благодарим остальных сотрудников редакции за помощь в оформлении этой книги.

Отзывы наших научных редакторов, Игоря Войды и Брента Уотсона, тоже были бесценны, как и комментарии рецензентов, прочитавших рукопись в процессе ее подготовки. Спасибо вам, Роберт Веннер, Алессандро Кампеис, Амит Ламба, Анджело Коста, Борис Василе, Брендан Грейнджер, Келвин Фернандеш, Кристофер Бейли, Кристофер Бортц, Конон Редмонд, Дилан Скотт, Филип Правица, Джейсон Ли, Джастин Ли, Кевин Опп, Николас Френкель, Павел Гайда, Рональд Тишляр, Тим Лейверс.

Спасибо всем, кто оставил отзывы во время публикаций по программе МЕАР и на форуме книги, — мы улучшили текст, учитывая замечания: Алессандро Кампеис, Боб Ресендес, Дидье Гарсия, Хаим Раман, Джеймс Уотсон, Жуан Мигель Пирес Диас, Хорхе Эзекиль Бо, Марк Котик, Шахрияр Анвар, Микаэль Даутри, Нитин Годе, Питер Сабо, Филипп Соренсен, Рани Шарим, Ричард Майнсен, Серхио Бритос, Симеон Лейзерзон, Стив Приор, Вальтер Александр Мата Лопес и Уильям Морган.

Мы благодарны всем участникам команды Kotlin, которым приходилось слушать фразы наподобие «Еще один раздел готов!» на протяжении всего процесса написания книги. Хотим поблагодарить коллег, помогавших нам составлять план книги и писавших отзывы на ее черновики, особенно Илью Рыженкова, Михаила Глухих, Илью Горбунова, Всеволода Толстопятова, Дмитрия Халанского и Хади Харири. Мы хотели бы также поблагодарить наших друзей, ведь они не только поддерживали нас, но и читали текст книги и оставляли отзывы (иногда на горнолыжных курортах во время отпуска): Льва Серебрякова, Павла Николаева, Алексея Семина, Алису Афонину.

Наконец, мы хотели бы поблагодарить наши семьи и кошек за то, что они делают этот мир лучше.

Во втором издании книги «Kotlin в действии» вы узнаете о языке программирования Kotlin и научитесь использовать его для создания приложений, работающих на виртуальной машине Java (JVM) и Android. Повествование начинается с описания базовых возможностей языка и переходит к рассмотрению более характерных аспектов Kotlin, таких как поддержка создания высокоуровневых абстракций и предметно-ориентированных языков. Кроме того, книга содержит информацию об интеграции Kotlin в существующие Java-проекты и о внедрении языка в текущую рабочую среду.

В книге рассматривается язык Kotlin версии 2.0. Чтобы постоянно получать информацию о новых функциях и изменениях, обратитесь к онлайн-документации, доступной по адресу <https://kotlinlang.org>.

ДЛЯ КОГО ЭТА КНИГА

Второе издание книги «Kotlin в действии» ориентировано в первую очередь на разработчиков, имеющих определенный опыт работы с Java. Kotlin опирается на многие концепции и методы Java, и материал книги подан так, чтобы вы могли быстро войти в курс дела, используя имеющиеся у вас знания.

Если у вас есть опыт работы с другими языками программирования, такими как C# или JavaScript, то вам может понадобиться информация из дополнительных источников, которая поможет понять более сложные аспекты взаимодействия Kotlin с JVM. Но изучить Kotlin с помощью нашей книги можно и без этих знаний. Материал сфокусирован на языке Kotlin в целом, а не на конкретной проблемной области, поэтому книга будет одинаково полезна и разработчикам серверной части, и разработчикам Android, и всем, кто создает проекты на базе JVM.

СТРУКТУРА ИЗДАНИЯ

Книга состоит из трех частей. Первая посвящена тому, как начать использовать Kotlin совместно с существующими библиотеками и API.

- В главе 1 рассказывается о ключевых целях, ценностях и областях применения Kotlin, а также показываются различные способы запуска кода Kotlin.
- В главе 2 приводится объяснение основных элементов любой программы на Kotlin, в том числе структур управления и объявлений переменных и функций.
- В главе 3 более подробно рассказывается о способах объявления функций в Kotlin и вводится понятие функций-расширений и свойств.

- Глава 4 посвящена объявлениям классов и знакомит читателя с понятиями классов данных и объектов-компаньонов.
- В главе 5 рассказывается об использовании лямбда-выражений в Kotlin и демонстрируется ряд функций стандартной библиотеки Kotlin, которыми можно воспользоваться с помощью лямбд.
- В главе 6 описаны подходы к работе с коллекциями в Kotlin, а также их отложенный аналог — последовательности.
- В главе 7 вы познакомитесь с понятием `null`-безопасности.
- В главе 8 описывается система типов Kotlin и отдельное внимание уделяется коллекциям.

Во второй части вы научитесь создавать собственные API и абстракции на языке Kotlin и узнаете о некоторых более сложных возможностях языка.

- В главе 9 рассказывается о принципе конвенций, которые позволяют наделить особым смыслом методы и свойства с определенными именами, и вводится понятие делегированных свойств.
- В главе 10 описывается объявление функций высшего порядка — они принимают другие функции и параметры или возвращают их. Вдобавок в этой главе представлена концепция встроенных функций.
- В главе 11 вы погрузитесь в тему обобщений в Kotlin. Описание начинается с базового синтаксиса и переходит к более сложным областям, таким как параметры вещественного типа и вариативность.
- Глава 12 посвящена использованию аннотаций и рефлексии; особое внимание уделяется небольшой библиотеке сериализации JSON под названием JKit — в ней активно используются эти концепции.
- В главе 13 вводится понятие предметно-ориентированных языков (DSL), описываются инструменты Kotlin для их создания и демонстрируется множество примеров применения DSL.

В третьей части рассказывается о корутинах и потоках, подходе к реализации конкурентного программирования в Kotlin.

- В главе 14 представлен обзор модели конкурентности в Kotlin, в том числе приостанавливаемых функций, корутин и основных способов написания конкурентного кода.
- В главе 15 рассказывается о концепции структурированной конкурентности, на которой строится управление конкурентными задачами. Кроме того, в этой главе вы познакомитесь с механизмами отмены и обработки ошибок.
- В главе 16 представлены потоки — абстракция на основе корутин. Она используется для моделирования последовательных потоков значений во времени.
- В главе 17 более подробно рассматриваются операторы преобразования потоков в Kotlin.
- В главе 18 разберем обработку ошибок и тестирование конкурентного кода.

В книге есть три приложения.

- В приложении А объясняется, как выполнять сборку кода Kotlin с помощью инструментов Gradle и Maven.
- Приложение Б посвящено написанию комментариев к документации и генерации документации API для модулей Kotlin.
- В приложении В приводится руководство по изучению экосистемы Kotlin и поиску наиболее актуальной информации в Сети.

Лучше, если вы прочтете книгу от начала до конца. Но можно читать и главы с интересующими вас темами по отдельности, а при обнаружении незнакомых концепций переходить по перекрестным ссылкам и находить нужную информацию.

УСЛОВНЫЕ ОБОЗНАЧЕНИЯ

В этой книге используются следующие условные обозначения.

Курсив

Курсивом выделены новые термины и важные понятия.

Моноширинный шрифт

Используется для листингов программ, а также внутри абзацев для обозначения пользовательского ввода и таких элементов, как переменные и функции, базы данных, типы данных, пути, переменные среды, классы, операторы и ключевые слова, имена файлов и их расширений.

Большая часть листингов сопровождается аннотациями, которые призваны выделить важные концепции в коде. Некоторый исходный код был переформатирован: мы добавили переносы строк и изменили отступы, чтобы код поместился на странице. В редких случаях даже этого было недостаточно и в листинги добавлялись маркеры переноса строки (↵).

Во многих листингах исходного кода показан код вместе с его выводом. В таких случаях вывод кода показан в виде комментария после строки с оператором вывода результата. Например:

```
fun main() {  
    println("Hello, world")  
    // Hello, world  
}
```

Одни примеры — законченные программы, а другие — фрагменты, используемые для демонстрации определенных концепций. В таких листингах могут содержаться пропуски (обозначенные ...) или синтаксические ошибки (описанные в тексте книги или в самих примерах). Запускаемые примеры можно скачать в виде ZIP-файла с сайта издательства Manning по адресу <https://www.manning.com/books/kotlin-in-action-second-edition>.

Шрифт без засечек

Используется для обозначения URL, адресов электронной почты, папок и каталогов, кнопок и элементов интерфейса.

ПРИМЕЧАНИЯ ИЛИ СОВЕТЫ

Оформлены таким образом.

ДРУГИЕ ИНТЕРНЕТ-РЕСУРСЫ

У Kotlin активное онлайн-сообщество, так что, если у вас есть вопросы или вы хотите пообщаться с другими пользователями Kotlin, можете воспользоваться следующими ресурсами:

- *Kotlin в Slack* — <https://slack-chats.kotlinlang.org/>;
- *официальные форумы Kotlin* — <https://discuss.kotlinlang.org>;
- *мег Kotlin на платформе вопросов и ответов Stack Overflow* — <http://stackoverflow.com/questions/tagged/kotlin>;
- *Kotlin на Reddit* — www.reddit.com/r/Kotlin.

ОТ ИЗДАТЕЛЬСТВА

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Мы выражаем огромную благодарность Анне Жарковой и Сергею Задорожному за помощь в работе над русскоязычным изданием книги и их вклад в повышение качества переводной литературы.

Анна Жаркова — энтузиаст мобильной разработки, эксперт Kotlin, руководитель мобильной практики в компании Usetech, ведет Telegram-канал «Записки разработчицы».

Сергей Задорожный — руководитель Platform Engineering и Enabling Team в банке «Центр-инвест». Разрабатывал backend на Java/Kotlin, сейчас внедряет DevOps-практики. Ведет Telegram-канал «IT Friday».

Об авторах

Себастьян Айгнер — адвокат разработчиков в компании JetBrains. Он регулярно выступает на конференциях и проводит семинары по темам, связанным с Kotlin. Кроме того, работает ведущим подкаста *Talking Kotlin* и создает видеоролики для официального канала Kotlin на YouTube. Будучи членом организации Kotlin Foundation, помогает поддерживать рост и устойчивость экосистемы.

Роман Елизаров был руководителем проекта Kotlin в компании JetBrains и в течение семи лет занимался разработкой языка Kotlin в роли ведущего дизайнера языка. Ранее проектировал и разрабатывал высокопроизводительное торговое программное обеспечение для ведущих брокерских компаний и служб доставки рыночных данных, которые регулярно обрабатывают миллионы событий в секунду. Работая над Kotlin в компании JetBrains, внес вклад в разработку корутин Kotlin и создание библиотеки корутин Kotlin.

Светлана Исакова начинала свою карьеру в команде разработчиков компилятора Kotlin, а сейчас занимает должность адвоката разработчиков в компании JetBrains. Преподает Kotlin и выступает на конференциях по всему миру. Кроме того, приняла участие в создании курса Kotlin для Java-разработчиков на платформе Coursera и стала соавтором книги *Atomic Kotlin* (Mindview LLC, 2021).

Дмитрий Жемеров был в числе первых разработчиков языка Kotlin на заре проекта. Хорошо знаком с устройством языка и причинами решений, принятых в ходе его разработки. Во время работы в компании JetBrains занимался различными темами, связанными с Kotlin, в том числе плагином IntelliJ IDEA для Kotlin и документацией по новому языку.

Иллюстрация на обложке

Иллюстрация называется «Одежда русской женщины на Валдае в 1764 году» и взята из книги Томаса Джеффериса, опубликованной между 1757 и 1772 годами.

В те времена по одежде можно было легко определить, где живут люди и каково их ремесло или положение в жизни. Компания Manning чествует изобретательность и инициативу ИТ-индустрии, помещая на обложки книг изображения, основанные на богатом разнообразии региональной жизни двухвековой давности.

Часть I

Введение в Kotlin

В этой части книги вы научитесь писать код на языке Kotlin, используя существующие API.

В главе 1 приводится общее описание Kotlin.

Из глав 2–4 вы узнаете, как основные концепции программирования — инструкции¹, функции, классы и типы — отображаются в коде Kotlin и как этот язык обогащает их. Чтобы быстро погрузиться в тему, вы можете опираться на свои знания других объектно-ориентированных языков, например Java, а также на такие инструменты, как вспомогательные функции IDE и конвертер Java — Kotlin.

В главе 5 рассказывается о том, как избежать повторений и эффективно решать поставленные задачи с помощью лямбда-выражений.

Из главы 6 вы узнаете о возможностях языка Kotlin в области изменения коллекций с помощью функционального программирования.

В главе 7 вы познакомитесь с одной из ключевых особенностей Kotlin — поддержкой работы со значениями `null`.

В главе 8 подробно описаны основы системы типов Kotlin, начиная с базовых чисел и заканчивая специальными типами `Any` и `Nothing`. Кроме того, вы получите дополнительные знания о типах коллекций, в том числе об их изменяемых и доступных только для чтения версиях, а также познакомитесь с массивами.

¹ В Kotlin слово `statements` означает «инструкции», а не «операторы». — *Примеч. пер.*

1

Kotlin. Что это за язык и зачем он нужен

В этой главе

- ✓ Демонстрация базовых возможностей Kotlin.
- ✓ Основные характеристики языка Kotlin.
- ✓ Возможности для серверной и Android-разработки.
- ✓ Обзор технологии Kotlin Multiplatform.
- ✓ Ключевые отличия Kotlin от других языков.
- ✓ Написание и запуск кода на Kotlin.

Kotlin — современный язык программирования на *виртуальной машине Java* (Java virtual machine, JVM) и не только. Это лаконичный, безопасный и прагматический язык программирования общего назначения. Его используют как независимые программисты, так и небольшие компании по разработке программного обеспечения и крупные предприятия. Множество разработчиков предпочитают Kotlin для написания мобильных приложений, создания приложений для серверов и персональных компьютеров (ПК), а также для достижения ряда других целей.

Kotlin задумывался как «улучшенная версия Java» — язык с расширенной эргономикой для разработчиков, позволившей исключить распространенные категории ошибок. Вдобавок в нем применяются современные парадигмы проектирования языка, но при этом сохранена возможность применения во всех задачах, где использовался язык Java. За последнее десятилетие Kotlin успел зарекомендовать себя

как прагматичный инструмент, подходящий для многих стилей разработки, типов проектов и платформ. Он стал преобладающим языком разработки для платформы Android. При создании программного обеспечения (ПО) на стороне сервера Kotlin стал мощной альтернативой Java. Для распространенных фреймворков, таких как Spring, или специализированных Kotlin-фреймворков, таких как Ktor, существует хорошая документация и поддержка.

В Kotlin сочетаются рабочие идеи существующих языков и инновационные подходы, такие как корутины для асинхронного программирования. Изначально язык был ориентирован только на JVM, но теперь существенно расширен: в нем есть поддержка кросс-платформенных решений. В этой главе мы подробно рассмотрим основные характеристики Kotlin.

1.1. ЗНАКОМСТВО С KOTLIN

Начнем с небольшого примера, чтобы продемонстрировать, как выглядит код на языке Kotlin (листинг 1.1). Даже в коротком ознакомительном фрагменте кода можно заметить множество интересных возможностей и концепций Kotlin — все они будут подробно обсуждаться в книге:

- определение класса данных `Person` со свойствами без необходимости указывать тело класса;
- объявление с помощью ключевого слова `val` свойств только для чтения (`name` и `age`);
- предоставление значений по умолчанию для аргументов;
- явная работа с нулевыми значениями (`Int?`) в системе типов, избегание так называемых ошибок на миллиард долларов — `NullPointerException`;
- определения функций верхнего уровня без необходимости оформлять вложенность в классы;
- именованные аргументы при вызове функций и конструкторов;
- использование висящих (или завершающих) запятых;
- использование операций над коллекциями с помощью лямбда-выражений;
- предоставление резервных значений с помощью оператора объединения с `null` (`?:`), когда переменная равна `null`;
- использование строковых шаблонов в качестве альтернативы выполнения конкатенации вручную;
- использование автоматически генерируемых функций (таких как `toString`) для классов данных.

К коду дано краткое пояснение, но если что-то пока будет вам непонятно, то не пугайтесь. В процессе изучения материала книги мы уделим достаточно времени описанию каждой детали этого листинга, и вы сможете писать подобный код самостоятельно.

Листинг 1.1. Знакомство с языком Kotlin

```

data class Person(  ← Клас данных
    val name: String,  ← Свойство только для чтения
    val age: Int? = null  ← Тип (Int?), допускающий значение null, —
                        ← значение по умолчанию для аргумента
)

fun main() {  ← Функция верхнего уровня
    val persons = listOf(
        Person("Alice", age = 29),  ← Именованный аргумент
        Person("Bob"),  ← Виисячая запятая
    )
    val oldest = persons.maxBy {  ← Лямбда-выражение
        it.age ?: 0  ← Оператор объединения с null
    }
    println("The oldest is: $oldest")  ← Строковый шаблон
}

// The oldest is: Person(name=Alice, age=29)  ← Автоматически сгенерированная
                                           ← функция toString

```

В этом фрагменте кода на языке Kotlin показано, как создать коллекцию, заполнить ее объектами класса `Person`, а затем найти самого старшего человека в коллекции, используя значения по умолчанию, если возраст не указан. При создании списка людей в нем не указан возраст записи с именем `Bob`, поэтому в качестве значения по умолчанию используется `null`. Для поиска самого старшего человека в списке используется функция `maxBy`. В функцию передается лямбда-выражение, которое принимает один параметр, по умолчанию названный `it` (хотя вы можете присвоить параметру и другие имена). Оператор объединения с `null` (`?:`) (Elvis operator) возвращает нулевое значение, если параметр `age` равен `null`. Возраст для записи `Bob` не указан, поэтому оператор `?:` заменяет данное значение нулем, и `Alice` считается записью, содержащей наибольшее значение возраста.

Попробуйте запустить пример самостоятельно. И самый простой способ сделать это — воспользоваться онлайн-площадкой на сайте <https://play.kotlinlang.org/>. Введите код примера и нажмите кнопку **Run** (Запуск), после чего код будет выполнен.

Вам понравился этот пример? Продолжайте читать книгу, изучайте язык и станьте экспертом в Kotlin. Надеемся, что вскоре такой код появится в ваших проектах, а не только на страницах нашей книги.

1.2. ОСНОВНЫЕ ХАРАКТЕРИСТИКИ KOTLIN

Kotlin — мультипарадигменный язык. Он статически типизирован, и это означает, что множество ошибок можно распознать во время компиляции, а не во время выполнения. В Kotlin сочетаются идеи объектно-ориентированных и функциональных языков, что помогает писать аккуратный код и использовать дополнительные мощные абстракции. Он предоставляет эффективный способ написания асинхронного кода, что важно во многих сферах разработки ПО.

Возможно, получив краткое описание, вы уже сформировали интуитивное представление о том, к какому типу языков относится Kotlin. Рассмотрим ключевые атрибуты языка более подробно. И для начала поговорим о том, какие типы приложений можно создавать с помощью Kotlin.

1.2.1. Варианты использования Kotlin: Android, серверная часть, везде, где работает Java, и не только

Диапазон целевых платформ Kotlin достаточно обширный. Язык не сосредоточен на какой-то одной проблемной области и не решает задачи какого-то определенного типа. Напротив, он позволяет повысить производительность для любых типов задач, возникающих в процессе разработки. При создании Kotlin в него была заложена идея высокого уровня интеграции с библиотеками, поддерживающими конкретные домены или парадигмы программирования.

В общем случае Kotlin применяется для создания:

- мобильных приложений, работающих на устройствах Android;
- кода на стороне сервера (как правило, бэкенд-веб-приложений).

Изначально Kotlin создавался в качестве более краткой, производительной и безопасной альтернативы Java, пригодной для использования во всех контекстах, в которых может использоваться Java. А это широкий спектр вариантов среды, от небольших периферийных устройств до крупнейших центров обработки данных. Во всех этих случаях Kotlin подходит идеально, и разработчики могут выполнять свою работу, имея дело с более кратким кодом и сталкиваясь с меньшим количеством неприятностей.

Однако Kotlin отлично работает и в других контекстах. С помощью технологии Kotlin Multiplatform можно создавать кросс-платформенные приложения для ПК, iOS и Android — и даже запускать программы на Kotlin в браузере. Эта книга преимущественно посвящена самому языку и тонкостям работы с JVM, а подробную информацию о других вариантах использования Kotlin можно найти на сайте <https://kotlin.in/>. Далее рассмотрим ключевые качества Kotlin как языка программирования.

1.2.2. Статическая типизация повышает производительность, надежность и удобство сопровождения

У языков программирования со статической типизацией есть ряд преимуществ: производительность, надежность, удобство сопровождения и инструментальная поддержка. Ключевое же преимущество заключается в том, что тип каждого выражения в программе известен во время компиляции. Kotlin относится именно к таким языкам программирования. Его компилятор может подтвердить, что методы и поля, к которым вы пытаетесь получить доступ в объекте, действительно существуют. Это позволяет устранить целый класс ошибок — если поле отсутствует или тип возвращаемой функции не соответствует ожидаемому, то эти проблемы

будут известны в процессе компиляции, а не во время выполнения программы. Благодаря этому вы получаете возможность исправить проблемы на более ранних этапах разработки.

Перечислим некоторые преимущества статической типизации.

- *Производительность*. Вызов методов происходит быстрее, поскольку нет необходимости определять, какой метод нужно вызвать во время выполнения.
- *Надежность*. Компилятор использует типы для проверки согласованности программы, что снижает вероятность сбоев во время выполнения.
- *Удобство сопровождения*. Работать с незнакомым кодом проще, поскольку сразу понятно, с какими типами работает код.
- *Инструментальная поддержка*. Статическая типизация обеспечивает надежное выполнение рефакторинга, точное завершение кода и позволяет использовать иные возможности IDE.

Это серьезные отличия от *динамически типизированных* языков программирования, например Python или JavaScript. В этих языках существует возможность определять переменные и функции, которые могут хранить или возвращать данные любого типа, и разрешать ссылки на методы и поля во время выполнения. При таком подходе сокращается объем кода и повышается гибкость при создании структур данных. Но есть и обратная сторона: такие проблемы, как неправильно написанные имена или недопустимые параметры функций, невозможно обнаружить при компиляции, что, в свою очередь, может привести к ошибкам во время выполнения.

Тип каждого выражения в программе должен быть *известен* во время компиляции, однако при написании кода на Kotlin нет необходимости явно *указывать* тип каждой переменной. Как правило, тип переменной автоматически определяется из контекста, что позволяет обойтись без его объявления. Ниже приведен простейший пример этого механизма.

```
val x: Int = 1    ← Можно явно определить тип переменной
val y = 1       ← Но зачастую этого не требуется
```

При объявлении переменной она инициализируется целочисленным значением, и Kotlin автоматически определяет, что ее тип — `Int`. Способность компилятора определять типы из контекста называется *выведением типов* (не путать с приведением типов. — *Примеч. пер.*). В Kotlin этот механизм позволяет сократить объем кода, поскольку нет необходимости объявлять типы в явном виде.

К числу особенностей системы типов Kotlin можно отнести множество концепций, характерных для других объектно-ориентированных языков программирования. Например, поведение классов и интерфейсов ничем не отличается. А если у вас уже есть опыт в Java-разработке, то ваши знания особенно легко переносятся на Kotlin, и это касается в том числе таких тем, как обобщения.

В Kotlin есть поддержка *типов, допускающих значение null* (nullable types), позволяющая писать более надежные программы. Этот механизм призван обнаруживать

возможные исключения указателя `null` во время компиляции, чтобы сбоев такого рода не возникало во время выполнения. Мы вернемся к данной теме в подразделе 1.4.3 и подробно обсудим ее в главе 7, в которой также приводится сравнение с другими, возможно знакомыми вам, подходами к значениям `null`.

Кроме того, система типов Kotlin поддерживает *функции первого класса*. Чтобы разобраться, что это такое, рассмотрим основные идеи функционального программирования и определим, как эта концепция поддерживается в языке Kotlin.

1.2.3. Сочетание функционального и объектно-ориентированного программирования делает Kotlin безопасным и гибким

Kotlin — мультипарадигменный язык программирования, поэтому сочетает в себе *объектно-ориентированный подход* и *функциональный стиль программирования*. К ключевым концепциям функционального программирования относятся:

- *функции (фрагменты поведения) первого класса* — с ними можно работать как со значениями. Допускается хранить их в переменных, передавать в качестве параметров или возвращать из других функций;
- *неизменяемость* — работа выполняется с неизменяемыми объектами, что гарантирует их перманентное состояние после создания;
- *отсутствие побочных эффектов* — в коде будут написаны *чистые функции*, которые возвращают один и тот же результат при одинаковых входных данных, не изменяют состояние других объектов и не взаимодействуют с внешним миром.

Какие преимущества дает написание кода в функциональном стиле? Первое — *краткость* кода. Функциональный код может быть более аккуратным и лаконичным по сравнению с *императивным* аналогом: вместо изменения переменных и формирования логики в расчете на циклы и условные ветвления вы, работая с функциями как со значениями, получаете гораздо больше возможностей для абстрагирования.

К тому же применение такого стиля программирования позволит избежать дублирования в коде. Если несколько фрагментов кода со схожей логикой выполняют аналогичные задачи, то можно легко извлечь общую составляющую логики в отдельную функцию, а различающиеся части передавать в качестве аргументов. И эти аргументы, в свою очередь, тоже могут быть функциями. В Kotlin их можно выразить с помощью лаконичного синтаксиса лямбда-выражений.

Второе преимущество функционального кода — *безопасное конкурентное выполнение*. В многопоточных программах изменение одних и тех же данных несколькими «актерами» (часто несколькими потоками) без надлежащей синхронизации часто становится серьезным источником проблем. Использование неизменяемых структур данных и чистых функций позволяет гарантировать, что такие небезопасные изменения данных не произойдут. Следовательно, нет необходимости создавать сложные схемы синхронизации.

Наконец, функциональное программирование влечет за собой *более легкое тестирование*. Функции без побочных эффектов можно тестировать изолированно. В таком случае не требуется писать большое количество кода настройки, чтобы создать среду, от которой зависят эти функции. Когда область действия функций ограничена, вам легче рассуждать о своем коде и проверять его поведение, не учитывая постоянно особенности большой и сложной системы.

В целом функциональный стиль программирования можно использовать со многими языками, и большое количество аспектов этого подхода пропагандируется как хороший стиль программирования. Но не во всех языках реализована синтаксическая и библиотечная поддержка его использования. При создании в Kotlin закладывался обширный набор возможностей для поддержки функционального программирования:

- *типы функций* — возможность для функций принимать другие функции в качестве аргументов или возвращать другие функции;
- *лямбда-выражения* — позволяют передавать блоки кода с минимальным количеством шаблонов;
- *ссылки на члены без вызова* — возможность использовать функции в виде значений и, например, передавать их в качестве аргументов;
- *классы данных* — предоставление краткого синтаксиса создания классов для хранения неизменяемых данных;
- *API стандартной библиотеки* — большой набор инструментов в стандартной библиотеке для работы с объектами и коллекциями в функциональном стиле.

Во фрагменте кода ниже показана цепочка действий, выполняемых с входной последовательностью. Получив заданную последовательность сообщений, код находит «всех уникальных отправителей непустых непрочитанных сообщений, отсортированных по их именам»:

```
messages
    .filter { it.body.isNotBlank() && !it.isRead }
    .map(Message::sender)
    .distinct()
    .sortedBy(Sender::name)
```

Вы можете использовать функции `filter`, `map` и `sortedBy` из стандартной библиотеки. Язык Kotlin поддерживает лямбда-выражения и ссылки на члены без вызова (например, `Message::sender`), поэтому аргументы для этих функций очень лаконичны.

Таким образом, при написании кода на языке Kotlin вы можете сочетать объектно-ориентированный и функциональный подходы и использовать инструменты, наиболее подходящие для решения поставленной задачи. Это позволяет вам использовать все возможности функционального стиля программирования в Kotlin, а при необходимости работать с изменяемыми данными и писать функции с побочными эффектами. И конечно, работа с фреймворками на основе интерфейсов и иерархии классов будет несложной.

1.2.4. Конкурентный и асинхронный код становится естественным и структурированным при использовании корутин

При создании приложения для сервера, ПК или мобильного телефона избежать *конкурентности* — одновременного выполнения нескольких частей кода — практически невозможно. Пользовательские интерфейсы должны оставаться отзывчивыми, пока в фоновом режиме выполняются длительные вычисления. При взаимодействии с сервисами в Интернете приложения часто реализуют несколько запросов одновременно. Аналогичным образом серверные приложения должны продолжать обслуживать входящие запросы, даже если один из запросов занимает гораздо больше времени, чем обычно. Все эти приложения должны работать *конкурентно*, одновременно выполняя несколько задач.

Создать конкурентность позволяют множество подходов: потоки, обратные вызовы, фьючерсы, промисы, реактивные расширения и многое другое. В Kotlin подход к решению задач конкурентного и асинхронного программирования реализован на основе *приостанавливаемых вычислений*, называемых *корутинами*. С их помощью код может приостановить свое выполнение и возобновить работу в более поздний момент.

В примере ниже происходит определение функции `processUser`, выполняющей три сетевых вызова: `authenticate`, `loadUserData` и `loadImage`.

```
suspend fun processUser(credentials: Credentials) {
    val user = authenticate(credentials)
    val data = loadUserData(user)
    val profilePicture = loadImage(data.imageID)
    // ...
}
```

← Даже продолжительные операции...

← ...можно определить последовательно, сверху вниз...

← ...без блокировки приложения

```
suspend fun authenticate(c: Credentials): User { /* ... */ }
suspend fun loadUserData(u: User): Data { /* ... */ }
suspend fun loadImage(id: Int): Image { /* ... */ }
```

← Для этого используется ключевое слово `suspend`

Сетевой вызов может занять продолжительное время. При осуществлении каждого запроса выполнение функции `processUser` *приостанавливается* в ожидании результата. Однако поток, в котором выполняется этот код (и, соответственно, само приложение), *не блокируется*. В ожидании результата работы функции `processUser` могут выполняться другие задачи, например ответ на пользовательский ввод. (Подробности приостановки функций описаны в разделе 14.1.)

Невозможно написать этот код последовательно в императивной манере, один вызов за другим, без блокировки основных потоков. А при использовании обратных вызовов или реактивных потоков такая простая последовательная логика сильно усложняется.

В следующем примере два изображения загружаются одновременно с помощью функции `async` (ее вы изучите в подразделе 14.6.3). Затем ожидается завершение

загрузки, инициируемое функцией `await`, а в качестве результата возвращается комбинация изображений (например, одно, наложенное на другое):

```
suspend fun loadAndOverlay(first: String, second: String): Image =
    coroutineScope {
        val firstDeferred = async { loadImage(first) }
        val secondDeferred = async { loadImage(second) }
        combineImages(firstDeferred.await(), secondDeferred.await())
    }
```

Начало загрузки первого изображения в новой корутине

Начало загрузки второго изображения в еще одной корутине

Когда оба изображения загружены, они накладываются друг на друга и результат возвращается

Структурированная конкурентность, описанная в главе 15, поможет управлять временем жизни создаваемых корутин. В этом примере два процесса загрузки запускаются структурированным образом (из одной и той же области видимости *корутины*). Это гарантирует, что при сбое одной загрузки вторая будет автоматически отменена.

Кроме того, корутины относятся к очень легковесным абстракциям. Это означает, что вы можете запускать миллионы конкурентных заданий, при этом не теряя в производительности. Корутины Kotlin вкупе с абстракциями *холодных* и *горячих потоков* (описаны в главе 16) становятся мощным инструментом создания конкурентных приложений.

Вся третья часть книги посвящена изучению тонкостей и особенностей корутин, а также тому, как применять их для наилучшего решения поставленных задач.

1.2.5. Kotlin можно использовать для любых целей: он бесплатный и имеет открытый исходный код

Язык Kotlin, в том числе компилятор, библиотеки и все сопутствующие инструменты, полностью открыт и может свободно использоваться в любых целях. Он доступен под лицензией Apache 2: разработка ведется в открытом доступе на GitHub (<http://github.com/jetbrains/kotlin>).

Внести свой вклад в развитие Kotlin и его сообщества вы можете множеством разных способов.

- Приветствуется вклад в разработку новых функций и исправлений, связанных с компилятором Kotlin и сопутствующим инструментарием.
- Можно помочь улучшить опыт разработки на Kotlin, предоставляя сообщения об ошибках и отзывы.
- Потенциальные новые возможности языка подробно обсуждаются в сообществе, и вклад разработчиков на языке Kotlin играет большую роль в продвижении и развитии языка.

Кроме того, доступен выбор между несколькими IDE с открытым исходным кодом для разработки приложений на Kotlin: IntelliJ IDEA Community Edition и Android Studio, обе имеют полную поддержку. (Конечно, версия IntelliJ IDEA Ultimate тоже подходит для работы.)

Мы обсудили, что представляет собой язык Kotlin, а теперь посмотрим, как преимущества Kotlin влияют на решение конкретных практических задач.

1.3. ОБЛАСТИ, В КОТОРЫХ ЧАСТО ИСПОЛЬЗУЕТСЯ KOTLIN

Как мы уже говорили, две основные сферы использования Kotlin — серверная часть ПО и разработка под операционную систему (ОС) Android. Рассмотрим эти области применения и выясним, почему Kotlin хорошо подходит для них.

1.3.1. Обеспечение работы бэкенда — разработка серверной части ПО на Kotlin

Программирование на стороне сервера — довольно широкое понятие. В него входят перечисленные ниже типы приложений и не только:

- веб-приложения, возвращающие браузеру HTML-страницы;
- бэкенд для мобильных или одностраничных приложений, предоставляющих JSON API по протоколу HTTP;
- микросервисы, взаимодействующие с другими микросервисами по протоколу RPC или шине сообщений.

Разработчики уже много лет создают подобные приложения на базе JVM и накопили огромный стек фреймворков и технологий, с помощью которых их можно создавать. Такие приложения обычно не разрабатываются изолированно или с нуля. Почти всегда есть определенная система. Ее расширяют, улучшают или заменяют, и новый код должен интегрироваться с существующими частями системы, возможно написанными много лет назад.

В такой среде Kotlin особенно выигрывает за счет своей легкой совместимости с существующим кодом Java. Kotlin отлично подойдет как для переноса существующего сервиса на его кодовую базу, так и для написания нового компонента. У вас не возникнет проблем, когда понадобится расширить Java-классы в Kotlin или аннотировать методы и поля класса определенным образом. В то же время вы получите преимущества в виде компактности кода, его повышенной надежности и простоты сопровождения.

Еще одно большое преимущество Kotlin — повышенная надежность создаваемого приложения. Система типов Kotlin с точным отслеживанием значений `null` делает проблему исключений с `null`-указателями гораздо менее актуальной. Большая часть кода на Java, вызывающего `NullPointerException` во время выполнения, в Kotlin

не компилируется. Поэтому исправить ошибку можно до того, как приложение попадет в среду эксплуатации.

Современные фреймворки, такие как Spring (<https://spring.io/>), обеспечивают поддержку функций первого класса в Kotlin. Помимо полной совместимости, эти фреймворки содержат дополнительные расширения и используют приемы, благодаря которым создается впечатление, что изначально они были разработаны для Kotlin.

В листинге 1.2 содержится простое приложение Spring Boot, передающее список объектов класса `Greeting`, стоящий из идентификатора и некоего текста в формате JSON, который передается по протоколу HTTP (рис. 1.1). Концепции из фреймворка Spring непосредственно переходят в Kotlin — используются те же аннотации (`@SpringBootApplication`, `@RestController`, `@GetMapping`), что и при работе с Java.

Листинг 1.2. Написание приложений Spring Boot на Kotlin

```
@SpringBootApplication
class DemoApplication
```

← Функциональность фреймворка
используется в виде аннотаций...

```
fun main(args: Array<String>) {
    runApplication<DemoApplication>(*args)
}

@RestController
class GreetingResource {
    @GetMapping
    fun index(): List<Greeting> = listOf(
        Greeting(1, "Hello!"),
        Greeting(2, "Bonjour!"),
        Greeting(3, "Guten Tag!")
    )
}
data class Greeting(val id: Int, val text: String)
```

← ...при этом применяются
выразительные
возможности Kotlin



Рис. 1.1. Благодаря сочетанию Kotlin с такими проверенными в индустрии фреймворками, как Spring, создать приложение для предоставления JSON-файла по протоколу HTTP можно, написав всего несколько десятков строк кода

На сайтах Kotlin или Spring вы можете получить дополнительную информацию об использовании Spring совместно с Kotlin (<https://kotlinlang.org/docs/jvm-get-started-spring-boot.html>).

Кроме того, у Kotlin постоянно растет экосистема собственных библиотек, в том числе серверных фреймворков. В качестве примера можно привести Ktor (<https://ktor.io/>) — созданный компанией JetBrains фреймворк подключенных приложений для Kotlin. Его можно использовать для разработки приложений на стороне сервера и выполнения сетевых запросов в клиентских и мобильных приложениях. На его основе были созданы такие продукты, как JetBrains Space (<https://jetbrains.space>) и Toolbox (<https://jetbrains.com/toolbox>). А такие компании, как Adobe, применяют его в работе.

Ktor — фреймворк Kotlin, поэтому с его помощью можно полноценно использовать возможности языка. Например, он определяет свой *предметно-ориентированный язык* (Domain-Specific Language, DSL), чтобы объявить, как HTTP-запросы направляются через приложение. Вместо того чтобы настраивать приложение с помощью аннотаций или XML-файлов, можно использовать DSL из Ktor для настройки маршрутизации приложения на стороне сервера. В решении этой задачи применяются полностью индивидуальные для фреймворка конструкции, выглядящие как часть языка Kotlin. Вы научитесь создавать такие решения самостоятельно в главе 13.

В листинге 1.3 определены три маршрута — `/world`, `/greet` и `/greet/{entityId}` — с помощью DSL-конструкций `get`, `post` и `route` фреймворка Ktor.

Листинг 1.3. Приложение Ktor использует DSL для маршрутизации HTTP-запросов

```
fun main() {
    embeddedServer(Netty, port = 8000) {
        routing {
            get("/world") {
                call.respondText("Hello, world!")
            }
            route("/greet") {
                get { /* . . . */ }
                post("/{entityId}") { /* . . . */ }
            }
        }
    }.start(wait = true)
}
```

Определение того, как Ktor обрабатывает HTTP-запросы...

...с помощью команд DSL, которые выглядят так, будто это встроенная функциональность Kotlin...

...и из которых легко составлять языковые конструкции

DSL обладает достаточно гибкой структурой, позволяет комбинировать возможности языка Kotlin и часто используется для конфигурирования, создания сложных объектов или задач *объектно-реляционного отображения* (Object-Relational Mapping, ORM), перевода объекты в их представление в базе данных и наоборот.

Другие фреймворки Kotlin для серверной разработки, например `http4k` (<https://http4k.org/>), строго учитывают функциональную природу кода Kotlin и предоставляют простые и единообразные абстракции для запросов и ответов. В обширной экосистеме Kotlin точно можно найти как проверенные в работе фреймворки промышленного стандарта для крупного проекта, так и легкие, подходящие для создания микросервиса.

1.3.2. Разработка мобильных приложений. Kotlin — первоочередной вариант для Android

Официальная поддержка Kotlin на самой распространенной мобильной ОС в мире, Android, в качестве языка для создания приложений появилась в 2017 году. И только в 2019 году, после множества положительных отзывов от разработчиков, Kotlin был принят для Android-устройств в качестве первоочередного языка разработки и стал выбором по умолчанию для новых приложений. С тех пор инструменты разработки Google, библиотеки Jetpack (<https://developer.android.com/jetpack>), примеры, документация и обучающие материалы — все в основном ориентированы на Kotlin.

Этот язык хорошо подходит для мобильных приложений — ПО такого типа обычно требуется создавать быстро, обеспечивая при этом надежную работу на большом количестве устройств. Возможности Kotlin превращают разработку для ОС Android в гораздо более продуктивное и приятное занятие. Общие задачи разработки можно решать с помощью гораздо меньшего количества кода. Созданная командой Android библиотека KTX (<https://developer.android.com/kotlin/kt>) еще больше расширяет возможности, добавляя адаптеры для Kotlin ко многим стандартным API Android.

Компания Google разработала набор инструментов под названием Jetpack Compose (<https://developer.android.com/jetpack/compose>). Он позволяет создавать нативные пользовательские интерфейсы для Android, а также разработан для Kotlin с нуля. Он использует возможности языка Kotlin и дает возможность при создании пользовательского интерфейса мобильных приложений писать более простой код меньшего объема, который легче поддерживать.

Ниже приведен пример работы в Jetpack Compose. Он даст представление о том, как выглядит Android-разработка на языке Kotlin. Этот фрагмент кода показывает сообщение и раскрывает или скрывает подробную информацию по щелчку кнопкой мыши:

```

@Composable
fun MessageCard(modifier: Modifier, message: Message) {
    var isExpanded by remember { mutableStateOf(false) }
    Column(modifier.clickable { isExpanded = !isExpanded }) {
        Text(message.body)
        if (isExpanded) {
            MessageDetails(message)
        }
    }
}

@Composable
fun MessageDetails(message: Message) { /* ... */ }

```

Кроме того, можно написать весь пользовательский интерфейс на языке Kotlin и использовать его обычный синтаксис, например выражения `if` или циклы. В этом примере мы показываем элемент пользовательского интерфейса, отображающий подробную информацию, только если пользователь нажал карточку для получения развернутого представления. В Kotlin можно извлечь пользовательскую

логику, представляющую различные части пользовательского интерфейса, и сохранить ее в функции, например, `MessageDetails`. В результате получится более аккуратный код.

Использование Kotlin в Android-разработке означает еще и то, что код будет более надежным: сократится количество исключений `NullPointerException` и сообщений с текстом `Unfortunately, process has stopped` (К сожалению, процесс остановлен). Например, когда в компании Google перешли к разработке новых функций на языке Kotlin, удалось сократить количество сбоев типа `NullPointerException` в приложении Google Home на 30 %.

Использование Kotlin не создает никаких сложностей с совместимостью или не приводит к снижению производительности в существующих приложениях. Этот язык полностью совместим с Java версии 8 и выше, а сгенерированный компилятором код выполняется эффективно. Среда выполнения Kotlin довольно небольшая, поэтому вы не заметите сильного увеличения размера скомпилированного пакета приложения. При использовании лямбд многие функции стандартной библиотеки Kotlin будут заменять их встроенными выражениями. Этот механизм гарантирует, что новые объекты не будут создаваться без необходимости и в работе приложения не будут возникать лишние паузы для сборки мусора. Вы воспользуетесь всеми новыми возможностями языка Kotlin, а пользователи смогут запустить приложение на своих устройствах, даже если они работают не на последней версии ОС Android.

1.3.3. Мультиплатформенность — совместное использование бизнес-логики и минимизация дублирования работы на iOS, JVM, JS и не только

Kotlin — *мультиплатформенный* язык. Помимо JVM, Kotlin поддерживает следующие целевые системы:

- его можно скомпилировать в JavaScript, что позволяет выполнять код Kotlin в браузере и таких средах выполнения, как Node.js;
- с помощью расширения Kotlin/Native можно компилировать код Kotlin в нативные двоичные файлы, что дает возможность работать на iOS и других платформах с помощью автономных программ (Self-Contained System, SCS);
- Wasm — целевая платформа, которая на момент написания книги еще разрабатывается. Инструмент Kotlin/Wasm позволит вам компилировать ваш код на Kotlin в бинарный формат инструкций WebAssembly, что даст возможность запускать его на виртуальных машинах WebAssembly, встроенных в современные браузеры и другие среды выполнения.

Кроме того, в Kotlin можно на очень детализированном уровне определить, какие части ПО должны совместно использоваться различными целевыми платформами, а какие требуют отдельной реализации. Данная область применения требует пристального контроля, поэтому у вас будет возможность смешивать фрагменты и подбирать оптимальное сочетание общего и специфического для платформы кода.

В Kotlin этот механизм называется *expect/actual*. Он позволяет использовать преимущества специфической для платформы функциональности из написанного кода Kotlin. Таким образом, эффективно смягчается классическая проблема «ориентации на наименьший общий знаменатель». С ней сталкиваются кросс-платформенные наборы инструментов при ограничении подмножеством операций, доступных на всех целевых платформах.

Один из основных примеров совместного использования кода — мобильные приложения для Android и iOS. Благодаря технологии Kotlin Multiplatform бизнес-логику нужно будет написать только один раз, а использовать в iOS и Android в полностью нативном виде и даже применять соответствующие API, наборы инструментов и возможности целевых платформ. Аналогичным образом, совместное использование кода между сервисом на стороне сервера и JavaScript-приложением, запущенным в браузере, позволяет сократить количество дубликатов, синхронизировать логику проверки и многое другое.

Более подробную информацию о специфике этих дополнительных платформ, а также о поддержке Kotlin для совместного использования кода и мультиплатформенного программирования, можно найти в разделе Kotlin Multiplatform на сайте Kotlin (<https://kotlinlang.org/docs/multiplatform.html>).

Теперь, когда мы рассмотрели ряд аспектов Kotlin, выделяющих его на фоне других языков, можно приступить к его философии — изучению основных отличительных характеристик этого языка.

1.4. ФИЛОСОФИЯ KOTLIN

Рассказывая о Kotlin, мы говорим, что это прагматический, лаконичный и безопасный язык с акцентом на совместимости. Но что именно мы подразумеваем под каждым из этих слов? Рассмотрим их все по порядку.

1.4.1. Прагматичность

Прагматичность — простое для нас понятие. Kotlin — практичный язык, созданный для решения реальных задач. Его дизайн основан на многолетнем опыте создания крупномасштабных систем, а функции подобраны для решения распространенных задач, встречающихся при разработке ПО. Кроме того, разработчики по всему миру используют Kotlin уже около десяти лет и от них постоянно поступает обратная связь. На основе этих данных формируется каждая новая версия языка, что позволяет нам с уверенностью говорить о том, что Kotlin может решать задачи в реальных проектах.

Но Kotlin нельзя отнести к исследовательским языкам. В основном в нем задействованы хорошо себя зарекомендовавшие функции и решения из других языков. Такой подход снижает сложность языка и облегчает его изучение, поскольку разработчики могут использовать уже знакомые им понятия. Новые возможности долгое время остаются в экспериментальном состоянии. Это позволяет команде разработчиков

языка собирать отзывы и дорабатывать окончательный дизайн функции, прежде чем она будет добавлена в качестве постоянной составляющей языка.

Kotlin не требует использования какого-то определенного стиля или парадигмы программирования. В работе можно применять знакомые вам стиль и приемы. И в процессе изучения языка вы постепенно откроете для себя более мощные возможности Kotlin, такие как функции-расширения (см. раздел 3.3), выразительная система типов (см. главы 7 и 8), функции высшего порядка (см. главу 10) и многое другое. Вы научитесь применять их в коде и сможете привести его в более лаконичный и идиоматичный вид.

Кроме того, к аспектам прагматизма Kotlin относится и ориентация на инструментарий. Интеллектуальная среда разработки так же важна для продуктивности разработчика, как и хороший язык, и поэтому не стоит относиться к поддержке IDE поверхностно. В нашем случае плагин IntelliJ IDEA разрабатывается параллельно с компилятором, а возможности языка всегда создаются с учетом инструментария.

Поддержка IDE играет важную роль и в раскрытии возможностей Kotlin. Во многих случаях инструменты определяют общие паттерны кода и предлагают автоматически заменить их более лаконичными конструкциями. Со временем вы достаточно хорошо изучите эти механизмы автоматизации и будете постоянно применять их в работе.

1.4.2. Лаконичность

Как правило, разработчики тратят больше времени на чтение существующего кода, чем на написание нового. Представьте, что вы работаете в команде, разрабатывающей большой проект. В какой-то момент вам требуется добавить в систему новую функциональную возможность или исправить ошибку. С чего вы начнете решать эту задачу? Сначала необходимо найти требуемый участок кода и только потом внедрить исправление. А для этого придется читать много кода. Возможно, он был написан недавно вашими коллегами или кем-то, кто больше не работает над проектом, или же давно написан вами. Поэтому необходимые изменения можно внести, только разобравшись в окружающем коде.

Чем проще и лаконичнее будет код, тем быстрее вы разберетесь в написанном. Конечно, хороший дизайн играет здесь важную роль, как и выбор выразительных имен, что гарантирует точное описание переменных, функций и классов. Но выбор языка и его краткость тоже важны. Язык *лаконичен*, если его синтаксис ясно выражает логику кода и не скрывает ее за шаблонами, необходимыми для уточнения реализации этой логики.

Kotlin задумывался так, чтобы весь код имел смысл, а не просто удовлетворял требованиям к структуре кода. Многие стандартные шаблоны объектно-ориентированных языков, такие как геттеры, сеттеры и логика присвоения параметров конструктора полям, неявно присутствуют в Kotlin и не загромождают исходный код. Вдобавок в этом языке можно опускать точки с запятой, избавляясь от лишнего беспорядка в кодовой базе. Продуманная система вывода типов избавляет от необходимости явно указывать типы там, где компилятор может вывести их из контекста.

В Kotlin есть механизм замены длинных, повторяющихся участков кода вызовами методов из обширной стандартной библиотеки. Поддержка лямбд и анонимных функций (литералов функций, используемых в качестве выражений) позволяет легко передавать небольшие блоки кода библиотечным функциям. Благодаря этому есть возможность инкапсулировать все общие фрагменты программы в библиотеку и оставить в пользовательском коде только уникальную, характерную для конкретной задачи часть.

В то же время в дизайне Kotlin не предусмотрено сворачивание исходного кода до минимально возможного количества символов. Например, этот язык поддерживает перегрузку фиксированного набора операторов, предоставляя возможность создавать собственные реализации для `+`, `-`, `in` или `[]`. Но возможности определять собственные операторы у пользователей нет. Вследствие этого ограничения разработчики не могут заменять имена методов загадочными последовательностями знаков препинания, что в противном случае осложнило бы чтение кода и поиск информации в системах документации.

Такой лаконичный код можно быстрее писать и, что еще важнее, быстрее читать и осмысливать. И следовательно, ваша производительность при решении рабочих задач возрастет.

1.4.3. Безопасность

Чтобы язык программирования мог называться *безопасным*, его дизайн должен предотвращать определенные виды ошибок в программе. Конечно, это не абсолютное качество — ни один язык не предотвращает все возможные ошибки. Кроме того, этот процесс обычно довольно затратный. Компилятору необходимо предоставить больше информации о предполагаемой работе программы, чтобы он проверял, соответствует ли эта информация поведению программы. Поэтому всегда присутствует компромисс между уровнем безопасности и потерей продуктивности, вызванной необходимостью вносить более подробные аннотации.

Запуск программы на JVM дает множество гарантий безопасности — например, безопасность памяти, предотвращение переполнения буфера и иных проблем, связанных с неправильным использованием динамически выделяемой памяти. Kotlin — статически типизированный язык для работы на JVM, поэтому обеспечивает безопасность типов в приложениях. Но эти идеи получили дальнейшее развитие. Язык дает возможность легко определять доступные только для чтения переменные (с помощью ключевого слова `val`) и быстро группировать их в неизменяемые (`data`) классы. Благодаря всему этому многопоточные приложения получают дополнительную защиту.

Кроме того, Kotlin нацелен на предотвращение ошибок во время выполнения программы, осуществляя проверки во время компиляции. Но самое главное, что в этом языке заложено стремление удалить исключения `NullPointerException` из создаваемой программы. Система типов Kotlin отслеживает значения, которые могут и не могут быть `null`, и запрещает операции, способные привести к возникновению `NullPointerException` во время выполнения. Дополнительные затраты на

это минимальны: пометить тип как допускающий значение `null` можно с помощью всего одного символа — вопросительного знака в конце:

```
fun main() {  
    var s: String? = null  ← Может принимать значение null  
    var s2: String = ""   ← Не может принимать значение null  
  
    println(s.length)     ← Не будет компилироваться, предотвращая сбой в работе программы  
    println(s2.length)    ← Будет работать, как задумано  
}
```

В дополнение к этому в Kotlin существует множество удобных способов работы с данными, допускающими значение `null`. Они помогают избежать сбоев в работе приложения.

Помимо всего прочего, Kotlin помогает избежать исключение типа `ClassCastException`. Оно возникает, когда в программе выполняется попытка привести объект к какому-либо типу без предварительной проверки корректности последнего. В Kotlin процессы проверки и приведения типа объединены в одну операцию. Это означает, что после проверки типа можно ссылаться на его члены, не выполняя какие-либо дополнительные приведения, повторные объявления или проверки.

В следующем примере оператор `is` выполняет проверку типа переменной `value` — она может быть типа `Any`. Компилятор знает, что в ветви `true` условия переменная `value` должна быть типа `String`, поэтому позволяет использовать методы данного типа. Этот процесс называется *умным преобразованием (smart cast)* и более подробно описывается в подразделе 2.3.6.

```
fun modify(value: Any) {  
    if (value is String) { ← Проверка типа  
        println(value.uppercase()) ← Выполнение метода, вызываемого для данных  
    }                               ← типа String, без последующего приведения типов  
}
```

Далее поговорим конкретно о применении Kotlin для целевых сред на JVM. Kotlin обеспечивает полное взаимодействие с Java.

1.4.4. Совместимость

В теме совместимости вас наверняка интересует возможность использовать существующие библиотеки. И Kotlin определенно удовлетворяет данную потребность. Независимо от того, какие API требуются библиотеке, можно работать с ними из Kotlin. Допускается вызывать методы Java, расширять классы Java и реализовывать интерфейсы, применять аннотации Java к классам Kotlin и т. д.

В отличие от некоторых других JVM-языков Kotlin улучшен настолько, что позволяет легко вызывать код Kotlin и из программ на Java. Никакие ухищрения не нужны — классы и методы Kotlin можно вызывать точно так же, как обычные классы и методы Java. Так вам доступна максимальная гибкость при смешивании кода Java и Kotlin в любом месте проекта. При внедрении кода Kotlin в Java-проект можно запустить конвертер из Java в Kotlin для любого класса в текущей кодовой

базе. При этом остальной код будет компилироваться и работать без каких-либо изменений. Это работает независимо от роли преобразуемого класса (данный механизм подробнее описывается в подразделе 1.5.1).

При создании Kotlin особое внимание уделялось такой области совместимости, как максимально возможное использование существующих библиотек Java. Например, коллекции Kotlin почти полностью используют классы стандартной библиотеки Java, добавляя в них дополнительные функции, увеличивая удобство их использования в Kotlin. (Этот механизм более подробно рассматривается в разделе 3.3.) А значит, нет необходимости оборачивать или преобразовывать объекты при вызове Java API из Kotlin, и наоборот. Реализация всех возможностей API, предоставляемых Kotlin, не требует затрат во время выполнения.

Кроме того, инструментарий Kotlin поддерживает и кросс-языковые проекты. Допустимо компилировать произвольную смесь исходных файлов Java и Kotlin, невзирая на то, в каких зависимостях друг от друга они находятся. Возможности IDE в средах IntelliJ IDEA и Android Studio тоже работают на разных языках. Благодаря этому можно:

- свободно перемещаться между исходными файлами Java и Kotlin;
- отлаживать проекты на разных языках и работать с написанным на разных языках кодом;
- выполнять рефакторинг методов Java и обеспечивать их корректное использование в коде Kotlin, и наоборот.

В качестве примера на рис. 1.2 показано, как в среде IntelliJ IDEA работает поиск упоминаний в смешанной кодовой базе Kotlin и Java.

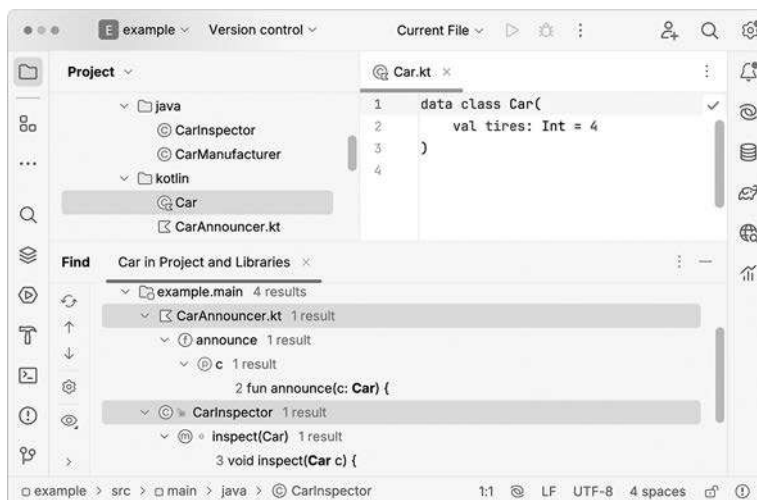


Рис. 1.2. При использовании действия Find Usages (Поиск случаев использования) в среде IntelliJ IDEA будут обнаружены результаты в файлах Kotlin и Java в одном проекте. И другие возможности IDE, такие как рефакторинг и навигация, полноценно работают в обоих языках

Надеемся, что теперь аргументов достаточно, чтобы вы попробовали поработать на Kotlin. Но с чего же начать? В следующем разделе описывается процесс компиляции и выполнения кода Kotlin как из командной строки, так и с помощью различных инструментов.

1.5. ИСПОЛЬЗОВАНИЕ ИНСТРУМЕНТОВ KOTLIN

Приведем обзор инструментов Kotlin. И начнем с обсуждения настройки среды для запуска кода Kotlin.

1.5.1. Настройка и запуск кода Kotlin

Можно запускать небольшие фрагменты онлайн или установить IDE. Но самый приятный опыт вы получите при работе в средах IntelliJ IDEA или Android Studio. В этом подразделе приведена базовая информация об инструментах, но лучшие актуальные руководства можно найти на сайте Kotlin. Подробная информация о настройке среды или о компиляции для различных целевых платформ находится в разделе *Getting Started* на сайте Kotlin (<https://kotlinlang.org/docs/getting-started.html>).

Попробуйте работать на Kotlin без установки, с помощью Kotlin Online Playground

Самый простой способ попробовать работать на Kotlin не требует установки или настройки. На сайте <https://play.kotlinlang.org/> расположена онлайн-площадка, на которой можно писать, компилировать и запускать небольшие программы на Kotlin. Там можно найти примеры кода, в которых демонстрируются возможности Kotlin, а также серию упражнений для интерактивного изучения Kotlin. Наряду с этим в документации по Kotlin (<https://kotlinlang.org/docs>) есть несколько интерактивных примеров, запускаемых прямо в браузере.

Эти способы запуска коротких фрагментов кода на Kotlin самые быстрые, но в случае их использования вы получите меньше помощи и советов. В минимально доступной среде разработки нет таких удобных возможностей, как автозаполнение или проверка, которые могут подсказать, как улучшить код на Kotlin. Кроме того, веб-версия не поддерживает взаимодействие с пользователем через стандартный поток ввода или работу с файлами и каталогами. Однако в средах разработки IntelliJ IDEA и Android Studio все это находится в удобном доступе.

Плагин для IntelliJ IDEA и Android Studio

Плагин Kotlin для IntelliJ IDEA разрабатывался параллельно с языком и является полнофункциональной средой разработки для языка Kotlin. Это достаточно развитый и стабильный инструмент, предоставляющий полный набор возможностей для разработки на Kotlin.

Плагин входит в комплект поставки IntelliJ IDEA и Android Studio, поэтому дополнительная настройка не требуется. Можно использовать бесплатные версии IntelliJ IDEA Community Edition с открытым исходным кодом или Android Studio

либо приобрести коммерческую IntelliJ IDEA Ultimate. После запуска IntelliJ IDEA выберите Kotlin в диалоговом окне **New Project** (Новый проект), и среда будет готова к работе. В Android Studio достаточно создать новый проект, чтобы сразу же приступить к написанию кода на языке Kotlin. Кроме того, вы можете ознакомиться с руководством *Get started with Kotlin/JVM*, в котором содержатся подробные инструкции и снимки экрана, где описывается создание проекта в IntelliJ IDEA: <https://kotlinlang.org/docs/jvm-get-started.html>.

Конвертер Java в Kotlin

Погружение в работу на новом языке никогда не бывает легким. К счастью, мы разработали небольшой быстрый способ, который позволяет ускорить процесс обучения и освоения и базируется на уже имеющихся знаниях о Java. Этот инструмент представляет собой автоматический конвертер из Java в Kotlin.

Поначалу не всегда можно вспомнить необходимый синтаксис Kotlin, и конвертер даст возможность выразить логику. Напишите необходимый фрагмент кода на Java, а затем вставьте его в файл Kotlin, и конвертер автоматически предложит перевести код в Kotlin. Результат не всегда будет самым идиоматичным, но код будет рабочим, и вы сможете продолжить работу над своей задачей.

Кроме того, конвертер отлично подходит для внедрения кода Kotlin в существующий Java-проект. Новый класс можно написать сразу на Kotlin. Но если нужно внести серьезные изменения в существующий класс, то вы, возможно, захотите использовать Kotlin и в этом процессе. В такой ситуации конвертер поможет очень сильно: сначала класс необходимо сконвертировать в Kotlin, а затем внести изменения, используя все преимущества современного языка.

Применять конвертер в IntelliJ IDEA очень просто. Можно скопировать фрагмент кода Java и вставить его в файл Kotlin. Либо при необходимости преобразовать весь файл, вызвав действие **Convert Java File to Kotlin File** (Конвертировать Java-файл в Kotlin-файл).

1.5.2. Компиляция кода Kotlin

Kotlin — компилируемый язык, а значит, прежде чем запускать программу, ее нужно скомпилировать. Код Kotlin можно скомпилировать под разные целевые платформы (см. подраздел 1.3.3):

- байт-код JVM (хранится в файлах `.class`) для запуска на JVM;
- байт-код JVM для дальнейшего преобразования и запуска на Android;
- нативные целевые платформы для запуска на различных операционных системах;
- JavaScript (и WebAssembly) для запуска в браузере.

Для компилятора Kotlin неважно, будет ли полученный байт-код JVM работать на виртуальной машине Java или будет дополнительно преобразован и запущен

на Android. Среда выполнения Android Runtime (ART) преобразует байт-код JVM в байт-код DEX и запускает его. Более подробную информацию о том, как это работает на Android, можно найти в документации: <https://source.android.com/devices/tech/dalvik>.

Основная тема нашей книги — Kotlin/JVM, поэтому обсудим такой процесс компиляции более подробно. Информация о других целевых платформах доступна на сайте Kotlin.

Процесс компиляции для Kotlin/Jvm

Исходный код Kotlin хранится в файлах с расширением `.kt`. При компиляции кода Kotlin для JVM компилятор анализирует исходный код и создает файлы с расширением `.class` точно так же, как это делает компилятор Java. После этого сгенерированные файлы `.class` упаковываются и выполняются с помощью стандартной процедуры для типа разрабатываемого приложения.

Проще всего использовать команду `kotlinc` для компиляции кода из командной строки и команду `java` для выполнения кода:

```
kotlinc <исходный файл или каталог> -include-runtime -d <имя jar-файла>  
java -jar <имя jar-файла>
```

Независимо от того, на каком из языков изначально был написан код, JVM может запускать файлы `.class`, скомпилированные из Kotlin. Однако встроенные классы Kotlin и их API отличаются от Java. Поэтому корректное выполнение скомпилированного кода JVM требует дополнительной информации в виде зависимости — *библиотеки среды выполнения Kotlin*. При компиляции кода из командной строки мы явно указали `-include-runtime`, чтобы добавить эту библиотеку в итоговый JAR-файл.

Эта библиотека должна входить в состав создаваемого приложения. Она содержит определения основных классов Kotlin, таких как `Int` и `String`, а также некоторые расширения, добавляемые в Kotlin к стандартным API Java. Упрощенное описание процесса сборки Kotlin показано на рис. 1.3.

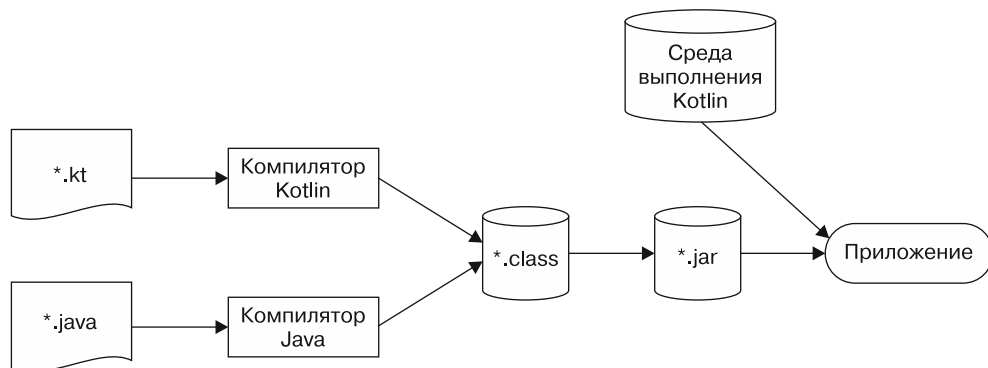


Рис. 1.3. Процесс сборки Kotlin

Кроме того, необходимо добавить в программу *стандартную библиотеку Kotlin* в качестве зависимости. Теоретически писать код Kotlin можно и без нее, но на практике такого не происходит. Стандартная библиотека содержит определения таких фундаментальных классов, как `List`, `Map` и `Sequence`, а также множество методов для работы с ними. В этой книге мы подробно рассмотрим наиболее важные классы и их API.

В работе по компиляции кода обычно используют совместимые с Kotlin системы сборки, например Gradle или Maven. Все эти системы поддерживают проекты на смешанных языках, сочетающие Kotlin и Java в одной кодовой базе.

Инструменты Maven и Gradle автоматически добавляют библиотеку среды выполнения и (для последних версий) стандартную библиотеку Kotlin в качестве зависимостей приложения. Поэтому нет необходимости указывать их в явном виде.

Самая актуальная информация по настройке и сборке проекта с выбранной системой сборки находится в следующих разделах документации Kotlin: <https://kotlinlang.org/docs/gradle.html> и <https://kotlinlang.org/docs/maven.html>. В данный момент вникать во все тонкости не обязательно: вы можете просто создать новый проект, и правильный файл сборки с необходимыми зависимостями будет сгенерирован автоматически.

РЕЗЮМЕ

- Kotlin статически типизирован и поддерживает вывод типов, что позволяет ему поддерживать корректность и производительность, сохраняя лаконичность исходного кода.
- Kotlin поддерживает как объектно-ориентированный, так и функциональный стиль программирования, позволяя создавать абстракции более высокого уровня с помощью функций первого класса. Кроме того, этот язык упрощает тестирование и многопоточную разработку благодаря поддержке неизменяемых значений.
- Корутины представляют собой легкую альтернативу потокам и помогают сделать асинхронный код естественным. С их помощью можно писать логику, похожую на последовательный код, и структурировать параллельный код в виде отношений «родитель — потомок».
- Kotlin отлично подходит для разработки серверных приложений благодаря таким фреймворкам, как Ktor и http4k. Вдобавок он полностью поддерживает все существующие Java-фреймворки, например Spring Boot.
- Kotlin стал первоочередным языком для ОС Android: инструменты разработки, библиотеки, примеры и документация ориентированы в первую очередь на Kotlin.
- Технология Kotlin Multiplatform переносит код Kotlin на целевые платформы, помимо JVM, в том числе iOS и веб-приложения.
- Kotlin — бесплатный язык с открытым исходным кодом, он поддерживает множество систем сборки и IDE.

- Среда разработки IntelliJ IDEA и Android Studio позволяют легко ориентироваться в коде, написанном как на Kotlin, так и на Java.
- На площадке Kotlin playground (<https://play.kotlinlang.org>) можно писать на Kotlin, не устанавливая дополнительное ПО.
- Автоматический конвертер из Java в Kotlin позволяет перенести существующий код и знания в Kotlin.
- Kotlin — прагматический, безопасный и лаконичный язык с высокой совместимостью. Он ориентирован на использование проверенных решений для общих задач, предотвращение распространенных ошибок, таких как `NullPointerException`, поддержку компактного и легкочитаемого кода, а также неограниченную интеграцию с Java.

2

Основы языка Kotlin

В этой главе

- ✓ Объявление функций, переменных, классов, перечислений и свойств.
- ✓ Управляющие структуры в Kotlin.
- ✓ Умное преобразование типов.
- ✓ Выброс и обработка исключений.

В данной главе вы изучите основы языка Kotlin, необходимые для разработки исполняемых программ. Вы узнаете о базовых строительных блоках программ, таких как переменные и функции, а также познакомитесь с различными способами представления данных в языке Kotlin с помощью перечислений, классов и их свойств.

Описанные в этой главе управляющие структуры необходимы для реализации условной логики в создаваемых программах, а также выполнения итераций с помощью циклов. Еще вы узнаете, чем эти конструкции отличаются от других языков, например Java.

Кроме того, вы познакомитесь с базовой механикой типов в Kotlin, начав с понятия *умного преобразования* (smart cast) — операции, объединяющей проверку и приведение типа. Вы увидите, как она помогает устранить избыточность в коде, не жертвуя безопасностью. Вдобавок мы кратко поговорим об обработке исключений и философии Kotlin, лежащей в ее основе. К концу главы вы сможете комбинировать базовые фрагменты языка Kotlin и писать собственный рабочий код, пусть и не самый идиоматичный.

ЧТО ТАКОЕ ИДИОМАТИЧНЫЙ KOTLIN

При обсуждении кода на языке Kotlin часто повторяется фраза: *идиоматичный Kotlin*. Вы наверняка будете встречать ее во время чтения этой книги, а также в разговорах с коллегами, на общественных мероприятиях и конференциях. Разумеется, необходимо понимать ее значение.

Попросту говоря, идиоматичный Kotlin — отражение того, *как* «носитель» Kotlin пишет код, используя возможности языка и «синтаксический сахар» там, где это необходимо. Такой код состоит из *идиом* — узнаваемых структур, способствующих решению задач в стиле Kotlin. Идиоматичный код вписывается в общепринятый в сообществе стиль программирования и следует рекомендациям разработчиков языка.

Как и любой другой навык, обучение идиоматичному написанию кода на языке Kotlin требует времени и практики. По мере прочтения этой книги, изучения предоставленных примеров кода и написания собственного кода у вас постепенно будет развиваться интуиция в отношении того, как выглядит идиоматичный код Kotlin. Вскоре вы сможете самостоятельно применять полученные знания в своих проектах.

2.1. ОСНОВНЫЕ ЭЛЕМЕНТЫ — ФУНКЦИИ И ПЕРЕМЕННЫЕ

В этом разделе вы познакомитесь с основными элементами, составляющими любую программу на языке Kotlin: с функциями и переменными. Вы напишете первую программу, увидите, как можно опускать многие объявления типов. Кроме того, вы узнаете, как избегать изменяемых данных, где это возможно, и почему это хорошо.

2.1.1. Написание первой программы на языке Kotlin: «Hello, world!»

Начнем путешествие по миру Kotlin с классического примера: программы, выводящей на экран фразу «Hello, world!». В языке Kotlin для этого требуется всего лишь одна функция (рис. 2.1).

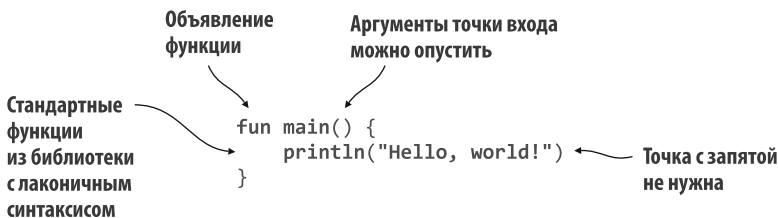


Рис. 2.1. Программа «Hello, world!» на языке Kotlin

Уже в этом простом фрагменте кода можно наблюдать несколько особенностей и элементов синтаксиса языка.

- Ключевое слово `fun` используется для объявления функции.
- Функция может быть объявлена в верхнем уровне любого файла Kotlin — нет необходимости помещать ее в класс.
- Функцию `main` можно указать в качестве точки входа для приложения на верхнем уровне и без дополнительных аргументов (в других языках необходимо принимать, например, массив параметров командной строки).
- Kotlin свойственна лаконичность: чтобы вывести текст в консоль, достаточно написать `println`. Стандартная библиотека Kotlin предоставляет множество оберток для стандартных функций библиотеки Java (например, `System.out.println`) с более лаконичным синтаксисом, и `println` — одна из них.
- Вы можете (и должны) опускать точку с запятой в конце строки, как и во многих других современных языках.

Пока все просто и понятно. А некоторые из этих тем мы более подробно обсудим позже. Теперь поговорим о синтаксисе объявления функции.

2.1.2. Объявление функций с параметрами и возвращаемыми значениями

Вы написали свою первую функцию, но она ничего не возвращает. Однако назначение функций часто состоит в том, чтобы вычислять некий результат и затем возвращать его. Например, вы можете написать простую функцию `max`, которая принимает два целых числа `a` и `b` и возвращает большее из них. Как же она будет выглядеть?

Объявление функции начинается с ключевого слова `fun`. За ним следует имя функции — в данном случае `max`. За ним в круглых скобках указывается список параметров. Здесь необходимо объявить два параметра, `a` и `b`, оба типа `Int`. В Kotlin сначала указывается имя параметра, а затем его тип, разделенные двоеточием. Тип возвращаемого значения размещается после списка параметров и отделяется от него двоеточием:

```
fun max(a: Int, b: Int): Int {  
    return if (a > b) a else b  
}
```

ПАРАМЕТРЫ И ТИП ВОЗВРАЩАЕМОГО ЗНАЧЕНИЯ ГЛАВНОЙ ФУНКЦИИ

В примере с выводом фразы «Hello, world!» показано, что точкой входа каждой программы Kotlin является ее главная функция — `main`. Она может быть объявлена либо без параметров, либо с массивом строк в качестве аргументов (`args: Array<String>`). В последнем случае каждый элемент массива соответствует параметру командной строки, передаваемому вашему приложению. В любом случае функция `main` не возвращает значения.

РАЗНИЦА МЕЖДУ ВЫРАЖЕНИЯМИ И ИНСТРУКЦИЯМИ

В языке Kotlin `if` — это выражение, а не инструкция. Разница между выражением и инструкцией состоит в том, что *выражение* содержит значение, которое может быть использовано как часть другого выражения. А *инструкция* всегда будет элементом верхнего уровня во вложенном блоке и не содержит собственного значения. В языке Kotlin большинство управляющих структур, за исключением циклов (`for`, `while` и `do/while`), — это выражения, и этим он отличается от других языков, таких как Java. В частности, возможность комбинировать управляющие структуры с другими выражениями позволяет лаконично выражать многие общие шаблоны. В следующем фрагменте кода показан наглядный пример конструкций языка Kotlin:

```
val x = if (myBoolean) 3 else 5
val direction = when (inputString) {
    "u" -> UP
    "d" -> DOWN
    else -> UNKNOWN
}
val number = try {
    inputString.toInt()
} catch (nfe: NumberFormatException) {
    -1
}
```

В то же время в Kotlin операции присвоения всегда относятся к инструкциям. То есть при присвоении значения переменной сама операция присвоения не возвращает значение.

Это различие позволяет избежать путаницы между операциями сравнения и присваивания. В таких языках, как Java или C/C++, присваивание рассматривается в качестве выражения, что зачастую становится источником ошибок в программе. Приведенный ниже код некорректен для Kotlin:

```
val number: Int
val alsoNumber = i = getNumber()
// ОШИБКА: Присваивания не являются выражениями,
// а в этом контексте допускается применение только выражений
```

На рис. 2.2 показана основная структура функции. Обратите внимание, что в Kotlin `if` — это выражение со значением результата вычисления. Представьте, что структура `if` возвращает значение из любой из своих ветвей. Поэтому она похожа на тернарный оператор в других языках, например в Java, где та же конструкция может выглядеть как `(a > b) ? a : b`.

Вызвать функцию можно по ее имени, указав аргументы в круглых скобках (о различных способах вызова функций Kotlin вы также узнаете в подразделе 3.2.1):

```
fun main() {
    println(max(1, 2))
    // 2
}
```

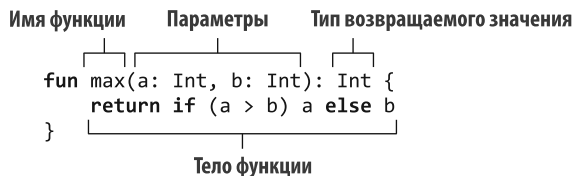


Рис. 2.2. Функция Kotlin представляется с помощью ключевого слова `fun`. Параметры и их типы следуют в круглых скобках, каждый из них аннотирован именем и типом, разделенными двоеточием. Тип возвращаемого значения указывается после окончания списка параметров. Функции, подобные этой, служат основными строительными блоками любой программы на языке Kotlin

2.1.3. Сокращенные определения функций с помощью тел выражений

Вы можете сократить функцию `max`. Ее тело состоит из однострочного выражения (`if (a > b) a else b`), поэтому можно использовать его в качестве всего тела функции, удалив фигурные скобки и оператор `return`. Однострочное выражение можно поместить сразу после знака равенства (=):

```
fun max(a: Int, b: Int): Int = if (a > b) a else b
```

Если функция написана так, что ее тело заключено в фигурные скобки, то она называется *функцией с блочным телом*. Если она возвращает непосредственно выражение, то у нее есть *тело выражения*.

Функции с телом выражения часто встречаются в коде Kotlin. Они очень удобны, когда функция представляет собой тривиальное однострочное выражение, позволяющее присвоить условной проверке или часто используемой операции запоминающееся имя. Но им можно найти применение еще и в том случае, когда функции вычисляют одно, более сложное выражение, например `if`, `when` или `try`. Такие функции мы рассмотрим позже в текущей главе, при обсуждении конструкции `when`.

Вы можете еще больше упростить свою функцию `max` и опустить тип возвращаемого значения:

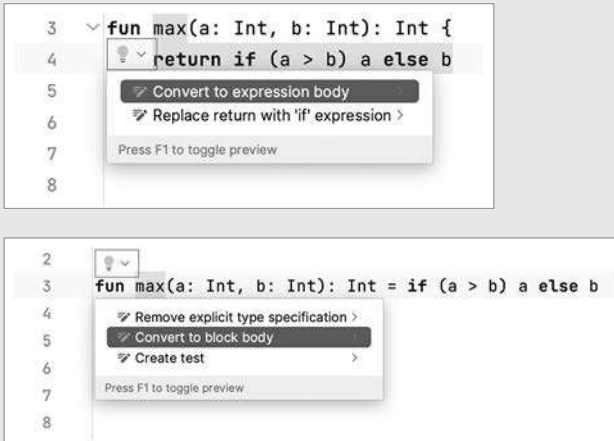
```
fun max(a: Int, b: Int) = if (a > b) a else b
```

На первый взгляд код может вызвать недоумение. Как могут существовать функции без объявления типа возвращаемого значения? Вы уже знаете, что Kotlin — статически типизированный язык, поэтому он требует, чтобы у каждого выражения использовался тип во время компиляции.

И действительно, у каждой переменной и выражения есть тип, а у каждой функции — тип возвращаемого значения. Но при использовании функций с телом выражения компилятор может проанализировать выражение, используемое в качестве тела функции, и применить его тип в качестве типа возвращаемого функцией значения, даже если он не указан в явном виде. Этот вид анализа обычно называют *выведением типов*.

ПРЕОБРАЗОВАНИЕ МЕЖДУ ТЕЛОМ ВЫРАЖЕНИЯ И БЛОЧНЫМ ТЕЛОМ В INTELLIJ IDEA И ANDROID STUDIO

Среды разработки IntelliJ IDEA и Android Studio предоставляют действия намерения для преобразования между двумя стилями функций: преобразование в тело выражения и преобразование в блочное тело. Их можно найти с помощью значка лампочки, который появляется при наведении указателя мыши на имя функции, или путем нажатия сочетания клавиш `Alt+Enter` (или `Option+Return` в macOS).



Обратите внимание: опускать возвращаемый тип можно только для функций с телом выражения. При использовании функций с блочным телом, возвращающих значение, необходимо указать тип возвращаемого значения и обязательно написать оператор `return`. Это должен быть осознанный выбор. Часто в работе встречаются длинные функции, содержащие несколько инструкций `return`. Поэтому явное указание типа возвращаемого значения и запись инструкций `return` поможет вам быстро понять, какой результат может быть возвращен. Далее рассмотрим синтаксис объявления переменных.

ПРИ РАЗРАБОТКЕ БИБЛИОТЕК ЯВНО УКАЗЫВАЙТЕ ТИПЫ ВОЗВРАЩАЕМЫХ ЗНАЧЕНИЙ

Если вы создаете библиотеки не только для личного пользования, то вам стоит воздержаться от использования вероятных типов возвращаемых значений для функций, которые являются частью вашего публичного API. Явно указывая типы своих функций, вы можете избежать случайных изменений сигнатур — действий, которые могут привести к ошибкам в коде пользователей вашей библиотеки. И Kotlin предоставляет инструменты в виде параметров компилятора. Они дают возможность автоматически проверять, что вы явно указываете типы возвращаемых значений. Подробнее этот *явный режим API* описывается в подразделе 4.1.3.

2.1.4. Объявление переменных для хранения данных

В программах на языке Kotlin есть еще один часто используемый базовый строительный блок — это переменные для хранения данных. Объявление переменной в Kotlin начинается с ключевого слова (`val` или `var`), за которым следует имя переменной. Kotlin позволяет не указывать тип во многих местах объявления переменных (благодаря мощному механизму *выведения типа*, описанному в подразделе 2.1.3), но вы всегда можете явно указать тип после имени переменной. Допустим, в Kotlin требуется сохранить распространенный вопрос и соответствующий ответ на него. Это можно сделать, указав две переменные, `question` и `answer`, с явными типами — `String` для текстового вопроса и `Int` для целочисленного ответа:

```
val question: String = "The Ultimate Question of Life, the Universe and Everything"
val answer: Int = 42
```

И можно опустить объявления типов, сделав пример более лаконичным:

```
val question = "The Ultimate Question of Life, the Universe and Everything"
val answer = 42
```

Как и в случае с функциями с телами выражений, если вы не указываете тип, то компилятор анализирует инициализирующее выражение и использует его тип в качестве типа переменной. В данном случае инициализатор `42` относится к типу `Int`, поэтому переменная `answer` получит тот же тип.

Если вы используете константу с плавающей запятой, то переменная получит тип `Double`:

```
val yearsToCompute = 7.5e6    ←     $7,5 \times 10^6 = 7\,500\,000$ 
```

Численные, а также другие базовые типы более подробно рассматриваются в разделе 8.1.

Если вы не инициализируете переменную сразу, а присваиваете ей значение позже, то компилятор не сможет определить тип переменной. В таком случае необходимо явно указать ее тип:

```
fun main() {
    val answer: Int
    answer = 42
}
```

2.1.5. Переназначаемые переменные и переменные только для чтения

Чтобы управлять присвоением значений переменным, в Kotlin для объявления переменных используется два ключевых слова, `val` и `var`.

- `val` (от слова *value* — «значение») объявляет *ссылку, доступную только для чтения*. Значение переменной, объявленной с помощью ключевого слова `val`, может быть присвоено только раз. После инициализации ей нельзя присвоить

другое значение. (Для сравнения: в Java это выражается с помощью модификатора `final`.)

- `var` (от слова *variable* — «переменная») объявляет *ссылку с возможностью переназначения*. Такой переменной можно присваивать другие значения даже после ее инициализации. (Такое поведение аналогично обычной переменной без модификатора в Java.)

По умолчанию объявляйте все переменные в Kotlin с помощью ключевого слова `val` — меняйте его на `var` только в случае необходимости. Использование ссылок только для чтения, неизменяемых объектов и функций без побочных эффектов позволяет воспользоваться преимуществами *функционального стиля* программирования. Мы кратко обсудили преимущества этого стиля в подразделе 1.2.3 и вернемся к данной теме в главе 5.

Переменная типа `val` должна быть инициализирована однократно во время выполнения блока, в котором она определена. Однако ее можно инициализировать разными значениями в зависимости от какого-либо условия, если компилятор способен обеспечить выполнение только одного из операторов инициализации.

У вас может сложиться ситуация, когда требуется присвоить содержимое переменной `result`, зависящей от возвращаемого значения другой функции, например `canPerformOperation`. Компилятор распознает, что будет выполнена только одна из двух потенциальных операций присвоения, поэтому вы можете указать `result` как ссылку только для чтения, используя ключевое слово `val`:

```
fun canPerformOperation(): Boolean {
    return true
}
fun main() {
    val result: String
    if (canPerformOperation()) {
        result = "Success"
    } else {
        result = "Can't perform operation"
    }
}
```

Обратите внимание: сама ссылка `val` предназначена только для чтения и не может быть изменена после присвоения, но объект, на который она указывает, может быть изменяемым. Например, добавлять элемент в изменяемый список, на который указывает ссылка, доступная только для чтения, вполне допустимо:

```
fun main() {
    val languages = mutableListOf("Java")
    languages.add("Kotlin")
}
```

Объявление ссылки только для чтения

Изменение объекта, на который указывает ссылка, путем добавления элемента

В подразделе 8.2.2 мы подробно рассмотрим изменяемые и доступные только для чтения объекты.

Ключевое слово `var` позволяет переменной менять свое значение, однако ее тип остается фиксированным. Например, если в середине программы вы решите, что переменная `answer` должна хранить строку, а не целое число, то возникнет ошибка компиляции:

```
fun main() {
    var answer = 42
    answer = "no answer"  ← Ошибка: несоответствие типов
}
```

В строковом литерале произошла ошибка, поскольку его тип (`String`) не соответствует ожидаемому (`Int`). Компилятор выводит тип переменной только из инициализатора и не учитывает последующие присвоения при определении типа.

Если требуется сохранить в переменной значение несоответствующего типа, то необходимо вручную преобразовать или привести его к нужному типу. О преобразовании чисел мы поговорим в подразделе 8.1.4.

Теперь, когда вы знаете, как определять переменные, пришло время рассмотреть несколько новых приемов, с помощью которых можно обращаться к значениям этих переменных. В частности, вы увидите, как получить более читабельные и структурированные результаты ваших первых программ на Kotlin.

2.1.6. Упрощенное форматирование строк с помощью шаблонов

Вернемся к примеру «Hello, world!» и расширим его некоторыми дополнительными функциями, часто встречающимися во всех видах Kotlin-программ. Добавим элемент персонализации, настроив в программе приветствие людей по имени: если пользователь указывает имя через стандартный ввод, то программа использует его в приветствии; если пользователь не вводит никаких данных, то программа просто поприветствует Kotlin. Такая приветственная программа может выглядеть следующим образом (листинг 2.1). Кстати, она содержит несколько новых для вас функций.

Листинг 2.1. Использование строковых шаблонов

```
fun main() {
    val input = readln()
    val name = if (input.isNotBlank()) input else "Kotlin"
    println("Hello, $name!")  ← Вывод фразы «Hello, Kotlin!» или «Hello, Bob!», если ввести «Bob»
                              через стандартный поток ввода (например, через терминал)
}
```

В этом примере представлен *строковый шаблон*, а также кратко показан пример считывания простого пользовательского ввода. В этом фрагменте кода данные для переменной `input` считываются из стандартного потока ввода с помощью функции `readln()` (которая, наряду с другими, есть в любом файле Kotlin). Далее объявляется переменная `name` и ее значение инициализируется с помощью выражения `if`. Если стандартный ввод использовался и в нем есть данные, то переменной `name` присваивается значение переменной `input`. В противном случае ей присваивается

значение по умолчанию "Kotlin". И наконец, она используется в строковом литерале, переданном в функцию `println`.

Как и многие сценарные языки, Kotlin позволяет ссылаться на локальные переменные в строковых литералах. Для этого необходимо указать перед именем переменной символ `$`. Это действие эквивалентно конкатенации строк в Java ("Hello, " + name + "!"), но более компактно и так же эффективно. И конечно, выражения проверяются статически, и код не скомпилируется, если обратиться к несуществующей переменной.

ПРИМЕЧАНИЕ

При использовании JVM 1.8 скомпилированный код создает `StringBuilder` и добавляет в него константные части и значения переменных. Приложения, ориентированные на JVM 9 и выше, компилируют конкатенации строк в более эффективные динамические вызовы, используя инструкцию `invokedynamic`.

Если необходимо добавить символ `$` в строку, то его следует экранировать обратным слешем:

```
fun main() {
    println("\$x")
    // $x
}
```

← `x` не интерпретируется как ссылка на переменную

Строковые шаблоны в Kotlin действуют весьма эффективно, поскольку не ограничивают разработчика ссылками на отдельные переменные. Если нужно, чтобы создаваемое приветствие было более авантюрным и приветствовало пользователя, называя количество букв в его имени, то можно указать более сложное выражение в строковом шаблоне. Для этого достаточно заключить выражение в фигурные скобки:

```
fun main() {
    val name = readln()
    if (name.isNotBlank()) {
        println("Hello, ${name.length}-letter person!")
    }
    // Пустой ввод: (без результата на выходе)
    // Введено "Seb" : Hello, 3-letter person!
}
```

← Синтаксис `${}` для вставки свойства длины переменной `name` в строку приветствия

На данный момент вы можете добавить произвольные выражения в строковые шаблоны и уже знаете, что `if` — это выражение в Kotlin. Объединив эти приемы, можно переписать программу приветствия, чтобы добавить условие непосредственно в строковый шаблон:

```
fun main() {
    val name = readln()
    println("Hello, ${if (name.isBlank()) "someone" else name}!")
    // Пустой ввод: Hello, someone!
    // Введено "Seb" : Hello, Seb!
}
```

Обратите внимание: в строковом шаблоне можно использовать выражение, содержащее строку с двойными кавычками. Когда оно вычисляется, то возвращает либо строку `someone`, либо указанное имя и вставляет его в окружающий шаблон. В разделе 3.5 мы вернемся к строкам и поговорим о том, что можно с ними делать.

Итак, мы обсудили несколько основных строительных блоков для написания собственных программ на языке Kotlin: функции и переменные. Теперь поднимемся на более высокую ступеньку иерархии и рассмотрим классы и то, как они помогают инкапсулировать и группировать связанные данные в объектно-ориентированном стиле. На этот раз вы воспользуетесь конвертером из Java в Kotlin, который поможет вам начать применять новые возможности языка.

2.2. ИНКАПСУЛЯЦИЯ ПОВЕДЕНИЯ И ДАННЫХ — КЛАССЫ И СВОЙСТВА

Как и другие объектно-ориентированные языки программирования, Kotlin предоставляет абстракцию *класса*. Концепции Kotlin в этой области вам знакомы, но многие распространенные задачи решаются путем написания гораздо меньшего количества кода, чем в других объектно-ориентированных языках. В этом разделе вы познакомитесь с основным синтаксисом объявления классов. Более подробно о них мы поговорим в главе 4.

Для начала рассмотрим простой Java-объект (Plain Old Java Object, POJO) класса `Person`, содержащий на данный момент только одно свойство — `name` (листинг 2.2).

Листинг 2.2. Простой класс Java под названием `Person`

```
/* Java */
public class Person {
    private final String name;

    public Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Очевидно, что в Java требуется довольно много кода. Тело конструктора полностью повторяется, поскольку только присваивает параметры полям с соответствующими именами. По принятому соглашению для доступа к полю `name` класс `Person` должен предоставлять геттер-функцию `getName`, которая просто возвращает содержимое поля. Подобные повторы часто встречаются в Java. В Kotlin эту логику можно выразить без использования большого количества шаблонов.

В главе 1 мы представили конвертер Java в Kotlin — инструмент, который автоматически заменяет код на Java кодом на Kotlin, выполняющим те же операции.

Посмотрим на конвертер в действии и преобразуем класс `Person` на язык Kotlin (листинг 2.3).

Листинг 2.3. Класс `Person` преобразован в Kotlin

```
class Person(val name: String)
```

Уже лучше. Если вы пробовали работать на другом современном объектно-ориентированном языке, то, возможно, видели нечто подобное. В Kotlin используется лаконичный синтаксис для объявления классов, особенно тех, которые содержат только данные, но не код. В главе 4 мы обсудим их связь с очень похожей концепцией в Java, доступной начиная с версии 14 и далее, — *записями*.

Обратите внимание, что модификатор `public` не применяется при переходе с Java на Kotlin. В Kotlin он используется по умолчанию, поэтому его можно опустить.

2.2.1. Ассоциирование данных с классом и обеспечение доступа к ним: свойства

Суть класса заключается в инкапсуляции данных и работающего с ними кода в единое целое. В Java данные хранятся в полях, как правило приватных. Если вам нужно предоставить клиентам класса доступ к этим данным, то вы предоставляете *методы доступа*: геттер и, возможно, сеттер. В классе `Person` показан пример такого метода. Кроме того, сеттер может содержать дополнительную логику, предназначенную для проверки переданного значения, отправки уведомлений об изменении и т. д.

В Java комбинацию поля и его методов доступа часто называют *свойством*, и многие фреймворки активно используют эту концепцию. В языке Kotlin свойства — это функция первого класса языка, которая полностью заменяет поля и методы доступа. Вы объявляете свойство в классе так же, как и переменную: с помощью ключевых слов `val` и `var`. Свойство, объявленное как `val`, доступно только для чтения, в то время как свойство `var` может быть изменено. Например, вы можете расширить класс `Person`, содержащий свойство `name`, доступное только для чтения, свойством `isStudent`, которое может измениться (листинг 2.4).

Листинг 2.4. Объявление изменяемого свойства в классе

```
class Person(
    val name: String,
    var isStudent: Boolean
)
```

Свойство только для чтения —
генерирует поле и тривиальный геттер

Записываемое свойство —
поле, геттер и сеттер

По сути, при объявлении свойства вы объявляете и соответствующие методы доступа (геттер для свойства, доступного только для чтения; геттер и сеттер для свойства, доступного для записи). По умолчанию реализация этих методов доступа тривиальна: создается поле для хранения значения, геттер возвращает значение этого поля, а сеттер обновляет его значение. Но при желании можно объявить собственный метод доступа, реализующий другую логику вычисления или обновления значения свойства.

Лаконичное объявление класса `Person` в листинге 2.4 скрывает ту же реализацию, что и исходный код Java: это класс с приватными полями, которые инициализируются в конструкторе и к которым можно получить доступ через соответствующий геттер. Это значит, что данный класс можно использовать из Java и из Kotlin одинаково, независимо от того, где он был объявлен. Применение будет идентичным. В листинге 2.5 показано, как можно использовать класс Kotlin `Person` из кода Java, создав новый объект `Person` с именем `Bob`. До выпускного он будет находиться в статусе студента.

Листинг 2.5. Использование класса `Person` из Java

```
public class Demo {
    public static void main(String[] args) {
        Person person = new Person("Bob", true);
        System.out.println(person.getName());
        // Bob
        System.out.println(person.isStudent());
        // истина
        person.setStudent(false); // Выпускной!
        System.out.println(person.isStudent());
        // ложь
    }
}
```

Обратите внимание, что логика выполнения одинакова при определении класса `Person` на Java и Kotlin. Свойство Kotlin `name` отображается в Java в виде геттер-метода `getName`. В правиле именования геттеров и сеттеров есть исключение: если имя свойства начинается со слова `is`, то дополнительный префикс для геттера не добавляется, а в имени сеттера `is` заменяется на `set`. Таким образом, из Java вы можете вызвать методы `isStudent()` и `setStudent()`, чтобы получить доступ к свойству `isStudent`.

Результат преобразования кода из листинга 2.5 в Kotlin показан в листинге 2.6.

Листинг 2.6. Использование класса `Person` из Kotlin

```
fun main() {
    val person = Person("Bob", true)
    println(person.name)
    // Bob
    println(person.isStudent)
    // истина
    person.isStudent = false // Выпускной!
    println(person.isStudent)
    // ложь
}
```

Вызов конструктора без ключевого слова `new`

Доступ к свойству осуществляется напрямую, но при этом вызывается геттер

Значение свойству присваивается напрямую, но при этом вызывается сеттер

Теперь вместо явного вызова геттера указывается ссылка на свойство напрямую. Логика осталась прежней, но код стал более лаконичным. Аналогичным образом работают и сеттеры изменяемых свойств. Если в Java для обозначения окончания обучения используется метод `person.setStudent(false)`, то в Kotlin применяется синтаксис свойства напрямую: `person.isStudent = false`.

СОВЕТ

Вы также можете использовать синтаксис свойств Kotlin для классов, определенных на Java. Геттеры в Java-классе могут быть доступны как свойства `val` из Kotlin, а пары «геттер — сеттер» — как свойства `var`. Например, если в Java-классе определены методы `getName` и `setName`, то можно получить к ним доступ как к свойству `name`. Если в классе определены методы `isStudent` и `setStudent`, то имя соответствующего свойства Kotlin будет иметь следующий вид: `isStudent`.

В большинстве случаев у свойства есть соответствующее *теневое поле* (backing field), которое просто хранит его значение (подробнее теневые поля в Kotlin описаны в подразделе 4.2.4). Но если значение может быть вычислено динамически (например, путем извлечения его из других свойств), то можно выразить это с помощью пользовательского геттера.

2.2.2. Вычисление свойств вместо хранения их значений: переопределенные методы доступа

Рассмотрим, как создать собственную реализацию метода доступа к свойству. Один из распространенных случаев — когда свойство является прямым результатом некоторых других свойств объекта. Если в программе есть класс `Rectangle`, содержащий свойства `width` и `height`, то можно предоставить свойство `isSquare`, получающее значение `true`, когда значения `width` и `height` равны. Поскольку это свойство, то его можно проверять динамически, вычисляя значение при доступе к нему — нет необходимости хранить эту информацию в отдельном поле. Вместо этого можно предоставить пользовательский геттер, реализация которого будет вычислять «квадратность» объекта класса `Rectangle` при каждом обращении к свойству:

```
class Rectangle(val height: Int, val width: Int) {
    val isSquare: Boolean
        get() { ◀— Объявление геттера свойства
            return height == width
        }
}
```

Обратите внимание, что не обязательно использовать полный синтаксис с фигурными скобками. Как и в любой другой функции, можно определить геттер, используя синтаксис с телом выражения, описанный в подразделе 2.1.3, и просто написать `val isSquare get() = height == width`. К тому же этот синтаксис позволяет не указывать тип свойства в явном виде, и компилятор выводит тип самостоятельно.

Независимо от выбранного вами синтаксиса вызов свойства `isSquare` остается неизменным:

```
fun main() {
    val rectangle = Rectangle(41, 43)
    println(rectangle.isSquare)
    // ложь
}
```

Если требуется получить доступ к этому свойству из Java, то вызывается метод `isSquare`, как и раньше.

Может возникнуть вопрос, что лучше: объявить свойство с пользовательским геттером или определить функцию внутри класса (в Kotlin она называется *функцией-членом* или *методом*). Оба варианта похожи: реализация или производительность ничем не отличается, разница лишь в удобстве чтения. Как правило, если вы описываете характеристику (свойство) класса, то необходимо объявить ее как свойство. Если же описываете поведение класса, то выберите функцию-член (метод).

В главе 4 приводится больше примеров, в которых используются классы, свойства и функции-члены, а также в ней описывается синтаксис для явного объявления конструкторов. Если вы нетерпеливы и знаете эквиваленты этих концепций на языке Java, то всегда можете воспользоваться конвертером из Java в Kotlin, чтобы заглянуть немного вперед. Прежде чем переходить к другим возможностям языка, кратко рассмотрим, как в целом структурируется код в проектах Kotlin.

2.2.3. Компоновка исходного кода Kotlin: каталоги и пакеты

Усложняясь, разрабатываемые программы содержат все больше функций, классов и других языковых конструкций. Поэтому вам неизбежно придется задуматься об организации исходного кода, чтобы проект оставался удобным для сопровождения и навигации. Рассмотрим, как обычно устроены проекты Kotlin.

В Kotlin используется концепция *пакетов* для организации классов (механизм, аналогичный Java). Каждый файл Kotlin может содержать в начале инструкцию `package`, и все объявления (классы, функции и свойства), определенные в файле, будут помещены в этот пакет.

В листинге 2.7 приведен пример исходного файла, содержащего синтаксис инструкции объявления пакета.

Листинг 2.7. Размещение объявления класса и функции в пакете

`package geometry.shapes` ← Объявление пакета

```
class Rectangle(val height: Int, val width: Int) {
    val isSquare: Boolean
        get() = height == width
}
```

```
fun createUnitSquare(): Rectangle {
    return Rectangle(1, 1)
}
```

Объявления, определенные в других файлах, можно использовать напрямую, если они находятся в том же пакете. Если же они расположены в другом, то их нужно *импортировать*. Импорт осуществляется с помощью ключевого слова `import` в начале файла, расположенного непосредственно под директивой `package`.

В Kotlin нет разницы между импортом классов или функций и с помощью ключевого слова `import` можно импортировать любые объявления. Если вы пишете демонстрационный проект в пакете `geometry.example`, то можете использовать класс `Rectangle` и функцию `createUnitSquare` из пакета `geometry.shapes`, просто импортировав их по имени (листинг 2.8).

Листинг 2.8. Импорт функции из другого пакета

```
package geometry.example

import geometry.shapes.Rectangle ← Импорт класса Rectangle по имени
import geometry.shapes.createUnitSquare ← Импорт функции createUnitSquare по имени

fun main() {
    println(Rectangle(3, 4).isSquare)
    // ложь
    println(createUnitSquare().isSquare)
    // истина
}
```

Кроме того, можно импортировать все объявления, определенные в конкретном пакете, написав `.*` после имени пакета. Обратите внимание, что при таком *импорте «со звездочкой»* (также называемом *импортом с подстановочным знаком*) становится доступно все, что определено в пакете: не только классы, но и функции и свойства верхнего уровня. Если в листинге 2.8 написать строку `import geometry.shapes.*` вместо явного импорта, то код будет корректно компилируемым.

В Java классы помещаются в структуру файлов и каталогов, соответствующую структуре пакета. Например, если у вас есть пакет `shapes` с несколькими классами, то вам нужно поместить каждый класс в отдельный файл с соответствующим именем и хранить эти файлы в каталоге `shapes`. На рис. 2.3 показано, как пакет `geometry` и его подпакеты могут быть организованы в Java. Предположим, что функция `createUnitSquare` находится в отдельном файле — `RectangleUtil`.

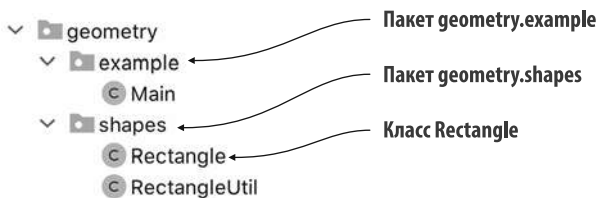


Рис. 2.3. В Java иерархия каталогов дублирует иерархию пакетов

В Kotlin можно поместить несколько классов в один файл и выбрать для него любое имя. Вдобавок язык не накладывает никаких ограничений на расположение исходных файлов на диске: вы можете использовать любую структуру каталогов для организации своих файлов. Например, можно определить все содержимое пакета `geometry.shapes` в файле `shapes.kt` и поместить этот файл в папку `geometry`, не создавая отдельной папки `shapes` (рис. 2.4).

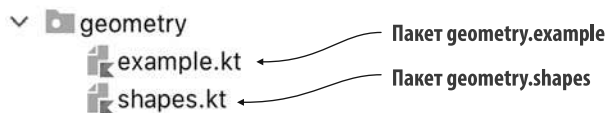


Рис. 2.4. Иерархия пакетов необязательно должна совпадать с иерархией каталогов

В большинстве случаев рекомендуется помещать исходные файлы в каталоги в соответствии со структурой пакета, следуя схеме каталогов Java. Придерживаться такой структуры особенно важно в проектах, где Kotlin смешивается с Java, поскольку код переносится постепенно, и при этом не возникает никаких проблем. Но можно и объединять несколько классов в один файл, особенно если классы небольшие (а в Kotlin это частое явление).

Теперь вы знаете структуру программ. Вернемся к базовым концепциям Kotlin и поговорим о том, как обрабатывать различные варианты выбора в Kotlin, помимо выражения `if`.

2.3. ПРЕДСТАВЛЕНИЕ И ОБРАБОТКА ВАРИАНТОВ ВЫБОРА: ПЕРЕЧИСЛЕНИЯ И ВЫРАЖЕНИЕ WHEN

В этом разделе мы рассмотрим пример объявления перечислений в Kotlin и поговорим о конструкции `when`. Последнюю можно рассматривать как более мощную и часто используемую замену конструкции `switch` в Java. Вдобавок мы обсудим концепцию умных преобразований, сочетающих в себе проверку и приведение типов.

2.3.1. Объявление классов и констант перечислений

Пришло время добавить в книгу ярких красок! Учитывая, что она черно-белая, вам придется использовать воображение: вам предстоит реализовать яркие образы в виде кода Kotlin, а именно в виде перечисления цветовых констант (листинг 2.9).

Листинг 2.9. Объявление простого класса перечисления

```
package ch02.colors

enum class Color {
    RED, ORANGE, YELLOW, GREEN, BLUE, INDIGO, VIOLET
}
```

Это редкий случай, когда в объявлении Kotlin используется больше ключевых слов, чем в соответствующем выражении на языке Java: `enum class` вместо просто `enum` в Java.

ENUM — МЯГКОЕ КЛЮЧЕВОЕ СЛОВО

В Kotlin `enum` — так называемое *мягкое ключевое слово*. Оно приобретает особое значение, когда указывается перед словом `class`, но вы можете использовать его как обычное имя (например, для функции, имени переменной или параметра) на других участках кода. В свою очередь, `class` — *жесткое ключевое слово*. Его нельзя использовать в качестве идентификатора: вам придется использовать альтернативное написание или формулировку, например `cLazz` или `aClass`.

Хранить цвета в перечислении полезно, но есть более подходящий метод. Значения цвета часто представляют с помощью красной, зеленой и синей составляющих. Константы перечисления используют тот же синтаксис объявления конструктора и свойства, который был представлен ранее для обычных классов. Он используется для расширения перечисления `Color` — можно связать каждую константу перечисления с ее значениями `r`, `g` и `b`. Кроме того, вы можете объявлять свойства, например `rgb`, которые могут создавать комбинированное числовое значение цвета из компонентов, и методы, например `printColor`, используя знакомый синтаксис (листинг 2.10).

Листинг 2.10. Объявление класса `enum` со свойствами

```
package ch02.colors
```

```
enum class Color(
    val r: Int,
    val g: Int,
    val b: Int
) {
    RED(255, 0, 0),
    ORANGE(255, 165, 0),
    YELLOW(255, 255, 0),
    GREEN(0, 255, 0),
    BLUE(0, 0, 255),
    INDIGO(75, 0, 130),
    VIOLET(238, 130, 238);
    val rgb = (r * 256 + g) * 256 + b
    fun printColor() = println("$this is $rgb")
}

fun main() {
    println(Color.BLUE.rgb)
    // 255
    Color.GREEN.printColor()
    // GREEN = 65280
}
```

Объявление свойств констант перечисления

Указание значения свойств при создании каждой константы

Здесь использование точки с запятой обязательно

Определение свойства класса перечислений

Определение метода класса перечислений

Обратите внимание, что в этом примере показано единственное место в синтаксисе Kotlin, где требуется использовать точку с запятой: если вы определяете

какие-либо методы в классе перечислений, то точка с запятой отделяет список констант перечислений от определений методов. Теперь, когда у нас есть полноценное перечисление `Colors`, посмотрим, как Kotlin позволяет работать с этими константами.

2.3.2. Обработка классов перечислений с помощью выражения `when`

Помните ли вы mnemonicские фразы, которые дети используют для запоминания цветов радуги? Вот одна из них: «*Каждый Охотник Желает Знать, Где Сидит Фазан*»¹. Представьте, что вам нужна функция с mnemonicкой для каждого цвета (и вы не хотите хранить эту информацию в самом перечислении). В Java для этого можно использовать оператор `switch` или, начиная с Java 13, выражение `switch`. Соответствующая конструкция в Kotlin носит название `when`.

Как и `if`, `when` представляет собой выражение, возвращающее значение. Поэтому можно написать функцию с телом выражения, возвращающую выражение `when` напрямую. При описании функций в начале главы мы обещали пример многострочной функции с телом выражения. В листинге 2.11 показан такой пример.

Листинг 2.11. Использование выражения `when` для выбора правильного значения перечисления

```
fun getMnemonic(color: Color) = ← Возврат выражения when напрямую
    when (color) {
        Color.RED -> "Richard"
        Color.ORANGE -> "Of"
        Color.YELLOW -> "York"
        Color.GREEN -> "Gave"
        Color.BLUE -> "Battle"
        Color.INDIGO -> "In"
        Color.VIOLET -> "Vain"
    }
    ← Возврат соответствующей строки,
    если цвет равен константе перечисления

fun main() {
    println(getMnemonic(Color.BLUE))
    // Battle
}
```

Код находит ветвь, соответствующую переданному значению `color`. Обратите внимание, что не нужно писать операторы `break` для каждой ветви. (В Java ошибки часто возникают из-за пропущенного ключевого слова `break` в операторе `switch`.) При успешном совпадении выполняется только соответствующая ветвь. Кроме того, можно объединить несколько значений в ветку, разделив их запятыми. Чтобы использовать различные ветви в зависимости от «теплоты» цвета, можно

¹ В английском языке используется mnemonicка *Richard Of York Gave Battle In Vain*, и в коде ниже приводится именно она. — *Примеч. пер.*

сгруппировать константы перечисления соответствующим образом в выражении `when` (листинг 2.12).

Листинг 2.12. Комбинирование вариантов в одной ветке `when`

```
fun measureColor() = Color.ORANGE
// как запасной вариант для более сложной логики измерений

fun getWarmthFromSensor(): String {
    val color = measureColor()
    return when(color) {
        Color.RED, Color.ORANGE, Color.YELLOW -> "warm (red = ${color.r})"
        Color.GREEN -> "neutral (green = ${color.g})"
        Color.BLUE, Color.INDIGO, Color.VIOLET -> "cold (blue = ${color.b})"
    }
}

fun main() {
    println(getWarmthFromSensor())
    // warm (red = 255)
}
```

В этих примерах константы перечисления используются по их полному имени, с указанием имени класса перечисления `Color` при каждой ссылке на одну из констант перечисления. Импортируя значения констант, вы можете упростить этот код и избавиться от повторений. Как это сделать, показано в листинге 2.13.

Листинг 2.13. Импорт констант перечисления для доступа к ним без указания класса

```
import ch02.colors.Color.* ← Прямой импорт констант перечисления
                             для использования их по именам

fun measureColor() = ORANGE

fun getWarmthFromSensor(): String {
    val color = measureColor()
    return when (color) {
        RED, ORANGE, YELLOW ->
            "warm (red = ${color.r})"
        GREEN ->
            "neutral (green = ${color.g})"
        BLUE, INDIGO, VIOLET -> 2*((014-4))
            "cold (blue = ${color.b})"
    }
}
```

Использование импортированных констант по именам

2.3.3. Захват субъекта выражения `when` в переменную

В предыдущих примерах субъектом выражения `when` была переменная `color`, полученная путем вызова функции `measureColor()`. Чтобы не загромождать окружающий код лишними переменными, как в данном случае `color`, выражение `when` также может передавать свой субъект в переменную (листинг 2.14). В этом случае область

видимости захваченной переменной ограничивается телом выражения `when`, но при этом обеспечивается доступ к ее свойствам внутри ветвей выражения `when`.

Листинг 2.14. Присвоение субъекта `when` переменной

```
import ch02.colors.Color.* ← Импорт констант (RED, ORANGE и т. д.)

fun measureColor() = ORANGE

fun getWarmthFromSensor() =
    when (val color = measureColor()) {
        RED, ORANGE, YELLOW -> "warm (red = ${color.r})"
        GREEN -> "neutral (green = ${color.g})"
        BLUE, INDIGO, VIOLET -> "cold (blue = ${color.b})"
    }
```

Захват субъекта выражения `when` в переменную `color`, область действия которой ограничена телом `when`

Можно получить доступ к свойствам захваченной переменной

В примерах далее будут использоваться короткие имена перечислений, но в целях упрощения примеров кода мы опустим указание импорта. Стоит отметить, что если `when` применяется как выражение (то есть его результат используется в присваивании или в качестве возвращаемого значения), то компилятор принудительно превращает конструкцию в *исчерпывающую* (exhaustive). Это означает, что *все возможные пути выполнения* должны возвращать значение.

В предыдущем примере конструкция `when` стала исчерпывающей, поскольку все константы перечисления были охвачены. Но можно было и предусмотреть случай по умолчанию с использованием ключевого слова `else`. Когда компилятор не может определить, охвачены ли все возможные пути выполнения, необходимо предоставить ему случай по умолчанию. Мы рассмотрим такой пример в следующем подразделе.

2.3.4. Использование выражения `when` с произвольными объектами

Конструкция `when` в Kotlin на самом деле более гибкая, чем привычные для других языков аналоги: любой объект можно использовать в качестве условия ветвления. Напишем функцию для смешивания двух цветов в небольшой палитре. В этой задаче не так много вариантов, можно перечислить их все без особого труда (листинг 2.15).

Листинг 2.15. Использование различных объектов в ветвях выражения `when`

```
fun mix(c1: Color, c2: Color) =
    when (setOf(c1, c2)) {
        setOf(RED, YELLOW) -> ORANGE
        setOf(YELLOW, BLUE) -> GREEN
        setOf(BLUE, VIOLET) -> INDIGO
        else -> throw Exception("Dirty color")
    }

fun main() {
    println(mix(BLUE, YELLOW))
    // GREEN
}
```

Аргументом выражения `when` может быть любой объект. Он проверяется на равенство условиями ветви

Перечисление пар цветов, доступных для смешивания

Выполняется, если ни в одной из ветвей не было обнаружено совпадения

Если цвета, обозначенные как `c1` и `c2`, — это `RED` (Красный) и `YELLOW` (Желтый), то результатом их смешивания станет `ORANGE` (Оранжевый) и т. д. Для реализации используется *сравнение множеств*. Стандартная библиотека Kotlin содержит функцию `setOf`. Она создает множество `Set` с объектами, указанными в качестве аргументов. *Множество* — это коллекция с неупорядоченными элементами. Два множества равны, если содержат одинаковые элементы. Таким образом, если множества `setOf(c1, c2)` и `setOf(RED, YELLOW)` равны, то либо `c1` равно `RED`, а `c2` равно `YELLOW`, либо наоборот. Именно это условие и требуется проверить.

Выражение `when` сопоставляет свой аргумент со всеми ветвями по порядку, пока не будет выполнено заданное условие ветви. Таким образом, `setOf(c1, c2)` проверяется на равенство сначала с `setOf(RED, YELLOW)`, а затем с другими множествами цветов поочередно. Если ни одно из иных условий ветви не выполняется, то запускается ветвь `else`. Компилятор Kotlin не может сделать вывод о том, что все возможные комбинации наборов цветов были охвачены, а результат выражения `when` используется в качестве возвращаемого значения для функции `mix`. Поэтому необходимо предусмотреть случай по умолчанию, чтобы выражение `when` гарантированно стало исчерпывающим.

Возможность использовать любое выражение в качестве условия ветвления `when` позволяет писать лаконичный и красивый код в большинстве случаев. В этом примере условием является проверка равенства, а позже вы увидите, что условием может быть любое логическое выражение.

2.3.5. Использование выражения `when` без аргумента

В листинге 2.15 привлекает внимание некоторая неэффективность кода. При каждом вызове функции создается несколько экземпляров `Set`. Они используются только для проверки того, совпадают ли два цвета с двумя другими. Обычно это не вызывает проблем, но если функция вызывается часто, то стоит переписать код, чтобы не создавать большое количество недолговечных объектов, которые нужно будет очищать сборщиком мусора. Для этого можно использовать выражение `when` без аргумента, как показано в листинге 2.16. Код в данном случае хуже читается, но такую цену часто приходится платить за повышение производительности.

Листинг 2.16. Использование выражения `when` без аргументов

```
fun mixOptimized(c1: Color, c2: Color) =
    when {
        (c1 == RED && c2 == YELLOW) ||
        (c1 == YELLOW && c2 == RED) ->
            ORANGE
        (c1 == YELLOW && c2 == BLUE) ||
        (c1 == BLUE && c2 == YELLOW) ->
            GREEN
        (c1 == BLUE && c2 == VIOLET) ||
        (c1 == VIOLET && c2 == BLUE) ->
            INDIGO
    }
```

← У выражения `when` нет аргументов

```

        else -> throw Exception("Dirty color")
    }
}

fun main() {
    println(mixOptimized(BLUE, YELLOW))
    // GREEN
}

```

Если конструкция `when` используется в качестве выражения, то должна быть исчерпывающей

Если для выражения `when` не задан аргумент, то условием ветвления будет любое логическое выражение. Функция `mixOptimized` делает то же самое, что и `mix` ранее. При ее использовании лишние объекты создаваться не будут, но и читать такой код станет труднее.

2.3.6. Умное преобразование: комбинирование проверки и приведения типов

Теперь, когда вы успешно смешали несколько цветов в программе на языке Kotlin, перейдем к более сложному примеру. Напишем функцию для вычисления простых арифметических выражений, например $(1 + 2) + 4$. Выражения будут содержать только один тип операций: сумму двух чисел. Другие арифметические операции (например, вычитание, умножение и деление) можно реализовать аналогичным образом, и вы можете сделать это в качестве дополнительного упражнения. В процессе вы узнаете, как умные преобразования значительно упрощают работу с объектами Kotlin разных типов.

Начнем с кодирования выражений. Традиционно они хранятся в древовидной структуре, где каждый узел — это либо сумма (`Sum`), либо число (`Num`). Узел `Num` всегда будет листовым, а у узла `Sum` есть два дочерних узла: аргументы операции `Sum`. В листинге 2.17 показана простая структура классов, используемых для кодирования выражений: интерфейс `Expr`, и два класса `Num` и `Sum`, позволяющие его реализовать (рис. 2.5). Обратите внимание: данный интерфейс не объявляет никаких методов — он используется как *маркерный интерфейс* для того, чтобы различные виды выражений имели общий вид. Чтобы отметить, что класс реализует интерфейс, используется двоеточие (:), за ним следует имя интерфейса, как показано в листинге 2.17 (более подробно интерфейсы описаны в подразделе 4.1.1).

Листинг 2.17. Иерархия классов выражений

```

interface Expr
class Num(val value: Int) : Expr
class Sum(val left: Expr, val right: Expr) : Expr

```

Простой класс с одним свойством, `value`, реализующий интерфейс `Expr`

Аргументом операции `Sum` может быть любой интерфейс `Expr`: либо `Num`, либо другой результат `Sum`

Класс `Sum` хранит ссылки на аргументы `left` и `right` типа `Expr`. В вашем примере это означает, что они могут быть либо типа `Num`, либо типа `Sum`. Для хранения

упоминавшегося ранее выражения $(1 + 2) + 4$ необходимо создать структуру объектов `Expr`, а именно `Sum(Sum(Num(1), Num(2)), Num(4))`. На рис. 2.6 показано ее древовидное представление.

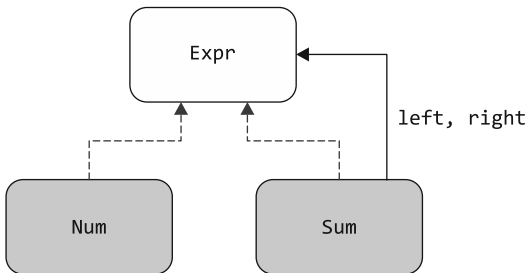


Рис. 2.5. Диаграмма классов, показывающая взаимосвязь `Expr`, `Num` и `Sum`. Классы `Num` и `Sum` реализуют маркерный интерфейс `Expr`. Вдобавок класс `Sum` связан с операндами `left` и `right`, которые опять же относятся к типу `Expr`

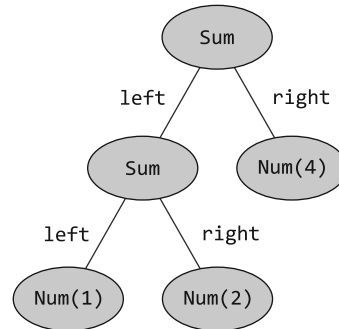


Рис. 2.6. Представление выражения `Sum(Sum(Num(1), Num(2)), Num(4))`, описывающего математическое выражение $(1 + 2) + 4$. Мы используем это представление в качестве входных данных для функции вычисления результата

Наша цель — вычислить такое выражение, состоящее из объектов классов `Sum` и `Num`, вычисляя итоговое значение. Рассмотрим дальнейшие действия.

Для интерфейса `Expr` существует две реализации, поэтому стоит попробовать два варианта вычисления значения результата для выражения.

1. Если выражение является числом, то будет возвращено соответствующее значение.
2. Если это сумма, то необходимо рекурсивно вычислить левое и правое выражения и вернуть их сумму.

Сначала рассмотрим реализацию этой функции в стиле, похожем на реализацию в коде Java. Затем выполним ее рефакторинг, чтобы показать идиоматичный Kotlin.

Сначала можно написать функцию в стиле других языков программирования, используя последовательность выражений `if` для проверки различных подтипов `Expr`. В Kotlin принадлежность переменной к конкретному типу определяется с помощью проверки `is`, поэтому реализация может выглядеть следующим образом (листинг 2.18).

Синтаксис `is` может быть вам знаком, если вы программировали на языке C#, а разработчикам Java он может быть знаком как эквивалент оператора `instanceof`.

Листинг 2.18. Вычисление выражений с помощью каскада if

```

fun eval(e: Expr): Int {
    if (e is Num) {
        val n = e as Num
        return n.value
    }
    if (e is Sum) {
        return eval(e.right) + eval(e.left)
    }
    throw IllegalArgumentException("Unknown expression")
}

fun main() {
    println(eval(Sum(Sum(Num(1), Num(2)), Num(4))))
    // 7
}

```

Проверка `is` в Kotlin дает дополнительные преимущества: если вы проверяете переменную на определенный тип, то вам не нужно приводить ее к определенному типу — ее можно использовать как уже имеющую тот тип, на который выполнялась проверка. По сути, компилятор выполняет приведение автоматически, и это называется *умным преобразованием*.

В функции `eval` после проверки того, относится ли переменная `e` к типу `Num`, компилятор интеллектуально интерпретирует ее как переменную типа `Num`. Затем можно получить доступ к свойству `value` класса `Num`, не выполняя приведение типа: `e.value`. То же самое касается свойств `right` и `left` для класса `Sum`: вы пишете только `e.right` и `e.left` в соответствующем контексте. В средах разработки IntelliJ IDEA и Android Studio эти значения умного преобразования выделяются цветом фона, поэтому легко понять, что данное значение было проверено заранее (рис. 2.7).

```

if (e is Sum) {
    return eval(e.right) + eval(e.left)
}

```

Рис. 2.7. IDE выделяет умные преобразования цветом фона, что хорошо видно в электронной версии книги

Умное преобразование работает только в том случае, если переменная не изменилась после проверки `is`. Когда вы используете этот механизм со свойством класса, как в этом примере, свойство необходимо объявить как `val` и у него не может быть пользовательского метода доступа. В противном случае было бы невозможно удостовериться, что каждое обращение к свойству возвращает одно и то же значение. Позже мы расскажем, что умные преобразования работают совместно и с другими возможностями языка Kotlin, такими как `null`-безопасность свойства (обсуждается в главе 7) и классы Kotlin с модификатором `final` по умолчанию (описаны в подразделе 4.1.2).

Явное приведение к конкретному типу выражается с помощью ключевого слова `as`:

```
val n = e as Num
```

Но такая реализация пока не считается идиоматичным Kotlin. Рассмотрим, как выполнить рефакторинг функции `eval`.

2.3.7. Рефакторинг: замена выражения `if` на `when`

В подразделе 2.1.2 говорилось, что `if` — это выражение в Kotlin. Именно поэтому в Kotlin нет тернарного оператора: выражение `if` уже может возвращать значение. Это означает, что можно переписать функцию `eval`, используя синтаксис с телом выражения, удалив оператор `return` и фигурные скобки, а в качестве тела функции использовать выражение `if` (листинг 2.19).

Листинг 2.19. Использование выражений `if`, возвращающих значения

```
fun eval(e: Expr): Int =
    if (e is Num) {
        e.value
    } else if (e is Sum) {
        eval(e.right) + eval(e.left)
    } else {
        throw IllegalArgumentException("Unknown expression")
    }

fun main() {
    println(eval(Sum(Num(1), Num(2))))
    // 3
}
```

Этот код можно сделать еще более кратким: фигурные скобки необязательны, если в ветке `if` есть только одно выражение: для ветки `if` с блоком последнее выражение возвращается в качестве результата. Сокращенная версия выражения `eval` с использованием каскада `if` может выглядеть следующим образом:

```
fun eval(e: Expr): Int =
    if (e is Num) e.value
    else if (e is Sum) eval(e.right) + eval(e.left)
    else throw IllegalArgumentException("Unknown expression")
```

Вы уже познакомились с улучшенной языковой конструкцией для выражения множественного выбора в подразделе 2.3.2. Поэтому доработаем этот код еще больше и перепишем его, используя выражение `when`.

Данное выражение не ограничивается проверкой значений на равенство, показанной ранее. Здесь используется другая форма ветвей `when`, позволяющая проверить тип значения аргумента `when`. Как и в примере `if` из листинга 2.19, при проверке типа применяется умное преобразование, поэтому вы можете получить

доступ к членам классов `Num` и `Sum`, не выполняя дополнительные приведения типов (листинг 2.20).

Листинг 2.20. Использование `when` вместо каскада `if`

```
fun eval(e: Expr): Int =
    when (e) {
        is Num -> e.value
        is Sum -> eval(e.right) + eval(e.left)
        else -> throw IllegalArgumentException("Unknown expr")
    }
```

Ветви `when` для проверки
и умного преобразования
типа аргумента

Сравните две последние Kotlin-версии функции `eval` и подумайте, как использовать `when` в качестве замены последовательностей выражений `if` и в собственном коде. Для логики ветвления, содержащей несколько операций, в качестве тела ветвления можно использовать блочное выражение. Посмотрим, как это работает.

2.3.8. Блоки как ветви конструкций `if` и `when`

В конструкциях `if` и `when` блоки могут выступать в роли ветвей. В этом случае последнее выражение в блоке является результатом. Допустим, вы хотите лучше понять, как ваша функция `eval` вычисляет результат. Один из способов сделать это — добавить операторы `println` для регистрации текущих вычислений функции. Их можно добавить в блок для каждой ветви в выражении `when` (листинг 2.21). Последнее выражение в блоке — то, что будет возвращено.

Листинг 2.21. Использование `when` с составными действиями в ветвях

```
fun evalWithLogging(e: Expr): Int =
    when (e) {
        is Num -> {
            println("num: ${e.value}")
            e.value
        }
        is Sum -> {
            val left = evalWithLogging(e.left)
            val right = evalWithLogging(e.right)
            println("sum: $left + $right")
            left + right
        }
        else -> throw IllegalArgumentException("Unknown expression")
    }
```

Последнее выражение
в блоке, и оно возвращается,
если `e` относится к типу `Num`

Выражение возвращается, если `e` относится к типу `Sum`

Теперь вы можете просмотреть журналы, выведенные функцией `evalWithLogging`, и проследить за порядком вычислений:

```
fun main() {
    println(evalWithLogging(Sum(Sum(Num(1), Num(2)), Num(4))))
    // num: 1
    // num: 2
```

}

return внутри.

самое время узнать о выполнении итераций.

ЦИКЛЫ WHILE И FOR

можно реализовать с помощью этих двух видов циклов.

цикл while

другим языкам программирования. Вкратце рассмотрим его:

`while (condition) {` ← Тело выполняется, пока условие истинно

$$/* \dots */$$

```
if (shouldExit) break
```

}

Оператор break завершает выполнение замкнутого цикла

```
do {
```

```
if (shouldSkip) continue
```

$$/* \dots */$$

Оператор continue выполняет переход к следующему шагу цикла

```
} while (condition)
```

Тело впервые выполняется безусловно. А после этого выполняется, пока условие истинно

Для вложенных циклов Kotlin допускается указать *метку*, на которую можно сослаться при использовании операторов `break` или `continue`. Метка — это идентификатор, за которым следует знак коммерческого эт (@):

```
outer@ while (outerCondition) { ← Маркировка цикла с помощью метки outer
    while (innerCondition) {
        if (shouldExitInner) break
        if (shouldSkipInner) continue
        if (shouldExit) break@outer
        if (shouldSkip) continue@outer
        // ...
    }
    // ...
}
```

Операторы `break` и `continue` без меток всегда относятся к ближайшему к ним циклу

Операторы `break` и `continue` с метками выполняют выход из указанного цикла

Обсудим различные варианты использования цикла `for` и поговорим о том, что он охватывает итерацию не только над элементами коллекции, но и над *диапазонами*.

2.4.2. Итерация над числами: диапазоны и прогрессии

Как мы только что упомянули, в Kotlin нет цикла `for` в стиле языка C, где переменная инициализируется, ее значение обновляется на каждом шаге цикла, и выход из цикла происходит, когда значение достигает определенной границы (классический `int i = 0; i < 10; i++`). Чтобы заменить наиболее распространенные случаи использования таких циклов, в Kotlin используются концепции *диапазонов*.

Диапазон — это, по сути, просто интервал между двумя значениями, обычно числами — начальным и конечным. Он записывается с помощью оператора `..` в следующем виде:

```
val oneToTen = 1..10
```

Обратите внимание, что в Kotlin эти диапазоны закрыты (инклюзивны), то есть второе значение всегда входит в диапазон.

Самое простое, что можно сделать с целочисленными диапазонами, — перебрать все значения в цикле. Если есть возможность перебирать все значения в диапазоне, то он называется *прогрессией*.

Воспользуемся диапазонами целых чисел, чтобы поиграть в игру Fizz-Buzz. Это хороший способ скоротать время в долгой поездке в машине и вспомнить забытые навыки деления. К тому же реализация этой игры стала популярным заданием на собеседованиях по программированию!

Чтобы сыграть в Fizz-Buzz, игроки по очереди считают по возрастающей, заменяя любое число, кратное 3, словом *fizz*, а любое число, кратное 5, словом *buzz*. Если число кратно и 3, и 5, необходимо сказать *fizz-buzz*.

Код в листинге 2.22 выводит правильные ответы для чисел от 1 до 100. Обратите внимание, как проверка возможных условий выполняется с помощью выражения `when` без аргумента.

Листинг 2.22. Использование выражения `when` для реализации игры Fizz-Buzz

```

fun fizzBuzz(i: Int) = when {
    i % 15 == 0 -> "FizzBuzz " ← Если i кратно 15, возвращается строка
                                FizzBuzz; % — оператор остатка
    i % 3 == 0 -> "Fizz " ← Если i кратно 3, возвращается строка Fizz
    i % 5 == 0 -> "Buzz " ← Если i кратно 5, возвращается строка Buzz
    else -> "$i " ← Иначе возвращается само число
}

fun main() {
    for (i in 1..100) { ← Итерация по диапазону
        print(fizzBuzz(i)) ← целых чисел 1..100
    }
    // 1 2 Fizz 4 Buzz Fizz 7 ...
}

```

Предположим, что после часа езды вы устанете от этих правил и захотите немного усложнить ситуацию. Начнем считать в обратном порядке от 100 и возьмем только четные числа (листинг 2.23).

Листинг 2.23. Итерация по диапазону с шагом

```

fun main() {
    for (i in 100 downTo 1 step 2) {
        print(fizzBuzz(i))
    }
    // Buzz 98 Fizz 94 92 FizzBuzz 88 ...
}

```

Теперь итерации выполняются над прогрессией с шагом, чтобы пропускать некоторые числа. Шаг может быть и отрицательным, в этом случае прогрессия идет не вперед, а назад. В данном примере `100 downTo 1` — прогрессия, которая идет назад (с шагом `-1`). Далее ключевое слово `step` изменяет абсолютное значение шага на 2, сохраняя направление (фактически устанавливая шаг `-2`).

Как мы уже говорили, синтаксис `..` всегда создает диапазон, содержащий конечную точку (значение справа от `..`). Во многих случаях удобнее выполнять итерацию по полужакрытым диапазонам, в которых нет указанной конечной точки. Чтобы создать такой диапазон, используйте синтаксис `..`. Например, цикл `for (x in 0..размер)` эквивалентен `for (x in 0..размер-1)`, но выражает идею несколько более четко.

Работа с диапазонами и прогрессиями помогла вам справиться со сложными правилами игры Fizz-Buzz. Но цикл `for` в Kotlin способен на большее. Рассмотрим другие примеры.

2.4.3. Итерации по картам

Мы уже говорили о том, что наиболее распространенным сценарием использования цикла `for (x in y)` является итерация по коллекции. Скорее всего, вы уже знакомы с его поведением: цикл выполняется для каждого элемента коллекции входных данных. В нашем случае просто выводится каждый элемент из коллекции цветов.

Внутри цикла к отдельным цветам можно обращаться с помощью переменной `color`, как показано в листинге 2.24, поскольку именно это имя используется в цикле `for`.

Листинг 2.24. Итерация по карте с помощью цикла `for`

```
fun main() {
    val collection = listOf("red", "green", "blue")
    for (color in collection) {
        print("$color ")
    }
    // red green blue
}
```

Не будем тратить время и взглянем на более интересную задачу — как выполнять итерации над картой. В качестве примера рассмотрим небольшую программу. Она выводит двоичные представления символов, предоставляя вам простую таблицу поиска, которая может помочь в расшифровке двоичного текста вручную, например, `1000100 1000101 1000011 1000001 1000110`. Эти двоичные представления будут храниться в карте (`map`) (просто для примера).

Код в листинге 2.25 создает карту, заполняет ее двоичными представлениями некоторых букв, а затем выводит ее содержимое на экран. Как видите, синтаксис `..` для создания диапазона допустим не только для чисел, но и для символов. Здесь он используется для перебора всех символов от `A` до `F` включительно.

Листинг 2.25. Инициализация и итерация по карте

```
fun main() {
    val binaryReps = mutableMapOf<Char, String>()
    for (char in 'A'..'F') {
        val binary = char.code.toString(radix = 2)
        binaryReps[char] = binary
    }

    for ((letter, binary) in binaryReps) {
        println("$letter = $binary")
    }
    // A = 1000001    D = 1000100
    // B = 1000010    E = 1000101
    // C = 1000011    F = 1000110
    // (вывод для краткости разбит на столбцы)
}
```

В Kotlin изменяемые карты сохраняют порядок итерации элементов

Итерации над символами от `A` до `F` с помощью диапазона символов

Преобразование ASCII-кода в двоичный

Сохранение значения в карте по символическому ключу

Итерация по карте, при этом ключу карты присваивается буквенное значение, а соответствующему значению — двоичное

В листинге 2.25 показано, что цикл `for` позволяет распаковать элемент коллекции, над которой выполняется итерация (в данном случае это коллекция пар «ключ — значение» в карте). Результат распаковки хранится в двух отдельных переменных: `letter` — ключ, `binary` — значение. В разделе 9.4 вы узнаете больше об этом синтаксисе деструктуризации.

Еще один интересный прием, использованный в листинге 2.25, — сокращенный синтаксис для получения и обновления значений карты по ключу. Вместо того чтобы

вызывать такие функции, как `get` и `put`, вы можете использовать `map[ключ]` для чтения значений и `map[ключ] = значение` для их установки. Это означает, что вместо версии `binaryReps.put(символ, двоичное_значение)` в стиле Java можно использовать эквивалентную, но более элегантную форму: `binaryReps[символ] = двоичное_значение`.

Можно использовать тот же синтаксис распаковки для итерации по коллекции, сохраняя при этом индекс текущего элемента. Это позволит не создавать отдельную переменную для хранения индекса и не увеличивать его вручную. В данном случае элементы коллекции выводятся с соответствующими индексами с помощью функции `withIndex`:

```
fun main() {
    val list = listOf("10", "11", "1001")
    for ((index, element) in list.withIndex()) {
        println("$index: $element")
    }
    // 0: 10
    // 1: 11
    // 2: 1001
}
```

← Итерация над коллекцией
с индексом

О функции `withIndex` мы поговорим в разделе 3.3.

Вы узнали, как ключевое слово `in` можно использовать для итерации по диапазону или коллекции. Кроме того, с помощью `in` можно проверять принадлежность значения к диапазону или коллекции. Рассмотрим эту тему подробнее.

2.4.4. Оператор `in` для проверки коллекции и принадлежности к диапазону

Оператор `in` используется, чтобы проверить, находится ли значение в диапазоне, или применяется его противоположность `!in`, чтобы проверить, не находится ли значение в диапазоне. Например, при проверке ввода пользователя часто приходится проверять, действительно ли вводимый символ является буквой или исключает цифры. Приведем пример, как `in` можно использовать для написания небольших вспомогательных функций `isLetter` и `isNotDigit`. Они проверяют, принадлежит ли символ к диапазону символов (листинг 2.26).

Листинг 2.26. Проверка принадлежности к диапазону с помощью `in`

```
fun isLetter(c: Char) = c in 'a'..'z' || c in 'A'..'Z'
fun isNotDigit(c: Char) = c !in '0'..'9'

fun main() {
    println(isLetter('q'))
    // истина
    println(isNotDigit('x'))
    // истина
}
```

Этот прием проверки того, является ли символ буквой, выглядит просто. Внутри не происходит ничего сверхъестественного: по-прежнему выполняется проверка, что код символа находится где-то между кодом первой буквы и кодом последней. Но эта логика скрыта в реализации классов диапазонов в стандартной библиотеке:

```
c in 'a'..'z'  ← Преобразование выражения в 'a' <= c && c <= 'z'
```

Операторы `in` и `!in` работают и в выражениях `when`, поэтому становятся еще более удобными, когда требуется проверить несколько различных диапазонов (листинг 2.27).

Листинг 2.27. Использование проверок `in` в качестве ветвей конструкции `when`

```
fun recognize(c: Char) = when (c) {
    in '0'..'9' -> "It's a digit!"
    in 'a'..'z', in 'A'..'Z' -> "It's a letter!"
else -> "I don't know..."
}

fun main() {
    println(recognize('8'))
    // It's a digit!
}
```

Проверка того, находится ли значение в диапазоне от 0 до 9

Можно комбинировать несколько диапазонов

Кроме того, диапазоны не ограничиваются символами. Используя класс, поддерживающий сравнение экземпляров (реализуя интерфейс `kotlin.Comparable`, о котором вы узнаете в подразделе 9.2.2), вы можете создавать диапазоны объектов этого типа. Перечислить объекты в таком диапазоне невозможно. Подумайте: можете ли вы, например, перечислить все строки, находящиеся в алфавитном порядке между *Java* и *Kotlin*? Нет, конечно. Но есть возможность проверить, принадлежит ли другой объект диапазону, с помощью оператора `in`:

```
fun main() {
    println("Kotlin" in "Java".."Scala")
    // истина
}
```

То же самое, что и `Java <= Kotlin && Kotlin <= Scala`

Обратите внимание: строки здесь сравниваются в алфавитном порядке, поскольку именно так класс `String` реализует интерфейс `Comparable`. При сортировке по алфавиту слово "Java" идет перед "Kotlin" и "Kotlin" идет перед "Scala", поэтому "Kotlin" находится в диапазоне между этими двумя строками.

Такая же проверка `in` работает и с коллекциями:

```
fun main() {
    println("Kotlin" in setOf("Java", "Scala"))
    // ложь
}
```

Это множество не содержит строку Kotlin

В подразделе 9.3.2 вы узнаете, как использовать диапазоны и прогрессии с собственными типами данных и с какими объектами вообще можно использовать

проверки `in`. Чтобы завершить обзор основных строительных блоков программ на языке Kotlin, в этой главе мы хотим рассмотреть еще одну тему — работу с исключениями.

2.5. ВЫБРАСЫВАНИЕ И ПЕРЕХВАТ ИСКЛЮЧЕНИЙ В KOTLIN

Обработка исключений в Kotlin схожа с Java и многими другими языками. Функция может завершиться обычным образом или выбросить исключение при возникновении ошибки. Вызывающая функция может перехватить это исключение и обработать его. В противном случае исключение будет выброшено дальше по стеку.

Исключение выбрасывается при наличии ключевого слова `throw` — в данном случае для того, чтобы указать, что вызывающая функция предоставила недопустимое значение процента:

```
if (percentage !in 0..100) {
    throw IllegalArgumentException(
        "A percentage value must be between 0 and 100: $percentage"
    )
}
```

Сейчас самое время напомнить, что в Kotlin нет ключевого слова `new`. Создание экземпляра исключения ничем не отличается.

В Kotlin конструкция `throw` является *выражением* и может использоваться как часть других выражений:

```
val percentage =
    if (number in 0..100)
        number
    else
        throw IllegalArgumentException( ← throw — это выражение
            "A percentage value must be between 0 and 100: $number"
        )
```

В этом примере, если условие выполняется, то программа ведет себя правильно, и переменная `percentage` инициализируется значением `number`. В противном случае будет выброшено исключение, и переменная не будет инициализирована. Технические подробности использования конструкции `throw` в составе других выражений мы рассмотрим в подразделе 8.1.7, в котором, помимо прочего, вы узнаете о типе возвращаемого им значения.

2.5.1. Обработка исключений и восстановление после ошибок: `try`, `catch` и `finally`

При работе над восстановлением системы после ошибок для обработки исключений используется блок `try` с операторами `catch` и `finally`. Пример такой конструкции показан в листинге 2.28. Здесь код считывает строку текста из файла `BufferedReader`,

пытается выполнить ее синтаксический анализ в качестве числа и возвращает число, или значение `null`, если строка не относится к допустимым числам.

Листинг 2.28. Использование блока `try` таким же образом, как и в Java

```
import java.io.BufferedReader
import java.io.StringReader

fun readNumber(reader: BufferedReader): Int? {
    try {
        val line = reader.readLine()
        return Integer.parseInt(line)
    } catch (e: NumberFormatException) {
        return null
    } finally {
        reader.close()
    }
}

fun main() {
    val reader = BufferedReader(StringReader("239"))
    println(readNumber(reader))
    // 239
}
```

В Kotlin нельзя явно указать исключения, которые могут быть выброшены из функции

Тип исключения указан справа

Оператор `finally` используется точно так же, как в Java

Важное отличие от Java — в Kotlin нет оператора `throws`. После объявления этой функции на Java необходимо явно написать `throws IOException` (листинг 2.29).

Листинг 2.29. В Java проверяемые исключения являются частью сигнатуры метода

```
Integer readNumber(BufferedReader reader) throws IOException
```

Это необходимо, поскольку в Java методы `readLine` и `close` могут выбросить *проверяемое исключение* `IOException`. В Java такой тип исключений требует прямой обработки. Необходимо объявить все проверяемые исключения, которые может выбросить ваша функция. А если вы вызываете другую функцию, то должны обработать ее проверяемые исключения или объявить, что ваша функция тоже может их выбросить.

Как и многие другие современные JVM-языки, Kotlin не делает различий между проверяемыми и непроверяемыми исключениями. Нет необходимости указывать исключения, выбрасываемые функцией, и можно обрабатывать (или нет) любые из них. Это проектное решение основано на практике использования проверяемых исключений в Java. Опыт показывает, что правила Java часто требуют большого количества бессмысленного кода, с помощью которого выполняется проброс или игнорирование исключений, и правила не всегда защищают от вероятных ошибок.

Например, в листинге 2.28 `NumberFormatException` не относится к проверяемым исключениям. Поэтому компилятор Java не обязывает перехватывать это исключение, и оно легко возникает во время выполнения программы. Это досадно, поскольку недействительные входные данные — обычная ситуация, и при ее возникновении действовать нужно аккуратно. В то же время метод `BufferedReader.close` может

выбросить проверяемое исключение `IOException`, и его потребуется обрабатывать. Большинство программ не могут предпринять никаких действенных мер при неудачной остановке потока, поэтому код, необходимый для перехвата исключения из метода `close`, является шаблонным.

В результате этого конструктивного решения вы сами решаете, какие исключения обрабатывать, а какие — нет. При желании можно реализовать функцию `readNumber` без каких-либо конструкций `try-catch`, как показано в листинге 2.30.

Листинг 2.30. В Kotlin компилятор не обязывает обрабатывать исключения

```
fun readNumber(reader: BufferedReader): Int {
    val line = reader.readLine()
    reader.close()
    return Integer.parseInt(line)
}
```

А что насчет конструкции `try-with-resources` из языка Java? В Kotlin для нее нет специального синтаксиса — она реализована как библиотечная функция. В главе 10 вы увидите, как ее использовать.

2.5.2. Использование блока `try` в качестве выражения

Пока мы использовали конструкцию `try` только в качестве инструкции. Но поскольку `try` представляет собой выражение (как и `if` или `when`), то можно немного изменить пример программы, чтобы получить все преимущества такого статуса и присвоить переменной значение выражения `try`. Для краткости уберем секцию `finally` (только потому, что вы уже видели, как она работает, — не используйте это как оправдание, чтобы не закрывать потоки!) и добавим код для вывода считанного из файла числа (листинг 2.31).

Листинг 2.31. Использование блока `try` в качестве выражения

```
fun readNumber(reader: BufferedReader) {
    val number = try {
        Integer.parseInt(reader.readLine())
    } catch (e: NumberFormatException) {
        return
    }

    println(number)
}

fun main() {
    val reader = BufferedReader(StringReader("not a number"))
    readNumber(reader)
}
```

Становится значением выражения `try`

Ничего не выводится на экран

Стоит отметить: в отличие от выражения `if`, в данном случае необходимо заключать тело оператора в фигурные скобки. Как и в других инструкциях, если тело содержит несколько выражений, то общим значением выражения `try` становится результат последнего из них.

В этом примере оператор `return` помещен в блок `catch`, поэтому выполнение функции не продолжается после блока `catch`. Если требуется продолжить выполнение, то оператор `catch` должен содержать значение, которое будет результатом последнего выражения в нем. Это работает следующим образом (листинг 2.32).

Листинг 2.32. Возврат значения в блок `catch`

```
fun readNumber(reader: BufferedReader) {
    val number = try {
        Integer.parseInt(reader.readLine())
    } catch (e: NumberFormatException) {
        null
    }

    println(number)
}

fun main() {
    val reader = BufferedReader(StringReader("not a number"))
    readNumber(reader)
    // null
}
```

Этот выражение используется, когда не возникает исключений

Значение null используется при возникновении исключения

Исключение выброшено, поэтому функция выводит null

Если блок кода `try` выполняется в нормальном режиме, то последнее выражение в блоке становится результатом. Если перехватывается исключение, то результатом будет последнее выражение в соответствующем блоке `catch`. В листинге 2.32 значение результата будет равно `null`, если возникнет исключение `NumberFormatException`.

Блоки `try` в качестве выражений позволяют сократить код и избавиться от дополнительных промежуточных переменных, а также помогают присваивать резервные значения или выполнять непосредственный возврат из замкнутой функции. Теперь вы можете писать программы на Kotlin, комбинируя описанные основные строительные блоки. Читая эту книгу, вы продолжите учиться тому, как изменить привычный образ мышления и использовать все возможности Kotlin.

РЕЗЮМЕ

- Ключевое слово `fun` используется для объявления функции. Ключевые слова `val` и `var` объявляют соответственно переменные, доступные только для чтения, и переменные с возможностью изменения.
- Ссылка `val` доступна только для чтения, но объект, на который она указывает, может быть изменяемым.
- Строковые шаблоны помогут вам избежать объемной конкатенации строк. Добавьте префикс `$` к имени переменной или окружите выражение символами `{}`, чтобы его значение было вставлено в строку.
- В Kotlin классы могут быть выражены в сжатой форме.
- Привычная конструкция `if` теперь является выражением с возвращаемым значением.

- Выражение `when` аналогично оператору `switch` в Java, но дает больше возможностей.
- Нет необходимости приводить тип переменной явно после проверки того, что она относится к определенному типу, — компилятор выполняет приведение типа автоматически с помощью умного преобразования.
- Циклы `for`, `while` и `do-while` похожи на свои аналоги в Java, но цикл `for` стал более удобным, особенно когда вам нужно выполнить итерацию по карте или коллекции с индексом.
- Лаконичный синтаксис `1..5` (и `1..<5`) создает диапазон. Диапазоны и прогрессии позволяют Kotlin использовать единый синтаксис и набор абстракций в циклах `for`, а также работать с операторами `in` и `!in`, проверяющими принадлежность значения к диапазону.
- Работа с исключениями в Kotlin очень похожа на Java, только Kotlin не требует объявлять исключения, которые могут быть выброшены функцией.

3

Определение и вызов функций

В этой главе

- ✓ Функции для работы с коллекциями, строками и регулярными выражениями.
- ✓ Использование именованных аргументов, значений параметров по умолчанию и инфиксного синтаксиса вызова.
- ✓ Адаптация библиотек Java к Kotlin с помощью функций-расширений и свойств.
- ✓ Структурирование кода с помощью функций и свойств верхнего и локального уровней.

Теперь вы уже можете достаточно уверенно использовать Kotlin на базовом уровне с той же легкостью, с которой писали код на других объектно-ориентированных языках, например на Java. Мы показали, как концепции Java переходят в Kotlin и как последний часто делает их более краткими и удобочитаемыми.

В этой главе мы расскажем об усовершенствовании одного из ключевых элементов каждой программы — об объявлении и вызове функций. Вдобавок мы рассмотрим возможности адаптации библиотек Java к стилю Kotlin с помощью функций-расширений, что позволит вам получить все преимущества Kotlin в проектах на разных языках. Чтобы повествование стало более полезным и менее абстрактным, мы сосредоточимся на работе с коллекциями, строками и регулярными выражениями. В качестве введения рассмотрим процесс создания коллекций в Kotlin.

3.1. СОЗДАНИЕ КОЛЛЕКЦИЙ В KOTLIN

Прежде чем использовать коллекции разными способами, необходимо научиться их создавать. В главе 2 приводился способ создания нового множества (коллекции, для которой порядок элементов не имеет значения: два набора равны, если содержат одинаковые элементы) — функция `setOf`. Тогда мы создавали множество цветов, а сейчас немного упростим задачу и будем работать с числами:

```
val set = setOf(1, 7, 53)
```

Аналогичным образом можно создать список, или *карту* (иногда карты называют *словарями*):

```
val list = listOf(1, 7, 53)
val map = mapOf(1 to "one", 7 to "seven", 53 to "fifty-three")
```

Обратите внимание, что `to` — не специальная конструкция, а обычная функция. Мы вернемся к этому вопросу позже, в подразделе 3.4.3.

Можете ли вы определить, какие классы объектов здесь создаются? Запустите этот пример кода — и увидите все сами:

```
fun main() {
    val set = setOf(1, 7, 53)
    val list = listOf(1, 7, 53)
    val map = mapOf(1 to "one", 7 to "seven", 53 to "fifty-three")

    println(set.javaClass)
    // класс java.util.LinkedHashSet

    println(list.javaClass)
    // класс java.util.Arrays$ArrayList

    println(map.javaClass)
    // класс java.util.LinkedHashMap
}
```

← Функция `javaClass` в Kotlin эквивалентна методу `getClass()` в Java

В Kotlin используются стандартные классы коллекций Java. Это хорошая новость для Java-разработчиков: в Kotlin нет новой реализации классов коллекций. Следовательно, все ваши знания о коллекциях Java будут актуальными. Однако стоит отметить, что интерфейсы коллекций в Kotlin по умолчанию доступны только для чтения. Данная тема и изменяемые аналоги этих интерфейсов более детально описаны в главе 8.

Использование стандартных коллекций Java значительно упрощает взаимодействие с кодом Java. Нет необходимости преобразовывать коллекции при вызове функций Java из Kotlin или наоборот.

Базовые коллекции Kotlin — точно такие же классы, как и коллекции Java. В Kotlin с ними можно выполнять гораздо больше операций. Например, можно получить

последний элемент из списка или перемешанную версию списка либо же просуммировать элементы коллекции (если это коллекция чисел):

```
fun main() {
    val strings = listOf("first", "second", "fourteenth")

    strings.last()
    // fourteenth

    println(strings.shuffled())
    // [fourteenth, second, first]

    val numbers = setOf(1, 14, 2)
    println(numbers.sum())
    // 17
}
```

В этой главе дается подробное описание этих механизмов обработки и источников всех новых методов в классах Java. В дальнейшем, при обсуждении лямбда-выражений, приводится большое количество операций с коллекциями, но при этом используются те же стандартные классы коллекций Java. А в главе 8 вы узнаете, как классы коллекций Java представлены в системе типов Kotlin. Прежде чем мы перейдем к обсуждению механизмов работы функций `last` и `sum` с коллекциями Java, познакомим вас с некоторыми новыми понятиями для объявления функции.

3.2. УПРОЩЕНИЕ ВЫЗОВА ФУНКЦИЙ

Мы рассмотрели способы создания коллекции, а теперь выполним простую операцию — вывод ее содержимого. Пусть вас не пугает простота кода — по мере его изучения вы познакомитесь с множеством важных понятий.

У коллекций Java по умолчанию есть реализация метода `toString`, но возможности форматировать вывод результата нет, а его вид не всегда соответствует вашим потребностям:

```
fun main() {
    val list = listOf(1, 2, 3)
    println(list)    ← Вызов функции toString()
    // [1, 2, 3]
}
```

Представьте, что в выводе результата требуются разделители в виде точек с запятой, и скобки должны быть круглыми, а не квадратными, как задано по умолчанию: `(1; 2; 3)`. В Java-проектах для решения этой задачи применяются сторонние библиотеки, такие как Guava и Apache Commons, или вносятся изменения в логику внутри проекта. В Kotlin для этого есть функция из стандартной библиотеки.

В этом разделе вы самостоятельно ее реализуете. Начать стоит с прямолинейного создания кода без использования возможностей Kotlin, которые помогают

упростить объявление функций, а после этого мы перепишем ее в более идиоматичном стиле.

В листинге 3.1 приведена функция `joinToString`. Она добавляет элементы коллекции в класс `StringBuilder` (изменяемую последовательность символов) с разделителем между ними, префиксом в начале и постфиксом в конце. Это функция обобщения — она работает с коллекциями, содержащими элементы любого типа. Синтаксис обобщений похож на синтаксис Java. (Функции этого типа более подробно рассматриваются в главе 11.)

Листинг 3.1. Первоначальная реализация функции `joinToString()`

```
fun <T> joinToString(
    collection: Collection<T>,
    separator: String,
    prefix: String,
    postfix: String
): String {

    val result = StringBuilder(prefix)

    for ((index, element) in collection.withIndex()) {
        if (index > 0) result.append(separator)
        result.append(element)
    }

    result.append(postfix)
    return result.toString()
}
```

← Перед первым элементом
разделитель не добавляется

Убедимся, что функция работает ожидаемым образом:

```
fun main() {
    val list = listOf(1, 2, 3)
    println(joinToString(list, "; ", "(", ")"))
    // (1; 2; 3)
}
```

Этот вариант реализации рабочий, и большинство разработчиков оставило бы его в таком виде. Но мы уделим внимание объявлению — как его можно изменить, чтобы сделать вызовы этой функции менее многословными? Возможно, не потребуется передавать четыре аргумента при каждом вызове функции. Подумаем, что можно сделать.

3.2.1. Именованные аргументы

Первая проблема касается удобочитаемости вызовов функций. Например, такой вызов `joinToString`:

```
joinToString(collection, " ", " ", ".")
```

Вам понятно, каким параметрам соответствуют все эти значения типа `String`? Элементы будут разделяться пробелом или точкой? На эти вопросы трудно ответить, не зная сигнатуры функции. Возможно, вы помните эту информацию или ваша IDE подскажет, но из вызывающего кода ответы неочевидны.

Эта проблема особенно характерна для логических флагов. Для ее решения в одних стилях программирования Java рекомендуется создавать перечисляемые типы вместо использования логических значений. В других случаях есть требование явно указывать имена параметров в комментариях, как в следующем примере:

```
/* Java */
joinToString(collection, /* separator */ " ", /* prefix */ " ",
    /* postfix */ ".");
```

В Kotlin эту задачу можно решить намного лучше:

```
joinToString(collection, separator = " ", prefix = " ", postfix = ".")
```

При вызове написанной на Kotlin функции можно указать имена некоторых (или всех) передаваемых ей аргументов. Если вы укажете имена их всех, то сможете даже изменить их порядок:

```
joinToString(
    postfix = ".",
    separator = " ",
    collection = collection,
    prefix = " "
)
```

Именованные аргументы особенно хорошо работают со значениями параметров по умолчанию.

СОВЕТ

Среды разработки IntelliJ IDEA и Android Studio могут поддерживать написанные в явном виде имена аргументов в актуальном состоянии при переименовании параметра вызываемой функции. Для этого вам не нужно редактировать имена вручную — используйте действие `Rename` (Переименовать) или `Change Signature` (Изменить сигнатуру). Оба действия можно вызвать, щелкнув правой кнопкой мыши на имени функции и выбрав в контекстном меню пункт `Refactor` (Рефакторинг).

3.2.2. Значения параметров по умолчанию

Еще одна распространенная проблема Java — избыток перегруженных методов в некоторых классах. Только взгляните на `java.lang.Thread` (<http://mng.bz/4KZC>) и на его восемь конструкторов! Иногда перегрузки предоставляются ради обратной совместимости, удобства пользователей API или по другим причинам, но конечный результат закономерен — дублирование. Имена параметров и типы повторяются

снова и снова, и если подходить к работе достаточно скрупулезно, то вам придется повторить большую часть документации в каждой перегрузке. В то же время при вызове сокращенного варианта перегрузки не всегда понятно, какие значения используются для отсутствующих параметров.

В Kotlin можно избежать создания перегрузок, указав в объявлении функции значения параметров по умолчанию. Воспользуемся этой возможностью, чтобы улучшить функцию `joinToString` (листинг 3.2). В большинстве случаев строковые значения можно разделять запятыми без префикса или постфикса. Поэтому сделаем запятые значениями по умолчанию.

Листинг 3.2. Объявление функции `joinToString()` со значениями по умолчанию для параметров

```
fun <T> joinToString(
    collection: Collection<T>,
    separator: String = ", ",
    prefix: String = "",
    postfix: String = ""
): String { /* ... */ }
```

| Параметры
со значениями
по умолчанию

Теперь можно либо вызвать функцию со всеми аргументами, либо опустить некоторые из них:

```
fun main() {
    joinToString(list, ", ", "", "")
    // 1, 2, 3
    joinToString(list)
    // 1, 2, 3
    joinToString(list, "; ")
    // 1; 2; 3
}
```

При использовании обычного синтаксиса вызова необходимо указывать аргументы в том же порядке, что и в объявлении функции, и опускать можно только концевые аргументы. Что касается именованных аргументов, то можно и опускать их из середины списка параметров, и указывать только нужные из них в любом порядке:

```
fun main() {
    joinToString(list, postfix = ";", prefix = "# ")
    // # 1, 2, 3;
}
```

Обратите внимание, что значения параметров по умолчанию закодированы в вызываемой функции, а не в точке вызова. В ситуации, когда значение по умолчанию изменяется и выполняется повторная компиляция класса с функцией, пользователи, не указавшие значение параметра при вызове, будут использовать новое значение по умолчанию.

ЗНАЧЕНИЯ ПО УМОЛЧАНИЮ В JAVA

В Java понятие значений параметров по умолчанию отсутствует. Поэтому при вызове функции Kotlin со значениями по умолчанию из Java необходимо указывать все значения параметров. Если функцию из Java приходится вызывать часто и вы хотите облегчить ее использование, то можно применить к ней аннотацию `@JvmOverloads`. Она дает указание компилятору генерировать перегруженные методы Java, опуская каждый из параметров по очереди, начиная с последнего. Например, можно задать аннотацию `@JvmOverloads` для функции `joinToString` (листинг 3.3).

Листинг 3.3. Объявление функции `joinToString()` со значениями по умолчанию для параметров

```
@JvmOverloads
fun <T> joinToString(
    collection: Collection<T>,
    separator: String = ", ",
    prefix: String = "",
    postfix: String = ""
): String { /* ... */ }
```

Это означает создание следующих перегрузок:

```
/* Java */
String joinToString(Collection<T> collection, String separator,
    String prefix, String postfix);

String joinToString(Collection<T> collection, String separator,
    String prefix);

String joinToString(Collection<T> collection, String separator);

String joinToString(Collection<T> collection);
```

Каждая перегрузка использует значения по умолчанию для параметров, опущенных в сигнатуре.

Все это время вы работали над своей служебной функцией, не обращая особого внимания на окружающий контекст. Конечно, это должен быть метод какого-то класса, не показанного в примерах, верно? На самом деле в Kotlin нет необходимости создавать такие классы.

3.2.3. Избавление от статических служебных классов: функции верхнего уровня и свойства

Все мы знаем, что Java, будучи объектно-ориентированным языком, требует, чтобы весь код был написан в виде методов классов. Как правило, это рабочий вариант, но в реальности почти в каждом большом проекте есть большой объем кода, не принадлежащего ни одному классу. В одних случаях операция работает с одинаково значимыми объектами двух разных классов. В других есть один

основной объект, но нет смысла расширять его API, добавляя операцию в качестве метода экземпляра.

В результате в коде появляются классы, которые не содержат ни состояния, ни методов экземпляра. Они выступают лишь в качестве контейнеров для нескольких статических методов. Отличным примером может служить класс `Collections` в наборе инструментов JDK. Вы можете обнаружить и другие примеры в собственном коде — ищите классы, в названии которых есть слово `Util`.

В Kotlin не нужно создавать все эти бессмысленные классы. Можно просто поместить функции непосредственно на верхний уровень исходного файла, за пределы какого-либо класса. Такие функции остаются членами пакета, объявленного в верхней части файла, и вам по-прежнему нужно импортировать их, если вы хотите вызывать их из других пакетов. Но в то же время ненужного дополнительного уровня вложенности больше не существует.

Поместим функцию `joinToString` непосредственно в пакет `strings`. Создайте файл под названием `join.kt` (выбор произволен, можно использовать любое удобное имя файла) со следующим содержимым (листинг 3.4).

Листинг 3.4. Объявление `joinToString()` в качестве функции верхнего уровня

```
package strings
```

```
fun joinToString( /* ... */ ): String { /* ... */ }
```

Как это работает? При компиляции файла будет создано несколько классов, поскольку JVM может выполнять код только в классах. Если вы работаете только с Kotlin, это все, что вам нужно знать. Но если потребуется вызвать такую функцию из Java, то вы должны понимать процесс ее компиляции. Чтобы внести ясность, посмотрим на код Java, который компилируется в тот же класс:

```
/* Java */
package strings;

public class JoinKt {
    public static String joinToString(/* ... */) { /* ... */ }
}
```

Соответствует join.kt — имени
файла из листинга 3.4

Компилятор Kotlin сгенерировал класс, и его имя соответствует имени файла, содержащего функцию, — оно начинается с заглавной буквы, чтобы соответствовать схеме именования Java, и имеет суффикс `Kt`. Все функции верхнего уровня в файле компилируются в статические методы данного класса. Поэтому вызвать данную функцию из Java так же просто, как и любой другой статический метод:

```
/* Java */
import strings.JoinKt;

/* ... */

JoinKt.joinToString(list, ", ", "", "");
```

ИЗМЕНЕНИЕ ИМЕНИ КЛАССА ФАЙЛА

По умолчанию имя класса, сгенерированное компилятором, соответствует имени файла с суффиксом `kt`. Можно изменить имя сгенерированного класса, содержащего функции верхнего уровня Kotlin. Для этого необходимо добавить аннотацию `@file:JvmName("...")` ко всему файлу. Поместите ее в начало файла, перед именем пакета:

```
@file:JvmName("StringFunctions")
```

← Аннотация для указания имени класса

```
package strings
```

← Инструкция `package` располагается после аннотации файла

```
fun joinToString(/* ... */): String { /* ... */ }
```

Теперь функцию можно вызвать следующим образом:

```
/* Java */
import strings.StringFunctions;
StringFunctions.joinToString(list, " ", " ", "", "");
```

Синтаксис аннотаций подробно рассматривается в главе 12.

Свойства верхнего уровня

Как и функции, свойства можно размещать на верхнем уровне файла. Хранить отдельные фрагменты данных вне класса полезно, но этот способ используется нечасто. Например, можно использовать свойство `var` для подсчета количества выполнений некоторой операции:

```
var opCount = 0
```

← Объявление свойства верхнего уровня

```
fun performOperation() {
    opCount++
    // ...
}
```

← Изменение значения свойства

```
fun reportOperationCount() {
    println("Operation performed $opCount times")
}
```

← Считывание значения свойства

Значение такого свойства будет храниться в статическом поле.

Кроме того, свойства верхнего уровня позволяют определять константы в коде:

```
val UNIX_LINE_SEPARATOR = "\n"
```

По умолчанию свойства верхнего уровня предоставляются коду Java в виде методов-доступа (геттер для свойства `val` и пара «геттер — сеттер» для свойства `var`). Если требуется представить константу в коде Java как поле `public static final`, чтобы сделать ее использование более естественным, то можно пометить ее

модификатором `const` (это разрешено для свойств примитивных типов, а также для данных типа `String`):

```
const val UNIX_LINE_SEPARATOR = "\n"
```

Эта строка представляет собой эквивалент следующего кода Java:

```
/* Java */
public static final String UNIX_LINE_SEPARATOR = "\n";
```

Первоначальная функция `joinToString` получила достаточно улучшений. Теперь посмотрим, как сделать ее еще более удобной.

ПРИМЕЧАНИЕ

Стандартная библиотека Kotlin тоже содержит несколько полезных функций и свойств верхнего уровня. Примером может служить пакет `kotlin.math`. В нем есть полезные функции для типичных математических и тригонометрических операций, например функция `max` для вычисления максимального значения двух чисел. В пакет входит и ряд математических констант, таких как число Эйлера или π :

```
fun main() {
    println(max(PI, E))
    // 3.141592653589793
}
```

3.3. ДОБАВЛЕНИЕ МЕТОДОВ К ЧУЖИМ КЛАССАМ: ФУНКЦИИ-РАСШИРЕНИЯ И СВОЙСТВА

Одна из главных тем Kotlin — плавная интеграция с существующим кодом. Даже проекты исключительно на Kotlin создаются на основе библиотек Java, таких как JDK, фреймворк Android и другие сторонние фреймворки. А при интеграции Kotlin в Java-проект предстоит работать с существующим кодом, который не был или не будет преобразован в Kotlin. И было бы очень полезно иметь возможность использовать все удобства Kotlin при работе с этими API, не переписывая их. Именно для этого и нужны *функции-расширения*.

Концептуально они достаточно простые — вызываются как член класса, но определены за его пределами. Для демонстрации этого механизма добавим метод вычисления последнего символа строки (игнорируя пока крайний случай работы с пустыми строками):

```
package strings

fun String.lastChar(): Char = this.get(this.length - 1)
```

Требуется только поместить имя расширяемого класса или интерфейса перед именем добавляемой функции. Имя этого класса называется *типом приемника*,

а значение, для которого вызывается функция-расширение, — *объектом-приемником*. Схема показана на рис. 3.1.

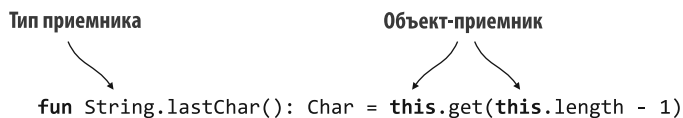


Рис. 3.1. В объявлении функции-расширения тип приемника — это тип, для которого определено расширение. Он используется для указания типа, расширяемого новой функцией. Объект-приемник — экземпляр данного типа. Он используется для доступа к свойствам и методам расширяемого типа

Функцию можно выбрать с помощью того же синтаксиса, что и для обычных членов класса:

```

fun main() {
    println("Kotlin".lastChar())
    // n
}
  
```

В этом примере `String` — тип приемника, а `"Kotlin"` — объект-приемник.

Можно сказать, что в класс `String` был добавлен новый метод. Даже если класс `String` не входит в вашу программу или у вас нет его исходного кода, то данный класс все равно можно расширить необходимыми методами. Неважно, на каком языке JVM написан `String`, и даже неважно, помечен ли он как `final`, что предотвращает создание подклассов. Пока он компилируется в класс Java, в него можно добавлять расширения.

В теле функции-расширения, как и в других методах, используется ключевое слово `this`. Как и в обычном методе, его можно опустить:

```
package strings
```

```

fun String.lastChar(): Char = get(length - 1)
  
```

← Доступ к членам объекта-приемника
можно получить и без `this`

В функции-расширении можно напрямую обращаться к методам и свойствам расширяемого класса, как к методам, определенным в самом классе. Но стоит отметить, что функции-расширения не позволяют нарушать инкапсуляцию. В отличие от методов, определенных в классе, функции-расширения не имеют доступа к приватным или защищенным членам класса.

В дальнейшем мы будем использовать термин «метод» как для членов класса, так и для функций-расширений. Например, можно сказать, что в теле функции-расширения без проблем вызывается любой метод приемника, то есть можно вызывать как члены, так и функции-расширения. С точки зрения вызывающей стороны функции-расширения неотличимы от членов класса, и часто не имеет значения, чем из них является конкретный метод.

3.3.1. Импорт и функции-расширения

Функция-расширение после объявления не становится автоматически доступной для всего проекта. Ее необходимо импортировать, как и любой другой класс или функцию. Это действие поможет избежать случайных конфликтов имен. В Kotlin можно импортировать отдельные функции с помощью синтаксиса для импорта классов:

```
import strings.lastChar

val c = "Kotlin".lastChar()
```

И конечно же, в данном случае полноценно работает импорт с постфиксом `*`:

```
import strings.*

val c = "Kotlin".lastChar()
```

С помощью ключевого слова `as` можно изменить имя импортируемого класса или функции:

```
import strings.lastChar as last

val c = "Kotlin".last()
```

Изменять имена при импорте очень полезно, когда у вас есть несколько функций с одинаковыми именами в разных пакетах и их нужно использовать в одном файле. Для обычных классов или функций в этой ситуации есть другое решение: использовать полное имя для ссылки на класс или функцию (можно ли вообще импортировать класс или функцию, зависит от модификатора видимости, о котором мы поговорим в подразделе 4.1.3). А для функций-расширений по правилам синтаксиса используется короткое имя, поэтому ключевое слово `as` в инструкции импорта — единственный способ разрешить конфликт.

3.3.2. Вызов функций-расширений из Java

По своей сути функция-расширение — статический метод, принимающий в качестве первого аргумента объект-приемник. Чтобы его вызвать, не нужно создавать объекты-адаптеры или делать что-то дополнительно во время выполнения.

Поэтому использовать функции-расширения из Java довольно просто: вызывается статический метод и ему передается экземпляр объекта-приемника. Как и в случае с другими функциями верхнего уровня, имя Java-класса, содержащего метод, определяется по имени файла, в котором объявлена функция. Допустим, он был объявлен в файле `StringUtil.kt`:

```
/* Java */
char c = StringUtilKt.lastChar("Java");
```

Эта функция-расширение объявлена как функция верхнего уровня, поэтому компилируется в статический метод. Метод `lastChar` можно статически импортировать из Java, упростив его использование до `lastChar("Java")`. Такой код не так удобно читать, как версию для Kotlin, но он идиоматичен с точки зрения Java.

3.3.3. Служебные функции в качестве расширений

Настало время написать финальную версию функции `joinToString` (листинг 3.5). Ее вид будет аналогичен тому, что можно найти в стандартной библиотеке Kotlin.

Листинг 3.5. Объявление функции `joinToString()` в качестве расширения

```
fun <T> Collection<T>.joinToString(
    separator: String = ", ",
    prefix: String = "",
    postfix: String = ""
): String {
    val result = StringBuilder(prefix)

    for ((index, element) in this.withIndex()) {
        if (index > 0) result.append(separator)
        result.append(element)
    }

    result.append(postfix)
    return result.toString()
}

fun main() {
    val list = listOf(1, 2, 3)
    println(
        list.joinToString(
            separator = "; ",
            prefix = "(",
            postfix = ")"
        )
    )
    // (1; 2; 3)
}
```

Объявление функции-расширения для типа `Collection<T>`

Присвоение параметрам значений по умолчанию

Ключевое слово `this` ссылается на объект-приемник — коллекцию `T`

Функция становится расширением для коллекции элементов (то есть работает со списками, множествами и т. п.), и ей предоставляются значения по умолчанию для всех аргументов. Теперь можно вызывать `joinToString` как член класса:

```
fun main() {
    val list = listOf(1, 2, 3)
    println(list.joinToString(" "))
    // 1 2 3
}
```

Функции-расширения — «синтаксический сахар» для статических вызовов методов, поэтому в качестве типа приемника может выступать не только класс, но и более конкретный тип. Допустим, вам требуется написать функцию `join`, которую можно вызвать только для коллекций строк:

```
fun Collection<String>.join(
    separator: String = ", ",
    prefix: String = "",
    postfix: String = ""
) = joinToString(separator, prefix, postfix)
```

```
fun main() {
    println(listOf("one", "two", "eight").join(" "))
    // one two eight
}
```

Вызов этой функции со списком объектов другого типа не работает:

```
fun main() {
    listOf(1, 2, 8).join()
    // Error: None of the following candidates is applicable because of
    // receiver type mismatch:
    // public fun Collection<String>.join(...): String
    // defined in root package
}
```

Статическая сущность расширений говорит еще и о том, что функции-расширения нельзя переопределить в подклассах. Рассмотрим пример.

3.3.4. Недопустимость переопределения для функций-расширений

Переопределение методов в Kotlin работает для функций-членов в обычном режиме, а функции-расширения переопределять нельзя. Допустим, даны два класса: `View` и `Button`. `Button` — подкласс `View`, и в нем переопределяется функция `click` из суперкласса. Чтобы это реализовать, необходимо пометить `View` и `click` модификатором `open`, позволяющим выполнять переопределение (листинг 3.6). А затем нужно использовать модификатор `override`, чтобы обеспечить реализацию в подклассе (этот синтаксис более подробно описан в подразделе 4.1.1, а информация о создании экземпляров подклассов приведена в подразделе 4.2.1).

Листинг 3.6. Переопределение функции-члена

```
open class View {
    open fun click() = println("View clicked")
}
class Button: View() {
    override fun click() = println("Button clicked")
}
```

Класс `View` помечается как `open`, чтобы из него можно было создавать подклассы

Метод `click` помечается как `open`, чтобы можно было выполнять его переопределение

Класс `Button` расширяет класс `View`

Метод `click` переопределяется в подклассе `Button`

Если переменная объявляется с типом `View`, то в ней можно хранить значение типа `Button`, поскольку `Button` является подтипом `View`. При вызове обычного метода, например `click`, для этой переменной будет использована его переопределенная реализация из класса `Button`:

```
fun main() {
    val view: View = Button()
    view.click()
    // Button clicked
}
```

Определение метода для вызова на основе фактического значения переменной `view`

Но для расширений такой принцип не работает. Функции-расширения не являются частью класса — они объявляются вне его (рис. 3.2).

Даже если вы можете определить функции-расширения с одинаковыми именами и типами параметров для базового класса и его подкласса, вызываемая функция зависит от объявленного статического типа переменной. А этот тип определяется во время компиляции. И функция не зависит от типа значения, хранящегося в этой переменной во время выполнения.

В листинге 3.7 показаны две функции-расширения `showOff`, объявленные в классах `View` и `Button`. При вызове `showOff` с переменной типа `View`, вызывается соответствующее расширение, даже если фактический тип значения `Button`.

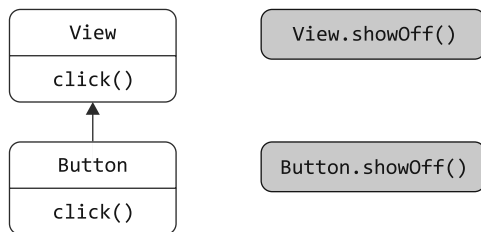


Рис. 3.2. Функции-расширения `View.showOff()` и `Button.showOff()` определены за пределами классов `View` и `Button`

Листинг 3.7. Отсутствие переопределения для функций-расширений

```
fun View.showOff() = println("I'm a view!")
fun Button.showOff() = println("I'm a button!")
```

```
fun main() {
    val view: View = Button()
    view.showOff() // I'm a view!
}
```

← Функция-расширение разрешается статически

Напоминаем, что в Java функция-расширение компилируется в статическую функцию с приемником в качестве первого аргумента. И в Java функция будет выбрана таким же образом:

```
/* Java */
class Demo {
    public static void main(String[] args) {
        View view = new Button();
        ExtensionsKt.showOff(view); // I'm a view!
    }
}
```

← Функции `showOff` объявлены в файле `extensions.kt`

Как видите, переопределение не распространяется на функции-расширения — Kotlin разрешает их статически.

ПРИМЕЧАНИЕ

Если в классе есть функция-член с той же сигнатурой, что и функция-расширение, то функция-член всегда получит приоритет. Об этом следует помнить при расширении API классов. Если добавить функцию-член с той же сигнатурой, что и у функции-расширения, определенной клиентом класса, а затем заново скомпилировать код, то результат изменит значение и будет ссылаться на новую функцию-член. Среда IDE также предупредит вас о том, что функция-расширение затенена функцией-членом.

Мы уже обсуждали, как предоставлять дополнительные методы для внешних классов. Теперь поговорим о том, как сделать то же самое со свойствами.

3.3.5. Свойства-расширения

Синтаксис объявления свойств Kotlin описан в подразделе 2.2.1, а *свойства-расширения* можно определять так же, как и функции-расширения. Они используются для расширения классов с помощью API, доступ к которым можно получить благодаря синтаксису свойств, а не функций. И хотя это *свойства*, у них не может быть никакого состояния, поскольку нет подходящего места для его хранения — невозможно добавить дополнительные поля к существующим экземплярам Java-объектов. В результате свойства-расширения всегда должны определять пользовательские методы доступа, подобные тем, которые описаны в подразделе 2.2.2. Тем не менее они делают возможным зачастую полезное более краткое и лаконичное обозначение.

Выше приводилось определение функции `lastChar()`. Теперь преобразуем ее в свойство, что позволит выполнять вызов в виде `"myText".lastChar`, а не `"myText".lastChar()` (листинг 3.8).

Листинг 3.8. Объявление свойства-расширения

```
val String.lastChar: Char
    get() = this.get(length - 1)
```

Как и в случае с функциями, свойство-расширение выглядит как обычное свойство с добавленным типом приемника. Геттер всегда должен быть определен, поскольку не существует теневого поля и, следовательно, нет реализации геттера по умолчанию. Использование инициализаторов не допускается по той же причине: хранить их значения попросту нелегко.

При определении того же свойства для класса `StringBuilder` можно объявить его с типом `var`, поскольку содержимое `StringBuilder` будет изменяться (листинг 3.9).

Листинг 3.9. Объявление изменяемого свойства-расширения

```
var StringBuilder.lastChar: Char
    get() = this.get(length - 1) ← Геттер свойства
    set(value) { ← Сеттер свойства
        this.setCharAt(length - 1, value)
    }
```

Доступ к свойствам-расширениям осуществляется точно так же, как и к свойствам членов:

```
fun main() {
    val sb = StringBuilder("Kotlin?")
    println(sb.lastChar)
    // ?
    sb.lastChar = '!'
    println(sb)
    // Kotlin!
}
```

Обратите внимание, что если необходимо получить доступ к свойству-расширению из Java, то его геттер и сеттер указываются в явном виде: `StringUtilKt.getLastChar("Java")` и `StringUtilKt.setLastChar(sb, '!')`.

Теперь, обсудив концепции расширений в целом, в следующем разделе вернемся к теме коллекций.

3.4. РАБОТА С КОЛЛЕКЦИЯМИ: VARARG, ИНФИКСНЫЕ ВЫЗОВЫ И ПОДДЕРЖКА БИБЛИОТЕК

В этом разделе описаны некоторые функции из стандартной библиотеки Kotlin, позволяющие работать с коллекциями, а также приводятся связанные с ними языковые возможности:

- ключевое слово `vararg` позволяет объявить функцию, принимающую произвольное количество аргументов;
- с помощью *инфиксной* нотации можно осуществить простой вызов функций с одним аргументом;
- *объявления деструктуризации* дают возможность распаковать одно составное значение в несколько переменных.

3.4.1. Расширение API коллекций Java

В начале этой главы прозвучала идея о том, что коллекции в Kotlin — это те же классы, что и в Java, но с расширенным API. Помимо прочих возможностей, приводились примеры получения последнего элемента в списке и вычисления суммы набора чисел:

```
fun main() {
    val strings: List<String> = listOf("first", "second", "fourteenth")
    strings.last()
    // fourteenth

    val numbers: Collection<Int> = setOf(1, 14, 2)
    numbers.sum()
    // 17
}
```

И самый большой интерес представляли механизмы выполнения этих операций — почему в Kotlin можно так свободно работать с коллекциями, несмотря на то что они являются экземплярами классов библиотеки Java. Теперь ответ должен быть очевиден: функции `last` и `sum` объявлены как функции-расширения и всегда импортируются по умолчанию в файлы Kotlin.

Функция `last` — расширение класса `List`, и она не сложнее, чем `lastChar` класса `String`, который вам знаком по разделу 3.3. А для `sum` используется упрощенное объявление (реальная библиотечная функция работает не только для чисел типа `Int`):

```
fun <T> List<T>.last(): T { /* возвращает последний элемент */ }
fun Collection<Int>.sum(): Int { /* суммирует все элементы */ }
```

В стандартной библиотеке Kotlin объявлено множество функций-расширений, и перечислять здесь их все бессмысленно. Вы можете захотеть изучить все содержимое стандартной библиотеки Kotlin. Но в этом нет необходимости. При создании каждой операции с коллекцией или любым другим объектом механизм завершения кода в IDE покажет вам все возможные функции, доступные для данного типа объекта. В списке будут показаны обычные методы и функции-расширения — остается выбрать нужный вариант. Кроме того, в стандартном справочнике библиотеки (<https://kotlinlang.org/api/latest/jvm/stdlib/>) перечислены все методы, доступные для каждого класса-члена библиотеки, а также расширения.

Кроме того, в начале главы вы познакомились с функциями для создания коллекций. Общая их черта — возможность вызова с произвольным количеством аргументов. В следующем подразделе приводится синтаксис для объявления таких функций.

3.4.2. Vararg — функции, принимающие произвольное количество аргументов

При вызове функции для создания списка ей можно передать любое количество аргументов:

```
val list = listOf(2, 3, 5, 7, 11)
```

В стандартной библиотеке сигнатура этой функции имеет следующий вид:

```
fun listOf<T>(vararg values: T): List<T> { /* реализация */ }
```

В этом методе используется функциональная возможность языка для передачи методу произвольного количества аргументов с упаковкой их в *массив с переменным количеством аргументов*. В Kotlin он похож на аналогичный массив Java, но синтаксис немного другой: вместо трех точек после типа в Kotlin используется модификатор **vararg** для параметра.

Еще одно различие заключается в синтаксисе вызова функции, когда передаваемые аргументы уже упакованы в массив. В Java массив передается непосредственно, тогда как Kotlin требует явной распаковки, чтобы каждый элемент массива стал отдельным аргументом вызываемой функции. Эта функция называется *оператором spread* (расширение). Применять ее просто: поставьте символ ***** перед соответствующим аргументом. Во фрагменте кода ниже выполняется расширение массива **args** с аргументами командной строки, переданными в функцию **main**, что позволит использовать его в качестве аргументов переменных для функции **listOf**:

```
fun main(args: Array<String>) {
    val list = listOf("args: ", *args)
    println(list)
}
```

← Оператор **spread** распаковывает содержимое массива

В этом примере показано, что оператор **spread** дает возможность объединить значения из массива и некоторые фиксированные значения в одном вызове. В Java такая возможность не поддерживается.

А теперь перейдем к картам. Кратко обсудим еще один способ улучшить читаемость вызовов функций Kotlin — инфиксный вызов.

3.4.3. Работа с парами: инфиксные вызовы и объявления деструктуризации

Для создания карты используется функция `mapOf`:

```
val map = mapOf(1 to "one", 7 to "seven", 53 to "fifty-three")
```

В начале главы мы обещали дать еще одно объяснение, и сейчас для этого самый подходящий момент. Слово `to` в этой строке кода не встроенная конструкция, а вызов метода особого типа — *инфиксный вызов*.

При таком вызове имя метода помещается непосредственно между именем целевого объекта и параметром, без дополнительных разделителей. Эти вызовы функций эквивалентны:

```
1.to("one")    ← Вызов функций обычным способом
1 to "one"     ← Вызов функции с использованием инфиксной нотации
```

Инфиксные вызовы можно использовать с обычными методами и функциями-расширениями, которым нужен строго один необходимый параметр. Чтобы разрешить вызов функции с помощью инфиксной нотации, необходимо пометить ее модификатором `infix`. Упрощенная версия объявления функции `to` выглядит так:

```
infix fun Any.to(other: Any) = Pair(this, other)
```

Функция `to` возвращает экземпляр `Pair` — предопределенного класса стандартной библиотеки Kotlin. Он представляет пару элементов. В фактических объявлениях `Pair` и `to` используются обобщения. Но в целях упрощения кода мы их опустим.

Обратите внимание на возможность инициализировать две переменные содержимым `Pair` непосредственно:

```
val (number, name) = 1 to "one"
```

Такая функциональная возможность называется *объявлением деструктуризации*. На рис. 3.3 показано, как это работает с парами.

Возможность объявления деструктуризации работает не только с парами. Например, можно инициализировать две переменные, `key` и `value`, содержимым записи карты (с чем вы уже вкратце познакомились в главе 2).

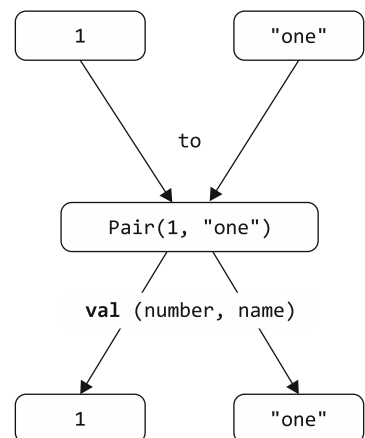


Рис. 3.3. Создание пары с помощью функции `to` и ее распаковка с помощью объявления деструктуризации

Этот механизм работает и с циклами, что было показано в реализации `joinToString`, использующей функцию `withIndex`:

```
for ((index, element) in collection.withIndex()) {  
    println("$index: $element")  
}
```

В главе 9 будут описаны общие правила деструктуризации выражения и его использования для инициализации нескольких переменных.

Функция `to` — функция-расширение. Из любых элементов можно создать пару, что означает расширение обобщенного приемника: можно написать `1 to "one"`, `"one" to 1`, `list to list.size()` и т. д. Рассмотрим сигнатуру функции `mapOf`:

```
fun <K, V> mapOf(vararg values: Pair<K, V>): Map<K, V>
```

Как и `listOf`, `mapOf` принимает переменное количество аргументов, однако в данном случае это должны быть пары ключей и значений. Создание новой карты может выглядеть в Kotlin как особая конструкция, но все же это обычная функция с лаконичным синтаксисом.

3.5. РАБОТА СО СТРОКАМИ И РЕГУЛЯРНЫМИ ВЫРАЖЕНИЯМИ

Строки в Kotlin ничем не отличаются от строк в Java. Можно передать строку из кода Kotlin любому методу Java, а также использовать любые методы стандартной библиотеки Kotlin для строк из кода Java. Не происходит никакого преобразования, и не создаются дополнительные объекты-обертки.

Но в Kotlin работать со стандартными строками Java удобнее, поскольку в нем есть несколько полезных функций-расширений. Кроме того, в этом языке скрыты некоторые запутанные методы и добавлены более понятные расширения. В качестве первого примера различий в API рассмотрим разделение строк в Kotlin.

3.5.1. Разбиение строк

Вероятно, вам знаком метод `split` для типа данных `String`. Им пользуются все, но иногда разработчики жалуются на него на сервисе Stack Overflow (<http://stackoverflow.com>): «Метод `split` в Java не работает с точками». Часто программисты пишут выражение вида `"12.345-6.A".split(".")` и ожидают в результате получить массив `[12, 345-6, A]`. Но в Java метод `split` вернет пустой массив! Это происходит потому, что он принимает в качестве параметра регулярное выражение и разбивает строку на несколько в соответствии с выражением. И в данном случае точка `(.)` — регулярное выражение, обозначающее любой символ.

В Kotlin этот запутанный метод скрыт, и для замены используется несколько перегруженных расширений с именем `split` и с различными аргументами. Один метод

принимает регулярное выражение и требует значения типа `Regex` или `Pattern` вместо `String`. Благодаря этому всегда понятно, как интерпретируется строка, переданная методу, — как обычный текст или как регулярное выражение.

Разбить строку с точкой или тире можно так:

```
fun main() {
    println("12.345-6.A".split("\\.|-".toRegex()))
    // [12, 345, 6, A]
```

← Создание регулярного выражения в явном виде

Синтаксис регулярных выражений в Kotlin и Java не имеет различий. Здесь шаблон соответствует точке (для нее применяется экранирование, чтобы показать, что имеется в виду буквальным символом, а не подстановочный знак) или тире. Кроме того, API для работы с регулярными выражениями похожи на стандартные API библиотеки Java, но более идиоматичны. Например, в Kotlin для преобразования строки в регулярное выражение используется функция-расширение `toRegex`.

Но для такого простого случая это не потребуется. Другая перегруженная версия функции-расширения `split` в Kotlin принимает произвольное количество разделителей в виде строк обычного текста:

```
fun main() {
    println("12.345-6.A".split(".", "-"))
    // [12, 345, 6, A]
```

← Указание нескольких разделителей

Обратите внимание, что вместо этого можно указать символьные аргументы и написать выражение вида `"12.345-6.A".split('.', '-')` — результат останется неизменным. Этот метод заменяет аналогичный метод языка Java, который принимает только один символ в качестве разделителя.

3.5.2. Регулярные выражения и строки в тройных кавычках

Рассмотрим другой пример с двумя разными реализациями: в первой применяются расширения для типа данных `String`, а во второй операции выполняются над регулярными выражениями. Задача — разобрать полное имя пути к файлу на составляющие: каталог, имя файла и расширение. Стандартная библиотека Kotlin содержит функции для получения подстроки до (или после) первого (или последнего) обнаружения заданного разделителя. На рис. 3.4 и в листинге 3.10 показано, как можно использовать их для решения этой задачи.

Листинг 3.10. Использование расширений `String` для синтаксического анализа пути

```
fun parsePath(path: String) {
    val directory = path.substringBeforeLast("/")
    val fullName = path.substringAfterLast("/")
    val fileName = fullName.substringBeforeLast(".")
    val extension = fullName.substringAfterLast(".")
}
```

```

println("Dir: $directory, name: $fileName, ext: $extension")
}

fun main() {
    parsePath("/Users/yole/kotlin-book/chapter.adoc")
    // Dir: /Users/yole/kotlin-book, name: chapter, ext: adoc
}

```



Рис. 3.4. Разделение пути на каталог, имя файла и его расширение с помощью функций `substringBeforeLast` и `substringAfterLast`

Подстрока перед последним символом слеша в пути (`path`) файла — это путь к вложенному каталогу, подстрока после последней точки — расширение файла, а имя файла находится между ними.

В Kotlin работа со строками упрощается, поскольку нет нужды использовать регулярные выражения. Они, несомненно, представляют собой мощный инструмент, но иногда, когда они написаны, их трудно понять. Если потребуется использовать регулярные выражения, то вам поможет стандартная библиотека Kotlin. В листинге 3.11 показано, как эту же задачу можно решить с помощью регулярных выражений.

Листинг 3.11. Использование регулярных выражений для синтаксического анализа путей

```

fun parsePathRegex(path: String) {
    val regex = """.(.)/(.)\.(.)""".toRegex()
    val matchResult = regex.matchEntire(path)
    if (matchResult != null) {
        val (directory, filename, extension) = matchResult.destructured
        println("Dir: $directory, name: $filename, ext: $extension")
    }
}

fun main() {
    parsePathRegex("/Users/yole/kotlin-book/chapter.adoc")
    // Dir: /Users/yole/kotlin-book, name: chapter, ext: adoc
}

```

В этом примере регулярное выражение записано в *строке с тройными кавычками*. В такой строке не нужно экранировать никакие символы, в том числе обратный слеш. Следовательно, регулярное выражение для литеральной точки можно закодировать с помощью `\.`, а не `\\.`, как в обычном строковом литерале (рис. 3.5).

Это регулярное выражение делит путь на три группы с разделителем в виде слеша и точки. Шаблон `.+` соответствует любому символу с начала, поэтому первая группа `(.+)` содержит подстроку до последнего слеша. Эта подстрока содержит все предыдущие слеша, поскольку они соответствуют шаблону «любой символ». Аналогично во второй группе содержится подстрока перед последней точкой (и после последнего слеша), а в третьей — оставшаяся часть.

Перейдем к обсуждению реализации функции `parsePathRegex`, показанной в предыдущем примере. В нем создается регулярное выражение, которое затем сопоставляется с входным путем. Если совпадение успешно (результат не равен `null`), то значение его свойства `destructured` присваивается соответствующим переменным. Такой же синтаксис используется при инициализации двух переменных с помощью класса `Pair` (более подробная информация приводится в разделе 9.4).

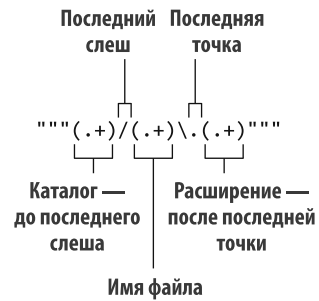


Рис. 3.5. Регулярное выражение для разделения пути на каталог, имя файла и его расширение

3.5.3. Многострочные строковые значения с тройными кавычками

Строки в тройных кавычках нужны не только для того, чтобы избежать экранирования символов. Такой строковый литерал может содержать любые символы, в том числе переносы строк. Поэтому можно легко добавлять в программы текст с переносами. В качестве примера нарисуем несколько ASCII-артов:

```
val kotlinLogo =
    """
    | //
    |//
    |/\
    """trimIndent()

fun main() {
    println(kotlinLogo)
    // | //
    // |//
    // |/\
}
```

Многострочная строка содержит все символы, заключенные в тройные кавычки. В ней есть переносы строк и отступы, используемые для форматирования кода. Обычно в подобных случаях интерес представляет только фактическое содержимое строки. Удалить общий минимальный отступ для всех строк, а также первую и последнюю пустые строки можно с помощью функции `trimIndent`.

ПРИМЕЧАНИЕ

В разных операционных системах для обозначения конца строки в файле используются разные символы: в Windows — CRLF (Carriage Return Line Feed), в Linux и macOS — LF (Line Feed). Независимо от используемой операционной системы Kotlin интерпретирует CRLF, LF и CR как перенос строки.

В предыдущем примере строка в тройных кавычках содержала переносы строк, но в ней нельзя использовать специальные символы, например `\n`. С другой стороны, не надо экранировать `\`, поэтому путь в стиле Windows `"C:\Users\yole\kotlin-book"` можно записать как `""C:\Users\yole\kotlin-book""`.

Кроме того, в строках такого типа можно использовать строковые шаблоны. Но многострочные строки не поддерживают экранирующие последовательности. И когда требуется использовать знак доллара буквально или экранированный символ Юникода в содержимом строки, придется использовать встроенные выражения. Поэтому не получится написать `val think = ""Hmm \uD83E\uDD14""`. Надо будет написать так: `val think = ""Hmm ${"\uD83E\uDD14"}""`, чтобы правильно интерпретировать символ, закодированный в этой строке.

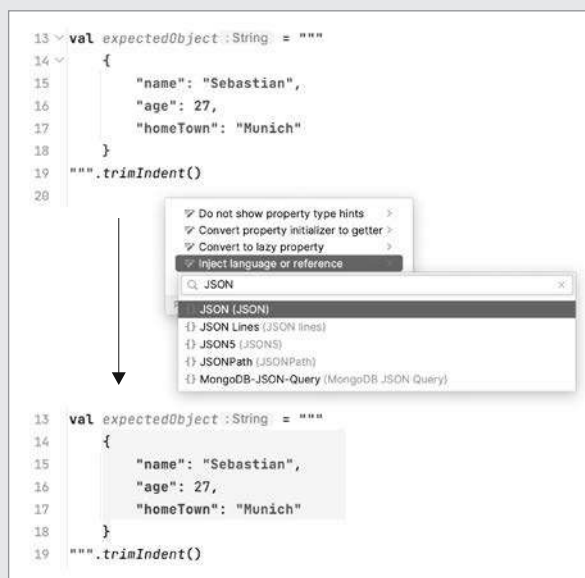
Многострочные строки могут быть полезны при создании тестов (помимо игр, использующих ASCII-арт). В тестах довольно часто требуется выполнить операцию, выдающую в результате многострочный текст (например, фрагмент веб-страницы или другой структурированный текст), и сравнить результат с ожидаемым. Использование строк такого типа — отличное решение, позволяющее добавлять ожидаемый вывод в тесты. Нет необходимости в неудобном экранировании или загрузке текста из сторонних файлов — просто поставьте несколько кавычек и поместите между ними ожидаемый HTML, XML, JSON или другой результат выполнения. А в целях лучшего форматирования используйте функцию `trimIndent`. Она, в свою очередь, служит еще одним примером функции-расширения:

```
val expectedPage = """
    <html lang="en">
        <head>
            <title>A page</title>
        </head>
        <body>
            <p>Hello, Kotlin!</p>
        </body>
    </html>
    """.trimIndent()

val expectedObject = """
    {
        "name": "Sebastian",
        "age": 27,
        "homeTown": "Munich"
    }
    """.trimIndent()
```

ВЫДЕЛЕНИЕ СИНТАКСИСА ВНУТРИ СТРОК С ТРОЙНЫМИ КАВЫЧКАМИ В INTELLIJ IDEA И ANDROID STUDIO

Использование строк в тройных кавычках для форматированного текста, например HTML или JSON, дает дополнительное преимущество: IntelliJ IDEA и Android Studio выделяют синтаксис внутри этих строковых литералов. Чтобы добавить эту функцию, поместите указатель мыши внутрь строки и нажмите клавиши Alt+Enter (или Option+Return на macOS). Вдобавок можно щелкнуть кнопкой мыши на плавающем желтом значке лампочки и выбрать из раскрывающегося списка пункт Inject Language or Reference (Вставка языка или ссылки). Затем нужно выбрать тип языка в строке (например, JSON). После этого многострочная строка превратится в формат JSON с правильным синтаксическим выделением цветом. Если в текстовом фрагменте окажется неправильно сформированный JSON, то система выдаст предупреждения и описательные сообщения об ошибках, и все это в строке Kotlin. По умолчанию это выделение текста временное. Чтобы указать IDE всегда вставлять строковый литерал с заданным языком, можно использовать аннотацию @Language("JSON").



```

14 @Language("JSON")
15 val expectedObject :String = """
16 {
17     "name": "Sebastian",
18     "age": 27,
19     "homeTown": "Munich"
20 }
21 """
22 .trimIndent()
  
```

Более подробную информацию о языковых вставках в IntelliJ IDEA и Android Studio можно узнать по ссылке <https://www.jetbrains.com/help/idea/using-language-injections.html>.

Можно сделать вывод, что функции-расширения — мощный способ, позволяющий расширить API существующих библиотек и адаптировать их к идиомам языка Kotlin. И действительно, большая часть стандартной библиотеки Kotlin состоит из функций-расширений для стандартных классов Java. Вдобавок существует множество разработанных сообществом библиотек. В них содержатся дружественные Kotlin расширения для сторонних библиотек.

Изучив улучшенные API для используемых в Kotlin библиотек, вернемся к вашему коду. Рассмотрим несколько новых вариантов использования функций-расширений, а также обсудим новую концепцию — *локальные функции*.

3.6. АККУРАТНЫЙ КОД: ЛОКАЛЬНЫЕ ФУНКЦИИ И РАСШИРЕНИЯ

Многие разработчики считают, что в хорошем коде не должно быть дублирования. Для этого принципа даже есть специальное название: не повторяться (Don't repeat yourself, DRY). Но при работе с кодом Java следовать этому принципу не всегда просто. Во многих случаях в IDE можно использовать функцию рефакторинга под названием Extract Method. Она разбивает длинные методы на более мелкие фрагменты, и их можно использовать в дальнейшем. Но это может усложнить понимание кода, поскольку в итоге будет сформирован класс со множеством мелких и не связанных между собой методов. Можно пойти еще дальше и сгруппировать извлеченные методы во внутренний класс. Это позволит сохранить структуру, но такой подход требует значительного количества шаблонов.

В Kotlin предлагается более аккуратное решение: можно вложить извлеченные функции в контейнирующую функцию. Таким образом, создается необходимая структура без лишних синтаксических накладок.

Рассмотрим, как с помощью локальных функций исправить довольно распространенный случай дублирования кода. В листинге 3.12 функция сохраняет данные пользователя в базе данных, и необходимо убедиться, что объект пользователя содержит корректные данные.

Листинг 3.12. Функция с повторяющимся кодом

```
class User(val id: Int, val name: String, val address: String)

fun saveUser(user: User) {
    if (user.name.isEmpty()) {
        throw IllegalArgumentException(
            "Can't save user ${user.id}: empty Name"
        )
    }

    if (user.address.isEmpty()) {
        throw IllegalArgumentException(
            "Can't save user ${user.id}: empty Address"
        )
    }

    // Сохранение данных пользователя в базе данных
}
```



Валидация полей продублирована

```
fun main() {
    saveUser(User(1, "", ""))
    // java.lang.IllegalArgumentException: Can't save user 1: empty Name
}
```

Здесь объем продублированного кода довольно мал. Поэтому кажется, что незачем хранить в классе полноценный метод для обработки одного особого случая валидации данных пользователя. Но если поместить код валидации в локальную функцию, то можно избавиться от дублирования и при этом сохранить четкую структуру кода. Этот механизм показан в листинге 3.13.

Листинг 3.13. Извлечение локальной функции во избежание повторений

```
class User(val id: Int, val name: String, val address: String)

fun saveUser(user: User) {
    fun validate(user: User,
                 value: String,
                 fieldName: String) {
        if (value.isEmpty()) {
            throw IllegalArgumentException(
                "Can't save user ${user.id}: empty $fieldName")
        }
    }

    validate(user, user.name, "Name")
    validate(user, user.address, "Address")

    // Сохранение данных пользователя в базе данных
}
```

Объявление локальной функции
для валидации любых полей

Вызов локальной
функции для валидации
определенных полей

Так код выглядит лучше: логика валидации не дублируется, но все еще ограничена областью действия функции `validate`. И вам не составит труда добавлять варианты валидации по мере развития проекта и появления новых полей у объекта `User`. Однако передача объекта `User` в функцию валидации выглядит несколько некрасиво. Но это совершенно не обязательно, поскольку у локальных функций есть доступ ко всем параметрам и переменным замкнутой функции. Воспользуемся этим и избавимся от лишнего параметра объекта `User` (листинг 3.14).

Листинг 3.14. Доступ к параметрам внешней функции в локальной

```
class User(val id: Int, val name: String, val address: String)

fun saveUser(user: User) {
    fun validate(value: String, fieldName: String) {
        if (value.isEmpty()) {
            throw IllegalArgumentException(
                "Can't save user ${user.id}: " +
                "empty $fieldName")
        }
    }
}
```

Теперь параметр объекта
пользователя не дублируется
в функции `saveUser`

Доступ к параметрам внешней
функции можно получить напрямую

```

        validate(user.name, "Name")
        validate(user.address, "Address")

    // Сохранение данных пользователя в базе данных
}

```

В качестве дальнейшего улучшения кода можно перенести логику валидации в функцию-расширение класса `User` (листинг 3.15).

Листинг 3.15. Извлечение логики в функцию-расширение

```

class User(val id: Int, val name: String, val address: String)

fun User.validateBeforeSave() {
    fun validate(value: String, fieldName: String) {
        if (value.isEmpty()) {
            throw IllegalArgumentException(
                "Can't save user $id: empty $fieldName")
        }
    }

    validate(name, "Name")
    validate(address, "Address")
}

fun saveUser(user: User) {
    user.validateBeforeSave()

    // Сохранение данных пользователя в базе данных
}

```

Можно напрямую получать доступ к свойствам класса `User`

Вызов функции-расширения

Извлечение части кода в функцию-расширение оказывается неожиданно полезным действием. Даже если класс `User` находится в вашей кодовой базе, а не в библиотеке, не стоит помещать эту логику в метод класса `User`, поскольку она относится к другим местам использования данного класса. Если следовать этому подходу, то API класса будет содержать только основные методы, а класс останется небольшим и в нем будет легко разобраться. С другой стороны, если функции работают с одним объектом и не нуждаются в доступе к его приватным данным, то они могут обращаться к его членам без дополнительных квалификаторов, как в листинге 3.15.

Функции-расширения тоже можно объявить как локальные. Поэтому код можно еще доработать и добавить функцию `User.validateBeforeSave` в качестве локальной в функцию `saveUser`. Но глубоко вложенные локальные функции обычно трудно читать, поэтому рекомендуется использовать не более одного уровня вложенности.

РЕЗЮМЕ

- Kotlin расширяет классы коллекций Java с помощью более богатого API.
- Определение значений по умолчанию для параметров функции значительно сокращает необходимость определения перегруженных функций, а синтаксис

именованных аргументов делает вызовы функций с большим количеством параметров гораздо более удобочитаемыми.

- Функции и свойства можно объявить не только как члены класса, но и непосредственно в файле. Это позволяет создать более гибкую структуру кода.
- Функции-расширения и свойства-расширения позволяют расширять API любого класса, в том числе классы из внешних библиотек, не изменяя его исходный код и не добавляя накладные расходы во время выполнения.
- С помощью инфиксных вызовов можно создать чистый синтаксис для вызова подобных операторам методов с одним аргументом.
- Kotlin предоставляет большое количество удобных функций для работы с обычными строками и с регулярными выражениями.
- Строки в тройных кавычках предоставляют чистый способ записи выражений, что в Java потребовало бы большого количества экранирования и конкатенации строк.
- Локальные функции помогут структурировать код более чисто и устранить дублирование.

4

Классы, объекты и интерфейсы

В этой главе

- ✓ Классы и интерфейсы.
- ✓ Нетривиальные свойства и конструкторы.
- ✓ Классы данных.
- ✓ Делегирование класса.
- ✓ Использование ключевого слова `object`.

В этой главе вы получите более глубокое представление о работе с классами и интерфейсами в Kotlin. Основной синтаксис объявления класса приведен в разделе 2.2, где вы узнали, как объявлять методы и свойства, использовать простые первичные конструкторы (удобные, не так ли?) и работать с перечислениями. Но в этой теме еще есть много интересного для изучения! Классы и интерфейсы в Kotlin немного отличаются от привычных вам по Java: например, интерфейсы могут содержать объявления свойств. По умолчанию в Kotlin объявления отмечены как `final` и `public`. Кроме того, вложенные классы не будут вложенными по умолчанию — они не содержат скрытой ссылки на свой внешний класс.

Конструкторы в большинстве случаев можно объявлять как первичные, с коротким синтаксисом. Но в Kotlin есть и полный. Он дает возможность объявлять конструкторы, имеющие нетривиальную логику инициализации. То же самое касается свойств: лаконичный синтаксис удобен, но есть возможность легко определить собственные реализации методов доступа.

Компилятор Kotlin может генерировать полезные методы, чтобы избежать много-словия. Объявляя класс с ключевым словом `data`, можно дать компилятору указание генерировать несколько стандартных методов для этого класса. Вдобавок можно не писать делегирующие методы вручную, поскольку паттерн делегирования изначально поддерживается в Kotlin.

Еще в данной главе содержится описание ключевого слова `object`. С помощью него выполняется объявление класса и создается его экземпляр. Это ключевое слово используется для создания одиночек (объектов-синглтонов), объектов-компаньонов и объектных выражений (аналог анонимных классов Java). Для начала поговорим о классах и интерфейсах, а также об особенностях определения иерархии классов в Kotlin.

4.1. ОПРЕДЕЛЕНИЕ ИЕРАРХИИ КЛАССОВ

В этом разделе приводится описание создания иерархии классов в Kotlin, рассматриваются модификаторы видимости и доступа Kotlin, а также то, какие значения выбираются для них по умолчанию. Вдобавок будет представлен модификатор `sealed`, предназначенный для ограничения возможных подклассов класса или реализации интерфейса.

4.1.1. Интерфейсы в Kotlin

Начнем с определения и реализации интерфейсов Kotlin. Они могут содержать определения абстрактных и реализации неабстрактных методов, но не могут содержать никакого состояния.

Для объявления интерфейса в Kotlin используется ключевое слово `interface`, а не `class`. В листинге 4.1 показан пример интерфейса, указывающего на кликабельный элемент, например на кнопку или гиперссылку.

Листинг 4.1. Объявление простого интерфейса

```
interface Clickable {  
    fun click()  
}
```

В этом фрагменте кода объявляется интерфейс с единственным абстрактным методом `click`. Он не возвращает никакого значения. Все неабстрактные классы для реализации интерфейса должны предоставить реализацию этого метода.

ПРИМЕЧАНИЕ

Технически данный метод возвращает значение `Unit`, эквивалент значения `void` в Java. Подробнее эту тему обсудим в подразделе 8.1.6.

Чтобы пометить кнопку как `Clickable`, необходимо поместить имя интерфейса за двоеточием после имени класса и предоставить реализацию функции `click` (листинг 4.2).

Листинг 4.2. Реализация простого интерфейса

```
class Button : Clickable {
    override fun click() = println("I was clicked")
}

fun main() {
    Button().click()
    // I was clicked
}
```

В Kotlin двоеточие после имени класса используется как для *композиции* (то есть реализации интерфейсов), так и для *наследования* (то есть создания подклассов; подробнее об этом поговорим в подразделе 4.1.2). Класс может реализовать произвольное количество интерфейсов, но расширять может только один класс.

Модификатор `override` используется для обозначения методов и свойств, переопределяющих методы и свойства суперкласса или интерфейса. В Java аннотация `@Override` применяется как дополнительная, но в Kotlin использование модификатора `override` является *обязательным*. И это условие не позволит случайно переопределить метод, если он был добавлен после написания реализации. Код не будет компилироваться, пока у метода не будет явного указания `override` или пока вы его не переименуете.

У метода интерфейса может быть реализация по умолчанию. Для этого достаточно указать тело метода. В этом случае можно добавить функцию вывода текста `showOff` с реализацией по умолчанию в интерфейс `Clickable` (листинг 4.3).

Листинг 4.3. Определение метода с телом в интерфейсе

```
interface Clickable {
    fun click()           ← Обычное объявление метода
    fun showOff() = println("I'm clickable!") ← Метод с реализацией по умолчанию
}
```

При реализации этого интерфейса необходимо предоставить реализацию для функции `click`. Можно переопределить поведение метода `showOff` или опустить его, если поведение по умолчанию подходит для решаемой задачи.

Теперь предположим, что другой интерфейс тоже определяет метод `showOff`, и его реализация выглядит следующим образом (листинг 4.4).

Листинг 4.4. Определение другого интерфейса, реализующего тот же метод

```
interface Focusable {
    fun setFocus(b: Boolean) =
        println("I ${if (b) "got" else "lost"} focus.")

    fun showOff() = println("I'm focusable!")
}
```

Что произойдет, если вам потребуется реализовать оба интерфейса в классе? Каждый из них содержит метод `showOff` с реализацией по умолчанию. Так какой же из них победит? Никакой. Если метод `showOff` не реализован в явном виде, то вместо выполнения мы получим следующую ошибку компилятора:

The class 'Button' must override public open fun showOff() because it inherits many implementations of it.

Компилятор Kotlin указывает, что необходимо предоставить собственную реализацию (листинг 4.5).

Листинг 4.5. Вызов реализации метода унаследованного интерфейса

```
class Button : Clickable, Focusable {
    override fun click() = println("I was clicked")

    override fun showOff() {
        super<Clickable>.showOff()
        super<Focusable>.showOff()
    }
}
```

Необходимо предоставить явную реализацию, если наследуется более одной реализации для одного и того же члена

Здесь ключевое слово `super` в угловых скобках — имя супертипа. Оно указывает на родителя, метод которого требуется вызвать

Теперь класс `Button` реализует два интерфейса. Функция `showOff()` создается вызовом обеих реализаций, унаследованных от супертипов. Для вызова такой реализации используется ключевое слово `super` с именем базового типа в угловых скобках: `super<Clickable>.showOff()` (синтаксис отличается от синтаксиса Java `Clickable.super.showOff()`).

Когда требуется вызвать только одну унаследованную реализацию, можно использовать синтаксис тела выражения:

```
override fun showOff() = super<Clickable>.showOff()
```

Чтобы проверить полученные знания, создайте экземпляр класса `Button` и вызовите все унаследованные методы — переопределенные функции `showOff` и `click`. И вместе с тем вызовите функцию `setFocus` из интерфейса `Focusable` с реализацией по умолчанию (листинг 4.6).

Листинг 4.6. Вызов унаследованных и переопределенных методов

```
fun main() {
    val button = Button()
    button.showOff()
    // I'm clickable!
    // I'm focusable!
    button.setFocus(true)
    // I got focus.
    button.click()
    // I was clicked.
}
```

РЕАЛИЗАЦИЯ ИНТЕРФЕЙСОВ С ПОМОЩЬЮ ТЕЛ МЕТОДОВ В JAVA

Kotlin компилирует каждый интерфейс с методами по умолчанию в комбинацию обычного интерфейса и класса. Тела методов в этом классе хранятся в виде статических методов. Интерфейс содержит только объявления, а класс — все реализации в виде статических методов. Поэтому, если нужно реализовать такой интерфейс в Java-классе, необходимо определить собственные реализации всех методов, в том числе те, у которых в Kotlin есть тела методов. Например, класс `JavaButton` с реализацией интерфейса `Clickable` должен предоставить реализации функций `click` и `showOff`. Хотя Kotlin и предоставляет реализацию по умолчанию для последней, что показано в листинге 4.7.

Листинг 4.7. Вызов унаследованных и переопределенных методов

```
class JavaButton implements Clickable {  
    @Override  
    public void click() {  
        System.out.println("I was clicked");  
    }  
  
    @Override  
    public void showOff() {  
        System.out.println("I'm showing off");  
    }  
}
```

Код на Java не может использовать реализации по умолчанию из Kotlin

Вы узнали, как в Kotlin реализуются методы из интерфейсов. Перейдем к другой части темы — к переопределению членов из базовых классов.

4.1.2. Модификаторы `open`, `final` и `abstract`: `final` по умолчанию

В Kotlin нельзя просто создать подкласс для класса или переопределить какие-либо методы базового класса — все классы и методы по умолчанию помечены как *final*.

В этом заключается отличие от Java, где разрешено создавать подклассы любого класса и можно переопределить любой метод без явной пометки ключевым словом `final`. Но почему при создании Kotlin разработчики языка не последовали этому подходу? Потому что он часто вызывает проблемы, хотя и достаточно удобен.

Так называемая проблема *хрупкого базового класса* возникает, когда модификации базового класса могут вызвать некорректное поведение подклассов. Это происходит по причине того, что измененный код базового класса больше не соответствует ожиданиям в его подклассах. Если класс не содержит точных правил выполнения подклассификации (какие методы должны быть переопределены и как), то клиенты рискуют переопределить методы таким способом, который не был предусмотрен автором базового класса. Невозможно проанализировать все подклассы, поэтому базовый класс считается «хрупким» в том смысле, что любое изменение в нем может привести к неожиданным изменениям в поведении подклассов.

Для предотвращения этой проблемы в книге *Effective Java*¹ Джошуа Блоха (Addison-Wesley, 2008), одной из самых известных книг о хорошем стиле программирования на Java, рекомендуется «проектировать и документировать наследование или просто запретить его». Это означает, что все не предназначенные для переопределения в подклассах классы и методы необходимо пометить как `final`. И Kotlin следует этой философии: все классы, методы и свойства получают модификатор `final` по умолчанию.

Если требуется разрешить создание подклассов, то необходимо пометить класс как открытый с помощью модификатора `open`. Кроме того, для переопределения нужно добавить этот модификатор к каждому свойству или методу.

Допустим, вы хотите разнообразить свой пользовательский интерфейс, не ограничиваясь простой кнопкой, и создать кликабельную кнопку класса `RichButton`. Его подклассы должны иметь возможность создавать собственные анимации. Но недопустимо нарушать базовое поведение, например отключать кнопку. Такой класс можно определить следующим образом (листинг 4.8).

Листинг 4.8. Объявление открытого класса с открытым методом

```
open class RichButton : Clickable {
    fun disable() { /* ... */ }
    open fun animate() { /* ... */ }
    override fun click() { /* ... */ }
}
```

← Открытый класс — наследование допустимо

← Эта функция помечена как `final` — ее нельзя переопределить в подклассе

← Открытая функция — ее можно переопределить в подклассе

← Эта функция переопределяет открытую функцию и сама принимает тот же модификатор

Это означает, что подкласс `RichButton` может, в свою очередь, выглядеть следующим образом (листинг 4.9).

Листинг 4.9. Объявление подкласса открытого класса с переопределением открытых методов

```
class ThemedButton : RichButton() {
    override fun animate() { /* ... */ }
    override fun click() { /* ... */ }
    override fun showOff() { /* ... */ }
}
```

← По умолчанию в классе `RichButton` у функции `disable` модификатор `final`, поэтому ее нельзя переопределить здесь

← Функция `animate` получила модификатор `open`, и ее можно переопределить

← Функцию `click` можно переопределить, поскольку в классе `RichButton` она не получила модификатора `final`

← Функцию `showOff` можно переопределить, даже если в классе `RichButton` для нее нет разрешения на переопределение

Обратите внимание, что при переопределении члена базового класса или интерфейса переопределяемый член тоже получит модификатор `open` по умолчанию. Если это состояние необходимо изменить и запретить подклассам текущего класса переопределять реализацию, то можно явно пометить переопределяемый член как `final` (листинг 4.10).

¹ Блох Дж. Java: эффективное программирование.

Листинг 4.10. Запрет на переопределение

```
open class RichButton : Clickable {
    final override fun click() { /* ... */ }
}
```

← Модификатор `final` здесь нелишний, поскольку переопределение без `final` подразумевает наличие модификатора `open`

ОТКРЫТЫЕ КЛАССЫ И УМНЫЕ ПРЕОБРАЗОВАНИЯ

У классов с модификатором `final` по умолчанию есть существенное преимущество: с ними можно применять умные преобразования в большем количестве сценариев. В подразделе 2.3.6 мы говорили, что компилятор может выполнить умное преобразование (автоматическое приведение типа для доступа к членам класса без дополнительного приведения типа вручную) только для переменных, которые не могли измениться после проверки типа.

В случае с классами это означает, что умное преобразование можно применять только к свойствам класса с модификатором `val` и без пользовательских методов доступа. Это требование говорит о том, что свойство должно быть помечено как `final`. В противном случае подкласс может переопределить свойство и определить собственный метод доступа, нарушив ключевое требование умных преобразований.

Поскольку свойства по умолчанию помечены как `final`, можно использовать умное преобразование с большинством свойств. А отсутствие явного указания модификатора улучшает выразительность кода.

Класс можно объявить и как `abstract`, и тогда создание экземпляров будет запрещено. Абстрактный класс обычно содержит абстрактные члены без реализаций — их необходимо переопределять в подклассах. Абстрактные члены всегда открыты, поэтому модификатор `open` не требуется (точно так же, как не требуется модификатор `open` в `interface`).

Примером абстрактного класса служит класс для определения свойств анимации, таких как скорость и количество кадров, а также поведение при запуске анимации. Эти свойства и методы работают только при реализации другим объектом, так что класс `Animated` помечен как `abstract` (листинг 4.11).

Листинг 4.11. Объявление абстрактного класса

```
abstract class Animated {
    abstract val animationSpeed: Double
    val keyframes: Int = 20
    open val frames: Int = 60

    abstract fun animate()
    open fun stopAnimating() { /* ... */ }
    fun animateTwice() { /* ... */ }
}
```

← Это абстрактный класс, и создавать его экземпляры невозможно

← Это абстрактное свойство: у него нет значения, а в подклассах необходимо переопределять его значение или метод доступа

← По умолчанию свойства в абстрактных классах закрыты, но их можно обозначить вручную с помощью модификатора `open`

← Это абстрактная функция: у нее нет реализации, и ее необходимо переопределить в подклассе

← Не абстрактные функции в абстрактном классе закрыты по умолчанию, но их можно пометить модификатором `open`

В табл. 4.1 перечислены модификаторы доступа в Kotlin. Комментарии в таблице относятся к модификаторам в классах. В интерфейсах ключевые слова `final`, `open` или `abstract` не используются. Член интерфейса всегда с модификатором `open`, и его нельзя изменить на `final`. Без тела он всегда будет помечен как `abstract`, но указания этого ключевого слова не требуется.

Таблица 4.1. Значение модификаторов доступа в классе

Модификатор	Соответствующий член	Комментарии
<code>final</code>	Нельзя переопределить	Используется по умолчанию для членов класса
<code>open</code>	Можно переопределить	Необходимо указать в явном виде
<code>abstract</code>	Необходимо переопределить	Можно использовать только в абстрактных классах; у абстрактных членов класса не может быть реализации
<code>override</code>	Переопределяет член в супер-классе или интерфейсе	Переопределенные члены открыты по умолчанию, если не помечены как <code>final</code>

Обсудив модификаторы управления наследованием, рассмотрим роль модификаторов видимости в Kotlin.

4.1.3. Модификаторы видимости: `public` по умолчанию

Модификаторы видимости помогают контролировать доступ к объявленным элементам в кодовой базе. Ограничение видимости подробностей реализации класса гарантирует, что их можно изменить, не нарушая код, зависящий от этого класса.

В Kotlin есть модификаторы `public`, `protected` и `private`. Они аналогичны Java: объявления типа `public` видимы во всей кодовой базе; `protected` видимы в подклассах; `private` видимы внутри класса или внутри файла, если относятся к объявлениям верхнего уровня. По умолчанию в Kotlin используется модификатор `public`, и его можно не указывать.

ПРИМЕЧАНИЕ

Модификатор общего доступа по умолчанию отличается от известного вам по Java, но это удобное соглашение для разработчиков приложений. В свою очередь, авторы библиотек обычно хотят быть уверенными, что никакие части их API не будут случайно доступны. Скрытие ранее общедоступного API по факту стало бы переломным моментом. Для такой ситуации в Kotlin предусмотрен явный режим API. В этом случае нужно указать видимость для объявлений, открытых как API в общем доступе, а также явно указать тип для свойств и функций из таких API. Явный режим API можно включить с помощью команды компилятора `-Xexplicit-api={strict|warning}` или через систему сборки.

Вдобавок в Kotlin есть модификатор видимости внутри модуля — `internal`. *Модуль* — это набор скомпилированных совместно файлов Kotlin. Это может быть исходный набор Gradle, проект Maven или модуль IntelliJ IDEA.

ПРИМЕЧАНИЕ

При использовании Gradle исходный набор `test` может получить доступ к объявлениям с модификатором `internal` из исходного набора `main`.

В KOTLIN НЕТ ПРИВАТНЫХ ПАКЕТОВ

В Kotlin нет концепции видимости внутри пакета (`package-private`), а в Java она используется по умолчанию. В Kotlin пакеты используются только как способ организации кода в пространствах имен — их не применяют для контроля области видимости.

Преимущество области видимости `internal` заключается в том, что она делает возможной реальную инкапсуляцию для деталей реализации модуля. В Java инкапсуляцию легко нарушить, поскольку сторонний код может определять классы в тех же пакетах, которые используются вашим кодом. Таким образом, этот код получит доступ к вашим закрытым объявлениям внутри пакета.

Кроме того, в Kotlin можно использовать область видимости `private` для объявлений верхнего уровня, в том числе классов, функций и свойств. Такие объявления будут видны только в том файле, в котором были объявлены. Это еще один полезный способ скрыть детали реализации подсистемы. В табл. 4.2 приведены все модификаторы видимости.

Таблица 4.2. Модификаторы видимости Kotlin

Модификатор	Член класса	Объявление верхнего уровня
<code>public</code> (по умолчанию)	Видимость везде	Видимость везде
<code>internal</code>	Видимость в модуле	Видимость в модуле
<code>protected</code>	Видимость в подклассе	—
<code>private</code>	Видимость в классе	Видимость в файле

Рассмотрим пример (листинг 4.12). Каждая строка в функции `giveSpeech` пытается нарушить правила видимости. Она компилируется с ошибкой.

Листинг 4.12. Использование различных модификаторов видимости

```
internal open class TalkativeButton {
    private fun yell() = println("Hey!")
    protected fun whisper() = println("Let's talk!")
}

fun TalkativeButton.giveSpeech() {
    yell()
    whisper()
}
```

Ошибка: член типа `public` предоставляет свой внутренний тип приемника класса `TalkativeButton`

Ошибка: невозможно получить доступ к функции `yell` — ее тип `private` в классе `TalkativeButton`

Ошибка: невозможно получить доступ к функции `whisper` — ее тип `protected` в классе `TalkativeButton`

Kotlin не дает ссылаться на менее видимый тип `TalkativeButton` (в данном случае `internal`) из функции `giveSpeech` типа `public`. Это частный случай более общего правила: видимость всех типов, используемых в списке базовых типов и параметров типа класса или сигнатуре метода, должна совпадать с видимостью класса или метода. Это правило означает, что у вас всегда есть доступ ко всем типам, которые могут понадобиться для вызова функции или расширения класса. Чтобы решить эту проблему, можно указать функции-расширению `giveSpeech` модификатор `internal` либо классу `TalkativeButton` модификатор `public`.

Обратите внимание на разницу в поведении модификатора `protected` в Java и в Kotlin. В Java можно получить доступ к члену с модификатором `protected` из того же пакета, но Kotlin этого не позволяет.

В Kotlin правила видимости просты, и член типа `protected` виден только в классе и его подклассах. К тому же стоит отметить, что функции-расширения класса не получают доступа к его членам типа `private` или `protected`. Именно поэтому нельзя вызвать функцию `protectedwhisper` из функции-расширения `giveSpeech`.

МОДИФИКАТОРЫ ВИДИМОСТИ KOTLIN И JAVA

В Kotlin модификаторы `public`, `protected` и `private` сохраняются при компиляции в байт-код Java. Эти объявления в Kotlin используются из кода Java, как если бы они были объявлены с той же видимостью в Java. Единственным исключением служит класс `private`: он компилируется в объявление видимости внутри пакета (`package-private`), поскольку в Java нельзя сделать класс типа `private`.

У вас может возникнуть вопрос, что происходит с модификатором `internal`? Прямого аналога в Java для него не существует. Видимость внутри пакета — совсем другое дело. Модуль обычно состоит из нескольких пакетов, и разные модули могут содержать объявления из одного и того же пакета. Таким образом, модификатор `internal` становится модификатором `public` в байт-коде.

Это соответствие между объявлениями Kotlin и их аналогами на Java (или их представлением в байт-коде) объясняет, почему иногда можно получить доступ к чему-то из кода Java, а из Kotlin это не работает. Например, можно обратиться к классу `internal` или объявлению верхнего уровня из кода Java в другом модуле или к члену типа `protected` из кода Java в том же пакете.

Но обратите внимание, что имена членов класса типа `internal` искажаются. Технически члены типа `internal` можно использовать из Java, но в коде на данном языке они выглядят некрасиво. Это помогает избежать неожиданных нестыковок в переопределениях при расширении класса из другого модуля, а также предотвращает случайное использование классов `internal`.

Еще одно различие в правилах видимости между Kotlin и Java заключается в том, что в Kotlin внешний класс не видит члены `private` своих внутренних (или вложенных) классов. Обсудим внутренние и вложенные классы в Kotlin и рассмотрим пример.

4.1.4. Внутренние и вложенные классы. По умолчанию вложенные

Если требуется инкапсулировать класс-помощник или хранить код рядом с местом его использования, то можно объявить класс внутри другого класса. Однако в отличие от Java вложенные классы в Kotlin не имеют доступа к экземпляру внешнего класса, если не сделать для этого специальный запрос. И почему это важно, покажем на примере.

Представьте, что требуется определить элемент **View**, состояние которого может быть сериализовано. Осуществление сериализации представления может оказаться сложной задачей. Но допускается скопировать все необходимые данные в другой класс-помощник. Для этого необходимо объявить интерфейс **State** для реализации состояния **Serializable**. В интерфейсе **View** объявляются методы **getCurrentState** и **restoreState** для сохранения состояния представления (листинг 4.13).

Листинг 4.13. Объявление представления с сериализуемым состоянием

```
interface State : Serializable

interface View {
    fun getCurrentState(): State
    fun restoreState(state: State) { /* ... */ }
}
```

Будет удобно определить класс для сохранения состояния кнопки в классе **Button**. Сначала посмотрим, как это можно сделать на Java (листинг 4.14) (аналогичный код на Kotlin будет приведен позже).

Листинг 4.14. Реализация интерфейса **View** в Java с помощью внутреннего класса

```
/* Java */
public class Button implements View {
    @Override
    public State getCurrentState() {
        return new ButtonState();
    }

    @Override
    public void restoreState(State state) { /*...*/ }

    public class ButtonState implements State { /*...*/ }
}
```

В этом фрагменте определен класс **ButtonState**. Он реализует интерфейс **State** и хранит специфическую информацию для класса **Button**. Новый экземпляр класса **ButtonState** создается в методе **getCurrentState**. В реальном случае вы бы осуществили инициализацию класса, используя все необходимые данные.

Что не так с этим кодом? Почему при попытке сериализовать состояние объявленной кнопки возникает исключение `java.io.NotSerializableException: Button`? Сначала

это может показаться странным, ведь тип переменной `state` для сериализации — `ButtonState`, а не `Button`.

Все становится понятным, если вспомнить, что в Java класс, объявленный в другом классе, по умолчанию становится внутренним. Класс `ButtonState` в примере скрыто хранит ссылку на свой внешний класс `Button`. Это объясняет, почему переменную типа `ButtonState` нельзя сериализовать: `Button` не сериализуется, и ссылка на него нарушает сериализацию `ButtonState`.

Для решения проблемы необходимо объявить класс `ButtonState` с модификатором `static`. Такое объявление вложенного класса удаляет скрытую ссылку из этого класса на его замкнутый класс.

В Kotlin поведение внутренних классов по умолчанию противоположно тому, что мы только что описали. И убедиться в этом можно, посмотрев листинг 4.15.

Листинг 4.15. Реализация View в Kotlin с помощью вложенного класса

```
class Button : View {
    override fun getCurrentState(): State = ButtonState()

    override fun restoreState(state: State) { /*...*/ }

    class ButtonState : State { /*...*/ } ← Это аналог вложенного static-класса в Java
}
```

Вложенный класс в Kotlin без явных модификаторов — то же самое, что вложенный `static`-класс в Java. Модификатор `inner` превратит его во внутренний класс со ссылкой на внешний. В табл. 4.3 описаны различия в этом поведении между Java и Kotlin, а разница между вложенными и внутренними классами показана на рис. 4.1.

Таблица 4.3. Соответствие между вложенными и внутренними классами в Java и Kotlin

Класс A объявлен внутри другого класса B	В Java	В Kotlin
Вложенный класс (не хранит ссылку на внешний)	<code>static class A</code>	<code>class A</code>
Внутренний класс (хранит ссылку на внешний)	<code>class A</code>	<code>inner class A</code>

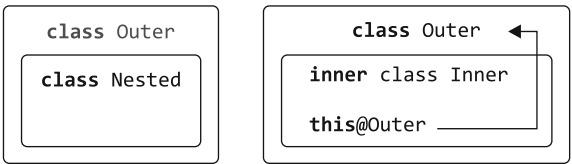


Рис. 4.1. Внутренние классы ссылаются на свой внешний класс, а вложенные — нет

Синтаксис для ссылки на экземпляр внешнего класса в Kotlin тоже отличается от Java.

Для доступа из класса `Inner` к классу `Outer` применяется аннотация `this@Outer`:

```
class Outer {
    inner class Inner {
        fun getOuterReference(): Outer = this@Outer
    }
}
```

Вы узнали, чем отличаются внутренние и вложенные классы в Java и в Kotlin. Далее мы обсудим, как создать иерархию, содержащую ограниченное количество классов.

4.1.5. Запечатанные классы: определение ограниченных иерархий классов

Иерархия классов Kotlin уже была представлена немного ранее. Вспомните пример иерархии выражений, приведенный в подразделе 2.3.6: мы применяли ее для кодирования выражений типа $(1 + 2) + 4$. Интерфейс `Expr` реализован в классах `Num` для представления числа и `Sum` для суммы двух выражений.

Обрабатывать все возможные случаи удобно в выражении `when`. Но необходимо указать ветвь `else`, чтобы определить, что произойдет, если ни одна из других ветвей не будет подходить (листинг 4.16).

Листинг 4.16. Выражения как реализации интерфейсов

```
interface Expr
class Num(val value: Int) : Expr
class Sum(val left: Expr, val right: Expr) : Expr

fun eval(e: Expr): Int =
    when (e) {
        is Num -> e.value
        is Sum -> eval(e.right) + eval(e.left)
        else -> ← Необходимо проверить ветвь else
                  throw IllegalArgumentException("Unknown expression")
    }
```

Конструкция `when` выступает в роли выражения (то есть используется ее возвращаемое значение), поэтому должна быть исчерпывающей — компилятор Kotlin обязывает проверить вариант по умолчанию. В этом примере нельзя вернуть осмысленный результат, поэтому выбрасывается исключение.

Добавлять ветку по умолчанию каждый раз не очень удобно. А еще это может стать проблемой, если вы (или ваш коллега, или автор используемой вами зависимости) добавите новый подкласс и компилятор не предупредит о том, что был упущен какой-то случай. Если забыть добавить ветку для работы с новым подклассом, то будет выбрана ветка по умолчанию. А это приведет к мелким ошибкам.

Kotlin предлагает решение этой проблемы — *запечатанные классы* (*sealed*). Суперкласс помечается модификатором `sealed`, и создание подклассов будет ограничено. Все «прямые» подклассы запечатанного класса должны быть известны во время

компиляции и объявлены в том же пакете, что и сам запечатанный класс, а все подклассы должны находиться в том же модуле.

Вместо использования интерфейса можно объявить `Expr` как `sealed class`, а `Num` и `Sum` — его подклассами (листинг 4.17).

Листинг 4.17. Выражения как запечатанные классы

```
sealed class Expr  ← Базовый класс помечен как запечатанный
class Num(val value: Int) : Expr()
class Sum(val left: Expr, val right: Expr) : Expr()  ← Перечисление всех
                                                    возможных подклассов

fun eval(e: Expr): Int =
    when (e) {
        is Num -> e.value
        is Sum -> eval(e.right) + eval(e.left)
    }  ← Выражение when охватывает все возможные
      случаи, и ветвь else не требуется
```

Если все подклассы запечатанного класса обрабатываются в выражении `when`, то необходимости в ветви по умолчанию нет: компилятор может убедиться, что охвачены все возможные ветви. Обратите внимание: модификатор `sealed` подразумевает абстрактность класса — явный модификатор `abstract` не требуется и можно объявлять абстрактные члены класса. Поведение запечатанных классов показано на рис. 4.2.

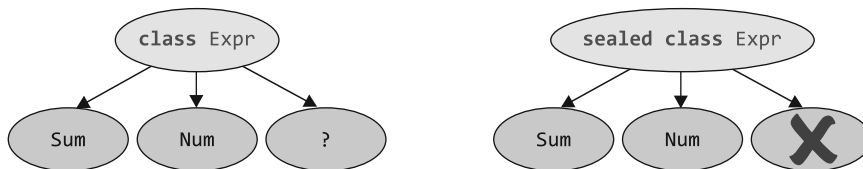


Рис. 4.2. Все прямые подклассы запечатанного класса должны быть известны во время компиляции

Если `when` используется с запечатанными классами и добавляется новый подкласс, то выражение `when` с возвращаемым значением не компилируется (листинг 4.18). Это указывает на требующий изменений код, например, когда вы определяете оператор умножения `Mul`, но не обрабатываете его в функции `eval`.

Листинг 4.18. Добавление нового класса в запечатанную иерархию

```
sealed class Expr
class Num(val value: Int) : Expr()
class Sum(val left: Expr, val right: Expr) : Expr()
class Mul(val left: Expr, val right: Expr): Expr()

fun eval(e: Expr): Int =
    when (e) {
        is Num -> e.value
        is Sum -> eval(e.right) + eval(e.left)
        // ERROR: 'when' expression must be exhaustive,
        // add necessary 'is Mul' branch or 'else' branch instead
    }
}
```

Помимо классов, модификатор `sealed` применяется для определения `sealed interface`. Запечатанные интерфейсы подчиняются тем же правилам: как только модуль, содержащий запечатанный интерфейс, скомпилирован, никаких новых реализаций для него быть не может:

```
sealed interface Toggleable {
    fun toggle()
}

class LightSwitch: Toggleable {
    override fun toggle() = println("Lights!")
}

class Camera: Toggleable {
    override fun toggle() = println("Camera!")
}
```

Запечатанные интерфейсы определяют функции и свойства...

...а классы предоставляют их реализацию

Обработка всех реализаций `sealed interface` в инструкции `when` тоже означает, что необходимости в ветви `else` нет.

В Kotlin двоеточие используется для расширения класса и реализации интерфейса:

```
class Num(val value: Int) : Expr()
class LightSwitch: Toggleable
```

Num — это подкласс

Класс LightSwitch реализует интерфейс

В следующем разделе об инициализации классов Kotlin мы обсудим наличие круглых скобок после имени класса `Expr()`.

4.2. ОБЪЯВЛЕНИЕ КЛАССА С НЕТРИВИАЛЬНЫМИ КОНСТРУКТОРАМИ ИЛИ СВОЙСТВАМИ

В объектно-ориентированных языках в классах обычно есть несколько конструкторов. В Kotlin тоже, но в нем различаются *первичный* конструктор (его чаще всего используют для инициализации класса, и он объявляется вне тела класса) и *вторичный* (объявляется в теле класса). К тому же в Kotlin есть возможность поместить дополнительную логику инициализации в *блоки инициализаторов*. Сначала мы рассмотрим синтаксис объявления первичного конструктора и блоков инициализаторов, а затем объясним, как объявить несколько конструкторов. После этого мы подробнее обсудим свойства.

4.2.1. Инициализация классов: первичные конструкторы и блоки инициализатора

В разделе 2.2 приводился пример объявления простого класса:

```
class User(val nickname: String)
```

Обычно все объявления в классе заключаются в фигурные скобки, но этот класс отличается от других. В нем нет фигурных скобок, а есть только объявление в круглых скобках. Этот блок кода в круглых скобках называется *первичным конструктором*.

Он служит двум целям: задает параметры конструктора и определяет свойства для инициализации этими параметрами. Разберемся в этом механизме и рассмотрим наиболее простой вариант кода для повторения этой логики:

```
class User constructor(_nickname: String) {  ← Первичный конструктор с одним параметром
    val nickname: String

    init {  ← Блок инициализатора
        nickname = _nickname
    }
}
```

В этом примере содержатся два новых ключевых слова: **constructor** начинает объявление первичного или вторичного конструктора, а **init** вводит *блок инициализатора*. Такие блоки применяются с первичными конструкторами и содержат код инициализации. Он выполняется при создании класса. Поскольку синтаксис первичного конструктора ограничен, то не может содержать кода инициализации: именно поэтому были созданы блоки инициализаторов. В случае необходимости можно объявить несколько блоков инициализаторов в одном классе.

Нижнее подчеркивание в параметре конструктора `_nickname` позволяет отличить имя свойства от имени параметра конструктора. Альтернативный вариант — использовать то же имя и указать ключевое слово **this**, чтобы устранить двусмысленность:

```
this.nickname = nickname
```

В этом примере не нужно размещать код инициализации в блоке инициализатора, поскольку его можно объединить с объявлением свойства `nickname`. Вдобавок можно опустить ключевое слово **constructor**, если для первичного конструктора не требуется аннотаций или модификаторов видимости. Внесем эти изменения и получим следующий код:

```
class User(_nickname: String) {  ← Первичный конструктор с одним параметром
    val nickname = _nickname  ← Свойство инициализируется с параметром
}
```

Это еще один способ объявить тот же класс. Обратите внимание на способ создания ссылок на параметры первичного конструктора в инициализаторах свойств и блоках инициализаторов.

В двух предыдущих примерах свойство было объявлено с помощью ключевого слова **val** в теле класса. Если свойство инициализируется соответствующим параметром конструктора, то код можно упростить, добавив перед параметром ключевое слово **val**. Такая форма заменяет определение свойства в теле класса:

```
class User(val nickname: String)  ← val означает, что для параметра конструктора
                                  генерируется соответствующее свойство
```

Результат всех этих объявлений класса **User** неизменен, но в последнем случае используется наиболее лаконичный синтаксис.

Значения по умолчанию для параметров конструктора можно объявлять так же, как и для параметров функции:

```
class User(
    val nickname: String,
    val isSubscribed: Boolean = true
)
```

Предоставление значения по умолчанию для параметра конструктора

Чтобы создать экземпляр класса, конструктор нужно вызывать напрямую, без дополнительных ключевых слов, например `new`. Создадим ситуацию с потенциальными пользователями: Alice подписалась на рассылку по умолчанию, Bob и Carol внимательно прочитали условия и отменили выбор по умолчанию, а Dave заинтересован в получении информации от отдела маркетинга:

```
fun main() {
    val alice = User("Alice")
    println(alice.isSubscribed)
    // истина
    val bob = User("Bob", false)
    println(bob.isSubscribed)
    // ложь
    val carol = User("Carol", isSubscribed = false)
    println(carol.isSubscribed)
    // ложь
    val dave = User(nickname = "Dave", isSubscribed = true)
    println(dave.isSubscribed)
    // истина
}
```

Используется значение по умолчанию true для параметра isSubscribed

Можно указать все параметры согласно порядку объявления

Можно указать имена для некоторых аргументов конструктора

Можно указать имена для всех аргументов конструктора

ПРИМЕЧАНИЕ

Если для всех параметров конструктора есть значения по умолчанию, то компилятор генерирует дополнительный конструктор без параметров. Он будет использовать все значения по умолчанию. Это облегчает использование Kotlin с библиотеками, которые создают экземпляры классов, используя конструкторы без параметров. Если в коде Java требуется вызвать конструктор Kotlin с определенными параметрами по умолчанию, то его следует пометить как `@JvmOverloads constructor`. Эта директива дает компилятору указание сгенерировать соответствующие перегрузки для использования из Java. Мы обсуждали это в подразделе 3.2.2.

Если конструктор суперкласса принимает аргументы, то первичный конструктор вашего класса должен их инициализировать. Для этого нужно указать параметры конструктора суперкласса после ссылки на суперкласс в списке базовых классов:

```
open class User(val nickname: String) { /* ... */ }

class SocialUser(nickname: String) : User(nickname) { /* ... */ }
```

Если не объявить конструкторы для класса, то будет сгенерирован конструктор по умолчанию без параметров и выполняемой логики:

```
open class Button
```

← Сгенерирован конструктор по умолчанию без аргументов

Если новый класс наследует класс `Button` без указания конструкторов, то необходимо явно вызвать конструктор суперкласса, даже если в нем не будет никаких параметров. Именно для этого нужны пустые круглые скобки после имени суперкласса:

```
class RadioButton: Button()
```

Обратите внимание на разницу с интерфейсами: у интерфейса нет конструкторов, поэтому при его инициализации скобки после его имени в списке супертипов не ставятся.

Если экземпляр класса не должен быть создан кодом за пределами самого класса, то необходимо указать модификатор `private` для конструктора. Сделать это можно следующим образом:

```
class Secret private constructor(private val agentName: String) {}
```

←
В классе будет `private`-конструктор

В классе `Secret` есть только конструктор `private`, поэтому код вне класса не может создать его экземпляр. Позже, в подразделе 4.4.2, мы обсудим объекты-компаньоны — в них удобно вызывать такие конструкторы.

АЛЬТЕРНАТИВЫ КОНСТРУКТОРАМ ТИПА `PRIVATE`

В Java конструктор `private` запрещает создание экземпляра класса, чтобы выразить более общую идею: класс служит контейнером статических служебных членов или представляет собой объект-одиночку. Для этих задач в Kotlin есть встроенные языковые возможности. В качестве статических служебных объектов используются функции верхнего уровня (были показаны в подразделе 3.2.3). Для обозначения объектов-одиночек используются объявления объектов. Этот способ был описан в подразделе 4.4.1.

В большинстве случаев конструктор класса прост — он не содержит параметров или присваивает параметры соответствующим свойствам. Именно поэтому в Kotlin есть лаконичный синтаксис для первичных конструкторов, и он отлично работает в большинстве случаев. Но перед разработчиками ставят разные задачи, а не только самые простые, поэтому Kotlin позволяет определить необходимое классу количество конструкторов. Посмотрим, как это работает.

4.2.2. Вторичные конструкторы: инициализация суперкласса различными способами

В общем случае классы с несколькими конструкторами встречаются в коде Kotlin гораздо реже, чем в Java. Разрешить большинство ситуаций, в которых понадобятся перегруженные конструкторы, можно благодаря поддержке в Kotlin значений параметров по умолчанию и синтаксиса именованных аргументов.

СОВЕТ

Не объявляйте несколько вторичных конструкторов для перегрузки и предоставления значений по умолчанию аргументов. Просто укажите значения по умолчанию напрямую.

Но все же иногда возникают ситуации, в которых требуется несколько конструкторов. Чаще всего это происходит, когда требуется расширить класс фреймворка с несколькими конструкторами, инициализирующими класс разными способами. Для примера рассмотрим класс `Downloader`. Он был объявлен на Java и содержит два конструктора: один принимает в качестве параметра значение типа `String`, а другой — `URI`:

```
import java.net.URI;

public class Downloader {
    public Downloader(String url) {
        // код
    }

    public Downloader(URI uri) {
        // код
    }
}
```

В Kotlin это объявление будет выглядеть следующим образом:

```
open class Downloader {
    constructor(url: String?) { ← Вторичные конструкторы
        // код
    }

    constructor(uri: URI?) {
        // код
    }
}
```

В этом классе не объявлен первичный конструктор (об этом можно судить по отсутствию круглых скобок после имени класса в заголовке класса), но объявлены два вторичных. Они обозначаются ключевым словом `constructor`. Можно объявить любое необходимое количество вторичных конструкторов.

Если потребуется расширить этот класс, можно объявить те же конструкторы:

```
class MyDownloader : Downloader {
    constructor(url: String?) : super(url) { ←
        // ...                               Вызов
    }                                         конструкторов
                                           суперкласса
    constructor(uri: URI?) : super(uri) { ←
        // ...
    }
}
```

Здесь определены два конструктора, и каждый из них вызывает соответствующий конструктор суперкласса с помощью ключевого слова `super()`. Эта схема представлена на рис. 4.3. Стрелка показывает, какой конструктор делегирован.

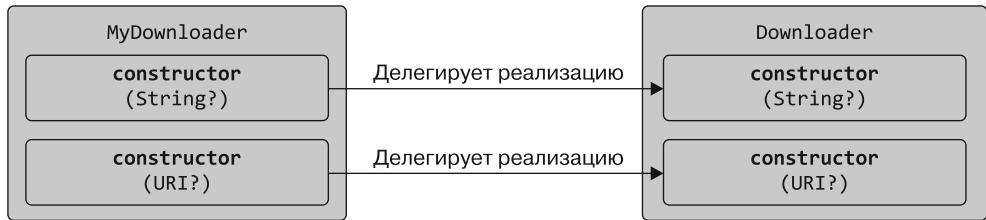


Рис. 4.3. Использование различных конструкторов суперклассов

Как и в Java, здесь есть возможность вызвать другой конструктор собственного класса из конструктора с помощью ключевого слова `this()`. Это работает следующим образом:

```

class MyDownloader : Downloader {
    constructor(url: String?) : this(URI(url))
    constructor(uri: URI?) : super(uri)
}
  
```

← Делегирует реализацию другому конструктору класса

Класс `MyDownloader` изменяется так, чтобы при создании объекта `URI` из строки `url` один из конструкторов делегировал реализацию другому конструктору того же класса (с помощью `this`) (рис. 4.4). Второй конструктор продолжает вызывать метод `super()`.

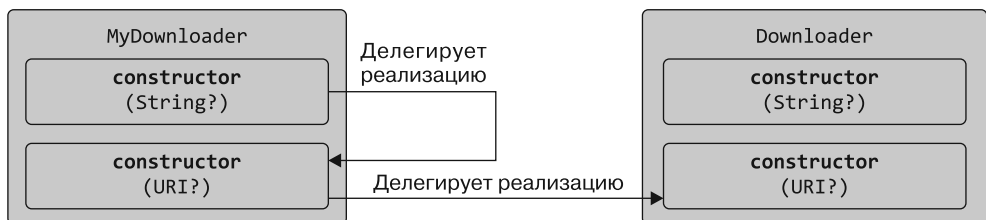


Рис. 4.4. Делегирование реализации конструктору того же класса

Если у класса нет первичного конструктора, то каждый вторичный конструктор должен инициализировать базовый класс или делегировать эту задачу другому конструктору. Выражаясь в терминах предыдущих рисунков, у каждого вторичного конструктора должна быть исходящая стрелка для обозначения пути к любому конструктору базового класса.

Чаще всего вторичные конструкторы используются для обеспечения совместимости с Java. Но возможен и другой случай — когда существует несколько способов создания экземпляров класса с разными списками параметров. Пример мы рассмотрим в подразделе 4.4.2.

Обсудив объявления нетривиальных конструкторов, перейдем к нетривиальным свойствам.

4.2.3. Реализация свойств, объявленных в интерфейсах

В Kotlin интерфейс может содержать объявления абстрактных свойств. Приведем пример определения интерфейса с таким объявлением:

```
interface User {
    val nickname: String
}
```

Это означает, что классы с реализацией интерфейса `User` должны предоставить способ получения значения `nickname`. В интерфейсе не указано, должно ли значение храниться в теновом поле или его необходимо получить с помощью геттера. Поэтому сам интерфейс не содержит никакого состояния. При необходимости классы, реализующие этот интерфейс, могут хранить значение. Или просто будут вычислять его при обращении, как было показано в подразделе 2.2.2.

Рассмотрим несколько возможных реализаций этого интерфейса: `PrivateUser` — те, кто будет заполнять только псевдоним; `SubscribingUser` для пользователей, обязанных предоставить адрес электронной почты; `SocialUser` — те, кто опровергметчиво поделился идентификатором своей учетной записи в социальной сети. Все эти классы по-разному реализуют абстрактное свойство в интерфейсе (листинг 4.19).

Листинг 4.19. Реализация свойства интерфейса

```
class PrivateUser(override val nickname: String) : User ← Свойство первичного конструктора

class SubscribingUser(val email: String) : User {
    override val nickname: String
        get() = email.substringBefore('@') ← Пользовательский геттер
}

class SocialUser(val accountId: Int) : User {
    override val nickname = getNameFromSocialNetwork(accountId) ← Инициализатор свойства
}

fun getNameFromSocialNetwork(accountId: Int) =
    "kodee$accountId"

fun main() {
    println(PrivateUser("kodee").nickname)
    // kodee
    println(SubscribingUser("test@kotlinlang.org").nickname)
    // test
    println(SocialUser(123).nickname)
    // kodee123
}
```

При создании класса `PrivateUser` применяется лаконичный синтаксис для объявления свойства непосредственно в первичном конструкторе. Оно реализует абстрактное свойство интерфейса `User`, поэтому помечается модификатором `override`.

Свойство `nickname` класса `SubscribingUser` реализовано с помощью пользовательского геттера. У этого свойства нет базового поля для хранения своего значения — есть только геттер, который при каждом обращении вычисляет псевдоним из электронной почты.

Значение свойству `nickname` класса `SocialUser` присваивается в его инициализаторе. Вдобавок здесь используется функция `getNameFromSocialNetwork`. Она должна вернуть имя пользователя социальной сети на основе идентификатора его учетной записи. На практике эта функция была бы затратной: чтобы получить нужные данные, необходимо установить соединение с провайдером социальной сети. Именно поэтому решено вызывать ее один раз на этапе инициализации.

Обратите внимание на разные реализации свойства `nickname` в классах `SubscribingUser` и `SocialUser`. Они выглядят похоже, но у свойства класса `SubscribingUser` указан пользовательский геттер. Он вычисляет функцию `substringBefore` при каждом обращении. А у свойства класса `SocialUser` есть теневое поле. В нем хранятся данные, вычисленные при инициализации класса.

КОГДА НУЖНО ВЫБИРАТЬ СВОЙСТВА ВМЕСТО ФУНКЦИЙ

В подразделе 2.2.2 мы вкратце рассказали о том, когда следует объявлять функцию без параметров или доступное только для чтения свойство с пользовательским геттером. Мы установили, что *характеристики* класса обычно объявляются как свойства, а *поведение* — как методы. В Kotlin есть несколько дополнительных стилистических соглашений для использования доступных только для чтения свойств вместо функций. Если код обладает одним из следующих качеств, то следует выбрать свойство:

- не выбрасывает исключения;
- вычисляется с небольшими затратами (или кэшируется при первом запуске);
- возвращает один и тот же результат при нескольких обращениях, если состояние объекта не изменилось.

В противном случае создавайте функцию.

В дополнение к объявлениям абстрактных свойств интерфейс может содержать свойства с геттерами и сеттерами, если они не ссылаются на теневое поле. (Теневое поле потребовало бы хранить состояние в интерфейсе, что недопустимо.) Рассмотрим пример:

```
interface EmailUser {
    val email: String
    val nickname: String
        get() = email.substringBefore('@')
}
```

У свойства нет теневого поля —
итоговое значение вычисляется
при каждом доступе

Этот интерфейс содержит абстрактное свойство `email`, а также свойство `nickname` с пользовательским геттером. Первое должно быть переопределено в подклассах, а второе может быть унаследовано (или переопределено, если возникнет такая необходимость).

У свойств, реализованных в интерфейсах, нет доступа к теневым полям, тогда как у свойств в классах он есть. Рассмотрим, как можно обращаться к ним из методов доступа.

4.2.4. Доступ к теневому полю из геттера или сеттера

Мы привели несколько примеров двух видов свойств: со значением и с пользовательскими методами доступа. Вторые вычисляют значения при каждом обращении. Теперь поговорим о том, как объединить эти два способа и реализовать свойство, которое хранит значение и предоставляет дополнительную логику. Она будет выполняться при обращении к значению или его изменении. Для этого необходимо получить доступ к базовому полю свойства из его методов доступа.

Допустим, вы создаете систему управления контактами. Она будет отслеживать объекты `User`, а также их свойства `name` и `address`. Программа должна регистрировать любое изменение данных, хранящихся в свойстве, например, `address`. Для этого свойство объявляется как изменяемое и требуется выполнение дополнительного фрагмента кода при каждом обращении к сеттеру (листинг 4.20).

Листинг 4.20. Доступ к теневому полю в сеттере

```
class User(val name: String) {
    var address: String = "unspecified"
    set(value: String) {
        println(
            """
            Address was changed for $name:
            "$field" -> "$value".  ← (Читывание значения теневого поля
            """).trimIndent()
        )
        field = value  ← Изменение значения теневого поля
                       на значение предоставленной строки
    }
}

fun main() {
    val user = User("Alice")
    user.address = "Christoph-Rapparini-Bogen 23"
    // Address was changed for Alice:
    // "unspecified" -> "Christoph-Rapparini-Bogen 23".
}
```

Значение свойства изменяется как обычно: `user.address = "new value"`, что скрытно вызывает его сеттер. В этом примере сеттер переопределен, поэтому выполняется дополнительный код журналирования (в целях упрощения в данном случае его результаты не выводятся на экран).

Для доступа к значению теневого поля в теле сеттера применяется специальный идентификатор `field`. В геттере можно только читать значение, а в сеттере — и читать, и изменять его.

Обратите внимание: если геттер или сеттер для изменяемого свойства тривиальны, то стоит определить только метод доступа, требующий особого поведения. Вторым переопределять не нужно. Например, геттер в листинге 4.20 тривиален и возвращает только значение поля, поэтому переопределять его не нужно.

Вы можете задаться вопросом, в чем разница между созданием свойства с тенью и без него. Способ доступа к свойству не зависит от наличия у него теневого поля: компилятор создаст тень для свойства, если есть явная ссылка на него или если используется реализация метода доступа по умолчанию. Если же создается пользовательский метод доступа без идентификатора `field` (для геттера, если свойство помечено как `val`, и для обоих методов доступа, если свойство изменяемое), то компилятор поймет, что свойству не нужно хранить какую-либо информацию, и тень не создается. В листинге 4.21 это показано с помощью свойства `ageIn2050`. Оно определяется исключительно свойством `birthYear` класса `Person`.

Листинг 4.21. Свойство, которое само не хранит информацию

```
class Person(var birthYear: Int) {
    var ageIn2050
        get() = 2050 - birthYear  ← В геттере нет ссылки на поле...
        set(value) {
            birthYear = 2050 - value ← ...или сеттер означает, что тень
        }                             поле не сгенерировано
}
```

Иногда не нужно менять стандартную реализацию метода доступа, но требуется изменить его видимость. Рассмотрим, как это сделать.

4.2.5. Изменение видимости метода доступа

По умолчанию видимость метода доступа согласована со свойством. Однако при необходимости это можно изменить. Для этого необходимо указать модификатор видимости перед ключевым словом `get` или `set`. Чтобы понять, как это использовать, рассмотрим пример: небольшой класс `LengthCounter` будет отслеживать общую длину добавленных в него слов (листинг 4.22).

Листинг 4.22. Объявление свойства с приватным сеттером

```
class LengthCounter {
    var counter: Int = 0
        private set  ← Нельзя изменить это свойство
                     из-за пределов класса

    fun addWord(word: String) {
        counter += word.length
    }
}
```

Свойство `counter` содержит общую длину и помечено как `public`, поскольку относится к API класса для клиентов. Но необходимо гарантировать, что оно будет изменяться только в классе, так как иначе внешний код может изменить его и сохранить неверное значение. Поэтому компилятор генерирует геттер с видимостью по умолчанию, а видимость сеттера изменяется на `private`.

Использовать этот класс можно следующим образом:

```
fun main() {
    val lengthCounter = LengthCounter()
    lengthCounter.addWord("Hi!")
    println(lengthCounter.counter)
    // 3
}
```

Здесь создается экземпляр класса `LengthCounter`, а после добавляется слово `"Hi!"` длиной 3 символа. Теперь свойство `counter` хранит значение 3. И естественно, попытка записать в свойство извне класса приводит к ошибке компиляции:

```
fun main() {
    // ...
    lengthCounter.counter = 0
    // Error: Cannot assign to 'counter': the setter is private in 'LengthCounter'
}
```

ПОДРОБНЕЕ О СВОЙСТВАХ

Мы продолжим обсуждение свойств чуть позже. Приведем несколько ссылок.

- Модификатор `lateinit` на ненулевом свойстве указывает, что оно инициализируется после вызова конструктора. Это распространенный случай в некоторых фреймворках. Данная функциональная возможность будет описана в разделе 7.9.
- Отложенная инициализация свойств как часть более общей возможности *делегированных свойств* описана в разделе 9.5.
- Для совместимости с Java-фреймворками можно использовать аннотации эмуляции функций Java в Kotlin. Например, аннотация `@JvmField` свойства предоставляет поле `public` без методов доступа. Подробнее об аннотациях поговорим в главе 12.
- Модификатор `const` облегчает работу с аннотациями. К тому же он позволяет использовать в качестве аргумента аннотации свойство примитивного типа или типа `String`. Подробности приведены в разделе 12.1.1.

На этом мы завершаем обсуждение написания нетривиальных конструкторов и свойств в Kotlin. Далее вы узнаете, как создать классы `data`, позволяющие хранить несколько фрагментов информации.

4.3. МЕТОДЫ, ГЕНЕРИРУЕМЫЕ КОМПИЛЯТОРОМ. КЛАССЫ ДАННЫХ И ДЕЛЕГИРОВАНИЕ КЛАССОВ

Платформа Java определяет несколько методов классов. Они реализуются механически, например `equals` (указывает, равны ли два объекта друг другу), `hashCode` (предоставляет хеш-код для объекта, как того требуют структуры данных вроде хеш-карт) и `toString` (возвращает текстовое представление объекта).

К счастью, IDE могут автоматизировать создание таких методов и не придется писать их вручную. Но в этом случае кодовая база все равно будет содержать шаблонный код, и его понадобится сопровождать. В компиляторе Kotlin еще больше возможностей: он может выполнять механическую генерацию кода в фоновом режиме, не загромождая файлы исходного кода результатами.

Мы уже показали, как эта схема работает для тривиальных конструкторов классов и методов доступа свойств. Теперь рассмотрим больше примеров, когда компилятор Kotlin генерирует типичные методы для простых классов данных. Это значительно упрощает паттерн делегирования классов.

4.3.1. Универсальные методы объектов

Во всех классах Kotlin есть методы, которые может потребоваться переопределить, как и в Java: `toString`, `equals` и `hashCode`. Рассмотрим, что это за методы и как Kotlin может помочь вам генерировать их реализацию автоматически. В качестве отправной точки используем простой класс `Customer` (листинг 4.23). Он хранит имя и почтовый индекс клиента.

Листинг 4.23. Начальное объявление класса `Customer`

```
class Customer(val name: String, val postalCode: Int)
```

Рассмотрим, как экземпляры классов представляются в виде строк.

Строковое представление: `toString()`

Все классы в Kotlin и в Java дают возможность получить строковое представление объектов класса. В основном эта функциональность используется для отладки и журналирования, хотя ее можно применять и в других контекстах. По умолчанию строковое представление объекта выглядит как `Customer@5e9f23b4` (имя класса и адрес памяти для хранения объекта), что на практике не очень удобно. Чтобы это изменить, необходимо переопределить метод `toString` (листинг 4.24).

Листинг 4.24. Реализация метода `toString()` для класса `Customer`

```
class Customer(val name: String, val postalCode: Int) {  
    override fun toString() = "Customer(name=$name, postalCode=$postalCode)"  
}
```


Теперь представление клиента выглядит следующим образом:

```
fun main() {
    val customer1 = Customer("Alice", 342562)
    println(customer1)
    // Customer(name=Alice, postalCode=342562)
}
```

Получилось более информативно, не так ли?

Равенство объектов: equals()

Все вычисления с классом `Customer` происходят за его пределами. Он просто хранит данные, и он должен быть простым и прозрачным. Тем не менее к поведению такого класса могут возникнуть определенные требования. Для примера предположим, что объекты должны считаться одинаковыми, если содержат одинаковые данные:

```
fun main() {
    val customer1 = Customer("Alice", 342562)
    val customer2 = Customer("Alice", 342562)
    println(customer1 == customer2)
    // ложь
}
```

В Kotlin оператор `==` проверяет, равны ли объекты, но не ссылки. Он составлен по принципу метода `equals`

Программа показывает, что объекты не равны. Чтобы решить эту проблему, необходимо переопределить метод `equals` для класса `Customer`.

ОПЕРАТОР == ДЛЯ РАВЕНСТВА

В Java для сравнения примитивных и ссылочных типов можно использовать оператор `==`. В случае применения к примитивным типам в Java он сравнивает значения, тогда как для ссылочных типов сравнивает ссылки. Так, в Java существует популярная практика всегда вызывать метод `equals`, а также хорошо известная проблема забыть это сделать.

В Kotlin оператор `==` представляет собой стандартный способ сравнения двух объектов: сравнивает их значения, вызывая метод `equals` в фоновом режиме. Таким образом, если в классе метод `equals` был переопределен, то можно смело сравнивать его экземпляры с помощью оператора `==`. Для сравнения ссылок можно использовать оператор `===`. Он работает точно так же, как оператор `==` в Java, сравнивая ссылки на объекты. Иными словами, он проверяет, указывают ли две ссылки на один и тот же объект в памяти. Поведение аналогов `==` и `===` с отрицанием — `!=` и `!==` — будет соответствующим.

Рассмотрим измененный класс `Customer` (листинг 4.25). В Kotlin функция `equals` принимает нулевой параметр `other` типа `Any` — суперкласс всех классов в Kotlin. С ним вы познакомитесь более подробно в подразделе 8.1.5.

Листинг 4.25. Реализация метода `equals()` для класса `Customer`

```

class Customer(val name: String, val postalCode: Int) {
    override fun equals(other: Any?): Boolean {
        if (other == null || other !is Customer)
            return false
        return name == other.name &&
            postalCode == other.postalCode
    }
    override fun toString() = "Customer(name=$name, postalCode=$postalCode)"
}

```

Any — аналог `java.lang.Object`: суперкласс всех классов в Kotlin. Тип `Any?`, допускающий значение `null`, означает, что параметр `other` может принимать это значение
 Проверка того, относится ли параметр `other` к классу `Customer`
 Проверка того, равны ли соответствующие свойства

Напомним, что оператор `is` проверяет, имеет ли значение указанный тип, и выполняет умное преобразование `other` к типу `Customer` (это аналог `instanceof` в Java). Оператор `!in` представляет отрицание проверки `in` (мы обсуждали оба оператора в подразделе 2.4.4), а оператор `!is` обозначает отрицание проверки `is`. Такие операторы облегчают чтение кода. В главе 7 мы подробно обсудим типы, допускающие значение `null`, и то, почему условие `other == null || other !is Customer` можно упростить до `other !is Customer`.

В Kotlin модификатор `override` является обязательным, поэтому вы защищены от случайного написания выражения `fun equals(other: Customer)`: оно добавит новый метод, а не переопределит `equals`. После переопределения метода `equals` клиенты с одинаковыми значениями свойств должны быть равны. И действительно, проверка равенства `customer1 == customer2` в предыдущем примере теперь возвращает значение `true`. Но если требуется выполнять более сложные операции с клиентами, такой подход не сработает. Обычный вопрос на собеседовании звучит так: «Что сломалось и в чем проблема?» Можете ответить, что проблема в отсутствии метода `hashCode`. Так и есть, и сейчас мы обсудим, почему это важно.

Хеш-контейнеры: `hashCode()`

Метод `hashCode` обязательно надо переопределить вместе с методом `equals`. В данном пункте текста объясняется, почему необходимо это сделать.

Создайте множество с одним элементом — клиентом по имени Alice. Затем создайте новый экземпляр класса `Customer` с теми же данными и проверьте, содержится ли он во множестве. Предполагается, что проверка вернет значение `true`, поскольку два экземпляра равны, однако на самом деле она возвращает ложное значение — `false`:

```

fun main() {
    val processed = hashSetOf(Customer("Alice", 342562))
    println(processed.contains(Customer("Alice", 342562)))
    // ложь
}

```

Причина в том, что в классе `Customer` отсутствует метод `hashCode`. Нарушается общий контракт `hashCode`: если два объекта равны, у них должен быть одинаковый хеш-код.

Множество `processed` относится к типу `HashSet`. Значения в `HashSet` сравниваются оптимизированным способом: сначала сравниваются их хеш-коды, и только если они равны, сравниваются реальные значения. В предыдущем примере хеш-коды двух разных экземпляров класса `Customer` различаются, поэтому программа определяет, что множество не содержит второй объект, хотя метод `equals` возвращает значение `true`. При несоблюдении этого правила структуры данных типа `HashSet` не смогут корректно работать с такими объектами. Но это можно исправить. Необходимо предоставить классу соответствующую реализацию метода `hashCode` (листинг 4.26).

Листинг 4.26. Реализация метода `hashCode()` для класса `Customer`

```
class Customer(val name: String, val postalCode: Int) {
    /* ... */
    override fun hashCode(): Int = name.hashCode() * 31 + postalCode
}
```

Теперь мы получили класс с ожидаемым во всех сценариях поведением. Но обратите внимание, как много кода пришлось написать. К счастью, компилятор Kotlin поможет в этой ситуации — сгенерирует все эти методы автоматически. Рассмотрим, как дать ему указание сделать это.

4.3.2. Классы данных: автоматически генерируемые реализации универсальных методов

Чтобы превратить класс в удобное хранилище данных, необходимо переопределить следующие методы: `toString`, `equals` и `hashCode`. Обычно их реализация проста, и такие IDE, как IntelliJ IDEA, помогут сгенерировать их автоматически и проверить правильность и последовательность их реализации.

Но в Kotlin генерировать все эти методы не потребуется. При добавлении в класс модификатора `data` необходимые методы будут сгенерированы автоматически. Пример показан в листинге 4.27.

Листинг 4.27. `Customer` в качестве класса данных

```
data class Customer(val name: String, val postalCode: Int)
```

Легко, не так ли? В результате получаем класс с переопределениями всех стандартных методов:

- `equals` для сравнения экземпляров;
- `hashCode` для использования экземпляров в качестве ключей в хеш-контейнерах, таких как `HashMap`;
- `toString` для генерации строковых представлений с отображениями всех полей в порядке объявления.

В методах `equals` и `hashCode` учитываются все свойства, объявленные в первичном конструкторе. Сгенерированный метод `equals` проверяет равенство значений всех свойств. Метод `hashCode` возвращает значение, зависящее от хеш-кодов всех

свойств. Обратите внимание: свойства, не объявленные в первичном конструкторе, не участвуют в проверке равенства и вычислении хеш-кода. В листинге 4.28 показано это «волшебство».

Листинг 4.28. Класс данных `Customer` содержит реализации всех стандартных методов

```
fun main() {
    val c1 = Customer("Sam", 11521)
    val c2 = Customer("Mart", 15500)
    val c3 = Customer("Sam", 11521)
    println(c1)
    // Customer(name=Sam, postalCode=11521)
    println(c1 == c2)
    // ложь
    println(c1 == c3)
    // истина
    println(c1.hashCode())
    // 2580770
    println(c3.hashCode())
    // 2580770
}
```

Это далеко не полный список полезных методов, сгенерированных для классов `data`. Далее мы поговорим еще об одном из них, а в разделе 9.4 описаны все остальные.

Классы данных и неизменяемость: метод `copy`

Обратите внимание: свойства класса данных не обязательно должны быть типа `val` (можно использовать и модификатор `var`), однако настоятельно рекомендуется использовать лишь свойства, доступные только для чтения. Экземпляры класса данных должны быть *неизменяемыми*. Это необходимо, если требуется использовать такие экземпляры в качестве ключей в `HashMap` или подобном контейнере. В противном случае контейнер может перейти в некорректное состояние, если объект-ключ изменится после добавления в контейнер. Создавать логические схемы на основе неизменяемых объектов тоже гораздо проще, особенно в многопоточном коде: после создания объекта он остается в своем первоначальном состоянии и не нужно задумываться о его изменении другими потоками во время работы кода.

Чтобы еще больше упростить использование классов данных в качестве неизменяемых объектов, компилятор Kotlin генерирует для них еще один метод. Он дает возможность *копировать* экземпляры классов, изменяя значения некоторых свойств. Создание копии более предпочтительно для модификации экземпляра в памяти. У такой копии будет отдельный жизненный цикл, и она не может повлиять на ссылки на оригинальный экземпляр. Метод `copy` будет выглядеть следующим образом при реализации вручную:

```
class Customer(val name: String, val postalCode: Int) {
    /* ... */
    fun copy(name: String = this.name,
            postalCode: Int = this.postalCode) =
        Customer(name, postalCode)
}
```

КЛАССЫ ДАННЫХ KOTLIN И ЗАПИСИ JAVA

Записи впервые появились в Java 14. Концептуально они очень похожи на классы данных Kotlin, поскольку содержат неизменяемые группы значений. Кроме того, записи автоматически генерируют определенные методы на основе их значений, например `toString`, `hashCode` и `equals`. Другие удобные функции, например метод `copy`, в записях отсутствуют.

Вдобавок записи Java накладывают больше структурных ограничений по сравнению с классами данных Kotlin:

- все объекты должны получить модификаторы `private` и `final`;
- запись не может расширять суперкласс;
- нельзя указывать дополнительные свойства в теле класса.

Для обеспечения совместимости следует объявлять классы записей в Kotlin. Для этого укажите классу `data` аннотацию `@JvmRecord`. В таком случае класс данных должен придерживаться структурных ограничений записей.

Использовать метод `copy` можно следующим образом:

```
fun main() {
    val bob = Customer("Bob", 973293)
    println(bob.copy(postalCode = 382555))
    // Customer(name=Bob, postalCode=382555)
}
```

Вы узнали, как модификатор `data` делает классы значений-объектов более удобными в использовании. Теперь обсудим делегирование классов. Эта возможность Kotlin позволяет избежать генерируемого IDE шаблонного кода.

4.3.3. Делегирование класса: использование ключевого слова `by`

Хрупкость при наследовании реализаций становится распространенной проблемой при проектировании больших объектно-ориентированных систем. В случае расширения класса и переопределения некоторых его методов код становится зависимым от деталей реализации расширяемого класса. Когда система развивается и реализация базового класса меняется или в него добавляются новые методы, ожидаемое при проектировании поведение может измениться. В итоге код может вести себя некорректно.

При разработке Kotlin эта проблема учитывалась, и по умолчанию классы обозначаются как `final`, что было показано в подразделе 4.1.2. Такой подход гарантирует наследование только предназначенных для расширения классов. Когда явно видно, что класс открыт, вы не забудете, что изменения должны быть совместимы с производными классами.

Но часто требуется добавить поведение в другой класс, даже если он не был предназначен для расширения. Такую логику зачастую реализуют с помощью паттерна

проектирования «*Декоратор*». Его суть заключается в создании нового класса с реализацией интерфейса исходного класса, а экземпляр исходного класса хранится в качестве поля. Методы с неизменным поведением исходного класса передаются экземпляру исходного класса.

Недостатком этого подхода стало требование довольно большого количества шаблонного кода (настолько большого, что в некоторых IDE, например в IntelliJ IDEA, есть специальные функции для генерации этого кода). Например, такой объем кода нужен для декоратора с реализацией такого простого интерфейса, как `Collection`, даже если вы не изменяете никакое поведение:

```
class DelegatingCollection<T> : Collection<T> {
    private val innerList = arrayListOf<T>()

    override val size: Int get() = innerList.size
    override fun isEmpty(): Boolean = innerList.isEmpty()
    override fun contains(element: T): Boolean = innerList.contains(element)
    override fun iterator(): Iterator<T> = innerList.iterator()
    override fun containsAll(elements: Collection<T>): Boolean =
        innerList.containsAll(elements)
}
```

Но в Kotlin есть поддержка делегирования первого класса как функциональной возможности языка. При реализации интерфейса можно сказать, что она *делегуется* другому объекту с помощью ключевого слова `by`. Переписать предыдущий пример, используя этот подход, можно так:

```
class DelegatingCollection<T>(  
    innerList: Collection<T> = mutableListOf<T>()  
) : Collection<T> by innerList
```

Все реализации методов в классе исчезли. Компилятор сгенерирует их, и реализация будет аналогична примеру `DelegatingCollection`. В коде мало интересного, поэтому нет смысла писать его вручную, если компилятор может сделать эту работу автоматически.

Теперь, когда требуется изменить поведение некоторых методов, их можно переопределить, и этот код будет вызываться вместо сгенерированных методов. Можно указывать методы, в отношении которых вас устраивает стандартная реализация делегирования базовому экземпляру.

Посмотрим, как с помощью этой техники можно реализовать коллекцию для подсчета количества попыток добавить в нее элемент. Например, при выполнении дедубликации можно использовать такую коллекцию для измерения эффективности процесса, сравнивая количество попыток добавить элемент с итоговым размером коллекции (листинг 4.29).

Здесь переопределяются методы `add` и `addAll`, предназначенные для увеличения значения подсчета, а остальная реализация интерфейса `MutableCollection` делегируется контейнеру.

Листинг 4.29. Использование делегирования классов

```

class CountingSet<T>{
    private val innerSet: MutableCollection<T> = HashSet<T>()
) : MutableCollection<T> by innerSet {
    var objectsAdded = 0

    override fun add(element: T): Boolean {
        objectsAdded++
        return innerSet.add(element)
    }

    override fun addAll(elements: Collection<T>): Boolean {
        objectsAdded += elements.size
        return innerSet.addAll(elements)
    }
}

fun main() {
    val cset = CountingSet<Int>()
    cset.addAll(listOf(1, 1, 2))
    println("Added ${cset.objectsAdded} objects, ${cset.size} uniques.")
    // Added 3 objects, 2 uniques.
}

```

Делегирование реализации коллекции *MutableCollection* коллекции *innerSet*

Указание отличающейся реализации вместо прямого делегирования

Важно, что не вводится никакой зависимости от того, как реализована базовая коллекция. Например, неважно, реализует ли эта коллекция метод `addAll` вызовом `add` в цикле или использует другую реализацию с оптимизацией для конкретного случая. Вы полностью контролируете происходящее при вызове клиентским кодом вашего класса и полагаетесь только на документированный API базовой коллекции для реализации своих операций. Поэтому можно рассчитывать на то, что программа будет работать.

Мы выяснили, как компилятор Kotlin может генерировать полезные методы для классов. Пора переходить к последней важной части из темы классов Kotlin — ключевому слову `object` и различным ситуациям его применения.

4.4. КЛЮЧЕВОЕ СЛОВО `OBJECT`: ОБЪЯВЛЕНИЕ КЛАССА И СОЗДАНИЕ ЕГО ЭКЗЕМПЛЯРА

Ключевое слово `object` встречается в Kotlin в разных случаях, но у всех общая суть: оно определяет класс и одновременно создает экземпляр (другими словами, объект) этого класса. Рассмотрим различные ситуации его применения.

- *Объявление объекта* — способ определения объекта-одиночки.
- *Объекты-компаньоны* — могут содержать фабричные методы и другие методы текущего класса, обходясь без вызова экземпляра класса. Доступ к их членам можно получить по имени класса.

- *Объектные выражения* — используются вместо анонимного внутреннего класса Java.

Пришло время обсудить эти возможности Kotlin более подробно.

4.4.1. Объявления объектов: объекты-одиночки — это просто

Довольно часто при проектировании объектно-ориентированных систем требуется класс всего с одним экземпляром. В таких языках, как Java, для этого используется паттерн «Одиночка»: определяется класс с конструктором типа `private` и статическим полем с единственным существующим экземпляром класса.

В Kotlin для этих целей есть поддержка языка первого класса — функциональная возможность *объявления объектов*. Она сочетает в себе *объявление класса* и *объявление одного экземпляра* этого класса.

Например, объявление объекта используется для представления платежной ведомости организации. Скорее всего, такая ведомость существует в единственном экземпляре, поэтому, в зависимости от общей сложности приложения, использование объекта для решения такой задачи может быть вполне разумным:

```
object Payroll {
    val allEmployees = mutableListOf<Person>()

    fun calculateSalary() {
        for (person in allEmployees) {
            /* ... */
        }
    }
}
```

Объявления объектов представлены ключевым словом `object`. Таким образом, в одном выражении фактически определяется класс и его переменная с тем же именем.

Объявление объекта, как и класса, может содержать объявления свойств, методов, блоков инициализаторов и т. д. Единственное, что запрещено, — конструкторы (как первичные, так и вторичные). В отличие от экземпляров обычных классов, объявления объектов создаются непосредственно в точке определения, а не через вызов конструктора из других мест кода. Поэтому определять конструктор для объявления объекта не имеет смысла. Аналогичным образом, любое начальное состояние объявления объекта должно быть предоставлено как часть тела объекта.

Объявление объекта, как и переменной, позволяет вызывать методы и обращаться к свойствам, используя имя объекта слева от символа `..`:

```
Payroll.allEmployees.add(Person(/* ... */))

Payroll.calculateSalary()
```

Кроме того, объявления объектов могут наследоваться от классов и интерфейсов. Это бывает полезно, когда используемый фреймворк требует от вас реализации

интерфейса, но ваша реализация не содержит никакого состояния. Например, рассмотрим интерфейс `Comparator`. Его реализация получает два объекта и возвращает целое число, указывающее, какой из объектов больше. Компараторы почти никогда не хранят данные, поэтому обычно требуется всего один экземпляр интерфейса `Comparator` для определенного способа сравнения объектов. Это идеальный случай использования объявления объекта.

В качестве примера реализуем компаратор для сравнения пути к файлам без учета регистра (листинг 4.30).

Листинг 4.30. Реализация интерфейса `Comparator` с объектом

```
object CaseInsensitiveFileComparator : Comparator<File> {
    override fun compare(file1: File, file2: File): Int {
        return file1.path.compareTo(file2.path,
            ignoreCase = true)
    }
}

fun main() {
    println(
        CaseInsensitiveFileComparator.compare(
            File("/User"), File("/user")
        )
    )
    // 0
}
```

ОБЪЕКТЫ-ОДИНОЧКИ И ВНЕДРЕНИЕ ЗАВИСИМОСТЕЙ

Как и паттерн «Одиночка», объявления объектов не всегда удобны для применения в больших программных системах. Они отлично подходят для небольших фрагментов кода с минимумом зависимостей или вовсе без них, но не для больших компонентов, взаимодействующих со множеством частей системы. Основная причина такого применения заключается в невозможности контролировать создание экземпляров объектов и указывать параметры для конструкторов.

Это означает, что нет возможности заменить реализацию самого объекта или других классов, от которых он зависит, в модульных тестах или различных конфигурациях программной системы. Если же это необходимо, то используйте обычные классы Kotlin и внедрение зависимостей.

Объекты-одиночки применяются в любом контексте, где может быть использован обычный объект (экземпляр класса). Например, этот объект можно передать в качестве аргумента в функцию, принимающую объект `Comparator`:

```
fun main() {
    val files = listOf(File("/Z"), File("/a"))
    println(files.sortedWith(CaseInsensitiveFileComparator))
    // [/a, /Z]
}
```

Здесь используется функция `sortedWith` — она возвращает список, отсортированный в соответствии с указанным объектом `Comparator`.

Объекты можно объявлять и внутри класса. Такие объекты существуют в единственном числе: у вас не будет отдельного объекта для каждого экземпляра контейнирующего класса. Например, компаратор для сравнения объектов определенного класса логично разместить внутри этого класса (листинг 4.31).

Листинг 4.31. Реализация интерфейса `Comparator` с вложенным объектом

```
data class Person(val name: String) {
    object NameComparator : Comparator<Person> {
        override fun compare(p1: Person, p2: Person): Int =
            p1.name.compareTo(p2.name)
    }
}

fun main() {
    val persons = listOf(Person("Bob"), Person("Alice"))
    println(persons.sortedWith(Person.NameComparator))
    // [Person(name=Alice), Person(name=Bob)]
}
```

ИСПОЛЬЗОВАНИЕ ОБЪЕКТОВ KOTLIN ИЗ JAVA

Объявление объекта в Kotlin компилируется как класс со статическим полем. В нем будет находиться единственный экземпляр класса `INSTANCE`. При реализации паттерна «Одиночка» в Java, вероятно, пришлось бы сделать то же самое вручную. Таким образом, чтобы использовать объект Kotlin из кода Java, необходимо обратиться к статическому полю `INSTANCE`:

```
/* Java */
CaseInsensitiveFileComparator.INSTANCE.compare(file1, file2);
Person.NameComparator.INSTANCE.compare(person1, person2)
```

В этом примере тип полей `INSTANCE` — это `CaseInsensitiveFileComparator` и `NameComparator` соответственно.

Теперь рассмотрим особый случай объектов, вложенных внутри класса, — *объекты-компаньоны*.

4.4.2. Объекты-компаньоны: место для фабричных методов и статических членов класса

В классах Kotlin не может быть статических членов. На самом деле в данном языке нет ключевого слова `static`, как в Java. Вместо этого Kotlin полагается на функции уровня пакета (они способны заменить статические методы во многих ситуациях) и объявления объектов (они служат для замены статических методов и таких же полей в иных случаях). В большинстве случаев рекомендуется использовать функции верхнего уровня. Но они не могут получить доступ к приватным членам класса

(рис. 4.5). *Фабричный метод* может служить примером функции, которой нужен доступ к приватным членам. Эти методы отвечают за создание объекта, и поэтому им часто требуется доступ к его приватным членам.

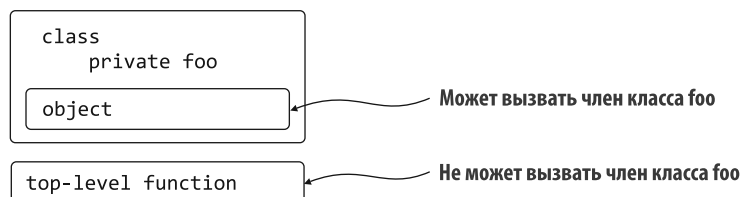


Рис. 4.5. В функциях верхнего уровня за пределами класса нельзя использовать приватные члены класса

Если записать функцию как член объявления объекта внутри класса, то ее можно будет вызвать без экземпляра класса, но доступа к внутренним функциям этого класса у нее не будет. Только одно из объявлений объектов в классе можно пометить специальным ключевым словом — `companion`. При этом вы получите возможность обращаться к методам и свойствам этого объекта непосредственно через имя содержащего его класса, не указывая имя объекта явно. В итоге синтаксис выглядит точно так же, как вызов статического метода в Java. Перед вами базовый пример демонстрации синтаксиса:

```

class MyClass {
    companion object {
        fun callMe() {
            println("Companion object called")
        }
    }
}

fun main() {
    MyClass.callMe()
    // Companion object called
}

```

Важно помнить, что объект-компаньон принадлежит к соответствующему классу. Нельзя получить доступ к членам объекта-компаньона в экземпляре класса. В этом заключается еще одно их отличие от статических членов в Java:

```

fun main() {
    val myObject = MyClass()
    myObject.callMe()
    // Error: Unresolved reference: callMe
}

```

В подразделе 4.2.1 мы обещали указать хорошее место для вызова конструктора `private`, и оно именно здесь. У объекта-компаньона есть доступ ко всем приватным членам класса, в том числе к приватному конструктору. Это делает его идеальным кандидатом для реализации паттерна «Фабрика».

Рассмотрим пример объявления двух конструкторов, а затем изменим его, чтобы использовать фабричные методы, объявленные в объекте-компаньоне. Продолжим развивать код листинга 4.19 на основе классов `SocialUser` и `SubscribingUser`.

Ранее эти сущности представляли собой различные классы, реализующие общий интерфейс `User`. Теперь обойдемся одним классом, но предоставим различные средства его создания (листинг 4.32).

Листинг 4.32. Определение класса с несколькими вторичными конструкторами

```
class User {
    val nickname: String

    constructor(email: String) {
        nickname = email.substringBefore('@')
    }

    constructor(socialAccountId: Int) {
        nickname = getSocialNetworkName(socialAccountId)
    }
}
```

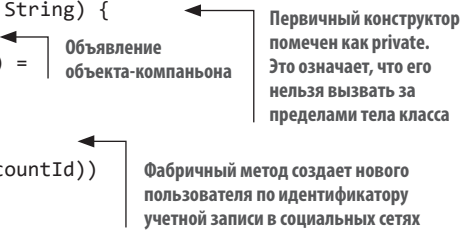


Альтернативный подход к выражению той же логики заключается в использовании фабричных методов для создания экземпляров класса. Это может быть полезно по многим причинам. Экземпляр класса `User` создается с помощью фабричных методов, а не нескольких конструкторов (листинг 4.33).

Листинг 4.33. Замена вторичных конструкторов на фабричные методы

```
class User private constructor(val nickname: String) {
    companion object {
        fun newSubscribingUser(email: String) =
            User(email.substringBefore('@'))

        fun newSocialUser(accountId: Int) =
            User(getNameFromSocialNetwork(accountId))
    }
}
```



Методы объекта `companion object` можно вызвать по имени класса:

```
fun main() {
    val subscribingUser = User.newSubscribingUser("bob@gmail.com")
    val socialUser = User.newSocialUser(4)
    println(subscribingUser.nickname)
    // bob
}
```

Фабричные методы очень полезны. Их можно назвать в соответствии с назначением, как показано в примере. Кроме того, фабричный метод может возвращать подклассы класса, в котором объявлен метод. В приведенном примере это классы `SubscribingUser` и `SocialUser`. Вдобавок можно не создавать новые объекты без необходимости. Например, можно сделать так, чтобы каждый адрес электронной

почты соответствовал уникальному экземпляру класса `User`, и возвращать существующий экземпляр вместо нового, когда фабричный метод вызывается с письмом, хранящимся в кэше. Но если такие классы требуется расширить, то использование нескольких конструкторов может оказаться лучшим решением, поскольку члены объекта-компаньона нельзя переопределить в подклассах.

4.4.3. Объекты-компаньоны в качестве обычных объектов

Объект-компаньон — это обычный объект, объявленный в классе. Как и другие объявления объектов, он может получить имя, реализовывать интерфейс, содержать функции-расширения или свойства. В этом подразделе мы рассмотрим один из таких примеров.

Предположим, вы работаете над веб-сервисом для расчета заработной платы компании, и вам нужно сериализовать и десериализовать объекты в формате JSON. Логике сериализации можно поместить в объект-компаньон (листинг 4.34).

Листинг 4.34. Объявление именованного объекта-компаньона

```
class Person(val name: String) {
    companion object Loader { ← Объявление объекта-компаньона Loader
        fun fromJSON(jsonText: String): Person = /* ... */
    }
}

fun main() {
    val person = Person.Loader.fromJSON("""{"name": "Dmitry"}""") ←
    println(person.name)                                           Вызвать функцию
    // Dmitry                                                       fromJSON можно
    val person2 = Person.fromJSON("""{"name": "Brent"}""")        обоими способами
    println(person2.name)
    // Brent
}
```

В большинстве случаев ссылка на объект-компаньон реализуется по имени содержащего его класса, поэтому придумывать новое не обязательно. Но при необходимости можно указать его имя, как в листинге 4.34, — `companion object Loader`.

У класса может быть только один объект-компаньон. Независимо от наличия у него имени всегда можно получить доступ к его членам через имя класса. Если имя объекта-компаньона не указать, то ему будет присвоено имя по умолчанию — `Companion`. Примеры использования этого имени приведены ниже в пункте «Расширения объектов-компаньонов».

Множество таких одиночных объектов-компаньонов можно найти и в стандартной библиотеке Kotlin. Например, объект `companion object Default` в классе `Kotlin Random` предоставляет доступ к генератору случайных чисел по умолчанию:

```
val chance = Random.nextInt(from = 0, until = 100)
val coin = Random.Default.nextBoolean()
```

Оба вызова используют стандартный генератор случайных чисел Kotlin

Реализация интерфейсов в объектах-компаньонах

В объявлении объекта-компаньона, как и в любом другом объявлении объекта, можно реализовывать интерфейсы. Имя содержащего класса можно использовать непосредственно в качестве экземпляра объекта, реализующего интерфейс.

ОБЪЕКТЫ-КОМПАЬОНЫ KOTLIN И СТАТИЧЕСКИЕ ЧЛЕНЫ

Объект-компаньон для класса компилируется так же, как и обычный объект: статическое поле класса ссылается на его экземпляр. Если у объекта-компаньона нет имени, то к нему можно получить доступ через ссылку `Companion` из кода Java:

```
/* Java */
Person.Companion.fromJSON("...");
```

Если у объекта-компаньона есть имя, то оно указывается вместо слова `Companion`.

Но вам может понадобиться работать с кодом Java, который требует, чтобы член класса был статическим. Этого можно добиться с помощью аннотации `@JvmStatic` для соответствующего члена класса. Если требуется объявить статическое поле, то используйте аннотацию `@JvmField` для свойства верхнего уровня или объявленного в объекте свойства. Эти функции существуют специально для обеспечения совместимости. И строго говоря, их нельзя считать частью языка. Подробно об аннотациях мы поговорим в главе 12. Стоит отметить, что в Kotlin можно обращаться к статическим методам и полям, объявленным в классах Java, с помощью синтаксиса Java.

Предположим, что в системе есть много видов объектов, в том числе `Person`. Требуется предоставить общий способ создания объектов всех типов. Допустим, для десериализуемых из формата JSON объектов есть интерфейс `JSONFactory`, и все объекты в системе должны создаваться с помощью этой фабрики. Для реализации такого интерфейса для класса `Person` можно использовать объект `companion object`. (В листинге 4.35 используются обобщения. С ними вы познакомитесь в главе 11, но будем надеяться, что код все равно понятен.)

Листинг 4.35. Реализация интерфейса в объекте-компаньоне

```
interface JSONFactory<T> {
    fun fromJSON(jsonText: String): T
}

class Person(val name: String) {
    companion object : JSONFactory<Person> {
        override fun fromJSON(jsonText: String): Person = /* ... */
    }
}
```

Реализация интерфейса
в объекте-компаньоне

Затем, при наличии функции с абстрактной фабрикой для загрузки сущностей, можно передать ей объект `Person`:

```
fun <T> loadFromJSON(factory: JSONFactory<T>): T {
    /* ... */
}
loadFromJSON(Person)
```

Передача экземпляра
объекта-компаньона
в функцию

Обратите внимание, что имя класса `Person` используется в качестве имени экземпляра `JSONFactory`.

Расширения объектов-компаньонов

В разделе 3.3 мы говорили о том, что в функциях-расширениях можно определять методы, которые вызываются для *экземпляров класса* из другого места кодовой базы. Но что, если требуются функции, доступные для вызова в *самом классе*? И вызывать их нужно с помощью синтаксиса методов объектов-компаньонов? Это можно сделать, если у класса есть объект-компаньон. Для него потребуется определить функцию-расширение. Более конкретно: если в классе `C` есть объект-компаньон и вы определяете функцию-расширение `func` для `C.Companion`, то вызвать ее можно как `C.func()`.

Например, представьте, что требуется реализовать более четкое разделение ответственностей для класса `Person`. Сам он будет частью основного модуля бизнес-логики, но привязывать этот модуль к какому-либо конкретному формату данных не нужно. Из-за этого функция десериализации должна быть определена в модуле, отвечающем за взаимодействие клиента и сервера. Чтобы сохранить тот же красивый синтаксис для вызова функции десериализации, можно использовать функцию-расширение для объекта-компаньона. Пример такого кода показан в листинге 4.36. Обратите внимание на использование имени по умолчанию (`Companion`) для ссылки на объект-компаньон — он был объявлен без указания имени.

Листинг 4.36. Определение функции-расширения для объекта-компаньона

```
// модуль бизнес-логики
class Person(val firstName: String, val lastName: String) {
    companion object {
    }
}

// модуль коммуникации "клиент – сервер"
fun Person.Companion.fromJSON(json: String): Person {
    /* ... */
}
val p = Person.fromJSON(json)
```

Объявление пустого
объекта-компаньона

Объявление
функции-расширения

Вызов функции `fromJSON` осуществляется так, словно она была определена как метод объекта-компаньона. Хотя на самом деле она определена вне его как функция-расширение. Как обычно, эта функция-расширение выглядит как член экземпляра, но таковым не является. И, как и другие функции-расширения, расширение объекта-компаньона не дает доступа к его приватным членам. Но учтите, что объявить объект-компаньон необходимо в своем классе, даже пустом. Только в этом случае у вас будет возможность определять для него расширения.

Вы узнали, насколько полезными могут быть объекты-компаньоны. Теперь перейдем к следующей возможности Kotlin. Она выражается с помощью того же ключевого слова `object`.

4.4.4. Объектные выражения: перефразирование анонимных внутренних классов

Ключевое слово `object` применяется для объявления не только именованных объектов-одиночек, но и *анонимных объектов*. В Java они заменяют использование анонимных внутренних классов.

Для примера рассмотрим, как можно создать типичный *слушатель событий* в Kotlin. Допустим, вы работаете с классом `Button`. Он принимает экземпляр интерфейса `MouseListener`, служащий для определения поведения при взаимодействии пользователя с кнопкой:

```
interface MouseListener {
    fun onEnter()
    fun onClick()
}
class Button(private val listener: MouseListener) { /* ... */ }
```

Можно использовать объектное выражение для создания специальной реализации интерфейса `MouseListener` и затем передать его конструктору класса `Button` (листинг 4.37).

Листинг 4.37. Реализация слушателя событий с помощью анонимного объекта

```
fun main() {
    Button(object : MouseListener {
        override fun onEnter() { /* ... */ }
        override fun onClick() { /* ... */ }
    })
}
```

Объявление анонимного объекта
для реализации интерфейса `MouseListener`

Переопределение методов
интерфейса `MouseListener`

Синтаксис такой же, как и при объявлении объектов, за исключением того, что имя объекта опущено. Но в отличие от объявлений объектов анонимные объекты не относятся к объектам-одиночкам. При каждом выполнении объектного выражения создается новый экземпляр объекта.

Объектное выражение объявляет класс и создает экземпляр этого класса, но не присваивает имя ни классу, ни экземпляру. Как правило, ни то ни другое не нужно, поскольку объект используется в качестве параметра в вызове функции. Если требуется присвоить объекту имя, то можно сохранить его в переменной:

```
val listener = object : MouseListener {
    override fun onEnter() { /* ... */ }
    override fun onClick() { /* ... */ }
}
```

Анонимные объекты в Kotlin довольно гибкие — могут реализовать один, несколько или ни одного интерфейса.

Код в объектном выражении может обращаться к переменным в функции, в которой был создан, — точно так же, как в анонимных классах Java. Но в отличие от Java эта

возможность не ограничивается неизменяемыми переменными. В Kotlin допускается изменять значения переменных внутри объектного выражения. Для примера рассмотрим, как использовать слушатель для подсчета количества щелчков левой кнопкой мыши в окне (листинг 4.38).

Листинг 4.38. Доступ к локальным переменным из анонимного объекта

```
fun main() {
    var clickCount = 0    ← Объявление локальной переменной
    Button(object : MouseListener {
        override fun onEnter() { /* ... */ }
        override fun onClick() {
            clickCount++    ← Обновление значения переменной
        }
    })
}
```

ПРИМЕЧАНИЕ

Пользу от использования объектных выражений чаще всего можно ощутить, когда требуется переопределить несколько методов в анонимном объекте. Если нужно реализовать интерфейс с одним методом (например, `Runnable`), то можно воспользоваться поддержкой `Single Abstract Method (SAM)` в Kotlin. Это преобразование литерала функции в реализацию интерфейса с одним абстрактным методом. И вы сможете написать свою реализацию в виде литерала функции (лямбда-выражения). Более подробно лямбда-выражения и преобразование SAM мы обсудим в разделе 5.2.

4.5. ДОПОЛНИТЕЛЬНАЯ БЕЗОПАСНОСТЬ ТИПОВ БЕЗ ДОПОЛНИТЕЛЬНЫХ ЗАТРАТ: ВСТРОЕННЫЕ КЛАССЫ

На примере классов данных можно было наблюдать, как сгенерированный компилятором код помогает сохранить удобочитаемость и порядок в коде. Рассмотрим еще один пример для демонстрации возможности компилятора Kotlin — *встроенные классы*.

Предположим, вы создаете простую систему для отслеживания своих расходов:

```
fun addExpense(expense: Int) {
    // сохранение расходов в центах США
}
```

Во время следующей поездки в Японию может потребоваться добавить счет за вкусную паровую булочку «Никуман», которая обойдется вам в ¥200:

```
addExpense(200) // Японская иена
```

Проблема становится очевидной довольно быстро: сигнатура функции принимает простое значение типа `Int`, поэтому ничто не мешает вызывающим ее пользователям передавать значения с другой семантикой. В этом случае вызывающая сторона

интерпретирует параметр как «иену», даже если в фактической реализации переданное значение означает «цент США».

Классический подход к предотвращению такой ситуации — использовать класс вместо обычного значения типа `Int`:

```
class UsdCent(val amount: Int)

fun addExpense(expense: UsdCent) {
    // сохранение расходов в центах США
}

fun main() {
    addExpense(UsdCent(147))
}
```

При таком подходе вероятность случайно передать в функцию значение с неправильной семантикой гораздо ниже, однако он сопряжен с некоторыми проблемами производительности. Для каждого вызова функции `addExpense` создается новый объект `UsdCent`, и внутри тела функции он разворачивается и отбрасывается.

Если эта функция вызывается часто, то необходимо выделить большое количество недолго существующих объектов и впоследствии собрать мусор.

Именно здесь на помощь приходят *встроенные классы*. Они позволяют повысить уровень безопасности типа, не причиняя ущерба для производительности.

Чтобы превратить класс `UsdCent` во встроенный, его необходимо пометить ключевым словом `value`, а затем аннотировать с помощью `@JvmInline`:

```
@JvmInline
value class UsdCent(val amount: Int)
```

Это небольшое изменение позволяет избежать ненужного создания экземпляров объектов. Но при этом не придется отказываться от безопасности типов, обеспечиваемой типом-оберткой `UsdCent`. Во время выполнения экземпляры класса `UsdCent` будут представлены как обернутое свойство. Отсюда же происходит название встроенных классов — данные класса *встраиваются* в места использования.

ПРИМЕЧАНИЕ

Если быть до конца точными, то компилятор Kotlin представляет встроенный класс как свой базовый тип везде, где это возможно. Бывают случаи, когда необходимо сохранить тип обертки, — в первую очередь, когда встроенный класс используется в качестве параметра типа. Обсуждение этих особых случаев приводится в документации Kotlin (<https://kotlinlang.org/docs/inline-classes.html>). Более подробно идею обертывания и разворачивания типов мы рассмотрим в главе 7.

Чтобы считаться встроенным, класс должен получить только одно свойство, и оно инициализируется в первичном конструкторе. Кроме того, встроенные классы не участвуют в иерархии классов — они не расширяют другие классы и не могут

быть расширены. Однако они все равно могут реализовывать интерфейсы, определять методы или предоставлять вычисляемые свойства (описанные в подразделе 2.2.2):

```
interface PrettyPrintable {
    fun prettyPrint()
}

@JvmInline
value class UsdCent(val amount: Int): PrettyPrintable {
    val salesTax get() = amount * 0.06
    override fun prettyPrint() = println("${amount}¢")
}

fun main() {
    val expense = UsdCent(1_99)
    println(expense.salesTax)
    // 11.94
    expense.prettyPrint()
    // 199¢
}
```

Чаше всего встроенные классы применяются для того, чтобы сделать семантику базовых значений явной. Например, при указании единиц измерения для простых типов чисел или при выявлении различий значений разных строк. Они предотвращают случайную передачу по вызову совместимых значений с различной семантикой. Еще один яркий пример использования встроенных классов мы рассмотрим в подразделе 8.1.2.

Обсуждение классов, интерфейсов и объектов завершено. В следующей главе мы перейдем к одной из самых интересных областей Kotlin — лямбда-выражениям и функциональному программированию.

ВСТРОЕННЫЕ КЛАССЫ И ПРОЕКТ VALHALLA

В настоящее время встроенные классы представляют собой функциональную особенность компилятора Kotlin. Ему известно, как выдать код без потери производительности за выделение объектов. Но так будет не всегда. Проект Valhalla (<https://openjdk.org/projects/valhalla/>) — это набор предложений по улучшению JDK (JDK Enhancement Proposals, JEP). Они направлены на внедрение поддержки встроенных классов (теперь их называют *примитивными классами*, а ранее именовали *классами значений*) в саму JVM. Это означает, что среда выполнения будет изначально воспринимать концепцию встроенных классов.

Поэтому проект Valhalla стал причиной того, что встроенные классы в Kotlin в настоящее время аннотируются `@JvmInline`. Это наглядно показывает, что они подвергаются особой обработке компилятора Kotlin. Когда в будущем появится реализация на базе Valhalla, можно будет объявлять встроенные классы в Kotlin без аннотации и использовать поддержку JVM по умолчанию.

РЕЗЮМЕ

- Интерфейсы в Kotlin похожи на интерфейсы Java, но могут содержать реализации и свойства по умолчанию.
- Все объявления по умолчанию относятся к типам `final` и `public`.
- Чтобы снять с объявления тип `final`, необходимо пометить его как `open`.
- Область видимости объявления типа `internal` ограничена модулем.
- Вложенные классы по умолчанию не являются внутренними. Ключевое слово `inner` используется для хранения ссылки на внешний класс.
- Все прямые подклассы запечатанных (`sealed`) классов и все реализации запечатанных интерфейсов должны быть известны во время компиляции.
- Блоки инициализаторов и вторичные конструкторы обеспечивают гибкость при инициализации экземпляров класса.
- Идентификатор `field` применяется для ссылки на теневое поле свойства из тела метода доступа.
- Классы данных предоставляют сгенерированные компилятором методы `equals`, `hashCode`, `toString`, `copy` и др.
- Делегирование классов позволяет избежать множества одинаковых делегирующих методов в коде.
- Объявление объектов — способ Kotlin определить класс-одиночку.
- Объекты-компаньоны (совместно с функциями и свойствами на уровне пакета) заменяют статические определения методов и полей в Java.
- Объекты-компаньоны, как и другие объекты, могут реализовывать интерфейсы, а также содержать функции- и свойства-расширения.
- Объектные выражения заменяют анонимные внутренние классы Java в Kotlin, предоставляя дополнительные возможности. Например, возможность реализовать несколько интерфейсов и изменять переменные, определенные в области видимости создания объекта.
- Встроенные классы добавляют в программу повышенный уровень безопасности типов, позволяя избежать потенциального падения производительности, связанного с выделением большого количества недолго существующих объектов.

5

Программирование с использованием лямбда-выражений

В этой главе

- ✓ Использование лямбда-выражений и ссылок на члены класса без вызова для передачи фрагментов кода и поведения в функции.
- ✓ Определение функциональных интерфейсов в Kotlin и применение функциональных интерфейсов Java.
- ✓ Использование лямбда-выражений с приемниками.

Лямбда-выражения, или просто *лямбды*, — это, по сути, небольшие фрагменты кода, которые можно передавать другим функциям. С помощью лямбд можно легко извлекать общие структуры кода в библиотечные функции, что позволяет повторно использовать больше кода и повышать его выразительность. Лямбды активно применяются в стандартной библиотеке Kotlin. В этой главе вы узнаете, что такое лямбда, увидите примеры типичных случаев использования лямбда-функций, посмотрите, как они выглядят в Kotlin, и узнаете, как они связаны со *ссылками на члены без вызова*.

Вдобавок мы покажем полную совместимость лямбд с API и библиотеками Java — даже с теми, которые изначально не были разработаны для взаимодействия с лямбдами. Вы узнаете об использовании функциональных интерфейсов в Kotlin, позволяющих сделать код, который работает с типами функций, еще более выразительным. Наконец, мы рассмотрим *лямбды с приемниками* — особый вид лямбд, в которых тело выполняется в другом контексте, отличающемся от окружающего кода.

5.1. ЛЯМБДА-ВЫРАЖЕНИЯ И ССЫЛКИ НА ЧЛЕНЫ КЛАССА БЕЗ ВЫЗОВА

Появление лямбд в Java 8 стало одним из самых долгожданных изменений в эволюции языка. Почему это было так важно? В данном разделе вы узнаете о полезности лямбд и о том, как выглядит их синтаксис в Kotlin.

5.1.1. Введение в лямбда-выражения: блоки кода как значения

Задача передачи и хранения фрагментов поведения в коде встречается часто. Например, обычно требуется выразить такие идеи, как «Когда происходит событие, запустить этот обработчик» или «Применить эту операцию ко всем элементам в структуре данных». В старых версиях Java это можно было сделать с помощью анонимных внутренних классов. Они справляются со своей задачей, но требуют многословного синтаксиса.

Есть и другой подход к решению этой задачи — возможность *рассматривать функции как значения*. Вместо того чтобы объявлять класс и передавать его экземпляр в функцию, можно передать функцию напрямую. Лямбда-выражения позволяют сделать код еще более лаконичным. Нет необходимости объявлять функцию — можно передать блок кода непосредственно в качестве ее параметра. Такой подход, когда функции рассматриваются как значения и комбинируются для выражения поведения, считается одним из столпов *функционального программирования*.

КРАТКОЕ РУКОВОДСТВО ПО ФУНКЦИОНАЛЬНОМУ ПРОГРАММИРОВАНИЮ

В подразделе 1.2.3 мы кратко рассказали о природе Kotlin как мультипарадигменного языка и о преимуществах функционального программирования, которые получают ваши проекты: лаконичность, акцент на неизменяемости и расширенные возможности абстракции. Напомним отличительные черты функционального программирования еще раз.

- *Функции первого класса.* С функциями (фрагментами поведения) можно работать как со значениями. Их можно хранить в переменных, передавать в качестве параметров или возвращать из других функций. Лямбда-выражения — одна из особенностей языка Kotlin, которая позволяет считать функции элементами первого класса.
- *Неизменяемость.* Объекты разрабатываются с гарантией, что их внутреннее состояние не может измениться после того, как они будут созданы.
- *Отсутствие побочных эффектов.* Функции создаются так, чтобы они возвращали один и тот же результат при одних и тех же входных данных, не изменяя состояние других объектов или внешнего мира. Такие функции называются *чистыми*.

Рассмотрим пример, в котором показаны достоинства подхода с использованием лямбда-выражений. Представьте, что требуется определить поведение при нажатии

кнопки. Для этого объекту `Button` может потребоваться передать экземпляр соответствующего интерфейса `OnClickListener`, что позволит обработать нажатие. В данном интерфейсе будет определен один метод — `onClick`. В Kotlin это можно реализовать с помощью объявления `object` (листинг 5.1) (такой способ реализации был показан в подразделе 4.4.1).

Листинг 5.1. Реализация слушателя с помощью объявления `object`

```
button.setOnClickListener(object: OnClickListener {
    override fun onClick(v: View) {
        println("I was clicked!")
    }
})
```

Объявление такого объекта требует большого количества кода, и это будет раздражать при многократном повторении. Нотация для выражения только *поведения* — того, что должно быть сделано при нажатии, — поможет избавиться от лишнего кода. Предыдущий фрагмент кода можно переписать с помощью лямбды (листинг 5.2).

Листинг 5.2. Реализация слушателя с помощью лямбды

```
button.setOnClickListener {
    println("I was clicked!")
}
```

Этот код Kotlin реализует поведение анонимного объекта, но является более кратким и удобочитаемым. Более подробно данный пример (и то, почему его можно использовать для реализации такого интерфейса, как `OnClickListener`) мы обсудим позже в текущем разделе.

Мы показали, как лямбда может использоваться в качестве альтернативы анонимному объекту с единственным методом. Теперь продолжим и кратко рассмотрим еще одно классическое применение лямбда-выражений — работу с коллекциями.

5.1.2. Лямбды и коллекции

Как мы уже упоминали, один из главных постулатов хорошего стиля программирования — избегать дублирования в коде. Большинство задач, решаемых с помощью коллекций, подчиняются нескольким общим закономерностям. Благодаря лямбдам Kotlin может предоставить хорошую, удобную стандартную библиотеку, которая содержит мощный функционал для работы с коллекциями.

Рассмотрим пример. В нем используется класс `Person` с информацией об имени и возрасте человека:

```
data class Person(val name: String, val age: Int)
```

Предположим, у вас есть список людей и необходимо найти самого старшего из них. При отсутствии опыта работы с лямбдами любой разработчик принял бы реализовывать поиск вручную. Создал бы две промежуточные переменные — одну для хранения максимального возраста и вторую для хранения первого

найденного человека этого возраста — и затем перебирал бы список, обновляя эти переменные (листинг 5.3).

Листинг 5.3. Поиск в коллекции вручную с помощью цикла `for`

```
fun findTheOldest(people: List<Person>) {
    var maxAge = 0           ← Сохранение максимального значения возраста
    var theOldest: Person? = null ← Сохранение данных человека, в которых
                                значение возраста максимально
    for (person in people) {
        if (person.age > maxAge) { ← Если следующий человек старше
            maxAge = person.age      текущего, то происходит изменение
            theOldest = person        максимального значения
        }
    }
    println(theOldest)
}

fun main() {
    val people = listOf(Person("Alice", 29), Person("Bob", 31))
    findTheOldest(people)
    // Person(name=Bob, age=31)
}
```

При большом количестве опыта такие циклы можно создавать достаточно быстро. Но здесь довольно много кода, и легко допустить ошибки. Например, вы можете ошибиться в сравнении и найти минимальный элемент вместо максимального.

В Kotlin есть куда более удобный способ. Можно использовать функцию из стандартной библиотеки (листинг 5.4).

Листинг 5.4. Поиск по коллекции с помощью функции `maxOrNull`

```
fun main() {
    val people = listOf(Person("Alice", 29), Person("Bob", 31))
    println(people.maxOrNull { it.age }) ← Поиск максимума путем
    // Person(name=Bob, age=31)           сравнения значений возраста
}
```

Функцию `maxOrNull` можно вызвать для любой коллекции, и они принимают один аргумент: функцию с указанием, какие значения следует сравнить, чтобы найти максимальный элемент. Код в фигурных скобках `{ it.age }` — это лямбда-выражение. Оно реализует логику селектора: принимает элемент коллекции в качестве аргумента и возвращает значение для сравнения. Лямбда принимает только один аргумент (элемент коллекции), и мы не указываем для него явного имени, поэтому ссылка на него реализуется с помощью скрытого имени `it`. В данном примере элементом коллекции будет объект `Person`, а значение возраста из свойства `age` будет добавлено к сравнению.

Если лямбда просто делегирует функцию или свойство, то ее можно заменить ссылкой на член без вызова (листинг 5.5).

Листинг 5.5. Поиск с помощью ссылки на член класса без вызова

```
people.maxOrNull(Person::age)
```


Этот код повторяет листинг 5.4. Подробнее об этом мы поговорим в подразделе 5.1.5.

Большую часть работы с коллекциями можно лаконично выразить с помощью библиотечных функций, использующих лямбды или ссылки на члены без вызовов. Получившийся код намного короче, его проще понимать, и он часто передает ваши цели (то есть то, чего ваш код пытается достичь) более четко, чем его аналог на основе циклов. Чтобы помочь вам освоиться, рассмотрим синтаксис лямбда-выражений. Позже, в главе 6, мы более подробно обсудим, какие функции используются для обработки коллекций с помощью лямбд.

5.1.3. Синтаксис лямбда-выражений

Лямбда кодирует небольшой фрагмент поведения и позволяет передать его в качестве значения. Иногда такой фрагмент объявляется независимо и хранится в переменной. Но чаще всего его объявляют непосредственно при передаче в функцию. На рис. 5.1 показан синтаксис объявления лямбда-выражений.

Лямбда-выражение в Kotlin всегда окружено фигурными скобками. Обратите внимание, что вокруг аргументов нет круглых скобок. Стрелка отделяет список аргументов от тела лямбды.

Лямбда-выражение можно хранить в переменной, а затем работать с ней как с обычной функцией (вызывать ее с соответствующими аргументами):

```
fun main() {  
    val sum = { x: Int, y: Int -> x + y }  
    println(sum(1, 2))  ← Вызов лямбды, хранящейся в переменной  
    // 3  
}
```

При желании лямбда-выражение можно вызвать напрямую:

```
fun main() {  
    { println(42) }()  
    // 42  
}
```

Но такой синтаксис неудобочитаем и не имеет особого смысла (это эквивалентно прямому выполнению тела лямбды). Если требуется заключить часть кода в блок, то применяется библиотечная функция `run` — она выполняет переданную ей лямбду:

```
fun main() {  
    run { println(42) } ← Выполнение кода в лямбде  
    // 42  
}
```

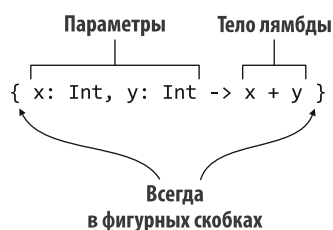


Рис. 5.1. Синтаксис лямбда-выражения: лямбда всегда окружена фигурными скобками, задает несколько параметров и предоставляет тело лямбды с фактической логикой

Функция `run` особенно полезна, когда требуется выполнить блок из нескольких операторов в точке, где ожидается выражение. Рассмотрим объявление переменной верхнего уровня. Она выполняет некоторые установки или дополнительную работу:

```
val myFavoriteNumber = run {  
    println("I'm thinking!")  
    println("I'm doing some more work...")  
    42  
}
```

В разделе 10.2 мы расскажем, почему такие вызовы — в отличие от создания лямбда-выражения и его прямого вызова через `{...}()` — не приводят к увеличению накладных расходов во время выполнения и по эффективности не уступают встроенным конструкциям языка. Вернемся к листингу 5.4. В нем выполняется поиск самого старшего человека в списке:

```
fun main() {  
    val people = listOf(Person("Alice", 29), Person("Bob", 31))  
    println(people.maxByOrNull { it.age })  
    // Person(name=Bob, age=31)  
}
```

Если переписать этот пример без использования синтаксических сокращений, то получится следующее:

```
people.maxByOrNull({ p: Person -> p.age })
```

Этот фрагмент кода должен быть вам понятен: часть кода в фигурных скобках — это лямбда-выражение, и оно передается в качестве аргумента функции. Лямбда-выражение принимает один аргумент типа `Person` и возвращает его возраст.

Но этот код очень многословен. Во-первых, слишком много знаков препинания, что ухудшает удобочитаемость. Во-вторых, тип может быть выведен из контекста, и, следовательно, его указание можно опустить. Наконец, в этом случае не надо присваивать имя аргументу лямбды.

Выполним эти улучшения. В Kotlin соглашение о синтаксисе позволяет вынести лямбда-выражение за скобки, если оно стало последним аргументом в вызове функции. В этом примере лямбда — единственный аргумент, поэтому ее можно поместить после круглых скобок:

```
people.maxByOrNull() { p: Person -> p.age }
```

Лямбда — единственный аргумент функции, поэтому пустые круглые скобки из вызова тоже можно удалить:

```
people.maxByOrNull { p: Person -> p.age }
```

Все три синтаксические формы означают одно и то же, но последняя — самая простая для чтения. Если лямбда — единственный аргумент, то ее определенно стоит написать без круглых скобок. Если функция принимает несколько аргументов и только

последний представлен лямбдой, то в Kotlin считается хорошим стилем вынести лямбду за пределы круглых скобок. Когда требуется передать несколько лямбд, вынести можно только одну, поэтому лучше оставить их все внутри круглых скобок.

Чтобы посмотреть, как выглядят эти варианты в более сложном вызове, вернемся к функции `joinToString` — мы часто использовали ее в главе 3. Она тоже определена в стандартной библиотеке Kotlin, с той лишь разницей, что в стандартной библиотеке функция принимается в качестве дополнительного параметра. Ее работа по преобразованию элемента в строку отличается от действий функции `toString`. В листинге 5.6 приведен пример, как можно использовать ее для вывода на экран только имен.

Листинг 5.6. Передача лямбды в качестве именованного аргумента

```
fun main() {
    val people = listOf(Person("Alice", 29), Person("Bob", 31))
    val names = people.joinToString(
        separator = " ",
        transform = { p: Person -> p.name }
    )
    println(names)
    // Alice Bob
}
```

Этот вызов можно переписать и вынести лямбду за круглые скобки (листинг 5.7).

Листинг 5.7. Передача лямбды за пределами круглых скобок

```
people.joinToString(" ") { p: Person -> p.name }
```

В листинге 5.6 для передачи лямбды используется именованный аргумент, и он указывает на цели использования лямбды. Листинг 5.7 более лаконичен, но в нем нет явного назначения лямбды. Поэтому такой код сложнее понять тем, кто не знаком с работой вызываемой функции.

СОВЕТ

Инструменты IntelliJ IDEA и Android Studio позволяют выполнять автоматическое преобразование двух синтаксических форм, в которых должна размещаться лямбда в качестве последнего аргумента. Чтобы преобразовать одну форму в другую, переведите указатель мыши на лямбду (рис. 5.2). Нажмите сочетание клавиш `Alt+Enter` (или `Option+Return` в macOS) или щелкните левой кнопкой мыши на плавающем желтом значке лампочки, а затем выберите пункты `Move Lambda Argument Out of Parenthes` (Переместить лямбда-аргумент за скобки) и `Move Lambda Argument Into Parenthes` (Переместить лямбда-аргумент в скобки).

Перейдем к упрощению синтаксиса и удалению типа параметра (листинг 5.8).

Листинг 5.8. Указание типа лямбда-параметра опущено

```
people.maxOrNull { p: Person -> p.age } ← Тип параметра указан в явном виде
people.maxOrNull { p -> p.age } ← Тип параметра определен
```

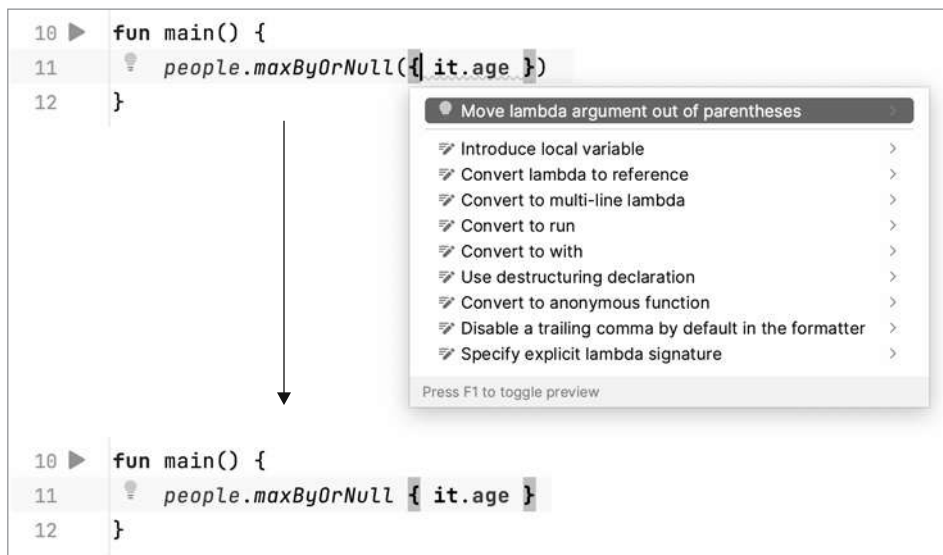


Рис. 5.2. Действие Move lambda (Переместить лямбда-аргумент) позволяет переместить лямбда-аргумент из окружающих круглых скобок

Как и в случае с локальными переменными, если тип лямбда-параметра может быть выведен, то его не нужно указывать явно. В функции `maxByOrNull` тип параметра всегда совпадает с типом элемента коллекции. Компилятор знает, что функция `maxByOrNull` вызвана для коллекции объектов класса `Person`, поэтому может сделать вывод, что параметр лямбды получит тип `Person`.

Бывают случаи, когда компилятор не может определить тип лямбда-параметра, но здесь мы их обсуждать не будем. Мы советуем запомнить простое правило: всегда начинайте писать код без типов, а если компилятор выдает сообщение о возможной ошибке, то укажите их.

Иногда требуется указать типы некоторых аргументов, а у остальных написать только имена. Это удобно, если компилятор не может вывести какой-то из типов или явное указание типа улучшает удобочитаемость.

Последнее упрощение для данного примера — заменить параметр именем параметра по умолчанию: `it` (листинг 5.9). Это имя генерируется, если в контексте ожидается лямбда с одним аргументом и ее тип может быть выведен. Все этапы упрощения кода показаны на рис. 5.3.

Листинг 5.9. Использование имени параметра по умолчанию — `it`

```
people.maxByOrNull { it.age } ← it — автоматически генерируемое имя параметра
```

Это имя по умолчанию генерируется только в том случае, если имя аргумента не было указано в явном виде.

- ❶ `people.maxByOrNull({ p: Person -> p.age })`
- ❷ `people.maxByOrNull() { p: Person -> p.age }`
- ❸ `people.maxByOrNull { p: Person -> p.age }`
- ❹ `people.maxByOrNull { p -> p.age }`
- ❺ `people.maxByOrNull { it.age }`
- ❻ `people.maxByOrNull(Person::age)`

Рис. 5.3. Упрощение вызова функции `maxByOrNull` для получения самого старшего человека из коллекции `people` в шесть шагов. ❶ Лямбда выносится за скобки. ❷ Пустая пара скобок удаляется. ❸ Выполняется сокращение явного указания типа параметра для `p`. ❹ Применяется вывод типа компилятора Kotlin. ❺ Скрытое имя `it` используется для единственного параметра лямбды. ❻ Вдобавок вы освоили дополнительное сокращение в виде ссылок на члены без вызова

ПРИМЕЧАНИЕ

Соглашение об имени `it` отлично подходит для сокращения кода, но не стоит злоупотреблять им. В частности, в случае вложенных лямбд лучше объявлять параметр каждой лямбды явно, иначе трудно понять, на какое значение ссылается `it` (и вы получите предупреждение вида `Implicit parameter it of enclosing lambda is shadowed` (Неявный параметр `it` вложенной лямбды затенен)). Кроме того, полезно объявлять параметры в явном виде, если значение или тип параметра не ясны из контекста.

При хранении лямбды в переменной контекст для выведения типов отсутствует, и поэтому их необходимо указывать в явном виде:

```
val getAge = { p: Person -> p.age }
people.maxByOrNull(getAge)
```

Пока мы приводили только примеры с лямбдами из одного выражения или инструкции. Но лямбды не ограничены таким маленьким размером и могут содержать несколько инструкций. В этом случае последнее выражение будет служить результатом — нет необходимости явно указывать оператор `return`:

```
fun main() {
    val sum = { x: Int, y: Int ->
        println("Computing the sum of $x and $y...")
        x + y
    }
    println(sum(1, 2))
    // Computing the sum of 1 and 2...
    // 3
}
```

Далее поговорим о концепции, тесно связанной с лямбда-выражениями, — о захвате переменных из контекста.

5.1.4. Доступ к переменным в области видимости

При объявлении анонимного внутреннего класса в функции можно ссылаться на параметры и локальные переменные этой функции из класса. При использовании лямбд можно поступать так же. Когда лямбда применяется в функции, можно получить доступ к параметрам этой функции, а также к локальным переменным, объявленным перед лямбдой (рис. 5.4).



Рис. 5.4. Лямбда `forEach` может обращаться к переменной `prefix` и другим переменным, определенным в окружающей области видимости, — вплоть до окружающих областей видимости классов и файлов

Для демонстрации воспользуемся стандартной библиотечной функцией `forEach`. Это одна из самых простых функций для работы с коллекциями. Она просто вызывает заданную лямбду на каждом элементе коллекции. Функция `forEach` несколько лаконичнее, чем обычный цикл `for`, но лишена многих других преимуществ, поэтому не стоит переводить все циклы в лямбды. В листинге 5.10 код принимает список сообщений и выводит каждое из них с одинаковым префиксом.

Листинг 5.10. Использование параметров функции в лямбде

```
fun printMessagesWithPrefix(messages: Collection<String>, prefix: String) {
    messages.forEach {
        println("$prefix $it")
    }
}

fun main() {
    val errors = listOf("403 Forbidden", "404 Not Found")
    printMessagesWithPrefix(errors, "Error:")
    // Error: 403 Forbidden
    // Error: 404 Not Found
}
```

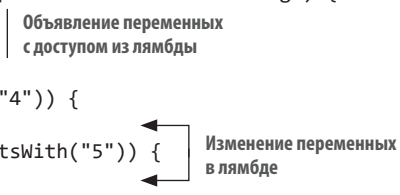
В качестве аргумента принимается лямбда, и она определяет, что делать с каждым элементом

Доступ к параметру префикса в лямбде

Одно из важных различий в использовании лямбд в Kotlin и Java заключается в том, что в Kotlin вы не ограничены доступом к конечным переменным — переменные можно изменять из лямбды. В листинге 5.11 подсчитывается количество ошибок клиента и сервера в заданном наборе кодов состояния ответа. Для этого нужно увеличить переменные `clientErrors` и `serverErrors`, определенные в функции `printProblemCounts` из лямбды `forEach`.

Листинг 5.11. Изменение локальных переменных из лямбды

```
fun printProblemCounts(responses: Collection<String>) {  
    var clientErrors = 0  
    var serverErrors = 0  
    responses.forEach {  
        if (it.startsWith("4")) {  
            clientErrors++  
        } else if (it.startsWith("5")) {  
            serverErrors++  
        }  
    }  
    println("$clientErrors client errors, $serverErrors server errors")  
}  
  
fun main() {  
    val responses = listOf("200 OK", "418 I'm a teapot",  
                           "500 Internal Server Error")  
    printProblemCounts(responses)  
    // 1 client errors, 1 server errors  
}
```



В Kotlin есть возможность обращаться к переменным без модификатора `final` в лямбде (и изменять их). В этих примерах выполняется обращение к внешним переменным (`prefix`, `clientErrors` и `serverErrors`), и этот процесс обычно называют *захватом* переменных лямбдой.

Обратите внимание, что по умолчанию время жизни локальной переменной ограничено функцией, в которой она объявлена. Но если она захвачена лямбдой, то код для работы с этой переменной можно сохранить и выполнить позже. Возникает вопрос, как это работает. Если захватывается переменная с модификатором `final`, то ее значение сохраняется вместе с использующим ее кодом лямбды. Для переменных без `final` значение заключено в специальную обертку. Она позволяет изменить переменную, а ссылка на обертку хранится вместе с лямбдой.

Есть важная оговорка: если лямбда используется в качестве обработчика событий или выполняется асинхронно, то изменения локальных переменных будут происходить только во время выполнения лямбды. Например, следующий код не может быть корректным способом подсчета нажатий кнопки:

```
fun tryToCountButtonClicks(button: Button): Int {  
    var clicks = 0  
    button.onClick { clicks++ }  
    return clicks  
}
```

Эта функция всегда будет возвращать значение `0`. Даже если обработчик `onClick` изменит значение переменной `clicks`, то данное изменение не будет отображаться, поскольку обработчик будет вызван после возврата результата функции. Правильная реализация функции должна хранить количество нажатий не в локальной переменной, а в доступном месте вне функции — например, в свойстве класса.

ДЕТАЛИ РЕАЛИЗАЦИИ ЗАХВАТА ИЗМЕНЯЕМОЙ ПЕРЕМЕННОЙ

В Java захват выполняется только для `final`-переменных. При необходимости захватить изменяемую переменную используются следующие приемы: объявление массива из одного элемента для хранения изменяемого значения; создание экземпляра класса-обертки для хранения изменяемой ссылки. Код в Kotlin с применением такого подхода выглядел бы следующим образом:

```
class Ref<T>(var value: T)  ← Класс для имитации захвата изменяемой переменной

fun main() {
    val counter = Ref(0)
    val inc = { counter.value++ } ← Формально захватывается неизменяемая
                                   переменная, но фактическое значение
                                   хранится в поле и может быть изменено
}
```

В реальном коде вам не нужно будет создавать такие обертки. Вместо этого можно изменять переменную напрямую:

```
fun main() {
    var counter = 0
    val inc = { counter++ }
}
```

Как это работает? В первом примере показана внутренняя работа второго примера. При каждом захвате `final`-переменной (`val`) ее значение копируется, как в Java. Когда захватывается изменяемая переменная (`var`), ее значение хранится как экземпляр класса `Ref`. Переменная типа `Ref` помечена как `final`, и ее можно захватывать. Хотя ее фактическое значение хранится в поле и может быть изменено из лямбды.

Мы рассмотрели синтаксис объявления лямбд и то, как переменные записываются в лямбды. Настало время обсудить ссылки на члены без вызова. Эта функциональная возможность позволяет легко передавать ссылки на существующие функции.

5.1.5. Ссылки на члены класса без вызова

Лямбды позволяют передавать блок кода в качестве параметра функции, но что, если такой передаваемый код уже определен как функция? Конечно, можно передать лямбду, вызывающую эту функцию, но это несколько избыточное действие. Можно ли передать функцию напрямую?

В Kotlin, как и в Java 8, это можно сделать, если преобразовать функцию в значение. Для этого применяется оператор `::` в следующем виде:

```
val getAge = Person::age
```

Это выражение называется *ссылкой на член класса без его непосредственного вызова* и представляет собой краткий синтаксис создания значения функции для вызова только одного метода или обращения к свойству. Двойное двоеточие отделяет имя класса от имени элемента, на который нужно сослаться (метод или свойство), как показано на рис. 5.5.

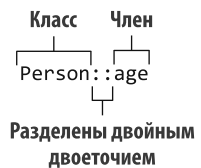


Рис. 5.5. Синтаксис ссылки на член класса без вызова

Более краткое выражение лямбды с той же логикой выглядит следующим образом:

```
val getAge = { person: Person -> person.age }
```

Обратите внимание: независимо от того, на что вы ссылаетесь — на функцию или свойство, — в ссылке на член не следует ставить круглые скобки после его имени. Ведь вы не вызываете член, а работаете со ссылкой на него.

Ссылка на член относится к тому же типу, что и лямбда для вызова этой функции или свойства, поэтому можно использовать эти два понятия как взаимозаменяемые:

```
people.maxOrNull(Person::age) | Эти вызовы
people.maxOrNull { person: Person -> person.age } | эквивалентны
```

Кроме того, можно создавать ссылку на функцию верхнего уровня (она не член класса):

```
fun salute() = println("Salute!")

fun main() {
    run(::salute) ← Ссылка на функцию верхнего уровня
    // Salute!
}
```

В таком случае имя класса необходимо опустить и поставить оператор `::` в начале выражения. Ссылка на член `::salute` передается в качестве аргумента библиотечной функции `run`, а та, в свою очередь, вызывает соответствующую функцию.

Особенно удобно применять ссылку на член без вызова, когда лямбда делегирует функцию с несколькими параметрами, — это позволяет избежать повторения имен параметров и их типов:

```
val action = { person: Person, message: String ->
    sendEmail(person, message)
}
val nextAction = ::sendEmail
```

← Эта лямбда делегирует выполнение функции `sendEmail`

← Вместо этого можно использовать ссылку на член

Сохранить или отложить действие по созданию экземпляра класса можно с помощью *ссылки на конструктор*. Она формируется путем указания имени класса после двойного двоеточия:

```
data class Person(val name: String, val age: Int)

fun main() {
    val createPerson = ::Person ← Действие по созданию экземпляра класса
    val p = createPerson("Alice", 29) | Person сохраняется в виде значения
    println(p)
    // Person(name=Alice, age=29)
}
```

Обратите внимание, что таким же образом можно ссылаться на функции-расширения:

```
fun Person.isAdult() = age >= 21
val predicate = Person::isAdult
```

Функция `isAdult` не является членом класса `Person`, но к ней можно получить доступ через ссылку так же, как и к члену экземпляра: `Person.isAdult()`.

5.1.6. Связанные вызываемые ссылки

До сих пор описанные ссылки на члены всегда указывали на член класса. С помощью связанных вызываемых ссылок можно применить тот же синтаксис ссылки на член для получения ссылки на метод конкретного экземпляра объекта. В этом примере переменная `personsAgeFunction` представляет собой обычную ссылку на член: если в качестве параметра указан объект типа `Person`, то ее вызов возвращает возраст этого человека. А переменная `sebsAgeFunction` возвращает возраст конкретного человека при вызове (и как таковая тоже не принимает аргументов):

```
fun main() {
    val seb = Person("Sebastian", 26)
    val personsAgeFunction = Person::age
    println(personsAgeFunction(seb))
    // 26
    val sebsAgeFunction = seb::age
    println(sebsAgeFunction())
    // 26
}
```

Ссылка на член для возврата значения
возраста указанного человека...

...принимает объект класса
Person в качестве аргумента

Ссылка на член возвращает значение
возраста определенного человека...

...связана с определенным объектом
и не принимает аргументов

Способ определения переменной `sebsAgeFunction` в этом примере — `seb::age` — эквивалентен явному написанию лямбды `{ seb.age }`, но более лаконичен (рис. 5.6).

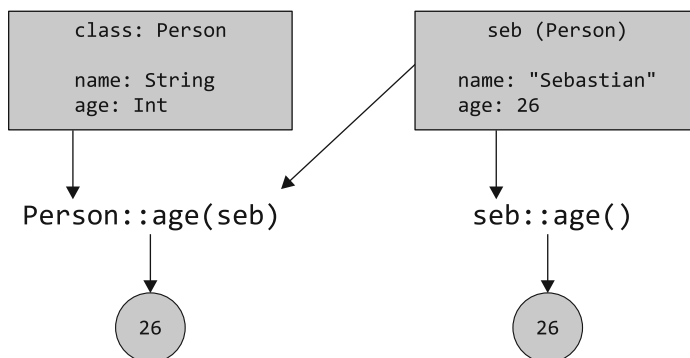


Рис. 5.6. Обычная ссылка на член, например `Person::age`, принимает экземпляр объекта в качестве параметра и возвращает значение члена. Связанная ссылка на член, например `seb::age`, не принимает никаких аргументов и возвращает значение члена, принадлежащего связанному с ней объекту

В следующем разделе мы рассмотрим множество библиотечных функций для работы с лямбда-выражениями, а также со ссылками на члены класса.

Мы подробно обсудили распространенные случаи применения лямбда-выражений — их использование в целях упрощения работы с коллекциями. Теперь затронем другую важную тему — использование лямбд с существующим API Java.

5.2. ИСПОЛЬЗОВАНИЕ ФУНКЦИОНАЛЬНЫХ ИНТЕРФЕЙСОВ JAVA — SINGLE ABSTRACT METHOD (SAM)

В экосистеме JVM уже существует множество библиотек на языке Kotlin, и они могут напрямую использовать лямбды Kotlin. Однако весьма вероятно, что вам потребуется использовать в своем проекте Kotlin библиотеку на Java. И лямбда-выражения Kotlin полностью совместимы с API Java. В данном разделе вы увидите, как именно это работает.

В начале главы был приведен пример передачи лямбды в метод Java:

```
button.setOnClickListener { ← Передача лямбды в качестве аргумента
    println("I was clicked!")
}
```

В классе `Button` задается новый слушатель кнопки с помощью метода `setOnClickListener`. Он принимает аргумент типа `OnClickListener`:

```
/* Java */
public class Button {
    public void setOnClickListener(OnClickListener l) { ... }
}
```

В интерфейсе `OnClickListener` объявлен метод `onClick`:

```
/* Java */
public interface OnClickListener {
    void onClick(View v);
}
```

Реализация интерфейса `OnClickListener` может быть довольно сложной, и все зависит от версии Java. До появления Java 8 нужно было создавать новый экземпляр анонимного класса, чтобы передать его в качестве аргумента методу `setOnClickListener`:

```
/* До Java 8 */
button.setOnClickListener(new OnClickListener() {
    @Override
    public void onClick(View v) {
        /* ... */
    }
})
```

```
/* Только начиная с Java 8 */
button.setOnClickListener(view -> { /* ... */ });
```

В Kotlin просто передается лямбда:

```
button.setOnClickListener { view -> /* ... */ }
```

С помощью лямбда-выражения необходимо указать, что в интерфейсе `OnClickListener` должен быть только один параметр `View`, как в методе `onClick`. Схема сопоставления показана на рис. 5.7.

```
public interface OnClickListener {
    void onClick(View v);
}
```

→ { view -> ... }

Рис. 5.7. Параметры лямбды соответствуют параметрам метода в интерфейсе с единственным абстрактным методом

Эта логика работает, поскольку в интерфейсе `OnClickListener` только один абстрактный метод. Такие интерфейсы называются *функциональными* или *интерфейсами с одним абстрактным методом* (Single Abstract Method, SAM). В API Java множество функциональных интерфейсов, таких как `Runnable` и `Callable`, а также методов для работы с ними. В Kotlin есть возможность использовать лямбды при вызове методов Java, принимающих в качестве параметров функциональные интерфейсы. Это позволяет коду Kotlin быть чистым и идиоматичным. Теперь посмотрим, что происходит при передаче лямбды методу, ожидающему аргумент типа функционального интерфейса.

5.2.1. Передача лямбды в качестве параметра в метод Java

Лямбду можно передать любому методу Java, ожидающему функциональный интерфейс. Для примера рассмотрим следующий метод с параметром типа `Runnable`:

```
/* Java */
void postponeComputation(int delay, Runnable computation);
```

В Kotlin его можно вызывать и передать лямбду в качестве аргумента. Компилятор автоматически преобразует ее в экземпляр `Runnable`:

```
postponeComputation(1000) { println(42) }
```

Обратите внимание, что словосочетание «экземпляр `Runnable`» подразумевает «экземпляр анонимного класса, реализующего интерфейс `Runnable`». Компилятор создаст его автоматически и будет использовать лямбду в качестве тела единственного абстрактного метода — в данном случае метода `run`.

Того же эффекта можно добиться, создав анонимный объект с открытой реализацией экземпляра `Runnable`:

```
postponeComputation(1000, object : Runnable {
    override fun run() {
        println(42)
    }
})
```

← Передача объектного выражения в качестве реализации функционального интерфейса

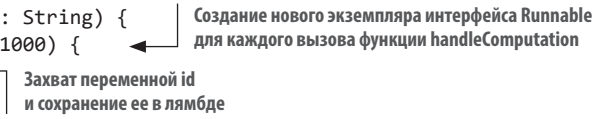
Но все же разница есть. Когда вы явно объявляете объект, при каждом обращении к нему создается новый экземпляр. С лямбдой ситуация иная: если лямбда не обращается к переменным из функции, в которой она определена, то соответствующий экземпляр анонимного класса используется в вызовах повторно:

```
postponeComputation(1000) { println(42) }
```

← Для всей программы создается один экземпляр интерфейса `Runnable`

Если лямбда захватывает переменные из окружающей области видимости, то повторно использовать один и тот же экземпляр для каждого вызова становится невозможно. В таком случае компилятор создает новый объект для каждого вызова и сохраняет значения захваченных переменных в этом объекте. Например, в следующей функции при каждом вызове используется новый экземпляр интерфейса `Runnable`. В нем значение `id` хранится в качестве поля:

```
fun handleComputation(id: String) {  
    postponeComputation(1000) {  
        println(id)  
    }  
}
```



Создание нового экземпляра интерфейса `Runnable` для каждого вызова функции `handleComputation`

Захват переменной `id` и сохранение ее в лямбде

Обратите внимание, что описание процесса создания анонимного класса и экземпляра этого класса для лямбды справедливо для методов Java, ожидающих функциональных интерфейсов. Эта информация не относится к работе с коллекциями с помощью методов расширения в Kotlin. Анонимные классы не создаются при передаче лямбды во встроенную функцию Kotlin. А большинство библиотечных функций помечены как встроенные — `inline`. Подробнее о том, как это работает, мы поговорим в разделе 10.2.

В большинстве случаев преобразование лямбды в экземпляр функционального интерфейса происходит автоматически, без каких-либо усилий с вашей стороны. Но бывают случаи, когда необходимо выполнить преобразование в явном виде. Посмотрим, как это сделать.

5.2.2. Конструкторы SAM: явное преобразование лямбд в функциональные интерфейсы

Конструктор SAM — генерируемая компилятором функция. Она выполняет явное преобразование лямбды в экземпляр интерфейса с помощью одного абстрактного метода. Ее можно использовать в ситуациях, когда компилятор не применяет автоматическое преобразование. Допустим, если есть метод для возврата экземпляра функционального интерфейса, то вы не можете возвращать лямбду напрямую — ее необходимо обернуть в конструктор SAM. Приведем простой пример (листинг 5.12).

Листинг 5.12. Использование конструктора SAM для возврата значения

```
fun createAllDoneRunnable(): Runnable {  
    return Runnable { println("All done!") }  
}  
  
fun main() {  
    createAllDoneRunnable().run()  
    // All done!  
}
```

Имя конструктора SAM совпадает с именем базового функционального интерфейса. Конструктор SAM принимает единственный аргумент — лямбду. Она будет

использоваться в качестве тела единственного абстрактного метода в функциональном интерфейсе. После он возвращает экземпляр класса, реализующего интерфейс.

Помимо возврата значений, конструкторы SAM применяются, когда требуется сохранить в переменной сгенерированный лямбдой экземпляр функционального интерфейса. Предположим, требуется повторно использовать один слушатель для нескольких кнопок, как в листинге 5.13 (в Android-приложении этот код может быть частью метода `Activity.onCreate`).

Листинг 5.13. Использование конструктора SAM для повторного применения экземпляра listener

```
val listener = OnClickListener { view ->  ← Вызов конструктора SAM с лямбдой
    val text = when (view.id) {           ← Использование переменной
        button1.id -> "First button"      view.id для определения того,
        button2.id -> "Second button"    какая кнопка была нажата
        else -> "Unknown button"
    }
    toast(text)  ← Отображение значения текста пользователю
}
button1.setOnClickListener(listener)
button2.setOnClickListener(listener)
```

Интерфейс `listener` проверяет, какая кнопка была источником события, и ведет себя соответствующим образом. Слушателя можно было определить с помощью объявления объекта, реализующего интерфейс `OnClickListener`, но конструкторы SAM предоставляют более лаконичный вариант.

Кроме того, хотя преобразование SAM в вызовах методов обычно происходит автоматически, бывают случаи, когда компилятор не может выбрать правильную перегрузку при передаче лямбды в качестве аргумента в перегруженный метод. В таких случаях непосредственное применение конструктора SAM — хороший способ устранить ошибку компиляции.

ЛЯМБДЫ И ДОБАВЛЕНИЕ/УДАЛЕНИЕ СЛУШАТЕЛЕЙ

Обратите внимание, что в лямбде нет ключевого слова `this`, как в анонимном объекте — нет способа сослаться на анонимный экземпляр класса, в который преобразуется лямбда. С точки зрения компилятора лямбда — это блок кода, а не объект, и вы не можете обращаться к ней как к объекту. Ссылка `this` в лямбде относится к окружающему классу.

Лямбду не получится применить, если слушателю события необходимо отписаться от события в процессе его обработки. Вместо этого используйте анонимный объект для реализации слушателя. В анонимном объекте ключевое слово `this` ссылается на экземпляр данного объекта, и можно передать его API для удаления слушателя.

5.3. ОПРЕДЕЛЕНИЕ ФУНКЦИОНАЛЬНЫХ ИНТЕРФЕЙСОВ В KOTLIN: FUN INTERFACE

В Kotlin допускается использовать типы функций для выражения поведения там, где в противном случае пришлось бы использовать функциональный интерфейс. В подразделе 11.3.7 мы рассмотрим способ давать более выразительные имена типам функций в Kotlin с помощью псевдонимов типов.

В некоторых случаях вам может потребоваться писать более прозрачный код. С помощью `fun interface` в Kotlin можно определять собственные функциональные интерфейсы.

В Kotlin они содержат только один абстрактный метод, но могут содержать несколько дополнительных неабстрактных методов. Это поможет выразить более сложные конструкции, не вмещающиеся в сигнатуру типа функции.

В листинге 5.14 приводится определение функционального интерфейса `IntCondition` с абстрактным методом `check`. В коде определен дополнительный, неабстрактный метод `checkString`. Он вызывает метод `check` после преобразования параметра в целое число. Как и в случае с SAM в Java, конструктор SAM используется для создания экземпляра интерфейса с лямбдой, определяющей реализацию метода `check`.

Листинг 5.14. Функциональный интерфейс в Kotlin содержит только один абстрактный метод

```
fun interface IntCondition {  
    fun check(i: Int): Boolean  
    fun checkString(s: String) = check(s.toInt())  
    fun checkChar(c: Char) = check(c.digitToInt())  
}  
  
fun main() {  
    val isOdd = IntCondition { it % 2 != 0 }  
    println(isOdd.check(1))  
    // истина  
    println(isOdd.checkString("2"))  
    // ложь  
    println(isOdd.checkChar('3'))  
    // истина  
}
```

Только один абстрактный метод...

...и дополнительные неабстрактные методы

Когда функция принимает параметр с типом `fun interface`, можно снова предоставить реализацию лямбды напрямую или передать ссылку на лямбду. В обоих случаях реализация интерфейса будет создаваться динамически.

В листинге 5.15 определяется функция `checkCondition`. Она принимает объявленный ранее интерфейс `IntCondition`. После этого будет доступно несколько вариантов вызова этой функции. Например, передача лямбды напрямую или передача ссылки

на функцию с правильным типом `((Int) -> Boolean)` — подробнее об этом синтаксисе и о лямбдах в целом рассказывается в разделе 10.1).

Листинг 5.15. Функциональный интерфейс создается динамически

```
fun checkCondition(i: Int, condition: IntCondition): Boolean {
    return condition.check(i)
}

fun main() {
    checkCondition(1) { it % 2 != 0 } ← Либо лямбда передается напрямую...
    // истина
    val isOdd: (Int) -> Boolean = { it % 2 != 0 }
    checkCondition(1, isOdd) ← ...либо передается ссылка на лямбду
    // истина                                     с подходящей сигнатурой
}
```

Как правило, простые функциональные типы хорошо работают, если API может принять любую функцию с определенным набором параметров, которая возвращает определенный тип. Если требуется выразить более сложные контракты или операции, недоступные для описания в функциональной сигнатуре типов, то функциональный интерфейс — хороший выбор.

ОЧИСТКА ТОЧЕК ВЫЗОВОВ JAVA С ПОМОЩЬЮ ФУНКЦИОНАЛЬНЫХ ИНТЕРФЕЙСОВ

Если вы пишете код для дальнейшего использования в Java и Kotlin, то применение `fun interface` поможет улучшить чистоту точек вызова Java. Типы функций Kotlin передаются как объекты, обобщенные типы которых представлены параметрами и возвращаемыми типами. Для функций без возвращаемых значений в Kotlin используется ключевое слово `Unit` — аналог `void` в Java (о необходимости и полезности типа `Unit` подробнее поговорим в подразделе 8.1.6).

Это означает еще и то, что, когда такой тип функции Kotlin вызывается из Java, вызывающая сторона должна вернуть `Unit.INSTANCE` в явном виде. Использование `fun interface` снимает это требование, и точка вызова становится более лаконичной. В данном примере функции `consumeHello` и `consumeHelloFunctional` выполняют одну и ту же логику. Но одна определена с помощью функционального интерфейса, а вторая — с использованием функциональных типов Kotlin:

```
fun interface StringConsumer {
    fun consume(s: String)
}

fun consumeHello(t: StringConsumer) {
    t.consume("Hello")
}

fun consumeHelloFunctional(t: (String) -> Unit) {
    t("Hello")
}
```


При использовании из Java вариант функции с `fun interface` вызывается с помощью простой лямбды, а вариант с функциональными типами Kotlin требует, чтобы лямбда явно возвращала `Unit.INSTANCE` из Kotlin:

```
import kotlin.Unit;

public class MyApp {
    public static void main(String[] args) {
        /* Java */
        MainKt.consumeHello(s -> System.out.println(s.toUpperCase()));
        MainKt.consumeHelloFunctional(s -> {
            System.out.println(s.toUpperCase());
            return Unit.INSTANCE;
        });
    }
}
```

Использование функциональных типов Kotlin из Java может потребовать возврата экземпляра `Unit.INSTANCE` в явном виде

Позже мы обсудим использование типов функций в объявлениях функций (см. раздел 10.1) и рассмотрим псевдонимы типов в Kotlin (см. подраздел 11.3.7).

Завершая обсуждение синтаксиса и использования лямбд, рассмотрим лямбды с приемниками и то, как с их помощью определять удобные библиотечные функции, похожие на встроенные конструкции.

5.4. ЛЯМБДЫ С ПРИЕМНИКАМИ: WITH, APPLY И ALSO

В этом разделе представлены функции `with`, `apply` и `also` из стандартной библиотеки Kotlin. Они очень удобны, и вы найдете им множество применений, даже не понимая, как они объявляются. В подразделе 13.2.1 будет показано, как можно объявить подобные функции для собственных нужд. Однако в текущем разделе мы дадим пояснения, которые помогут вам познакомиться с уникальной особенностью лямбд Kotlin, недоступной в Java, — с возможностью вызывать методы другого объекта в теле лямбды без каких-либо дополнительных уточнений. Такие лямбды называются *лямбдами с приемниками*. Для начала рассмотрим функцию `with` — она использует такую лямбду.

5.4.1. Выполнение нескольких операций над одним объектом: функция `with`

Во многих языках есть специальные операторы, с помощью которых можно выполнять несколько операций над одним и тем же объектом, не повторяя его имени. В Kotlin тоже есть такая возможность, но она предоставляется в виде библиотечной функции `with`, а не как специальная языковая конструкция. Рассмотрим следующий пример, чтобы оценить полезные стороны такого подхода (листинг 5.16). А после выполним рефакторинг этого кода с помощью функции `with`.

Листинг 5.16. Создание алфавита

```

fun alphabet(): String {
    val result = StringBuilder()
    for (letter in 'A'..'Z') {
        result.append(letter)
    }
    result.append("\nNow I know the alphabet!")
    return result.toString()
}

fun main() {
    println(alphabet())
    // ABCDEFGHIJKLMNOPQRSTUVWXYZ
    // Now I know the alphabet!
}

```

В этом примере вызывается несколько различных методов для экземпляра `result`, а имя `result` повторяется в каждом вызове. Это не слишком плохо, но что, если бы используемое выражение было длиннее или повторялось большее количество раз? Переписать код с помощью функции `with` можно следующим образом (листинг 5.17).

Листинг 5.17. Использование функции `with` для создания алфавита

```

fun alphabet(): String {
    val stringBuilder = StringBuilder()
    return with(stringBuilder) {
        for (letter in 'A'..'Z') {
            this.append(letter)
        }
        this.append("\nNow I know the alphabet!")
        this.toString()
    }
}

```

Указание значения приемника, для которого будут вызваны методы

`stringBuilder` становится ссылкой `this...`

...чтобы можно было вызывать методы, например `append`

Возврат значения из функции `with`

Структура `with` выглядит как специальная конструкция, но это функция, принимающая два аргумента: в данном случае `stringBuilder` и лямбду. Соглашение о вынесении лямбды за круглые скобки здесь работает, и весь вызов выглядит как встроенная функциональность языка. В качестве альтернативы можно написать `with(stringBuilder, { ... })`, но такой формат неудобочитаем.

Функция `with` преобразует свой первый аргумент в *приемник* лямбды, переданной в качестве второго аргумента. Получить доступ к этому приемнику можно через явную ссылку `this`. В качестве альтернативы можно опустить ссылку `this` и получить доступ к методам или свойствам этого значения без дополнительных квалификаторов (рис. 5.8).

```

val stringBuilder = StringBuilder()
with(stringBuilder) { this: StringBuilder
    // ...
}

```

Рис. 5.8. Внутри лямбды функции `with` первый аргумент доступен в виде типа приемника `this`. В таких IDE, как IntelliJ IDEA и Android Studio, есть возможность визуализировать этот тип приемника с помощью внутритекстовой подсказки после открывающей круглой скобки

В листинге 5.17 ссылка `this` относится к переменной `stringBuilder`, она передается в функцию `with` в качестве первого аргумента. К методам `stringBuilder` можно обращаться через явные ссылки `this`, как в `this.append(letter)`, или напрямую, что сделает код еще более лаконичным (листинг 5.18).

Листинг 5.18. Внутри лямбды функции `with` не нужно указывать ссылку `this` в явном виде

```
fun alphabet(): String {  
    val stringBuilder = StringBuilder()  
    return with(stringBuilder) {  
        for (letter in 'A'..'Z') {  
            append(letter)  
        }  
        append("\nNow I know the alphabet!")  
        toString()  
    }  
}
```

Внутри лямбды
можно опустить
указание ссылки `this`

ЛЯМБДЫ С ПРИЕМНИКОМ И ФУНКЦИЯМИ-РАСШИРЕНИЯМИ

Возможно, вы помните похожую концепцию, когда ссылка `this` указывала на функцию-приемник. В теле функции-расширения `this` ссылается на экземпляр типа, который расширяет функция. В таком случае ссылку можно опустить, чтобы получить прямой доступ к членам приемника.

Обратите внимание: функция-расширение — это в некотором смысле функция с приемником. Можно провести следующую аналогию:

Обычная функция	Обычная лямбда
Функция-расширение	Лямбда с приемником

Лямбда — способ определить поведение, аналогичное обычной функции. Лямбда с приемником — способ определить поведение, подобное функции расширения.

Выполним дополнительный рефакторинг исходной функции `alphabet` и избавимся от лишней переменной `stringBuilder` (листинг 5.19).

Листинг 5.19. Использование функции `with` и тела выражения для создания алфавита

```
fun alphabet() = with(StringBuilder()) {  
    for (letter in 'A'..'Z') {  
        append(letter)  
    }  
    append("\nNow I know the alphabet!")  
    toString()  
}
```

Теперь эта функция возвращает только выражение, поэтому переписана с использованием синтаксиса тела выражения. Создается новый экземпляр интерфейса `StringBuilder`, и он будет непосредственно передан в качестве аргумента. После этого можно ссылаться на него без явного указания `this` в лямбде.

КОНФЛИКТЫ ИМЕН МЕТОДОВ

Что произойдет, если в объекте, передаваемом в качестве параметра в функцию `with`, есть метод с тем же именем, что и у класса, в котором используется `with`? В таком случае стоит добавить явную метку к ссылке `this`, чтобы указать, какой метод необходимо вызвать.

Представьте, что функция `alphabet` — это метод класса `OuterClass`. Если требуется обратиться к методу `toString` из внешнего класса, а не из интерфейса `StringBuilder`, то сделать это можно, используя следующий синтаксис:

```
this@OuterClass.toString()
```

Возвращаемое функцией `with` значение — результат выполнения лямбда-кода. Результатом будет последнее выражение в лямбде. Но иногда нужно, чтобы вызов возвращал объект-приемник, а не результат выполнения лямбды. Именно в таком случае может пригодиться библиотечная функция `apply`.

5.4.2. Инициализация и конфигурирование объектов: функция `apply`

Функция `apply` работает аналогично функции `with`. Основное различие заключается в том, что `apply` всегда возвращает объект, переданный ей в качестве аргумента (иными словами — объект-приемник). Снова выполним рефакторинг функции `alphabet` и на этот раз используем функцию `apply` (листинг 5.20).

Листинг 5.20. Использование функции `apply` для создания алфавита

```
fun alphabet() = StringBuilder().apply {
    for (letter in 'A'..'Z') {
        append(letter)
    }
    append("\nNow I know the alphabet!")
}.toString()
```

Функцию `apply` можно вызвать в качестве расширения для любого типа. В данном случае она вызывается для только что созданного экземпляра интерфейса `StringBuilder`. Объект, для которого вызывается `apply`, становится приемником лямбды и передается в качестве аргумента (рис. 5.9). Результатом выполнения функции `apply` будет значение типа `StringBuilder`, поэтому необходимо вызвать метод `toString`, чтобы в дальнейшем результат можно было преобразовать в тип данных `String`.

```
val result : StringBuilder = StringBuilder().apply { this: StringBuilder
    // . . .
}
```

Рис. 5.9. Функция `apply`, как и `with`, делает объект, для которого она была вызвана, типом-приемником внутри лямбды и возвращает его в качестве результата. Внутритекстовые подсказки в IntelliJ IDEA и Android Studio помогают визуализировать этот процесс

Зачастую такой подход оказывается полезным при создании экземпляра объекта, когда необходимо сразу же инициализировать некоторые свойства. В Java для этого обычно используется отдельный объект `Builder`, а в Kotlin можно использовать функцию `apply` для любого объекта без какой-либо специальной поддержки со стороны библиотеки.

Рассмотрим пример, в котором функция `apply` используется в таких случаях. В нем создается компонент `TextView` для среды Android с некоторыми пользовательскими атрибутами (листинг 5.21).

Функция `apply` позволяет использовать компактный стиль тела выражения. Новый экземпляр компонента `TextView` создается и немедленно передается в `apply`. В лямбде, переданной в функцию `apply`, экземпляр `TextView` становится приемником. Поэтому для него можно вызывать методы и устанавливать свойства. После выполнения лямбды функция `apply` возвращает инициализированный экземпляр — он становится результатом функции `createViewWithCustomAttributes`.

Листинг 5.21. Использование функции `apply` для инициализации компонента `TextView`

```
fun createViewWithCustomAttributes(context: Context) =
    TextView(context).apply {
        text = "Sample Text"
        textSize = 20.0
        setPadding(10, 0, 0, 0)
    }
```

Функции `with` и `apply` представляют собой базовые примеры использования лямбд с приемниками. Более специфические функции тоже могут использовать этот паттерн. Например, функцию `alphabet` можно еще больше упростить с помощью функции `buildString` из стандартной библиотеки (листинг 5.22). Она автоматически создаст экземпляр `StringBuilder` и вызовет метод `toString`. Аргументом `buildString` будет лямбда с приемником, а приемником всегда становится `StringBuilder`.

Листинг 5.22. Использование функции `buildString` для создания алфавита

```
fun alphabet() = buildString {
    for (letter in 'A'..'Z') {
        append(letter)
    }
    append("\nNow I know the alphabet!")
}
```

Функция `buildString` — хорошее решение для задачи по созданию значения типа `String` с помощью `StringBuilder`. Кроме того, в стандартную библиотеку Kotlin входят функции создания коллекций. Они позволяют создавать доступные только для чтения коллекции, `List`, `Set` или `Map`, притом на этапе создания коллекции можно рассматривать ее как изменяемую (листинг 5.23).

Листинг 5.23. Использование функций `buildList`, `buildSet` и `buildMap` для создания коллекций

```
val fibonacci = buildList {
    addAll(listOf(1, 1, 2))
    add(3)
```

```

    add(index = 0, element = 3)
}

val shouldAdd = true

val fruits = buildSet {
    add("Apple")
    if (shouldAdd) {
        addAll(listOf("Apple", "Banana", "Cherry"))
    }
}

val medals = buildMap<String, Int> {
    put("Gold", 1)
    putAll(listOf("Silver" to 2, "Bronze" to 3))
}

```

5.4.3. Выполнение дополнительных действий с объектом: функция `also`

Функцию `also`, как и `apply`, можно использовать, чтобы принять объект-приемник, выполнить над ним действие, а затем вернуть его (рис. 5.10). Основное различие заключается в том, что внутри лямбды `also` обращение к объекту-приемнику осуществляется как к аргументу: ему либо задается имя, либо используется имя по умолчанию `it`. Благодаря этому функция `also` хорошо подходит для выполнения действий, которые принимают исходный объект-приемник в качестве аргумента (в отличие от операций, работающих со свойствами и функциями объекта). Если в коде вы встречаете `also`, то можете интерпретировать логику этого фрагмента как выполнение дополнительных эффектов: «... а также сделайте с объектом следующее».

```

val x: List<Int> = listOf(1, 2, 3).also { it: List<Int>
    // . . .
}

```

Рис. 5.10. При использовании `also` объект не становится типом-приемником внутри лямбды, но будет доступным в качестве аргумента с именем по умолчанию `it`. Функция `also` возвращает объект, для которого она была вызвана. Это подтверждают внутритекстовые подсказки

В следующем примере (листинг 5.24) коллекция фруктов сопоставляется с их названиями в верхнем регистре. Результат этого сопоставления добавляется в дополнительную коллекцию. Затем из коллекции отфильтровываются фрукты с названиями длиннее пяти символов. Этот результат также выводится, после чего порядок записей в списке будет изменен на обратный.

Листинг 5.24. Использование функции `also` для реализации дополнительных эффектов

```

fun main() {
    val fruits = listOf("Apple", "Banana", "Cherry")
    val uppercaseFruits = mutableList<String>()
    val reversedLongFruits = fruits
        .map { it.uppercase() }
}

```

```
.also { uppercaseFruits.addAll(it) }  
.filter { it.length > 5 }  
.also { println(it) }  
.reversed()  
// [BANANA, CHERRY]  
println(uppercaseFruits)  
// [APPLE, BANANA, CHERRY]  
println(reversedLongFruits)  
// [CHERRY, BANANA]  
}
```

Более интересные примеры использования лямбд с приемниками будут показаны в разделе 13.2, когда мы перейдем к обсуждению предметно-ориентированных языков (domain-specific languages, DSL). Лямбды с приемниками — отличный инструмент для создания DSL. Мы покажем, как использовать их для этой цели и как определять собственные функции, вызывающие лямбды с приемниками.

РЕЗЮМЕ

- Лямбды позволяют передавать фрагменты кода функциям в качестве аргументов, благодаря чему удобно извлекать общие структуры кода.
- Kotlin дает возможность передавать лямбды в функции за круглыми скобками, чтобы сделать код чистым и лаконичным.
- Если лямбда принимает только один параметр, то на нее можно ссылаться с помощью скрытого имени `it`. Это избавит вас от необходимости присваивать имя единственному параметру в коротких и простых лямбдах.
- Лямбды могут *захватывать* внешние переменные. Это означает, что лямбды можно использовать, например, для доступа к переменным и изменения их в функции, содержащей вызов лямбды.
- Создавать ссылки на методы, конструкторы и свойства можно с помощью префикса `::` к имени. Такие ссылки передаются в функции вместо лямбд в качестве сокращения.
- Чтобы реализовать интерфейсы с одним абстрактным методом (так называемые SAM или функциональные интерфейсы), можно просто передавать лямбды, вместо того чтобы создавать объект для реализации интерфейса в явном виде.
- *Лямбды с приемниками* — это лямбды, имеющие возможность напрямую вызывать методы специального объекта приемника. Тело таких лямбд выполняется в контексте, который отличается от окружающего кода, поэтому они помогают структурировать код.
- Функция `with` из стандартной библиотеки позволяет вызывать несколько методов для одного объекта, не повторяя ссылки на объект. Функция `apply` дает возможность сконструировать и инициализировать любой объект, используя API в стиле конструктора. С помощью функции `also` можно выполнять дополнительные действия с объектом.

Работа с коллекциями и последовательностями

В этой главе

- ✓ Работа с коллекциями в функциональном стиле.
- ✓ Последовательности: отложенное выполнение операций коллекций.

В главе 5 рассказывалось о лямбдах как о способе передачи небольших блоков кода другим функциям. Одно из самых распространенных применений лямбд — *работа с коллекциями*. В этой главе показано, как замена распространенных шаблонов доступа к коллекции комбинацией функций стандартной библиотеки и собственных лямбд может сделать код более выразительным, элегантным и лаконичным. Это касается задач фильтрации данных на основе предикатов, необходимости группировки данных или преобразования элементов коллекции из одной формы в другую.

Кроме того, вы изучите последовательности как альтернативный способ эффективного применения нескольких операций сбора, не требующий особых накладных расходов. Вы узнаете, в чем разница между *немедленным* и *отложенным* выполнением операций коллекций в Kotlin и как использовать оба варианта в своих программах.

6.1. ФУНКЦИОНАЛЬНЫЕ API ДЛЯ КОЛЛЕКЦИЙ

Функциональный стиль программирования дает множество преимуществ при работе с коллекциями. Для большинства задач можно использовать функции из стандартной библиотеки и настраивать их поведение, передавая лямбды в качестве аргументов этим функциям. По сравнению с переходами по коллекциям

и агрегированием данных вручную благодаря этой возможности вы можете выражать распространенные операции последовательно с помощью словаря функций. Он используется совместно с другими разработчиками Kotlin.

В этом разделе мы рассмотрим некоторые функции для работы с коллекциями из стандартной библиотеки Kotlin. Мы начнем с таких основных функций, как `filter` и `map` для преобразования коллекций, а также с концепций, лежащих в основе этих функций. Кроме того, мы расскажем о других полезных функциях и дадим советы о том, как избежать их чрезмерного использования и что поможет писать четкий и понятный код.

Обратите внимание: эти функции не были придуманы разработчиками Kotlin. Они, как и другие подобные функции, доступны для всех языков, поддерживающих лямбда-выражения, в том числе C#, Groovy и Scala. Если эти концепции вам уже знакомы, то можете быстро просмотреть следующие примеры и пропустить объяснения.

6.1.1. Удаление и преобразование элементов: функции `filter` и `map`

Функции `filter` и `map` лежат в основе работы с коллекциями. С их помощью можно выразить многие операции коллекций. Если перед вами стоит задача отфильтровать коллекцию на основе определенного предиката или преобразовать каждый ее элемент в другую форму, то эти функции окажутся полезными.

Для каждой функции приведем один пример с числами и один с использованием уже знакомого вам класса `Person`:

```
data class Person(val name: String, val age: Int)
```

Функция `filter` проходит через коллекцию и выбирает элементы, для которых заданная лямбда возвращает значение `true`. Например, эта функция поможет извлечь из списка чисел только четные (остаток деления на 2 равен 0):

```
fun main() {  
    val list = listOf(1, 2, 3, 4)  
    println(list.filter { it % 2 == 0 }) ← Остались только четные числа  
    // [2, 4]  
}
```

В результате получается новая коллекция. В ней содержатся элементы входной коллекции, удовлетворяющие предикату (рис. 6.1).

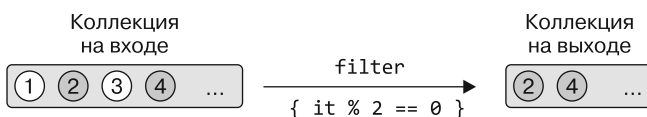


Рис. 6.1. Функция `filter` выбирает элементы, соответствующие заданному предикату

Если требуется получить коллекцию, состоящую из данных только о людях старше 30 лет, то следует снова использовать функцию `filter`:

```
fun main() {
    val people = listOf(Person("Alice", 29), Person("Bob", 31))
    println(people.filter { it.age > 30 })
    // [Person(name=Bob, age=31)]
}
```

Эта функция может создать новую коллекцию элементов, соответствующих заданному предикату. Но она *не преобразует* элементы в процессе — результатом по-прежнему будет коллекция объектов класса `Person`. Можно считать, что записи «извлекаются» из коллекции без изменения их типа.

Для сравнения: функция `map` позволяет преобразовывать элементы входной коллекции. Она применяет заданную функцию к каждому элементу коллекции и собирает возвращаемые значения в новую коллекцию. С ее помощью можно преобразовать список чисел в список их квадратов, как в следующем примере:

```
fun main() {
    val list = listOf(1, 2, 3, 4)
    println(list.map { it * it })
    // [1, 4, 9, 16]
}
```

В результате получается новая коллекция. Она хранит то же количество элементов, но каждый элемент преобразован в соответствии с заданной предикатной функцией (рис. 6.2).

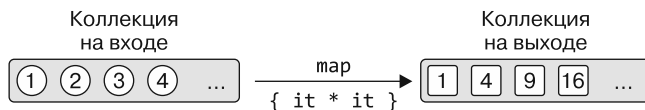


Рис. 6.2. Функция `map` применяет лямбду ко всем элементам коллекции

Если требуется вывести только список имен, а не список всех данных людей, то можно преобразовать его с помощью функции `map`. Это действие превращает список объектов класса `Person` в список объектов типа `String`, представляющих их имена. И такой список можно легко вывести на экран:

```
fun main() {
    val people = listOf(Person("Alice", 29), Person("Bob", 31))
    println(people.map { it.name })
    // [Alice, Bob]
}
```

Обратите внимание, что этот пример можно отлично переписать, используя ссылки на члены:

```
people.map(Person::name)
```

Кроме того, можно составить цепочку из нескольких подобных вызовов. Например, найдем имена людей старше 30 лет:

```
println(people.filter { it.age > 30 }.map(Person::name))  
// [Bob]
```

Допустим, требуется найти самых старших людей в группе. Можно найти максимальный возраст людей в группе и вернуть данные всех, кто соответствует этому возрасту (или значение `null`, если таких людей не существует; подробнее о `null`-безопасности рассказывается в главе 7). Такой код легко написать с помощью лямбд:

```
people.filter {  
    val oldestPerson = people.maxByOrNull(Person::age)  
    it.age == oldestPerson?.age  
}
```

Но обратите внимание, что этот код повторяет процесс поиска максимального значения возраста для каждого человека. Поэтому, если в коллекции 100 человек, поиск максимального возраста будет выполнен 100 раз!

Следующее решение улучшает эту задачу и вычисляет максимальный возраст всего один раз:

```
val maxAge = people.maxByOrNull(Person::age)?.age  
people.filter { it.age == maxAge }
```

Не стоит повторять вычисления, если в этом нет необходимости! Простой на вид код, в котором используются лямбда-выражения, иногда может скрывать сложность базовых операций. Всегда помните о том, что происходит в создаваемом вами коде.

Если операции фильтрации и преобразования зависят от индекса элементов в дополнение к их фактическим значениям, то можно применить родственные функции `filterIndexed` и `mapIndexed`. Они предоставляют лямбде индекс элемента (начиная с нуля), а также сам элемент. В примере ниже список чисел фильтруется так, чтобы в нем остались только значения больше 3 и с четным индексом. Кроме того, сопоставляется второй список, чтобы можно было суммировать номер индекса и числовое значение каждого элемента:

```
fun main() {  
    val numbers = listOf(1, 2, 3, 4, 5, 6, 7)  
    val filtered = numbers.filterIndexed { index, element ->  
        index % 2 == 0 && element > 3  
    }  
    println(filtered)  
    // [5, 7]  
  
    val mapped = numbers.mapIndexed { index, element ->  
        index + element  
    }  
    println(mapped)  
    // [1, 3, 5, 7, 9, 11, 13]  
}
```

К картам тоже можно применять функции фильтрации и преобразования:

```
fun main() {
    val numbers = mapOf(0 to "zero", 1 to "one")
    println(numbers.mapValues { it.value.uppercase() })
    // {0=ZERO, 1=ONE}
}
```

Для работы с ключами и значениями существуют отдельные функции: `filterKeys` и `mapKeys` фильтруют и преобразуют ключи карты, а `filterValues` и `mapValues` — соответствующие значения.

6.1.2. Накопление значений для коллекций: функции `reduce` и `fold`

Функции `reduce` и `fold`, в дополнение к `filter` и `map`, представляют собой еще два важных базовых элемента при работе с коллекциями в функциональном стиле. Эти функции используются для *объединения* информации из коллекции — они возвращают одно значение из заданной коллекции. Это значение постепенно накапливается в аккумуляторе. Лямбда вызывается для каждого элемента и должна вернуть новое значение для аккумулятора.

При использовании функции `reduce` выполнение начинается с первого элемента коллекции в аккумуляторе (поэтому не вызывайте ее для пустой коллекции!). Затем вызывается лямбда с аккумулятором и вторым элементом. В примере ниже функция `reduce` суммирует и умножает значения входной коллекции соответственно (аналогичное поведение показано на рис. 6.3):

```
fun main() {
    val list = listOf(1, 2, 3, 4)
    val summed = list.reduce { acc, element ->
        acc + element
    }
    println(summed)
    // 10
    val multiplied = list.reduce { acc, element ->
        acc * element
    }
    println(multiplied)
    // 24
}
```

Функция `fold` концептуально очень похожа на `reduce`, но вместо того, чтобы сохранять первый элемент коллекции в аккумулятор в самом начале, можно выбрать произвольное начальное значение. В примере выше выполняется конкатенация свойств `name` двух объектов класса `Person` с помощью функции `fold` (эту задачу обычно выполняет описанная ранее функция `joinToString`, и тем не менее это наглядный пример).

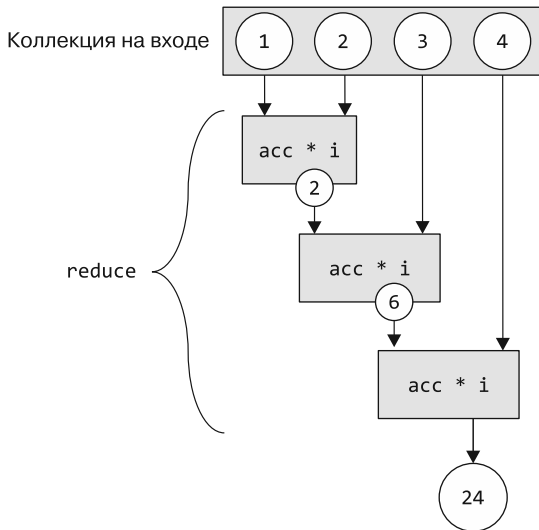


Рис. 6.3. Функция `reduce` постепенно накапливает результат в своем аккумуляторе, вызывая лямбду несколько раз с каждым элементом и предыдущим значением аккумулятора

Аккумулятор инициализируется пустой строкой. Позже окончательный текст постепенно сформируется в лямбде (это поведение показано на рис. 6.4):

```
fun main() {
    val people = listOf(
        Person("Alex", 29),
        Person("Natalia", 28)
    )
    val folded = people.fold("") { acc, person ->
        acc + person.name
    }
    println(folded)
}
// AlexNatalia
```

Существует множество алгоритмов, которые можно выразить в краткой форме с помощью операции `fold` или `reduce`. Когда необходимо получить все периодические значения операций `reduce` или `fold`, используют функции `runningReduce` и `runningFold`. Единственное их отличие от рассмотренных ранее функций `reduce` и `fold` заключается в том, что эти функции возвращают список. Он содержит все периодические значения аккумулятора вместе с конечным результатом. В примере ниже применяется `running`-аналог фрагментов кода, приведенных в предыдущих разделах:

```
fun main() {
    val list = listOf(1, 2, 3, 4)
    val summed = list.runningReduce { acc, element ->
        acc + element
    }
}
```

```

println(summed)
// [1, 3, 6, 10]
val multiplied = list.runningReduce { acc, element ->
    acc * element
}
println(multiplied)
// [1, 2, 6, 24]
val people = listOf(
    Person("Alex", 29),
    Person("Natalia", 28)
)
println(people.runningFold("") { acc, person ->
    acc + person.name
})
// [, Alex, AlexNatalia]
}

```

Конечный результат один и тот же, но выполняемые функции возвращают и все промежуточные значения

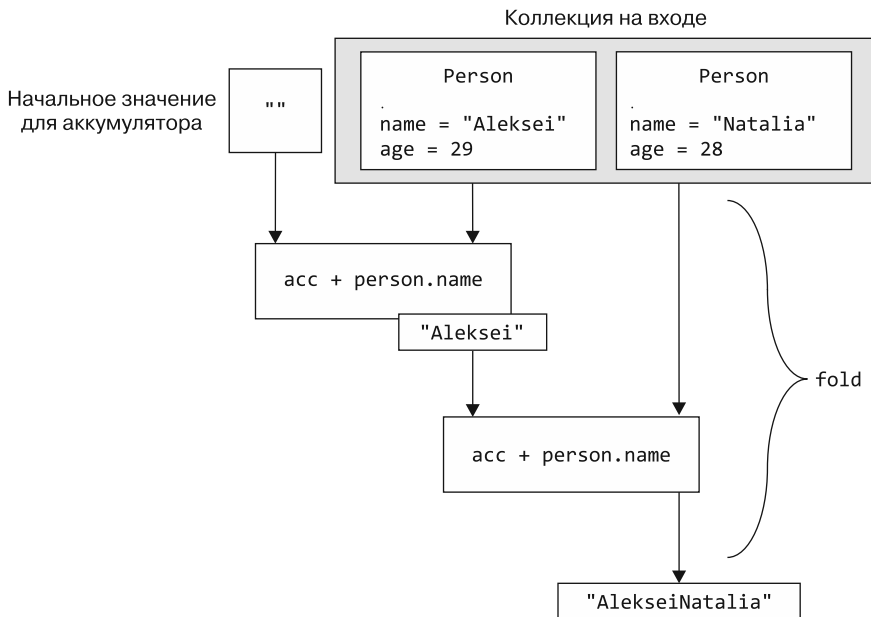


Рис. 6.4. Функция fold позволяет задать начальное значение и тип аккумулятора. Результат постепенно накапливается в аккумуляторе, многократно применяя лямбду к каждой паре «аккумулятор — элемент»

6.1.3. Применение предиката к коллекции: функции all, any, none, count и find

Другая распространенная задача — проверка соответствия всех или некоторых элементов в коллекции либо ни одного из них определенному условию. В Kotlin для этого используются функции all, any и none. Функция count проверяет, сколько элементов удовлетворяют предикату, а find возвращает первый совпадающий элемент.

Продemonстрируем работу этих функций и сначала определим предикат `canBeInClub27` для проверки возраста человека — ему должно быть 27 лет или меньше:

```
val canBeInClub27 = { p: Person -> p.age <= 27 }
```

Если вас интересует, *все* ли элементы удовлетворяют этому предикату, то необходимо применить функцию `all`:

```
fun main() {  
    val people = listOf(Person("Alice", 27), Person("Bob", 31))  
    println(people.all(canBeInClub27))  
    // ложь  
}
```

Если требуется проверить, есть ли хотя бы один подходящий элемент, то используйте функцию `any`:

```
fun main() {  
    println(people.any(canBeInClub27))  
    // истина  
}
```

Стоит отметить, что функцию `!all` («не все») с условием можно заменить функцией `any` с отрицанием условия и наоборот. Чтобы улучшить удобочитаемость кода, стоит выбирать функции без знака отрицания:

```
fun main() {  
    val list = listOf(1, 2, 3)  
    println(!list.all { it == 3 })  
    // истина  
    println(list.any { it != 3 })  
    // истина  
}
```

Отрицание функции (!) не привлекает внимания, поэтому в таком случае лучше использовать функцию `any`

Условие в аргументе заменено противоположным

Первая проверка гарантирует, что не все элементы равны 3. Это то же самое, что и наличие хотя бы одного значения, не равного 3, и именно это условие проверяется с помощью функции `any` во второй строке.

Аналогичным образом можно заменить `!any` на `none`:

```
fun main() {  
    val list = listOf(1, 2, 3)  
    println(!list.any { it == 4 })  
    // истина  
    println(list.none { it == 4 })  
    // истина  
}
```

Вместо того чтобы вызывать функцию `any` и отрицать ее результат...

...можно использовать функцию `none` с тем же условием

Первый вызов проверяет, равны ли какие-либо элементы коллекции числу 4, и отрицает данный результат. Более естественный способ выразить это как в коде, так и в словах, — убедиться, что ни один элемент не равен 4.

Поначалу такая ситуация может удивить, но при дальнейшем изучении вы поймете, что это очень разумное возвращаемое значение. Нельзя назвать элемент,

нарушающий предикат, поэтому предикат явно должен быть истинным для *всех* элементов коллекции — даже если их нет! Данная концепция известна как «*пустая истина*». В большинстве случаев она хорошо подходит для условных выражений, которые должны работать и с пустыми коллекциями.

ПРЕДИКАТЫ И ПУСТЫЕ КОЛЛЕКЦИИ

Читая описание различных типов предикатов — `any`, `none` и `all`, — вы могли задаться вопросом, что возвращают эти функции при вызове для пустых коллекций. Разберемся в данном вопросе.

При вызове `any` коллекция не содержит элементов, удовлетворяющих данному предикату. В таком случае код возвращает значение `false`:

```
fun main() {
    println(emptyList<Int>().any { it > 42 })
    // ложь
}
```

Обратите внимание, что в этом фрагменте демонстрируется более выразительный способ создания пустых списков в Kotlin — функция `emptyList`.

Функция `none` обратная по отношению к функции `any`. Это видно и в случае пустой коллекции: нет ни одного элемента, который мог бы нарушить предикат, поэтому функция возвращает значение `true`:

```
fun main() {
    println(emptyList<Int>().none { it > 42 })
    // истина
}
```

Потенциально самой сложной для понимания будет функция `all`. Независимо от предиката она возвращает значение `true` при вызове для пустой коллекции:

```
fun main() {
    println(emptyList<Int>().all { it > 42 })
    // истина
}
```

Если требуется узнать, сколько элементов удовлетворяют предикату, то используйте функцию `count`:

```
fun main() {
    val people = listOf(Person("Alice", 27), Person("Bob", 31))
    println(people.count(canBeInClub27))
    // 1
}
```

Для поиска элемента, удовлетворяющего предикату, используйте функцию `find`:

```
fun main() {
    val people = listOf(Person("Alice", 27), Person("Bob", 31))
    println(people.find(canBeInClub27))
    // Person(name=Alice, age=27)
}
```


ИСПОЛЬЗОВАНИЕ ПРАВИЛЬНОЙ ФУНКЦИИ ДЛЯ РАБОТЫ: COUNT И SIZE

Легко отбросить функцию `count` и реализовать на основе фильтрации и получения размера коллекции:

```
println(people.filter(canBeInClub27).size)
// 1
```

Но в этом случае создается промежуточная коллекция для хранения всех элементов, удовлетворяющих предикату. В свою очередь, метод `count` отслеживает только количество совпадающих элементов, а не сами элементы и поэтому считается более эффективным. Как правило, лучше найти наиболее подходящую вашим потребностям операцию.

Если совпадений много, то программа вернет первый подходящий элемент; если удовлетворяющих предикату результатов нет — будет возвращено значение `null`. Функция `firstOrNull` аналогична `find`. Используйте ее, если в таком случае логика вам более понятна.

6.1.4. Разбиение одного списка на два: функция `partition`

В некоторых ситуациях необходимо разделить коллекцию на две группы на основе заданного предиката: одни элементы выполняют логический предикат, а другие — нет. Если для работы требуются обе группы, то можно использовать функцию `filter` и родственную ей `filterNot` (она инвертирует предикат) для создания этих двух списков. В примере ниже выясняется, кого можно пускать в клуб, а кого — нет:

```
fun main() {
    val people = listOf(
        Person("Alice", 26),
        Person("Bob", 29),
        Person("Carol", 31)
    )
    val comeIn = people.filter(canBeInClub27)
    val stayOut = people.filterNot(canBeInClub27)
    println(comeIn)
    // [Person(name=Alice, age=26)]
    println(stayOut)
    // [Person(name=Bob, age=29), Person(name=Carol, age=31)]
}
```

Но есть и более лаконичный способ выполнения этой операции — можно использовать функцию `partition`. Она возвращает пару списков, не повторяя предикат и дважды итерацию входной коллекции. Это означает, что логику из предыдущего фрагмента кода можно реализовать следующим образом (рис. 6.5):

```
val (comeIn, stayOut) = people.partition(canBeInClub27)
println(comeIn)
// [Person(name=Alice, age=26)]
println(stayOut)
// [Person(name=Bob, age=29), Person(name=Carol, age=31)]
```

Объявление деструктуризации разобьет возвращаемую пару списков на две переменные на основе логического предиката

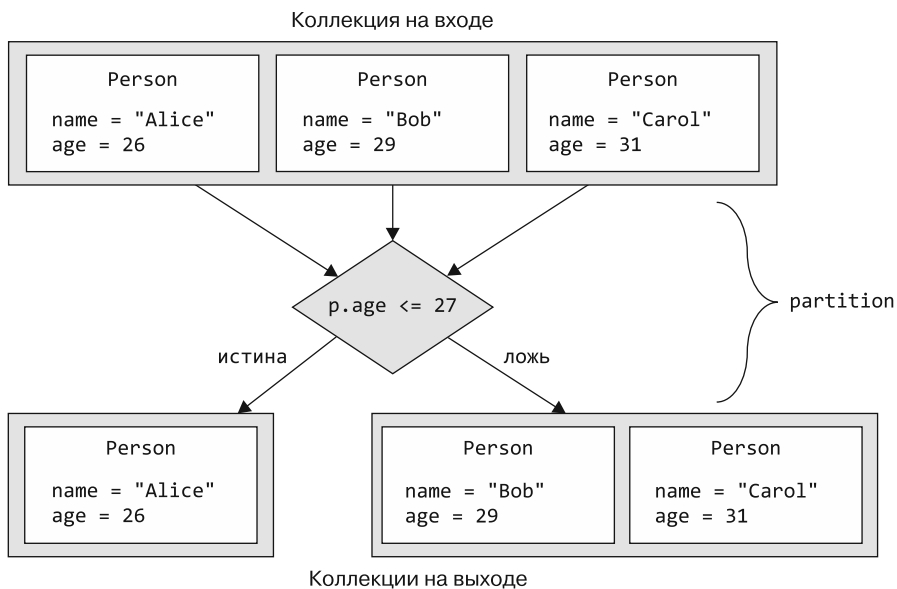


Рис. 6.5. Функция `partition` возвращает пару, состоящую из двух списков: из элементов, удовлетворяющих заданному логическому предикату и не удовлетворяющих

6.1.5. Преобразование списка в карту групп: функция `groupBy`

Функция `partition` возвращает «истинные» и «ложные» значения, но зачастую элементы коллекции нельзя объединить всего в две группы. Может потребоваться разделить все элементы на разные группы по какому-либо *качеству*. Например, требуется сгруппировать людей одного возраста. Удобно передавать это качество непосредственно в виде параметра. Функция `groupBy` может выполнить эту операцию:

```
fun main() {
    val people = listOf(
        Person("Alice", 31),
        Person("Bob", 29),
        Person("Carol", 31)
    )
    println(people.groupBy { it.age })
}
```

Результатом этой операции является сопоставление ключа группировки элементов (в данном случае `age`) с группой элементов — людьми (рис. 6.6).

Вывод результата будет следующим:

```
{31=[Person(s=Alice, age=31), Person(s=Carol, age=31)],
 29=[Person(s=Bob, age=29)]}
```

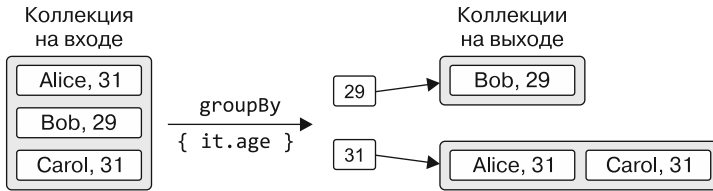


Рис. 6.6. Результат применения функции `groupBy`

Каждая группа хранится в списке, поэтому тип результата — `Map<Int, List<Person>>`. Можно вносить дальнейшие изменения в эту карту, используя такие функции, как `mapKeys` и `mapValues`.

В качестве другого примера рассмотрим возможность сгруппировать строки по первому символу с помощью ссылок на члены:

```
fun main() {
    val list = listOf("apple", "apricot", "banana", "cantaloupe")
    println(list.groupBy(String::first))
    // {a=[apple, apricot], b=[banana], c=[cantaloupe]}
}
```

Обратите внимание, что здесь `first` не является членом класса `String` — это расширение. Тем не менее к нему можно получить доступ как к ссылке на член.

6.1.6. Преобразование коллекций в карты: функции `associate`, `associateWith` и `associateBy`

Мы обсудили способы, позволяющие создать ассоциативную структуру данных из списка с помощью функции `groupBy`. Она группирует элементы на основе общего свойства. Если требуется создать карту из элементов коллекции, *не группируя* элементы, то можно использовать функцию `associate`. Ей необходимо предоставить лямбду для преобразования элемента из входной коллекции в пару «ключ — значение», которая будет помещена в карту.

В примере ниже функция `associate` превращает список объектов `Person` в карту имен для возраста и запрашивает пример значения, как это делается с любым другим объектом `Map<String, Int>`. Используется функция `to`, описанная в подразделе 3.4.3, чтобы задать отдельные пары «ключ — значение» (рис. 6.7):

```
fun main() {
    val people = listOf(Person("Joe", 22), Person("Mary", 31))
    val nameToAge = people.associate { it.name to it.age }
    println(nameToAge)
    // {Joe=22, Mary=31}
    println(nameToAge["Joe"])
    // 22
}
```

Инфиксная функция `to` создает пару из левого и правого операндов

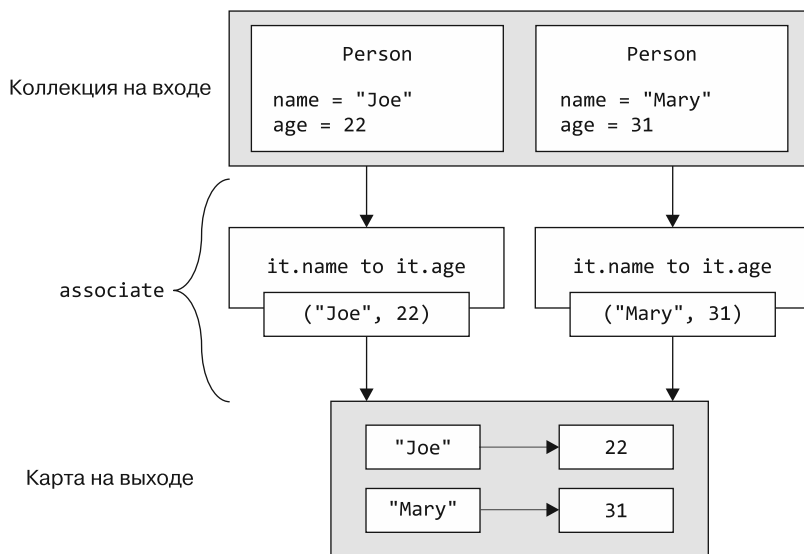


Рис. 6.7. Функция `associate` превращает список в карту на основе пар ключей и значений, возвращаемых лямбдой

Вместо того чтобы создавать пары пользовательских ключей *и* значений, может потребоваться связать элементы коллекции и другое определенное значение. Это можно сделать с помощью функций `associateWith` и `associateBy`.

Функция `associateWith` использует исходные элементы коллекции в качестве ключей. Предоставляемая лямбда генерирует соответствующее значение для каждого ключа. В противоположность ей функция `associateBy` использует исходные элементы коллекции в качестве значений и использует лямбду для генерации ключей карты.

В примере ниже сначала создается карта из имен людей и их возраста с помощью функции `associateWith`. Затем функция `associateBy` помогает создать обратную карту с сопоставлением от значений возраста к людям:

```
fun main() {
    val people = listOf(
        Person("Joe", 22),
        Person("Mary", 31),
        Person("Jamie", 22)
    )
    val personToAge = people.associateWith { it.age }
    println(personToAge)
    // {Person(name=Joe, age=22)=22, Person(name=Mary, age=31)=31,
    // Person(name=Jamie, age=22)=22} ← Joe и Jamie одинакового возраста...
    val ageToPerson = people.associateBy { it.age } ← ...и он используется в качестве ключа...
    println(ageToPerson)
    // {22=Person(name=Jamie, age=22), 31=Person(name=Mary, age=31)} ←
}
```

...поэтому в карте отображается только последнее значение возраста

Не забывайте, что ключи для карт должны быть уникальными, и ключи, генерируемые функциями `associate`, `associateBy` и `associateWith`, не являются исключением. Если функция преобразования добавит один и тот же ключ несколько раз, то последний результат перезапишет все предыдущие назначения.

6.1.7. Замена элементов в изменяемых коллекциях: функции `replaceAll` и `fill`

Как правило, функциональный стиль программирования поощряет работу с неизменяемыми коллекциями. Но периодически возникают ситуации, когда эргономичнее работать с изменяемыми коллекциями. Для таких случаев в стандартной библиотеке Kotlin есть несколько удобных функциональных возможностей. С их помощью можно изменять содержимое коллекций.

При использовании функции `replaceAll` со списком `MutableList` каждый его элемент заменяется результатом указанной вами лямбды. Для особого случая замены всех элементов в изменяемом списке одним и тем же значением в качестве сокращения можно использовать функцию `fill`. В примере ниже сначала заменяются все элементы во входной коллекции на их вариант в верхнем регистре, а затем все имена заменяются текстом-заполнителем:

```
fun main() {
    val names = mutableListOf("Martin", "Samuel")
    println(names)
    // [Martin, Samuel]
    names.replaceAll { it.uppercase() }
    println(names)
    // [MARTIN, SAMUEL]
    names.fill("(redacted)")
    println(names)
    // [(redacted), (redacted)]
}
```

6.1.8. Обработка особых случаев для коллекций: функция `ifEmpty`

Часто программа выполняет обработку только в том случае, когда коллекция входных данных не пуста, то есть в ней хранятся какие-то реальные элементы для обработки. С помощью функции `ifEmpty` можно предоставить лямбду для генерации значения по умолчанию, если коллекция пуста:

```
fun main() {
    val empty = emptyList<String>()
    val full = listOf("apple", "orange", "banana")
    println(empty.ifEmpty { listOf("no", "values", "here") })
    // [no, values, here]
    println(full.ifEmpty { listOf("no", "values", "here") })
    // [apple, orange, banana]
}
```

ifBlank: РОДСТВЕННАЯ ФУНКЦИЯ ifEmpty ДЛЯ СТРОК

При работе с текстом (который мы часто рассматриваем просто как набор символов) требование «пустой» иногда ослабляется до «незаполненный» — строки, содержащие исключительно пробельные символы, обычно менее выразительны, чем пустые строки. Чтобы сгенерировать для них значение по умолчанию, можно использовать функцию `ifBlank`:

```
fun main() {
    val blankName = " "
    val name = "J. Doe"
    println(blankName.ifEmpty { "(unnamed)" })
    //
    println(blankName.ifBlank { "(unnamed)" })
    // (unnamed)
    println(name.ifBlank { "(unnamed)" })
    // J. Doe
}
```

6.1.9. Разбиение коллекций на части: функции `chunked` и `windowed`

Иногда данные в коллекции представляют собой серию фрагментов информации. Вам может потребоваться работать с несколькими последовательными значениями за один раз. Для примера рассмотрим список ежедневных измерений датчика температуры:

```
val temperatures = listOf(27.7, 29.8, 22.0, 35.5, 19.1)
```

Чтобы получить среднее значение за три дня для каждого набора дней в этом списке значений, можно использовать *скользящее окно* размером 3. Сначала усредняются первые три значения: 27.7, 29.8 и 22.0. Затем осуществляется «скольжение» окна на один индекс, усредняя значения 29.8, 22.0 и 35.5. Эти смещения будут продолжаться до достижения трех последних значений — 22.0, 35.5 и 19.1.

Для создания таких скользящих окон можно использовать функцию `windowed`. В качестве дополнительной возможности эта функция позволяет передать лямбду для преобразования результата. В случае измерения температуры это может быть вычисление среднего значения для каждого окна (рис. 6.8):

```
fun main() {
    println(temperatures.windowed(3))
    // [[27.7, 29.8, 22.0], [29.8, 22.0, 35.5], [22.0, 35.5, 19.1]]
    println(temperatures.windowed(3) { it.sum() / it.size })
    // [26.5, 29.099999999999998, 25.533333333333333]
}
```

Вместо того чтобы перемещать скользящее окно по коллекции входных данных, можно разбить ее на отдельные части заданного размера. Это можно сделать с помощью функции `chunked`. Опять же можно передать лямбду для преобразования результата (рис. 6.9):

```
fun main() {  
    println(temperatures.chunked(2))  
    // [[27.7, 29.8], [22.0, 35.5], [19.1]]  
    println(temperatures.chunked(2) { it.sum() })  
    // [57.5, 57.5, 19.1]  
}
```

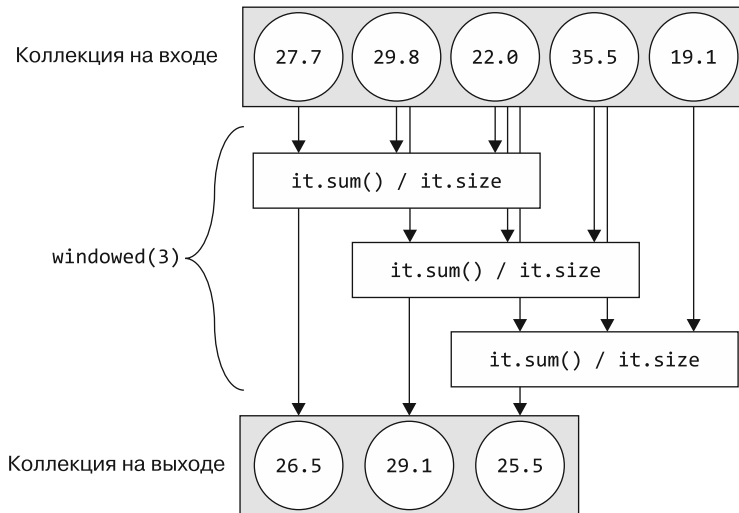


Рис. 6.8. Функция `windowed` обрабатывает коллекцию входных данных с помощью скользящего окна

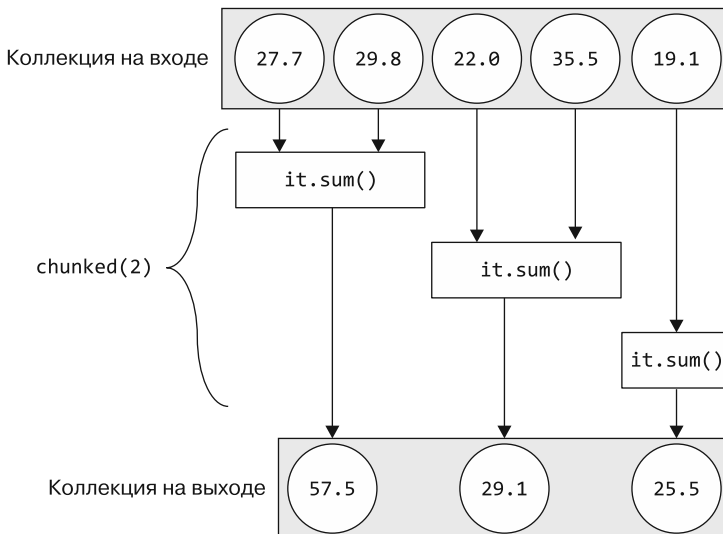


Рис. 6.9. Функция `chunked` обрабатывает входную коллекцию в непересекающихся сегментах указанного размера

Обратите внимание, что в предыдущем примере, даже если указать размер части равным 2, последняя сгенерированная часть может оказаться более мелкой. Во входной коллекции нечетное количество элементов, поэтому функция `chunked` создает две части размером 2 и помещает оставшийся элемент в третью часть.

6.1.10. Объединение коллекций: функция `zip`

Иногда приходится работать с отдельными списками, содержащими связанные данные, и агрегировать эту информацию. Например, вместо списка объектов класса `Person` есть два списка — для имен и возрастов людей соответственно:

```
val names = listOf("Joe", "Mary", "Jamie")
val ages = listOf(22, 31, 31, 44, 0)
```

Если известно, что значения в каждом списке соответствуют их индексу (то есть имя с индексом 0, `Joe`, соответствует возрасту с индексом 0, 22), то можно использовать функцию `zip` для создания списка пар из значений с одинаковыми индексами из двух коллекций. Вдобавок передача лямбды в функцию дает возможность указать, как должен быть преобразован вывод. В примере ниже создается список объектов класса `Person` из каждой пары `name` и `age` (рис. 6.10):

```
fun main() {
    val names = listOf("Joe", "Mary", "Jamie")
    val ages = listOf(22, 31, 31, 44, 0)
    println(names.zip(ages))
    // [(Joe, 22), (Mary, 31), (Jamie, 31)]
    println(names.zip(ages) { name, age -> Person(name, age) })
    // [Person(name=Joe, age=22), Person(name=Mary, age=31),
    //   Person(name=Jamie, age=31)]
}
```

Функция `zip` игнорирует значения без соответствий во второй коллекции

Обратите внимание, что размер итоговой коллекции равен меньшему из двух списков. Это связано с тем, что функция `zip` обрабатывает элементы только по индексам, хранящимся в обеих входных коллекциях.

Функцию `zip`, как и `to` для создания объектов типа `Pair`, можно вызвать как *инфиксную* (процесс был описан в подразделе 3.4.3). Но в этом случае невозможно передать преобразующую лямбду:

```
println(names zip ages)
// [(Joe, 22), (Mary, 31), (Jamie, 31)]
```

Как и в случае с любой другой функцией, несколько вызовов функции `zip` для объединения нескольких списков можно объединить в цепочку, даже при использовании инфиксного синтаксиса. Данная функция всегда оперирует двумя списками, поэтому следует знать, что итоговая структура будет состоять из вложенных пар, а не просто из списка списков:

```
val countries = listOf("DE", "NL", "US")
println(names zip ages zip countries)
// [((Joe, 22), DE), ((Mary, 31), NL), ((Jamie, 31), US)]
```

Результатом сочетания множества вызовов функции `zip` станет список вложенных пар. Обратите внимание на дополнительные скобки для имен и значений возраста

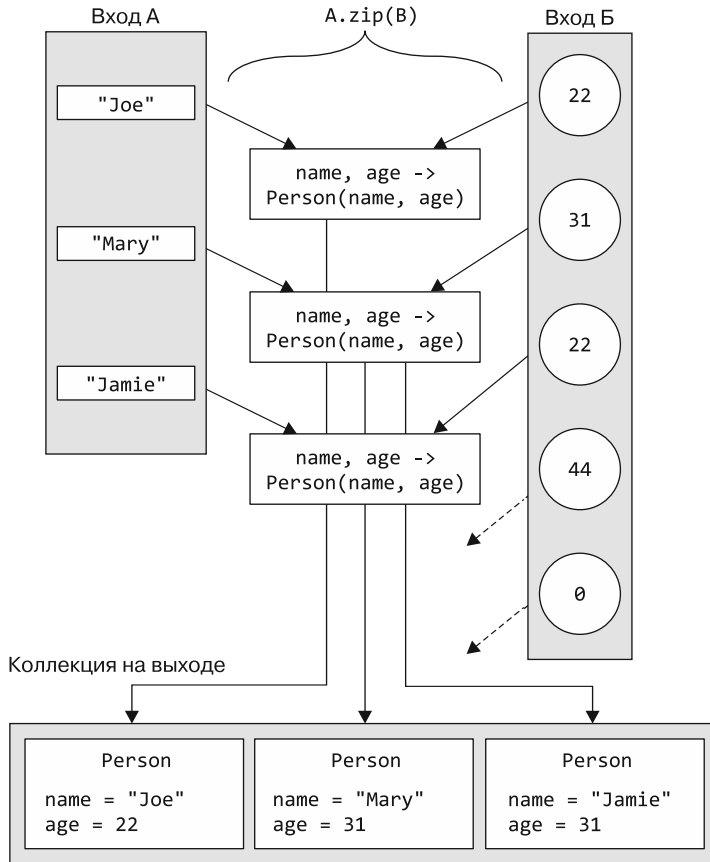


Рис. 6.10. Функция `zip` сопоставляет каждый элемент двух своих входов по одному и тому же индексу с помощью переданной ей лямбды. Иными словами, создает пары элементов. Элементы, не имеющие соответствий в другой коллекции, игнорируются

6.1.11. Обработка элементов во вложенных коллекциях: функции `flatMap` и `flatten`

Отложим разговор о людях и перейдем к книгам. Предположим, у вас есть коллекция книг, представленная классом `Book`:

```
class Book(val title: String, val authors: List<String>)
```

Каждая книга была написана одним или несколькими авторами, и в библиотеке есть несколько книг:

```
val library = listOf(
    Book("Kotlin in Action", listOf("Isakova", "Elizarov", "Aigner", "Jemerov")),
    Book("Atomic Kotlin", listOf("Eckel", "Isakova")),
    Book("The Three-Body Problem", listOf("Liu"))
)
```

Если требуется вычислить всех авторов в библиотеке, то стоит начать с функции `map`, которая была представлена в подразделе 6.1.1:

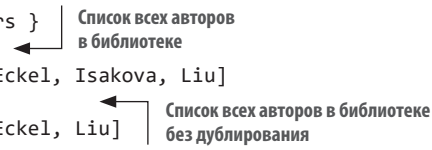
```
fun main() {
    val authors = library.map { it.authors }
    println(authors)
    // [[Isakova, Elizarov, Aigner, Jemerov], [Eckel, Isakova], [Liu]]
}
```

Скорее всего, такого результата вы не ожидали. Переменная `authors` сама по себе представлена элементом `List<String>`, поэтому итоговая коллекция примет вид вложенной: `List<List<String>>`.

Функция `flatMap` вычислит множество всех авторов в вашей библиотеке, создавая лишнюю вложенность. Она выполняет две задачи: сначала преобразует (или сопоставляет) каждый элемент в коллекцию в соответствии с функцией, заданной в качестве аргумента (точно так же, как функция `map`), а затем объединяет (или выравнивает) эти списки (листинг 6.1).

Листинг 6.1. Функция `flatMap` превращает коллекцию коллекций в «плоский» список

```
fun main() {
    val authors = library.flatMap { it.authors }
    println(authors)
    // [Isakova, Elizarov, Aigner, Jemerov, Eckel, Isakova, Liu]
    println(authors.toSet())
    // [Isakova, Elizarov, Aigner, Jemerov, Eckel, Liu]
}
```



Список всех авторов в библиотеке

Список всех авторов в библиотеке без дублирования

Каждая книга может быть написана несколькими авторами, и свойство `book.authors` хранит список авторов. В листинге 6.1 функция `flatMap` применяется для объединения авторов всех книг в один «плоский» список. Вызов `toSet` удаляет дубликаты из итоговой коллекции, поэтому в данном примере `Isakova` указана в выходных данных только раз.

Функция `flatMap` может понадобиться, когда вы работаете с коллекцией коллекций элементов (например, `List<List<Int>>`) и вам требуется свести их в одну (то есть `List<Int>`). Обратите внимание: если не нужно ничего преобразовывать, а требуется просто выровнять такую коллекцию коллекций, то можно использовать следующую `flatten`-функцию: `listOfLists.flatten()`.

Мы рассмотрели несколько функций работы с коллекциями в стандартной библиотеке Kotlin, но их гораздо больше. Не будем описывать их все как из соображений экономии книжных страниц, так и потому, что показывать длинный список функций скучно. В общем случае при написании кода для работы с коллекциями мы рекомендуем подумать о том, как операция может быть выражена в виде общего преобразования, и поискать стандартную библиотечную функцию, с помощью которой его можно выполнить. Варианты функций представлены в предложениях автозаполнения в вашей IDE, либо их стоит поискать в справочнике по стандартной библиотеке Kotlin (<https://kotlinlang.org/api/latest/jvm/stdlib/>). Вполне вероятно, что

вы сможете найти и использовать ее для решения своей задачи быстрее, чем при реализации вручную.

Теперь подробнее рассмотрим производительность кода, содержащего цепочки операций коллекций. В следующем разделе представлены различные способы выполнения таких операций.

6.2. ОТЛОЖЕННЫЕ ОПЕРАЦИИ КОЛЛЕКЦИЙ: ПОСЛЕДОВАТЕЛЬНОСТИ

В предыдущем разделе приводились примеры цепочек функций коллекций, таких как `map` и `filter`. Эти функции создают промежуточные коллекции *немедленно*. То есть промежуточный результат каждого шага хранится во временном списке. *Последовательности* предоставляют альтернативный способ выполнения таких вычислений. Он позволяет избежать создания промежуточных временных объектов, подобно тому как это делают потоки в Java 8. Например:

```
people.map(Person::name).filter { it.startsWith("A") }
```

В справочнике стандартной библиотеки Kotlin говорится, что функции `map` и `filter` возвращают список. Это означает, что данная цепочка вызовов создаст два списка: один для хранения результатов функции `map`, а второй — для результатов `filter`. У вас не возникнет проблем, если исходный список содержит два элемента. Но такой подход становится гораздо менее действенным, если элементов миллион.

Код станет более эффективным, если преобразовать операцию для использования последовательностей, а не коллекций напрямую:

```
people
    .asSequence()
    .map(Person::name)
    .filter { it.startsWith("A") }
    .toList()
```

Преобразование исходной коллекции в последовательность

Последовательности поддерживают API для работы с коллекциями

Преобразование результирующей последовательности обратно в список

Результат применения операции тот же, что и в предыдущем примере: список имен людей, начиная с буквы А. Но во втором примере не создаются промежуточные коллекции для хранения элементов, поэтому производительность для большого количества элементов будет заметно выше.

Точкой входа для отложенных операций коллекций в Kotlin служит интерфейс `Sequence`. Он представляет собой последовательность элементов, которые можно перечислять один за другим. `Sequence` предоставляет только один метод, `iterator`, с помощью которого можно получить значения из последовательности.

Сила интерфейса `Sequence` заключается в способе реализации операции над ним. Элементы последовательности вычисляются *отложено*. Поэтому с помощью последовательностей можно эффективно выполнять цепочки операций над элементами коллекции, не создавая коллекции для хранения промежуточных результатов

обработки. Вдобавок стоит отметить, что функции для обработки обычных списков, такие как `map` и `filter`, применяются и к последовательностям.

Любую коллекцию можно преобразовать в последовательность, если вызвать функцию-расширение `asSequence`. Чтобы выполнить обратное преобразование из последовательности в обычный список, необходимо вызвать функцию `toList`.

Зачем преобразовывать последовательность обратно в коллекцию? Не удобнее ли использовать последовательности вместо коллекций, если они намного лучше? Не всегда это утверждение справедливо. Если нужно выполнить итерацию только по элементам последовательности, то можно использовать последовательность напрямую. Если требуются другие методы API, например обращение к элементам по индексу, то нужно преобразовать последовательность в список.

ПРИМЕЧАНИЕ

Как правило, последовательность используется во всех случаях, когда выполняется цепочка операций над большой коллекцией. В разделе 10.2 мы обсудим, почему в Kotlin немедленные операции над обычными коллекциями эффективны, несмотря на создание промежуточных коллекций. Но если коллекция содержит большое количество элементов, то промежуточная перестановка элементов обходится очень дорого, поэтому отложенное вычисление предпочтительнее.

Операции над последовательностью относятся к отложенным, поэтому для их запуска необходимо выполнять итерации над элементами последовательности напрямую или путем преобразования ее в коллекцию. В следующем разделе рассказывается о данном процессе.

6.2.1. Выполнение последовательных операций: промежуточные и терминальные операции

Операции над последовательностью делятся на две категории: промежуточные и терминальные. *Промежуточная операция* возвращает другую последовательность, которой известно, как преобразовать элементы исходной. *Терминальная операция* возвращает результат. Он может быть коллекцией, элементом, числом или любым другим объектом, полученным в результате последовательности преобразований исходной коллекции (рис. 6.11).

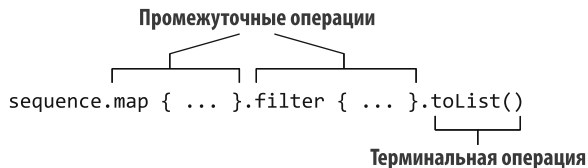


Рис. 6.11. Промежуточные и терминальные операции над последовательностями

Промежуточные операции всегда отложены. Посмотрите этот пример, в нем нет терминальной операции:

```
fun main() {
    println(
        listOf(1, 2, 3, 4)
            .asSequence()
            .map {
                print("map($it) ")
                it * it
            }.filter {
                print("filter($it) ")
                it % 2 == 0
            }
    )
    // kotlin.sequences.FilteringSequence@506e1b77
}
```

Когда этот фрагмент кода будет выполнен, вы не увидите ожидаемого результата этих операций в консоли: вместо него будет отображен только сам объект `Sequence`. Выполнение преобразований `map` и `filter` откладывается и будет применено только при получении результата (то есть при вызове терминальной операции):

```
fun main() {
    listOf(1, 2, 3, 4)
        .asSequence()
        .map {
            print("map($it) ")
            it * it
        }.filter {
            print("filter($it) ")
            it % 2 == 0
        }.toList()
}
```

Терминальная операция (`toList`) приводит к выполнению всех отложенных вычислений.

Еще один важный момент в этом примере — порядок выполнения вычислений. При простом прямом подходе сначала вызывается функция `map` для каждого элемента, а затем функция `filter` для каждого элемента полученной последовательности. Так функции `map` и `filter` работают с коллекциями, но не с последовательностями. В случае с последовательностями все операции применяются к каждому элементу по очереди: обрабатывается первый элемент (сопоставляется, затем фильтруется), затем второй и т. д.

Такой подход означает, что некоторые элементы вообще не преобразуются, если результат получен до их достижения. Рассмотрим пример с операциями `map` и `find`.

Сначала число сопоставляется с его квадратом, а затем выполняется поиск первого элемента со значением больше 3:

```
fun main() {
    println(
        listOf(1, 2, 3, 4)
            .asSequence()
            .map { it * it }
            .find { it > 3 }
    )
    // 4
}
```

Если те же операции применяются не к последовательности, а к коллекции, то сначала вычисляется результат функции `map`, преобразующий все элементы исходной коллекции. На втором этапе в промежуточной коллекции обнаруживается элемент, удовлетворяющий предикату. В случае с последовательностями отложенный подход означает, что можно пропустить обработку некоторых элементов. На рис. 6.12 показана разница между вычислением этого кода в немедленном режиме (с использованием коллекций) и в отложенном (с использованием последовательностей).

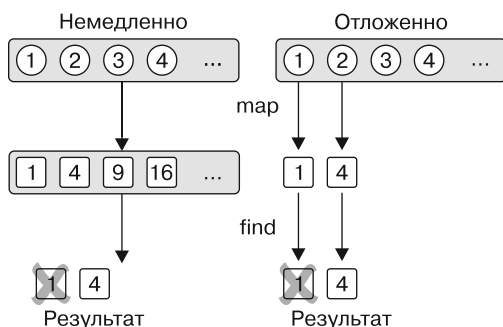


Рис. 6.12. При немедленном вычислении каждая операция выполняется над всей коллекцией; при отложенном элементы обрабатываются по одному

В первом случае, при работе с коллекциями, список преобразуется в другой, поэтому преобразование `map` применяется к каждому элементу, в том числе 3 и 4. После этого будет найден первый элемент, удовлетворяющий предикату, — квадрат числа 2.

Во втором случае вызов функции `find` начинает обрабатывать элементы по одному. Из исходной последовательности берется число, преобразуется с помощью функции `map`. Затем проверяется, соответствует ли оно предикату, переданному в функцию `find`. Дойдя до числа 2, можно обнаружить, что его квадрат больше числа 3. И 2 будет возвращено как результат операции `find`. Нет необходимости обрабатывать числа 3 и 4, поскольку результат был найден до того, как до них дошла очередь выполнения.

Порядок операций, выполняемых над коллекцией, может влиять и на производительность. Представьте, что у вас есть коллекция с данными о людях и требуется

вывести их имена, если они короче определенного значения. Необходимо сопоставить каждого человека с его именем, а затем отфильтровать слишком длинные имена. В таком случае операции `map` и `filter` применяются в любом порядке. Оба подхода дадут одинаковый результат, но общее количество преобразований будет отличаться (рис. 6.13):

```
fun main() {
    val people = listOf(
        Person("Alice", 29),
        Person("Bob", 31),
        Person("Charles", 31),
        Person("Dan", 21)
    )
    println(
        people
            .asSequence()
            .map(Person::name) ← Операция map выполняется перед filter
            .filter { it.length < 4 }
            .toList()
    )
    // [Bob, Dan]
    println(
        people
            .asSequence()
            .filter { it.name.length < 4 }
            .map(Person::name)
            .toList() ← Операция map выполняется после filter
    )
    // [Bob, Dan]
}
```

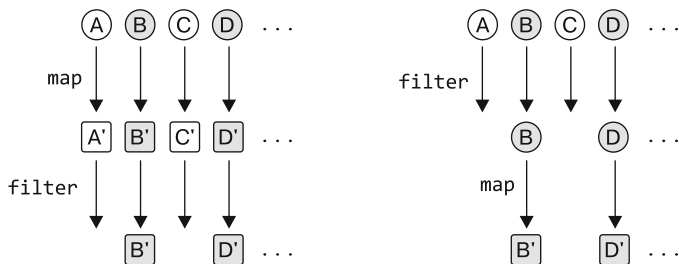


Рис. 6.13. Применение операции `filter` первой помогает сократить общее количество преобразований

Если сначала выполняется `map`, то каждый элемент преобразуется, даже в том случае, когда он отбрасывается на следующем шаге и больше никогда не используется. Если сначала применить операцию `filter`, то несоответствующие элементы будут отфильтрованы сразу и преобразования к ним применяться не будут. Как правило, чем раньше элементы будут удалены из цепочки операций (разумеется, без ущерба для логики кода), тем более производительным будет код.

6.2.2. Создание последовательностей

В предыдущих примерах для создания последовательности к коллекции применялся один и тот же метод — `asSequence()`. Другая возможность — использовать функцию `generateSequence`. Она вычисляет следующий элемент в последовательности, учитывая предыдущий.

Приведем пример, как функцию `generateSequence` можно использовать для вычисления суммы всех натуральных чисел до 100 (листинг 6.2). Сначала генерируется последовательность всех натуральных чисел. Затем применяется функция `takeWhile`, чтобы извлечь из этой последовательности элементы со значением, которое меньше или равно 100. Наконец, для вычисления суммы этих чисел используется функция `sum`.

Листинг 6.2. Генерация и использование последовательности натуральных чисел

```
fun main() {
    val naturalNumbers = generateSequence(0) { it + 1 }
    val numbersTo100 = naturalNumbers.takeWhile { it <= 100 }
    println(numbersTo100.sum())
    // 5050
}
```

← Все отложенные операции выполняются после получения результата функции `sum`

Обратите внимание: `naturalNumbers` (бесконечная) и `numbersTo100` (конечная) в данном примере — последовательности с отложенным вычислением. Фактические числа в этих последовательностях не будут вычисляться до тех пор, пока будет вызвана терминальная операция (в данном случае `sum`).

Другой распространенный вариант использования — последовательность родительских элементов. Если у элемента есть родители того же типа, то вас могут заинтересовать качества последовательности всех его предков. Типичными примерами могут быть родословная человека или структура родительской папки для заданного файла (на JVM и файлы, и папки обычно представлены одним и тем же типом: `File`). В листинге 6.3 выясняется, находится ли файл в скрытом каталоге. Для этого создается последовательность его родительских каталогов и выполняется проверка этого атрибута в каждом из них.

Листинг 6.3. Создание и использование последовательности родительских каталогов

```
import java.io.File

fun File.isInsideHiddenDirectory() =
    generateSequence(this) { it.parentFile }.any { it.isHidden }

fun main() {
    val file = File("/Users/svtk/.HiddenDir/a.txt")
    println(file.isInsideHiddenDirectory())
    // истина
}
```

Как и ранее, вы создаете последовательность, предоставляя первый элемент и способ получения каждого последующего. Заменяв функцию `any` на `find`, вы получите

фактический скрытый каталог, а не просто логическое значение, указывающее на наличие скрытого файла где-то по пути. Обратите внимание, что использование последовательностей позволяет прекратить обход родительского каталога, как только будет найден нужный каталог.

РЕЗЮМЕ

- Вместо того чтобы вручную перебирать элементы коллекции, большинство пространственных операций можно выполнять, комбинируя существующие функции из стандартной библиотеки с вашими собственными лямбдами. В Kotlin встроено большое количество таких функций.
- Функции `filter` и `map` лежат в основе работы с коллекциями, а также позволяют легко извлекать элементы, соответствующие определенному предикату, или преобразовывать элементы в новую форму.
- Операции `reduce` и `fold` *объединяют* информацию из коллекции, помогая вычислить одно значение из коллекции элементов.
- Функции из семейств `associate` и `groupBy` помогут превратить «плоские» списки в карты, чтобы вы могли структурировать данные по собственным критериям.
- Когда данные в коллекциях связаны по индексам, функции `chunked`, `windowed` и `zip` дают возможность создавать подгруппы элементов коллекции или объединять несколько коллекций.
- Используя *предикаты* — лямбда-функции, возвращающие `Boolean`, `all`, `any`, `none` и другие родственные функции, — вы можете проверить, применимы ли определенные инварианты к вашим коллекциям.
- При работе со вложенными коллекциями функция `flatten` извлечет вложенные элементы, а функция `flatMap` выполнит преобразование на том же шаге.
- Последовательности позволяют *отложено* комбинировать несколько операций над коллекцией, не создавая коллекций для хранения промежуточных результатов, что делает код более эффективным. Для работы с последовательностями можно использовать те же функции, которые применяются и для взаимодействия с коллекциями.



Работа с nullable-значениями

В этой главе

- ✓ Типы, допускающие значение `null`.
- ✓ Синтаксис для работы с потенциально нулевыми значениями.
- ✓ Преобразование между типами, допускающими и не допускающими значение `null`.
- ✓ Взаимодействие концепции null-безопасности в Kotlin и кода Java.

К настоящему моменту вы уже увидели работу большей части синтаксиса Kotlin. Вы вышли за рамки создания базового кода на Kotlin и готовы воспользоваться функциями этого языка, которые призваны повысить производительность. Они помогут сделать ваш код более компактным и удобочитаемым. Одна из важнейших возможностей Kotlin, которая позволяет повысить надежность кода, — поддержка *типов, допускающих значение null* (nullable types). Рассмотрим ее более подробно.

7.1. ПРЕДОТВРАЩЕНИЕ NULLPOINTEREXCEPTION И ОБРАБОТКА ОТСУТСТВИЯ ЗНАЧЕНИЙ: NULL-БЕЗОПАСНОСТЬ

Null-безопасность (nullability) — функциональная особенность системы типов Kotlin для предотвращения ошибок типа `NullPointerException` (NPE). Как пользователь программы, вы наверняка видели сообщение об ошибке, похожее на An

error has occurred: java.lang.NullPointerException (Произошла ошибка: java.lang.NullPointerException), не содержащее никаких подробностей. На Android вы могли видеть другую версию этого сообщения, например *Unfortunately, application X has stopped* (К сожалению, приложение X остановлено). Его причина обычно — `NullPointerException`. Такие ошибки во время выполнения программы доставляют неудобства и пользователям, и разработчикам.

Подход современных языков, в том числе Kotlin, заключается в том, чтобы переместить эти ошибки из времени выполнения во время компиляции. Поддерживая `null`-безопасность как часть системы типов, компилятор может обнаружить множество вероятных ошибок во время компиляции и уменьшить вероятность возникновения исключений во время выполнения.

В этом разделе мы обсудим типы в Kotlin, допускающие значение `null`: как этот язык помечает значения, которые могут принять `null`, и какие инструменты в нем есть для работы с такими значениями. Далее мы рассмотрим детали смешивания кода Kotlin и Java в отношении этих типов.

7.2. ЯВНОЕ ОПРЕДЕЛЕНИЕ ДОПУСТИМОСТИ NULL-ПЕРЕМЕННЫХ С ПОМОЩЬЮ ТИПОВ, ДОПУСКАЮЩИХ ЗНАЧЕНИЕ NULL

Первое и, вероятно, самое важное различие между системами типов Kotlin и Java — явная поддержка в Kotlin *типов, допускающих значение null*. Что это значит? Это способ указать, какие переменные или свойства в программе могут принимать значение `null`. Когда переменная может хранить `null`, вызов метода для нее может вызвать исключение `NullPointerException`. Kotlin запрещает такие вызовы и тем самым предотвращает множество возможных исключений. Чтобы увидеть, как это работает на практике, рассмотрим следующую функцию Java. Она принимает значение типа `String` и вызывает функцию `length` для него:

```
/* Java */
int strLen(String s) {
    return s.length();
}
```

Работа этой функции безопасна? Опытный разработчик, вероятно, быстро поймет, что если вызвать функцию с аргументом `null`, то она выбросит исключение `NullPointerException`. Нужно ли добавить в функцию проверку значения `null`? Это зависит от предназначения функции.

Перепишем эту функцию на языке Kotlin. Первый вопрос, требующий ответа: ожидаете ли вы, что функция будет вызвана с аргументом `null`? Мы имеем в виду не только непосредственно литерал `null`, как в `strLen(null)`, но и любую переменную или другое выражение, которое может содержать значение `null` во время выполнения.

Если же это, скорее всего, не произойдет, то объявите эту функцию в Kotlin следующим образом:

```
fun strLen(s: String) = s.length
```

Вызов функции `strLen` с аргументом с возможным значением `null` недопустим и будет отмечен как ошибка во время компиляции:

```
fun main() {
    strLen(null)
    // ERROR: Null can not be a value of a non-null type String
}
```

Параметр объявлен как тип `String`, а в Kotlin это означает, что он всегда должен содержать экземпляр `String`. Компилятор следит за соблюдением этого условия, поэтому передать аргумент с `null` не получится. Это гарантирует, что функция `strLen` никогда не выбросит исключение `NullPointerException` во время выполнения.

Если требуется допустить использование этой функции со всеми аргументами, в том числе `null`, то необходимо явно указать на это, поставив вопросительный знак после имени типа:

```
fun strLenSafe(s: String?) = ...
```

Знак вопроса можно поставить после любого типа, чтобы указать, что переменные этого типа могут хранить `null`-ссылки: `String?`, `Int?`, `MyCustomType?` и т. д. (рис. 7.1).

Type? = Type или null

Рис. 7.1. Вопросительный знак после имени указывает на тип, допускающий значение `null`. Переменная такого типа может хранить ссылку `null`

Повторим, что имя типа без вопросительного знака означает, что переменные этого типа не могут хранить ссылки `null`. Это означает, что все обычные типы по умолчанию не допускают значение `null`, если только явно не помечены как `nullable`.

При наличии значения типа, допускающего значение `null`, набор доступных с ним операций ограничен. Например, для него нельзя вызывать методы. Теперь компилятор будет «жаловаться» на вызов `length` в теле функции:

```
fun strLenSafe(s: String?) = s.length()
// ERROR: only safe (?.) or non-null asserted (!!) calls are allowed
// on a nullable receiver of type kotlin.String?
```

Кроме того, нельзя присвоить значение типа, допускающего значение `null`, переменной типа, который таковым не является:

```
fun main() {
    val x: String? = null
    var y: String = x
    // ERROR: Type mismatch:
    // inferred type is String? but String was expected
}
```

Нельзя передать значение типа, допускающего значение `null`, в качестве аргумента в функцию с параметром, не допускающим такое значение:

```
fun main() {
    val x: String? = null
    strlen(x)
    // ERROR: Type mismatch:
    // inferred type is String? but String was expected
}
```

Что же можно сделать со значением типа, допускающего значение `null`? Самое важное — сравнить его с `null`. И как только сравнение будет выполнено, компилятор запомнит это и будет рассматривать значение как не допускающее `null` в области видимости, где была выполнена проверка. Например, код в листинге 7.1 вполне корректен.

Листинг 7.1. Обработка значений `null` с проверками `if`

```
fun strlenSafe(s: String?): Int =
    if (s != null) s.length else 0
fun main() {
    val x: String? = null
    println(strlenSafe(x))
    // 0
    println(strlenSafe("abc"))
    // 3
}
```

После добавления проверки на `null` код компилируется

Если бы проверки `if` были единственным инструментом для решения проблемы `null`-безопасности, то код очень быстро стал бы многословным. К счастью, Kotlin предоставляет ряд других инструментов для более лаконичной работы с nullable-значениями. Но прежде, чем мы их рассмотрим, уделим немного внимания обсуждению понятия «`null`-безопасность» и того, что такое типы переменных.

7.3. БОЛЕЕ ПРИСТАЛЬНОЕ ИЗУЧЕНИЕ ЗНАЧЕНИЯ ТИПОВ

Подумаем над самыми общими вопросами: что такое типы и зачем они нужны переменным? В 1976 году Дэвид Парнас уже определил типы как набор возможных значений и набор операций, которые можно выполнять над этими значениями. (Статья *Abstract types defined as classes of variables*, <https://dl.acm.org/doi/10.1145/800237.8071133>.)

Попробуем применить это определение к некоторым типам Java, начиная с типа `double`. Значения типа `double` — это 64-битные числа с плавающей запятой. Над этими значениями выполняются стандартные математические операции. Все функции одинаково применимы ко всем значениям типа `double`. Следовательно, если есть переменная типа `double`, то вы можете быть уверены, что любая разрешенная компилятором операция над ее значением будет выполнена успешно.

Теперь сравним эту ситуацию с переменной типа `String`. В Java такая переменная может содержать одно из двух значений: экземпляр класса `String` или `null`. Эти

типы значений совершенно не похожи друг на друга. И даже собственный оператор `Java instanceof` сообщит, что `null` не относится к `String`. Над значением переменной тоже можно выполнять совершенно разные операции: реальный экземпляр класса `String` позволяет вызывать любые методы для строки, а значение `null` дает возможность выполнять лишь ограниченный набор операций.

Это означает, что система типов `Java` в данном случае не справляется со своей задачей. Даже если для переменной объявляется тип `String`, без дополнительных проверок неизвестно, что можно делать с ее значениями. Часто эти проверки пропускают, поскольку из общего потока данных программы известно, что значение не может быть равно `null` в определенный момент. Иногда разработчик ошибается, и тогда программа завершается с исключением `NullPointerException`.

ДРУГИЕ СПОСОБЫ БОРЬБЫ С ОШИБКАМИ `NULLPOINTEREXCEPTION`

В `Java` есть инструменты для решения проблемы `NullPointerException`. Например, некоторые люди используют аннотации (`@Nullable` и `@NotNull`), чтобы выразить null-безопасность значений. Существуют инструменты (например, встроенная инспекция кода в `IntelliJ IDEA`), использующие эти аннотации для обнаружения точек, в которых может возникнуть `NullPointerException`. Но такие инструменты не являются частью стандартного процесса компиляции `Java`, поэтому трудно сделать так, чтобы они применялись последовательно. Кроме того, сложно аннотировать всю кодовую базу, в том числе библиотеки, используемые в проекте, чтобы выявить все возможные места ошибок. Наш опыт работы в `JetBrains` показывает, что даже повсеместное использование аннотаций null-безопасности в `Java` не полностью решает проблему исключений `NPE`.

Другой путь решения этой проблемы — никогда не использовать в коде значения `null` и применять специальный тип-обертку. В `Java 8` был введен такой тип `Optional` для представления значений, которые могут быть определены или не определены. Этот подход имеет несколько минусов: код становится более многословным, дополнительные экземпляры обертки влияют на производительность во время выполнения, и данный тип не используется последовательно во всей экосистеме. Даже если получится применять `Optional` повсеместно, вам все равно придется работать со значениями `null`, поскольку они возвращаются из методов `JDK`, фреймворка `Android` и других сторонних библиотек.

Типы в `Kotlin`, допускающие значение `null`, предоставляют комплексное решение этой проблемы. Различение типов, которые допускают значение `null` и которые таковыми не являются, дает четкое понимание того, какие операции со значением разрешены, а какие могут привести к исключениям во время выполнения и поэтому запрещены.

ПРИМЕЧАНИЕ

Объекты обоих типов во время выполнения одинаковы: тип, допускающий значение `null`, не является оберткой для типа, который таковым не является. Работа с первыми типами в `Kotlin` практически не влечет никаких накладных расходов во время выполнения. Исключениями выступают некоторые автоматически генерируемые внутренние проверки (<https://kotlinlang.org/docs/java-to-kotlin-nullability-guide.html#support-for-nullable-types>), не оказывающие сильного влияния на производительность.

Теперь обсудим подходы к работе с типами в Kotlin, допускающими значение `null`, и то, почему работа с ними не вызывает неприязни. Начнем со специального оператора для безопасного доступа к nullable-значению.

7.4. КОМБИНИРОВАНИЕ ПРОВЕРОК NULL И ВЫЗОВОВ МЕТОДОВ С ПОМОЩЬЮ ОПЕРАТОРА БЕЗОПАСНОГО ВЫЗОВА ?.

Оператор безопасного вызова (`?.`) стал одним из самых полезных инструментов в арсенале Kotlin. Этот оператор объединяет проверку `null` и вызов метода в одну операцию. Например, выражение `str?.uppercase()` эквивалентно следующему, более громоздкому выражению: `if (str != null) str.uppercase() else null`.

Другими словами, если значение, для которого вызывается метод, не равно `null`, то вызов метода будет выполнен нормально. Если значение `null`, то вызов пропускается и вместо него в качестве значения используется `null`. Это показано на рис. 7.2.

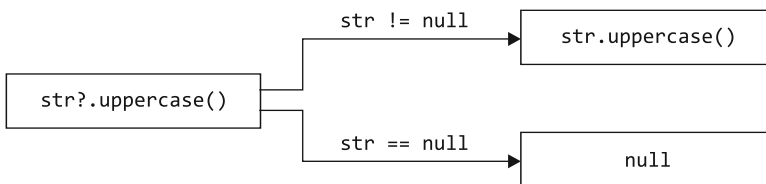


Рис. 7.2. Оператор безопасного вызова запускает методы только для значений, которые не являются `null`. Если значение окажется `null`, то вызов не выполняется, а `null` возвращается напрямую. Это позволяет безопасно вызывать методы, не прибегая к проверке `null` вручную

Обратите внимание, что тип результата такого вызова допускает значение `null`. Функция `String.uppercase` возвращает значение типа `String`, однако тип результата выражения `s?.uppercase()`, когда `s` допускает значение `null`, будет `String?`:

```

fun printAllCaps(str: String?) {
    val allCaps: String? = str?.uppercase()
    println(allCaps)
}

fun main() {
    printAllCaps("abc")
    // ABC
    printAllCaps(null)
    // null
}
  
```

← Переменная `allCaps` может принимать значение `null`

Безопасные вызовы можно использовать и для доступа к свойствам, а не только для вызова методов. В листинге 7.2 показан простой класс Kotlin `Employee` со свойством `manager`, допускающим значение `null`, и продемонстрировано, как

с помощью оператора безопасного вызова осуществляется доступ к этому свойству в функции `managerName`.

Листинг 7.2. Использование безопасных вызовов для работы со свойствами, допускающими значение `null`

```
class Employee(val name: String, val manager: Employee?)

fun managerName(employee: Employee): String? = employee.manager?.name

fun main() {
    val ceo = Employee("Da Boss", null)
    val developer = Employee("Bob Smith", ceo)
    println(managerName(developer))
    // Da Boss
    println(managerName(ceo))
    // null
}
```

Если у вас есть граф объектов и у нескольких свойств в нем типы допускают значение `null`, то удобно использовать несколько безопасных вызовов в одном выражении. Допустим, вы храните информацию о человеке, его компании и адресе компании, используя разные классы. Данные о компании и ее адрес могут быть опущены. К свойству `country` класса `Person` можно получить доступ с помощью оператора `?..` Код займет одну строку, и в нем не будет никаких дополнительных проверок (листинг 7.3).

Листинг 7.3. Цепочка из нескольких операторов безопасного вызова

```
class Address(val streetAddress: String, val zipCode: Int,
              val city: String, val country: String)

class Company(val name: String, val address: Address?)

class Person(val name: String, val company: Company?)

fun Person.countryName(): String {
    val country = this.company?.address?.country
    return if (country != null) country else "Unknown"
}

fun main() {
    val person = Person("Dmitry", null)
    println(person.countryName())
    // Unknown
}
```

Несколько операторов безопасного вызова можно объединить в цепочку

Последовательности вызовов с проверкой `null` часто встречаются в коде на Java, а Kotlin делает их более лаконичными. Но в листинге 7.3 есть ненужные повторения: значение сравнивается с `null` и возвращается либо это значение, либо что-то другое, если оно равно `null`. Посмотрим, как Kotlin помогает избавиться от повторений.

7.5. ПРЕДОСТАВЛЕНИЕ ЗНАЧЕНИЙ ПО УМОЛЧАНИЮ В NULL-СЛУЧАЯХ С ПОМОЩЬЮ ОПЕРАТОРА ?:

В Kotlin есть удобный оператор для предоставления значений по умолчанию вместо null. Он называется *оператором Elvis* (или *оператором объединения с null*, если вы предпочитаете более серьезные названия). Выглядит он так: `?:` (при взгляде сбоку похож на прическу Элвиса Пресли). Он применяется следующим образом:

```
fun greet(name: String?) {
    val recipient: String = name ?: "unnamed"
    println("Hello, $recipient!")
}
```

Если имя равно null, то вместо него получателю будет присвоено значение unnamed

Оператор принимает два значения, и его результатом является первое значение, если оно не равно null, или второе в противном случае. На рис. 7.3 показано, как это работает.

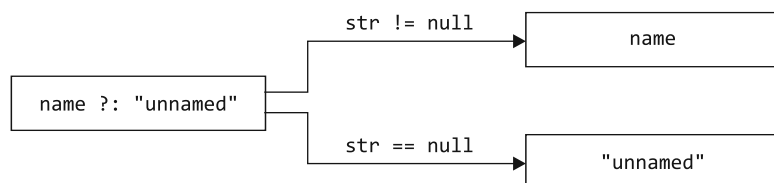


Рис. 7.3. Оператор `?:` заменяет указанное значение на null. Это позволяет определить значение по умолчанию, если левое выражение окажется равным null

Оператор `?:` часто используют с оператором безопасного вызова для подстановки значения, отличного от null, когда объект, для которого вызывается метод, равен null. В листинге 7.4 показано, как с помощью этого паттерна можно упростить код в листинге 7.1.

Листинг 7.4. Использование оператора `?:` для работы с null-значениями

```
fun strLenSafe(s: String?): Int = s?.length ?: 0

fun main() {
    println(strLenSafe("abc"))
    // 3
    println(strLenSafe(null))
    // 0
}
```

Реализация функции `countryName` из листинга 7.3 теперь тоже укладывается в одно аккуратное выражение:

```
fun Person.countryName() = company?.address?.country ?: "Unknown"
```

Особенно удобно, что при использовании в Kotlin оператора `?:` такие функции, как `return` и `throw`, работают как выражения. Следовательно, их можно использовать в правой части оператора. В этом случае, если значение в левой части равно `null`, функция немедленно вернет значение или выбросит указанное вами исключение. Это полезно для проверки предварительных условий в функции.

Рассмотрим варианты использования этого оператора для реализации функции печати этикетки с адресом компании человека. В листинге 7.5 повторяются объявления всех классов — в Kotlin они настолько лаконичны, что это не проблема.

Листинг 7.5. Использование выражения `throw` с оператором `?:`

```
class Address(val streetAddress: String, val zipCode: Int,
              val city: String, val country: String)

class Company(val name: String, val address: Address?)

class Person(val name: String, val company: Company?)

fun printShippingLabel(person: Person) {
    val address = person.company?.address
    ?: throw IllegalArgumentException("No address") ← Если адрес отсутствует,
                                                    то выбрасывается
                                                    исключение
    with (address) { ← Переменная address не равна null
        println(streetAddress)
        println("$zipCode $city, $country")
    }
}

fun main() {
    val address = Address("Elsestr. 47", 80687, "Munich", "Germany")
    val jetbrains = Company("JetBrains", address)
    val person = Person("Dmitry", jetbrains)
    printShippingLabel(person)
    // Elsestr. 47
    // 80687 Munich, Germany
    printShippingLabel(Person("Alexey", null))
    // java.lang.IllegalArgumentException: No address
}
```

Если все правильно, то функция `printShippingLabel` выводит этикетку на печать. Когда адрес не указан, то код не просто выбрасывает `NullPointerException` с номером строки, а сообщает о значимой ошибке. Если адрес указан, то метка состоит из адреса улицы, почтового индекса, города и страны. Обратите внимание на способ использования функции `with` (с которой вы познакомились в подразделе 5.4.1), который препятствует тому, чтобы переменная `address` повторялась четыре раза подряд.

Вы изучили реализацию проверок «если не `null`» в Kotlin. Далее обсудим безопасную версию проверки `instanceof` в Kotlin — оператор безопасного приведения типа. Он часто применяется с безопасными вызовами и оператором объединения с `null`.

7.6. БЕЗОПАСНОЕ ПРИВЕДЕНИЕ ЗНАЧЕНИЙ БЕЗ ВЫБРОСА ИСКЛЮЧЕНИЙ: ОПЕРАТОР AS?

В главе 2 описан обычный оператор Kotlin для приведения типов — `as`. Как и при обычном приведении типа в Java, `as` выбрасывает исключение `ClassCastException`, если отсутствует тип значения, к которому выполняется приведение. Конечно, можно совместить его с проверкой `is`, чтобы убедиться в правильности типа. Kotlin — безопасный и лаконичный язык, нет ли в нем лучшего решения? Несомненно, есть.

Оператор `as?` пытается привести значение к указанному типу и возвращает `null`, если у значения неподходящий тип. Это показано на рис. 7.4.

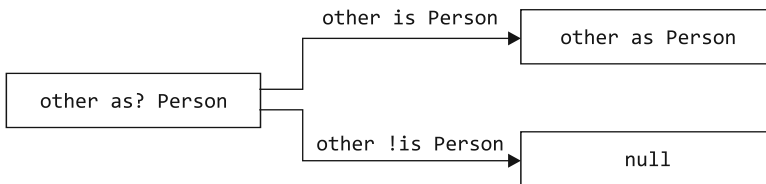


Рис. 7.4. Оператор безопасного приведения `as?` предоставляет инструменты для безопасной работы с вероятностью неудачного приведения типа. Он пытается привести значение к заданному типу и возвращает значение `null`, если тип отличается

Один из распространенных вариантов использования безопасного приведения из листинга 7.6 — сочетание его с оператором `?:`. Это может быть удобно, например, при реализации метода `equals`.

Листинг 7.6. Использование безопасного приведения типа для реализации метода `equals`

```

class Person(val firstName: String, val lastName: String) {
    override fun equals(other: Any?): Boolean {
        val otherPerson = other as? Person ?: return false
        return otherPerson.firstName == firstName &&
            otherPerson.lastName == lastName
    }

    override fun hashCode(): Int =
        firstName.hashCode() * 37 + lastName.hashCode()
}

fun main() {
    val p1 = Person("Dmitry", "Jemerov")
    val p2 = Person("Dmitry", "Jemerov")
    println(p1 == null)
    // ложь
    println(p1 == p2)
    // истина
    println(p1.equals(42))
    // ложь
}
  
```

Проверка типа и возврат значения false, если совпадения не обнаружено

После безопасного приведения к переменной `otherPerson` применяется умное преобразование к типу `Person`

Оператор `==` вызывает метод `equals`

С помощью этого паттерна можно легко проверить соответствие типа, привести его и вернуть значение `false`, если тип не подходит, — и все это в одном выражении. Разумеется, умные преобразования применимы и в данном контексте. После проверки типа и отбрасывания значений `null` компилятор знает, что тип значения переменной `otherPerson` будет `Person`. Поэтому допускается использовать его соответствующим образом.

Операторы безопасного вызова, безопасного приведения и операторы объединения с `null` полезны и часто встречаются в коде Kotlin. Но иногда поддержка языка в работе со значениями `null` не требуется — просто нужно сообщить компилятору, что значение на самом деле не равно `null`. Рассмотрим, как этого добиться.

7.7. ОБЕЩАНИЯ КОМПИЛЯТОРУ С ПОМОЩЬЮ ОПЕРАТОРА УТВЕРЖДЕНИЯ «НЕ NULL»: !!

Утверждение «не `null`» — самый простой и грубый инструмент в Kotlin для работы со значением типа, допускающего значение `null`. Оно обозначается двойным восклицательным знаком и преобразует любое значение в тип, который такое значение не допускает. Для значений `null` выбрасывается исключение. Логика показана на рис. 7.5.

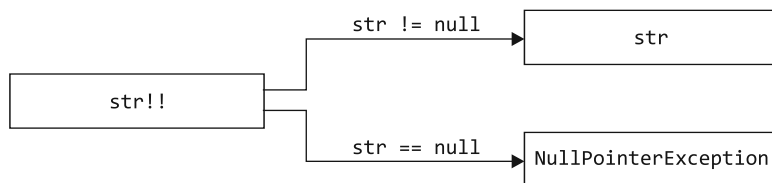


Рис. 7.5. Используя утверждение «не `null`», нет необходимости явно обрабатывать значение как `null`. Вместо этого при обнаружении значения `null` выбрасывается исключение

Приведем тривиальный пример функции, в которой утверждение используется для преобразования аргумента, допускающего значение `null`, в его антипода (листинг 7.7).

Листинг 7.7. Использование утверждения «не `null`»

```

fun ignoreNulls(str: String?) {
    val strNotNull: String = str!! ← Исключение указывает на эту строку
    println(strNotNull.length)
}

fun main() {
    ignoreNulls(null)
    // Exception in thread "main" java.lang.NullPointerException
    // at <...>.ignoreNulls(07_NotnullAssertions.kt:2)
}
  
```

Что произойдет, если в этой функции аргумент `str` примет значение `null`? В Kotlin нет особого выбора — во время выполнения будет выброшено исключение. Но обратите внимание, что точкой возникновения исключения становится само утверждение, а не последующая строка с попыткой использовать значение. По сути, вы говорите компилятору: «Я знаю, что это значение не равно `null`, и готов к исключению, если окажется, что я ошибаюсь».

ПРИМЕЧАНИЕ

Можно заметить, что двойной восклицательный знак выглядит немного грубо, как будто вы кричите на компилятор. Это сделано намеренно. Разработчики Kotlin пытаются подтолкнуть вас к поиску лучшего решения, не требующего утверждений, которые не могут быть проверены компилятором.

Но бывают ситуации, когда утверждения «не `null`» становятся подходящим решением задачи. Когда выполняется проверка `null` в одной функции, но значение используется в другой, компилятор не может распознать, что использование безопасно. Если проверка всегда выполняется в другой функции, то можно не дублировать ее перед использованием значения. В таком случае вместо проверки можно использовать утверждение «не `null`».

На практике это происходит с классами действий, встречающимися в UI-фреймворках. В классе действия есть отдельные методы для обновления состояния действия (включения или выключения) и для его выполнения. Проверки в методе `update` гарантируют, что метод `execute` не будет вызван, если условия не выполняются. Но компилятор не способен распознать эту логику.

Рассмотрим гипотетический пример класса действия с применением утверждения «не `null`» в такой ситуации (листинг 7.8). Действие `CopyRowAction`, работающее с классом `SelectableTextList`, должно копировать значение выбранной строки списка в буфер обмена. Мы опустили все ненужные детали и оставили только код проверки выбора строки (это значит, действие может быть выполнено) и код получения значения для выбранной строки. Здесь подразумевается, что функция `executeCopyRow` вызывается только в случае, если функция `isActionEnabled` содержит значение `true`. Это означает еще и то, что список элементов `list.selectedIndex` никогда не будет равен `null` при вызове функции `executeCopyRow` (даже если компилятор этого не знает).

Листинг 7.8. Использование утверждения «не `null`» в классе действия

```
class SelectableTextList(
    val contents: List<String>,
    var selectedIndex: Int? = null,
)

class CopyRowAction(val list: SelectableTextList) {
    fun isActionEnabled(): Boolean =
        list.selectedIndex != null
}
```

```
fun executeCopyRow() {
    val index = list.selectedIndex!!
    val value = list.contents[index]
    // копирование значения в буфер обмена
}
```

← Функция `executeCopyRow` вызывается, только если функция `isEnabled` возвращает значение `true`

В данном случае можно и не использовать оператор `!!`, а написать выражение `val index = list.selectedIndex ?: return`, чтобы получить индекс в виде типа, не допускающего значение `null`. Если вы примените этот паттерн, то nullable-значение `list.selectedIndex` приведет к преждевременному возврату из функции, поэтому значение `value` всегда будет не nullable. Проверка «не `null`» с помощью оператора `?:` здесь будет излишеством, но ее можно использовать, чтобы избежать усложнения функции `isEnabled` в будущем.

Дадим еще одно предостережение: если при использовании оператора `!!` выбрасывается исключение, то в трассировке стека указывается только номер строки, в которой оно возникло, но не конкретное выражение. Чтобы было понятно, какое именно значение содержало `null`, лучше избегать использования нескольких утверждений с оператором `!!` в одной строке:

```
person.company!!.address!!.country ← Писать такой код не стоит!
```

Если исключение укажет на эту строку, то определить, где было значение `null`: в переменной `company` или `address`, не получится.

Мы обсуждали, как получить доступ к значениям типов, допускающих значение `null`. Но что делать, если требуется передать nullable-значение в качестве аргумента в функцию, которая ожидает значение, не являющееся nullable? Компилятор не позволяет осуществлять это без проверки из-за снижения безопасности. В языке Kotlin нет специальной поддержки для этого случая, но есть подходящая функция стандартной библиотеки — `let`.

7.8. РАБОТА С ВЫРАЖЕНИЯМИ, ДОПУСКАЮЩИМИ ЗНАЧЕНИЕ NULL: ФУНКЦИЯ LET

Функция `let` облегчает работу с выражениями, допускающими значение `null`. Эта функция с оператором безопасного вызова позволяет вычислить выражение, проверить результат на `null` и сохранить результат в переменной — и все это в одном, кратком выражении.

Один из самых распространенных вариантов применения — обработка аргумента, допускающего значение `null`, который должен быть передан в функцию, ожидающую параметр, не допускающий такое значение. Допустим, функция `sendEmailTo` принимает один параметр типа `String` и отправляет письмо на этот адрес. Эта функция написана на языке Kotlin и требует параметра, который не допускает значение `null`:

```
fun sendEmailTo(email: String) { /*...*/ }
```

Нельзя передать в эту функцию значение типа, допускающего значение `null`:

```
fun main() {
    val email: String? = "foo@bar.com"
    sendEmailTo(email)
    //ERROR: Type mismatch: inferred type is String? but String was expected
}
```

Необходимо выполнить проверку на `null` в явном виде:

```
if (email != null) sendEmailTo(email)
```

Но можно пойти другим путем: применить функцию `let` и вызвать ее через безопасный вызов. Функция `let` только превращает обрабатываемый объект в параметр лямбды. В этом смысле она похожа на некоторые другие функции области видимости, представленные в разделе 5.4. Но если сочетать ее с синтаксисом безопасного вызова, то она эффективно преобразует объект типа, допускающего значение `null`, обрабатываемый функцией `let`, в объект типа, который это значение не допускает (рис. 7.6).

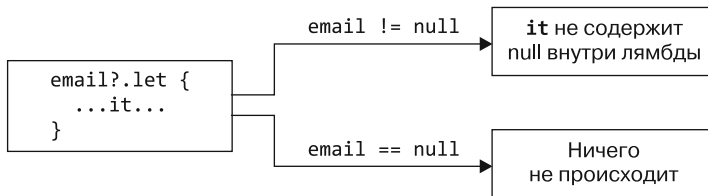


Рис. 7.6. Функция `let` с оператором безопасного вызова позволяет указать лямбду, и она будет выполнена, только если выражение не содержит `null`. Это особенно полезно при работе с результатом цепочки выражений, поскольку он может быть nullable-значением

Функция `let` будет вызвана, только если значение адреса электронной почты не является `null`, поэтому переменная `email` используется в качестве аргумента лямбды, допускающего значение `null`:

```
email?.let { email -> sendEmailTo(email) }
```

Если перейти к короткому синтаксису с использованием автоматически генерируемого имени `it`, то результат будет гораздо более лаконичным: `email?.let {sendEmailTo(it)}`. В листинге 7.9 приведен более полный пример, в котором показан этот паттерн.

Листинг 7.9. Использование функции `let` для вызова функции с параметром, не допускающим значение `null`

```
fun sendEmailTo(email: String) {
    println("Sending email to $email")
}

fun main() {
    var email: String? = "yole@example.com"
```

```

email?.let { sendEmailTo(it) }
// Отправка электронного письма на yole@example.com
email = null
email?.let { sendEmailTo(it) }
}

```

Обратите внимание, что нотация функции `let` особенно удобна, когда нужно использовать значение длинного выражения, если оно не принимает значение `null`. В этом случае не нужно создавать отдельную переменную. Сравните этот вариант кода с проверкой `if` в явном виде:

```

val person: Person? = getTheBestPersonInTheWorld()
if (person != null) sendEmailTo(person.email)

```

с тем же фрагментом кода, но без дополнительной переменной:

```

getTheBestPersonInTheWorld()?.let { sendEmailTo(it.email) }

```

Эта функция возвращает значение `null`, поэтому код в лямбда-выражении не выполняется:

```

fun getTheBestPersonInTheWorld(): Person? = null

```

Если требуется проверить несколько значений на `null`, то можно применять вложенные вызовы функции `let` для их обработки. Но в большинстве случаев такой код получается довольно многословным и сложным для понимания. Обычно проще использовать обычное выражение `if` для совместной проверки всех значений.

СРАВНЕНИЕ ФУНКЦИЙ ОБЛАСТИ ВИДИМОСТИ В KOTLIN: КОГДА ИСПОЛЬЗОВАТЬ WITH, APPLY, LET, RUN И ALSO

В последних нескольких главах подробно описаны несколько функций с очень похожими сигнатурами: `with`, `apply`, `let`, `run` и `also`.

Все эти *функции области видимости* (также их называют *функциями контекста*) выполняют блок кода в контексте объекта. Они отличаются способом обращения к объекту внутри лямбды, а также возвращаемым значением.

Функция	Доступ к <code>x</code> с помощью	Возвращаемое значение
<code>x.let { ... }</code>	<code>it</code>	Результат выполнения лямбды
<code>x.also { ... }</code>	<code>it</code>	<code>x</code>
<code>x.apply { ... }</code>	<code>this</code>	<code>x</code>
<code>x.run { ... }</code>	<code>this</code>	Результат выполнения лямбды
<code>with(x) { ... }</code>	<code>this</code>	Результат выполнения лямбды

Различия между ними довольно тонкие. Поэтому стоит еще раз обратить внимание на то, для каких именно задач подходит каждая из них, чтобы можно было сравнить их между собой.

- Функция `let` используется вместе с оператором безопасного вызова `?.` для выполнения блока кода только тогда, когда обрабатываемый объект не равен `null`. Функция `let` используется отдельно для превращения выражения в переменную, ограниченную областью видимости его лямбды.
- Функция `apply` позволяет настраивать свойства объекта с помощью API в стиле конструктора (например, при создании экземпляра).
- Функция `also` применяется для выполнения дополнительных действий, использующих ваш объект. При этом исходный объект передается в дальнейшие цепочки операций.
- С помощью функции `with` можно группировать вызовы функций одного и того же объекта, не повторяя его имя.
- Функция `run` используется для конфигурирования объекта и вычисления пользовательского результата.

Использование различных функций области видимости различается нюансами, поэтому вы можете оказаться в ситуации, когда несколько таких функций покажутся подходящими. В таких случаях имеет смысл договориться об общих соглашениях для команды или разрабатываемого проекта.

Еще одна распространенная ситуация — невозможность инициализировать в конструкторе «не `null`»-значением свойство, относящееся к типу, который не допускает значение `null`. Рассмотрим предложенные в Kotlin решения для этой ситуации.

7.9. «НЕ NULL»-ТИПЫ С ПОЗДНЕЙ ИНИЦИАЛИЗАЦИЕЙ: СВОЙСТВА С ОТЛОЖЕННОЙ ИНИЦИАЛИЗАЦИЕЙ

Многие фреймворки инициализируют объекты в специальных методах, вызываемых после создания экземпляра объекта. Например, в Android инициализация объекта `activity` происходит в методе `onCreate`. В фреймворке JUnit принято помещать логику инициализации в методы с аннотацией `@BeforeAll` или `@BeforeEach`.

Но нельзя оставить свойство, не допускающее значение `null`, без инициализатора в конструкторе и инициализировать его только в специальном методе. Kotlin обычно необходимо инициализировать все свойства в конструкторе, и если свойство имеет тип, не допускающий значение `null`, то вы должны предоставить «не `null`»-значение инициализатора. Если сделать это невозможно, то вместо него следует использовать тип, допускающий значение `null`. При этом каждое обращение к свойству потребует либо проверки `null`, либо оператора `!!` (листинг 7.10).

Листинг 7.10. Использование утверждений «не null» для доступа к свойству, допускающему значение null

```
class MyService {
    fun performAction(): String = "Action Done!"
}

@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class MyTest {
    private var myService: MyService? = null

    @BeforeAll fun setUp() {
        myService = MyService()
    }

    @Test fun testAction() {
        assertEquals("Action Done!", myService!!.performAction())
    }
}
```

Объявление свойства типа, допускающего значение null, для инициализации его с таким значением

Предоставление действительного инициализатора в методе setUp

Необходимо позаботиться о null-безопасности — используйте оператор !! или ?

Такой код выглядит некрасиво, особенно при многократном обращении к свойству. Чтобы решить эту проблему, можно объявить свойство `myService` с *отложенной инициализацией*. Это делается с помощью модификатора `lateinit` (листинг 7.11).

Листинг 7.11. Использование свойства с отложенной инициализацией

```
class MyService {
    fun performAction(): String = "Action Done!"
}

@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class MyTest {
    private lateinit var myService: MyService

    @BeforeAll fun setUp() {
        myService = MyService()
    }

    @Test fun testAction() {
        assertEquals("Action Done!", myService.performAction())
    }
}
```

Объявление свойства «не null»-типа без инициализатора

Обычная инициализация свойства в методе setUp

Получение доступа к свойству без дополнительных null-проверок

Обратите внимание, что поздняя инициализация свойства всегда помечена ключевым словом `var`, поскольку необходимо сохранить возможность изменять его значение вне конструктора. Тогда как свойства `val` компилируются в конечные поля, и должны быть инициализированы в конструкторе. Но больше нет необходимости инициализировать свойство в конструкторе, даже если его тип не допускает значение null. При обращении к свойству до его инициализации вы получите следующий результат:

```
kotlin.UninitializedPropertyAccessException:
    lateinit property myService has not been initialized
```

Здесь четко определено, что произошло, и такое сообщение гораздо легче понять, чем общее исключение `NullPointerException` (рис. 7.7).

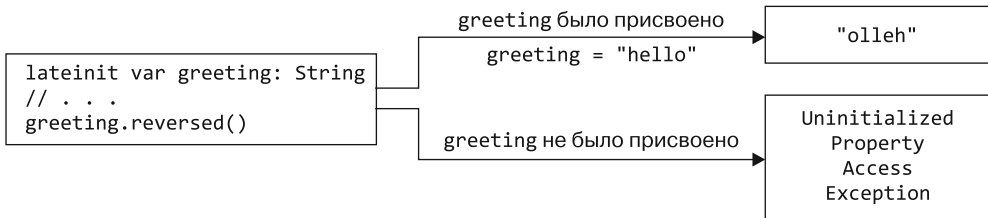


Рис. 7.7. Свойство `lateinit` типа, не допускающего значение `null`, но ему не нужно сразу присваивать значение. Не следует обращаться к переменной до того, как ей будет присвоено значение

Свойства `lateinit` обычно используются в сочетании с Java-фреймворками, предназначенными для внедрения зависимостей, такими как Google Guice. В этом сценарии значения свойств `lateinit` задаются фреймворком извне. Чтобы обеспечить совместимость с широким спектром Java-фреймворков, Kotlin генерирует поле с той же видимостью, что и у свойства `lateinit`. Если свойство объявляется с модификатором `public`, то поле также станет `public`.

ПРИМЕЧАНИЕ

Модификатор `lateinit` не ограничивается свойствами классов. Можно указать локальные переменные внутри тела функции или лямбды и свойства верхнего уровня с отложенной инициализацией.

Теперь посмотрим, как можно расширить набор инструментов Kotlin для работы со значениями `null`. Вы научитесь делать это, определяя функции-расширения для типов, допускающих это значение.

7.10. РАСШИРЕНИЕ ТИПОВ БЕЗ ОПЕРАТОРА БЕЗОПАСНОГО ВЫЗОВА: РАСШИРЕНИЯ ДЛЯ ТИПОВ, ДОПУСКАЮЩИХ ЗНАЧЕНИЕ NULL

Определение функций-расширений для типов, допускающих значение `null`, — это еще один мощный способ работы с такими значениями. Вместо того чтобы гарантировать, что переменная не может быть `null` перед вызовом метода, можно разрешить вызовы с `null` в качестве приемника и работать со значением `null` в функции. Это возможно только для функций-расширений. Обычные же вызовы членов пересылаются через экземпляр объекта, и, следовательно, их нельзя выполнить, когда экземпляр содержит `null`.

В качестве примера рассмотрим функции `isEmpty` и `isBlank`. Они определены как расширения класса `String` в стандартной библиотеке Kotlin. Первая проверяет, пуста ли строка `""`, а вторая — пуста ли строка или состоит только из пробельных

символов. Обычно эти функции используются для проверки того, что строка не-тривиальна, чтобы выполнить с ней значимую операцию. Может показаться, что было бы полезно обрабатывать `null` так же, как и тривиальные пустые или незаполненные строки. И осуществить это возможно: функции `isEmpty` и `isNullOrEmpty` можно вызвать с приемником типа `String?` (листинг 7.12).

Листинг 7.12. Вызов функции-расширения с приемником, допускающим значение `null`

```
fun verifyUserInput(input: String?) {
    if (input.isNullOrEmpty()) { ← Безопасный вызов не требуется
        println("Please fill in the required fields")
    }
}

fun main() {
    verifyUserInput(" ")
    // Please fill in the required fields
    verifyUserInput(null)
    // Please fill in the required fields ← При вызове функции isNullOrEmpty
    //                                     со значением null исключение
    //                                     не выбрасывается
}
```

Функцию-расширение, объявленную для приемника, допускающего значение `null`, можно вызвать без безопасного доступа (рис. 7.8). Функции обрабатывают возможные значения `null`.

Следовательно, к ним можно обращаться без безопасного вызова.

Функция `isNullOrEmpty` выполняет проверку `null` в явном виде и возвращает значение `true` при обнаружении. А после вызывает функцию `isBlank`, которую можно вызвать только для типа `String`, не допускающего значение `null`:

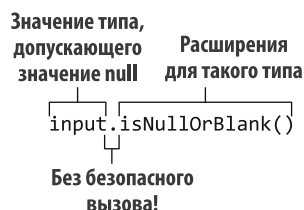


Рис. 7.8. Расширения для типов, допускающих значение `null`, сами знают, как обрабатывать случай `null` для своего приемника

```
fun String?.isNullOrEmpty(): Boolean = ← Расширение для типа String,
    this == null || this.isBlank()      не допускающего значение null
    ← Ко второй ссылке this применяется умное преобразование типа
```

При объявлении функции-расширения для типа, допускающего значение `null` (с модификатором `?` в конце), можно вызывать эту функцию на nullable-значениях. Ссылка `this` в теле функции может принимать значение `null`, поэтому необходимо проверять это в явном виде. В Java `this` всегда относится к не `null`-значениям, поскольку ссылается на экземпляр текущего класса. Для Kotlin это уже несправедливо: в функции-расширении для типа, допускающего значение `null`, ссылка `this` может принимать `null`.

Обратите внимание, что функцию `let` можно вызвать и на приемнике, допускающем значение `null`, но она не проверяет значение на `null`. Если вызвать ее для типа, допускающего значение `null`, без использования оператора безопасного вызова, то аргумент лямбды тоже будет допускать это значение. Кроме того, это означает,

что переданный блок кода всегда будет выполняться, независимо от того, окажется значение `null` или нет:

```
fun sendEmailTo(email: String) {
    println("Sending email to $email")
}

fun main() {
    val recipient: String? = null
    recipient.let { sendEmailTo(it) } ← Безопасный вызов отсутствует,
    // ERROR: Type mismatch: следовательно, будет тип,
    // inferred type is String? but String was expected допускающий значение null
}
```

Поэтому, если требуется проверить аргументы на «не `null`» с помощью функции `let`, необходимо использовать оператор безопасного вызова `?..` Ранее мы уже приводили такой пример: `recipient?.let { sendEmailTo(it) }`.

ПРИМЕЧАНИЕ

Когда вы определяете собственную функцию-расширение, то необходимо подумать, следует ли определять ее как расширение для типа, допускающего значение `null`. По умолчанию определяйте ее как расширение для типа, который таковым не является. Позже ее можно смело изменить — код не будет нарушен. Такая необходимость может возникнуть, если окажется, что функция используется в основном для nullable-значений и значение `null` может быть адекватно обработано.

В этом разделе вы узнали нечто неожиданное. Если вы разыменовываете переменную, не выполняя дополнительную проверку, как в функции `s.isNullOrBlank()`, это не означает, что переменная становится «не `null`» — функция может быть расширением для типа, допускающего значение `null`. Далее обсудим еще один удивительный случай — типовой параметр может допускать это значение, даже не имея вопросительного знака в конце.

7.11. NULL-БЕЗОПАСНОСТЬ ТИПОВЫХ ПАРАМЕТРОВ

По умолчанию все обобщенные типовые параметры функций и классов в Kotlin относятся к типу, допускающему значение `null`. Любой тип, в том числе и такой, может быть заменен на типовой параметр. В таком случае объявления с ним в качестве типа могут принимать значение `null`, даже если типовой параметр `T` не заканчивается вопросительным знаком. Рассмотрим следующий пример (листинг 7.13).

Листинг 7.13. Работа с типовым параметром, допускающим значение `null`

```
fun <T> printHashCode(t: T) {
    println(t?.hashCode()) ← Необходимо использовать
}                                     безопасный вызов, поскольку t
                                     может принимать значение null

fun main() {
    printHashCode(null) ← T определяется как Any?
    // null
}
```

В вызове функции `printHashCode` выведенным типом для типового параметра `T` стал тип, допускающий значение `null`, — `Any?`. Поэтому параметр `t` может содержать значение `null`, даже не имея вопросительного знака после `T`.

Чтобы сделать типовой параметр не допускающим значение `null`, нужно указать для него верхнюю границу, которая не допускает такое значение (листинг 7.14). Это отклонит nullable-значение в качестве аргумента.

Листинг 7.14. Объявление верхней границы, не допускающей значение `null`, для типового параметра

```
fun <T: Any> printHashCode(t: T) { ← Теперь T не может быть nullable
    println(t.hashCode())
}

fun main() {
    printHashCode(null) ←
    // Error: Type parameter bound for `T` is not satisfied
    printHashCode(42)
    // 42
}
```

Этот код не компилируется —
нельзя передать значение `null`,
поскольку ожидается «не `null`»

В подразделе 11.1.4 мы более подробно рассмотрим обобщения в Kotlin. Обратите внимание, что типовые параметры — единственное исключение из правила, согласно которому для обозначения типа как допускающего значение `null` требуется вопросительный знак в конце, а типы без вопросительного знака относятся к не допускающим это значение. В следующем разделе показан еще один особый случай null-безопасности — типы, поступающие из кода Java.

7.12. NULL-БЕЗОПАСНОСТЬ И JAVA

Итак, мы рассмотрели инструменты для работы со значениями `null` в Kotlin. Но он знаменит своей совместимостью с Java, а этот язык не поддерживает null-безопасность в своей системе типов. Что же происходит при объединении кода Kotlin и Java? Утратится ли вся безопасность, или придется проверять каждое значение на `null`? А может быть, существует более удачное решение? Выясним это.

Начнем с того, что иногда код Java содержит информацию о null-безопасности, выраженную с помощью аннотаций. Когда эта информация есть в коде, Kotlin использует ее. Таким образом, `@Nullable String` в Java рассматривается как `String?` в Kotlin, а `@NotNull String` — это просто `String` (рис. 7.9).

Kotlin распознает множество различных вариантов аннотаций null-безопасности, в том числе аннотации из стандарта JSR-305 (в пакете `javax.annotation`),

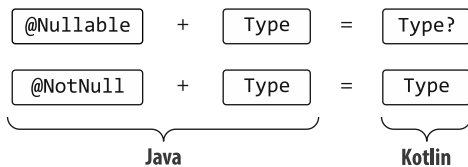


Рис. 7.9. Аннотированные типы Java представлены в Kotlin как типы, допускающие и не допускающие значение `null`, в соответствии с аннотациями. Эти типы могут либо хранить значения `null` в явном виде, либо не будут типами, допускающими данное значение

аннотации для Android (`android.support.annotation`), поддерживаемые инструментами JetBrains (`org.jetbrains.annotations`). Интересно, что происходит, когда аннотации отсутствуют. В этом случае тип код Java становится платформенным типом в Kotlin.

7.12.1. Платформенные типы

Для *платформенного типа* у Kotlin нет информации о `null`-безопасности. Работать с ним можно как типом, допускающим или не допускающим значение `null` (рис. 7.10).

Это означает, что, как и в Java, вся ответственность за операции с этим типом лежит на разработчике. Компилятор разрешит выполнение всех операций. Он также не будет выделять любые `null`-безопасные операции над такими значениями как избыточные. Это обычно происходит при выполнении `null`-безопасной операции над значением типа, не допускающего значение `null`. Если известно, что значение может принимать `null`, то можно сравнить его с `null` перед использованием.

Если известно, что значение не будет принимать `null`, то можно использовать его напрямую. Как и в Java, в случае ошибки в точке использования значения будет выброшено исключение `NullPointerException`. Допустим, класс `Person` объявлен на языке Java (листинг 7.15).

Листинг 7.15. Класс Java без аннотаций `null`-безопасности

```
/* Java */
public class Person {
    private final String name;

    public Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Может ли функция `getName` вернуть значение `null`? В данном случае компилятор Kotlin ничего не знает о `null`-безопасности типа `String`, поэтому вам предстоит решать данную задачу самостоятельно. Если вы уверены, что переменная имени не принимает значение `null`, то можно разыменовать ее обычным способом, как в Java, без дополнительных проверок (листинг 7.16). Но в таком случае будьте готовы получить исключение.

Другой вариант — интерпретировать возвращаемый функцией `getName()` тип как допускающий значение `null` и безопасно обращаться к нему (листинг 7.17).

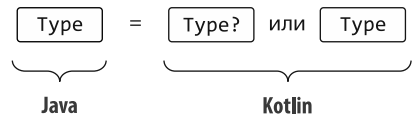


Рис. 7.10. Типы Java без специальных аннотаций представлены в Kotlin как платформенные. Можно использовать их как типы, допускающие или не допускающие значение `null`

Листинг 7.16. Доступ к классу Java без null-проверок

```

fun yellAt(person: Person) {
    println(person.name.uppercase() + "!!!")
}

fun main() {
    yellAt(Person(null))
    // java.lang.NullPointerException: person.name must not be null
}

```

← Приемник person.name вызова функции uppercase() равен null, поэтому возникает исключение

Листинг 7.17. Доступ к классу Java с использованием проверки null

```

fun yellAtSafe(person: Person) {
    println((person.name ?: "Anyone").uppercase() + "!!!")
}

fun main() {
    yellAtSafe(Person(null))
    // ANYONE!!!
}

```

В этом примере значения null обрабатываются правильно и исключение во время выполнения не возникает.

Следует проявить осторожность при работе с API Java. Большинство библиотек не аннотированы, поэтому вы можете интерпретировать все типы как не допускающие значение null, но это приведет к ошибкам. Чтобы избежать ошибок, следует проверить документацию (и, если нужно, реализацию) используемых методов Java. В ней можно узнать, когда методы могут возвращать значение null, и добавить проверки для них.

ПОЧЕМУ ИМЕННО ПЛАТФОРМЕННЫЕ ТИПЫ?

Не будет ли безопаснее для Kotlin рассматривать все значения, приходящие из Java, как nullable? Такой подход возможен, но он потребует большого количества избыточных проверок null для значений, которые вообще не принимают null, поскольку компилятор Kotlin не сможет увидеть эту информацию.

Особенно плохо обстоят дела с обобщениями. Например, каждый элемент `ArrayList<String>` из Java примет вид `ArrayList<String?>` в Kotlin. И вам придется проверять значения на null при каждом обращении или использовать приведение типов, что перечеркнет все преимущества безопасности. Написание таких проверок очень раздражает, поэтому создатели Kotlin поступили прагматично и позволили разработчикам взять на себя ответственность за правильную обработку значений, поступающих из Java.

В Kotlin нельзя объявить переменную платформенного типа — эти типы могут только поступить из кода Java. Но их можно встретить в сообщениях об ошибках и в IDE:

```

val i: Int = person.name
// ERROR: Type mismatch: inferred type is String! but Int was expected

```


Нотация `String!` — это то, как компилятор и среды разработки Kotlin (такие как IntelliJ IDEA и Android Studio) обозначают платформенные типы, поступающие из кода Java (рис. 7.11). Такой синтаксис нельзя использовать при написании кода. И обычно этот восклицательный знак не связан с источником проблемы, поэтому его можно игнорировать. Он просто подчеркивает, что null-безопасность типа неизвестна.

Интерпретировать такие типы можно как угодно: и как допускающие, и как не допускающие значение `null`, поэтому оба следующих объявления действительны:

```
val s: String? = person.name
val s1: String = person.name
```

← Свойство Java может быть представлено как допускающее значение `null`...
← ...или как не допускающее

В такой ситуации, как и в случае с вызовами методов, необходимо убедиться, что null-безопасность указана верно. Если попытаться присвоить переменной Kotlin, не допускающей значение `null`, это значение, поступившее из Java, то в точке присвоения возникнет исключение.

Мы поговорили о том, как типы Java воспринимаются в Kotlin. Теперь обсудим некоторые трудности, возникающие при создании смешанных иерархий Kotlin и Java.

7.12.2. Наследование

При переопределении метода Java в Kotlin можно объявлять параметры и возвращаемый тип как допускающий или не допускающий значение `null`. Для примера рассмотрим интерфейс `StringProcessor` в Java (листинг 7.18).

Листинг 7.18. Интерфейс Java с параметром `String`

```
/* Java */
interface StringProcessor {
    void process(String value);
}
```

Компилятор Kotlin примет обе следующие реализации (листинг 7.19).

Листинг 7.19. Реализация интерфейса Java с различной null-безопасностью параметров

```
class StringPrinter : StringProcessor {
    override fun process(value: String) {
        println(value)
    }
}

class NullableStringPrinter : StringProcessor {
    override fun process(value: String?) {
```

```
fun main() {
    val s: String! = p.name
}
```

Рис. 7.11. При использовании вывода типа для свойства Java IntelliJ IDEA и Android Studio указывают, что вы работаете с платформенным типом, если включены внутритекстовые подсказки для типов Kotlin. Восклицательный знак позволяет с первого взгляда определить эти платформенные типы

```
        if (value != null) {  
            println(value)  
        }  
    }  
}
```

Обратите внимание, что при реализации методов из классов или интерфейсов Java важно правильно определить `null`-безопасность. Методы реализации могут быть вызваны из стороннего кода, поэтому компилятор Kotlin будет генерировать «не `null`»-утверждения для каждого параметра, который вы объявляете с типом, не допускающим значение `null`. Если код Java передаст методу данное значение, то утверждение сработает и будет выброшено исключение. Это произойдет, даже если в своей реализации вы никогда не обращались к значению параметра.

РЕЗЮМЕ

- Поддержка типов в Kotlin, допускающих значение `null`, позволяет обнаружить возможные ошибки типа `NullPointerException` во время компиляции.
- Обычные типы по умолчанию не допускают значение `null`, если явно не помечены как типы, которые допускают это значение. Вопросительный знак после имени типа указывает на то, что он относится к типу, допускающему значение `null`.
- Kotlin предоставляет множество инструментов для лаконичной работы с типами, допускающими значение `null`.
- Безопасные вызовы (`?.`) позволяют вызывать методы и получать доступ к свойствам объектов, допускающих значение `null`.
- Оператор объединения с `null` (`?:`) позволяет задать значение по умолчанию для выражения, которое может принимать `null`, прервать выполнение или вызвать исключение.
- С помощью утверждений «не `null`» (`!!`) можно пообещать компилятору, что данное значение не принимает `null` (но будьте готовы к возникновению исключения, если обещание будет нарушено).
- Функция области видимости `let` превращает обрабатываемый объект в параметр для лямбда-выражения. Вместе с оператором безопасного вызова она эффективно преобразует объект типа, допускающего значение `null`, в объект типа, не допускающего это значение.
- Использование оператора `as?` — простой способ приведения значения к типу и обработки случая, когда его тип отличается.

8

Базовые типы, коллекции и массивы

В этой главе

- ✓ Примитивы и другие базовые типы, а также их соответствие типам Java.
- ✓ Коллекции и массивы Kotlin, их истории null-безопасности и совместимости.

Помимо поддержки null-безопасности, система типов Kotlin обладает несколькими важными функциями для повышения надежности кода и реализует множество решений, полученных на основе работы других систем типов, в том числе Java. Эти решения определяют, как выполняется работа со всеми сущностями в коде Kotlin, от примитивных значений и базовых типов до иерархии коллекций в стандартной библиотеке Kotlin. В этом языке вводятся такие функциональные возможности, как *коллекции, доступные только для чтения*. Кроме того, в нем доработаны или скрыты проблематичные или бесполезные части системы типов, например поддержка массивов первого класса. Рассмотрим их подробнее, начиная с базовых строительных блоков.

8.1. ПРИМИТИВЫ И ДРУГИЕ БАЗОВЫЕ ТИПЫ

В этом разделе приводится описание базовых типов для программ: `Int`, `Boolean` и `Any`. В отличие от Java Kotlin не различает примитивные типы и обертки. Вскоре вы узнаете, в чем причина и как это работает внутри системы. Вдобавок мы поговорим о соответствии между типами Kotlin и такими типами Java, как `Object` и `Void`.

8.1.1. Представление целых чисел, чисел с плавающей запятой, символов и логических значений с помощью примитивных типов

В Java существует различие между примитивными и ссылочными типами. Переменная *примитивного типа* (например, `int`) непосредственно хранит свое значение. А переменная *ссылочного типа* (например, `String`) содержит ссылку на область памяти, содержащую объект.

Значения примитивных типов можно хранить и передавать более эффективно, но вызывать методы для таких значений или хранить их в коллекциях невозможно. Java предоставляет специальные типы-обертки (например, `java.lang.Integer`). Они инкапсулируют примитивные типы в ситуациях, в которых требуется объект. Следовательно, чтобы определить коллекцию целых чисел, нельзя написать `Collection<int>`, вместо этого нужно использовать выражение `Collection<Integer>`.

В Kotlin нет различия между примитивными типами и типами-обертками — всегда используется один и тот же тип (например, `Int`):

```
val i: Int = 1
val list: List<Int> = listOf(1, 2, 3)
```

Это очень удобно. Более того, можно вызывать методы для значений числового типа. В качестве примера рассмотрим фрагмент кода ниже. В нем используется функция `coerceIn` из стандартной библиотеки для ограничения диапазона возможных значений:

```
fun showProgress(progress: Int) {
    val percent = progress.coerceIn(0, 100)
    println("We're $percent % done!")
}

fun main() {
    showProgress(146)
    // We're 100 % done!
}
```

Если примитивные и ссылочные типы одинаковы, то значит ли это, что Kotlin представляет все числа как объекты? Разве это не было бы очень неэффективно? Действительно, это крайне затратно, поэтому в Kotlin такого нет.

Во время выполнения программы типы чисел представляются наиболее эффективным способом. В большинстве случаев — для переменных, свойств, параметров и возвращаемых типов — тип `Int` в Kotlin компилируется в примитивный тип Java `int`. Это невозможно только при использовании классов обобщений, таких как коллекции. Примитивный тип, используемый в качестве аргумента типа класса обобщения, компилируется в соответствующий тип-обертку Java. Например, если аргумент типа коллекции — `Int`, то коллекция будет хранить экземпляры `java.lang.Integer` соответствующего типа-обертки.

Ниже приведен полный список типов, соответствующих примитивным типам Java:

- *целочисленные типы* — Byte, Short, Int и Long;
- *типы чисел с плавающей запятой* — Float и Double;
- *символьный тип* — Char;
- *логический тип* — Boolean.

8.1.2. Использование всего диапазона битов для представления положительных чисел: типы беззнаковых чисел

Иногда возникают ситуации, когда необходимо использовать весь битовый диапазон целого положительного числа. Например, при работе на битовом и байтовом уровне, обрабатывая пиксели в растровом изображении, байты в файле или другие двоичные данные. В подобных случаях Kotlin расширяет обычные примитивные типы виртуальной машины Java (JVM) типами для беззнаковых целых чисел. В частности, существует четыре типа беззнаковых чисел. Они показаны в табл. 8.1.

Таблица 8.1. Типы беззнаковых чисел в Kotlin

Тип	Размер	Диапазон значений
UByte	8 бит	0–255
UShort	16 бит	0–65535
UInt	32 бита	0–2 ³² –1
ULong	64 бита	0–2 ⁶⁴ –1

Беззнаковые типы чисел «сдвигают» диапазон значений по сравнению со своими знаковыми аналогами. Это дает возможность хранить большие неотрицательные числа в том же объеме памяти. Обычный тип Int, например, позволяет хранить числа примерно от отрицательных 2 миллиардов до положительных 2 миллиардов. А тип UInt может представлять числа от 0 до примерно 4 миллиардов (рис. 8.1).

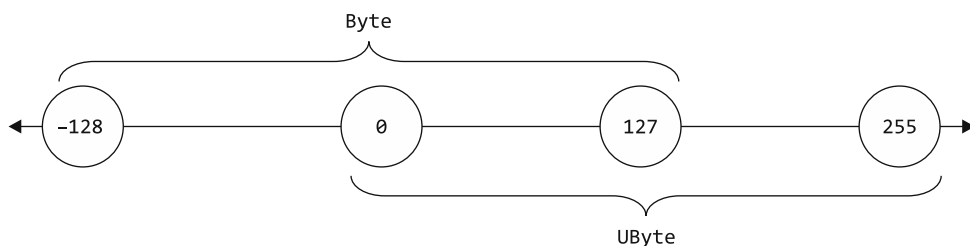


Рис. 8.1. Типы беззнаковых чисел смещают диапазон значений, позволяя хранить большие неотрицательные числа в том же объеме памяти. Если обычный знаковый Byte может хранить значения от –128 до 127, то UByte может хранить значения от 0 до 255

Как и другие примитивные типы, беззнаковые числа в Kotlin оборачиваются только при необходимости. В остальном они обладают характеристиками примитивных типов.

ПРИМЕЧАНИЕ

Может возникнуть соблазн использовать беззнаковые целые числа в ситуациях, в которых необходимо указать, что требуются неотрицательные целые числа. Однако цель типов беззнаковых чисел в Kotlin заключается не в этом. Когда полный диапазон битов не нужен, лучше использовать обычные целые числа и проверять, что в функцию было передано неотрицательное значение.

ПОДРОБНОСТИ РЕАЛИЗАЦИИ ТИПОВ БЕЗЗНАКОВЫХ ЧИСЕЛ

В спецификации JVM (<http://mng.bz/nJa4>) указано, что сама JVM не определяет и не предоставляет примитивы для беззнаковых чисел. Kotlin не может этого изменить, поэтому предоставляет собственные абстракции поверх существующих знаковых примитивов.

Для этого используется концепция, описанная в разделе 4.5, — встроенные классы. Каждый класс, представляющий беззнаковое число, на самом деле является встроенным и использует свой знаковый аналог в качестве хранилища. Именно так: внутри системы указанный `UInt` — это обычный `Int`. Компилятор Kotlin по возможности заменяет встроенные классы на базовое свойство, которое они оборачивают, поэтому можно сделать вывод, что беззнаковые типы чисел будут работать так же, как и знаковые.

Компилятор Kotlin может легко преобразовать тип `Int` в соответствующий примитивный тип на JVM, поскольку оба типа способны представлять один и тот же набор значений (и ни один из них не может хранить ссылку `null`). Аналогично при использовании объявления Java в программе на Kotlin примитивные типы Java становятся типами, не допускающими значение `null` (а не платформенными). Все потому, что они не могут содержать значения `null`. Теперь обсудим версии тех же типов, допускающие значение `null`.

8.1.3. Примитивные типы, допускающие значение `null`: `Int?`, `Boolean?` и другие

Типы из языка Kotlin, допускающие значение `null`, нельзя представить с помощью примитивов Java, поскольку в Java значение `null` можно хранить только в переменной ссылочного типа. Это означает, что при каждом использовании в Kotlin примитивного типа, допускающего значение `null`, он компилируется в соответствующий тип-обертку.

Чтобы увидеть, как используются типы, допускающие значение `null`, вернемся к начальному примеру книги и вспомним объявленный там класс `Person`.

Он представляет человека, имя которого всегда известно, а возраст может быть как известным, так и неопределенным. Добавим функцию для проверки того, старше ли один человек другого (листинг 8.1).

Листинг 8.1. Использование примитивных типов, допускающих значение `null`

```
data class Person(val name: String,
                 val age: Int? = null) {

    fun isOlderThan(other: Person): Boolean? {
        if (age == null || other.age == null)
            return null
        return age > other.age
    }
}

fun main() {
    println(Person("Sam", 35).isOlderThan(Person("Amy", 42)))
    // ложь
    println(Person("Sam", 35).isOlderThan(Person("Jane")))
    // null
}
```

Обратите внимание на то, как здесь действуют обычные правила `null`-безопасности. Нельзя просто сравнить два значения типа `Int?`, поскольку одно из них может быть равно `null`. Поэтому перед сравнением нужно проверить, что оба значения не равны `null`. После этого компилятор позволяет работать с ними в обычном режиме.

Значение свойства `age`, объявленного в классе `Person`, хранится как `java.lang.Integer`. Но этот нюанс имеет значение только при работе с классом из Java. Чтобы выбрать правильный тип в Kotlin, следует только подумать, может ли переменная или свойство принимать значение `null`.

Классы обобщений — еще один случай использования типов-оберткок. Если в качестве аргумента типа класса вы используете примитивный тип, то Kotlin использует обернутое представление типа. Например, создается список значений `Integer`, даже если вы никогда не указывали тип, допускающий значение `null`, или не использовали это значение:

```
val listOfInts = listOf(1, 2, 3)
```

Это происходит из-за способа реализации обобщений в JVM. В настоящее время JVM не поддерживает использование примитивного типа в качестве аргумента типа, поэтому класс обобщений (как в Java, так и в Kotlin) всегда должен использовать обернутое представление типа. Как следствие, если нужно эффективно хранить большие коллекции примитивных типов, то необходимо использовать стороннюю библиотеку с поддержкой таких коллекций, например `Eclipse Collections` (<https://github.com/eclipse/eclipse-collections>), либо хранить их в массивах. О массивах мы поговорим в конце текущей главы. Теперь рассмотрим, как можно преобразовывать значения между различными примитивными типами.

8.1.4. В Kotlin преобразования чисел стали явными

Одно из важных различий между Kotlin и Java заключается в том, как эти языки обрабатывают числовые преобразования. Kotlin не преобразует числа из одного типа в другой автоматически. Даже если большему по размеру типу попытаться присвоить значение меньшего размера. Например, следующий код не будет компилироваться в Kotlin:

```
val i = 1
val l: Long = i    ← Ошибка: несоответствие типов
```

Поэтому необходимо применить преобразование в явном виде:

```
val i = 1
val l: Long = i.toLong()
```

Функции преобразования определены для каждого примитивного типа (кроме `Boolean`): `toByte()`, `toShort()`, `toChar()` и т. д. Они поддерживают преобразование в обоих направлениях: расширение меньшего типа в больший, например `Int.toLong()`, и усечение большего типа до меньшего, например `Long.toInt()`.

Kotlin делает это преобразование явным, чтобы избежать неожиданностей, особенно при сравнении обернутых значений. Метод `equals` для двух значений в ячейках проверяет тип ячейки, а не только хранящееся в ней значение. Так, в Java функция `Integer.valueOf(42).equals(Long.valueOf(42))` возвращает значение `false`. Если бы в Kotlin поддерживались неявные преобразования, то вы могли бы написать примерно такой код:

```
val x = 1            ← Переменная типа Int
val list = listOf(1L, 2L, 3L)    ← Список значений типа Long
x in list           ← Ложно, если бы Kotlin поддерживал скрытые преобразования
```

Вопреки ожиданиям, результатом выполнения будет `false`. Строка `x in list` из данного примера не компилируется, поэтому Kotlin требует явного преобразования типов, чтобы сравнивались только значения одного типа:

```
fun main() {
    val x = 1
    println(x.toLong() in listOf(1L, 2L, 3L))
    // истина
}
```

Если в коде одновременно используются разные типы чисел, то во избежание неожиданного поведения необходимо явно преобразовывать переменные.

Обратите внимание, что при записи литерала числа обычно не нужно использовать функции преобразования. Одна из возможностей — с помощью специального синтаксиса явно указывать тип константы, например `42L` или `42.0f`. И даже если

литерал не используется, необходимое преобразование применяется автоматически, когда числовой литерал применяется для инициализации переменной известного типа или передается в качестве аргумента в функцию. Кроме того, арифметические операторы перегружены, чтобы принимать все подходящие числовые типы. Например, код ниже работает корректно без явных преобразований:

```
fun printALong(l: Long) = println(l)
```

```
fun main() {
    val b: Byte = 1
    val l = b + 1L
    printALong(42)
    // 42
}
```

Постоянное значение приобретает правильный тип

Оператор + работает с аргументами типов Byte и Long

Компилятор интерпретирует число 42 как значение типа Long

ЛИТЕРАЛЫ ПРИМИТИВНЫХ ТИПОВ

Kotlin поддерживает следующие способы записи числовых литералов в исходном коде, помимо простых десятичных чисел:

- в литералах типа Long используется суффикс L: 123L;
- для литералов типа Double используется стандартное представление числа с плавающей запятой: 0.12, 2.0, 1.2e10 и 1.2e-10;
- в литералах типа Float используется суффикс f или F: 123.4f, .456F и 1e3f;
- для шестнадцатеричных литералов применяется префикс 0x или 0X (например, 0xCAFEBAFE или 0xbcdL);
- для двоичных литералов применяется префикс 0b или 0B (например, 0b000000101);
- для литералов беззнаковых чисел используется суффикс U: 123U, 123UL и 0x10cU.

Для символьных литералов используется в основном тот же синтаксис, что и в Java. Символ записывается в одинарных кавычках. При необходимости можно использовать экранирующие последовательности. Примеры допустимых символьных литералов Kotlin выглядят так: '1', '\t' (символ табуляции) и '\u0009' (символ табуляции, представленный с помощью экранирующей последовательности Unicode).

Обратите внимание, что поведение арифметических операторов Kotlin в отношении переполнения и недостаточного наполнения диапазона чисел точно такое же, как и в Java: Kotlin не вводит никаких дополнительных проверок на переполнение:

```
fun main() {
    println(Int.MAX_VALUE + 1)
    -2147483648
    println(Int.MIN_VALUE - 1)
    2147483647
}
```

Переполнение приводит к тому, что значение оборачивается к минимуму...

...а недостаточное наполнение — к максимуму

ПРЕОБРАЗОВАНИЕ ИЗ ТИПА STRING

Стандартная библиотека Kotlin предоставляет набор функций-расширений для преобразования строки в примитивный тип: `toInt`, `toByte`, `toBoolean` и т. д. Каждая из этих функций пытается выполнить синтаксический анализ содержимого строки как соответствующий тип и выбрасывает исключение `NumberFormatException`, если анализ завершился неудачей:

```
fun main() {
    println("42".toInt())
    // 42
}
```

Но если ожидается, что преобразование из строки в примитивный тип будет происходить часто, то постоянная обработка исключения `NumberFormatException` в явном виде может оказаться трудоемкой. Для этого случая у каждой из указанных функций-расширений есть аналог. Он возвращает значение `null`, если преобразование не удалось: `toIntOrNull`, `toByteOrNull` и т. д.:

```
fun main() {
    println("seven".toIntOrNull())
    // null
}
```

В качестве особого случая стоит выделить преобразование строк в логические значения. Эти функции преобразования определяются на приемнике, допускающем значение `null`, как было описано в главе 7.

Функция `toBoolean` возвращает значение `true`, если вызываемая ею строка не равна `null` и ее содержимое равно слову `true` (без учета регистра). В противном случае она возвращает значение `false`:

```
fun main() {
    println("true".toBoolean())
    // истина
    println("7".toBoolean())
    // ложь
    println(null.toBoolean())
    // ложь
}
```

Чтобы строки `"true"` и `"false"` точно совпали при преобразовании, необходимо применить функцию `toBooleanStrict`. Она принимает только эти два значения и выбрасывает исключение в противном случае.

Прежде чем мы перейдем к другим типам, необходимо упомянуть еще три особых типа: `Any`, `Unit` и `Nothing`.

8.1.5. Типы `Any` и `Any?`: корень иерархии типов Kotlin

Подобно тому как `Object` является корнем иерархии классов в Java, тип `Any` служит супертипом всех типов в Kotlin, не допускающих значение `null`. Но в Java класс `Object` представляет собой супертип только всех ссылочных типов, а примитивные типы не входят в иерархию. Поэтому необходимо использовать типы-обертки,

такие как `java.lang.Integer`, для представления значения примитивного типа, когда требуется использовать экземпляр `Object`. В Kotlin `Any` — супертип для всех типов, в том числе примитивных, таких как `Int`.

Как и в Java, присвоение значения примитивного типа переменной типа `Any` выполняет автоматическое обертывание:

```
val answer: Any = 42
```

 ← Значение 42 обертывается, поскольку `Any` — ссылочный тип

Поскольку `Any` относится к типам, не допускающим значение `null`, переменная типа `Any` не может содержать это значение. Если требуется переменная для хранения любого возможного в Kotlin значения, в том числе `null`, то можно использовать тип `Any?`.

Внутри системы тип `Any` соответствует `java.lang.Object`. Если тип `Object` используется в параметрах и возвращаемых типах методов Java, то будет представлен как `Any` в Kotlin. (Точнее, он рассматривается как платформенный тип, поскольку его `null`-безопасность неизвестна.) Когда функции Kotlin используют тип `Any`, он компилируется в байт-код Java как `Object`.

В главе 4 мы рассказали, что во всех классах Kotlin есть три метода: `toString`, `equals` и `hashCode`. Они наследуются от `Any`. Другие методы определены на основе `java.lang.Object` (например, `wait` и `notify`). Они недоступны с типом `Any`, но их можно вызвать, если вручную привести значение к типу `java.lang.Object`.

8.1.6. Тип `Unit` — `void` для Kotlin

Тип `Unit` в Kotlin выполняет ту же функцию, что и `void` в Java. Его можно использовать в качестве возвращаемого типа функции, которая не может вернуть ничего полезного:

```
fun f(): Unit { /* ... */ }
```

Синтаксически это то же самое, что написать функцию с телом блока без объявления типа:

```
fun f() { /* ... */ }
```

 ← Явное объявление типа `Unit` опускается

В большинстве случаев разница между `void` и `Unit` незаметна. Если функция Kotlin имеет возвращаемый тип `Unit` и не переопределяет функцию обобщения, то компилируется в хорошо известную функцию `void` внутри системы. Если переопределить ее из Java, то функция Java просто должна возвращать `void`.

Чем же тогда различаются тип `Unit` из Kotlin и `void` из Java? `Unit` — полноценный тип, и, в отличие от `void`, его можно использовать в качестве аргумента типа. Существует только одно значение данного типа, оно называется `Unit` и возвращается *скрыто*. Это полезно, когда функция возвращает параметр обобщения, а вы переопределяете ее и даете указание возвращать значение типа `Unit`:

```
interface Processor<T> {  
    fun process(): T  
}
```

```
class NoResultProcessor : Processor<Unit> {
    override fun process() { ← Возврат Unit, но определение типа было опущено
        // некий код
    } ← Писать оператор return в явном виде здесь не нужно
}
```

Сигнатура интерфейса требует, чтобы функция `process` возвращала значение, а поскольку у типа `Unit` есть значение, то вернуть его из метода не составит труда.

Но писать оператор `return` в функции `NoResultProcessor.process` в явном виде не нужно, поскольку строка `return Unit` скрыто добавляется компилятором.

Это решение Kotlin намного приятнее любой из возможностей Java, предназначенной для решения проблемы использования «значения нет» в качестве аргумента типа. Один из вариантов — применять отдельные интерфейсы (например, `Callable` и `Runnable`) для представления возвращающих и возвращающих значение интерфейсов. Вдобавок можно использовать в качестве типового параметра специальный тип `java.lang.Void`. В этом случае вам все равно придется добавить оператор `return null`, чтобы вернуть единственное возможное значение, соответствующее данному типу. Ведь если тип возврата окажется не `void`, то в коде всегда должен быть указан оператор `return`.

Может показаться удивительным, почему мы выбрали другое название для типа `Unit` и не назвали его `Void`. Название `Unit` традиционно используется в функциональных языках для обозначения «только одного экземпляра», и именно этим `Unit` в Kotlin отличается от `void` в Java. Можно было использовать привычное имя `Void`, но в Kotlin есть тип `Nothing`, и он выполняет совершенно другую функцию. А наличие двух типов с названиями `Void` и `Nothing` может запутать, поскольку их значения очень близки. Так что же это за тип — `Nothing`? Давайте выясним.

8.1.7. Тип `Nothing`: «Эта функция никогда не возвращает результат»

Для некоторых функций в Kotlin понятие «возвращаемое значение» не имеет смысла, поскольку они никогда не завершаются успешно. Например, во многих библиотеках тестирования есть функция `fail`. Она проваливает текущий тест, выбрасывая исключение с заданным сообщением. Функция с бесконечным циклом тоже не завершится успешно.

При анализе кода с вызовами таких функций полезно знать, что они никогда не завершаются нормально. В Kotlin это выражается с помощью специального типа возврата `Nothing`:

```
fun fail(message: String): Nothing {
    throw IllegalStateException(message)
}

fun main() {
    fail("Error occurred")
    // java.lang.IllegalStateException: Error occurred
}
```

Тип `Nothing` не содержит значений, поэтому его имеет смысл использовать только как тип возврата функции или как аргумент типа для типового параметра, который используется как тип возврата функции обобщения. Во всех остальных случаях объявлять переменную без возможности хранить в ней значение не имеет смысла.

Обратите внимание, что возвращающие `Nothing` функции можно использовать в правой части оператора `?:` для проверки предварительного условия:

```
val address = company.address ?: fail("No address")
println(address.city)
```

Этот пример показывает, почему наличие `Nothing` в системе типов чрезвычайно полезно. Компилятору известно, что функция с таким типом возврата никогда не завершается нормально. И эта информация используется при анализе кода, вызывающего функцию. В предыдущем примере компилятор делает вывод, что у переменной `address` тип «не `null`», поскольку ветвь, обрабатывающая случай, когда переменная равна `null`, всегда выбрасывает исключение и выполнение кода прекращается. В стандартной библиотеке Kotlin тоже есть такая функция: функция `error` выбрасывает `IllegalStateException`, а ее тип возврата — `Nothing`.

На этом мы завершаем обсуждение базовых примитивных типов в Kotlin: `Any`, `Unit` и `Nothing`. Теперь рассмотрим типы коллекций и их отличия от аналогов в Java.

8.2. КОЛЛЕКЦИИ И МАССИВЫ

Мы привели множество примеров кода с использованием различных API коллекций. В разделе 3.1 рассказывается, что Kotlin опирается на библиотеку коллекций Java и дополняет ее возможностями с помощью функций-расширений. История не завершается поддержкой коллекций в Kotlin и соответствия между коллекциями Java и Kotlin, и сейчас самое время изучить подробности.

8.2.1. Коллекции nullable-значений и коллекции, допускающие значение null

В главе 7 мы говорили о концепции типов, допускающих значение `null`, но `null`-безопасность аргументов типов упоминалась вскользь. Однако это важная тема применительно к последовательной системе типов: необходимо знать, может ли коллекция содержать значения `null`. Не менее важно знать и то, может ли значение переменной быть равным `null`. В Kotlin есть полная поддержка `null`-безопасности для аргументов типов. К типу переменной добавляется символ `?` для обозначения того, что переменная может содержать `null`, и тип, используемый в качестве аргумента типа, может быть помечен точно так же. Чтобы понять, как это работает, рассмотрим пример функции (листинг 8.2). Она принимает входной текст и пытается выполнить синтаксический анализ каждой строки в строке ввода как число.

Листинг 8.2. Создание коллекции nullable-значений

```

fun readNumbers(text: String): List<Int?> {
    val result = mutableListOf<Int?>()
    for (line in text.lineSequence()) {
        val numberOrNull = line.toIntOrNull()
        result.add(numberOrNull)
    }
    return result
}

```

Создание изменяемого списка nullable-значений типа Int

Построчные итерации по входной строке

Добавление целого числа или значения null в список, если невозможно выполнить анализ текущей строки

Список `List<Int?>` может содержать значения типа `Int?`, другими словами — `Int` или `null`. Если анализ строки можно выполнить, то в список `result` добавляется целое число, если нет — значение `null`.

Обратите внимание, что `null`-безопасность типа переменной и типа, используемого в качестве аргумента типа, различается. Разница между списком nullable-значений типа `Int` и списком значений типа `Int`, допускающим значение `null`, показана на рис. 8.2.

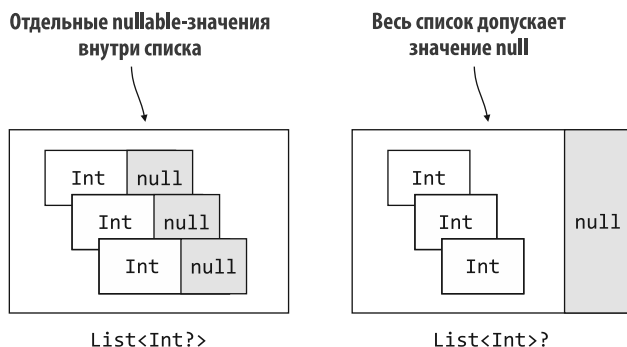


Рис. 8.2. При обсуждении решения относительно `null`-безопасности следует тщательно продумать, как планируется использовать коллекцию. Должна ли вся коллекция допускать значение `null`, или его должны допускать отдельные элементы `List<Int?>` `List<Int>?` внутри нее?

В первом случае сам список всегда будет «не `null`», но каждое значение в списке может быть равно `null`. Переменная второго типа может содержать `null`-ссылку вместо экземпляра списка, но элементы списка гарантированно будут «не `null`».

На основе знаний о функциональном программировании и лямбдах можно сократить этот пример с помощью функции `map`, описанной в главе 6. Она применяет заданную функцию — в данном случае `toIntOrNull` — к каждому элементу входной последовательности. Затем эти элементы будут собраны в список результатов (листинг 8.3).

Листинг 8.3. Сокращение метода `readNumbers` с помощью `map`

```

fun readNumbers2(text: String): List<Int?> =
    text.lineSequence().map { it.toIntOrNull() }.toList()

```

У вас может возникнуть ситуация, когда потребуется объявить переменную, содержащую список чисел, допускающих значение `null`, при этом и сам список допускает данное значение. Это позволяет выразить мысль о том, что отдельные элементы в списке могут отсутствовать, а также о том, что список в целом может отсутствовать. В Kotlin это выглядит так: `List<Int?>?`, с двумя вопросительными знаками (рис. 8.3). Внутренний вопросительный знак указывает на то, что *элементы* списка могут допускать значение `null`. Внешний вопросительный знак указывает на то, что *сам список* может допускать данное значение. При использовании значения переменной и значения каждого элемента в списке необходимо применять проверку `null`.

Весь список или его отдельные значения допускают значение `null`

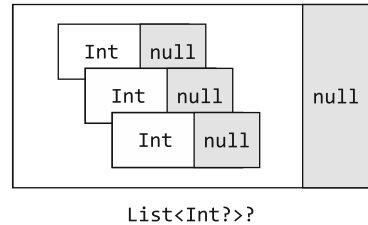


Рис. 8.3. Nullable-коллекция целых чисел, допускающих значение `null`, может сама быть `null` или хранить потенциально значения `null`

Чтобы увидеть, как работать со списком значений, допускающих значение `null`, напомним функцию. Она будет складывать все допустимые числа и отдельно подсчитывать недопустимые (листинг 8.4).

Листинг 8.4. Работа с коллекциями значений, допускающих значение `null`

```
fun addValidNumbers(numbers: List<Int?>) {
    var sumOfValidNumbers = 0
    var invalidNumbers = 0
    for (number in numbers) {
        if (number != null) {
            sumOfValidNumbers += number
        } else {
            invalidNumbers++
        }
    }
    println("Sum of valid numbers: $sumOfValidNumbers")
    println("Invalid numbers: $invalidNumbers")
}

fun main() {
    val input = """
        1
        abc
        42
    """.trimIndent()
    val numbers = readNumbers(input)
    addValidNumbers(numbers)
    // Sum of valid numbers: 43
    // Invalid numbers: 1
}
```

Считывание из списка значения, допускающего значение `null`

Проверка значения на `null`

Определение многострочной входной строки

Здесь нет ничего особенного. При обращении к элементу списка вы получаете значение типа `Int?`, и необходимо проверить его на `null`, прежде чем использовать в арифметических операциях.

Отфильтровать `null` из коллекции nullable-значений — настолько распространенная операция, что Kotlin предоставляет для ее выполнения стандартную библиотечную функцию `filterNotNull`. С ее помощью можно значительно упростить предыдущий пример (листинг 8.5).

Листинг 8.5. Использование функции `filterNotNull` с коллекцией nullable-значений

```
fun addValidNumbers(numbers: List<Int?>) {
    val validNumbers = numbers.filterNotNull()
    println("Sum of valid numbers: ${validNumbers.sum()}")
    println("Invalid numbers: ${numbers.size - validNumbers.size}")
}
```

Конечно, фильтрация тоже влияет на тип коллекции. Тип списка `validNumbers` — `List<Int>`, поскольку фильтрация гарантирует, что коллекция не содержит элементов `null` (рис. 8.4).

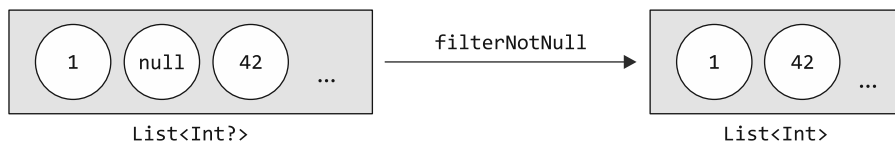


Рис. 8.4. Функция `filterNotNull` возвращает новую коллекцию без `null`-элементов входной коллекции. Эта новая коллекция не будет допускать значение `null`, и значит, в дальнейшем не придется выполнять обработку таких значений

Теперь вы понимаете, как Kotlin различает коллекции с элементами, которые допускают и не допускают значение `null`. В следующем подразделе рассмотрим еще одно важное нововведение Kotlin — коллекции только для чтения и изменяемые коллекции.

8.2.2. Коллекции только для чтения и изменяемые коллекции

В Kotlin есть важная отличительная особенность — разделение интерфейсов для доступа к данным в коллекции и для их модификации. Это различие существует, начиная с самого базового интерфейса для работы с коллекциями: `kotlin.collections.Collection`. С помощью данного интерфейса можно перебирать элементы коллекции, узнавать ее размер, проверять, содержит ли она определенный элемент, и выполнять другие операции со считыванием данных из коллекции. Но в этом интерфейсе нет методов для добавления или удаления элементов.

Для изменения данных в коллекции применяется интерфейс `kotlin.collections.MutableCollection`. Он расширяет обычный интерфейс `kotlin.collections.Collection` и предоставляет методы для добавления и удаления элементов, очистки коллекции и т. д. На рис. 8.5 показаны ключевые методы из этих двух интерфейсов.

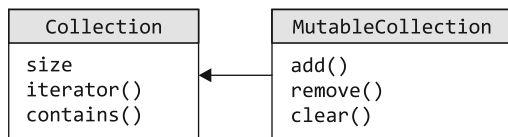


Рис. 8.5. Интерфейс `Collection` предназначен только для чтения. Тип `MutableCollection` расширяет его и добавляет методы для изменения содержимого коллекций

Как правило, в коде следует повсеместно использовать интерфейсы, доступные только для чтения. Изменяемые варианты применяются только в том случае, если код будет изменять коллекцию.

Как и разделение между `val` и `var`, разделение интерфейсов коллекций на изменяемые и только для чтения значительно упрощает понимание того, что происходит с данными в программе. Если функция принимает параметр типа `Collection`, но не `MutableCollection`, то известно, что она не собирается изменять коллекцию, а только считывает данные из нее. Когда в функцию требуется передать `MutableCollection`, можно предположить, что она будет изменять данные. И если есть коллекция, представляющая часть внутреннего состояния компонента, то может потребоваться создать ее копию, прежде чем передавать ее в такую функцию. (Такой паттерн обычно называют *защитной копией*.) Например, следующая функция `copyElements` изменит целевую коллекцию, но не исходную (листинг 8.6).

Листинг 8.6. Использование интерфейсов коллекций только для чтения и изменяемых

```

fun <T> copyElements(source: Collection<T>,
                    target: MutableCollection<T>) {

    for (item in source) { ← Перебор в цикле всех элементов в исходной коллекции
        target.add(item) ← Добавление элементов в целевую
    }                      изменяемую коллекцию
}

fun main() {
    val source: Collection<Int> = arrayListOf(3, 5, 7)
    val target: MutableCollection<Int> = arrayListOf(1)
    copyElements(source, target)
    println(target)
    // [1, 3, 5, 7]
}
  
```

Нельзя передать переменную типа коллекции только для чтения в качестве аргумента аннотации `target`, даже если ее значение представлено изменяемой коллекцией:

```

fun main() {
    val source: Collection<Int> = arrayListOf(3, 5, 7)
    val target: Collection<Int> = arrayListOf(1)
    copyElements(source, target) ← Ошибка аргумента аннотации target
    // Error: Type mismatch: inferred type is Collection<Int>
    // but MutableCollection<Int> was expected
}
  
```

При работе с интерфейсами коллекций следует помнить о том, что коллекции только для чтения не обязательно будут неизменяемыми. Если вы работаете с переменной типа интерфейса только для чтения, это может быть лишь одна из многих ссылок на одну и ту же коллекцию. У других ссылок может быть изменяемый тип интерфейса (рис. 8.6).

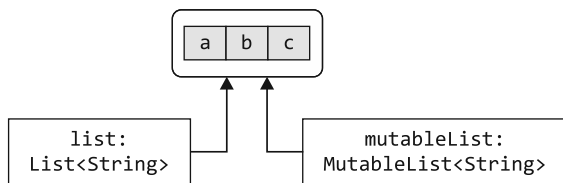


Рис. 8.6. Две разные ссылки, одна для чтения, другая для изменения, указывают на один и тот же объект коллекции. Код обращения к списку `list` не может изменить базовую коллекцию, но будет работать с ее изменениями. Их будет вносить код, работающий с `mutableList`

Допустим, одна часть кода хранит ссылку на изменяемую коллекцию. А другая — хранит «представление» этой же коллекции только для чтения и не может полагаться на предположение, что первая часть не изменит коллекцию во время обработки. Если во время работы кода с коллекцией та изменяется, это может привести к возникновению ошибок `ConcurrentModificationException` и к другим проблемам.

Поэтому важно понимать, что *коллекции только для чтения не всегда потокобезопасны*, — то, что функция воспринимает как «представление» коллекции, на самом деле может быть изменяемой коллекцией. Поэтому при работе с данными в многопоточной среде необходимо убедиться, что код правильно синхронизирует доступ к данным или использует структуры данных, поддерживающие конкурентный доступ.

ПРИМЕЧАНИЕ

В стандартной библиотеке неизменяемые коллекции недоступны, но библиотека `kotlinx.collections.immutable` (<https://github.com/Kotlin/kotlinx.collections.immutable>) предоставляет интерфейсы неизменяемых коллекций и прототипы реализации для Kotlin.

Как происходит разделение между изменяемыми коллекциями и доступными только для чтения? Разве мы не говорили ранее, что коллекции Kotlin — это то же самое, что и коллекции Java? Нет ли здесь противоречия? Разберемся, в чем тут дело.

8.2.3. Коллекции Kotlin и Java глубоко связаны между собой

На самом деле каждая коллекция Kotlin представляет собой экземпляр соответствующего интерфейса коллекции Java. При переходе между Kotlin и Java не происходит преобразования, нет необходимости в обертках или копировании данных. Но для каждого интерфейса коллекции в Java есть два *представления* в Kotlin — только для чтения и изменяемое (рис. 8.7).

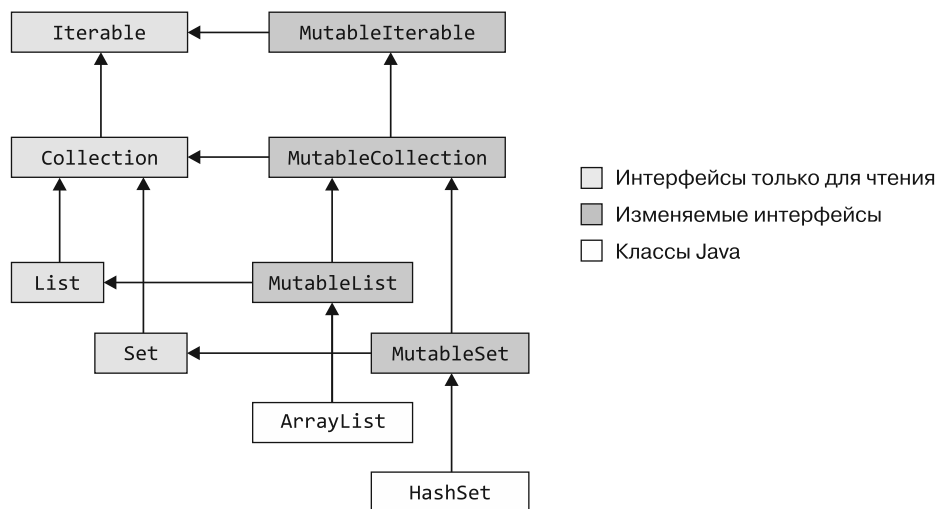


Рис. 8.7. Иерархия интерфейсов коллекций Kotlin. Классы `ArrayList` и `HashSet` из Java среди прочих расширяют изменяемые интерфейсы Kotlin

Все интерфейсы коллекций на этом рисунке объявлены на языке Kotlin. Можно провести параллель между базовой структурой интерфейсов Kotlin только для чтения и структурой изменяемых интерфейсов коллекций Java в пакете `java.util`. Кроме того, каждый изменяемый интерфейс расширяет соответствующий интерфейс, предназначенный только для чтения. Изменяемые интерфейсы соответствуют интерфейсам из пакета `java.util`, а в версиях только для чтения отсутствуют все методы изменения.

На рис. 8.7 также есть классы `java.util.ArrayList` и `java.util.HashSet` из языка Java, которые показывают, как стандартные классы Java обрабатываются в Kotlin. В Kotlin они представлены так, как если бы они были унаследованными от интерфейсов Kotlin `MutableList` и `MutableSet`. Другие реализации из библиотеки коллекций Java (`LinkedList`, `SortedSet` и т. д.) здесь не представлены, но с точки зрения Kotlin в них содержатся похожие супертипы. Таким образом, мы получаем и совместимость, и четкое разделение изменяемых и доступных только для чтения интерфейсов.

Помимо коллекций, класс `Map` (который не расширяет интерфейсы `Collection` или `Iterable`) тоже представлен в Kotlin в виде двух различных версий: `Map` и `MutableMap`. В табл. 8.2 показаны функции для создания коллекций различных типов.

Стоит отметить, что методы `setOf()` и `mapOf()` возвращают экземпляры доступных только для чтения интерфейсов `Set` и `Map`, но внутри системы те хранятся как изменяемые. (В JVM коллекции можно обернуть вызовом `Collections.unmodifiable`, чтобы сделать изменения невозможными. Однако из-за накладных расходов на перенаправление в Kotlin это не выполняется автоматически.) Но не стоит применять этот способ: возможно, в будущей версии Kotlin в качестве возвращаемых значений методов `setOf` и `mapOf` будут использоваться действительно неизменяемые классы реализации.

Таблица 8.2. Функции для создания коллекций

Тип коллекции	Только для чтения	Изменяемая
List	listOf, List	mutableListOf, MutableList, arrayListOf, buildList
Set	setOf	mutableSetOf, hashSetOf, linkedSetOf, sortedSetOf, buildSet
Map	mapOf	mutableMapOf, hashMapOf, linkedMapOf, sortedMapOf, buildMap

Когда требуется вызвать метод Java и передать коллекцию в качестве аргумента, это можно сделать напрямую, без каких-либо дополнительных действий. Например, если у вас есть метод Java и он принимает в качестве параметра `java.util.Collection`, то можно передать этому параметру в качестве аргумента любое значение `Collection` или `MutableCollection`.

Это оказывает очень сильное влияние на изменяемость коллекций. В Java нет различий коллекций только для чтения и изменяемых, поэтому код Java может изменять коллекцию, даже если на стороне Kotlin она объявлена как доступная только для чтения `Collectio`. Компилятор Kotlin не может полностью проанализировать, что делается с коллекцией в коде Java. Поэтому у Kotlin нет возможности отклонить вызов, передающий `Collection` только для чтения в код Java, который ее изменяет. Например, следующие два фрагмента кода образуют компилируемую межязыковую программу Kotlin-Java:

```

/* Java */
// CollectionUtils.java
public class CollectionUtils {
    public static List<String> uppercaseAll(List<String> items) {
        for (int i = 0; i < items.size(); i++) {
            items.set(i, items.get(i).toUpperCase());
        }
        return items;
    }
}

// Kotlin
// collections.kt
fun printInUppercase(list: List<String>) {
    println(CollectionUtils.uppercaseAll(list))
    println(list.first())
}

fun main() {
    val list = listOf("a", "b", "c")
    printInUppercase(list)
    // [A, B, C]
    // A
}

```

Объявление параметра только для чтения

Вызов функции Java, которая изменяет коллекцию

Отображение того, что коллекция была изменена

Как следствие, если вы пишете функцию Kotlin, которая принимает коллекцию и передает ее в Java, то *на вас лежит ответственность за использование правильного типа для параметра*. Он будет зависеть от того, будет ли вызываемый вами код Java изменять коллекцию.

Обратите внимание: это предостережение относится и к коллекциям с «не null»-типами элементов. Если вы передадите такую коллекцию методу Java, то он может поместить в нее значение null. И в Kotlin нет способа запретить это действие или даже обнаружить его, не навредив производительности. Из-за этого при передаче коллекций коду Java, который может их изменить, необходимо соблюдать особые меры предосторожности: типы Kotlin должны корректно отражать все доступные изменения коллекций. Теперь подробнее рассмотрим, как Kotlin обрабатывает коллекции из кода Java.

8.2.4. Объявленные в Java коллекции в Kotlin воспринимаются как платформенные типы

Напомним, что типы, определенные в коде Java, в Kotlin воспринимаются как *платформенные*. В Kotlin нет информации о их null-безопасности, поэтому компилятор позволяет коду Kotlin рассматривать их как допускающие значение null или «не null». Таким же образом переменные типов коллекций, объявленные в Java, рассматриваются как платформенные типы. Коллекция с платформенным типом — это, по сути, коллекция с неизвестной изменяемостью. Следовательно, код Kotlin может рассматривать ее как только для чтения или как изменяемую. Обычно это неважно, поскольку в действительности все выполняемые операции просто работают.

Разница становится важной при переопределении или реализации метода Java, в сигнатуре которого присутствует тип коллекции. Здесь необходимо решить, какой тип Kotlin будет использоваться для представления типа Java, поступающего из обрабатываемого метода.

В этой ситуации нужно принять несколько решений. Все они будут отражены в итоговом типе параметра в Kotlin.

- Эта коллекция допускает значение null?
- Элементы коллекции допускают значение null?
- Будет ли метод изменять коллекцию?

Рассмотрим следующие случаи, чтобы оценить различия. В первом примере интерфейс Java представляет объект для обработки текста в файле (листинг 8.7).

Листинг 8.7. Java-интерфейс с параметром коллекции

```
/* Java */
interface FileContentProcessor {
    void processContents(
        File path,
        byte[] binaryContents,
        List<String> textContents
    );
}
```

При реализации этого интерфейса в Kotlin необходимо сделать следующий выбор:

- список будет допускать значение `null`, поскольку некоторые файлы будут двоичными и их содержимое не может быть представлено в виде текста;
- элементы списка не будут допускать значение `null`, поскольку строки в файле не принимают это значение;
- список будет доступен только для чтения, так как представляет собой содержимое файла, и оно не будет изменено.

Эта реализация выглядит следующим образом (листинг 8.8).

Листинг 8.8. Реализация интерфейса `FileContentProcessor` в Kotlin

```
class FileIndexer : FileContentProcessor {
    override fun processContents(
        path: File,
        binaryContents: ByteArray?,
        textContents: List<String>?
    ) {
        // ...
    }
}
```

Противопоставим этот код другому интерфейсу (листинг 8.9). Здесь реализации интерфейса выполняют синтаксический анализ некоторых данных из текстовой формы в список объектов. Затем добавляют эти объекты в выходной список и сообщают об ошибках, обнаруженных при анализе. Сообщения сохраняются в отдельном списке.

Листинг 8.9. Еще один интерфейс Java с параметром коллекции

```
/* Java */
interface DataParser<T> {
    void parseData(
        String input,
        List<T> output,
        List<String> errors
    );
}
```

В этом случае выбор будет другим:

- список `List<String>` будет «не `null`», поскольку вызывающие программы всегда должны получать сообщения об ошибках;
- элементы в списке будут допускать значение `null`, поскольку не каждый элемент в выходном списке получит связанное с ним сообщение об ошибке;
- список `List<String>` будет изменяемым, поскольку код реализации должен добавлять в него элементы.

Этот интерфейс можно реализовать в Kotlin следующим образом (листинг 8.10).

Листинг 8.10. Реализация интерфейса `DataParser` в Kotlin

```
class PersonParser : DataParser<Person> {
    override fun parseData(
        input: String,
        output: MutableList<Person>,
        errors: MutableList<String?>
    ) {
        // ...
    }
}
```

Обратите внимание, как один и тот же тип Java — `List<String>` — представлен двумя разными типами Kotlin: `List<String>?` (список строк, допускающий значение `null`) в одном случае и `MutableList<String?>` (изменяемый список строк, допускающих значение `null`) в другом. Чтобы сделать правильный выбор, вы должны знать, какому именно контракту должен следовать Java-интерфейс или класс. Обычно это легко понять по тому, какие действия должна выполнять ваша реализация.

Итак, мы обсудили коллекции и можем переходить к теме массивов. По умолчанию лучше использовать коллекции, а не массивы. Но многие API Java по-прежнему используют массивы, поэтому мы расскажем, как работать с ними в Kotlin.

8.2.5. Создание массивов объектов и примитивные типы для обеспечения совместимости и производительности

Вы уже сталкивались с массивами в самом начале изучения Kotlin, поскольку массив может быть частью сигнатуры функции Kotlin `main`. Напомним, как это выглядит (листинг 8.11).

Листинг 8.11. Использование массивов

```
fun main(args: Array<String>) {
    for (i in args.indices) {
        println("Argument $i is: ${args[i]}")
    }
}
```

Для итераций по массиву индексов используется свойство-расширение `array.indices`

Получение доступа к элементам по индексу с помощью выражения `массив[индекс]`

Массив в Kotlin — это класс с типовым параметром, а тип элемента указывается в качестве соответствующего аргумента типа.

Создать массив в Kotlin можно следующими способами.

- Функция `arrayOf` создает массив элементов, указанных в качестве ее аргументов.
- Функция `arrayOfNulls` создает массив заданного размера с элементами `null`. Конечно же, ее можно использовать только для создания массивов с типом элемента, допускающего значение `null`.

- Конструктор `Array` принимает размер массива и лямбду и инициализирует каждый элемент массива, вызывая лямбду. Так можно инициализировать массив с «не null»-типом элемента, не передавая явно каждый элемент.

В качестве простого примера приведем вариант использования функции `Array` для создания массива строк от `a` до `z` (листинг 8.12).

Листинг 8.12. Создание массива символов

```
fun main() {
    val letters = Array<String>(26) { i -> ('a' + i).toString() }
    println(letters.joinToString(""))
    // abcdefghijklmnopqrstuvwxyz
}
```

Лямбда принимает индекс элемента массива и возвращает значение, которое должно быть помещено в массив по этому индексу. Здесь значение вычисляется путем добавления индекса к символу `'a'` и преобразования результата в строку. Тип элемента массива показан для наглядности — при разработке кода его обычно не указывают, так как компилятор может вычислить его:

```
fun main() {
    val letters = Array(26) { i -> ('a' + i).toString() }
    println(letters.joinToString())
    // a, b, c, d, e, f, g, h, i, j
}
```

ПРИМЕЧАНИЕ

На самом деле этот тип создания не был разработан исключительно для массивов. В Kotlin есть функции `List` и `MutableList`. Они создают экземпляры своих элементов на основе параметра размера и лямбды инициализации.

При этом одним из наиболее распространенных случаев создания массива в коде Kotlin будет вызов метода Java, принимающего массив, или функции Kotlin с параметром `vararg`. Пример показан в листинге 8.13. В таких ситуациях данные часто уже хранятся в коллекции, и остается просто преобразовать их в массив. Это можно сделать с помощью метода `toArray`.

Листинг 8.13. Передача коллекции в метод `vararg`

```
fun main() {
    val strings = listOf("a", "b", "c")
    println("%s/%s/%s".format(*strings.toArray()))
    // a/b/c
}
```

Оператор `spread (*)` применяется для передачи массива, когда ожидается поступление параметров метода `vararg`

Как и в случае с другими типами, аргументы типов массивов всегда становятся типами объектов. Поэтому, если объявить что-то подобное `Array<Int>`, это станет массивом целых чисел (его тип в Java будет `java.lang.Integer[]`). Если нужно создать массив значений примитивного типа без обертывания, то нужно использовать один из специализированных классов для массивов примитивных типов.

Для представления массивов примитивных типов Kotlin предлагает несколько отдельных классов — по одному для каждого примитивного типа. Например, массив значений типа `Int` называется `IntArray`. Для остальных типов в Kotlin есть `ByteArray`, `CharArray`, `BooleanArray` и т. д. Все эти типы компилируются в обычные массивы примитивных типов Java, такие как `int[]`, `byte[]`, `char[]` и т. д. Поэтому значения в таком массиве хранятся наиболее эффективным образом, без обертки.

ПРИМЕЧАНИЕ

В Kotlin есть массивы беззнаковых типов, например `UByteArray`, `UShortArray`, `UIntArray` и `ULongArray`. Они схожи с другими массивами числовых типов и не допускают обертывания. На момент написания данной книги беззнаковые массивы и операции над ними были еще нестабильны.

Создать массив примитивного типа можно следующими способами.

- Конструктор типа принимает параметр `size` и возвращает массив, инициализированный значениями по умолчанию для соответствующего примитивного типа (обычно нулями).
- Фабричная функция (`IntArrayOf` для `IntArray` и т. д. для других типов массивов) принимает в качестве аргументов переменное количество значений и создает из них массив.
- Другой конструктор принимает размер и лямбду, используемую для инициализации каждого элемента.

Первые два варианта для создания целочисленного массива, содержащего пять нулей, работают следующим образом:

```
val fiveZeros = IntArray(5)
val fiveZerosToo = IntArray(5) { 0 }
```

А так можно использовать конструктор, принимающий лямбду:

```
fun main() {
    val squares = IntArray(5) { i -> (i+1) * (i+1) }
    println(squares.joinToString())
    // 1, 4, 9, 16, 25
}
```

Если у вас есть массив или коллекция с обернутыми значениями примитивного типа, то можно преобразовать их в массив этого примитивного типа, используя соответствующую функцию преобразования, например `toIntArray`.

Далее рассмотрим некоторые действия с массивами. Помимо основных операций (получение длины массива, получение и добавление элементов), стандартная библиотека Kotlin поддерживает тот же набор функций-расширений для массивов, что и для коллекций. Все функции, описанные в главе 6 (`filter`, `map` и т. д.), работают и с массивами, в том числе примитивных типов. (Обратите внимание, что возвращаемые значения этих функций — списки, а не массивы.)

Посмотрим, как переписать листинг 8.11 с помощью функции `forEachIndexed` и лямбды. Переданная этой функции лямбда вызывается для каждого элемента массива и получает два аргумента: индекс элемента и сам элемент (листинг 8.14).

Листинг 8.14. Использование функции `forEachIndexed` с массивом

```
fun main(args: Array<String>) {  
    args.forEachIndexed { index, element ->  
        println("Argument $index is: $element")  
    }  
}
```

Теперь вы знаете, как использовать массивы в своем коде. Работа с ними так же проста, как работа с коллекциями в Kotlin.

РЕЗЮМЕ

- Типы, представляющие основные числа (например, `Int`), выглядят и функционируют как обычные классы, но, как правило, компилируются в примитивные типы Java. У классов беззнаковых чисел Kotlin нет точного эквивалента в JVM. Они преобразуются с помощью встроенных классов, чтобы их поведение и выполнение совпадали с примитивными типами.
- Примитивные типы, допускающие значение `null` (например, `Int?`), соответствуют обернутым примитивным типам в Java (например, `java.lang.Integer`).
- Тип `Any` является супертипом всех типов и аналогичен типу `Object` в Java. Тип `Unit` — аналог `void`.
- Тип `Nothing` используется в качестве возвращаемого типа функций, которые не завершаются обычным образом.
- Поступающие из Java типы интерпретируются в Kotlin как платформенные, что позволяет разработчику рассматривать их как допускающие или не допускающие значение `null`.
- Kotlin использует стандартные классы Java для коллекций и расширяет их. В них добавляется различие между коллекциями только для чтения и изменяемыми коллекциями.
- При расширении Java-классов или реализации Java-интерфейсов в Kotlin необходимо строго учитывать `null`-безопасность и изменяемость параметров.
- В Kotlin можно использовать массивы, но по умолчанию рекомендуется отдавать предпочтение коллекциям.
- Класс `Array` в Kotlin выглядит как обычный класс обобщения, но компилируется в массив Java.
- Массивы примитивных типов представлены специальными классами, такими как `IntArray`.

Часть II

Освоение Kotlin

На данном этапе вы уже должны быть хорошо знакомы с тем, как использовать Kotlin для доступа к существующим API. В этой части книги вы узнаете, как создавать собственные API на Kotlin. Важно помнить, что создание API не является исключительным правом авторов библиотек: каждый раз, когда в вашей программе появляется два взаимодействующих класса, один из них предоставляет API другому.

В главе 9 вы узнаете о принципе *соглашений*. Он используется в Kotlin для реализации перегрузки операторов и других техник абстракции, таких как делегированные свойства.

В главе 10 мы более подробно рассмотрим лямбды, и вы узнаете, как можно объявлять собственные функции, принимающие лямбды в качестве параметров.

В главах 11–12 вы познакомитесь с тем, как Kotlin работает с некоторыми более сложными концепциями, такими как обобщения, аннотации и рефлексия. Кроме того, в главе 12 вы поработаете с довольно большим реальным проектом на языке Kotlin: JKid — библиотекой для сериализации и десериализации JSON.

И наконец, в главе 13 узнаете об одном из важнейших свойств Kotlin — поддержке создания предметно-ориентированных языков.

9

Перегрузка операторов и другие соглашения

В этой главе

- ✓ Перегрузка операторов.
- ✓ Соглашения — специальные именованные функции, поддерживающие различные операции.
- ✓ Делегированные свойства.

В Kotlin есть ряд возможностей, где конкретные языковые конструкции реализуются путем вызова функций, определяемых вами в собственном коде. Возможно, вам уже знакомы подобные конструкции по языку Java. В них объекты, реализующие интерфейсы `java.lang.Iterable` и `java.lang.AutoCloseable`, могут использоваться в циклах `for` и операторах `try-with-resources` соответственно.

В Kotlin такие возможности привязаны к функциям с определенными именами (а не к каким-то специальным интерфейсам в стандартной библиотеке, как в Java). Например, если в классе определяется специальный метод `plus`, то по соглашению можно использовать оператор `+` для экземпляров данного класса. Поэтому в Kotlin мы называем эту технику *соглашениями*. В текущей главе мы рассмотрим различные соглашения в Kotlin и способы их применения.

В Java приходится использовать типы. А в Kotlin применяется принцип соглашений, поскольку в таком случае разработчики могут адаптировать существующие классы Java к требованиям особенностей языка Kotlin. Код Kotlin не может изменить сторонние классы так, чтобы они реализовывали дополнительные интерфейсы. С другой стороны, определение новых методов для класса возможно через механизм функций-расширений. Можно определить любые методы, описываемые

соглашениями, как расширения и таким образом адаптировать любой существующий класс Java, не изменяя его код.

В качестве примера в этой главе мы будем использовать простой класс `Point`, представляющий точку на экране. Такие классы есть в большинстве фреймворков пользовательского интерфейса, и вы легко сможете адаптировать приведенные здесь определения к своей среде:

```
data class Point(val x: Int, val y: Int)
```

Начнем с определения некоторых арифметических операторов для класса `Point`.

9.1. ПЕРЕГРУЗКА АРИФМЕТИЧЕСКИХ ОПЕРАТОРОВ ДЕЛАЕТ ОПЕРАЦИИ ДЛЯ ПРОИЗВОЛЬНЫХ КЛАССОВ БОЛЕЕ УДОБНЫМИ

Самый простой пример использования соглашений в Kotlin — арифметические операторы. В Java полный набор арифметических операций можно использовать только с примитивными типами. Но оператор `+` можно использовать и со значениями типа `String`. Однако эти операции могут быть удобны и в других случаях. Например, при работе с числами с помощью класса `BigInteger` будет удобнее суммировать их, используя `+`, чем вызывать метод `add`. Чтобы добавить элемент в коллекцию, можно использовать оператор `+=`. Kotlin позволяет это сделать, и в текущем разделе мы рассмотрим принципы работы данного механизма.

9.1.1. Функции `plus`, `times`, `divide` и другие: перегрузка операций двоичной арифметики

Сначала обеспечим поддержку операции сложения двух точек. Эта операция суммирует координаты `x` и `y` точек. Реализовать ее можно следующим образом (листинг 9.1).

Листинг 9.1. Определение оператора `plus`

```
data class Point(val x: Int, val y: Int) {  
    operator fun plus(other: Point): Point {  
        return Point(x + other.x, y + other.y)  
    }  
}  
  
fun main() {  
    val p1 = Point(10, 20)  
    val p2 = Point(30, 40)  
    println(p1 + p2)  
    // Point(x = 40, y = 60)  
}
```

Определение функции `plus` с модификатором `operator`

Добавление координат и возврат новой точки

Вызов функции `plus` с помощью знака `+`

Обратите внимание на использование ключевого слова `operator` для объявления функции `plus`. Все функции для перегрузки операторов должны быть помечены этим ключевым словом. Такое действие дает понять, что вы намерены использовать

функцию в качестве реализации соответствующего соглашения и что совпадение имени не случайно.

После того как функция `plus` будет объявлена с модификатором `operator`, можно суммировать объекты с помощью только знака `+`. Внутри кода операция ссылается на функцию `plus` (рис. 9.1).



Рис. 9.1. Оператор `+` преобразуется в вызов функции `plus`

Вместо того чтобы объявлять оператор в качестве члена класса, можно определить его как функцию-расширение (листинг 9.2).

Листинг 9.2. Определение оператора в качестве функции-расширения

```
operator fun Point.plus(other: Point): Point {  
    return Point(x + other.x, y + other.y)  
}
```

Реализация в точности повторяется. В последующих примерах будет использоваться синтаксис функции-расширения, поскольку это обычная схема определения функций-расширений соглашения для классов сторонних библиотек. Кроме того, данный синтаксис будет хорошо работать и для ваших собственных классов.

По сравнению с некоторыми другими языками определять и использовать перегруженные операторы в Kotlin проще, поскольку здесь нельзя определять собственные операторы. Для случаев, когда необходимо применять функцию между двумя операндами (например, `a times b`), Kotlin предлагает *инфиксные функции*, описанные в разделе 3.4, и мы еще вернемся к ним в подразделе 13.4.1. Они позволяют использовать главное синтаксическое преимущество пользовательских операторов — наличие операндов на каждой стороне вызова функции, — не вводя произвольные комбинации символов. Ведь такие произвольные названия вам пришлось бы запоминать или записывать.

В Kotlin есть ограниченный набор операторов для перегрузки, и каждый из них соответствует имени функции, которую вам нужно определить в своем классе. В табл. 9.1 перечислены все доступные для определения двоичные операторы и соответствующие имена функций.

Таблица 9.1. Перегружаемые операторы двоичной арифметики

Выражение	Имя функции
<code>a * b</code>	<code>times</code>
<code>a / b</code>	<code>div</code>
<code>a % b</code>	<code>mod</code>
<code>a + b</code>	<code>plus</code>
<code>a - b</code>	<code>minus</code>

Приоритет выполнения операторов ваших собственных типов не будет отличаться от приоритета стандартных числовых типов. Например, в выражении `a + b * c` умножение всегда будет выполняться раньше сложения, даже если вы сами определили эти операторы. У операторов `*`, `/` и `%` одинаковый приоритет, и он выше, чем приоритет операторов `+` и `-`.

ОПЕРАТОРНЫЕ ФУНКЦИИ И JAVA

Операторы Kotlin легко вызывать из Java. Каждый перегруженный оператор определяется как функция Kotlin (с модификатором `operator`), поэтому их вызывают как обычные функции с помощью полного имени. При вызове Java из Kotlin можно использовать синтаксис оператора для любых методов с именами, соответствующими соглашениям Kotlin. В Java не определен синтаксис для обозначения операторной функции, так что требование использовать модификатор `operator` не применяется, и единственными ограничениями будут выступать совпадение имени и количества параметров. Если в классе Java определен метод с нужным вам поведением, но его имя не подходит, то можно определить функцию-расширение с правильным именем. И она будет делегировать существующий метод Java.

При определении оператора не обязательно использовать одинаковые типы для двух операндов. Для примера определим оператор для масштабирования точки на определенное число (листинг 9.3). С его помощью можно переводить точки между различными системами координат.

Листинг 9.3. Определение оператора с различными типами операндов

```
operator fun Point.times(scale: Double): Point {
    return Point((x * scale).toInt(), (y * scale).toInt())
}

fun main() {
    val p = Point(10, 20)
    println(p * 1.5)
    // Point(x=15, y=30)
}
```

Обратите внимание, что операторы Kotlin автоматически не поддерживают *коммутативность* — возможность поменять местами левую и правую части оператора. Если требуется, чтобы пользователи могли писать `1.5 * p` и `p * 1.5`, то необходимо определить для этого отдельный оператор: `operator fun Double.times(p: Point): Point`.

Кроме того, тип возврата операторной функции может отличаться от типа операнда. Например, можно определить оператор для создания строки, повторяя символ определенное количество раз (листинг 9.4).

Листинг 9.4. Определение оператора с другим возвращаемым типом

```
operator fun Char.times(count: Int): String {
    return toString().repeat(count)
}

fun main() {
    println('a' * 3)
    // aaa
}
```

Повторение строки с помощью
встроенной функции repeat

Этот оператор принимает значения типа `Char` в качестве левого операнда и типа `Int` в качестве правого операнда, тип результата — `String`. Такие комбинации типов операндов и результатов вполне допустимы.

Обратите внимание, что допускается перегружать функции `operator`: можно определить несколько методов с разными типами параметров для одного и того же имени метода.

ДЛЯ ПОБИТОВЫХ ОПЕРАЦИЙ НЕТ СПЕЦИАЛЬНЫХ ОПЕРАТОРОВ

Kotlin не определяет побитовые операторы для стандартных типов чисел (как знаковых, так и беззнаковых, что было описано в разделе 8.1). Следовательно, невозможно определить их для ваших собственных типов. Вместо этого в Kotlin используются обычные функции, поддерживающие инфиксный синтаксис вызова, описанный в разделе 3.4. Допускается определять аналогичные функции для работы с собственными типами.

Далее приведен полный список функций, предоставляемых Kotlin для выполнения побитовых операций:

- `shl` — знаковый сдвиг влево;
- `shr` — знаковый сдвиг вправо;
- `ushr` — беззнаковый сдвиг вправо;
- `and` — побитовое И;
- `or` — побитовое ИЛИ;
- `xor` — побитовое исключающее ИЛИ;
- `inv` — побитовое инвертирование.

В примере ниже показано использование некоторых из перечисленных функций:

```
fun main() {
    println(0xF and 0xF0)
    // 0
    println(0xF or 0xF0)
    // 255
    println(0x1 shl 4)
    // 16
}
```

Теперь обсудим операторы наподобие `+=`. Они объединяют два действия: присваивание и соответствующий арифметический оператор.

9.1.2. Применение операции и немедленное присвоение ее значения: перегрузка составных операторов присваивания

Обычно при определении оператора наподобие `plus`, которое вы выполняли в подразделе 9.1.1, Kotlin поддерживает не только операцию `+`, но и `+=`. Такие операторы, как `+=`, `-=` и т. д., называются *составными операторами присваивания*. Приведем пример:

```
fun main() {
    var point = Point(1, 2)
    point += Point(3, 4)
    println(point)
    // Point(x=4, y=6)
}
```

Это выражение соответствует следующему: `point = point + Point(3, 4)`. Конечно же, код будет работать, только если переменная `point` изменяемая.

В некоторых случаях имеет смысл определить операцию `+=` для изменения объекта, на который ссылается переменная, не присваивая ссылку повторно. Один из таких случаев — добавление элемента в изменяемую коллекцию:

```
fun main() {
    val numbers = mutableListOf<Int>()
    numbers += 42
    println(numbers[0])
    // 42
}
```

Если определить функцию `plusAssign` с типом возврата `Unit` и обозначить ее ключевым словом `operator`, то Kotlin будет вызывать ее при использовании оператора `+=`. Названия других операторов двоичной арифметики аналогичны: `minusAssign`, `timesAssign` и т. д.

В стандартной библиотеке Kotlin определена функция `plusAssign` для изменяемой коллекции, и в предыдущем примере она использовалась (`+=`):

```
operator fun <T> MutableCollection<T>.plusAssign(element: T) {
    this.add(element)
}
```

Когда в коде вы пишете оператор `+=`, теоретически могут быть вызваны обе функции: `plus` и `plusAssign` (рис. 9.2). Если так происходит и обе функции определены и применимы, то компилятор сообщает об ошибке. Один из способов решить эту проблему — заменить использование оператора на обычный вызов функции. Другой — заменить объявление `var` на `val`, и тогда операция `plusAssign` становится неприменимой. Но в целом лучше всего разрабатывать новые классы последовательно — старайтесь не добавлять операции `plus` и `plusAssign` одновременно. Если класс неизменяемый, как `Point` в одном из предыдущих примеров, то

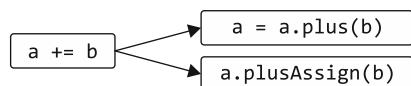


Рис. 9.2. Оператор `+=` можно преобразовать в `plus` или вызов функции `plusAssign`

становится неприменимой. Но в целом лучше всего разрабатывать новые классы последовательно — старайтесь не добавлять операции `plus` и `plusAssign` одновременно. Если класс неизменяемый, как `Point` в одном из предыдущих примеров, то

необходимо предоставлять только операторы, возвращающие новое значение (такие как `plus`). При разработке изменяемого класса, например строителя, предоставляйте только `plusAssign` и подобные операции.

Стандартная библиотека Kotlin поддерживает оба подхода для коллекций. Операторы `+` и `-` всегда возвращают новую коллекцию. Операторы `+=` и `-=` работают с изменяемыми коллекциями и изменяют их в месте хранения, а коллекции только для чтения обрабатываются с помощью возврата измененной копии. (Это означает, что `+=` и `-=` можно использовать с коллекцией только для чтения в том случае, если ссылающаяся на нее переменная объявлена как `var`.) В качестве операндов этих операторов можно использовать как отдельные элементы, так и другие коллекции с соответствующим типом элементов:

```
fun main() {
    val list = mutableListOf(1, 2)
    list += 3  ← Оператор += изменяет список
    val newList = list + listOf(4, 5) ← Оператор + возвращает новый
    println(list)                      список со всеми элементами
    // [1, 2, 3]
    println(newList)
    // [1, 2, 3, 4, 5]
}
```

До сих пор мы обсуждали перегрузку *бинарных* (или *двоичных*) операторов. Они применяются к двум значениям, например `a + b`. В Kotlin есть возможность перегружать и *унарные* операторы. Они применяются к одному значению, например `-a`.

9.1.3. Операторы с одним операндом: перегрузка унарных операторов

Процедура перегрузки унарного оператора аналогична представленной ранее: объявите функцию (член или расширение) с предопределенным именем, а затем пометьте ее модификатором `operator`. Рассмотрим пример (листинг 9.5).

Листинг 9.5. Определение унарного арифметического оператора

```
operator fun Point.unaryMinus(): Point { ← Унарная функция minus
    return Point(-x, -y) ← не содержит параметров
}

fun main() {
    val p = Point(10, 20)
    println(-p)
    // Point(x=-10, y=-20)
}
```

Отрицание координат объекта класса point и его возврат

Функции перегрузки унарных операторов не принимают никаких аргументов. Как показано на рис. 9.3, унарный оператор `+` работает аналогичным образом. В табл. 9.2 перечислены все перегружаемые унарные операторы.

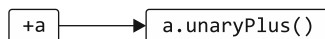


Рис. 9.3. Унарный оператор `+` преобразуется в вызов функции `unaryPlus`

Таблица 9.2. Перегружаемые операторы унарной арифметики

Выражение	Имя функции
<code>+a</code>	<code>unaryPlus</code>
<code>-a</code>	<code>unaryMinus</code>
<code>!a</code>	<code>not</code>
<code>++a, a++</code>	<code>inc</code>
<code>--a, a--</code>	<code>dec</code>

При определении функций `inc` и `dec`, выполняемой в целях перегрузки операторов инкремента и декремента, компилятор автоматически поддерживает ту же семантику для операторов пре- и постинкремента, как и в случае с обычными числовыми типами. Рассмотрим следующий пример (листинг 9.6). Здесь оператор `++` перегружен для класса `BigDecimal` из стандартной библиотеки Java.

Листинг 9.6. Определение инкрементного оператора

```
import java.math.BigDecimal

operator fun BigDecimal.inc() = this + BigDecimal.ONE

fun main() {
    var bd = BigDecimal.ZERO
    println(bd++) ← Инкремент выполняется после первого
                  // 0                               выполнения инструкции println
    println(bd)
    // 1
    println(++bd) ← Инкремент выполняется после второго
                  // 2                               выполнения инструкции println
}
```

Постфиксная операция `++` сначала возвращает текущее значение переменной `bd`, а затем увеличивает его, в то время как префиксная выполняется в обратном порядке. Выведенные на экран значения совпадают с теми, которые вы получили бы при использовании переменной типа `Int`. Ничего особенного для поддержки такого решения делать не нужно.

9.2. ПЕРЕГРУЗКА ОПЕРАТОРОВ СРАВНЕНИЯ УПРОЩАЕТ ПРОВЕРКУ ОТНОШЕНИЙ МЕЖДУ ОБЪЕКТАМИ

Как и в случае с арифметическими операторами, Kotlin позволяет использовать операторы сравнения (`==`, `!=`, `>`, `<` и т. д.) с любыми объектами, а не только с примитивными типами. Вместо того чтобы вызывать функции `equals` или `compareTo`, как в Java, допускается использовать операторы сравнения напрямую, что интуитивно понятно и лаконично. В этом разделе мы рассмотрим соглашения для поддержки этих операторов.

9.2.1. Операторы равенства: equals (==)

Мы затронули тему равенства в главе 4. В ней говорилось, что использование оператора `==` в Kotlin преобразуется в вызов метода `equals`. Это еще одно применение описанного ранее принципа соглашений.

Применение оператора `!=` тоже преобразуется в вызов метода `equals`. Разница лишь в том, что результат инвертируется. Обратите внимание: в отличие от всех остальных операторов `==` и `!=` можно использовать с операндами, допускающими значение `null`, поскольку эти операторы скрыто проверяют равенство `null`. Сравнение `a == b` проверяет, не равна ли переменная `a` значению `null`, и если нет, то вызывает метод `a.equals(b)` (рис. 9.4). В противном случае результат будет равен `true`, только если оба аргумента представлены ссылками `null`.

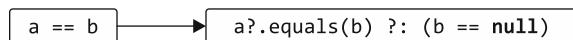


Рис. 9.4. Проверка равенства `==` преобразуется в вызов метода `equals` и проверку `null`

Для класса `Point` реализация метода `equals` генерируется компилятором автоматически, поскольку он был отмечен как класс `data` (подробности этого процесса были описаны в подразделе 4.3.2). Но если реализовать его вручную, то код может выглядеть следующим образом (листинг 9.7) (обратите внимание, что полная реализация должна предоставлять еще и реализацию метода `hashCode`; мы опустили ее здесь для краткости).

Листинг 9.7. Реализация метода `equals`

```

class Point(val x: Int, val y: Int) {
    override fun equals(other: Any?): Boolean {
        if (other === this) return true
        if (other !is Point) return false
        return other.x == x && other.y == y
    }
}

fun main() {
    println(Point(10, 20) == Point(10, 20))
    // истина
    println(Point(10, 20) != Point(5, 5))
    // истина
    println(null == Point(1, 2))
    // ложь
}
  
```

Переопределение метода из класса `Any`
 Оптимизация: проверка того, является ли параметр тем же объектом, что и выражение `this`
 Проверка типа параметра
 Применение умного преобразования для класса `Point`, чтобы получить доступ к свойствам `x` и `y`

Оператор тождества или равенства ссылок (`===`) применяется для проверки того, является ли параметр `equals` тем же объектом, что и тот, для которого вызывается `equals`. Оператор равенства ссылок делает то же самое, что и оператор `==` в Java: проверяет, ссылаются ли оба его аргумента на один и тот же объект (или хранят одно и то же значение, если относятся к примитивному типу). Использование

этого оператора — обычная оптимизация при реализации метода `equals`. Следует отметить, что перегрузка оператора `===` не допускается.

Функция `equals` помечена как `override`, поскольку, в отличие от других соглашений, реализующий ее метод определен в классе `Any` (сравнение равенства поддерживается для всех объектов в Kotlin). Это объясняет еще и то, почему нет необходимости помечать его как `operator`: базовый метод в `Any` помечен как таковой, а модификатор `operator` применяется ко всем методам, которые его реализуют или переопределяют. Кроме того, обратите внимание на то, что метод `equals` не может быть реализован как расширение. Это связано с тем, что реализация, унаследованная от класса `Any`, всегда получает приоритет над расширением.

Еще в листинге 9.7 показано, что использование оператора `!=` также преобразуется в вызов метода `equals`. Компилятор автоматически отрицает возвращаемое значение, поэтому корректная работа не требует выполнения дополнительных действий.

Теперь перейдем к другим операторам сравнения.

9.2.2. Упорядочение операторов: `compareTo` (<, >, ? и >=)

В Java классы могут реализовывать интерфейс `Comparable` для использования в алгоритмах со сравнениями значений, таких как поиск максимума или сортировка. Метод `compareTo` из этого интерфейса используется для определения, какой из объектов больше. Но в Java не существует сокращенного синтаксиса для вызова данного метода. Только значения примитивных типов можно сравнивать с помощью операторов < и >. Для всех остальных типов требуется непосредственная запись `element1.compareTo(element2)`.

Этот же интерфейс `Comparable` поддерживается в Kotlin. Но метод `compareTo` из данного интерфейса можно вызвать по соглашению, и применение операторов сравнения (<, >, ? и >=) преобразуется в вызовы метода `compareTo` (рис. 9.5). Типом возвращаемого `compareTo` значения должен быть `Int`. Выражение `p1 < p2` эквивалентно выражению `p1.compareTo(p2) < 0`. Остальные операторы сравнения работают точно так же.

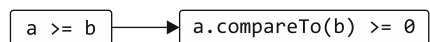


Рис. 9.5. Сравнение двух объектов преобразуется в сравнение результата вызова метода `compareTo` с нулем

Не существует однозначно правильного способа сравнения двумерных точек друг с другом, поэтому воспользуемся уже известным вам классом `Person`, чтобы показать, как можно реализовать данный метод (листинг 9.8). В реализации будет использоваться упорядочение адресной книги (сравнение по фамилии, а затем, если она совпадает, по имени).

В этом случае интерфейс `Comparable` реализуется затем, чтобы объекты класса `Person` можно было сравнивать с помощью не только кода Kotlin, но и функций Java, предназначенных, например, для сортировки коллекций. Как и в случае с методом `equals`, модификатор `operator` применяется к функции в базовом интерфейсе, поэтому повторять ключевое слово при переопределении функции не требуется.

Листинг 9.8. Реализация метода `compareTo`

```

class Person(
    val firstName: String, val lastName: String
) : Comparable<Person> {

    override fun compareTo(other: Person): Int {
        return compareValuesBy(this, other,
                                Person::lastName, Person::firstName)
    }
}

fun main() {
    val p1 = Person("Alice", "Smith")
    val p2 = Person("Bob", "Johnson")
    println(p1 < p2)
    // ложь
}

```

← Вычисление заданной функции или ссылки на свойства по порядку и сравнение значений

Обратите внимание, что использовать функцию `compareValuesBy` можно из стандартной библиотеки Kotlin. Это позволит реализовать метод `compareTo` просто и лаконично. Функция получает список функций выбора (селекторов), вычисляющих значения для сравнения. Она вызывает каждую функцию выбора по порядку для обоих объектов и сравнивает возвращаемые значения. Если они различаются, то возвращается результат сравнения. Если одинаковы, то выполняется следующий селектор или возвращается 0, если больше нечего вызывать. Эти функции выбора можно передавать как лямбды или как ссылки на свойства.

Стоит отметить, что выполнить прямую реализацию для сравнения полей вручную будет быстрее, хотя кода для нее потребуется больше. Как всегда, предпочтение отдается лаконичной версии, а вопрос производительности следует учитывать, если известно, что эта реализация будет вызываться очень часто.

Все классы, реализующие интерфейс `Comparable`, можно сравнивать в Kotlin с помощью лаконичного синтаксиса операторов. Это относится и к типу `String`, например:

```

fun main() {
    println("abc" < "bac")
    // истина
}

```

Для этого не нужно добавлять никакие расширения.

9.3. СОГЛАШЕНИЯ ДЛЯ КОЛЛЕКЦИЙ И ДИАПАЗОНОВ

К самым распространенным операциям при работе с коллекциями относятся получение и размещение элементов по индексу, а также проверка принадлежности элемента к коллекции. Все эти операции поддерживаются с помощью синтаксиса оператора: чтобы получить или задать элемент по индексу, используется синтаксис `a[b]` (так называемый *оператор индексированного доступа*). Оператор `in` может применяться для проверки принадлежности элемента к коллекции или диапазону,

а также для итерации по коллекции. Эти операции вы можете добавить для собственных классов, работающих как коллекции. Теперь рассмотрим соглашения, используемые для поддержки этих операций.

9.3.1. Доступ к элементам по индексу: соглашения для `get` и `set`

В Kotlin обращаться к элементам карты можно аналогично обращению к массивам в Java — с помощью квадратных скобок:

```
val value = map[key]
```

Этот же оператор можно использовать для изменения значения ключа в изменяемой карте:

```
mutableMap[key] = newValue
```

Разберемся, как это работает. В Kotlin оператор индексированного доступа — еще одно соглашение. Чтение элемента с помощью оператора индексированного доступа преобразуется в вызов метода оператора `get`, а запись элемента — в вызов `set`. Методы уже определены для интерфейсов `Map` и `MutableMap`. А теперь рассмотрим, как добавить аналогичные методы в собственный класс.

Указать координаты точки можно с помощью квадратных скобок: `p[0]` для доступа к координате *x* и `p[1]` для доступа к координате *y*. В листинге 9.9 показано, как их реализовать и применить.

Листинг 9.9. Реализация соглашения `get`

```
operator fun Point.get(index: Int): Int {
    return when(index) {
        0 -> x
        1 -> y
        else ->
            throw IndexOutOfBoundsException("Invalid coordinate $index")
    }
}

fun main() {
    val p = Point(10, 20)
    println(p[1])
    // 20
}
```

Определение операторной функции под названием `get`

Получение координат, соответствующих заданному индексу

Требуется только определить функцию под названием `get` и пометить ее директивой `operator`. После этого выражение вида `p[1]`, где `p` относится к типу `Point`, будет преобразовано в вызов метода `get`.

Стоит отметить, что параметр метода `get` может быть любого типа, а не только `Int`. Например, если оператор индексирования применяется к карте, то тип параметра является типом ключа карты и может быть произвольным. Вдобавок метод `get`

можно определить с несколькими параметрами. Например, при реализации класса, который будет представлять двумерный массив или матрицу, можно определить метод `operator fun get(rowIndex: Int, colIndex: Int)` и вызвать его как `matrix[row, col]` (рис. 9.6).

Можно определить несколько перегруженных методов `get` с различными типами параметров, если есть возможность обращаться к коллекции, используя разные типы ключей.



Рис. 9.6. Обращение с помощью квадратных скобок преобразуется в вызов метода `get`

Аналогичным образом, используя синтаксис со скобками, можно определить функцию для изменения значения по заданному индексу. Класс `Point` неизменяемый, поэтому не имеет смысла определять для него такой метод. Определим другой класс для представления изменяемой точки и используем его в качестве примера (листинг 9.10).

Листинг 9.10. Реализация соглашения `set`

```

data class MutablePoint(var x: Int, var y: Int)

operator fun MutablePoint.set(index: Int, value: Int) {
    when(index) {
        0 -> x = value
        1 -> y = value
        else ->
            throw IndexOutOfBoundsException("Invalid coordinate $index")
    }
}

fun main() {
    val p = MutablePoint(10, 20)
    p[1] = 42
    println(p)
    // MutablePoint(x=10, y=42)
}
  
```

Определение операторной функции `set`

Изменение координат, соответствующих заданному индексу

Этот пример тоже простой: чтобы разрешить использование оператора индексированного доступа в операциях присваивания, достаточно определить функцию `set`. Последний параметр `set` получает значение для правой части операции присваивания, а остальные аргументы (в случае класса `Point` только `index`) будут получены из индексов в скобках.



Рис. 9.7. Присваивание с помощью квадратных скобок преобразуется в вызов функции `set`

В общем виде это показано на рис. 9.7.

9.3.2. Проверка принадлежности объекта к коллекции: соглашение `in`

Еще один оператор, поддерживаемый коллекциями, — `in`. Он используется для проверки принадлежности объекта к коллекции. Соответствующая ему функция называется `contains`. Реализуем ее для того, чтобы с помощью оператора `in` проверять принадлежность точки к прямоугольнику (листинг 9.11).

Листинг 9.11. Реализация соглашения `in`

```
data class Rectangle(val upperLeft: Point, val lowerRight: Point)

operator fun Rectangle.contains(p: Point): Boolean {
    return p.x in upperLeft.x..<lowerRight.x &&
           p.y in upperLeft.y..<lowerRight.y
}

fun main() {
    val rect = Rectangle(Point(10, 20), Point(50, 50))
    println(Point(20, 30) in rect)
    // истина
    println(Point(5, 5) in rect)
    // ложь
}
```

Создание диапазона значений
и проверка, что координата *x*
принадлежит этому диапазону

Использование оператора `<`
для создания открытого диапазона

Объект в правой части оператора `in` становится объектом, для которого вызывается метод `contains`. Объект же в левой части становится аргументом, передаваемым методу (рис. 9.8).

В реализации метода `Rectangle.contains` используется оператор `..<` для создания открытого диапазона. Затем оператор `in` применяется к диапазону, чтобы проверить принадлежность к нему точки.



Рис. 9.8. Оператор `in` преобразуется в вызов функции `contains`

Открытый диапазон не содержит конечной точки. Например, если создать обычный (закрытый) диапазон с помощью выражения `10..20`, то в него войдут все числа от 10 до 20, в том числе 20. Открытый диапазон `10...<20` содержит числа от 10 до 19, но не 20. Класс прямоугольника часто определяется таким образом, что его нижняя и правая координаты не считаются частью прямоугольника. Именно поэтому здесь уместно использовать открытые диапазоны.

9.3.3. Создание диапазонов из объектов: соглашения `rangeTo` и `rangeUntil`

Для создания диапазона используется синтаксис `..` — например, `1..10` представляет числа от 1 до 10. Описание диапазонов приводилось в главе 2, а здесь мы обсудим соглашение, которое помогает их создавать. Оператор `..` представляет собой лаконичную форму вызова функции `rangeTo` (рис. 9.9).



Рис. 9.9. Оператор `..` преобразуется в вызов функции `rangeTo`

Функция `rangeTo` возвращает диапазон `start..end` `start.rangeTo(end)`. Данный оператор можно определить для разрабатываемого класса. Но в этом нет необходимости, если в классе реализован интерфейс `Comparable`: можно создавать диапазон сравниваемых элементов, используя стандартную библиотеку Kotlin.

В библиотеке определена функция `rangeTo`, и ее можно вызвать для любого сравниваемого элемента:

```
operator fun <T: Comparable<T>> T.rangeTo(that: T): ClosedRange<T>
```

Эта функция возвращает диапазон для проверки принадлежности ему различных элементов.

В качестве примера создадим диапазон дат с помощью класса `LocalDate` (определен в стандартной библиотеке Java 8) (листинг 9.12).

Листинг 9.12. Работа с диапазоном дат

```
import java.time.LocalDate

fun main() {
    val now = LocalDate.now()
    val vacation = now..now.plusDays(10)
    println(now.plusWeeks(1) in vacation)
    // истина
}
```

Создание 10-дневного диапазона, начинающегося с данного момента

Проверка того, принадлежит ли определенная дата заданному диапазону

Выражение `now..now.plusDays(10)` преобразуется компилятором в `now.rangeTo(now.plusDays(10))`. Функция `rangeTo` не относится к членам класса `LocalDate`, но представляет собой функцию-расширение интерфейса `Comparable`, как было показано ранее.

Приоритет оператора `rangeTo` ниже, чем у арифметических операторов. Но лучше использовать круглые скобки для его аргументов, чтобы избежать путаницы:

```
fun main() {
    val n = 9
    println(0..(n + 1))
    // 0..10
}
```

Можно написать просто `0..n + 1`, но со скобками будет понятнее

Кроме того, обратите внимание, что выражение `0..n.forEach { }` не скомпилируется, поскольку для вызова метода выражение диапазона необходимо окружить круглыми скобками:

```
fun main() {
    val n = 9
    (0..n).forEach { print(it) }
    // 0123456789
}
```

Диапазон необходимо поместить в скобки, чтобы для него можно было вызвать метод

Оператор `rangeUntil (<..<)`, аналог оператора `rangeTo`, возвращает диапазон с исключающей верхней границей:

```
fun main() {
    (0..<9).forEach { print(it) }
    // 012345678
}
```

А теперь обсудим, как соглашения позволяют выполнять итерацию по коллекции или диапазону.

9.3.4. Создание возможности циклического перехода по типам: соглашение `iterator`

В разделе 2.4 мы обсуждали, что циклы `for` в Kotlin используют тот же оператор `in`, что и проверки диапазона. Но в данном контексте он имеет иное значение и используется для выполнения итерации. Это означает, что такое выражение, как `for (x in list) { ... }`, будет преобразовано в вызов функции `list.iterator()`, и для получения результатов ее выполнения, как и в Java, будут многократно вызваны методы `hasNext` и `next`.

Обратите внимание, что в Kotlin данный механизм также относится к соглашениям. А это означает, что метод `iterator` можно определить как расширение. Это объясняет, почему можно выполнять итерацию по обычной строке: стандартная библиотека Kotlin определяет функцию-расширение `iterator` для `CharSequence` — суперкласса типа `String`:

```
operator fun CharSequence.iterator(): CharIterator
fun main() {
    for (c in "abc") { }
```

← Эта библиотечная функция позволяет выполнять итерации по строке

Функцию `iterator` вы можете определить как метод в собственных классах или функцию-расширение для используемых вами сторонних классов. Как в следующем примере (листинг 9.13), можно определить функцию-расширение для итерации по объектам класса `LocalDate`. Функция `iterator` должна возвращать объект для реализации интерфейса `Iterator<LocalDate>`. Поэтому необходимо использовать объявление объекта (было описано в подразделе 4.4.1), чтобы определить реализации функций `hasNext` и `next`, ожидаемых интерфейсом.

Листинг 9.13. Реализация итератора диапазона дат

```
import java.time.LocalDate

operator fun ClosedRange<LocalDate>.iterator(): Iterator<LocalDate> =
    object : Iterator<LocalDate> {
        var current = start

        override fun hasNext() =
            current <= endInclusive

        override fun next(): LocalDate {
            val thisDate = current
            current = current.plusDays(1)
            return thisDate
        }
    }
```

← Этот объект реализует интерфейс `Iterator` для элементов класса `LocalDate`

← Соглашение `compareTo` используется для дат

← Инкремент текущей даты на один день

← Возврат преинкрементированной даты

```
fun main() {
    val newYear = LocalDate.ofYearDay(2042, 1)
    val daysOff = newYear.minusDays(1)..newYear
    for (dayOff in daysOff) { println(dayOff) }
    // 2041-12-31
    // 2042-01-01
}
```

Итерация по коллекции `daysOff`,
если доступна соответствующая
функция-итератор

Обратите внимание на определение метода `iterator` для пользовательского типа диапазона: в качестве аргумента типа используется `LocalDate`. Библиотечная функция `rangeTo`, описанная выше, возвращает экземпляр коллекции `ClosedRange`, а расширение `ClosedRange<LocalDate>` под названием `iterator` позволяет использовать экземпляр диапазона в цикле `for`.

9.4. СОЗДАНИЕ ОБЪЯВЛЕНИЙ ДЕСТРУКТУРИЗАЦИИ С ПОМОЩЬЮ КОМПОНЕНТНЫХ ФУНКЦИЙ

При обсуждении классов данных в главе 4 мы упомянули, что некоторые из их возможностей будут представлены позже. Теперь, когда вы знакомы с принципом соглашений, можно рассмотреть последнюю функциональную возможность — *объявление дедекструктуризации*. С его помощью можно распаковать одно составное значение и использовать его для инициализации нескольких отдельных локальных переменных. Логика работы этого функционала выглядит так:

```
fun main() {
    val p = Point(10, 20)
    val (x, y) = p
    println(x)
    // 10
    println(y)
    // 20
}
```

Объявление переменных `x` и `y`,
инициализированных в качестве
компонентов `p`

Объявление дедекструктуризации выглядит как обычное объявление переменной, но содержит несколько переменных, сгруппированных в круглых скобках.

Внутри системы объявление дедекструктуризации снова использует принцип соглашений. Для инициализации каждой переменной в объявлении дедекструктуризации вызывается функция с именем `componentN`, где `N` — позиция переменной в объявлении. Другими словами, предыдущий пример будет преобразован следующим образом (рис. 9.10).

<code>val (a, b) = p</code>	→	<code>val a = p.component1()</code> <code>val b = p.component2()</code>
-----------------------------	---	--

Рис. 9.10. Объявления дедекструктуризации преобразуются в вызовы функций `componentN`

В `data`-классе компилятор генерирует функцию `componentN` для каждого свойства, объявленного в первичном конструкторе. В примере ниже показано, как можно объявить эти функции вручную для класса, не имеющего модификатора `data`:

```
class Point(val x: Int, val y: Int) {
    operator fun component1() = x
    operator fun component2() = y
}
```

Один из основных примеров применения объявления деструктуризации — возврат нескольких значений из функции. Если это нужно сделать, то можно определить класс данных для хранения возвращаемых значений и использовать его в качестве возвращаемого типа функции. Синтаксис объявления деструктуризации позволяет легко распаковать и использовать значения после вызова функции. Для демонстрации напишем простую функцию разделения имени файла на имя и расширение (листинг 9.14).

Листинг 9.14. Использование объявления деструктуризации для возврата нескольких значений

```
data class NameComponents(val name: String,
                          val extension: String)

fun splitFilename(fullName: String): NameComponents {
    val result = fullName.split('.', limit = 2)
    return NameComponents(result[0], result[1])

fun main() {
    val (name, ext) = splitFilename("example.kt")
    println(name)
    // example
    println(ext)
    // kt
}
```

Объявление класса данных для хранения значений

Возврат экземпляра класса данных из функции

Использование синтаксиса объявления деструктуризации для распаковки класса

Пример можно доработать, если обратить внимание, что функции `componentN` определяются еще и для массивов и коллекций. Это полезно, когда работа ведется с коллекциями известного размера, а это как раз такой случай: метод `split` возвращает список из двух элементов (листинг 9.15).

Листинг 9.15. Использование объявления деструктуризации с коллекцией

```
data class NameComponents(
    val name: String,
    val extension: String)

fun splitFilename(fullName: String): NameComponents {
    val (name, extension) = fullName.split('.', limit = 2)
    return NameComponents(name, extension)
}
```

Конечно, невозможно и не очень полезно определять бесконечное количество функций `componentN`, у которых синтаксис будет работать с произвольным количеством элементов. Стандартная библиотека позволяет использовать этот синтаксис для доступа к первым пяти элементам контейнера.

Более простой способ вернуть несколько значений из функции — использовать классы `Pair` и `Triple` из стандартной библиотеки. Для этого может потребоваться меньше кода, чем для определения собственного класса. Но придется отказаться от ценной выразительности кода, поскольку `Pair` и `Triple` не дают понять, что содержится в возвращаемом объекте.

9.4.1. Объявления деструктуризации и циклы

Объявления деструктуризации работают не только как инструкции верхнего уровня в функциях, но и в других конструкциях, где можно объявлять переменные, например, в циклах. Одним из хороших вариантов использования этого метода может служить перечисление записей на карте. В листинге 9.16 приведен небольшой пример, в котором показано, как с помощью этого синтаксиса можно вывести на экран все записи в заданной карте.

Листинг 9.16. Использование объявления деструктуризации для итерации по карте

```
fun printEntries(map: Map<String, String>) {
    for ((key, value) in map) {
        println("$key -> $value")
    }
}

fun main() {
    val map = mapOf("Oracle" to "Java", "JetBrains" to "Kotlin")
    printEntries(map)
    // Oracle -> Java
    // JetBrains -> Kotlin
}
```

← Объявление
деструктуризации
в цикле

В этом простом примере используются два соглашения Kotlin: для итерации по объекту и для объявлений деструктуризации. В стандартной библиотеке Kotlin есть функция-расширение `iterator` для коллекции `Map`. Она возвращает итератор по записям карты. Таким образом, в отличие от Java, здесь можно выполнять итерации по карте напрямую. К тому же в библиотеке есть функции-расширения `component1` и `component2` для элемента `Map.Entry`, возвращающие ключ и значение соответственно. По сути, приведенный выше цикл преобразуется в эквивалент следующего кода:

```
for (entry in map.entries) {
    val key = entry.component1()
    val value = entry.component2()
    // ...
}
```

Кроме того, можно использовать объявления деструктуризации, когда лямбда получает составное значение, например, `data class` или карту. В этом примере все записи в заданной карте снова выводятся на экран. Однако на этот раз используется функция `.forEach`, описанная в главе 5:

```
map.forEach { (key, value) ->
    println("$key -> $value")
}
```

Благодаря этим примерам вы снова можете увидеть, насколько функции-расширения важны для поддержки соглашений Kotlin.

9.4.2. Игнорирование деструктурированных значений с помощью символа `_`

Если деструктуризации подвергается объект с большим количеством компонентов, то возникает вероятность, что все они вам не понадобятся. В листинге 9.17 деструктурируется класс `Person`, но в действительности необходимо использовать только поля `firstName` и `age`.

Листинг 9.17. Деструктуризация объекта `Person`

```
data class Person(
    val firstName: String,
    val lastName: String,
    val age: Int,
    val city: String,
)

fun introducePerson(p: Person) {
    val (firstName, lastName, age, city) = p
    println("This is $firstName, aged $age.")
}
```

В данном случае объявление локальных переменных `lastName` и `city` не представляет никакой ценности для нашего кода. Скорее, из-за этого тело функции загромождается неиспользуемыми переменными, чего, как правило, лучше избегать.

Не требуется деструктурировать весь объект, поэтому можно не добавлять в объявление деструктуризации последующие объявления (в данном случае `city`). Деструктуризации подвергаются только первых три элемента:

```
val (firstName, lastName, age) = p
```

Чтобы избавиться от объявления параметра `lastName`, нужно выполнить несколько иные действия. Если просто удалить его (оставив `(firstName, age)`), то содержимое `Person.lastName` будет ошибочно присвоено переменной `age`. Помните, что внутри системы это объявление деструктуризации вызывает только функции `component1` и `component2`, независимо от того, какое имя вы им дадите. Чтобы справиться с этой

проблемой, Kotlin позволяет назначать неиспользуемые объявления во время деструктуризации, присваивая им зарезервированный символ `_`.

Теперь, обладая этими знаниями, вы сможете сделать реализацию функции `introducePerson` более лаконичной: переименовать `lastName` в `_` и полностью удалить параметр `city` в процессе деструктуризации:

```
fun introducePerson(p: Person) {
    val (firstName, _, age) = p
    println("This is $firstName, aged $age.")
}
```

Чтобы игнорировать компонент во время деструктуризации, назначьте вместо него символ `_`

ОГРАНИЧЕНИЯ И НЕДОСТАТКИ ДЕСТРУКТУРИЗАЦИИ В KOTLIN

Реализация объявлений деструктуризации в Kotlin *позиционная*. То есть результат операции деструктуризации полностью зависит от позиции аргументов. Для класса данных `Person` из листинга 9.17 это означает, что переменным при деструктуризации всегда будут присваиваться значения в том же порядке, в каком они появляются в конструкторе:

```
val (firstName, lastName, age, city) = p
```

Имена переменных, которым присваивается результат деструктуризации, неважны. Объявления деструктуризации выполняют итерации по функциям `componentN` по очереди, поэтому данный код работает так же хорошо:

```
val (f, l, a, c) = p
```

Это может привести к трудноуловимым проблемам, когда в процессе рефакторинга меняется порядок свойств в классе данных:

```
data class Person(
    val lastName: String,
    val firstName: String,
    val age: Int,
    val city: String,
)
```

Компоненты `firstName` и `lastName` поменялись местами

Теперь предыдущий фрагмент кода все еще работает, но ложно присваивает значение `lastName` компоненту `firstName`, и наоборот:

```
val (firstName, lastName, age, city) = p
```

Такое поведение означает, что объявления деструктуризации лучше использовать только для небольших контейнерных классов (например, пар «ключ — значение» или пар «индекс — значение») или классов с минимальной вероятностью изменений в будущем. Таких объявлений следует избегать, если вы работаете с более сложными сущностями.

Потенциальным решением этой проблемы послужит введение деструктуризации на основе имен. Данная тема на момент написания книги рассматривалась для классов значений Kotlin (<http://mng.bz/v17r>). Многофайловые классы значений планируется добавить в одной из будущих версий Kotlin.

9.5. ПОВТОРНОЕ ИСПОЛЬЗОВАНИЕ ЛОГИКИ МЕТОДОВ ДОСТУПА К СВОЙСТВАМ: ДЕЛЕГИРОВАННЫЕ СВОЙСТВА

Завершая эту главу, рассмотрим еще одну функциональную возможность на базе соглашений, которая стала одной из самых уникальных и эффективных в Kotlin, — *делегированные свойства*. Она позволяет легко реализовать свойства, которые работают более сложным образом, чем хранение значений в теневых полях, не дублируя логику в каждом методе доступа. Например, свойства могут хранить свои значения в таблицах базы данных, в сеансе браузера, в карте и т. д.

В основе этой возможности лежит паттерн проектирования «Делегирование». При его использовании задача делегируется другому объекту-помощнику, а не выполняется самостоятельно. Объект-помощник называется *делегатом*. Этот паттерн мы представили вам в подразделе 4.3.3, когда обсуждали делегирование классов. В данном случае он применяется к свойству, а оно, в свою очередь, может делегировать логику своих методов доступа объекту-помощнику. Можно реализовать этот механизм вручную или воспользоваться поддержкой языка Kotlin. Чуть ниже мы представим примеры для обоих вариантов, но сначала дадим общее объяснение.

9.5.1. Основной синтаксис и работа делегированных свойств

Общий синтаксис делегированного свойства выглядит так:

```
var p: Type by Delegate()
```

Свойство `p` делегирует логику своих методов доступа другому объекту, в данном случае новому экземпляру класса `Delegate`. Объект получается при вычислении выражения, следующего за ключевым словом `by`. Выражение может быть любым, но должно удовлетворять правилам соглашения для делегатов свойств.

Рассмотрим, что происходит внутри класса, определяющего делегированное свойство:

```
class Foo {  
    var p: Type by Delegate()  
}
```

Компилятор создает скрытое вспомогательное свойство, инициализированное экземпляром объекта-делегата, которому делегируется исходное свойство `p`. Для простоты назовем его `delegate`:

```
class Foo {  
    private val delegate = Delegate()  ← Это вспомогательное свойство  
                                       ← сгенерировано компилятором  
    var p: Type                       ← Сгенерированные методы доступа свойства p  
                                       ← вызывают методы getValue и setValue делегата  
        set(value: Type) = delegate.setValue(/* ... */, value)  
        get() = delegate.getValue(/* ... */)   
}
```

По соглашению класс `Delegate` должен содержать операторные функции `getValue` и `setValue`. Хотя последняя требуется только для изменяемых свойств (то есть при определении `var delegate = ...`). Кроме того, они могут (но не обязаны) предоставлять реализацию функции `provideDelegate`. В ней можно выполнить логику проверки или изменить способ создания экземпляра делегата. Как обычно, они могут быть реализованы в виде членов класса или расширений. Чтобы упростить объяснение, мы опустим их параметры, а точные сигнатуры будут рассмотрены позже в этой главе. В простой форме класс `Delegate` может выглядеть так:

```

class Delegate {
    operator fun getValue(/* ... */) { /* ... */ }

    operator fun setValue(/* ... */, value: Type) { /* ... */ }

    operator fun provideDelegate(/* ... */): Delegate { /* ... */ }
}

class Foo {
    var p: Type by Delegate()
}

fun main() {
    val foo = Foo()
    val oldValue = foo.p
    foo.p = newValue
}

```

Метод `getValue` содержит логику для реализации геттера

Метод `setValue` содержит логику для реализации сеттера

Ключевое слово `by` связывает свойство с объектом-делегатом (в данном случае с новым экземпляром класса `Delegate`)

Метод `provideDelegate` содержит логику для предоставления или создания делегата

При создании типа с делегированным свойством вызывается метод `delegate.provideDelegate()`, если таковой имеется

Доступ к свойству `foo.p` вызывает внутри системы метод `delegate.getValue(...)`

Изменение значения свойства вызывает метод `delegate.setValue(..., newValue)`

Свойство `foo.p` используется как обычное, но внутри системы вызываются методы вспомогательного свойства типа `Delegate`. Чтобы изучить использование этого механизма на практике, начнем с одного примера силы делегированных свойств — библиотечной поддержки отложенной инициализации. Затем рассмотрим, как определить собственные делегированные свойства и когда это бывает полезно.

9.5.2. Использование делегированных свойств: отложенная инициализация и выполняемая с помощью функции `lazy()`

Отложенная инициализация — распространенный паттерн, и он подразумевает создание части объекта по требованию, когда к нему обращаются в первый раз. Это полезно, когда процесс инициализации потребляет значительные ресурсы, а данные не всегда нужны при использовании объекта.

Для примера рассмотрим класс `Person`. Он дает возможность получить доступ к списку писем, написанных человеком. Письма хранятся в базе данных, и доступ к ним занимает много времени. Требуется загружать письма при первом обращении

к свойству и делать это только один раз. Допустим, есть функция `loadEmails` для извлечения электронных писем из базы данных:

```
class Email { /*...*/ }
fun loadEmails(person: Person): List<Email> {
    println("Load emails for ${person.name}")
    return listOf(/*...*/)
}
```

Реализовать отложенную загрузку можно с помощью дополнительного свойства `_emails` (листинг 9.18). Оно хранит значение `null` до загрузки и список писем после. Свойство `emails` само использует пользовательские методы доступа (о них мы говорили в подразделе 2.2.2).

Листинг 9.18. Реализация отложенной инициализации с помощью теневого свойства

```
class Person(val name: String) {
    private var _emails: List<Email>? = null
    val emails: List<Email>
        get() {
            if (_emails == null) {
                _emails = loadEmails(this)
            }
            return _emails!!
        }
}

fun main() {
    val p = Person("Alice")
    p.emails
    // Загрузка свойства emails для Alice
    p.emails
}
```

Свойство `_emails`, в котором хранятся данные и которому свойство `emails` делегирует задачи

Загрузка данных при обращении

Если данные были загружены ранее, то оператор их возвращает

Адреса электронной почты загружены при первом обращении

Здесь используется так называемый прием *теневого свойства*. В коде есть свойство `_emails`, которое хранит значение, а другое, `emails`, служит для обеспечения доступа на чтение к нему. Необходимо использовать два свойства, поскольку у них разные типы: `_emails` допускает значение `null`, а `emails` является не `null`. Если в классе есть два свойства для представления одной и той же концепции, то у приватного свойства указывается префикс с подчеркиванием (`_emails`), а у публичного префикса нет (`email`).

Стоит ознакомиться с этим приемом, поскольку он используется довольно часто. Но код несколько громоздкий, и представьте, насколько длиннее он стал после добавления нескольких отложенных свойств. Более того, он не всегда работает корректно: эта реализация не является потокобезопасной. При одновременном обращении к свойству `emails` двух потоков нет механизма, который позволил бы предотвратить двойной вызов дорогостоящей функции `loadEmails`. В лучшем случае

это лишь трата ресурсов, но в худшем вы получите непоследовательное состояние приложения. Безусловно, Kotlin предлагает лучшее решение.

Код значительно упрощается, если использовать делегированное свойство. Оно может инкапсулировать теневое свойство, в котором хранится значение, и логику, обеспечивающую инициализацию значения только один раз. Делегат для текущего примера возвращается стандартной библиотечной функцией `lazy` (листинг 9.19).

Листинг 9.19. Реализация отложенной инициализации с помощью делегированного свойства

```
class Person(val name: String) {  
    val emails by lazy { loadEmails(this) }  
}
```

Функция `lazy` возвращает объект, содержащий метод под названием `getValue`, с соответствующей сигнатурой. Поэтому можно использовать его с ключевым словом `by` для создания делегированного свойства. В роли аргумента `lazy` выступает лямбда, которую эта функция вызывает для инициализации значения. Функция `lazy` по умолчанию потокобезопасна. При необходимости можно задать дополнительные параметры, чтобы указать ей, какую блокировку использовать. Или же допускается полностью обойти синхронизацию, если класс не будет применяться в многопоточной среде.

В следующем подразделе мы подробно рассмотрим, как работает механизм делегированных свойств, и обсудим используемые соглашения.

9.5.3. Реализация собственных делегированных свойств

Чтобы узнать, как реализуются делегированные свойства, рассмотрим другой пример — задачу уведомления слушателей об изменении свойства объекта. Такой подход используется очень часто. Например, когда объекты представлены в пользовательском интерфейсе и требуется автоматически обновлять его, если они изменяются.

Обычно это называется *наблюдаемостью*. Опишем ее реализацию в Kotlin. Сначала рассмотрим вариант без делегированных свойств. Затем выполним рефакторинг кода, чтобы использовать их.

Класс `Observable` управляет списком объектов `Observers`. При вызове функции `notifyObservers` инициируется вызов функции `onChange` для каждого зарегистрированного объекта `Observer` со старыми и новыми значениями свойств. От класса `Observer` требуется только предоставить реализацию метода `onChange`. Поэтому уместно использовать функциональный интерфейс, как было показано в главе 5:

```
fun interface Observer {  
    fun onChange(name: String, oldValue: Any?, newValue: Any?)  
}
```

```

open class Observable {
    val observers = mutableListOf<Observer>()
    fun notifyObservers(propName: String, oldValue: Any?, newValue: Any?) {
        for (obs in observers) {
            obs.onChange(propName, oldValue, newValue)
        }
    }
}

```

Теперь напомним класс `Person` (листинг 9.20). Необходимо определить доступное только для чтения свойство (имя человека, которое обычно не меняется) и два свойства, доступных для записи: возраст и зарплата. Класс будет уведомлять своих наблюдателей при изменении возраста или зарплаты человека.

Листинг 9.20. Реализация уведомлений наблюдателя об изменении свойств вручную

```

class Person(val name: String, age: Int, salary: Int): Observable() {
    var age: Int = age
    set(newValue) {
        val oldValue = field ← Идентификатор поля позволяет получить
                               доступ к теневого полю свойства
        field = newValue
        notifyObservers(
            "age", oldValue, newValue ← Уведомление наблюдателей
                                       об изменении свойства
        )
    }

    var salary: Int = salary
    set(newValue) {
        val oldValue = field
        field = newValue
        notifyObservers(
            "salary", oldValue, newValue
        )
    }
}

fun main() {
    val p = Person("Seb", 28, 1000)
    p.observers += Observer { propName, oldValue, newValue -> ←
        println(
            """
            Property $propName changed from $oldValue to $newValue!
            """.trimIndent()
        )
    }
    p.age = 29
    // Property age changed from 28 to 29!
    p.salary = 1500
    // Property salary changed from 1000 to 1500!
}

```

Обратите внимание на то, как этот код использует идентификатор `field` для доступа к теневого полю свойств `age` и `salary`, о чем мы говорили в главе 4.

В сеттерах довольно много повторяющегося кода. Попробуем создать класс для хранения значения свойства и отправки необходимых уведомлений (листинг 9.21).

Листинг 9.21. Использование уведомлений наблюдателя об изменении свойств с помощью дополнительного класса

```
class ObservableProperty(
    val propName: String,
    var propValue:
        Int,
    val observable: Observable
) {
    fun getValue(): Int = propValue
    fun setValue(newValue: Int) {
        val oldValue = propValue
        propValue = newValue
        observable.notifyObservers(propName, oldValue, newValue)
    }
}

class Person(val name: String, age: Int, salary: Int): Observable() {
    val _age = ObservableProperty("age", age, this)
    var age: Int
        get() = _age.getValue()
        set(newValue) {
            _age.setValue(newValue)
        }

    val _salary = ObservableProperty("salary", salary, this)
    var salary: Int
        get() = _salary.getValue()
        set(newValue) {
            _salary.setValue(newValue)
        }
}
```

Мы стали на шаг ближе к пониманию того, как работают делегированные свойства в Kotlin. Был создан класс для хранения свойства и автоматического уведомления наблюдателей о его изменении. Мы сократили продублированный код в логике. Но потребовалось довольно много повторений шаблона создания экземпляра `ObservableProperty` для каждого свойства и делегирования ему геттера и сеттера. Функциональная возможность делегированных свойств в Kotlin позволяет избавиться от этого шаблона. Но прежде, чем это сделать, необходимо изменить сигнатуры методов `ObservableProperty`, чтобы они соответствовали требованиям соглашений Kotlin (листинг 9.22).

Листинг 9.22. Класс `ObservableProperty` в качестве делегата свойств

```
import kotlin.reflect.KProperty

class ObservableProperty(var propValue: Int, val observable: Observable) {
    operator fun getValue(thisRef: Any?, prop: KProperty<*>): Int = propValue

    operator fun setValue(thisRef: Any?, prop: KProperty<*>, newValue: Int) {
        val oldValue = propValue
    }
}
```

```
        propValue = newValue
        observable.notifyObservers(prop.name, oldValue, newValue)
    }
}
```

По сравнению с предыдущей версией в этом коде есть изменения.

- Функции `getValue` и `setValue` получили директиву `operator`. Она требуется для всех функций, используемых через соглашения.
- В эти функции было добавлено два параметра: один — для получения экземпляра, для которого свойство получено или задано (`thisRef`), а второй — для представления самого свойства (`prop`). Оно представлено как объект типа `KProperty`. Мы рассмотрим этот момент подробнее в главе 12, а пока вам достаточно знать, что получить доступ к имени свойства можно в виде `KProperty.name`.
- Свойство `name` было удалено из первичного конструктора, поскольку теперь можно получить доступ к имени свойства через `KProperty`.

Наконец-то вы можете использовать возможности делегированных свойств Kotlin (листинг 9.23). Заметили, насколько короче становится код?

Листинг 9.23. Использование делегированных свойств для обеспечения наблюдаемости свойств

```
class Person(val name: String, age: Int, salary: Int) : Observable() {
    var age by ObservableProperty(age, this)
    var salary by ObservableProperty(salary, this)
}
```

При использовании ключевого слова `by` компилятор Kotlin автоматически выполняет то, что в предыдущей версии кода выполнялось вручную. Сравните этот код с предыдущей версией класса `Person`: сгенерированный код при использовании делегированных свойств очень похож. Объект справа от ключевого слова `by` называется *делегатом*. Kotlin автоматически сохраняет делегата в скрытом свойстве и вызывает для него методы `getValue` и `setValue` при обращении или изменении основного свойства.

Вместо того чтобы реализовывать логику наблюдаемых свойств вручную, можно воспользоваться стандартной библиотекой Kotlin. Оказывается, она уже содержит собственный класс `ObservableProperty`. Однако он ничего не знает об определенном ранее интерфейсе `Observable`. Поэтому необходимо передать ему лямбду, которая содержит указание о том, как сообщать об изменении значения свойства. В листинге 9.24 показано, как это можно сделать.

Листинг 9.24. Использование делегата `Delegates.observable` для уведомлений об изменении свойств

```
import kotlin.properties.Delegates

class Person(val name: String, age: Int, salary: Int) : Observable() {
    private val onChange = { property: KProperty<*>, oldValue: Any?,
        ➤ newValue: Any? ->
        notifyObservers(property.name, oldValue, newValue)
    }
}
```

```

var age by Delegates.observable(age, onChange)
var salary by Delegates.observable(salary, onChange)
}

```

Выражение справа от ключевого слова `by` не обязательно должно быть вновь созданным экземпляром. Это может быть вызов функции, другое свойство или любое другое выражение, если значение последнего будет объектом, для которого компилятор может вызвать методы `getValue` и `setValue` с правильными типами параметров. Как и в других соглашениях, `getValue` и `setValue` могут быть либо методами, объявленными в самом объекте, либо функциями-расширениями.

Обратите внимание: в целях упрощения примеров мы показали, как работать только с делегированными свойствами типа `Int`. Но этот механизм универсален и работает с любыми другими типами.

9.5.4. Делегированные свойства переводятся в скрытые свойства с пользовательскими методами доступа

Подытожим правила работы делегированных свойств. Предположим, у вас есть класс с делегированным свойством:

```

class C {
    var prop: Type by MyDelegate()
}

```

```
val c = C()
```

Экземпляр `MyDelegate` будет храниться в скрытом свойстве. Мы будем обращаться к нему как `<delegate>`. Вдобавок компилятор будет использовать объект типа `KProperty` для представления свойства. К нему мы будем обращаться как к `<property>`.

Компилятор генерирует следующий код:

```

class C {
    private val <delegate> = MyDelegate()

    var prop: Type
        get() = <delegate>.getValue(this, <property>)
        set(value: Type) = <delegate>.setValue(this, <property>, value)
}

```

Таким образом, внутри каждого метода доступа свойства компилятор генерирует вызовы соответствующих методов `getValue` и `setValue` (рис. 9.11).

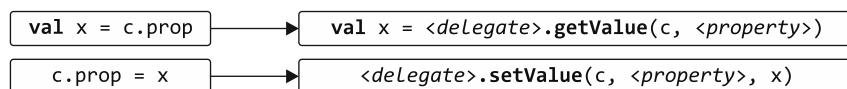


Рис. 9.11. При обращении к свойству функции `getValue` и `setValue` вызываются для экземпляра `<delegate>`

Механизм довольно прост, но позволяет реализовать множество интересных сценариев. Можно настроить, где будет храниться значение свойства (в карте, таблице базы данных или в куки-файлах пользовательской сессии), а также что будет происходить при обращении к свойству (можно добавлять валидацию, уведомления об изменениях и т. д.). Все это можно реализовать с помощью компактного кода.

Далее мы представим еще один способ применения делегированных свойств в стандартной библиотеке, а затем покажем, как вы можете использовать их в собственных фреймворках.

9.5.5. Доступ к динамическим атрибутам путем делегирования карт

Другой распространенный паттерн, в котором делегированные свойства играют важную роль, — это объекты с динамически определяемым набором связанных с ними атрибутов. (В других языках, например в С#, они называются *expand-объектами*.) Для примера рассмотрим систему управления контактами, в которой можно хранить произвольную информацию о записях.

У каждого человека в системе есть несколько необходимых свойств (например, имя), которые обрабатываются особым образом. Кроме того, у каждого человека есть некоторое количество дополнительных атрибутов. Они могут быть разными для каждой записи (например, день рождения младшего ребенка).

Один из способов реализации такой системы — хранение всех атрибутов человека в карте и предоставление свойств для доступа к информации, требующей специальной обработки. Приведем пример (листинг 9.25).

Листинг 9.25. Определение свойства, хранящего свое значение в карте

```
class Person {
    private val _attributes = mutableMapOf<String, String>()

    fun setAttribute(attrName: String, value: String) {
        _attributes[attrName] = value
    }

    var name: String
        get() = _attributes["name"]!!
        set(value) {
            _attributes["name"] = value
        }
}

fun main() {
    val p = Person()
    val data = mapOf("name" to "Seb", "company" to "JetBrains")
    for ((attrName, value) in data)
        p.setAttribute(attrName, value)
    println(p.name)
    // Seb
```

```

    p.name = "Sebastian"
    println(p.name)
    // Sebastian
}

```

Здесь используется общий API для загрузки данных в объект (в реальном проекте это может быть десериализация JSON или что-то подобное), а затем специфический API для доступа к значению одного свойства. Изменить этот код для использования делегированного свойства очень просто: можно поместить карту непосредственно после ключевого слова `by` (листинг 9.26).

Листинг 9.26. Использование делегированного свойства, хранящего свое значение в карте

```

class Person {
    private val _attributes = mutableMapOf<String, String>()

    fun setAttribute(attrName: String, value: String) {
        _attributes[attrName] = value
    }

    var name: String by _attributes
}

```

Использование карты
как делегированного свойства

Этот код работает потому, что стандартная библиотека определяет функции-расширения `getValue` и `setValue` на стандартных интерфейсах `Map` и `MutableMap`. Имя свойства автоматически используется в качестве ключа для хранения значения в карте. Как и в листинге 9.25, `p.name` скрывает вызов метода `_attributes.getValue(p, prop)`, который, в свою очередь, реализуется как `_attributes[prop.name]`.

9.5.6. Как использовать делегированные свойства во фреймворке

Менять способ хранения и модификации свойств объекта чрезвычайно полезно для разработчиков фреймворков. В этом подразделе показан пример того, как делегированные свойства улучшают разработку и использование фреймворка, и подробно описаны механизмы их работы.

Допустим, в базе данных хранится таблица `Users` с двумя столбцами: `name` строкового типа и `age` целочисленного типа. Можно определить классы `Users` и `User` в Kotlin. Затем в коде Kotlin с помощью экземпляров класса `User` можно загрузить и изменить все пользовательские сущности, хранящиеся в базе данных (листинг 9.27).

Объект `Users` описывает таблицу базы данных — он объявлен как объект, поскольку описывает таблицу в целом. Следовательно, требуется получить только один его экземпляр. Свойства объекта представляют собой столбцы таблицы.

Класс `Entity` — суперкласс класса `User` — содержит отображение столбцов базы данных на их значения для сущности. В свойствах конкретного объекта `User` хранятся значения `name` и `age`, указанные в базе данных для этого пользователя.

Листинг 9.27. Доступ к столбцам базы данных с помощью делегированных свойств

```
object Users : IdTable() {
    val name = varchar("name", length = 50).index()
    val age = integer("age")
}

class User(id: EntityID) : Entity(id) {
    var name: String by Users.name
    var age: Int by Users.age
}
```

← Объект соответствует таблице в базе данных

← Свойства соответствуют столбцам в этой таблице

← Каждый экземпляр класса User соответствует определенной сущности в таблице

← Для этого пользователя в базе данных хранится значение имени

Использовать фреймворк особенно удобно потому, что обращение к свойству автоматически извлекает соответствующее значение из отображения в классе `Entity`, а его изменение помечает объект как модифицированный, чтобы при необходимости его можно было сохранить в базе данных. Можно написать `user.age += 1` в коде на языке Kotlin, и соответствующая сущность в базе данных будет автоматически обновлена.

К текущему моменту вы получили достаточно знаний, чтобы понять, как реализовать фреймворк с таким API. Каждый из атрибутов сущности (`name`, `age`) реализован как делегированное свойство, использующее объект столбца (`Users.name`, `Users.age`) в качестве делегата:

```
class User(id: EntityID) : Entity(id) {
    var name: String by Users.name
    var age: Int by Users.age
}
```

← Объект `Users.name` — делегат для свойства `name`

Рассмотрим явно указанные типы столбцов:

```
object Users : IdTable() {
    val name: Column<String> = varchar("name", 50).index()
    val age: Column<Int> = integer("age")
}
```

Для класса `Column` фреймворк определяет методы `getValue` и `setValue`, что соответствует соглашению Kotlin для делегатов:

```
operator fun <T> Column<T>.getValue(o: Entity, desc: KProperty<*>): T
{
    // получение значения из базы данных
}
operator fun <T> Column<T>.setValue(o: Entity, desc: KProperty<*>, value: T)
{
    // обновление значения в базе данных
}
```

Свойство класса `Column` (`Users.name`) можно использовать в качестве делегата для делегированного свойства (`name`). Если написать выражение `user.age += 1` в коде, то программа выполнит нечто похожее на `user.ageDelegate.setValue(user.ageDelegate.getValue() + 1)` (опустив параметры для свойств и экземпляров объектов).

Методы `getValue` и `setValue` занимаются получением и обновлением информации в базе данных.

Полная реализация классов в этом примере находится в исходном коде фреймворка Exposed (<https://github.com/JetBrains/Exposed>). Мы вернемся к нему в главе 13, чтобы изучить используемые в ней методы проектирования DSL.

РЕЗЮМЕ

- Kotlin позволяет перегружать некоторые стандартные математические операции, определяя функции с соответствующими именами. Вы не можете определять собственные операторы, но можете использовать инфиксные функции в качестве более выразительной альтернативы.
- Операторы сравнения (`==`, `!=`, `>`, `<` и т. д.) можно использовать с любыми объектами. Они соответствуют вызовам методов `equals` и `compareTo`.
- Определение функций `get`, `set` и `contains` дает возможность поддерживать операторы `[]` и `in`, что позволяет сделать класс похожим на коллекции Kotlin.
- Создание диапазонов и итерация по коллекциям и массивам работают по соглашениям.
- Объявления деструктуризации позволяют инициализировать несколько переменных, распаковывая один объект, что удобно для возврата нескольких значений из функции. Они работают с классами данных автоматически. А для собственных классов вы можете их применять, определяя функции с именем `componentN`.
- Делегированные свойства позволяют повторно использовать логику, управляющую хранением и инициализацией свойств, доступом к ним и изменением их значений. Это мощный инструмент для создания фреймворков.
- Функция `lazy` стандартной библиотеки предоставляет простой способ реализовать отложено инициализируемые свойства.
- Функция `Delegates.observable` позволяет добавить объект-наблюдатель за изменениями свойств.
- Делегированные свойства могут использовать любую карту в качестве делегата свойства. Так вы получаете гибкий способ обработки объектов, имеющих переменные наборы атрибутов.

10

Функции высшего порядка: лямбды в качестве параметров и возвращаемых значений

В этой главе

- ✓ Типы функций.
- ✓ Функции высшего порядка и их использование в целях структурирования кода.
- ✓ Встроенные функции.
- ✓ Нелокальные выражения `return` и метки.
- ✓ Анонимные функции.

Мы рассмотрели общую концепцию лямбда-выражений в главе 5, а более углубленное описание функций стандартной библиотеки, использующих лямбды, — в главе 6. Лямбды — отличный инструмент создания абстракций, и их возможности не ограничиваются коллекциями и другими классами стандартной библиотеки. В текущей главе вы узнаете, как создавать *функции высшего порядка* — ваши собственные функции. Они будут принимать лямбда-выражения в качестве аргументов или возвращать их. Мы приведем примеры того, как упростить код, устранить дублирование и создать красивые абстракции с помощью функций высшего порядка. Рассмотрим *встроенные функции* — мощную функциональную возможность Kotlin.

10.1. ОБЪЯВЛЕНИЕ ФУНКЦИЙ, ВОЗВРАЩАЮЩИХ ИЛИ ПОЛУЧАЮЩИХ ДРУГИЕ ФУНКЦИИ: ФУНКЦИИ ВЫСШЕГО ПОРЯДКА

Ключевой идеей этой главы будет концепция *функций высшего порядка*. Согласно определению, функция высшего порядка — это функция, которая принимает в качестве аргумента другую функцию или возвращает ее. В Kotlin функции можно представить в виде значений с помощью лямбд или ссылок на функции. Следовательно, функции высшего порядка можно передать в качестве аргумента лямбду или ссылку на функцию, либо она будет возвращать обе или одну из этих сущностей. Например, стандартная библиотечная функция `filter` принимает в качестве аргумента предикатную функцию. Следовательно, она представляет собой функцию высшего порядка:

```
list.filter { x > 0 }
```

В главе 6 вы познакомились с другими функциями высшего порядка из стандартной библиотеки Kotlin: `map`, `with` и т. д. А здесь мы поговорим о том, как объявлять такие функции в собственном коде. Для этого необходимо сначала изучить *типы функций*.

10.1.1. Типы функций определяют типы параметров и возвращаемых значений лямбды

Чтобы объявить функцию, принимающую в качестве аргумента лямбду, нужно знать, как объявить тип соответствующего параметра. Но сначала рассмотрим более простой случай и сохраним лямбду в локальной переменной. Вы уже видели, как это можно сделать без объявления типа, полагаясь на выводение типа в Kotlin:

```
val sum = { x: Int, y: Int -> x + y }
val action = { println(42) }
```

В данном случае компилятор делает вывод, что переменные `sum` и `action` получают типы функций (среда IDE поможет визуализировать это (рис. 10.1)).

```
val sum: (Int, Int) -> Int = { x: Int, y: Int -> x + y }
val action: () -> Unit = { println(42) }
```

Рис. 10.1. Дополнительные подсказки в IntelliJ IDEA и Android Studio помогают визуализировать предполагаемые типы функций лямбд, таких как `sum` и `action`

Теперь посмотрим, как выглядит явное объявление типа этих переменных:

```
val sum: (Int, Int) -> Int = { x, y -> x + y }
val action: () -> Unit = { println(42) }
```

← Эта функция принимает два параметра типа `Int` и возвращает значение типа `Int`

← Эта функция не принимает аргументы и не возвращает значения

Чтобы объявить тип функции, необходимо заключить типы параметров функции в круглые скобки, за которыми следует стрелка и возвращаемый тип функции (рис. 10.2).

В главе 8 мы обсуждали, что с помощью типа `Unit` можно указать, что функция не возвращает никакого значимого значения. Тип возвращаемого значения `Unit` можно опустить при объявлении обычной функции. Но объявление типа функции всегда требует явного указания типа возврата, поэтому в данном контексте нельзя опускать `Unit`.

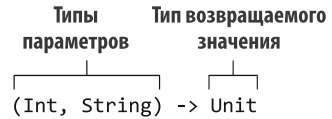


Рис. 10.2. Синтаксис типа функции в Kotlin

Обратите внимание, что типы параметров `x`, `y` можно не указывать в лямбда-выражении `{ x, y -> x + y }`. Они указываются в типе функции как часть объявления переменной, поэтому нет нужды повторять их в самой лямбде.

Как и любая другая функция, возвращаемый тип функции может быть помечен как допускающий значение `null`:

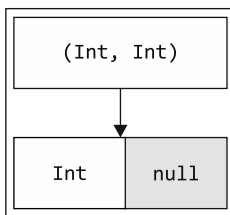
```
var canReturnNull: (Int, Int) -> Int? = { x, y -> null }
```

Кроме того, можно определять переменную типа функции, допускающую значение `null`. Чтобы указать, что именно переменная, а не возвращаемый тип функции относится к типу `nullable`, нужно заключить все определение типа функции в круглые скобки и поставить знак вопроса после закрывающей скобки:

```
var funOrNull: ((Int, Int) -> Int)? = null
```

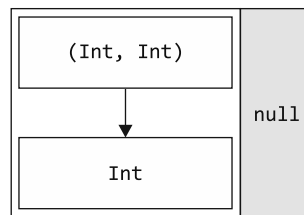
Обратите внимание на тонкое различие между этим примером и предыдущим. Если опустить круглые скобки, будет объявлен тип функции с `nullable`-типом возвращаемого значения. Но не переменная типа функции, допускающая значение `null`. Это показано на рис. 10.3.

Функция, которая принимает два целых числа и возвращает целое число, допускающее значение `null`



`(Int, Int) -> Int?`

Функция, допускающая значение `null`, которая принимает два целых числа и возвращает целое число типа «не `null`»



`((Int, Int) -> Int)?`

Рис. 10.3. Скобки определяют, будет ли у типа функции возвращаемый тип, допускающий значение `null`, или сам тип функции допускает это значение

10.1.2. Вызов функций, переданных в качестве аргументов

Вы узнали, как в Kotlin указать функциональный тип в локальной переменной. Теперь обсудим, как реализовать функцию высшего порядка. Первый пример максимально прост и использует то же объявление типов, что и упоминавшаяся ранее лямбда `sum`. Функция выполняет произвольную операцию с двумя числами, 2 и 3, и выводит результат (листинг 10.1).

Листинг 10.1. Определение простой функции высшего порядка

```
fun twoAndThree(operation: (Int, Int) -> Int) {
    val result = operation(2, 3)
    println("The result is $result")
}

fun main() {
    twoAndThree { a, b -> a + b }
    // The result is 5
    twoAndThree { a, b -> a * b }
    // The result is 6
}
```

Объявление параметра типа функции

Вызов параметра типа функции

Синтаксис вызова функции, переданной в качестве аргумента, такой же, как и при вызове обычной функции: после имени функции ставятся круглые скобки, а внутри скобок — параметры.

ИМЕНА ПАРАМЕТРОВ ТИПОВ ФУНКЦИЙ

Для параметров типа функции можно задавать имена:

```
fun twoAndThree(
    operation: (operandA: Int, operandB: Int) -> Int
) {
    val result = operation(2, 3)
    println("The result is $result")
}

fun main() {
    twoAndThree { operandA, operandB -> operandA + operandB }
    // The result is 5
    twoAndThree { alpha, beta -> alpha + beta }
    // The result is 5
}
```

У типа функции теперь именованные параметры

В качестве имен аргументов лямбды можно использовать имена, предоставленные в API...

...или их можно изменить

Имена параметров не влияют на сопоставление типов. При объявлении лямбды не обязательно использовать те же имена параметров, что и при объявлении типа функции. Однако имена улучшают читаемость кода и могут быть использованы в IDE для завершения кода.

В качестве более интересного примера переделаем одну из наиболее часто используемых функций стандартной библиотеки — `filter`. Мы уже использовали

СОВЕТ ДЛЯ INTELLIJ IDEA

Среда IntelliJ IDEA поддерживает интеллектуальный переход к лямбда-коду в отладчике. При анализе предыдущего примера можно заметить, как выполнение перемещается между телом функции `filter` и лямбдой, передаваемой через нее, по мере того как функция обрабатывает каждый элемент входного списка.

10.1.3. Лямбды Java автоматически преобразуются в типы функций Kotlin

В главе 5 мы обсудили, что лямбду можно передать любому методу Java, ожидающему функциональный интерфейс, благодаря автоматическому преобразованию SAM. Это означает, что код на языке Kotlin может использовать библиотеки Java и без проблем вызывать функции высшего порядка, определенные на Java. Аналогично функции Kotlin, использующие типы функций, можно легко вызывать из Java. Лямбды из Java автоматически преобразуются в значения типов функций, что показано в листинге 10.3.

Листинг 10.3. Исходный файл `ProcessTheAnswer.kt`

```
/* Объявление Kotlin */
fun processTheAnswer(f: (Int) -> Int) {
    println(f(42))
}

/* Вызов в Java */
processTheAnswer(number -> number + 1);
// 43
```

В Java можно использовать функции-расширения из стандартной библиотеки Kotlin — те, которые ожидают лямбды в качестве аргументов. Но обратите внимание, что они выглядят не так красиво, как в Kotlin, — придется передавать объект-приемник в качестве первого аргумента в явном виде:

```
/* Java */
import kotlin.collections.CollectionsKt;

// ...
public static void main(String[] args) {
    List<String> strings = new ArrayList();
    strings.add("42");
    CollectionsKt.forEach(strings, s -> {
        System.out.println(s);
        return Unit.INSTANCE;
    });
}
```

Можно использовать функцию из стандартной библиотеки Kotlin в коде Java

Вы должны явно вернуть значение типа Unit

Ваша функция или лямбда в Java смогут возвращать тип `Unit`. Но в Kotlin у типа `Unit` есть значение, поэтому возвращать его необходимо в явном виде. Нельзя передать лямбду с возвращаемым типом `void` в качестве аргумента функции, возвращающей тип `Unit`, как `(String) -> Unit` в предыдущем примере.

ТИПЫ ФУНКЦИЙ. ПОДРОБНОСТИ РЕАЛИЗАЦИИ

Внутри системы типы функций Kotlin — это обычные интерфейсы: переменная типа функции представляет собой реализацию интерфейса `FunctionN`. Список интерфейсов по количеству аргументов функции: `Function0<R>` (эта функция не принимает никаких аргументов и определяет только тип возврата), `Function1<P1, R>` (эта функция принимает один аргумент) и т. д. Каждый интерфейс определяет один метод `invoke`, и его вызов приводит к выполнению функции. (Более подробно оператор `invoke` мы рассмотрим в главе 13.) Вкратце интерфейсы `FunctionN` выглядят следующим образом:

```
interface Function1<P1, out R> {
    operator fun invoke(p1: P1): R
}
```

Переменная типа функции — это экземпляр класса с реализацией соответствующего интерфейса `FunctionN`. Он содержит метод `invoke`, хранящий тело лямбды. Внутри системы это означает, что листинг 10.3 выглядит примерно так:

```
fun processTheAnswer(f: Function1<Int, Int>) {
    println(f.invoke(42))
}
```

Эти функциональные типы на самом деле представляют собой просто интерфейсы Kotlin, так что их можно использовать везде, где разрешено применение интерфейса. Например, класс может наследоваться от интерфейса `FunctionN` или эквивалентного ему функционального типа (хотя на практике это используется редко):

```
class Adder : (Int, Int) -> Int {
    override operator fun invoke(
        p1: Int,
        p2: Int
    ): Int {
        return p1 + p2
    }
}
```

← Эквивалентно выражению `Function2<Int, Int, Int>`

Интерфейсы `FunctionN` относятся к *синтетическим типам, генерируемым компилятором*. То есть найти их объявления в стандартной библиотеке Kotlin невозможно. Компилятор генерирует их по мере необходимости. Это дает возможность использовать интерфейс для функции, имеющей любое количество параметров, не сталкиваясь с искусственными ограничениями на возможное количество типовых параметров функции.

10.1.4. Параметры с типами функций могут задавать значения по умолчанию или допускать значение `null`

При объявлении параметра типа функции можно указать его значение по умолчанию. Чтобы увидеть, где это может быть полезно, вернемся к функции `joinToString`, описанной в главе 3. Тогда мы получили реализацию следующего вида (листинг 10.4).

Листинг 10.4. Функция `joinToString` с жестко закодированным соглашением `toString`

```

fun <T> Collection<T>.joinToString(
    separator: String = ", ",
    prefix: String = "",
    postfix: String = ""
): String {
    val result = StringBuilder(prefix)

    for ((index, element) in this.withIndex()) {
        if (index > 0) result.append(separator)
        result.append(element)
    }

    result.append(postfix)
    return result.toString()
}

```

← Преобразование объекта
в строку с помощью метода
по умолчанию `toString`

Эта реализация достаточно гибкая, но не позволяет контролировать один ключевой аспект преобразования — то, как отдельные значения в коллекции преобразуются в строки. Код использует выражение `StringBuilder.append(o: Any?)`, и оно всегда преобразует объект в строку с помощью метода `toString`. Такой подход хорошо работает во многих случаях, но не всегда. Известно, что можно передать лямбду и с ее помощью указать, как значения преобразуются в строки. Но требовать передавать эту лямбду при каждом вызове было бы обременительно, поскольку большинству из точек вызова достаточно поведения по умолчанию. Для решения этой проблемы можно определить параметр типа функции и указать для него значение по умолчанию в виде лямбды (листинг 10.5).

Листинг 10.5. Указание значения по умолчанию для параметра типа функции

```

fun <T> Collection<T>.joinToString(
    separator: String = ", ",
    prefix: String = "",
    postfix: String = "",
    transform: (T) -> String = { it.toString() }
): String {
    val result = StringBuilder(prefix)

    for ((index, element) in this.withIndex()) {
        if (index > 0) result.append(separator)
        result.append(transform(element))
    }

    result.append(postfix)
    return result.toString()
}

fun main() {
    val letters = listOf("Alpha", "Beta")
    println(letters.joinToString())
    // Alpha, Beta
    println(letters.joinToString { it.lowercase() })
    // alpha, beta
}

```

← Объявление параметра типа
функции с лямбдой в качестве
значения по умолчанию

← Вызов функции, переданной
в качестве аргумента
для параметра `transform`

← Использование функции
преобразования
по умолчанию

← Передача лямбды
в качестве аргумента

```
println(letters.joinToString(separator = "! ", postfix = "! ",
transform = { it.uppercase() }))
// ALPHA! BETA!
```

Использование синтаксиса именованных аргументов для передачи нескольких аргументов, включая лямбду

Обратите внимание, что это обобщенная функция: у нее есть параметр типа `T`, обозначающий тип элемента в коллекции. Лямбда `transform` будет принимать аргумент данного типа.

Объявление значения по умолчанию для типа функции не требует специального синтаксиса: значение просто помещается в виде лямбды после знака `=`. В листинге 10.5 были показаны различные способы вызова функции: полное отсутствие лямбды (в этом случае используется стандартное преобразование `toString()`), передача ее вне круглых скобок (поскольку она стала последним аргументом функции `joinToString`) и передача ее в качестве именованного аргумента.

Альтернативный подход заключается в объявлении параметра типа функции, допускающего значение `null`. Обратите внимание, что вызвать напрямую функцию, переданную в таком параметре, невозможно: Kotlin откажется компилировать такой код, поскольку обнаружит возможность возникновения в этом случае исключения `null-указателя`. В качестве альтернативы можно выполнить проверку `null` в явном виде:

```
fun foo(callback: (() -> Unit)?) {
    // ...
    if (callback != null) {
        callback()
    }
}
```

Более короткая версия использует тот факт, что функциональный тип — это реализация интерфейса с методом `invoke`. Его, как и обычный метод, можно вызвать с помощью синтаксиса безопасного вызова: `callback?.invoke()`. В листинге 10.6 показано, как с помощью этого приема переписать функцию `joinToString`.

Листинг 10.6. Использование параметра типа функции, допускающего значение `null`

```
fun <T> Collection<T>.joinToString(
    separator: String = ", ",
    prefix: String = "",
    postfix: String = "",
    transform: ((T) -> String)? = null
): String {
    val result = StringBuilder(prefix)
    for ((index, element) in this.withIndex()) {
        if (index > 0) result.append(separator)
        val str = transform?.invoke(element)
        result.append(str)
    }

    result.append(postfix)
    return result.toString()
}
```

Объявление параметра типа функции, допускающего значение `null`

Использование синтаксиса безопасного вызова функции

Использование оператора `?:` для обработки случая, когда обратный вызов не был указан

Этот пример также служит напоминанием о синтаксисе функциональных типов, который был представлен на рис. 10.2: `transform` — параметр типа функции, допускающий значение `null`, но его возвращаемый тип не допускает это значение. Если параметр `transform` не равен `null`, то от него гарантированно вернется «не `null`»-значение типа `String`.

Мы выяснили, как писать функции, принимающие функции в качестве аргументов. Далее рассмотрим другой вид функций высшего порядка — функции, возвращающие другие функции.

10.1.5. Возврат функций из функций

Требование вернуть функцию из другой функции встречается не так часто, как передача функций другим функциям, но все же этот механизм может быть полезен. Представьте логику в программе, которая может меняться в зависимости от состояния программы или других условий. Например, стоимость доставки нужно вычислять в зависимости от выбранного способа доставки. Можно определить функцию для выбора подходящего варианта логики и возврата его в виде другой функции. В листинге 10.7 показано, как это выглядит в виде кода.

Листинг 10.7. Определение функции, возвращающей другую функцию

```
enum class Delivery { STANDARD, EXPEDITED }

class Order(val itemCount: Int)

fun getShippingCostCalculator(delivery: Delivery): (Order) -> Double {
    if (delivery == Delivery.EXPEDITED) {
        return { order -> 6 + 2.1 * order.itemCount }
    }

    return { order -> 1.2 * order.itemCount }
}

fun main() {
    val calculator = getShippingCostCalculator(Delivery.EXPEDITED)
    println("Shipping costs ${calculator(Order(3))}")
    // Shipping costs 12.3
}
```

Чтобы объявить функцию, возвращающую другую функцию, необходимо указать тип функции в качестве возвращаемого типа. В листинге 10.7 функция `getShippingCostCalculator` возвращает функцию, которая принимает экземпляр `Order` и возвращает тип `Double`. Для возврата функции необходимо написать выражение `return`, за которым следует лямбда, ссылка на член класса или другое выражение типа функции, например локальная переменная.

Рассмотрим еще один пример полезного применения этого механизма. Предположим, вы работаете над приложением для управления контактами, имеющим графический интерфейс. Вам нужно определить, какие контакты должны быть

отображены, основываясь на состоянии пользовательского интерфейса. Допустим, он позволяет ввести строку, а затем показать только контакты с именами, начинающимися с этой строки. Вдобавок он скрывает контакты без номера телефона. Для хранения состояния вариантов будет использоваться класс `ContactListFilters`:

```
class ContactListFilters {
    var prefix: String = ""
    var onlyWithPhoneNumber: Boolean = false
}
```

Когда пользователь набирает букву D, чтобы увидеть контакты, имя или фамилия которых начинаются с нее, значение `prefix` обновляется. Мы опустили код для осуществления необходимых изменений. (Код полноценного приложения с пользовательским интерфейсом был бы слишком объемным для данной книги, поэтому мы приводим упрощенный пример.)

Чтобы отделить логику отображения списка контактов от пользовательского интерфейса фильтрации, можно определить функцию для создания предиката. Он будет использоваться для фильтрации списка контактов, как показано в листинге 10.8. Этот предикат проверяет префикс и наличие телефонного номера при необходимости.

Листинг 10.8. Использование функций, возвращающих функции, в коде пользовательского интерфейса

```
data class Person(
    val firstName: String,
    val lastName: String,
    val phoneNumber: String?
)

class ContactListFilters {
    var prefix: String = ""
    var onlyWithPhoneNumber: Boolean = false

    fun getPredicate(): (Person) -> Boolean {
        val startsWithPrefix = { p: Person ->
            p.firstName.startsWith(prefix) || p.lastName.startsWith(prefix)
        }
        if (!onlyWithPhoneNumber) {
            return startsWithPrefix
        }
        return { startsWithPrefix(it)
            && it.phoneNumber != null }
    }
}

fun main() {
    val contacts = listOf(
        Person("Dmitry", "Jemerov", "123-4567"),
        Person("Svetlana", "Isakova", null)
    )
}
```

Объявление функции, возвращающей другую функцию

Возврат переменной типа функции

Из этой функции возвращается лямбда

```

val contactListFilters = ContactListFilters()
with (contactListFilters) {
    prefix = "Dm"
    onlyWithPhoneNumber = true
}
println(
    contacts.filter(contactListFilters.getPredicate())
)
// [Person(firstName=Dmitry, lastName=Jemerov, phoneNumber=123-4567)]
}

```

Передача функции, возвращенной методом `getPredicate` в качестве аргумента для функции `filter`

Метод `getPredicate` возвращает значение функции, и оно передается в функцию `filter` в качестве аргумента. Типы функций Kotlin позволяют делать это так же легко, как и для значений других типов, например строк.

Функции высшего порядка — чрезвычайно мощный инструмент улучшения структуры кода и устранения дублирования.

Далее посмотрим, как лямбды помогают извлечь повторяющуюся логику из вашего кода.

10.1.6. Повышение удобства использования кода за счет сокращения дублирования с помощью лямбд

Сочетание типов функций и лямбда-выражений — отличный инструмент создания многократно используемого кода. Раньше многие виды дублирования кода можно было исключить только с помощью громоздких конструкций. А теперь это можно реализовать, используя лаконичные лямбда-выражения.

Рассмотрим пример с анализом посещений сайта (листинг 10.9). Класс `SiteVisit` хранит путь каждого посещения, его продолжительность и ОС пользователя. Различные ОС представлены в виде перечисления.

Листинг 10.9. Определение данных о посещении сайта

```

data class SiteVisit(
    val path: String,
    val duration: Double,
    val os: OS
)

enum class OS { WINDOWS, LINUX, MAC, IOS, ANDROID }

val log = listOf(
    SiteVisit("/", 34.0, OS.WINDOWS),
    SiteVisit("/", 22.0, OS.MAC),
    SiteVisit("/login", 12.0, OS.WINDOWS),
    SiteVisit("/signup", 8.0, OS.IOS),
    SiteVisit("/", 16.3, OS.ANDROID)
)

```

Представьте, что требуется отобразить среднюю продолжительность посещений с машин Windows. Эту задачу можно выполнить с помощью функции `average` (листинг 10.10).

Листинг 10.10. Анализ данных о посещении сайта с жестко закодированными фильтрами

```
val averageWindowsDuration = log
    .filter { it.os == OS.WINDOWS }
    .map(SiteVisit::duration)
    .average()

fun main() {
    println(averageWindowsDuration)
    // 23.0
}
```

Теперь предположим, что нужно рассчитать ту же статистику для пользователей Mac. Чтобы избежать дублирования, можно извлечь платформу в качестве параметра (листинг 10.11).

Листинг 10.11. Удаление дублирования с помощью обычной функции

```
fun List<SiteVisit>.averageDurationFor(os: OS) =
    filter { it.os == os }.map(SiteVisit::duration).average()

fun main() {
    println(log.averageDurationFor(OS.WINDOWS))
    // 23.0
    println(log.averageDurationFor(OS.MAC))
    // 22.0
}
```

← Продублированный
код извлекается
в функцию

Обратите внимание, что превращение этой функции в расширение улучшает удобочитаемость. Можно даже объявить данную функцию как локальную функцию-расширение, если она будет использоваться только в локальном контексте.

Но это не очень эффективное решение. Представьте, что вас интересует средняя продолжительность посещений с мобильных платформ (на данный момент их две: iOS и Android) (листинг 10.12).

Листинг 10.12. Анализ данных о посещении сайта с помощью сложного жестко закодированного фильтра

```
fun main() {
    val averageMobileDuration = log
        .filter { it.os in setOf(OS.IOS, OS.ANDROID) }
        .map(SiteVisit::duration)
        .average()
    println(averageMobileDuration)
    // 12.15
}
```

Теперь простой параметр, представляющий платформу, не справится с этой задачей. Вполне вероятно, что потребуются запросить журнал, используя более сложные условия, например: «Какова средняя продолжительность посещения страницы регистрации с iOS?». В этой ситуации помогут лямбды. Можно использовать типы функций для извлечения требуемого условия в параметр (листинг 10.13).

Листинг 10.13. Удаление дублирования с помощью функции высшего порядка

```

fun List<SiteVisit>.averageDurationFor(predicate: (SiteVisit) -> Boolean) =
    filter(predicate).map(SiteVisit::duration).average()

fun main() {
    println(
        log.averageDurationFor {
            it.os in setOf(OS.ANDROID, OS.IOS)
        }
    )
    // 12.15
    println(
        log.averageDurationFor {
            it.os == OS.IOS && it.path == "/signup"
        }
    )
    // 8.0
}

```

Типы функций помогают устранить дублирование кода. Если у вас возникнет соблазн скопировать и вставить часть кода, то, скорее всего, можно обойтись без дублирования. С помощью лямбд можно извлекать не только повторяющиеся данные, но и поведение.

ПРИМЕЧАНИЕ

Некоторые известные паттерны проектирования можно упростить, используя типы функций и лямбда-выражения. Рассмотрим, например, паттерн под названием «Стратегия». Если лямбда-выражения не используются, то он требует объявления интерфейса с несколькими реализациями для каждой возможной стратегии. Если в используемом языке есть функциональные типы, то можно использовать общий тип функции для описания стратегии и передавать различные лямбда-выражения в качестве различных стратегий.

Мы обсудили, как создавать функции высшего порядка. Далее поговорим об их производительности. Не станет ли код работать медленнее, если вы начнете использовать функции высшего порядка для всего, вместо того чтобы писать привычные циклы и условия? В следующем разделе мы обсудим, почему это не всегда справедливо и как ключевое слово `inline` может помочь.

10.2. УСТРАНЕНИЕ НАКЛАДНЫХ РАСХОДОВ НА ЛЯМБДЫ С ПОМОЩЬЮ ВСТРОЕННЫХ ФУНКЦИЙ

Сокращенный синтаксис передачи лямбды в качестве аргумента функции в Kotlin похож на синтаксис обычных инструкций, таких как `if` и `for`. Мы показали его во время обсуждения функций `with` и `apply` в главе 5. Но как же производительность? Не создаем ли мы неприятные сюрпризы, определяя более медленные функции, похожие на операторы Java?

В главе 5 мы объяснили, что лямбды обычно компилируются в анонимные классы. Но это означает, что при каждом использовании лямбда-выражения создается

дополнительный класс. А если лямбда захватывает некоторые переменные, то при каждом вызове создается новый объект. Это приводит к накладным расходам во время выполнения, в результате чего реализация с использованием лямбды будет менее эффективна, чем функция, выполняющая тот же код напрямую.

Можно ли дать компилятору указание генерировать код, столь же эффективный, как и прямое выполнение кода? И при этом он должен позволять извлекать повторяющуюся логику в библиотечную функцию. Безусловно, компилятор Kotlin позволяет это сделать. Если пометить функцию модификатором `inline`, то компилятор не будет генерировать вызов функции при ее использовании. Вместо этого он заменит каждое обращение к ней на фактический код для реализации данной функции. Давайте более детально разберемся в этом механизме и рассмотрим несколько примеров.

10.2.1. Встраивание означает подстановку тела функции в каждую точку вызова

Когда функция объявляется с модификатором `inline`, ее тело встраивается. Другими словами, оно подставляется непосредственно в точки вызова функции вместо обычного вызова. Рассмотрим пример, чтобы разобраться в итоговом коде.

Функцию из листинга 10.14 можно использовать для предотвращения одновременных обращений несколькими потоками к общему ресурсу. Она блокирует объект `Lock`, выполняет заданный блок кода, а затем снимает блокировку.

Листинг 10.14. Определение встроенной функции

```
import java.util.concurrent.locks.Lock
import java.util.concurrent.locks.ReentrantLock

inline fun <T> synchronized(lock: Lock, action: () -> T): T {
    lock.lock()
    try {
        return action()
    }
    finally {
        lock.unlock()
    }
}

fun main() {
    val l = ReentrantLock()
    synchronized(l) {
        // ...
    }
}
```

Тело функции, помеченной как `inline`, подставляется в точки вызова этой функции

Синтаксис вызова этой функции выглядит точно так же, как использование оператора `synchronized` в Java. Разница в том, что этот оператор Java можно использовать с любым объектом, в то время как приведенная функция требует передачи экземпляра `Lock`. Приведенное здесь определение — лишь пример. В стандартной библиотеке Kotlin определена реализация оператора `synchronized`, и она принимает любой объект в качестве аргумента.

Но использование явных блокировок для синхронизации позволяет получить более надежный и удобный в сопровождении код. В подразделе 10.2.5 будет представлена функция `withLock` из стандартной библиотеки Kotlin. Для выполнения заданного действия под блокировкой следует использовать именно ее.

Функция `synchronized` была объявлена как `inline`, поэтому генерируемый для каждого ее вызова код соответствует коду оператора `synchronized` в Java. Рассмотрим следующий пример использования функции `synchronized()`:

```
fun foo(l: Lock) {
    println("Before sync")
    synchronized(l) {
        println("Action")
    }
    println("After sync")
}
```

На рис. 10.5 показан эквивалентный код, который будет скомпилирован в тот же байт-код.

```
fun __foo__(l: Lock) {
    println("Before sync")
    l.lock()
    try {
        println("Action")
    } finally {
        l.unlock()
    }
    println("After sync")
}
```

Рис. 10.5. Скомпилированная версия функции `foo`

Обратите внимание, что встраивание применяется как к лямбда-выражению, так и к реализации функции `synchronized`. Байт-код, генерируемый лямбдой, становится частью определения вызывающей функции и не оборачивается в анонимный класс для реализации интерфейса функции.

Кроме того, можно вызвать встроенную функцию и передать параметр типа функции из переменной:

```
class LockOwner(val lock: Lock) {
    fun runUnderLock(body: () -> Unit) {
        synchronized(lock, body)
    }
}
```

В качестве аргумента передается переменная типа функции, а не лямбда

В этом случае код лямбды недоступен в точке вызова встроенной функции. Поэтому встраивается только тело функции `synchronized`, а лямбда вызывается как обычно.

Функция `runUnderLock` будет скомпилирована в байт-код, подобный следующей функции:

```
class LockOwner(val lock: Lock) {
    fun __runUnderLock__(body: () -> Unit) {
        lock.lock()
        try {
            body()
        } finally {
            lock.unlock()
        }
    }
}
```

Эта функция похожа на байт-код, в который компилируется настоящая функция `runUnderLock`

Тело не вставляется, поскольку в вызове нет лямбды

Если в программе две точки применения встроенной функции в разных местах с разными лямбдами, то в каждой точке встраивание будет выполняться независимо. Код встроенной функции будет скопирован в оба места использования с подстановкой в него разных лямбд.

Кроме того, ключевым словом `inline` можно пометить методы доступа к свойствам (`get`, `set`). Это становится полезным при использовании *вещественных* (*reified*) *обобщений* Kotlin. Примеры и подробности мы рассмотрим в главе 11.

10.2.2. Ограничения для встроенных функций

Из-за способа встраивания этот механизм можно применить не к каждой функции с лямбда-выражениями. При встраивании функции тело лямбда-выражения, переданного в качестве аргумента, подставляется непосредственно в итоговый код. Из-за этого возможные варианты использования соответствующего параметра в теле функции ограничиваются. Если вызывается параметр лямбды, то такой код можно встраивать без труда. Но если параметр хранится где-то для дальнейшего использования, то код лямбда-выражения нельзя встраивать, поскольку должен существовать объект, содержащий этот код:

```
class FunctionStorage {
    var myStoredFunction: ((Int) -> Unit)? = null
    inline fun storeFunction(f: (Int) -> Unit) {
        myStoredFunction = f
    }
}
```

Сохранение переданного параметра. Это означает, что компилятор не может подставлять код в каждом месте вызова, и сообщает о неверном использовании встроенного параметра `f`

Как правило, параметр можно встроить, если он вызывается напрямую или передается в качестве аргумента другой функции с директивой `inline`. В противном случае компилятор запретит встраивание параметра, выдав следующее сообщение об ошибке: `Illegal usage of inline-parameter` (Незаконное использование встроенного параметра).

Например, различные функции для работы с последовательностями возвращают экземпляры классов, которые представляют соответствующие операции

с последовательностями и получают лямбду в качестве параметра конструктора. Функция `Sequence.map` определяется следующим образом:

```
fun <T, R> Sequence<T>.map(transform: (T) -> R): Sequence<R> {
    return TransformingSequence(this, transform)
}
```

Функция `map` не вызывает функцию, переданную в качестве параметра `transform`, напрямую. Она передает эту функцию в конструктор класса, и он хранит ее в свойстве. Для этого лямбду, передаваемую в качестве аргумента `transform`, необходимо скомпилировать в стандартное невстраиваемое представление как анонимный класс для реализации интерфейса функции.

Если в программе есть функция, принимающая в качестве аргументов две или несколько лямбд, то можно выбрать встраивание только некоторых из них. В этом есть смысл, когда ожидается, что одна из лямбд будет содержать много кода. Или она используется таким образом, что встраивание недопустимо. Параметры, принимающие такие неиндексируемые лямбды, помечаются с помощью модификатора `noinline`:

```
inline fun foo(inlined: () -> Unit, noinline notInlined: () -> Unit) {
    // ...
}
```

Обратите внимание, что компилятор полностью поддерживает встраивание функций из модулей или функций, определенных в сторонних библиотеках. Большинство встроенных функций можно вызвать из кода Java. Такие вызовы не встраиваются, а будут скомпилированы как вызовы обычных функций. В главе 11 приведен еще один случай обоснованного применения модификатора `noinline` (правда, с некоторыми ограничениями, касающимися совместимости с Java).

10.2.3. Встраивание операций с коллекциями

Рассмотрим производительность функций для работы с коллекциями из стандартной библиотеки Kotlin. Большинство этих функций принимают в качестве аргументов лямбда-выражения. Будет ли эффективнее реализовать эти операции напрямую, а не использовать функции стандартной библиотеки? Для примера сравним способы фильтрации списка людей, показанные в листингах 10.15 и 10.16.

Листинг 10.15. Фильтрация коллекции с помощью лямбды

```
data class Person(val name: String, val age: Int)

val people = listOf(Person("Alice", 29), Person("Bob", 31))

fun main() {
    println(people.filter { it.age < 30 })
    // [Person(name=Alice, age=29)]
}
```

Приведенный выше код можно переписать без лямбда-выражений (листинг 10.16).

Листинг 10.16. Фильтрация коллекции вручную

```
fun main() {
    val result = mutableListOf<Person>()
    for (person in people) {
        if (person.age < 30) result.add(person)
    }
    println(result)
    // [Person(name=Alice, age=29)]
}
```

Функция `filter` в Kotlin объявляется как встроенная. Это означает, что байт-код функции `filter` вместе с байт-кодом переданной ей лямбды будет встроен в точку вызова `filter`. В результате байт-код, сгенерированный для первой версии с функцией `filter`, будет очень похож на байт-код для второй версии. Можно с уверенностью использовать идиоматические операции с коллекциями, а поддержка встроенных функций в Kotlin позволяет не беспокоиться о производительности.

Представьте, что в программе применяется цепочка вызовов двух операций — `filter` и `map`:

```
fun main() {
    println(
        people.filter { it.age > 30 }
            .map(Person::name)
    )
    // [Bob]
}
```

В этом примере используется лямбда-выражение и ссылка на член. Опять же, функции `filter` и `map` объявлены с модификатором `inline`. Поэтому их тела встраиваются, и не создаются никакие дополнительные классы или объекты. Однако код создает промежуточную коллекцию, в которой будут храниться результаты фильтрации списка. Код, генерируемый из функций `filter` и `map`, соответственно добавляет элементы в эту коллекцию и считывает их из нее.

Если количество обрабатываемых элементов велико и накладные расходы на промежуточную коллекцию могут стать значительными, то можно использовать последовательность, добавив в цепочку вызов метода `asSequence`. Последовательности мы обсуждали в главе 6, но лямбды, используемые для обработки последовательности, не встраиваются (см. раздел 10.2.2). Каждая промежуточная последовательность представлена в виде объекта, хранящего в своем поле лямбду. А терминальная операция инициирует цепочку вызовов по каждой промежуточной последовательности. И хотя операции с последовательностями отложены, не стоит стремиться вставлять вызов метода `asSequence` в каждую цепочку операций над коллекцией в вашем коде. Это помогает только при работе с большими коллекциями, небольшие же можно обрабатывать с помощью обычных операций коллекций.

10.2.4. Когда объявлять функции как встроенные

Мы выяснили преимущества, которое дает использование ключевого слова `inline`. Теперь можно рассмотреть возможность применения `inline` во всей вашей кодовой базе, позволяющую ускорить ее выполнение. Как оказалось, это не самая лучшая идея. Скорее всего, использование ключевого слова `inline` повысит производительность только в функциях, принимающих лямбды в качестве аргументов. Во всех остальных случаях требуется дополнительное исследование, измерение и профилирование разрабатываемого приложения.

Для обычных вызовов функций JVM уже предоставляет мощную поддержку встраивания. Виртуальная машина анализирует выполнение кода и встраивает вызовы, когда это приносит наибольшую пользу. Это происходит автоматически при переводе байт-кода в машинный код. В байт-коде реализация каждой функции повторяется только один раз, и ее не нужно копировать в каждую точку вызова, как в случае с `inline`-функциями Kotlin. Кроме того, если функция вызывается напрямую, то трассировка стека будет более ясной.

С другой стороны, встраивание функций с лямбда-аргументами оказывает положительный эффект. Во-первых, благодаря встраиванию урезаются существенные накладные расходы. Вы экономите не только на вызове, но и на создании дополнительного класса для каждой лямбды и объекта для экземпляра лямбды. Во-вторых, JVM в настоящее время недостаточно умна, чтобы всегда выполнять встраивание через вызов и лямбду. Наконец, встраивание позволяет использовать возможности, недоступные при реализации с помощью обычных лямбд. Например, нелокальные операторы возврата (о них мы поговорим позже в этой главе).

И все равно необходимо обращать внимание на размер кода, принимая решение о том, использовать ли модификатор `inline`. Если во встраиваемой функции много кода, то копирование ее байт-кода в каждую точку вызова может быть дорогостоящим с точки зрения размера байт-кода. В этом случае следует попытаться извлечь код, не связанный с лямбда-аргументами, в отдельную невстроенную функцию. Обратите внимание на размер `inline`-функций в стандартной библиотеке Kotlin — они всегда маленькие.

Далее вы узнаете о том, как улучшить код с помощью функций высшего порядка.

10.2.5. Использование встроенных лямбд для управления ресурсами с помощью функций `withLock`, `use` и `useLines`

В паттерне управления ресурсами лямбды могут устранить дублирование кода. Этот паттерн представляет собой получение ресурса перед выполнением операции и его высвобождение после завершения. В данном случае под словом «ресурс» может скрываться множество понятий: файл, блокировка, транзакция базы данных

и т. д. При стандартной реализации такого паттерна используется оператор `try/finally` — в нем ресурс приобретается до блока `try` и высвобождается в блоке `finally`. А иногда применяются специализированные языковые конструкции, такие как `try-with-resources` в Java.

В подразделе 10.2.1 был показан пример заключения логики оператора `try/finally` в функцию, а код, использующий ресурс, передавался в качестве лямбды в эту функцию. В примере была показана функция `synchronized`. Ее синтаксис аналогичен синтаксису оператора `synchronized` в Java — в качестве аргумента она принимает объект блокировки. В стандартной библиотеке Kotlin определена другая функция `withLock`. У нее более идиоматичный API для решения той же задачи — это функция-расширение интерфейса `Lock`. Применить ее можно следующим образом:

```
val l: Lock = ReentrantLock()
l.withLock {
    // обращение к ресурсу, защищенному этой блокировкой
}
```

Выполнение заданного действия
во время блокировки

Функция `withLock` в библиотеке Kotlin определяется так:

```
inline fun <T> Lock.withLock(action: () -> T): T {
    lock()
    try {
        return action()
    } finally {
        unlock()
    }
}
```

Идиомы работы с блокировками
вынесены в отдельную функцию

В подразделе 14.7.4 мы углубимся в корутины Kotlin и конкурентное программирование и вы познакомитесь с функцией `Mutex`. Она также предоставляет функцию `withLock`, выступающую в качестве аналога только что описанного варианта `Lock`.

Файлы — еще один распространенный тип ресурсов для применения этого паттерна. В листинге 10.17 показана функция Kotlin для считывания первой строки из файла. Для этого используется функция `use` из стандартной библиотеки Kotlin. Это функция-расширение, и она вызывается на закрываемом ресурсе (объект, реализующий интерфейс `Closeable`), а в качестве аргумента получает лямбду. Функция вызывает лямбду и обеспечивает закрытие ресурса, независимо от того, завершается ли лямбда нормально или выбрасывает исключение. В этом примере она делает так, чтобы методы `BufferedReader` и `FileReader` закрылись успешно, они оба реализуют интерфейс `Closeable` после использования.

Разумеется, функция `use` встроена, поэтому ее использование не приводит к снижению производительности.

Листинг 10.17. Использование функции `use` для управления ресурсами

```
import java.io.BufferedReader
import java.io.FileReader

fun readFirstLineFromFile(fileName: String): String {
    BufferedReader(FileReader(fileName)).use { br ->
        return br.readLine()
    }
}
```

Создание функции `BufferedReader`, вызов функции `use` и передача лямбды для выполнения операции с файлом

Возврат строки из функции

Кроме того, в стандартную библиотеку Kotlin входят более специализированные функции-расширения. Функция `use` предназначена для работы с любым типом интерфейса `Closeable`, а `useLines` определена для объектов `File` и `Path`. Она предоставляет лямбде доступ к последовательности строк (с ней вы познакомились в главе 6). Это позволяет сделать код более лаконичным и идиоматичным (листинг 10.18).

Листинг 10.18. Специализированная функция-расширение `useLines`

```
import kotlin.io.path.Path
import kotlin.io.path.useLines

fun readFirstLineFromFile(fileName: String): String {
    Path(fileName).useLines {
        return it.first()
    }
}
```

`it` — последовательность строк из текста входного файла

В KOTLIN НЕТ КОНСТРУКЦИИ TRY-WITH-RESOURCES

В Java есть специальный синтаксис для работы с закрываемыми ресурсами, такими как файлы, — оператор `try-with-resources`. На языке Java код, равнозначный листингу 10.17, для чтения первой строки из файла будет выглядеть так:

```
/* Java */
static String readFirstLineFromFile(String fileName) throws IOException {
    try (BufferedReader br =
        new BufferedReader(new FileReader(fileName))) {
        return br.readLine();
    }
}
```

В Kotlin нет прямого эквивалента специального синтаксиса, поскольку данную задачу можно решить с помощью функции `use`. Это еще одна хорошая иллюстрация универсальных возможностей функций высшего порядка (функций, принимающих лямбды в качестве аргументов).

Обратите внимание: в теле лямбды (в листингах 10.17 и 10.18) используется нелокальный оператор `return` для возврата значения из функции `readFirstLineFromFile` — возврат выполняется из функции, тело которой содержит вызов лямбды, а не только из самой лямбды. Далее мы подробно обсудим использование выражений `return` в лямбдах.

10.3. ВОЗВРАТ ИЗ ЛЯМБД. ПОТОК УПРАВЛЕНИЯ В ФУНКЦИЯХ ВЫСШЕГО ПОРЯДКА

При использовании лямбд для замены императивных конструкций кода, таких как циклы, вы быстро столкнетесь с проблемой выражений `return`. Размещать оператор `return` в середине цикла теперь бесполезно. Но что, если преобразовать цикл в использование функции, например `filter`? Как оператор `return` будет работать в таком случае? Рассмотрим несколько примеров.

10.3.1. Выражения `return` в лямбдах: возврат из замкнутых функций

Мы сравним два разных способа итерации над коллекцией. В листинге 10.19 видно, что если человека зовут Alice, то выполняется возврат из функции `lookForAlice`.

Листинг 10.19. Использование оператора `return` в обычном цикле

```
data class Person(val name: String, val age: Int)

val people = listOf(Person("Alice", 29), Person("Bob", 31))

fun lookForAlice(people: List<Person>) {
    for (person in people) {
        if (person.name == "Alice") {
            println("Found!")
            return
        }
    }
    println("Alice is not found")
}

fun main() {
    lookForAlice(people)
    // Found!
}
```

← Эта строка выводится, если среди переменных `people` нет Alice

Можно ли переписать этот код, используя итерацию `forEach`? Будет ли оператор `return` означать то же самое? Да, можно изменить код с помощью функции `forEach` (листинг 10.20).

Листинг 10.20. Использование оператора `return` в лямбде, переданной в функцию `forEach`

```
fun lookForAlice(people: List<Person>) {
    people.forEach {
        if (it.name == "Alice") {
            println("Found!")
            return
        }
    }
    println("Alice is not found")
}
```

← Возврат из функции, как в листинге 10.19

При использовании ключевого слова `return` в лямбде *выполняется возврат из функции, в которой была вызвана лямбда*, а не только из самой лямбды. Такой оператор `return` называется *нелокальным*, поскольку выполняет возврат из большего блока, чем блок, содержащий оператор `return`.

Чтобы понять логику этого правила, поразмышляйте об использовании ключевого слова `return` в цикле `for` или блока `synchronized` в методе Java. Очевидно, что поток выполнения возвращается из функции, а не из цикла или блока. Kotlin позволяет сохранить такое же поведение при переходе от возможностей языка к функциям, принимающим лямбды в качестве аргументов.

Обратите внимание: возврат из внешней функции возможен *только в том случае, если функция, принимающая лямбду в качестве аргумента, будет встроенной*. В листинге 10.20 тело функции `forEach` встроено вместе с телом лямбды. Поэтому скомпилировать выражение `return` так, чтобы возврат происходил из вложенной функции, несложно. Использование выражения `return` в лямбдах, передаваемых невстроенным функциям, недопустимо, поскольку такая функция может хранить переданную ей лямбду в переменной. Это значит, что она может выполнить лямбду позже, когда возврат уже произойдет. И следовательно, лямбда уже не сможет повлиять на время выполнения возврата из окружающей функции.

10.3.2. Возврат из лямбд: оператор `return` с меткой

Существует возможность написать *локальный* оператор `return` из лямбда-выражения. Локальный возврат останавливает выполнение лямбды и продолжает выполнение кода, из которого была вызвана лямбда. Чтобы отличить локальный возврат от нелокального, используются метки, кратко описанные в главе 2. Можно пометить лямбда-выражение, из которого требуется осуществить возврат, а затем сослаться на эту метку после ключевого слова `return`. В листинге 10.21 функция `forEach` используется для итерации по всем элементам во входной коллекции `people`. А `return` с меткой применяется для пропуска элементов, где свойство `name` не равно строке "Alice".

Листинг 10.21. Использование локального возврата с меткой

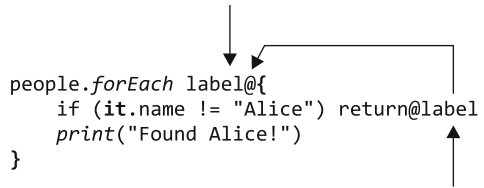
```
fun lookForAlice(people: List<Person>) {
    people.forEach label@{           ← Установка метки лямбда-выражению
        if (it.name != "Alice") return@label    ← Выражение return@label
        print("Found Alice!")                 ← Эта строка
    }                                           выводится только
                                              в том случае, если
                                              возврат не был
                                              выполнен
}

fun main() {
    lookForAlice(people)
    // Found Alice!
}
```

Чтобы обозначить лямбда-выражение, поместите имя метки (это может быть любой идентификатор), за которой следует символ `@`, перед открывающей фигурной скобкой

лямбды. Чтобы выполнить возврат из лямбды, поставьте символ @ после ключевого слова `return`, за которым следует имя метки (рис. 10.6).

Кроме того, в качестве метки можно использовать имя функции, принимающей эту лямбду в качестве аргумента (листинг 10.22).



```
people.forEach label@{
    if (it.name != "Alice") return@label
    print("Found Alice!")
}
```

Рис. 10.6. В возвратах из лямбды используется символ @ для обозначения метки

Листинг 10.22. Использование имени функции в качестве метки `return`

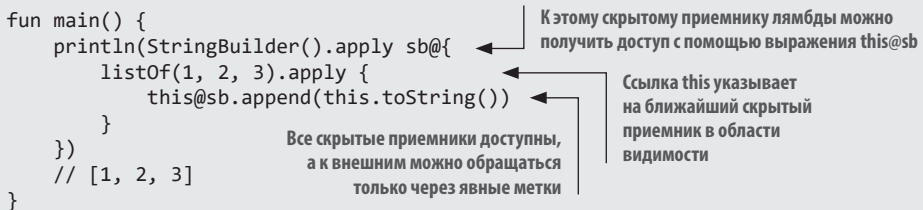
```
fun lookForAlice(people: List<Person>) {
    people.forEach {
        if (it.name != "Alice") return@forEach
        print("Found Alice!")
    }
}
```

Выражение `return@forEach` выполняет возврат из лямбда-выражения

Обратите внимание: если метка лямбда-выражения указывается в явном виде, то пометка по имени функции не работает. В лямбда-выражении может быть только одна метка.

ВЫРАЖЕНИЕ `THIS` С МЕТКОЙ

Те же правила применяются к меткам выражений `this`. В главе 5 мы обсуждали лямбды с приемниками — лямбды, содержащие скрытый объект-приемник, к которому можно обратиться через ссылку `this` в лямбде. (В главе 13 рассказывается о том, как писать собственные функции, ожидающие в качестве аргументов лямбды с приемниками.) Если указать метку лямбды с приемником, то можно получить доступ к ее скрытому приемнику, используя соответствующее выражение `this` с меткой:



```
fun main() {
    println(StringBuilder().apply sb@{
        listOf(1, 2, 3).apply {
            this@sb.append(this.toString())
        }
    })
    // [1, 2, 3]
}
```

К этому скрытому приемнику лямбды можно получить доступ с помощью выражения `this@sb`

Ссылка `this` указывает на ближайший скрытый приемник в области видимости

Все скрытые приемники доступны, а к внешним можно обращаться только через явные метки

Как и в случае с метками для выражений `return`, здесь можно указать метку лямбда-выражения явно или использовать вместо нее имя функции.

Синтаксис нелокального возврата довольно многословен и становится громоздким, если лямбда содержит несколько выражений возврата. В качестве решения можно использовать альтернативный синтаксис передачи блоков кода — анонимные функции.

10.3.3. Анонимные функции: локальные операторы return по умолчанию

Анонимные функции — иная синтаксическая форма записи лямбда-выражений. Таким образом, это еще один способ написать блоки кода, которые можно передавать другим функциям. Однако они используют выражения `return` несколько иным способом. Рассмотрим их подробнее и начнем с примера (листинг 10.23).

Листинг 10.23. Использование оператора `return` в анонимной функции

```
fun lookForAlice(people: List<Person>) {
    people.forEach(fun (person) {
        if (person.name == "Alice") return
        println("${person.name} is not Alice")
    })
}

fun main() {
    lookForAlice(people)
    // Bob is not Alice.
}
```

Использование анонимной функции вместо лямбда-выражения

Оператор `return` относится к ближайшей функции — анонимной

Анонимная функция выглядит как обычная, за исключением того, что ее имя опускается, а типы параметров могут быть выведены. Приведем еще один пример (листинг 10.24).

Листинг 10.24. Использование анонимной функции с методом `filter`

```
people.filter(fun (person): Boolean {
    return person.age < 30
})
```

Возвращаемый тип анонимной функции определяется согласно правилам для обычных функций. Анонимные функции с блочным телом, как, например, в листинге 10.24, требуют явного указания возвращаемого типа. При использовании тела выражения можно опустить тип возвращаемого значения (листинг 10.25).

Листинг 10.25. Использование анонимной функции с телом выражения

```
people.filter(fun (person) = person.age < 30)
```

Выражение `return` без метки внутри анонимной функции выполняет возврат именно из анонимной функции, а не из замкнутой. Правило простое: оператор `return` выполняет возврат из ближайшей функции, объявленной с помощью ключевого слова `fun`. В лямбда-выражениях не используется ключевое слово `fun`, поэтому `return` в лямбде выполняет возврат из внешней функции. При объявлении анонимных функций действительно используется ключевое слово `fun`. Поэтому в предыдущем примере анонимная функция будет наиболее близкой из подходящих. Следовательно, выражение `return` выполняет возврат из анонимной функции, а не из замыкающей. Разница показана на рис. 10.7.

Необходимо отметить, что хотя объявления анонимной и обычной функции похожи, анонимная функция представляет собой иную синтаксическую форму

лямбда-выражения. Чаще всего вы будете использовать синтаксис лямбды, описанный в этой книге. По большей части анонимные функции помогают сократить код, содержащий большое количество ранних выражений `return`, которым требуются метки при использовании синтаксиса лямбд.

```
fun lookForAlice(people: List<Person>) {
    people.forEach(fun(person) {
        if (person.name == "Alice") return
    })
}

fun lookForAlice(people: List<Person>) {
    people.forEach {
        if (it.name == "Alice") return
    }
}
```

Рис. 10.7. Выражение `return` выполняет возврат из функции, объявленной с помощью ключевого слова `fun`

Описание того, как реализуются лямбда-выражения и как они встраиваются во встроенные функции, применимо и к анонимным функциям.

РЕЗЮМЕ

- Типы функций дают возможность объявить переменную, параметр или возвращаемое функцией значение, содержащие ссылку на функцию.
- Функции высшего порядка принимают другие функции в качестве аргументов или возвращают их. Такие функции создаются с использованием типа функции в качестве типа параметра функции или возвращаемого значения.
- При компиляции встроенной функции ее байт-код, а также байт-код переданной ей лямбды вставляется непосредственно в код вызывающей функции. Это гарантирует, что вызов произойдет без лишних затрат по сравнению с аналогичным кодом, написанным непосредственно.
- Функции высшего порядка облегчают повторное использование кода в рамках одного компонента и позволяют создавать мощные библиотеки общего назначения.
- Во встроенных функциях можно использовать *нелокальные возвраты* — выражения `return`, помещенные в лямбду, которые возвращаются из вложенной функции.
- Анонимные функции предоставляют альтернативный синтаксис лямбда-выражений с различными правилами для разрешения выражений `return`. Их можно использовать, когда вам нужно написать блок кода с несколькими точками выхода.

11

Обобщения

В этой главе

- ✓ Объявление функций и классов обобщений.
- ✓ Стирание типов и параметры вещественного типа.
- ✓ Вариативность точек объявления и использования.
- ✓ Псевдонимы типов.

В этой книге приводилось несколько примеров кода с использованием *обобщений* (еще их называют *дженериками*). Основные концепции объявления и использования классов обобщений и функций в Kotlin схожи с Java, поэтому предыдущие примеры должны были быть понятны без подробного объяснения. В этой главе мы вернемся к некоторым примерам и погрузимся в них более глубоко.

Затем мы перейдем к теме обобщений и обсудим новые понятия, появившиеся в Kotlin, такие как параметры вещественного типа и вариативность точки объявления. Эти понятия могут быть для вас новыми, но не волнуйтесь — в главе они подробно описаны.

Параметры вещественного типа с модификатором `reified` позволяют во время выполнения ссылаться на конкретные типы, используемые в качестве аргумента типа в вызове встроенной функции. (Для обычных классов или функций это невозможно, поскольку аргументы типа стираются во время выполнения.)

Вариативность точки объявления позволяет определить, является ли тип обобщения с аргументом типа подтипом или супертипом другого типа обобщения с тем же

базовым типом и другим аргументом типа. Например, она регулирует, можно ли передавать аргументы типа `List<Int>` в функции, ожидающие `List<Any>`.

Вариативность точки использования достигает той же цели для конкретного использования типа обобщения и решает ту же задачу, что и подстановочные знаки в Java.

Итак, обсудим эти темы подробнее, начав с типовых параметров обобщений в целом.

11.1. СОЗДАНИЕ ТИПОВ С АРГУМЕНТАМИ ТИПА — ТИПОВЫЕ ПАРАМЕТРЫ ОБОБЩЕНИЙ

Обобщения позволяют определять типы с *типовыми параметрами*. Когда создается экземпляр такого типа, типовые параметры заменяются определенными типами, называемыми *аргументами типа*. Например, если в коде есть переменная типа `List`, то полезно знать, что хранится в этом списке. Типовой параметр позволяет ответить именно на этот вопрос: вместо «Эта переменная содержит список» можно сказать что-то вроде «Эта переменная содержит список строк». Синтаксис Kotlin для указания «списка строковых значений» выглядит так же, как и в Java: `List<String>`. Вдобавок можно объявить несколько типовых параметров для класса. Например, в классе `Map` есть типовые параметры для типов ключа и значения: `class Map<K, V>`. Его можно создать с определенными аргументами: `Map<String, Person>`. Пока что все выглядит точно так же, как и в Java.

Как и в случае с типами в целом, аргументы типа часто могут быть выведены компилятором Kotlin:

```
val authors = listOf("Sveta", "Seb", "Dima", "Roman")
```

Все значения, передаваемые функции `listOf`, относятся к строковому типу, поэтому компилятор делает вывод, что вы создаете список вида `List<String>` (ваша среда IDE поможет визуализировать это; рис. 11.1).

```
val authors: List<String> = listOf("Sveta", "Seb", "Dima", "Roman")
```

Рис. 11.1. Дополнительные внутритекстовые подсказки в IntelliJ IDEA и Android Studio помогают визуализировать выведенные типы обобщений

Но если требуется создать пустой список, то аргумент типа не из чего вывести, поэтому его нужно указать явно. Создавая список, вы можете выбирать, что указывать: тип в объявлении переменной или аргумента типа для функции, создающей список. В примере ниже показано, как это осуществить:

```
val readers: MutableList<String> = mutableListOf()
```

```
val readers = mutableListOf<String>()
```

Эти объявления эквивалентны. Напомним, что функции создания коллекций были описаны в разделе 8.2.

В KOTLIN НЕТ СЫРЫХ ТИПОВ

В отличие от Java Kotlin всегда требует, чтобы аргументы типа либо указывались явно, либо выводились компилятором. Поскольку обобщения были добавлены в Java только в версии 1.5, то необходимо было поддерживать совместимость с кодом, написанным для более старых версий. Поэтому в Java допускается использовать обобщенный тип без аргументов типа — так называемый *сырой тип*. Например, в Java можно объявить переменную типа `ArrayList`, не указывая, что именно она содержит:

```
List aList = new ArrayList();
```

В Kotlin обобщения использовались с самого начала, поэтому он не поддерживает сырые типы и аргументы типа всегда должны быть определены. Если программа получает из кода Java переменную с сырым типом, то она рассматривается как имеющая обобщенный параметр типа `Any!` — платформенный тип (подробнее об этом мы говорили в разделе 7.12).

11.1.1. Функции и свойства, работающие с обобщенными типами

Если планируется написать функцию для работы со списком и требуется, чтобы она работала с любым списком (обобщенным), а не со списком элементов определенного типа, то нужно написать *обобщенную функцию*. У нее есть свои типовые параметры. Их необходимо заменить определенными аргументами типа при каждом вызове функции.

Большинство библиотечных функций для работы с коллекциями представляют собой обобщения. Для примера рассмотрим объявление функции `slice` (рис. 11.2). Эта функция возвращает список, содержащий только элементы с индексами в указанном диапазоне.

Объявление типового параметра

```
fun <T> List<T>.slice(indices: IntRange): List<T>
```

Типовой параметр используется в типах приемника и возвращаемого значения

Рис. 11.2. Обобщенная функция `slice` содержит типовой параметр `T`, что позволяет ей работать со списками из произвольных элементов. Этот типовой параметр используется как в типе приемника функции-расширения, так и в типе возврата функции

Типовой параметр `T` функции используется в типах приемника и возврата — `List<T>`. При вызове такой функции для конкретного списка можно явно указать аргумент типа. Но в большинстве случаев этого делать не нужно, поскольку компилятор сам выводит тип, как показано далее (листинг 11.1).

**НЕЛЬЗЯ ОБЪЯВИТЬ ОБОБЩЕННОЕ СВОЙСТВО,
НЕ ЯВЛЯЮЩЕЕСЯ РАСШИРЕНИЕМ**

У обычного свойства (не являющегося расширением) не может быть типовых параметров. Невозможно хранить несколько значений разных типов в свойстве класса, поэтому объявлять обобщенное обычное свойство не имеет смысла. При попытке сделать это компилятор сообщит об ошибке:

```
val <T> x: T = TODO()
// Error: type parameter of a property must be used in its receiver type
```

Теперь вспомним, как объявлять обобщенные классы.

**11.1.2. Обобщенные классы объявляются
с помощью синтаксиса угловых скобок**

Как и в Java, обобщенный класс или интерфейс Kotlin объявляется с помощью угловых скобок после имени класса, а внутри них помещаются типовые параметры. После этого параметры можно использовать в теле класса, как и любые другие типы. Рассмотрим, как объявить в Kotlin базовый интерфейс, такой как `List` из стандартной библиотеки. Для упрощения мы опустили большинство методов:

```
interface List<T> {
    operator fun get(index: Int): T
    // ...
}
```

← Интерфейс `List` определяет типовой параметр `T`

← `T` можно использовать в качестве обычного типа в интерфейсе или классе

Позже, в разделе 11.3, при обсуждении вариативности, мы доработаем этот пример, и вы увидите, как объявляется интерфейс `List` в стандартной библиотеке Kotlin.

Если класс расширяет обобщенный класс (или реализует обобщенный интерфейс), то необходимо предоставить аргумент типа для обобщенного параметра базового типа. Это может быть как конкретный тип, так и другой типовой параметр:

```
class StringList: List<String> {
    override fun get(index: Int): String = TODO()
    // ...
}

class ArrayList<T> : List<T> {
    override fun get(index: Int): T = TODO()
    // ...
}
```

← В этом классе реализуется интерфейс `List` и предоставляется определенный аргумент — `String`

← Обратите внимание, что `String` используется вместо `T`

← Теперь обобщенный типовой параметр `T` класса `ArrayList` стал аргументом типа `List`

Класс `StringList` объявлен как содержащий только элементы типа `String`, поэтому `String` в нем выступает в роли аргумента типа для базового типа. Любая функция из подкласса подставляет этот правильный тип вместо `T`. То есть вместо выражения `fun get(Int): T` мы получаем сигнатуру `fun get(Int): String`.

Класс `ArrayList` определяет свой типовой параметр `T` и указывает его в качестве аргумента типа для суперкласса. Обратите внимание на отличие от `List<T>`. В данном случае `T` в `ArrayList<T>` — новый типовой параметр, и его имя не обязательно должно совпадать с именем класса.

Класс может даже ссылаться на себя как на аргумент типа. Классы, реализующие интерфейс `Comparable`, служат классическим примером этого паттерна. Любой сравниваемый элемент должен определять, как сравнивать его с объектами того же типа:

```
interface Comparable<T> {
    fun compareTo(other: T): Int
}

class String : Comparable<String> {
    override fun compareTo(other: String): Int = TODO()
}
```

В классе `String` реализован обобщенный интерфейс `Comparable`, а также для типового параметра `T` предоставлен тип `String`.

Пока что обобщения выглядят так же, как и в Java. О различиях мы поговорим позже, в разделах 11.2 и 11.3.

Теперь обсудим другую аналогичную Java концепцию, позволяющую писать полезные функции для работы со сравниваемыми элементами.

11.1.3. Ограничение типа, который могут использовать обобщенный класс или функция: ограничения типовых параметров

Ограничения типовых параметров дают возможность ограничить типы, которые могут быть использованы в качестве аргументов типа для класса или функции. В качестве примера рассмотрим функцию для вычисления суммы элементов в списке. Ее можно использовать для списков `List<Int>` или `List<Double>`, но не для списка `List<String>`. Чтобы выразить это, можно определить ограничение типового параметра. Оно будет указывать, что типовой параметр `sum` должен быть числом.

Когда вы указываете тип в качестве *верхней границы* для типового параметра обобщенного типа, соответствующие аргументы типа в конкретных экземплярах обобщенного типа должны быть либо указанным типом, либо его подтипами. (Пока можете считать *подтип* синонимом *подкласса*. Разница будет описана в подразделе 11.3.2.)

Чтобы задать ограничение, после имени типового параметра ставится двоеточие, а затем указывается тип верхней границы (рис. 11.3).

Типовой параметр

```

      ┌──┴──┐
      │      │
fun <T : Number> List<T>.sum(): T
      └──┬──┘
          │
        Верхняя граница
```

Рис. 11.3. Ограничения задаются путем указания верхней границы после типового параметра. В данном случае функция `sum` ограничивается списками типа, верхняя граница которого представлена типом `Number`

В Java для выражения той же концепции используется ключевое слово `extends`: `<T extends Number> T sum(List<T> list)`.

Этот вызов функции разрешен, поскольку фактический аргумент типа (`Int` в следующем примере) расширяет абстрактный класс `Number` — суперкласс всех классов числовых значений из стандартной библиотеки Kotlin:

```
fun main() {
    println(listOf(1, 2, 3).sum())
    // 6
}
```

Когда граница для типового параметра `T` будет установлена, можно использовать значения типа `T` в качестве значений его верхней границы. Например, можно вызывать методы класса, используемого в качестве границы:

```
fun <T : Number> oneHalf(value: T): Double {
    return value.toDouble() / 2.0
}

fun main() {
    println(oneHalf(3))
    // 1.5
}
```

← Определение `Number` в качестве верхней границы типового параметра

← Вызов метода из класса `Number`

Теперь напомним обобщенную функцию для нахождения максимального элемента из двух заданных. Поскольку такое значение можно получить только при работе с элементами, которые допускается сравнивать друг с другом, необходимо указать это в сигнатуре функции. Это можно сделать так: задать функции `max` ограничение на прием параметров `first` и `second` типа `T`. Далее ограничить `T` реализацией интерфейса `Comparable<T>`, что позволит использовать только объекты, которые можно сравнить с `T` (листинг 11.3).

Листинг 11.3. Объявление функции с ограничением типового параметра

```
fun <T: Comparable<T>> max(first: T, second: T): T {
    return if (first > second) first else second
}

fun main() {
    println(max("kotlin", "java"))
    // kotlin
}
```

← В качестве аргументов этой функции должны выступать сравнимые элементы

Если вы попытаетесь вызвать функцию `max` на несравнимых элементах, код не скомпилируется:

```
println(max("kotlin", 42))
ERROR: Type parameter bound for T is not satisfied:
    inferred type Any is not a subtype of Comparable<Any>
```

Верхняя граница `T` — обобщенный тип `Comparable<T>`. Класс `String` расширяет интерфейс `Comparable<String>`, что делает `String` действительным аргументом типа для функции `max`.

Помните, что короткая форма `first > second` компилируется в выражение `first.compareTo(second) > 0` в соответствии с соглашениями об операторах Kotlin (см. подраздел 9.2.2). Тип `T` элемента `first` расширяется из интерфейса `Comparable<T>`, поэтому `first` можно сравнить с другим элементом типа `T`.

В редких случаях, когда необходимо указать несколько ограничений для типового параметра, используется немного другой синтаксис. Например, листинг 11.4 служит образцом, который позволяет убедиться, что у заданной последовательности `CharSequence` есть точка в конце. В этом случае необходимо указать, что тип, используемый в качестве аргумента типа, должен реализовывать интерфейсы `CharSequence` и `Appendable`. Это означает, что операции доступа (`endsWith`) и изменения (`append`) данных можно использовать со значениями этого типа. Один из классов с реализацией этих двух интерфейсов — `StringBuilder`. Он представляет изменяемую последовательность символов (краткое описание этого класса приводилось в разделе 3.2).

Листинг 11.4. Указание нескольких ограничений для типового параметра

```
fun <T> ensureTrailingPeriod(seq: T)
    where T : CharSequence, T : Appendable {
    if (!seq.endsWith('.')) {
        seq.append('.')
    }
}

fun main() {
    val helloWorld = StringBuilder("Hello World")
    ensureTrailingPeriod(helloWorld)
    println(helloWorld)
    //Hello World.
}
```

Список ограничений
типового параметра

Вызов функции-расширения, определенной
для интерфейса `CharSequence`

Вызов метода
из интерфейса
`Appendable`

Далее обсудим распространенную ситуацию применения ограничений, когда требуется объявить типовой параметр, который не является `null`.

11.1.4. Исключение аргументов типа, допускающих значение `null`, путем явного обозначения типовых параметров как «не `null`»

При объявлении обобщенного класса или функции, любые аргументы типа, в том числе допускающие значение `null`, можно заменить типовыми параметрами. По сути, типовой параметр без указания верхней границы получит верхнюю границу `Any?`.

Рассмотрим следующий пример:

```
class Processor<T> {
    fun process(value: T) {
        value?.hashCode()
    }
}
```

Параметр `value` допускает значение `null`, поэтому
необходимо использовать безопасный вызов

Параметр `value` функции `process` допускает значение `null`, даже если `T` не помечен вопросительным знаком. Так происходит потому, что конкретные экземпляры

класса `Processor` могут использовать тип, допускающий значение `null`, для `T` нет ограничений, запрещающих типу `T` допускать это значение (например, `String?`):

```

Вместо T подставляется тип String?, допускающий значение null
val nullableStringProcessor = Processor<String?>()
nullableStringProcessor.process(null)

```

Этот код нормально компилируется со значением `null` в качестве аргумента `value`

Необходимо указать ограничение, если требуется, чтобы типовой параметр гарантированно относился к типу «не `null`». Если кроме `null`-безопасности иных ограничений нет, можно использовать тип `Any` в качестве верхней границы, заменив используемый по умолчанию `Any?`:

```

class Processor<T : Any> {
    fun process(value: T) {
        value.hashCode()
    }
}

```

Определение «не `null`» верхней границы
Значение типа `T` теперь «не `null`»

Ограничение `<T : Any>` гарантирует, что тип `T` всегда не будет допускать значение `null`. Компилятор не примет код `Processor<String?>`, поскольку тип аргумента `String?` не относится к подтипам `Any` (это подтип менее специфичного `Any?`):

```

val nullableStringProcessor = Processor<String?>()
// Error: Type argument is not within its bounds: should be subtype of 'Any'

```

Стоит отметить, что можно сделать типовой параметр «не `null`»-типа, указав в качестве верхней границы любой «не `null`»-тип, а не только `Any`.

ПОМЕТКА ОБОБЩЕННЫХ ТИПОВ КАК «ОПРЕДЕЛЕННО НЕ ДОПУСКАЮЩИХ ЗНАЧЕНИЕ NULL» ПРИ ВЗАИМОДЕЙСТВИИ С JAVA

Следует обратить внимание на особый случай — реализацию общих интерфейсов с аннотациями `null`-безопасности из Java (подробнее мы говорили об этом в разделе 7.12). Например, этот обобщенный интерфейс `JBox` ограничивает метод `put`, чтобы он вызывался только с «не `null`»-параметром типа `T`. Заметим, что сам интерфейс не накладывает таких ограничений на тип `T` в целом, позволяя другим методам, таким как `putIfNotNull`, принимать `nullable`-значения:

```

import org.jetbrains.annotations.NotNull;

public interface JBox<T> {
    /**
     * Помещение «не null»-значения в поле.
     */
    /**
     * Значение помещается в поле, если оно не равно null,
     * для значений null операций не выполняется.
     */
    void putIfNotNull(T t);
}

```


С показанным ранее синтаксисом нельзя напрямую преобразовать это ограничение в код Kotlin. Если в реализации Kotlin «не null»-ограничение для обобщенного типа задано выражением `T : Any`, то nullable-значения вообще нельзя использовать в этой реализации, что отличается от ограничения интерфейса Java:

```
class KBox<T> : Any { JBox<T> {
    override fun put(t: T) { /* ... */ }
    override fun putIfNotNull(t: T) { /* Проблема! */ }
}
```

Обобщенный тип `T` уже получил ограничение, не допускающее значение `null`...

...поэтому нельзя ослабить данное ограничение для функции, которая ожидает параметр, допускающий данное значение

Теперь `T` будет допускать значение `null` везде в реализации класса `KBox`, а не только в методе `put`.

Чтобы решить эту проблему, в Kotlin предусмотрен способ пометить тип как *определенно не допускающий значение `null`* в точке его использования (а не в точке первоначального определения обобщенного параметра). Синтаксически это выражается как `T & Any` (возможно, данная форма вам знакома по обозначению типов пересечений в других языках):

```
class KBox<T>: JBox<T> {
    override fun put(t: T & Any) { /* ... */ }
    override fun putIfNotNull(t: T) { /* ... */ }
}
```

Используя типы, определенно не допускающие значение `null`, в Kotlin можно выразить ограничения `null`-безопасности из кода Java.

Мы рассмотрели основы обобщений — тему, наиболее схожую с аналогичной темой в Java. Теперь обсудим другую концепцию: поведение обобщений во время выполнения. Она может в некоторой степени быть вам знакома, если вы занимались разработкой на Java.

11.2. ОБОБЩЕНИЯ ВО ВРЕМЯ ВЫПОЛНЕНИЯ: СТИРАНИЕ И ПАРАМЕТРЫ ВЕЩЕСТВЕННОГО ТИПА

С точки зрения реализации, обобщения в виртуальной машине Java (JVM) обычно реализуются через *стирание типов*. Это означает, что аргументы типа экземпляра обобщенного класса не сохраняются во время выполнения. В текущем разделе мы обсудим практические последствия стирания типов для Kotlin, а также то, как можно обойти его ограничения, объявив функцию с модификатором `inline`. Можно объявить `inline`-функцию так, чтобы ее аргументы типа не стирались (в Kotlin это обозначается ключевым словом `reified`). Мы подробно обсудим параметры вещественного типа (`reified`) и рассмотрим примеры их применения.

11.2.1. Ограничения на поиск информации о типе обобщенного класса во время выполнения: проверка и приведение типов

В Kotlin обобщения *стираются* во время выполнения. Это означает, что экземпляр обобщенного класса не несет информации об аргументах типа, применявшихся для создания этого экземпляра. Например, если создать список вида `List<String>` и поместить в него множество строк, то во время выполнения он будет отображаться просто как `List` (этот эффект можно наблюдать и в Java). Невозможно определить, для какого типа элементов предназначался список. (Конечно, можно получить элемент и проверить его тип, но это действие не даст никаких гарантий, поскольку другие элементы списка могут быть другого типа.)

Рассмотрим, что произойдет с этими двумя списками при выполнении кода:

```
val list1: List<String> = listOf("a", "b")
val list2: List<Int> = listOf(1, 2, 3)
```

Компилятор видит два разных типа списков, однако во время выполнения они выглядят одинаково (рис. 11.4). Несмотря на это, обычно список `List<String>` гарантированно будет хранить только строки, а `List<Int>` — только целые числа. Это связано с тем, что компилятору известны аргументы типа и он гарантирует, что в каждом списке будут храниться только элементы правильного типа. (Можно обмануть компилятор, используя приведение типов или сырые типы Java, чтобы получить доступ к списку, но для этого нужно приложить особые усилия.)

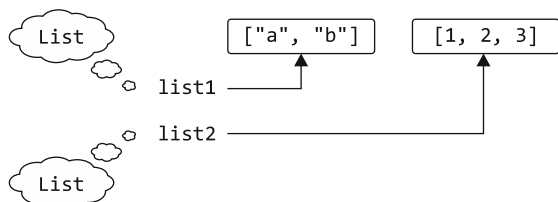


Рис. 11.4. Во время выполнения неизвестно, были ли `list1` и `list2` объявлены как списки строк или целых чисел. Каждый из них обозначен просто как `List`. Это накладывает дополнительные ограничения на работу с аргументами типа

Далее поговорим об ограничениях, связанных со стиранием информации о типе. Аргументы типа не хранятся, поэтому проверить их невозможно. Например, не получится проверить, является ли список именно списком строк. Как правило, в проверках `is` нельзя использовать типы с аргументами типа. Это может стать препятствием, когда требуется создать функцию, поведение которой меняется в зависимости от аргумента типа ее параметра.

Например, у вас может быть функция `readNumbersOrWords`. В зависимости от ввода пользователя она возвращает либо `List<String>`, либо `List<Int>`. Попытка выявить

различия списка чисел и слов с помощью проверки `is` внутри функции `printList` не компилируется:

```
fun readNumbersOrWords(): List<Any> {
    val input = readln()
    val words: List<String> = input.split(",")
    val numbers: List<Int> = words.mapNotNull { it.toIntOrNull() }
    return numbers.isEmpty { words }
}

fun printList(l: List<Any>) {
    when(l) {
        is List<String> -> println("Strings: $l")
        is List<Int> -> println("Integers: $l")
    }
}

fun main() {
    val list = readNumbersOrWords()
    printList(list)
}
```

Ошибка: невозможно проверить
экземпляр стертого типа

Даже если во время выполнения можно узнать, что значение относится к типу `List`, не получится определить, чем оно является: списком строк, людей или иных объектов, — эта информация была стерта. Обратите внимание, что стирание информации обобщенных типов дает и преимущества: общий объем памяти для приложения будет меньше, поскольку в памяти хранится меньше информации о типах.

Kotlin не позволяет использовать обобщенный тип без указания аргументов типа. Поэтому может возникнуть вопрос, как убедиться, что значение представляет собой список, а не множество или другой объект. Для этого можно использовать специальный синтаксис *star-projection* (или «звездную» проекцию):

```
if (value is List<*>) { /* ... */ }
```

По сути, необходимо добавить символ `*` для каждого параметра типа, содержащегося в типе. Мы подробно обсудим синтаксис *star-projection* (в том числе и то, почему он называется *проекцией*) в подразделе 11.3.6, а пока считайте, что это тип с неизвестными аргументами (или аналог `List<?>` в Java).

В предыдущем примере выполнялась проверка того, относится ли `value` к типу `List`, но никакой информации о типе элемента не поступило.

Обратите внимание, что обычные обобщенные типы в операциях приведения `as` и `as?` все еще доступны. Но приведение типа завершится успешно, если у класса правильный базовый тип и неправильный аргумент типа, поскольку аргумент типа скрыт во время выполнения приведения. Из-за этого при выполнении такого приведения компилятор выдаст предупреждение `unchecked cast` (приведение типа без проверки). Это всего лишь предупреждение, поэтому в дальнейшем значение

можно использовать как получившееся необходимый тип, как в следующем примере (листинг 11.5).

Листинг 11.5. Использование приведения типа с обобщенным типом

```
fun printSum(c: Collection<*>) {
    val intList = c as? List<Int> ← Здесь появится предупреждение:
    ? : throw IllegalArgumentException("List is expected")      Unchecked cast: List<*> to List<Int>
    println(intList.sum())
}
```

Этот код считается действительным, поскольку компилятор при проверке выдает только предупреждение. Если вызвать функцию `printSum` для списка или набора целых чисел, то она сработает должным образом: в первом случае выведет сумму, а во втором — выбросит исключение `IllegalArgumentException`:

```
fun main() {
    printSum(listOf(1, 2, 3)) ← Со списками все работает должным образом
    // 6
    printSum(setOf(1, 2, 3)) ← Множество не список, поэтому выбрасывается исключение
    // IllegalArgumentException: List is expected
}
```

Но при передаче значения не того типа будет выброшено исключение `ClassCastException` во время выполнения:

```
fun main() {
    printSum(listOf("a", "b", "c")) ← Приведение типа выполнено успешно,
    // ClassCastException: String cannot be cast to Number      но строки нельзя суммировать, поэтому
                                                                позже будет выброшено другое исключение
}
```

Обсудим исключение, возникающее при вызове функции `printSum` для списка строк. Ошибки `IllegalArgumentException` не возникает, поскольку нельзя проверить, относится ли аргумент к типу `List<Int>`. Поэтому приведение проходит успешно, и допускается вызов функции `sum` для такого списка. А во время ее выполнения выбрасывается исключение. Это происходит потому, что функция пытается получить из списка значения типа `Number` и сложить их вместе. Попытка использовать значение типа `String` в качестве `Number` приведет к возникновению исключения `ClassCastException` во время выполнения.

Стоит отметить, что компилятор Kotlin достаточно умен, чтобы разрешить проверку `is`, когда соответствующая информация о типе уже известна во время компиляции (листинг 11.6).

В листинге 11.6 есть возможность проверить принадлежность элемента к типу `List<Int>`, поскольку во время компиляции известно, что эта коллекция (неважно, список это или другой тип коллекции) содержит целые числа. А в примере из листинга 11.5 ситуация другая: там информации о типе не было.

Листинг 11.6. Использование проверки типа с известным аргументом типа

```

fun printSum(c: Collection<Int>) {
    when (c) {
        is List<Int> -> println("List sum: ${c.sum()}")
        is Set<Int> -> println("Set sum: ${c.sum()}")
    }
}

fun main() {
    printSum(listOf(1,2,3))
    // List sum: 6
    printSum(setOf(3,4,5))
    // Set sum: 12
}

```

Поскольку тип элемента `Int` известен во время компиляции...

...то эти проверки действительны

Как правило, компилятор Kotlin оповещает о доступных и об опасных проверках (запрещая проверки `is` и выдавая предупреждения для приведения типа `as`). Нужно только изучить значение этих предупреждений и понимать, какие операции безопасны.

В Kotlin есть специальная конструкция для использования аргументов определенного типа в теле функции, но ее допускается применять только для `inline`-функций. Далее мы рассмотрим эту функциональную возможность.

11.2.2. Функции с параметрами вещественного типа могут ссылаться на фактические аргументы типа во время выполнения

Обобщения Kotlin стираются во время выполнения, а это значит, что если в коде есть экземпляр обобщенного класса, то узнать аргументы типа, использованные при создании данного экземпляра, не получится. То же самое относится и к типовым аргументам функции. При вызове обобщенной функции в ее теле нельзя определить аргументы типа, с которыми она была вызвана:

```

fun <T> isA(value: Any) = value is T
// Error: Cannot check for instance of erased type: T

```

В целом это справедливо, но есть возможность, позволяющая избежать данного ограничения, — встроенные функции. Параметры вещественного типа встроенных функций могут быть помечены как `reified`. Это означает, что допускается ссылаться на фактические аргументы типа во время выполнения.

В разделе 10.2 `inline`-функции были описаны более подробно. Если пометить функцию модификатором `inline`, то компилятор заменит каждое обращение к функции на фактический код реализации этой функции. Создание `inline`-функции может повысить производительность, если эта функция использует лямбды в качестве

аргументов — код лямбд также может быть встроен, поэтому анонимный класс не создается. В текущем подразделе показан еще один случай полезного применения `inline-функций` — их аргументы типа могут быть помечены как `reified`.

Если объявить предыдущую функцию `isA` как `inline` и пометить типовой параметр ключевым словом `reified`, можно проверить, относится ли `value` к экземплярам типа `T` (листинг 11.7).

Листинг 11.7. Объявление функции с параметрами вещественного типа

`inline fun <reified T> isA(value: Any) = value is T` ← Теперь этот код компилируется

```
fun main() {
    println(isA<String>("abc"))
    // истина
    println(isA<String>(123))
    // ложь
}
```

Рассмотрим несколько менее тривиальных примеров использования параметров вещественного типа. И одним из них станет функция стандартной библиотеки `filterIsInstance`. Функция принимает коллекцию, выбирает экземпляры указанного класса и возвращает только эти экземпляры. Применить ее можно следующим образом (листинг 11.8).

Листинг 11.8. Использование функции `filterIsInstance` из стандартной библиотеки

```
fun main() {
    val items = listOf("one", 2, "three")
    println(items.filterIsInstance<String>())
    // [one, three]
}
```

Сообщить программе, что нам требуются только строки, можно с помощью `<String>` в качестве аргумента типа для функции. И тогда возвращаемый тип функции будет `List<String>`. В этом случае *аргумент типа будет известен во время выполнения*, и функция `filterIsInstance` использует его для проверки значений на принадлежность к экземплярам класса, указанного в качестве аргумента типа.

Упрощенная версия объявления функции `filterIsInstance` из стандартной библиотеки Kotlin показана в листинге 11.9.

Листинг 11.9. Упрощенная реализация функции `filterIsInstance`

```
inline fun <reified T>
    Iterable<*>.filterIsInstance(): List<T> {
    val destination = mutableListOf<T>()
    for (element in this) {
        if (element is T) {
            destination.add(element)
        }
    }
    return destination
}
```

← Ключевое слово `reified` указывает, что этот типовой параметр не будет стерт во время выполнения

← Можно проверить, относится ли элемент к экземплярам класса, указанного в качестве аргумента типа

ПОЧЕМУ МОДИФИКАТОР REIFIED РАБОТАЕТ ТОЛЬКО ДЛЯ ВСТРОЕННЫХ ФУНКЦИЙ

Как работает этот механизм? Почему допускается писать `element is T` в `inline`-функции, но не в обычном классе или функции?

В разделе 10.2 мы говорили, что компилятор вставляет байт-код, реализующий встроенную функцию, в каждую точку ее вызова. При каждом вызове функции с параметром вещественного типа компилятору точно известен тип, использованный в качестве аргумента типа в этом конкретном вызове. Таким образом, компилятор может сгенерировать байт-код со ссылкой на конкретный класс, используемый в качестве аргумента типа. В результате для вызова функции `filterIsInstance<String>` из листинга 11.8, сгенерированный код будет эквивалентен следующему:

```
for (element in this) {
    if (element is String) { ← Ссылка на определенный класс
        destination.add(element)
    }
}
```

Сгенерированный байт-код ссылается на конкретный класс, а не на типовой параметр, поэтому в нем не происходит стирание аргумента типа во время выполнения.

Обратите внимание, что `inline`-функцию с `reified` типовыми параметрами нельзя вызвать из кода Java. Обычные встроенные функции доступны в Java как обычные функции — их можно вызывать, но они не встраиваются. Функции с параметрами вещественного типа требуют дополнительной обработки для подстановки значений аргументов типа в байт-код, поэтому всегда должны быть встроенными. Как следствие, их невозможно вызвать обычным способом, как в коде Java.

У встроенной функции может быть несколько параметров вещественного типа, и дополнительно несколько параметров без модификатора `reified`. Обратите внимание, что функция `filterIsInstance` помечена как `inline`, хотя не ожидает никаких лямбд в качестве аргументов. В подразделе 10.2.4 мы говорили о том, что пометка функции как встроенной дает преимущества в производительности только в том случае, если у нее будут типовые параметры, а соответствующие аргументы — лямбды — встраиваются вместе с функцией. Но в данном случае функция помечается как `inline` не из соображений производительности. Это необходимо, чтобы разрешить использование параметров вещественного типа.

Чтобы гарантировать хорошую производительность, необходимо следить за размером функции с модификатором `inline`. Если она становится большой, то лучше извлечь код, не зависящий от параметров вещественного типа, в отдельные встроенные функции.

11.2.3. Исключение параметров класса `java.lang.Class` путем замены ссылок на классы параметрами вещественного типа

Один из распространенных случаев использования параметров вещественного типа представлен созданием адаптеров для API — они принимают параметры типа `java.lang.Class`. Примером такого API служит механизм `ServiceLoader` из JDK. Он принимает объект `java.lang.Class`, представляющий интерфейс или

абстрактный класс, и возвращает экземпляр сервисного класса с реализацией данного интерфейса на основе ранее предоставленной конфигурации. Рассмотрим, как можно использовать параметры вещественного типа для упрощения вызова этих API.

Чтобы загрузить сервис с помощью стандартного API `ServiceLoader` языка Java, вы можете использовать следующий вызов:

```
val serviceImpl = ServiceLoader.load(Service::class.java)
```

Синтаксис `::class.java` показывает, как получить объект `java.lang.Class`, соответствующий классу Kotlin. Это точный эквивалент класса `Service.class` в Java. Мы более подробно поговорим об этом в разделе 12.2 при обсуждении рефлексии.

Теперь перепишем данный пример с помощью функции с параметром вещественного типа и укажем класс загружаемого сервиса в качестве аргумента типа функции `loadService`:

```
val serviceImpl = loadService<Service>()
```

Так будет намного короче, не правда ли? Указание класса в качестве аргумента типа легче читать, поскольку оно короче, чем используемый в противном случае синтаксис `::class.java`.

Далее посмотрим, как определить функцию `loadService`:

```
inline fun <reified T> loadService() {
    return ServiceLoader.load(T::class.java)
}
```

Типовой параметр помечен как `reified`

Получение доступа к классу типового параметра в форме `T::class`

Можно использовать тот же синтаксис `::class.java` для параметров вещественного типа, что и для обычных классов. При использовании этого синтаксиса будет получен объект `java.lang.Class`. Он соответствует классу, указанному в качестве типового параметра, и его можно использовать обычным образом.

УПРОЩЕНИЕ ФУНКЦИИ `STARTACTIVITY` НА `ANDROID`

Если вы разработчик на Android, то вам может показаться более знакомым другой пример — отображение активностей. Вместо того чтобы передавать класс активности как `java.lang.Class`, можно использовать параметр вещественного типа:

```
inline fun <reified T : Activity>
    Context.startActivity() {
    val intent = Intent(this, T::class.java)
    startActivity(intent)
}

startActivity<DetailActivity>()
```

Типовой параметр помечен как `reified`

Получение доступа к классу параметра вещественного типа в форме `T::class`

Вызов метода для отображения активности

11.2.4. Объявление методов доступа с параметрами вещественного типа

Не только функции Kotlin можно встраивать и применять в них параметры вещественного типа. В подразделе 2.2.2 мы обсуждали, что методы доступа свойств могут предоставлять пользовательские реализации геттеров и сеттеров. Если метод доступа свойства определен для обобщенного типа, то, пометив свойство как `inline` и типовой параметр как `reified`, вы сможете ссылаться на конкретный класс, используемый в качестве аргумента типа.

В примере ниже предоставляется свойство-расширение `canonical`. Оно возвращает каноническое имя обобщенного класса. Как и в подразделе 11.2.3, вы получаете более удобный способ доступа к свойству `canonicalName`, оборачивая вызов к `T::class.java`:

```
inline val <reified T> T.canonical: String
    get() = T::class.java.canonicalName

fun main() {
    println(listOf(1, 2, 3).canonical)
    // java.util.List
    println(1.canonical)
    // java.lang.Integer
}
```

11.2.5. У параметров вещественного типа есть ограничения

Несмотря на то что параметры вещественного типа представляют собой удобный инструмент, у них есть определенные ограничения. Некоторые из них присущи концепции, а другие определяются текущей реализацией и, возможно, будут смягчены в будущих версиях Kotlin.

Если более конкретно, то использовать параметр вещественного типа можно следующим образом:

- в проверке и приведении типов (`is`, `!is`, `as`, `as?`);
- для использования API рефлексии Kotlin, что описано в главе 12 (`::class`);
- для получения соответствующего объекта `java.lang.Class` (`::class.java`);
- в качестве аргумента типа для вызова других функций.

Но при этом нельзя:

- создавать новые экземпляры класса, указанного в качестве типового параметра;
- вызывать методы для объекта-компаньона класса типового параметра;
- использовать типовой параметр без модификатора `reified` в качестве аргумента типа при вызове функции с параметром вещественного типа;
- помечать типовые параметры классов или невстроенных функций как `reified`.

Последнее ограничение приводит к интересному следствию: параметры вещественного типа могут использоваться только во встроенных функциях, поэтому использование такого параметра означает, что функция вместе со всеми переданными ей лямбдами будет встроенной. Если лямбды нельзя встроить из-за способа их использования во встроенной функции или из соображений производительности, то можно использовать модификатор `noinline`, описанный в подразделе 10.2.2. С его помощью лямбда-выражения помечаются как невстраиваемые.

Обсудив работу обобщений в качестве языковой возможности, рассмотрим понятия подтипов и вариативности. Для этого мы более подробно поговорим о самых распространенных обобщенных типах, которые встречаются в каждой программе на Kotlin, — о коллекциях и их подклассах.

11.3. ВАРИАТИВНОСТЬ ОПИСЫВАЕТ ОТНОШЕНИЯ ПОДТИПИЗАЦИИ МЕЖДУ ОБОБЩЕННЫМИ АРГУМЕНТАМИ

Вариативность характеризует то, как типы с одинаковым базовым типом и разными аргументами типа соотносятся друг с другом: например, `List<String>` и `List<Any>`. Сначала мы обсудим, почему это отношение важно в целом, а затем рассмотрим, как оно выражается в Kotlin. Понимать вариативность очень важно при написании собственных обобщенных классов или функций. Эта концепция помогает создавать API без неудобных ограничений для пользователей и не нарушает их ожиданий относительно безопасности типов.

11.3.1. Вариативность определяет, безопасно ли передавать аргумент в функцию

Допустим, есть функция, которая принимает в качестве аргумента список `List<Any>`. Безопасно ли передавать в нее переменную типа `List<String>`? Конечно, это безопасно и можно передавать строку в функцию, ожидающую тип `Any`, поскольку класс `String` расширяет `Any`. Но когда `Any` и `String` становятся аргументами типа интерфейса `List`, ответ уже не столь однозначный.

Для примера рассмотрим функцию вывода содержимого списка:

```
fun printContents(list: List<Any>) {  
    println(list.joinToString())  
}  
  
fun main() {  
    printContents(listOf("abc", "bac"))  
    // abc, bac  
}
```

Похоже, что список строк здесь работает нормально. Функция рассматривает каждый элемент как тип `Any`, а поскольку каждая строка тоже относится к типу `Any`, то применение функции абсолютно безопасно.

Теперь рассмотрим функцию для изменения списка (следовательно, она принимает интерфейс `MutableList` в качестве параметра):

```
fun addAnswer(list: MutableList<Any>) {
    list.add(42)
}
```

Может ли произойти что-то плохое, если передать в эту функцию список строк?

```
fun main() {
    val strings = mutableListOfOf("abc", "bac")
    addAnswer(strings)
    println(strings.maxBy { it.length })
    // ClassCastException: Integer cannot be cast to String
}
```

← Если эта строка пройдет компиляцию... → ...то во время выполнения будет выброшено исключение

Переменная `strings` объявляется как тип `MutableList<String>`. А после мы пытаемся передать ее в функцию. Если бы компилятор принял это (а он этого не делает), то можно было бы добавить целое число в список строк. А это привело бы к исключению во время выполнения при попытке получить доступ к содержимому списка как к строкам. По этой причине данный вызов не компилируется. Этот пример показывает, что передавать `MutableList<String>` в качестве аргумента, когда ожидается `MutableList<Any>`, небезопасно. Запрет компилятора Kotlin здесь будет совершенно правильным.

Теперь вы можете ответить на вопрос, безопасно ли передавать список строк в функцию, которая ожидает список объектов типа `Any`. Небезопасно, если функция добавляет или заменяет элементы в списке, поскольку создается вероятность возникновения несоответствия типов. В противном случае безопасно (подробнее об этом мы поговорим чуть позже). В Kotlin этот момент можно легко контролировать. Для этого необходимо выбрать нужный интерфейс, в зависимости от изменяемости списка. Если функция принимает список, доступный только для чтения, то можно передать тип `List` с более конкретным типом элемента. Если список изменяемый, то сделать это не получится.

Позже мы поднимем тот же вопрос для любого класса обобщения, а не только для `List`. Вдобавок вы узнаете, почему интерфейсы `List` и `MutableList` различаются по аргументу типа. Но перед этим необходимо обсудить понятия *типа* и *подтипа*.

11.3.2. Изучение различий между классами, типами и подтипами

В разделе 7.3 мы говорили, что тип переменной определяет возможные для нее значения. Иногда мы используем термины «тип» и «класс» как взаимозаменяемые, но это неверно, и сейчас самое время обсудить их различия.

В простейшем случае, когда класс не обобщенный, его имя можно использовать непосредственно в качестве типа. Например, если написать `var x: String`, то будет объявлена переменная, способная хранить экземпляры класса `String`. Но стоит

отметить, что то же самое имя класса может быть использовано для объявления типа, допускающего значение `null`: `var x: String?`. Это означает, что каждый класс Kotlin может быть использован для создания как минимум двух типов.

С обобщенными классами все еще сложнее. Чтобы получить допустимый тип, нужно подставить определенный тип в качестве аргумента типа в типовой параметр класса. `List` — не тип (а класс), но все следующие подстановки станут допустимыми типами: `List<Int>`, `List<String?>`, `List<List<String>>` и т. д. Каждый обобщенный класс порождает потенциально бесконечное количество типов.

Чтобы вы могли понять отношения между типами, вам необходимо познакомиться с термином «*подтип*». Тип `B` считается подтипом типа `A`, если допускается использовать значение типа `B` везде, где требуется значение типа `A`. Например, `Int` — подтип `Number`, но `Int` — не подтип `String`. Данное определение также указывает на то, что тип считается подтипом самого себя. На рис. 11.5 это показано наглядно.

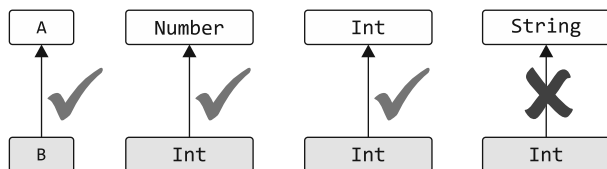


Рис. 11.5. `B` — подтип `A`, если его можно использовать, когда ожидается значение типа `A`. `Int` можно использовать там, где ожидается тип `Number`, так что это подтип. Аналогично тип `Int` используется там, где ожидается тип `Int`. Значит, он является подтипом самого себя. Нельзя использовать тип `Int` там, где ожидается `String`, поэтому он не может считаться подтипом

Термин «*супертип*» противоположен термину «*подтип*». Если `A` — подтип `B`, то `B` — супертип `A`.

Почему важно, является ли один тип подтипом другого? Компилятор выполняет эту проверку при каждом присвоении значения переменной или передаче аргумента в функцию. Рассмотрим следующий пример (листинг 11.10).

Листинг 11.10. Проверка того, является ли тип подтипом другого типа

```

fun test(i: Int) {
    val n: Number = i  ← Компиляция успешна, поскольку Int — подтип типа Number

    fun f(s: String) { /*...*/ }
    f(i)  ← Компиляция не выполнится, так как Int — не подтип типа String
}
  
```

Хранить значение в переменной допустимо только в том случае, если тип значения является подтипом типа переменной. Например, тип `Int` инициализатора переменной `i` представляет подтип типа переменной `Number`, поэтому объявление `n` будет допустимым. Передавать выражение в функцию разрешается, только если тип

выражения представляет собой подтип типа параметра функции. В примере тип `Int` аргумента `i` не является подтипом параметра функции типа `String`, поэтому вызов функции `f` не компилируется.

В простых случаях *подтип* означает практически то же самое, что и *подкласс*. Например, класс `Int` — это подкласс `Number`. Следовательно, тип `Int` — это подтип типа `Number`. Если класс реализует интерфейс, то его тип становится подтипом типа интерфейса: `String` — это подтип `CharSequence`.

Типы, допускающие значение `null`, — пример того, что *подтип* иногда не тождественен *подклассу* (рис. 11.6).

«Не `null`»-тип представляет собой подтип своего варианта, допускающего значение `null`, но оба они соответствуют одному классу. Всегда можно хранить значение типа, не допускающего значение `null`, в переменной типа, который такое значение допускает, но не наоборот (значение `null` недопустимо для переменной типа, не допускающего данное значение). Следовательно, тип, не допускающий значение `null`, будет подтипом типа, который такое значение допускает:

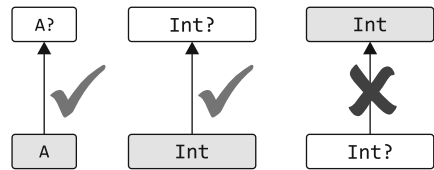


Рис. 11.6. «Не `null`»-тип `A` — подтип типа `A?`, допускающего значение `null`, но не наоборот. `Int` можно использовать там, где ожидается `Int?`, но нельзя использовать `Int?` там, где ожидается `Int`

```
val s: String = "abc"
val t: String? = s
```

← Эта операция присваивания допускается, поскольку `String` — подтип типа `String?`

Разница между подклассами и подтипами становится особенно важной, когда речь идет об обобщенных типах. В предыдущем подразделе мы поднимали вопрос о безопасности передачи переменной типа `List<String>` в функцию, ожидающую `List<Any>`. Теперь его можно переформулировать в понятиях подтипов: `List<String>` — это подтип `List<Any>`? Мы разобрались, почему небезопасно рассматривать `MutableList<String>` как подтип `MutableList<Any>`. Очевидно, что обратное тоже неверно: `MutableList<Any>` — не подтип `MutableList<String>`.

Обобщенный класс, например `MutableList`, называется *инвариантным* по типовому параметру, если для любых двух различных типов `A` и `B` интерфейс `MutableList<A>` не представляет собой подтип или супертип `MutableList<B>`. В Java все классы инвариантны (даже если конкретное их использование может быть помечено как неинвариантное).

В предыдущем подразделе приводился класс, правила подтипизации которого различаются: `List`. Интерфейс `List` в Kotlin представляет коллекцию, доступную только для чтения. Если `A` — подтип типа `B`, то `List<A>` — подтип `List<B>`. Такие классы или интерфейсы называются *ковариантными*. В следующем подразделе мы поговорим о понятии ковариантности и выясним, когда можно объявить класс или интерфейс ковариантным.

11.3.3. Ковариантность сохраняет отношение подтипа

Ковариантный класс — обобщенный (в качестве примера мы будем использовать `Producer<T>`), и для него справедливо следующее правило: `Producer<A>` — это подтип `Producer<B>`, если `A` — подтип `B`. В таком случае можно сказать, что *подтипизация сохраняется*. Например, `Producer<Cat>` — подтип `Producer<Animal>`, поскольку `Cat` — подтип `Animal`.

В Kotlin, чтобы объявить класс ковариантным по определенному параметру типа, необходимо поместить ключевое слово `out` перед именем типового параметра:

```
interface Producer<out T> {
    fun produce(): T
}
```

← Этот класс объявлен как ковариантный для T

Пометка типового параметра класса как ковариантного позволяет передавать значения этого класса в качестве аргументов функции и возвращаемых значений, когда аргументы типа *не совпадают в точности* с теми, которые указаны в определении функции. Допустим, есть функция для обеспечения кормления группы животных и она представлена классом `Herd`. Типовой параметр класса `Herd` определяет тип животного в стае (листинг 11.11).

Листинг 11.11. Определение инвариантного класса, подобного коллекции

```
open class Animal {
    fun feed() { /* ... */ }
}

class Herd<T : Animal> {
    val size: Int get() = /* ... */
    operator fun get(i: Int): T { /* ... */ }
}

fun feedAll(animals: Herd<Animal>) {
    for (i in 0..animals.size) {
        animals[i].feed()
    }
}
```

← Типовой параметр не объявлен как ковариантный

Предположим, у пользователя вашего кода есть стая кошек и ему нужно за ними ухаживать (листинг 11.12).

Листинг 11.12. Использование инвариантного класса, подобного коллекции

```
class Cat : Animal() {
    fun cleanLitter() { /* ... */ }
}

fun takeCareOfCats(cats: Herd<Cat>) {
    for (i in 0..cats.size) {
        cats[i].cleanLitter()
    }
    // feedAll(cats)
}
```

← Кошка (Cat) относится к типу животных (Animal)

← Ошибка: выведенный тип Herd<Cat>, но ожидалось получение Herd<Animal>

К сожалению, кошки останутся голодными. Если бы вы попытались передать стаю в функцию `feedAll`, то при компиляции получили бы ошибку несоответствия типов. Модификатор вариативности не применялся к типовому параметру `T` в классе `Herd` (а значит, он инвариантный), поэтому стая кошек не будет считаться подклассом стаи животных. Для решения проблемы можно использовать приведение типа в явном виде. Но такой подход многословен, чреват ошибками и почти всегда будет неверным способом решения проблемы несоответствия типов.

У класса `Herd` есть API, аналогичный `List`, и клиенты класса не могут добавлять или изменять животных в стае, поэтому можно сделать его ковариантным и соответствующим образом изменить код вызова (листинг 11.13).

Листинг 11.13. Использование ковариантного класса, подобного коллекции

```
class Herd<out T : Animal> {  ← Теперь параметр T ковариантный
    /* ... */
}

fun takeCareOfCats(cats: Herd<Cat>) {
    for (i in 0..

```

Нельзя сделать любой класс ковариантным, так как это было бы небезопасно. Если сделать класс ковариантным по определенному типовому параметру, то это действие ограничит возможные варианты использования данного типового параметра в классе. Чтобы гарантировать безопасность типов, его можно использовать только в так называемых позициях `out`, то есть класс будет производить значения типа `T`, но не потреблять их.

Использование типового параметра в объявлениях членов класса можно разделить на позиции `in` и `out`. Рассмотрим класс с параметром типа `T` и функцией, использующей данный параметр. Если `T` используется в качестве возвращаемого типа функции, то находится в позиции `out`. В этом случае функция *производит* значения типа `T`. Если же `T` используется как тип параметра функции, то находится в позиции `in`. Такая функция *потребляет* значения типа `T`. Это показано на рис. 11.7.

Ключевое слово `out` для типового параметра класса требует, чтобы все методы, использующие тип `T`, хранили его только в позиции `out`. Это ключевое слово ограничивает возможное использование типа `T`, что гарантирует безопасность соответствующего отношения подтипа.

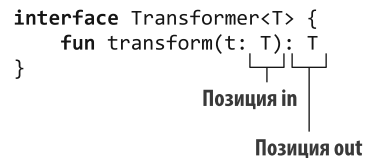


Рис. 11.7. В зависимости от того, где используется обобщенный параметр, его положение обозначается по-разному. Тип параметра функции вызывается в позиции `in`, а тип возвращаемого значения функции — в позиции `out`

Для примера рассмотрим класс `Herd`. Он использует типовой параметр `T` только в одном месте — в возвращаемом значении метода `get`:

```
class Herd<out T : Animal> {
    val size: Int get() = /* ... */
    operator fun get(i: Int): T { /* ... */ }
}
```

Использование `T` в качестве типа возвращаемого значения

Это позиция `out`, и она позволяет объявить класс ковариантным. Любой код, вызывающий метод `get` для коллекции `Herd<Animal>`, будет работать идеально, если метод возвращает значение типа `Cat`, поскольку `Cat` — подтип `Animal`.

Повторимся, что ключевое слово `out` в типовом параметре `T` означает две вещи:

- подтипизация сохраняется (`Producer<Cat>` — это подтип `Producer<Animal>`);
- `T` можно использовать только в позиции `out`.

Теперь рассмотрим интерфейс `List<T>`. Класс `List` в Kotlin предназначен только для чтения. Поэтому в нем есть метод `get` для возврата элемента типа `T`, нет методов для сохранения значения типа `T` в списке. Как следствие, интерфейс тоже ковариантен.

```
interface List<out T> : Collection<T> {
    operator fun get(index: Int): T
    // ...
}
```

Интерфейс только для чтения определяет лишь методы, возвращающие `T` (поэтому `T` находится в позиции `out`)

Обратите внимание, что типовой параметр может использоваться не только непосредственно как тип параметра или тип возвращаемого значения, но и как типовой аргумент другого типа. Например, интерфейс `List` содержит метод `subList`, который возвращает тип `List<T>`.

```
interface List<out T> : Collection<T> {
    fun subList(fromIndex: Int, toIndex: Int): List<T>
    // ...
}
```

Здесь `T` тоже в позиции `out`

В данном случае `T` в функции `subList` используется в позиции `out`. Мы не будем вдаваться в подробности. Если вас интересует точный алгоритм определения позиций `out` и `in`, поищите эту информацию в документации по языку Kotlin.

Обратите внимание, что нельзя объявить список `MutableList<T>` ковариантным по типовому параметру, поскольку он содержит методы, которые принимают значения типа `T` в качестве параметров и возвращают их (поэтому `T` появляется в позициях `in` и `out`). Компилятор соблюдает данное ограничение. Следующий код для объявления интерфейса ковариантным с помощью ключевого слова `out` выдает ошибку: `Type parameter T is declared as 'out' but occurs in the 'in' position` (Параметр типа `T` объявлен как `out`, но находится в позиции `in`):

```
interface MutableList<out T>
    : List<T>, MutableCollection<T> {
    override fun add(element: T): Boolean
}
```

Список `MutableList` нельзя объявить (с помощью ключевого слова `out`) как ковариантный для `T...`

...поскольку `T` используется в позиции `in` (`T` используется в качестве типа параметра функции)

Стоит отметить, что параметры конструктора не находятся в позициях `in` и `out`. Даже если типовой параметр объявлен как `out`, все равно можно использовать его в объявлении параметров конструктора:

```
class Herd<out T: Animal>(vararg animals: T) { /* ... */ }
```

Вариативность защищает экземпляр класса от нецелевого использования, если вы работаете с ним как с экземпляром обобщенного типа, — вызывать потенциально опасные методы просто не получится. Конструктор нельзя вызвать позже (после того как экземпляр будет создан), поэтому он не может быть потенциально опасным.

При использовании ключевого слова `val` или `var` с параметром конструктора происходит и объявление геттера и сеттера (если свойство изменяемое). Поэтому типовой параметр используется в позиции `out` для свойства, доступного только для чтения, и в позициях `out` и `in` для изменяемого свойства:

```
class Herd<T: Animal>(var leadAnimal: T, vararg animals: T) { /* ... */ }
```

В этом случае `T` нельзя пометить как `out`, поскольку класс содержит сеттер для свойства `leadAnimal`, а оно использует `T` в позиции `in`.

Кроме того, обратите внимание, что правила позиционирования распространяются только на видимый извне (`public`, `protected` и `internal`) API класса. Параметры частных методов не находятся в позициях `in` или `out`. Правила вариативности защищают класс от неправильного использования внешними клиентами и не участвуют в реализации самого класса:

```
class Herd<out T: Animal>(private var leadAnimal: T,
    vararg animals: T) { /* ... */ }
```

Теперь можно сделать класс `Herd` ковариантным по `T`, поскольку свойство `leadAnimal` стало приватным.

Возникает вопрос: что происходит с классами или интерфейсами, в которых типовой параметр используется только в позиции `in`? В этом случае действует обратное отношение типов. И в следующем подразделе мы рассмотрим данный вопрос подробнее.

11.3.4. Контравариантность меняет отношение подтипизации на противоположное

Понятие *контравариантности* можно рассматривать как зеркальное отражение ковариантности. Для контравариантного класса отношение подтипизации противоположно отношению подтипизации классов, используемых в качестве его типовых аргументов. Начнем с примера — интерфейс `Comparator`. В этом интерфейсе определен один метод `compare` для сравнения двух заданных объектов:

```
interface Comparator<in T> {
    fun compare(e1: T, e2: T): Int { /* ... */ } ← Использование T в позиции in
}
```

Метод этого интерфейса потребляет только значения типа T. Это означает, что T используется лишь в позиции `in`, поэтому прежде, чем объявлять его, можно использовать ключевое слово `in`.

Компаратор, определенный для значений конкретного типа, может сравнивать значения любого подтипа этого типа. Например, в программе может быть простая иерархия фруктов — яблок и апельсинов — с общим свойством, `weight`:

```
sealed class Fruit {
    abstract val weight: Int
}

data class Apple(
    override val weight: Int,
    val color: String,
): Fruit()

data class Orange(
    override val weight: Int,
    val juicy: Boolean,
): Fruit()
```

Если сейчас создать экземпляр `Comparator<Fruit>`, то можно использовать его для сравнения значений любого заданного типа:

```
fun main() {
    val weightComparator = Comparator<Fruit> { a, b ->
        a.weight - b.weight
    }
    val fruits: List<Fruit> = listOf(
        Orange(180, true),
        Apple(100, "green")
    )
    val apples: List<Apple> = listOf(
        Apple(50, "red"),
        Apple(120, "green"),
        Apple(155, "yellow")
    )
    println(fruits.sortedWith(weightComparator))
    // [Apple(weight=100, color=green), Orange(weight=180, juicy=true)]
    println(apples.sortedWith(weightComparator))
    // [Apple(weight=50, color=red), Apple(weight=120, color=green),
        Apple(weight=155, color=yellow)]
}
```

Можно использовать компаратор веса для любой коллекции объектов, представляющих подтип `Fruit`, например яблок и апельсинов

Функция `sortedWith` ожидает значение типа `Comparator<String>` (компаратора для сравнения строк). Но в нее безопаснее передать компаратор более общих типов. Если требуется выполнить сравнение объектов определенного типа, то можно использовать компаратор для работы с этим типом или с любым из его супертипов. Это означает, что `Comparator<Any>` представляет собой *подтип* `Comparator<String>`, где `Any` — *супертип* `String`. Отношение подтипизации между компараторами для двух разных типов идет в противоположном направлении от отношения подтипизации между этими типами.

Теперь вы готовы к полному определению контравариантности.

Класс, *контравариантный* по параметру типа, — это обобщенный класс (рассмотрим в качестве примера `Consumer<T>`), для которого справедливо следующее: `Consumer<A>` — это подтип `Consumer<B>`, если `B` — подтип `A`. Аргументы типа `A` и `B` меняются местами, поэтому говорится, что подтипизация обратная. Например, `Consumer<Animal>` — подтип `Consumer<Cat>`.

На рис. 11.8 показано различие между отношениями подтипизации для классов, ковариантных и контравариантных по типовому параметру. Для класса `Producer` отношение подтипизации повторяет отношение подтипизации для его аргументов типа, в то время как для класса `Consumer` отношение обратное.

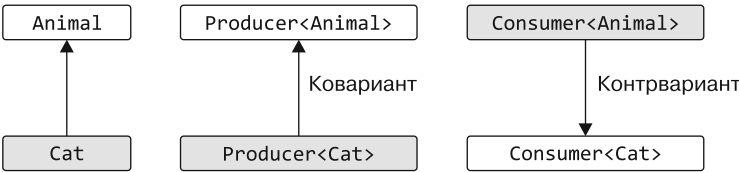


Рис. 11.8. Для ковариантного типа `Producer<T>` подтипизация сохраняется, а для контравариантного типа `Consumer<T>` подтипизация меняется на противоположную

Ключевое слово `in` означает, что значения соответствующего типа передаются в методы этого класса и потребляются ими. Как и в ковариантном случае, ограничение на использование типового параметра приводит к специфическому отношению подтипизации. Ключевое слово `in` для типового параметра `T` означает, что подтипизация будет обратной и `T` можно использовать только в позициях `in`.

В табл. 11.1 приведены различия между возможными типами вариативности.

Таблица 11.1. Ковариантные, контравариантные и инвариантные классы

Ковариант	Контравариант	Инвариант
<code>Producer<out T></code>	<code>Consumer<in T></code>	<code>MutableList<T></code>
Подтипизация для класса сохраняется: <code>Producer<Cat></code> — подтип <code>Producer<Animal></code>	Подтипизация обратная: <code>Consumer<Animal></code> — подтип <code>Consumer<Cat></code>	Без подтипизации
<code>T</code> только в позициях <code>out</code>	<code>T</code> только в позициях <code>in</code>	<code>T</code> в любой позиции

Класс или интерфейс может быть ковариантным по одному типовому параметру и контравариантным — по другому. Классическим примером может служить интерфейс `Function`. В следующем объявлении показан `Function` с одним параметром:

```
interface Function1<in P, out R> {  
    operator fun invoke(p: P): R  
}
```

Нотация Kotlin $(P) \rightarrow R$ — это иная, более удобочитаемая форма выражения `Function1<P, R>`. Здесь P (тип параметра) используется только в позиции `in` и помечается ключевым словом `in`, а R (тип возвращаемого значения) используется лишь в позиции `out` и помечается ключевым словом `out`. Это означает, что подтипизация для типа функции изменяется на противоположную для ее первого аргумента типа и сохраняется для второго. Например, если есть функция высшего порядка для перечисления ваших кошек, то можно передать ей лямбду, принимающую любых животных:

```
fun enumerateCats(f: (Cat) -> Number) { /* ... */ }
fun Animal.getIndex(): Int = /* ... */

fun main() {
    enumerateCats(Animal::getIndex)
}
```

Этот код действителен для Kotlin. `Animal` — супертип `Cat`, а `Int` — подтип `Number`

На рис. 11.9 отображены отношения подтипов, показанных в предыдущем примере.

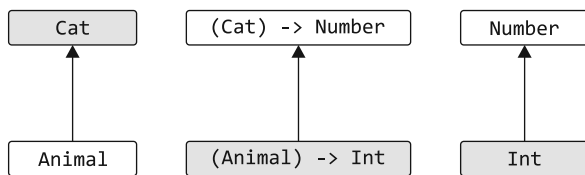


Рис. 11.9. Функция $(T) \rightarrow R$ контравариантна по аргументу (обратная подтипизация) и ковариантна по возвращаемому типу (подтипизация сохранена)

Обратите внимание: во всех приведенных примерах вариативность класса указывается непосредственно в его объявлении и применяется ко всем точкам использования данного класса. В Java этот способ не поддерживается, и вместо него обозначить различия между конкретными вариантами использования класса можно с помощью подстановочных знаков. Разберем разницу между этими двумя подходами и посмотрим, как можно использовать второй подход в Kotlin.

11.3.5. Определение вариативности встречающихся типов через вариативность точек использования

Возможность указывать модификаторы вариативности в объявлениях классов удобна тем, что модификаторы применяются во всех точках использования класса. Это называется *вариативностью в точке объявления*. Если вам знакомы подстановочные типы Java (`? extends` и `? super`), то вы поймете, что Java по-другому обрабатывает вариативность. В данном языке при каждом использовании типа с типовым параметром допускается указать возможность замены типового параметра на его подтипы или супертипы. Это называется *вариативностью точки использования*.

ВАРИАТИВНОСТЬ В ТОЧКЕ ОБЪЯВЛЕНИЯ В KOTLIN И ПОДСТАНОВОЧНЫЕ ЗНАКИ В JAVA

Вариативность в точке объявления позволяет сделать код более лаконичным. Модификаторы вариативности указываются только раз, и клиентам класса не нужно о них задумываться. В Java для создания API, которые ведут себя в соответствии с ожиданиями пользователей, автор библиотеки должен постоянно использовать подстановочные знаки: `Function<? super T, ? extends R>`. Если более детально изучить исходный код стандартной библиотеки Java 8, то можно заметить подстановочные знаки при каждом использовании интерфейса `Function`. Например, метод `Stream.map` объявлен следующим образом:

```
/* Java */
public interface Stream<T> {
    <R> Stream<R> map(Function<? super T, ? extends R> mapper);
}
```

Указав вариативность один раз в объявлении, вы сделаете код гораздо более лаконичным и элегантным.

Кроме того, Kotlin поддерживает вариативность в точке использования. Благодаря этому можно указать вариативность для конкретного применения типового параметра, даже если он не может быть объявлен как ковариантный или контравариантный в объявлении класса. Посмотрим, как это работает.

Многие интерфейсы, например `MutableList`, не являются ковариантными или контравариантными в общем случае, поскольку могут как производить, так и потреблять значения типов, указанных в их типовых параметрах. Но обычно переменная такого типа в конкретной функции используется только в роли производителя или потребителя. Для примера рассмотрим эту простую функцию (листинг 11.14).

Листинг 11.14. Функция копирования данных с инвариантными типами параметров

```
fun <T> copyData(source: MutableList<T>,
                destination: MutableList<T>) {
    for (item in source) {
        destination.add(item)
    }
}
```

Эта функция копирует элементы из одной коллекции в другую. У обеих коллекций неизменяемый тип, однако исходная используется только для чтения, а целевая — только для записи. В такой ситуации элементы определенного типа можно скопировать в коллекцию, хранящую супертип этих элементов. Например, вполне допустимо скопировать коллекцию строк в коллекцию с обобщенным типом `Any`.

Чтобы дать этой функции указание работать со списками разных типов, вы можете ввести второй обобщенный параметр (листинг 11.15).

Листинг 11.15. Функция копирования данных с двумя типовыми параметрами

```
fun <T: R, R> copyData(source: MutableList<T>,
                      destination: MutableList<R>) {
    for (item in source) {
        destination.add(item)
    }
}

fun main() {
    val ints = mutableListOf(1, 2, 3)
    val anyItems = mutableListOf<Any>()
    copyData(ints, anyItems)
    println(anyItems)
    // [1, 2, 3]
}
```

Тип элемента источника должен быть подтипом типа элемента назначения

Эту функцию можно вызвать, поскольку Int — подтип Any

Здесь объявлены два обобщенных параметра. Они представляют типы элементов в списках источника и назначения. Чтобы вы могли получить возможность копировать элементы из одного списка в другой, тип исходного элемента должен быть подтипом элементов в списке назначения — `destination`. Так, в листинге 11.16 тип `Int` — подтип `Any`.

Но Kotlin предлагает более элегантный способ выразить эти отношения. Если реализация функции вызывает только методы с типовым параметром в позиции `out` (или только в позиции `in`), то можно воспользоваться этим и добавить модификаторы вариативности к конкретным случаям применения типового параметра в определении функции (листинг 11.16).

Листинг 11.16. Функция копирования данных с предполагаемым типовым параметром `out`

```
fun <T> copyData(source: MutableList<out T>,
                destination: MutableList<T>) {
    for (item in source) {
        destination.add(item)
    }
}
```

Ключевое слово `out` можно добавить к использованию типа — методы с `T` в позиции `in` применяться не будут

Модификатор вариативности можно указать для любого использования типового параметра в объявлении типа, для типа параметра (как в листинге 11.16), типа локальной переменной, типа возвращаемой функции и т. д. Все это называется *проекцией типа*: мы говорим, что источник (`source`) не обычный интерфейс `MutableList`, а *спроецированный* (ограниченный). Можно вызывать лишь методы, которые возвращают обобщенный типовой параметр, а точнее, используют его только в позиции `out`. Компилятор запрещает вызывать методы, в которых этот типовой параметр используется в качестве аргумента (в позиции `in`):

```
fun main() {
    val list: MutableList<out Number> = mutableListOfOf()
    list.add(42)
    // Error: Out-projected type 'MutableList<out Number>' prohibits
    // the use of 'fun add(element: E): Boolean'
}
```

Не удивляйтесь, что некоторые методы не получится вызвать при использовании проецируемого типа. Если их необходимо вызвать, то нужно использовать обычный тип, а не проекцию. Может потребоваться объявить второй типовой параметр, зависящий от того, который изначально был проекцией, как в листинге 11.15.

Конечно, правильнее всего для реализации функции `copyData` будет использовать `List<T>` в качестве типа аргумента `source`, поскольку применяются лишь методы, объявленные в интерфейсе `List`, а не `MutableList`. А вариативность типового параметра `List` указана в его объявлении. Но этот пример все равно важен для иллюстрации концепции, особенно если учесть, что у большинства классов нет отдельного ковариантного интерфейса чтения и инвариантного интерфейса чтения-записи, как, например, `List` и `MutableList`.

Нет смысла в проекции `out` типового параметра, у которого уже есть вариативность `out`, например `List<out T>`. Такая форма эквивалентна `List<T>`, поскольку класс `List` объявлен как `class List<out T>`. Компилятор Kotlin предупредит, что такая проекция избыточна.

Аналогичным образом можно использовать модификатор `in` для типового параметра. Это позволит указать, что в данном месте соответствующее значение выступает в роли потребителя, а типовой параметр можно заменить любым из его супертипов. Переписать листинг 11.16 с помощью проекции `in` можно следующим образом (листинг 11.17).

Листинг 11.17. Функция копирования данных с in-проекцией типового параметра

```
fun <T> copyData(source: MutableList<T>,
                destination: MutableList<in T>) {
    for (item in source) {
        destination.add(item)
    }
}
```

Позволяет типу элемента назначения быть супертипом типа элемента источника

ПРИМЕЧАНИЕ

Объявления вариативности точки использования в Kotlin напрямую соответствуют ограниченным подстановочным знакам в Java. Так, `MutableList<out T>` в Kotlin означает то же самое, что и `MutableList<? extends T>` в Java. В Kotlin in-проекция `MutableList<in T>` соответствует `MutableList<? super T>` в Java.

Проекции в точке вызова помогают расширить диапазон приемлемых типов. Теперь обсудим крайний случай, когда допустимыми становятся типы со всеми возможными типовыми аргументами.

11.3.6. «Звездная» проекция: использование символа * для указания на недостаток информации об обобщенном аргументе

Ранее в этой главе, при обсуждении проверки и приведения типов, мы упоминали специальный *синтаксис «звездной» проекции*. Его используют, чтобы указать на отсутствие информации об обобщенном аргументе. Например, список элементов неизвестного типа выражается с помощью этого синтаксиса как `List<*>`. Теперь рассмотрим семантику «звездных» проекций более подробно.

Для начала отметим, что `MutableList<*>` не то же самое, что `MutableList<Any?>` (здесь важно отметить, что `MutableList<T>` — это инвариант `T`). Известно, что список `MutableList<Any?>` может содержать элементы любого типа. С другой стороны, список `MutableList<*>` содержит элементы определенного, неизвестного вам типа. Список был создан как список элементов определенного типа, например `String` (нельзя создать новый `ArrayList<*>`), и создавший его код ожидает, что список будет содержать только элементы этого типа. Тип неизвестен, поэтому поместить объекты в список не получится, ведь любое значение может нарушить ожидания вызывающего кода. Но получить элементы из списка можно, поскольку известно, что все хранящиеся в нем значения будут соответствовать типу `Any?`. А он служит супертипом всех типов Kotlin:

```
import kotlin.random.Random
```

```
fun main() {
    val list: MutableList<Any?> = mutableListOfOf('a', 1, "qwe")
    val chars = mutableListOfOf('a', 'b', 'c')
    val unknownElements: MutableList<*> = ← MutableList<*> отличается от MutableList<Any?>
        if (Random.nextBoolean()) list else chars
    println(unknownElements.first())      ← Получение элементов безопасно:
    // a                                  функция first() возвращает
    unknownElements.add(42) ← Компилятор запрещает вызывать этот метод
    // Error: Out-projected type 'MutableList<*>' prohibits
    // the use of 'fun add(element: E): Boolean'
}
```

Почему компилятор ссылается на `MutableList<*>` как на *out*-проецируемый тип? В данном контексте `MutableList<*>` проецируется на (действует как) `MutableList<out Any?>`. Когда ничего не известно о типе элемента, можно безопасно получать элементы типа `Any?`, но небезопасно помещать элементы в список. Что касается подстановочных знаков Java, то `MyType<*>` в Kotlin соответствует `MyType<?>` в Java.

ПРИМЕЧАНИЕ

Для контравариантных типовых параметров, таких как `Consumer<in T>`, «звездная» проекция эквивалентна `<in Nothing>`. По сути, на такой «звездной» проекции нельзя вызывать методы, в сигнатуре которых содержится `T`. Если типовой параметр контравариантен, то действует только как потребитель, и неизвестно, какие типы он будет потреблять. Поэтому нельзя передать ему ничего для потребления. Более подробную информацию можно найти в онлайн-документации Kotlin (<http://mng.bz/3Ed7>).

Использовать синтаксис «звездной» проекции можно, когда информация об аргументах типа не важна, — не используются методы со ссылками на параметр типа в сигнатуре или выполняется только считывание данных, и их конкретный тип неважен. Например, можно реализовать функцию `printFirst`, принимая в качестве параметра `List<*>`:

```
fun printFirst(list: List<*>) {
    if (list.isNotEmpty()) {
        println(list.first())
    }
}

fun main() {
    printFirst(listOf("Sveta", "Seb", "Dima", "Roman"))
    // Sveta
}
```

← Каждый список — возможный аргумент

← Функция `isNotEmpty()` не использует обобщенный типовой параметр

← Функция `first()` возвращает тип `Any?`, но в данном случае этого достаточно

Как и в случае с вариативностью точки использования, доступна альтернатива — ввести обобщенный типовой параметр:

```
fun <T> printFirst(list: List<T>) {
    if (list.isNotEmpty()) {
        println(list.first())
    }
}
```

← И снова каждый список — это возможный аргумент

← Теперь функция `first()` возвращает значение типа `T`

Синтаксис «звездной» проекции более лаконичен, но работает только в том случае, если точное значение обобщенного типового параметра неважно, — используются только методы, которые выдают значения, и типы этих значений не играют особой роли.

Теперь рассмотрим другой пример использования типа со «звездной» проекцией и типичные проблемы, возникающие при использовании этого подхода. Допустим, требуется проверять вводимые пользователем данные, и вы объявляете интерфейс `FieldValidator`. Он содержит типовой параметр только в позиции `in`, поэтому его можно объявить как контравариантный. На самом деле лучше использовать валидатор, проверяющий любые элементы, когда требуется валидатор строк (именно это позволяет объявить его контравариантным). Вдобавок необходимо объявить два валидатора для обработки ввода значений типа `String` и `Int` (листинг 11.18).

Листинг 11.18. Интерфейсы для проверки ввода

```
interface FieldValidator<in T> {
    fun validate(input: T): Boolean
}

object DefaultStringValidator : FieldValidator<String> {
    override fun validate(input: String) = input.isNotEmpty()
}

object DefaultIntValidator : FieldValidator<Int> {
    override fun validate(input: Int) = input >= 0
}
```

← Интерфейс, объявленный контравариантным по `T`

← `T` используется только в позиции `in` (этот метод потребляет значения типа `T`)

Теперь представьте, что планируется хранить все валидаторы в одном контейнере и получать требуемый в зависимости от типа ввода. Для их хранения можно сначала попытаться использовать карту. Необходимо хранить валидаторы для любых типов, поэтому объявляется карта типа `KClass` (он представляет класс Kotlin — в главе 12 мы подробно рассмотрим `KClass`) в интерфейсе `FieldValidator<*>` (который может ссылаться на валидатор любого типа):

```
import kotlin.reflect.KClass

fun main() {
    val validators = mutableMapOf<KClass<*>, FieldValidator<*>>()
    validators[String::class] = DefaultStringValidator
    validators[Int::class] = DefaultIntValidator
}
```

Если реализовать этот подход, то у вас могут возникнуть трудности при попытке использовать валидаторы. Проверить строку с помощью валидатора типа `FieldValidator<*>` не получится. Это небезопасно, поскольку компилятору неизвестно, какой валидатор используется:

```
validators[String::class]!!.validate("")
// Error: Out-projected type 'FieldValidator<*>' prohibits
// the use of 'fun validate(input: T): Boolean'
```

← У значения в карте будет тип `FieldValidator<*>`

Эта ошибка возникала ранее, при попытке поместить элемент в список `MutaleList<*>`. В данном случае эта ошибка означает, что передавать значение определенного типа валидатору неизвестного типа небезопасно. Чтобы исправить ситуацию, можно явно привести валидатор к нужному типу (листинг 11.19). Это небезопасно и не рекомендуется. Но мы решили показать такой вариант как способ быстро заставить ваш код компилироваться. А после вы сможете выполнить рефакторинг кода программы.

Листинг 11.19. Получение валидатора с помощью явного приведения типа

```
val stringValidator = validators[String::class] as FieldValidator<String>
println(stringValidator.validate(""))
// ложь
```

← Предупреждение: «приведение типа без проверки»

Компилятор выдает предупреждение о приведении типа без проверки. Однако стоит отметить, что этот код вызовет проблемы только при проверке компилятором, а не во время выполнения приведения. Это связано с тем, что во время выполнения вся информация об обобщенном типе стирается (листинг 11.20).

Листинг 11.20. Неправильное получение валидатора

```
val stringValidator = validators[Int::class]
                        as FieldValidator<String>
stringValidator.validate("")
// java.lang.ClassCastException:
// java.lang.String cannot be cast to java.lang.Number
// at DefaultIntValidator.validate
```

← Здесь неправильный валидатор (возможно, по ошибке), но код компилируется

← Это всего лишь предупреждение

← Настоящая ошибка скрыта до момента применения валидатора

Этот неверный код и листинг 11.19 похожи в том смысле, что в обоих случаях выдается только предупреждение. Необходимо выполнять приведение типа только значений правильного типа.

Данное решение не отвечает требованиям безопасности типов и чревато ошибками. Итак, разберемся, какие еще есть варианты, если требуется хранить валидаторы для разных типов в одном месте.

Для решения в листинге 11.21 использовалась та же карта `validators`, но весь доступ к ней инкапсулирован в два обобщенных метода, отвечающих за регистрацию и возврат только корректных валидаторов. К тому же этот код выдает предупреждение о приведении типа без проверки (то же самое), но здесь объект `Validators` контролирует весь доступ к карте, что гарантирует отсутствие в ней неправильных изменений.

Листинг 11.21. Инкапсуляция доступа к коллекции валидаторов

```
object Validators {
    private val validators = mutableMapOf<KClass<*>, FieldValidator<*>>()

    fun <T: Any> registerValidator(
        kClass: KClass<T>, fieldValidator: FieldValidator<T>) {
        validators[kClass] = fieldValidator
    }

    @Suppress("UNCHECKED_CAST")
    operator fun <T: Any> get(kClass: KClass<T>): FieldValidator<T> =
        validators[kClass] as? FieldValidator<T>
        ?: throw IllegalArgumentException(
            "No validator for ${kClass.simpleName}")
}

fun main() {
    Validators.registerValidator(String::class, DefaultStringValidator)
    Validators.registerValidator(Int::class, DefaultIntValidator)

    println(Validators[String::class].validate("Kotlin"))
    // истина
    println(Validators[Int::class].validate(42))
    // истина
}
```

Используется та же карта, что и ранее. Но теперь к ней нельзя получить доступ извне

Когда валидатор соответствует классу, в карту помещаются только корректные пары «ключ — значение»

Подавляет предупреждение о приведении к типу `FieldValidator<T>` без проверки

Теперь мы получили API с безопасностью типов. Вся небезопасная логика скрыта в теле класса, и локализация обеспечивает ее корректное использование. Компилятор не позволяет использовать неправильный валидатор, поскольку объект `Validators` всегда предоставляет правильную реализацию валидатора:

```
println(Validators[String::class].validate(42))
// Error: The integer literal does not conform to the expected type String
```

Теперь метод `get` возвращает экземпляр интерфейса `FieldValidator<String>`

Этот шаблон можно легко расширить до хранения любых пользовательских обобщенных классов. Локализация небезопасного кода в отдельном месте предотвращает

злоупотребления и делает использование контейнера безопасным. Стоит отметить, что описанный здесь паттерн неисключителен для Kotlin — в Java можно применять такой же подход.

Обобщения и вариативность в Java обычно считаются самой сложной частью языка. В Kotlin мы постарались придумать дизайн, который облегчил бы понимание этой темы и упростил работу с кодом, и сохранить при этом совместимость с Java.

11.3.7. Псевдонимы типов

При работе с типами, объединяющими несколько обобщений, иногда бывает сложно уследить за смыслом, скрытым за сигнатурой типа. Назначение коллекции с типом `List<(String, Int) -> String>` может быть очевидным не сразу. Вдобавок может возникнуть желание избежать повторения одной и той же сложной комбинации обобщенных и функциональных типов при каждом обращении к этой коллекции.

Для таких случаев Kotlin предоставляет возможность определять *псевдонимы типов* — альтернативные имена для существующих типов. Псевдоним типа вводится с помощью ключевого слова `typealias`, а за ним следует псевдоним. Затем, после знака `=`, необходимо указать исходный, базовый тип.

Псевдонимы типов могут оказаться особенно полезными при сокращении длинных обобщенных типов. В примере ниже объявляется функция `combineAuthors` для объединения имен четырех авторов этой книги. Поведение того, *как* данные авторов объединяются, можно передать с помощью параметра функционального типа. Он будет принимать четыре строки и возвращать новую, объединенную строку. Это удобно, так как сигнатура типа `(String, String, String, String) -> String` несколько громоздкая и ее неудобно вводить при каждом обращении. Этому функциональному типу с помощью псевдонима можно дать новое имя — `NameCombiner`. И его можно использовать везде, где раньше использовался базовый тип:

```
typealias NameCombiner = (String, String, String, String) -> String

val authorsCombiner: NameCombiner = { a, b, c, d -> "$a et al." }
val bandCombiner: NameCombiner = { a, b, c, d -> "$a, $b & The Gang" }

fun combineAuthors(combiner: NameCombiner) {
    println(combiner("Sveta", "Seb", "Dima", "Roman"))
}

fun main() {
    combineAuthors(bandCombiner)
    // Sveta, Seb & The Gang
    combineAuthors(authorsCombiner)
    // Sveta et al.
    combineAuthors { a, b, c, d -> "$d, $c & Co." }
    // Roman, Dima & Co.
}
```

Псевдоним типа определен с помощью ключевого слова `typealias`, псевдонима и базового типа

Псевдонимы типов можно использовать везде, где применялся бы базовый тип, например, в объявлении переменной...

Псевдоним типа разрешается в базовый тип. Поэтому совершенно нормально передавать `NameCombiner...`

...или лямбду, которая принимает четыре и возвращает одну строку

...или в объявлении параметра функции

Псевдоним типа наделяет функциональный тип дополнительным контекстом, что может помочь при чтении кода. Однако стоит помнить, что разработчикам, не знакомым с вашей кодовой базой, придется тратить время на понимание псевдонима `NameCombiner` и переход к его базовому типу при чтении кода или внесении изменений. Решение о применении псевдонимов типов в кодовой базе в конечном счете зависит от того, какой компромисс вам придется выбрать.

Стоит также отметить, что с точки зрения компилятора псевдонимы типов не вводят никаких новых ограничений или изменений — во время компиляции псевдонимы будут полностью развернуты до своего базового типа. Они помогают сократить код, но не обеспечивают никакой дополнительной безопасности типов.

КОГДА ИСПОЛЬЗОВАТЬ ВСТРОЕННЫЕ КЛАССЫ И ПСЕВДОНИМЫ ТИПОВ

Псевдонимы типов представляют собой полезное сокращение, но не обеспечивают никакой дополнительной безопасности типов. Это означает, что их нельзя изолировать в целях получения дополнительных гарантий, предотвращающих случайное использование двух типов вместо друг друга. Рассмотрим это в следующем примере. Введение псевдонима `ValidatedInput` для типа `String` помогает сделать сигнатуру функции `save` более понятной. Название сообщает о том, что ожидаются подтвержденные входные данные. Однако компилятор примет любое значение типа `String` без претензий:

```
typealias ValidatedInput = String
```

```
fun save(v: ValidatedInput): Unit = TODO()
```

```
fun main() {
    val rawInput = "needs validating!"
    save(rawInput)
}
```

Псевдонимы типов не предоставляют дополнительные гарантии во время компиляции

Если ваша цель — обеспечить дополнительную безопасность типов, обходясь минимальными накладными расходами во время выполнения, то следует использовать встроенные классы (см. раздел 4.5). Типы встроенных классов проверяются так же, как и любые другие, поэтому предыдущий пример не скомпилируется из-за несоответствия типов `ValidatedInput` и `String`. Это вынуждает пользователя функции `save` выполнить преобразование из `String` в `ValidatedInput` в явном виде. Благодаря этому вы сможете выявить потенциальные ошибки на ранней стадии:

```
@JvmInline
value class ValidatedInput(val s: String)
```

```
fun save(v: ValidatedInput): Unit = TODO()
```

```
fun main() {
    val rawInput = "needs validating!"
    save(rawInput)
}
```

Этот код не пройдет компиляцию из-за несовпадения типов `ValidatedInput` и `String`

РЕЗЮМЕ

- Обобщения в Kotlin очень похожи на те, что есть в Java, — объявление обобщенной функции или класса происходит таким же образом.
- Как и в Java, аргументы типа для обобщенных типов существуют только во время компиляции.
- Нельзя использовать типы с аргументами типа совместно с оператором `is`, поскольку аргументы типа стираются во время выполнения.
- Типовые параметры встроенных функций могут быть помечены как `reified`. Это позволяет использовать их во время выполнения для осуществления проверки `is` и получения экземпляров `java.lang.Class`.
- Вариативность — способ указать, является ли один из двух обобщенных типов, имеющих один и тот же базовый класс и разные аргументы типа, подтипом или супертипом другого типа, когда один из аргументов типа является подтипом другого типа.
- Можно объявить класс ковариантным по типовому параметру, если параметр используется только в позиции `out`.
- Для контравариантных случаев верно обратное: можно объявить класс контравариантным по типовому параметру, если он используется только в позиции `in`.
- Доступный только для чтения интерфейс `List` в Kotlin объявлен как ковариантный, а это означает, что `List<String>` — подтип типа `List<Any>`.
- Интерфейс функции объявлен как контравариантный по первому типовому параметру и ковариантный по второму, что делает `(Animal) -> Int` подтипом `(Cat) -> Number`.
- Kotlin позволяет задавать вариативность как для обобщенного класса в целом (*вариативность точки объявления*), так и для конкретного случая применения обобщенного типа (*вариативность точки вызова*).
- Синтаксис «звездной» проекции можно использовать, когда аргументы типа точно не известны или неважны.
- Псевдонимы типов позволяют предоставлять альтернативные или сокращенные имена типов. Во время компиляции они разворачиваются до своего базового типа.

12

Аннотации и рефлексия

В этой главе

- ✓ Применение и определение аннотаций.
- ✓ Использование рефлексии для интроспекции классов во время выполнения.
- ✓ Реальный пример проекта на Kotlin.

Мы уже показали большое количество возможностей для работы с классами и функциями. Но все они требуют указания точных имен используемых классов и функций в исходном коде программы. Чтобы вызвать функцию, необходимо знать класс, в котором она была определена, а также ее имя и типы параметров. *Аннотации* и *рефлексия* дают вам возможность выйти за эти рамки и написать код, работающий с произвольными, заранее не известными классами. С помощью аннотаций можно добавлять дополнительные метаданные и семантику к объявлениям. Например, благодаря аннотациям можно указать, что объявление устаревшее (см. подраздел 12.1.1). Кроме того, аннотации применяются для интеграции с компилятором, средой IDE и внешними инструментами (см. подраздел 12.1.2). Еще они используются для создания пользовательской и специфической для библиотеки семантики. Рефлексия же дает возможность анализировать объявления во время выполнения.

Применять аннотации достаточно просто, а вот писать собственные аннотации и код для их обработки — не столь тривиальная задача. Синтаксис для их использования не отличается от Java, а синтаксис для объявления собственных классов аннотаций немного другой. К тому же общая структура API-интерфейсов рефлексии похожа на Java, но различается в деталях.

Как вариант использования аннотаций и рефлексии мы покажем вам реализацию реального проекта — библиотеки сериализации и десериализации JSON под названием JKit. С помощью рефлексии она получает доступ к свойствам произвольных объектов Kotlin во время выполнения, а также создает объекты на основе данных из файлов JSON. Аннотации дают возможность настраивать, как определенные классы и свойства будут сериализоваться и десериализоваться библиотекой.

12.1. ОБЪЯВЛЕНИЕ И ПРИМЕНЕНИЕ АННОТАЦИЙ

Аннотации позволяют связать с объявлением дополнительные *метаданные*. В зависимости от настроек аннотации, к метаданным могут обращаться инструменты для работы с исходным кодом или скомпилированными файлами, а также код во время выполнения.

12.1.1. Применение аннотаций для меток объявлений

В Kotlin, чтобы применить аннотацию, следует поместить ее имя с префиксом `@` в начало аннотируемого объявления. Допускается аннотировать различные объявления в коде, например функции и классы.

Например, если вы используете библиотеку `kotlin.test` с фреймворком JUnit (<https://junit.org/junit5/>), то можете пометить метод тестирования аннотацией `@Test`:

```
import kotlin.test.*
class MyTest {
    @Test
    fun testTrue() {
        assertTrue(1 + 1 == 2)
    }
}
```

Аннотация `@Test` дает фреймворку указание вызывать этот метод при тестировании

В качестве более интересного примера рассмотрим аннотацию `@Deprecated`. Она помечает объявление как устаревшее, указывая на то, что его больше не следует использовать в коде. Обычно это связано с тем, что его заменили другим объявлением или предоставляемая им функциональность больше не поддерживается.

Аннотация `@Deprecated` принимает до трех параметров. Первый, `message`, содержит объяснение причины устаревания. Дополнительный параметр `replaceWith` позволяет предоставить заменяющий шаблон для поддержки плавного перехода на новую версию API. Вдобавок можно указать параметр `level` для обозначения степени устаревания элемента. В нем `WARNING` служит простым уведомлением для пользователей объявления, `ERROR` и `HIDDEN` предотвращают компиляцию нового кода Kotlin под эти API, при этом последний сохраняет бинарную совместимость только для ранее скомпилированного кода.

В примере ниже показано, как можно предоставить аргументы для аннотации (в частности, сообщение об устаревании и образец замены). Аргументы передаются в круглых скобках, как и в обычном вызове функции. Здесь функция `remove`

аннотирована, чтобы указать, что функция `removeAt(index)` будет предпочтительной заменой:

```
@Deprecated("Use removeAt(index) instead.", ReplaceWith("removeAt(index)"))
fun remove(index: Int) { /* ... */ }
```

Если кто-то использует функцию `remove` с таким объявлением, то IntelliJ IDEA не только покажет, какую функцию следует использовать вместо нее (в данном случае `removeAt`), но и предложит быстрое решение для ее автоматической замены (рис. 12.1).



Рис. 12.1. IntelliJ IDEA предлагает быстрое решение для замены вызовов функций, аннотированных с помощью `@Deprecated`

В качестве параметров аннотаций могут выступать только примитивные типы, строки, перечисления, ссылки на классы, другие классы аннотаций и их массивы. Синтаксис для указания аргументов аннотации выглядит следующим образом.

- *Указание класса в качестве аргумента аннотации* — выражение `::class` после имени класса: `@MyAnnotation(MyClass::class)`. Например, библиотека сериализации (описывается далее в этой главе) может предоставить аннотацию, которая ожидает класс в качестве аргумента, чтобы установить соответствие между интерфейсами и реализацией, используемой в процессе десериализации: `@DeserializeInterface(CompanyImpl::class)`.
- *Указание другой аннотации в качестве аргумента* — отсутствие символа `@` перед именем аннотации. Например, параметр `ReplaceWith` в предыдущем примере — это аннотация и необходимо опустить символ `@`, чтобы указать ее в качестве аргумента аннотации `Deprecated`.

- *Указание массива в качестве аргумента* — можно использовать скобки: `@RequestMapping(path = ["/foo", "/bar"])`. Кроме того, указать массив можно с помощью функции `ArrayOf`. (Если использовать класс аннотации из Java, то при необходимости параметр `value` автоматически преобразуется в параметр `vararg`.)

Аргументы аннотаций должны быть известны во время компиляции, поэтому нельзя ссылаться на произвольные свойства в качестве аргументов. Чтобы использовать свойство в качестве аргумента аннотации, необходимо пометить его модификатором `const` — он сообщает компилятору, что свойство является *константой времени компиляции*. Приведенная ниже аннотация `@Timeout` из библиотеки JUnit задает тайм-аут теста в секундах:

```
const val TEST_TIMEOUT = 10L ← Если опустить модификатор const...
```

```
class MyTest {
    @Test
    @Timeout(TEST_TIMEOUT) ← ...то возникнет ошибка времени
    fun testMethod() {      компиляции: «В константных выражениях
        // ...              можно использовать только const val»
    }
}
```

В подразделе 3.2.3 мы обсуждали, что свойства, аннотированные как `const`, должны быть объявлены на верхнем уровне файла или в классе `object` и инициализированы значениями примитивных типов либо `String`. При попытке использовать обычное свойство в качестве аргумента аннотации возникнет следующая ошибка: «В константных выражениях можно использовать только `const val`».

12.1.2. Указание точного объявления, на которое ссылается аннотация: цели аннотации

Во многих случаях одно объявление в исходном коде Kotlin порождает несколько объявлений Java, и каждое из них может содержать аннотации. Например, свойство Kotlin соответствует полю Java, геттеру и, возможно, сеттеру и его параметру. У объявленного в первичном конструкторе свойства есть еще один соответствующий элемент — параметр конструктора. Поэтому может потребоваться указать, какой из данных элементов должен быть аннотирован.

Это можно сделать с помощью объявления *целевой точки использования*. Она помещается между знаком и именем аннотации и отделяется от имени двоеточием. Слово `get` на рис. 12.2 приводит к тому, что к геттеру свойства применяется аннотация `@JvmName`.

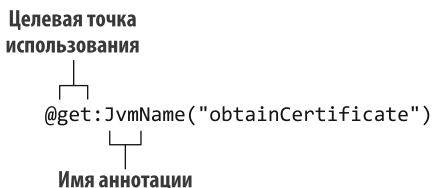


Рис. 12.2. Целевая точка использования (например, `get` или `set`) помещается между знаком `@` и именем аннотации и отделяется от имени двоеточием

Если требуется изменить способ доступа к функции или свойству из Java, то можно использовать аннотацию `@JvmName` (кратко описана в подразделе 3.2.3). Здесь она используется, чтобы сделать функцию `calculate` вызываемой из кода Java с помощью функции `performCalculation()`:

```
@JvmName("performCalculation")
fun calculate(): Int {
    return (2 + 2) - 1
}
```

То же самое можно делать и со свойствами в Kotlin — они автоматически определяют геттер и сеттер (мы обсуждали это в подразделе 2.2.1). Чтобы применить аннотацию `@JvmName` в явном виде к геттеру или сеттеру свойства, нужно использовать выражения `@get:JvmName()` и `@set:JvmName()` соответственно:

```
class CertificateManager {
    @get:JvmName("obtainCertificate") ◀— Установка имени JVM для геттера
    @set:JvmName("putCertificate") ◀— Установка имени JVM для сеттера
    var certificate: String = "-----BEGIN PRIVATE KEY-----"
}
```

С этими аннотациями код Java получит доступ к свойству `certificate` благодаря переименованным функциям `obtainCertificate` и `putCertificate`:

```
class Foo {
    public static void main(String[] args) {
        var certManager = new CertificateManager();
        var cert = certManager.obtainCertificate();
        certManager.putCertificate("-----BEGIN CERTIFICATE-----");
    }
}
```

Если используемая аннотация объявлена в Java, то по умолчанию она будет применяться к соответствующему полю в Kotlin. Определенные в Kotlin аннотации можно объявить так, чтобы они могли непосредственно применяться к свойствам.

Вот полный список поддерживаемых целевых точек использования:

- **property** — свойство (Java-аннотации невозможно применять к этой целевой точке использования);
- **field** — сгенерированное из свойства поле;
- **get** — геттер свойства;
- **set** — сеттер свойства;
- **receiver** — параметр-приемник функции- или свойства-расширения;
- **param** — параметр конструктора;
- **setparam** — параметр сеттера свойств;
- **delegate** — поле, хранящее экземпляр делегата для делегированного свойства;
- **file** — класс с функциями и свойствами верхнего уровня, объявленными в файле.

Любая аннотация с целью `file` должна быть размещена на верхнем уровне файла, перед директивой `package`. К файлам часто применяется аннотация `@JvmName` — она изменяет имя соответствующего класса. В подразделе 3.2.3 был приведен пример: `@file:JvmName("StringFunctions")`.

Kotlin позволяет применять аннотации к произвольным выражениям, а не только к объявлениям классов и функций или типов. В качестве наиболее распространенного примера можно привести аннотацию `@Suppress`. Ее можно использовать для подавления определенного предупреждения компилятора в контексте аннотированного выражения. Ниже приведен пример, в котором объявление локальной переменной аннотируется в целях подавления предупреждения о приведении типа без проверки:

```
fun test(list: List<*>) {
    @Suppress("UNCHECKED_CAST")
    val strings = list as List<String>
    // ...
}
```

СОВЕТ

Обратите внимание, что IntelliJ IDEA и Android studio предлагают функцию подавления в качестве быстрого решения, когда вы нажимаете комбинацию клавиш `Alt+Enter` при появлении предупреждения компилятора. Если выбрать это предложение, то среда вставит в код аннотацию `@Suppress`.

УПРАВЛЕНИЕ JAVA API С ПОМОЩЬЮ АННОТАЦИЙ

Kotlin предоставляет множество аннотаций для управления процессом того, как объявления Kotlin компилируются в байт-код Java и предоставляются для вызовов из Java. Одни из этих аннотаций заменяют соответствующие ключевые слова языка Java (например, аннотация `@Volatile` служит прямой заменой ключевого слова `volatile`). Другие используются для изменения того, как объявления Kotlin отображаются для вызовов из Java:

- `@JvmName` изменяет имя метода или поля Java, сгенерированного из объявления Kotlin;
- `@JvmStatic` можно применить к методам объявления объекта или объекта-компаньона, чтобы отобразить их как статические методы Java;
- `@JvmOverloads` (упоминается в подразделе 3.2.2) предписывает компилятору Kotlin генерировать перегрузки для функции или конструктора, у которых есть значения параметров по умолчанию;
- `@JvmField` можно применить к свойству, чтобы предоставить это свойство как публичное поле Java без геттеров и сеттеров;
- `@JvmRecord` можно применить к `data class`, чтобы объявить класс записи в Java, как показано в подразделе 4.3.2.

Более подробную информацию об использовании этих аннотаций можно найти в комментариях к документации и в разделе `Java interop` онлайн-документации.

12.1.3. Использование аннотаций для настройки сериализации JSON

Один из классических примеров использования аннотаций — настройка сериализации объектов. *Сериализация* — процесс преобразования объекта в двоичное или текстовое представление. После этого его можно сохранить или передать по сети. Обратный процесс, *десериализация*, преобразует такое представление обратно в объект. Одним из наиболее распространенных форматов для сериализации стал JSON. Существует несколько широко используемых библиотек Kotlin для сериализации объектов Kotlin в JSON, в том числе `kotlinx.serialization` (<https://github.com/Kotlin/kotlinx.serialization>) — она была разработана командой Kotlin в JetBrains. Кроме того, существуют такие библиотеки, как Jackson (<https://github.com/FasterXML/jackson>) и Gson (<https://github.com/google/gson>). Они разработаны в целях преобразования объектов Java в JSON и полностью совместимы с Kotlin.

В этой главе мы рассмотрим реализацию библиотеки сериализации исключительно для Kotlin. Она называется JKid. Ее размер позволяет легко прочитать весь исходный код, и мы рекомендуем вам сделать это во время чтения данной главы.

ИСХОДНЫЙ КОД БИБЛИОТЕКИ JKID И УПРАЖНЕНИЯ

Полная реализация доступна как в исходном коде книги, так и онлайн на странице <https://github.com/Kotlin/kotlin-in-action-2e-jkid>. Чтобы изучить реализацию библиотеки и примеры, откройте репозиторий как проект Gradle в своей IDE. Примеры можно найти в проекте в разделе `src/test/kotlin/examples`. Эта библиотека не такая полнофункциональная и гибкая, как `kotlinx.serialization` или другие, но представляет убедительный пример того, как выполнять обработку аннотаций и рефлексии в Kotlin.

Поскольку мы будем рассматривать наиболее значимые части целой библиотеки, то рекомендуем вам открыть проект на своем компьютере во время чтения данной главы. Это даст вам возможность изучить структуру и понять, как сочетаются между собой отдельные аспекты JKid.

В проекте JKid есть серия упражнений. Выполните их после прочтения главы, чтобы убедиться, что поняли концепции. Описание упражнений можно найти в файле проекта `README.md` или прочитать на странице проекта на GitHub.

Начнем с самого простого примера для тестирования библиотеки — сериализации и десериализации экземпляра класса, представляющего `Person`. Сначала экземпляр передается в функцию `serialize`, и она возвращает строку, содержащую его JSON-представление:

```
data class Person(val name: String, val age: Int)

fun main() {
    val person = Person("Alice", 29)
    println(serialize(person))
    // {"age": 29, "name": "Alice"}
}
```

JSON-представление объекта состоит из пар «ключ — значение» — пар имен свойств и их значений для конкретного экземпляра, например `"age": 29`.

Чтобы получить объект Kotlin из JSON-представления, необходимо вызвать функцию `deserialize`. При создании экземпляра из данных JSON следует явно указать класс в качестве аргумента типа, поскольку JSON не хранит типы объектов. В данном случае передается класс `Person`:

```
fun main() {
    val json = """"{"name": "Alice", "age": 29}""""
    println(deserialize<Person>(json))
    // Person(name=Alice, age=29)
}
```

На рис. 12.3 показана эквивалентность между объектом и его JSON-представлением. Обратите внимание, что сериализованный класс может содержать не только значения примитивных типов или строк, но и коллекции и экземпляры других классов объектов значений.

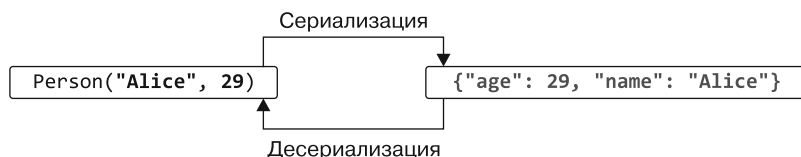


Рис. 12.3. Сериализация и десериализация экземпляра класса `Person`. Объекты Kotlin преобразуются в текстовое представление JSON и обратно

Настроить способ сериализации и десериализации объектов можно с помощью аннотаций. При сериализации объекта в JSON по умолчанию библиотека пытается сериализовать все свойства и использует имена свойств в качестве ключей. Аннотации позволяют изменить настройки по умолчанию. В этом подразделе мы рассмотрим две аннотации, `@JsonExclude` и `@JsonName`, а их реализация будет показана чуть позже.

- Аннотация `@JsonExclude` используется для обозначения свойства, которое должно быть исключено из сериализации и десериализации.
- Аннотация `@JsonName` позволяет указать, что ключом в паре «ключ — значение», представляющей свойство, должна быть заданная строка, а не имя свойства.

Рассмотрим следующий пример. В нем свойство `firstName` аннотируется для изменения ключа, используемого для его представления в JSON. Вдобавок аннотируется свойство `age`, чтобы исключить его из сериализации и десериализации:

```
data class Person(
    @JsonName("alias") val firstName: String,
    @JsonExclude val age: Int? = null
)
```

Обратите внимание, что необходимо указать значение по умолчанию для свойства `age`. В противном случае не получится создать новый экземпляр класса `Person` во время десериализации. На рис. 12.4 показано, как меняется представление экземпляра класса `Person`.



Рис. 12.4. Сериализация и десериализация экземпляра `Person` с аннотацией. Она изменяет поле `firstName` так, чтобы оно сериализовалось в поле `alias` (и десериализовалось из него)

Итак, вы ознакомились с большинством функций, доступных в `JKid`: `serialize()`, `deserialize()`, `@JsonName` и `@JsonExclude`. Теперь приступим к изучению его реализации, начав с объявлений аннотаций.

12.1.4. Создание собственных объявлений аннотаций

В этом разделе вы узнаете, как объявлять аннотации, на примере кода библиотеки `JKid`. Самая простая форма — у аннотации `@JsonExclude`, поскольку она не содержит никаких параметров. Синтаксис выглядит как обычное объявление класса, с модификатором `annotation` перед ключевым словом `class`:

```
annotation class JsonExclude
```

СРАВНЕНИЕ С АННОТАЦИЯМИ JAVA

В Java та же аннотация объявляется следующим образом:

```
/* Java */
public @interface JsonName {
    String value();
}
```

Обратите внимание, что в аннотации Java есть метод `value`, а в аннотации Kotlin — свойство `name`. В Java метод `value` особый: при использовании аннотации необходимо предоставить явные имена для всех указанных атрибутов, кроме `value`.

В Kotlin же применение аннотации представляет собой обычный вызов конструктора. Можно использовать синтаксис именованных аргументов, чтобы сделать имена аргументов явными, — допускается указать имена только для некоторых аргументов или полностью их опустить. В случае с аннотацией `JsonName` выражение `@JsonName(name = "first_name")` идентично выражению на практике `@JsonName("first_name")`, поскольку `name` — первый параметр конструктора `JsonName`. Но если нужно применить аннотацию из Java к элементу Kotlin, то следует использовать синтаксис именованного аргумента для всех аргументов, кроме `value`. Его Kotlin также распознает как особый.

Классы аннотаций используются только для определения структуры метаданных, связанных с объявлениями и выражениями, поэтому не содержат никакого кода. Как следствие, компилятор запрещает указывать тело для класса аннотации.

Параметры для аннотаций объявляются в первичном конструкторе класса. Для этого используется обычный синтаксис объявления первичного конструктора, и все параметры помечаются как `val` (это обязательно для параметров класса аннотации):

```
annotation class JsonName(val name: String)
```

Далее мы обсудим, как контролировать использование аннотаций и применять аннотации к другим аннотациям.

12.1.5. Метааннотации: управление обработкой аннотации

Сам класс аннотации Kotlin может быть аннотирован. К классам аннотаций применяются *метааннотации*. В стандартной библиотеке определены несколько таких метааннотаций, и они управляют тем, как компилятор обрабатывает аннотации. Метааннотации используются и в других фреймворках. Например, многие библиотеки внедрения зависимостей с их помощью обозначают аннотации для идентификации различных внедряемых объектов одного типа.

`@Target` — одна из самых распространенных метааннотаций стандартной библиотеки. Объявления `JsonExclude` и `JsonName` в `JKid` используют ее для определения допустимых целей этих аннотаций. Она применяется следующим образом:

```
@Target(AnnotationTarget.PROPERTY)
annotation class JsonExclude
```

Метааннотация `@Target` определяет типы элементов, к которым может применяться аннотация. Если ее не использовать, то аннотация будет применяться ко всем объявлениям. Для `JKid` в этом нет смысла, поскольку библиотека обрабатывает только аннотации свойств.

Список значений перечисления `AnnotationTarget` дает полный набор возможных целей для аннотации. В него входят классы, файлы, функции, свойства, методы доступа свойств, типы, все выражения и т. д. При необходимости можно объявить несколько целей: `@Target(AnnotationTarget.CLASS, AnnotationTarget.METHOD)`.

Чтобы объявить собственную метааннотацию, используйте `ANNOTATION_CLASS` в качестве ее цели:

```
@Target(AnnotationTarget.ANNOTATION_CLASS)
annotation class BindingAnnotation
```

```
@BindingAnnotation
annotation class MyBinding
```

Обратите внимание: нельзя использовать аннотации с целью `property` из кода Java. Чтобы сделать такую аннотацию пригодной для использования из кода Java, можно добавить вторую цель `AnnotationTarget.FIELD`. В этом случае аннотация будет применяться к свойствам в Kotlin и полям в Java.

АННОТАЦИЯ @RETENTION

Возможно, вы встречали в Java еще одну важную метааннотацию — `@Retention`. С ее помощью можно указать, будет ли объявленная аннотация храниться в файле с расширением `.class` и будет ли она доступна во время выполнения через рефлексия. Java по умолчанию сохраняет аннотации в файлах `.class`, но не делает их доступными во время выполнения. Это означает, что по умолчанию аннотации Java видны только во время компиляции и лишь программам, работающим непосредственно с файлами `.class`, таким как инструменты анализа байт-кода. Обычно большинство аннотаций должны быть доступны во время выполнения, поэтому в Kotlin по умолчанию установлено другое значение, а именно доступность аннотаций — `RUNTIME`. То есть, даже если у аннотаций JKid нет явно указанной доступности, к ним можно получить доступ для рефлексии, что будет показано в подразделе 12.2.3.

12.1.6. Передача классов в качестве параметров аннотации в целях дальнейшего управления поведением

Вы узнали, как определить аннотацию со статическими данными в качестве аргументов. Но иногда требуется нечто иное — возможность ссылаться на класс в качестве метаданных объявления. Это можно сделать, объявив класс аннотации со ссылкой на класс в качестве параметра. В библиотеке JKid это проявляется в аннотации `@DeserializeInterface`. Она дает возможность управлять десериализацией свойств, имеющих тип интерфейса. Создать экземпляр интерфейса напрямую нельзя, поэтому необходимо указать, какой класс будет использоваться в качестве реализации, создаваемой при десериализации.

Приведем простой пример, в котором покажем, как с помощью аннотации `@DeserializeInterface` указать класс, который должен быть использован для реализации интерфейса:

```
interface Company {
    val name: String
}

data class CompanyImpl(override val name: String) : Company

data class Person(
    val name: String,
    @DeserializeInterface(CompanyImpl::class) val company: Company
)
```

При десериализации каждый раз, когда JKid считывает вложенный объект `company` экземпляра `Person`, она создает и десериализует экземпляр `CompanyImpl` и сохраняет его в свойстве `company`. Чтобы указать это, выражение `CompanyImpl::class` применяется в качестве аргумента аннотации `@DeserializeInterface`. Обычно для ссылки на класс используется его имя с последующим ключевым словом `::class`.

Теперь посмотрим, как объявляется сама аннотация. Ее единственным аргументом будет ссылка на класс, как в выражении `@DeserializeInterface(CompanyImpl::class)`:

```
annotation class DeserializeInterface(val targetClass: KClass<out Any>)
```

Тип `KClass` используется для хранения ссылок на классы Kotlin. Его возможности по работе с этими классами будут показаны в разделе 12.2.

Типовой параметр в `KClass` определяет, на какие классы Kotlin может указывать данная ссылка. Например, у класса `CompanyImpl::class` тип `KClass<CompanyImpl>`, который представляет собой подтип типа параметра аннотации (рис. 12.5).

Если бы мы написали `KClass<Any>` без модификатора `out`, то передать `CompanyImpl::class` в качестве аргумента не получилось бы. Единственным доступным аргументом стал бы класс `Any::class`.

Ключевое слово `out` указывает, что можно ссылаться на классы, расширяющие `Any`, а не только на сам класс `Any`.

В следующем подразделе показана еще одна аннотация. Она принимает в качестве параметра ссылку на обобщенный класс.

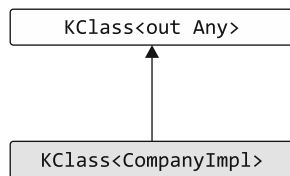


Рис. 12.5. Тип аргумента аннотации `CompanyImpl::class` (`KClass<CompanyImpl>`) — это подтип типа параметра аннотации (`KClass<out Any>`)

12.1.7. Обобщенные классы как параметры аннотаций

По умолчанию `JKid` сериализует свойства непримитивных типов как вложенные объекты. Но это поведение можно изменить и предоставить собственную логику сериализации для некоторых значений.

Аннотация `@CustomSerializer` принимает в качестве аргумента ссылку на класс пользовательского сериализатора. Он должен реализовывать интерфейс `ValueSerializer` и обеспечивать преобразование объекта Kotlin в его JSON-представление и соответственно JSON-значения обратно в объект Kotlin:

```
interface ValueSerializer<T> {
    fun toJsonValue(value: T): Any?
    fun fromJsonValue(jsonValue: Any?): T
}
```

Предположим, вам нужно поддерживать сериализацию дат и вы создали для этого собственный класс `DateSerializer` с реализацией интерфейса `ValueSerializer<Date>`.

Этот класс приведен в качестве примера в исходном коде `JKid` (<http://mng.bz/e1vQ>). Он применяется к классу `Person` следующим образом:

```
data class Person(
    val name: String,
    @CustomSerializer(DateSerializer::class) val birthDate: Date
)
```

Теперь рассмотрим объявление аннотации `@CustomSerializer`. Класс `ValueSerializer` является обобщенным и определяет типовой параметр, поэтому необходимо предоставлять значение аргумента типа при каждой ссылке на тип. Типы свойств данной аннотации вам неизвестны, поэтому вы можете использовать в качестве аргумента «звездную» проекцию (см. подраздел 11.3.6):

```
annotation class CustomSerializer(  
    val serializerClass: KClass<out ValueSerializer<*>>  
)
```

На рис. 12.6 показан тип параметра `serializerClass` и объясняются его различные части. Необходимо убедиться, что аннотация может ссылаться только на классы, реализующие интерфейс `ValueSerializer`. Например, написать `@CustomSerializer(Date::class)` нельзя, поскольку класс `Date` не реализует интерфейс `ValueSerializer`.

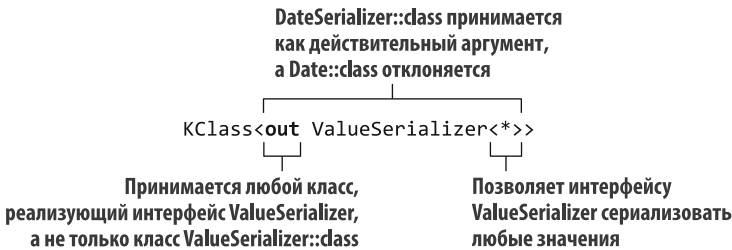


Рис. 12.6. Тип параметра аннотации `serializerClass`. Только ссылки на классы, расширяющие интерфейс `ValueSerializer`, будут действительными аргументами аннотации

Эта тема может показаться сложной, но в ней есть и положительные стороны: можно применять один и тот же паттерн при каждом использовании класса в качестве аргумента аннотации. Можно написать `KClass<out YourClassName>`, и если у параметра `YourClassName` есть собственные аргументы типа, то заменить их на `*`.

Вы ознакомились со всеми важными аспектами объявления и применения аннотаций в Kotlin. Следующий шаг — выяснить, как получить доступ к данным, хранящимся в аннотациях. Для этого предстоит использовать *рефлексию*.

12.2. РЕФЛЕКСИЯ: ИНТРОСПЕКЦИЯ ОБЪЕКТОВ KOTLIN ВО ВРЕМЯ ВЫПОЛНЕНИЯ

Проще говоря, рефлексия — способ *динамического* доступа к свойствам и методам объектов во время выполнения программы. При этом заранее неизвестно, что это за свойства. Обычно при обращении к методу или свойству объекта исходный код программы ссылается на определенное объявление, а компилятор *статически* разрешает эту ссылку и убеждается, что объявление существует. Но иногда требуется написать код, который может работать с объектами любого типа или в котором имена запрашиваемых методов и свойств становятся известны только во время выполнения. Библиотека сериализации — отличный пример такого кода. Она должна

быть способна сериализовать любой объект в JSON, поэтому не может ссылаться на конкретные классы и свойства. Вот в такой ситуации и применяется рефлексия.

При работе с рефлексией в Kotlin обычно приходится взаимодействовать с API рефлексии Kotlin. Он определен в пакетах `kotlin.reflect` и `kotlin.reflect.full`. Они предоставляют доступ ко всем концепциям Kotlin, таким как классы данных, свойства и типы, допускающие значение `null`. Важно отметить, что API рефлексии Kotlin не ограничивается классами Kotlin — можно использовать тот же API для доступа к классам на любом языке JVM.

В качестве запасного варианта можно использовать стандартную рефлексию Java из пакета `java.lang.reflect`. Классы Kotlin компилируются в обычный байт-код Java, поэтому API рефлексии Java прекрасно поддерживает их. В частности, это означает, что библиотеки Java с API рефлексии полностью совместимы с кодом Kotlin.

ПРИМЕЧАНИЕ

Чтобы уменьшить размер библиотеки времени выполнения на платформах, где это важно, таких как Android, API рефлексии Kotlin упакован в отдельный JAR-файл `kotlin-reflect.jar`. По умолчанию он не добавляется в качестве зависимости в новые проекты. При использовании API рефлексии Kotlin необходимо убедиться, что библиотека добавлена в качестве зависимости. Идентификатор группы/артефакта Maven для этой библиотеки — `org.jetbrains.kotlin:kotlin-reflect`.

В этом разделе будет показано, как JKid использует API рефлексии. Сначала проведем обзор сериализации, поскольку она более понятна и проста для объяснения, а затем перейдем к разбору и десериализации JSON. Но сначала поговорим о содержимом API рефлексии.

12.2.1. API рефлексии Kotlin: `KClass`, `KCallable`, `KFunction` и `KProperty`

Основной точкой входа в API рефлексии Kotlin служит класс `KClass`. С его помощью можно перечислить все объявления, содержащиеся в классе, его суперклассах и т. д., и получить доступ ко всем этим объявлениям. Чтобы получить экземпляр класса `KClass`, напишите `MyClass::class`. А для получения класса объекта `myObject` во время выполнения напишите `myObject::class`:

```
import kotlin.reflect.full.*

class Person(val name: String, val age: Int)

fun main() {
    val person = Person("Alice", 29)
    val kClass = person::class  ◀ Возврат экземпляра KClass<out Person>
    println(kClass.simpleName)
    // Person
    kClass.memberProperties.forEach { println(it.name) }
    // age
    // name
}
```

Этот простой пример выводит на экран имя класса и имена его свойств и использует расширение `memberProperties` для сбора всех нерасширяемых свойств данного класса и всех его суперклассов.

Объявление `KClass` содержит множество полезных методов, позволяющих получить доступ к содержимому класса:

```
interface KClass<T : Any> {
    val simpleName: String?
    val qualifiedName: String?
    val members: Collection<KCallable*>
    val constructors: Collection<KFunction<T>>
    val nestedClasses: Collection<KClass*>
    // ...
}
```

Многие другие полезные функциональные возможности `KClass`, показанные в предыдущем примере, в том числе `memberProperties`, объявляются как расширения. Полный список методов `KClass` (включая расширения) можно посмотреть в стандартной библиотеке по ссылке (<http://mng.bz/em4i>).

ПРИМЕЧАНИЕ

Можно ожидать, что свойства `simpleName` и `qualifiedName` не будут допускать значение `null`. Однако стоит вспомнить описание способов использования выражения `object` в целях создания анонимных объектов, приведенное в подразделе 4.4.4. Эти объекты все еще представляют собой экземпляры класса, однако данный класс является анонимным. Как таковой, он не содержит свойства `simpleName` и `qualifiedName`. Доступ к этим полям из экземпляра `KClass` вернет значение `null`.

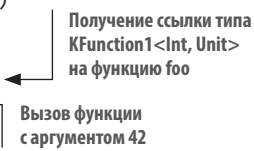
Можно заметить, что список всех членов класса — `members` и он представляет собой коллекцию экземпляров `KCallable`. А она является суперинтерфейсом для функций и свойств, в котором объявлен метод `call` для вызова соответствующей функции или геттера свойства:

```
interface KCallable<out R> {
    fun call(vararg args: Any?): R
    // ...
}
```

Аргументы функции предоставляются в виде списка `vararg`. Во фрагменте кода ниже показано, как можно использовать метод `call` для вызова функции через рефлексия:

```
fun foo(x: Int) = println(x)

fun main() {
    val kFunction = ::foo
    kFunction.call(42)
    // 42
}
```



Синтаксис `::foo` уже использовался в подразделе 5.1.5, а теперь видно, что значение этого выражения — экземпляр класса `KFunction` из API рефлексии. Для вызова

указанной функции используется метод `KCallable.call`. В этом случае необходимо указать единственный аргумент: 42. При попытке вызвать функцию с неправильным количеством аргументов, например `kFunction.call()`, будет выброшено исключение времени выполнения: `IllegalArgumentException: Callable expects 1 argument, but 0 were provided` (`IllegalArgumentException: Callable` ожидает 1 аргумент, но предоставлено 0).

Но в этом случае для вызова функции можно использовать более специфический метод. Типом выражения `::foo` будет `KFunction1<Int, Unit>`, и он содержит информацию о типах параметров и возврата. Имя экземпляра `KFunction1` обозначает, что эта функция принимает один параметр. Для вызова функции через данный интерфейс используется метод `invoke`. Он принимает фиксированное количество аргументов (в данном случае один), а их типы соответствуют типовым параметрам интерфейса `KFunction1`. Параметр имеет тип `Int`, а возвращаемым типом функции является `Unit`. Вызвать `kFunction` можно и напрямую (в разделе 13.3 рассказывается, почему можно вызвать `kFunction`, не указывая явно метод `invoke`):

```
import kotlin.reflect.KFunction2

fun sum(x: Int, y: Int) = x + y

fun main() {
    val kFunction: KFunction2<Int, Int, Int> = ::sum
    println(kFunction.invoke(1, 2) + kFunction(3, 4))
    // 10
    kFunction(1)
    // ERROR: No value passed for parameter p2
}
```

Использование метода `invoke` вместо `call` в классе `kFunction` предотвращает случайную передачу неправильного количества аргументов функции — код не будет компилироваться. Поэтому, если есть класс `KFunction` определенного типа с известными параметрами и типом возврата, то предпочтительнее использовать его метод `invoke`. Использование метода `call` — общий подход, он работает для всех типов функций, но не обеспечивает безопасность типов.

КАК И ГДЕ ОПРЕДЕЛЯЮТСЯ ИНТЕРФЕЙСЫ KFUNCTIONN

Такие типы, как `KFunction1`, представляют функции с различным количеством параметров. Каждый тип расширяет `KFunction` и добавляет один дополнительный член класса — `invoke`, имеющий соответствующее количество параметров. Например, `KFunction2` объявляет оператор `fun invoke(p1: P1, p2: P2): R`, где `P1` и `P2` представляют собой типы параметров функции, а `R` — тип возврата.

Эти типы функций — *синтетические типы, генерируемые компилятором*, и их объявления в пакете `kotlin.reflect` отсутствуют. Это означает, что не получится использовать интерфейс для функции с любым количеством параметров, не имея искусственных ограничений на возможное количество типовых параметров функции.

Можно вызвать метод `call` для экземпляра класса `KProperty`, и он вызовет геттер свойства. Но интерфейс свойств предоставляет улучшенный способ получить значение свойства — метод `get`.

Чтобы получить доступ к методу `get`, необходимо использовать правильный интерфейс для свойства, в зависимости от того, как оно объявлено. Доступные только для чтения и изменяемые свойства верхнего уровня представлены экземплярами интерфейсов `KProperty0` и `KMutableProperty0` соответственно. И у обоих есть метод `get` без аргументов:

```
var counter = 0

fun main() {
    val kProperty = ::counter
    kProperty.setter.call(21)
    println(kProperty.get())
    // 21
}
```

Свойство члена класса представлено экземпляром `KProperty1` или `KMutableProperty1`. Они оба предоставляют метод `get` с одним аргументом. Чтобы получить доступ к его значению, необходимо указать экземпляр объекта, для которого требуется получить значение. В примере ниже ссылка на свойство хранится в переменной `memberProperty`. Затем выполняется вызов функции `memberProperty.get(person)`, что позволяет получить значение этого свойства для конкретного экземпляра `person`. Так что если `memberProperty` ссылается на свойство `age` класса `Person`, то функция `memberProperty.get(person)` представляет собой способ динамически получить значение свойства `person.age`. Вы уже сталкивались с этой концепцией в подразделе 5.1.6:

```
class Person(val name: String, val age: Int)

fun main() {
    val person = Person("Alice", 29)
    val memberProperty = Person::age
    println(memberProperty.get(person))
    // 29
}
```

Обратите внимание, что `KProperty1` — обобщенный класс. Тип переменной `memberProperty` — `KProperty1<Person, Int>`, где первый типовой параметр обозначает тип приемника, а второй — тип свойства. Таким образом, можно вызвать его метод `get` только с приемником правильного типа. А вызов функции `memberProperty.get("Alice")` не пройдет компиляцию.

Вдобавок стоит отметить, что рефлексию можно использовать только для доступа к свойствам, определенным на верхнем уровне или в классе, но не к локальным переменным функции. Если определить локальную переменную `x` и попытаться получить ссылку на нее с помощью выражения `::x`, то возникнет ошибка

компиляции: `References to variables aren't supported yet` (Ссылки на переменные пока не поддерживаются).

На рис. 12.7 показана иерархия интерфейсов, используемых для доступа к элементам исходного кода во время выполнения программы. Аннотировать можно все объявления, поэтому интерфейсы с объявлениями во время выполнения, такие как `KClass`, `KFunction` и `KParameter`, расширяют интерфейс `KAnnotatedElement`. `KClass` используется для представления классов и объектов. `KProperty` может представлять любое свойство, в то время как его подкласс, `KMutableProperty`, представляет изменяющееся свойство. Оно объявляется с помощью ключевого слова `var`.

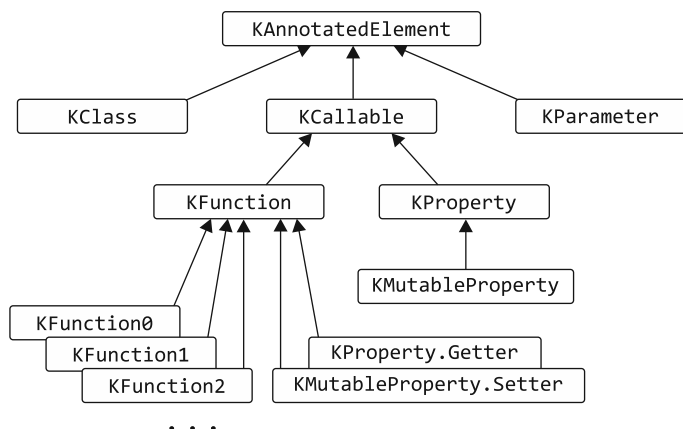


Рис. 12.7. Иерархия интерфейсов в API рефлексии Kotlin

Для работы с методами доступа свойств как с функциями (например, если требуется получить их аннотации) используются специальные интерфейсы `Getter` и `Setter`, объявленные в `Property` и `KMutableProperty`. Оба интерфейса для методов доступа расширяют интерфейс `KFunction`. Для простоты мы опустили на рисунке конкретные интерфейсы для свойств типа `KProperty0`.

Теперь, когда вы познакомились с основами API рефлексии Kotlin, мы переходим к изучению реализации библиотеки `JKid`.

12.2.2. Сериализация объектов с помощью рефлексии

Сначала вспомним объявление функции сериализации в `JKid`:

```
fun serialize(obj: Any): String
```

Эта функция принимает объект и возвращает его JSON-представление в виде строки. Она соберет полученный JSON в экземпляр класса `StringBuilder`. По мере сериализации свойств объекта и их значений функция будет добавлять их в этот объект `StringBuilder`. Чтобы сделать вызовы функции `append` более краткими, поместим

реализацию в функцию-расширение класса `StringBuilder`. И тогда можно будет удобно вызывать метод `append` без квалификатора:

```
private fun StringBuilder.serializeObject(x: Any) {
    append(/*...*/)
}
```

Преобразование параметра функции в приемник функции-расширения — пространственный паттерн в коде Kotlin, и мы подробно рассмотрим его в подразделе 13.2.1. Обратите внимание, что метод `serializeObject` не расширяет API класса `StringBuilder` — он выполняет операции, актуальные только для этого контекста, и поэтому помечен как `private`. Это позволяет гарантировать, что метод не будет использоваться в других местах. Он объявляется как расширение, что позволяет выделить определенный объект как основной для данного блока кода и облегчить работу с ним.

Следовательно, функция `serialize` делегирует всю работу методу `serializeObject`:

```
fun serialize(obj: Any): String = buildString { serializeObject(obj) }
```

В подразделе 5.4.1 рассказывалось, что метод `buildString` создает интерфейс `StringBuilder` и позволяет заполнить его содержимым в лямбде. В этом случае содержимое предоставляется с помощью вызова метода `serializeObject(obj)`.

Теперь обсудим поведение функции сериализации. По умолчанию она сериализует все свойства объекта. Примитивные типы и строки будут сериализованы в JSON как числа, логические и строковые значения — в зависимости от ситуации. Коллекции будут сериализованы в виде массивов JSON. Свойства других типов будут сериализованы как вложенные объекты. Это поведение можно настроить с помощью аннотаций, как в предыдущем разделе.

Теперь перейдем к реализации функции `serializeObject`, и вы сможете увидеть API рефлексии в реальном сценарии.

ПРИМЕЧАНИЕ

В репозитории эта функция называется `serializeObjectWithoutAnnotation`, и мы переработаем ее.

Листинг 12.1. Сериализация объекта

```
private fun StringBuilder.serializeObject(obj: Any) {
    val kClass = obj::class as KClass<Any>  ← Получение класса KClass для объекта
    val properties = kClass.memberProperties ← Получение всех свойств класса

    properties.joinToStringBuilder(
        this, prefix = "{", postfix = "}" { prop ->
            serializeString(prop.name)  ← Получение имени свойства
            append(": ")
            serializePropertyValue(prop.get(obj))  ← Получение значения свойства
        }
    )
}
```

Реализация этой функции должна быть понятна: каждое свойство класса сериализуется по очереди. Полученный результат JSON будет выглядеть следующим образом: `{ "prop1": value1, "prop2": value2 }`. Функция `joinToStringBuilder` обеспечивает разделение свойств запятыми, `serializeString` экранирует специальные символы, как того требует формат JSON. Функция `serializePropertyValue` проверяет, чем является значение: примитивным значением, строкой, коллекцией или вложенным объектом, и соответствующим образом сериализует его содержимое.

В предыдущем подразделе мы рассмотрели способ получения значения экземпляра `KProperty` — метод `get`. В том случае мы работали со ссылкой на член класса `Person::age` типа `KProperty1<Person, Int>`, что позволяет компилятору узнать точные типы приемника и значения свойства. А в этом примере точные типы неизвестны, поскольку перечисляются все свойства класса объекта. Следовательно, тип переменной `prop` — `KProperty1<Any, *>`, и метод `prop.get(obj)` возвращает значение типа `Any?`. Проверки типа приемника во время компиляции не выполняются, но поскольку передается тот же объект, из которого был получен список свойств, то тип приемника будет правильным.

12.2.3. Настройка сериализации с помощью аннотаций

Ранее в этой главе вы познакомились с определениями аннотаций для настройки процесса сериализации JSON. В частности, мы обсудили аннотации `@JsonExclude`, `@JsonName` и `@CustomSerializer`. Пришло время рассказать вам, как эти аннотации обрабатываются функцией `serializeObject`.

Начнем с аннотации `@JsonExclude`. Она позволяет исключить некоторые свойства из сериализации. Рассмотрим изменения в реализации функции `serializeObject` для поддержки этого процесса.

Напомним, что для получения всех свойств членов класса используется свойство-расширение `memberProperties` экземпляра `KClass`. Но теперь задача усложняется: свойства с аннотацией `@JsonExclude` необходимо отфильтровать. Посмотрим, как это осуществить.

Интерфейс `KAnnotatedElement` определяет свойство `annotations`, коллекцию всех экземпляров аннотаций (с сохранением во время выполнения), применявшихся к элементу в исходном коде. Поскольку `KProperty` расширяет интерфейс `KAnnotatedElement`, то доступ ко всем аннотациям свойств осуществляется с помощью метода `property.annotations`.

Но код, отвечающий за исключение свойств, на самом деле должен найти конкретную аннотацию. В этом случае используется функция `findAnnotation`, и вызывается она для элемента `KAnnotatedElement`. Она возвращает аннотацию типа, указанного в качестве аргумента, если такая аннотация существует.

Комбинация `findAnnotation` с функцией стандартной библиотеки `filter` отфильтрует свойства с аннотацией `@JsonExclude`:

```
val properties = kClass.memberProperties
    .filter { it.findAnnotation<JsonExclude>() == null }
```

Следующая аннотация — `@JsonName`. В качестве напоминания повторим ее объявление и пример использования:

```
annotation class JsonName(val name: String)
```

```
data class Person(
    @JsonName("alias") val firstName: String,
    val age: Int
)
```

В данном случае интерес представляет не только наличие свойства, но и его аргумент — имя, которое должно использоваться для аннотируемого свойства в JSON. И снова обратимся к функции `findAnnotation`:

```
val jsonNameAnn = prop.findAnnotation<JsonName>()
val propName = jsonNameAnn?.name ?: prop.name
```

Если у свойства нет аннотации `@JsonName`, то переменная `jsonNameAnn` равна `null` и `prop.name` остается в качестве имени свойства в JSON. Если свойство аннотировано, то вместо имени по умолчанию используется указанное имя.

Рассмотрим сериализацию экземпляра объявленного ранее класса `Person`. Во время сериализации свойства `firstName` переменная `jsonNameAnn` содержит соответствующий экземпляр аннотации класса `JsonName`. Следовательно, `jsonNameAnn?.name` возвращает «не `null`»-значение `"alias"`, используемое в качестве ключа в JSON. Когда свойство `age` сериализовано, аннотация не обнаружена, поэтому в качестве ключа используется имя свойства `age`. Таким образом, сериализованный вывод JSON для объекта `Person("Alice", 35)` — это `{ "alias": "Alice", "age": 35 }`.

Объединим описанные изменения и посмотрим на получившуюся реализацию логики сериализации, показанную в листинге 12.2.

Листинг 12.2. Сериализация объекта с фильтрацией свойств

```
private fun StringBuilder.serializeObject(obj: Any) {
    (obj::class as KClass<Any>)
        .memberProperties
        .filter { it.findAnnotation<JsonExclude>() == null }
        .joinToStringBuilder(this, prefix = "{", postfix = "}") {
            serializeProperty(it, obj)
        }
}
```

Теперь свойства с аннотацией `@JsonExclud` отфильтровываются. Вдобавок мы вынесли логику сериализации свойств в отдельную функцию `serializeProperty` (листинг 12.3).

Листинг 12.3. Сериализация одного свойства

```
private fun StringBuilder.serializeProperty(
    prop: KProperty1<Any, *>, obj: Any
) {
    val jsonNameAnn = prop.findAnnotation<JsonName>()
    val propName = jsonNameAnn?.name ?: prop.name
    serializeString(propName)
    append(": ")

    serializePropertyValue(prop.get(obj))
}
```

Имя свойства обрабатывается в соответствии с описанной ранее аннотацией `@JsonName`.

Далее реализуем оставшуюся аннотацию — `@CustomSerializer`. Ее реализация базируется на функции `getSerializer`. Она возвращает экземпляр `ValueSerializer`, зарегистрированный с помощью аннотации `@CustomSerializer`. Например, если объявить класс `Person` и вызвать функцию `getSerializer()` при сериализации свойства `birthDate`, то будет возвращен экземпляр класса `DateSerializer`:

```
import java.util.Date

data class Person(
    val name: String,
    @CustomSerializer(DateSerializer::class) val birthDate: Date
)
```

Напомним, как объявляется аннотация `@CustomSerializer`, чтобы вам было проще понять реализацию функции `getSerializer`:

```
annotation class CustomSerializer(
    val serializerClass: KClass<out ValueSerializer<*>>
)
```

Функция `getSerializer` реализуется следующим образом.

Листинг 12.4. Получение сериализатора значений для свойства

```
fun KProperty<*>.getSerializer(): ValueSerializer<Any?>? {
    val customSerializerAnn = findAnnotation<CustomSerializer>()
    ?: return null
    val serializerClass = customSerializerAnn.serializerClass

    val valueSerializer = serializerClass.objectInstance
    ?: serializerClass.createInstance()
    @Suppress("UNCHECKED_CAST")
    return valueSerializer as ValueSerializer<Any?>
}
```

`getSerializer` — функция-расширение класса `KProperty`, поскольку функция работает со свойством. Она вызывает функцию `findAnnotation`, чтобы получить экземпляр аннотации `@CustomSerializer`, если таковой существует. Ее аргумент, `serializerClass`, указывает класс, для которого нужно получить экземпляр.

Наиболее интересным здесь будет способ работы с классами и с объектами (объектами-одиночками Kotlin) в качестве значений аннотации `@CustomSerializer`. Оба они представлены классом `KClass`. Разница в том, что у объектов есть «не null»-значение свойства `objectInstance` и его можно использовать для доступа к экземпляру объекта-одиночки, созданному для класса `object`. Например, интерфейс `DateSerializer` объявлен как `object`, поэтому его свойство `objectInstance` хранит объект-одиночку экземпляра `DateSerializer`. Этот экземпляр будет использоваться для сериализации всех объектов, и метод `createInstance` не будет вызываться. Если `KClass` представляет собой обычный класс, то новый экземпляр создается с помощью метода `createInstance()`.

И наконец, интерфейс `getSerializer` можно использовать в реализации функции `serializeProperty`. В листинге 12.5 показана окончательная версия функции.

Листинг 12.5. Сериализация свойства с поддержкой пользовательского сериализатора

```
private fun StringBuilder.serializeProperty(
    prop: KProperty1<Any, *>, obj: Any
) {
    val jsonNameAnn = prop.findAnnotation<JsonName>()
    val propName = jsonNameAnn?.name ?: prop.name
    serializeString(propName)
    append(": ")

    val value = prop.get(obj)
    val jsonValue = prop.getSerializer()?.toJsonValue(value)
    // ?: value
    serializePropertyValue(jsonValue)
}
```

Применение пользовательского сериализатора для свойства, если оно существует

В противном случае используется значение свойства, как и ранее

Функция `serializeProperty` использует сериализатор для преобразования значения свойства в JSON-совместимый формат с помощью функции `toJsonValue`. Если у свойства нет пользовательского сериализатора, то используется значение свойства.

Мы рассмотрели реализацию в библиотеке сериализации JSON и переходим к синтаксическому анализу и десериализации. Для нее требуется гораздо больше кода, поэтому мы не будем рассматривать его весь. Но мы покажем структуру реализации и объясним, как рефлексия используется для десериализации объектов.

12.2.4. Синтаксический анализ JSON и десериализация объектов

Начнем со второй части истории — реализации логики десериализации. Для начала напомним, что API, как и API для сериализации, состоит из одной функции. Во время выполнения функция должна иметь доступ к типовому параметру, чтобы создать правильный итоговый объект при десериализации. Это означает (см. раздел 11.2),

что типовой параметр необходимо пометить как `reified`, то есть функция должна быть помечена как `inline`:

```
inline fun <reified T: Any> deserialize(json: String): T
```

Приведем пример ее использования:

```
data class Author(val name: String)
data class Book(val title: String, val author: Author)

fun main() {
    val json = """"{"title": "Catch-22", "author": {"name": "J. Heller"}}""""
    val book = deserialize<Book>(json)
    println(book)
    // Book(title=Catch-22, author=Author(name=J. Heller))
}
```

Тип десериализуемого объекта передается в качестве параметра вещественного типа в функцию `deserialize`, а из нее поступает новый экземпляр объекта.

Десериализация JSON — более сложная задача, чем сериализация, поскольку содержит синтаксический анализ входной JSON-строки в дополнение к использованию рефлексии для получения доступа к внутренним данным объекта. Десериализатор JSON в библиотеке JKid реализован достаточно традиционно и состоит из трех основных этапов: лексического анализатора, или *лексера*, синтаксического анализатора, или *парсера*, и собственно компонента десериализации.

Лексический анализ разбивает входную строку из символов на список *лексем* (или *токенов*). Существует два вида лексем: *символьные* представляют собой символы со специальными значениями в синтаксисе JSON (запятая, двоеточие, фигурные и квадратные скобки), *лексемы значений* соответствуют константам строк, чисел, логических значений и констант `null`. Левая фигурная скобка (`{`), строковое значение (`"Catch-22"`) и целочисленное значение (`42`) — примеры разных лексем.

Парсер зачастую отвечает за преобразование обычного списка лексем в структурированное представление. Его задача в библиотеке JKid — понять высокоуровневую структуру JSON и преобразовать отдельные лексемы в семантические элементы, поддерживаемые в JSON: пары «ключ — значение», объекты и массивы.

Интерфейс `JsonObject` отслеживает десериализуемый в данный момент объект или массив. Парсер вызывает соответствующие методы при обнаружении новых свойств текущего объекта (простые значения, составные свойства или массивы) (листинг 12.6).

Листинг 12.6. Интерфейс обратного вызова парсера JSON

```
interface JsonObject {
    fun setSimpleProperty(propertyName: String, value: Any?)
    fun createObject(propertyName: String): JsonObject

    fun createArray(propertyName: String): JsonObject
}
```

Параметр `propertyName` в этих методах получает JSON-ключ. Таким образом, когда синтаксический анализатор встречает свойство `author` с объектом в качестве значения, вызывается метод `createObject("author")`.

Для свойств с простыми значениями вызывается функция `setSimpleProperty`, при этом фактическое значение токена передается в качестве аргумента `value`. Реализации `JsonObject` отвечают за создание новых объектов для свойств и хранение ссылок на них во внешнем объекте.

На рис. 12.8 показаны входные и выходные данные каждого этапа лексического и синтаксического анализа при десериализации образца строки. И снова лексический анализ разбивает входную строку на список лексем. Затем синтаксический анализ (парсер) обрабатывает этот список лексем и вызывает соответствующий метод интерфейса `JsonObject` для каждого нового значимого элемента.



Рис. 12.8. Процесс синтаксического анализа JSON: сначала лексер получает входной текст и разделяет его на лексемы. Затем парсер обрабатывает различные семантические элементы. И наконец, десериализатор превращает их в конечные объекты Kotlin

Затем десериализатор предоставляет реализацию для интерфейса `JsonObject`. Она постепенно создает новый экземпляр соответствующего типа. Ей необходимо найти соответствие между свойствами класса и ключами JSON (`title`, `author` и `name` на рис. 12.8) и создать вложенные значения объектов (экземпляр класса `Author`). И только после этого будет создан экземпляр нужного класса (`Book`).

Библиотека `JKid` предназначена для использования с классами данных и поэтому передает все пары «имя — значение» из файла JSON в качестве параметров в конструктор десериализуемого класса. Она не поддерживает установку свойств для экземпляров объектов после их создания. Это означает, что нужно где-то хранить данные во время чтения их из JSON, прежде чем начнется создание фактического объекта.

Требование сохранять компоненты перед созданием объекта похоже на традиционный паттерн «*Строитель*», с той лишь разницей, что конструкторы обычно

предназначены для создания объектов определенного типа. В случае с десериализацией решение должно быть полностью универсальным. Забавы ради мы используем термин «семя» (seed) для обозначения реализации. В JSON необходимо создавать различные типы составных структур: объекты, коллекции и карты. Классы `ObjectSeed`, `ObjectListSeed` и `ValueListSeed` отвечают за правильное создание объектов и списков составных объектов или простых значений. Создание карт мы оставили вам в качестве упражнения.

Базовый интерфейс `Seed` расширяет интерфейс `JsonObject` и предоставляет дополнительный метод `spawn`, позволяющий получить итоговый экземпляр после завершения процесса создания. К тому же в нем объявлен метод `createCompositeProperty` для создания вложенных объектов и списков (они используют одну и ту же логику создания экземпляров через «семена») (листинг 12.7).

Листинг 12.7. Интерфейс для создания объектов из данных JSON

```
interface Seed : JsonObject {
    fun spawn(): Any?

    fun createCompositeProperty(
        propertyName: String,
        isList: Boolean
    ): JsonObject

    override fun createObject(propertyName: String) =
        createCompositeProperty(propertyName, false)

    override fun createArray(propertyName: String) =
        createCompositeProperty(propertyName, true)

    // ...
}
```

Функцию `spawn` можно считать аналогом `build` — метода для возврата значения результата. Он возвращает созданный объект для класса `ObjectSeed` и итоговый список для классов `ObjectListSeed` или `ValueListSeed`. Мы не будем подробно описывать, как происходит десериализация списков. Вместо этого сосредоточимся на создании объектов — более сложном процессе, который лучше подходит для демонстрации общей идеи.

Но прежде обсудим главную функцию `deserialize`, показанную в листинге 12.8. Она выполняет всю работу по десериализации значения.

Листинг 12.8. Функция десериализации верхнего уровня

```
fun <T: Any> deserialize(json: Reader, targetClass: KClass<T>): T {
    val seed = ObjectSeed(targetClass, ClassInfoCache())
    Parser(json, seed).parse()
    return seed.spawn()
}
```


Чтобы начать синтаксический анализ, вы создаете экземпляр класса `ObjectSeed`, предназначенный для хранения свойств десериализуемого объекта. Затем вызываете парсер, и ему передается считыватель входного потока `json`. Как только поток выполнения достигает конца входных данных, вызывается функция `spawn` для создания результирующего объекта.

Теперь сосредоточимся на реализации класса `ObjectSeed`, который хранит состояние создаваемого объекта (листинг 12.9). Класс `ObjectSeed` принимает ссылку на итоговый класс и объект `classInfoCache`, содержащий кэшированную информацию о свойствах класса. Она будет использоваться позже для создания экземпляров этого класса. `ClassInfoCache` и `ClassInfo` — вспомогательные классы. Их мы обсудим в следующем подразделе.

Листинг 12.9. Десериализация объекта

```
class ObjectSeed<out T: Any>(  
    targetClass: KClass<T>,  
    override val classInfoCache: ClassInfoCache  
) : Seed {  
  
    private val classInfo: ClassInfo<T> =  
        classInfoCache[targetClass]  
  
    private val valueArguments = mutableMapOf<KParameter, Any?>()  
    private val seedArguments = mutableMapOf<KParameter, Seed>()  
  
    private val arguments: Map<KParameter, Any?>  
    get() = valueArguments +  
        seedArguments.mapValues { it.value.spawn() }  
  
    override fun setSimpleProperty(propertyName: String, value: Any?) {  
        val param = classInfo.getConstructorParameter(propertyName)  
        valueArguments[param] =  
            classInfo.deserializeConstructorArgument(param, value)  
    }  
  
    override fun createCompositeProperty(  
        propertyName: String, isList: Boolean  
    ): Seed {  
        val param = classInfo.getConstructorParameter(propertyName)  
        val deserializeAs =  
            classInfo.getDeserializeClass(propertyName)?.starProjectedType  
        val seed = createSeedForType(  
            deserializeAs ?: param.type, isList  
        )  
        return seed.apply { seedArguments[param] = this }  
    }  
  
    override fun spawn(): T =  
        classInfo.createInstance(arguments)  
}
```

Кэширование информации, необходимой для создания экземпляра `targetClass`

Создание карты от параметров конструктора к их значениям

Запись значения для параметра конструктора, если это простое значение

Загрузка значения аннотации `DeserializeInterface` для свойства, если таковое имеется

Создание класса `ObjectSeed` или `CollectionSeed` в соответствии с типом параметра...

...и запись его в карту `seedArguments`

Создание итогового экземпляра `targetClass` с помощью передачи ему карты аргументов

Класс `ObjectSeed` создает карту от параметров конструктора к их значениям. Для этого используются две изменяемые карты: `valueArguments` для простых свойств значений и `seedArguments` — для составных. Пока результат создается, новые аргументы добавляются в карты `valueArguments` и `seedArguments` с помощью вызовов функции `setSimpleProperty` и функции `createCompositeProperty` соответственно. Новые составные «семена» добавляются в пустом состоянии, а затем заполняются данными, поступающими из входного потока. Наконец, метод `spawn` рекурсивно создает все вложенные «семена», вызывая сам себя для каждого из них.

Обратите внимание, что вызов переменной `arguments` в теле метода `spawn` запускает рекурсивное создание составных (`seed`) аргументов — пользовательский геттер `arguments` вызывает методы `spawn` для каждой переменной `seedArguments`. Функция `createSeedForType` анализирует тип параметра и создает `ObjectSeed`, `ObjectListSeed` или `ValueListSeed` в зависимости от того, является ли параметр какой-либо коллекцией. Разбор того, как это реализовано, мы оставим на ваше усмотрение. Далее рассмотрим, как функция `ClassInfo.createInstance` создает экземпляр класса `targetClass`.

12.2.5. Последний этап десериализации: функция `callBy()` и создание объектов с помощью рефлексии

Обсудим оставшуюся часть — класс `ClassInfo`. Он создает итоговый экземпляр и хранит информацию о параметрах конструктора. Он используется в классе `ObjectSeed`. Но прежде, чем погружаться в детали реализации, посмотрим на API для создания объектов с помощью рефлексии.

Вы знакомы с методом `KCallable.call` — он вызывает функцию или конструктор, принимая список аргументов. Этот метод отлично работает во многих случаях, но у него есть ограничение: он не поддерживает значения параметров по умолчанию. В этом случае, если пользователь пытается десериализовать объект с конструктором, содержащим значения параметров по умолчанию, нельзя требовать, чтобы эти аргументы были указаны в JSON. Поэтому необходимо использовать метод `KCallable.callBy` с поддержкой значений параметров по умолчанию:

```
interface KCallable<out R> {
    fun callBy(args: Map<KParameter, Any?>): R
    // ...
}
```

Метод принимает карту параметров с соответствующими им значениями, которые будут переданы в качестве аргументов. Если параметр отсутствует в карте, то по возможности будет использовано его значение по умолчанию. И это удобно, поскольку вам не нужно размещать параметры в правильном порядке. Можно просто считать пары «имя — значение» из JSON, найти параметр, соответствующий каждому имени аргумента, и поместить его значение в карту.

Но нужно позаботиться о правильном подборе типов. Тип значения в карте `args` должен соответствовать типу параметра конструктора. В противном случае во время

выполнения будет выброшено исключение `IllegalArgumentException`. Это особенно важно для числовых типов — необходимо знать, принимает ли параметр значение `Int`, `Long`, `Double` или другой примитивный тип, и нужно преобразовать числовое значение, поступающее из JSON в правильный тип. Для этого используется свойство `KParameter.type`.

Преобразование типов происходит через интерфейс `ValueSerializer`, используемый для пользовательской сериализации. Если у свойства нет аннотации `@CustomSerializer`, то у вас будет стандартная реализация на основе его типа.

Для этого можно предоставить небольшую функцию `serializerForType`. Она дает возможность выполнить сопоставление между типом `KType` и соответствующими встроенными объектами интерфейса `ValueSerializer`. Чтобы получить представление известных `JKid` типов во время выполнения — `Byte`, `Int`, `Boolean` и т. д., — можно использовать функцию `typeOf<>()` для возврата соответствующих типу `KType` экземпляров (листинг 12.10).

Листинг 12.10. Получение сериализатора на основе типа значения

```
fun serializerForType(type: KType): ValueSerializer<out Any?>? =
    when (type) {
        typeOf<Byte>() -> ByteSerializer
        typeOf<Int>() -> IntSerializer
        typeOf<Boolean>() -> BooleanSerializer
        // ...
        else -> null
    }
```

После этого соответствующие реализации интерфейса `ValueSerializer` выполняют необходимую проверку или преобразование типов. В следующем примере показано, что сериализатор для значений типа `Boolean` проверяет, действительно ли переменная `jsonValue` относится к типу `Boolean` при десериализации (листинг 12.11).

Листинг 12.11. Сериализатор для значений типа `Boolean`

```
object BooleanSerializer : ValueSerializer<Boolean> {
    override fun fromJsonValue(jsonValue: Any?): Boolean {
        if (jsonValue !is Boolean) throw JKidException("Boolean expected")
        return jsonValue
    }

    override fun toJsonValue(value: Boolean) = value
}
```

Метод `callBy` дает возможность вызвать первичный конструктор объекта, передав ему карту параметров и соответствующих значений. Механизм `ValueSerializer` гарантирует, что у значений в карте правильные типы. Теперь посмотрим, как вызывать этот API.

Класс `ClassInfoCache` предназначен для снижения накладных расходов на операции рефлексии. Напомним, что аннотации для управления процессом сериализации и десериализации (`@JsonName` и `@CustomSerializer`) применяются к свойствам,

а не к параметрам. При десериализации объекта вы работаете с параметрами конструктора, а не со свойствами. Чтобы получить аннотации, необходимо найти соответствующее свойство. Выполнение такого поиска при чтении каждой пары «ключ — значение» будет очень медленным, поэтому вы делаете это один раз для каждого класса и кэшируете информацию. В листинге 12.12 показана вся реализация класса `ClassInfoCache`.

Листинг 12.12. Хранение кэшированных данных рефлексии

```
class ClassInfoCache {
    private val cacheData = mutableMapOf<KClass<*>, ClassInfo<*>>()

    @Suppress("UNCHECKED_CAST")
    operator fun <T : Any> get(cls: KClass<T>): ClassInfo<T> =
        cacheData.getOrPut(cls) { ClassInfo(cls) } as ClassInfo<T>
}
```

Здесь применяется паттерн, который был описан в подразделе 11.3.6, — при сохранении значений в карте информация о типе удаляется, но реализация метода `get` гарантирует, что у возвращаемого класса `ClassInfo<T>` правильный аргумент типа. Обратите внимание на использование метода `getOrPut`: если карта `cacheData` уже содержит запись для аргумента `cls`, то вызов метода просто возвращает эту запись. В противном случае вызывается переданная лямбда для вычисления значения ключа, сохранения его в карте и возврата.

Класс `ClassInfo` отвечает за создание нового экземпляра целевого класса и кэширование необходимой информации (листинг 12.13). Чтобы упростить код, мы опустили некоторые функции и тривиальные инициализаторы. Вдобавок стоит отметить, что вместо !! реальный код `JKid` в репозитории выбрасывает исключение с информационным сообщением (это хороший паттерн и для вашего собственного кода). Здесь оно просто опущено для краткости.

Листинг 12.13. Кэш параметров конструктора и данных аннотаций

```
class ClassInfo<T : Any>(cls: KClass<T>) {
    private val constructor = cls.primaryConstructor!!

    private val jsonNameToParamMap = hashMapOf<String, KParameter>()
    private val paramToSerializerMap =
        hashMapOf<KParameter, ValueSerializer<out Any?>>()
    private val jsonNameToDeserializeClassMap =
        hashMapOf<String, KClass<out Any?>>()

    init {
        constructor.parameters.forEach { cacheDataForParameter(cls, it) }
    }

    fun getConstructorParameter(propertyName: String): KParameter =
        jsonNameToParam[propertyName]!!

    fun deserializeConstructorArgument(
        param: KParameter, value: Any?): Any? {
        val serializer = paramToSerializer[param]
        if (serializer != null) return serializer.fromJsonValue(value)
    }
}
```

```

        validateArgumentType(param, value)
        return value
    }

    fun createInstance(arguments: Map<KParameter, Any?>): T {
        ensureAllParametersPresent(arguments)
        return constructor.callBy(arguments)
    }
    // ...
}

```

При инициализации этот код находит свойство, соответствующее каждому параметру конструктора, и извлекает его аннотации. Он хранит данные в трех картах: `jsonNameToParam` определяет параметр, соответствующий каждому ключу в JSON-файле; `paramToSerializer` хранит сериализатор для каждого параметра; `jsonNameToDeserializeClass` хранит класс, указанный в качестве аргумента `@DeserializeInterface`, если таковой имеется. Класс `ClassInfo` может указать параметр конструктора по имени свойства, а вызывающий код использует этот параметр в качестве ключа для карты «параметр — аргумент».

Функции `cacheDataForParameter`, `validateArgumentType` и `ensureAllParametersPresent` приватны для этого класса. В листинге 12.14 показана реализация функции `ensureAllParametersPresent`. Код остальных функций вы можете посмотреть самостоятельно.

Листинг 12.14. Проверка предоставления необходимых параметров

```

private fun ensureAllParametersPresent(arguments: Map<KParameter, Any?>) {
    for (param in constructor.parameters) {
        if (arguments[param] == null &&
            !param.isOptional && !param.type.isMarkedNullable) {
            throw JKidException("Missing value for parameter ${param.name}")
        }
    }
}

```

Эта функция проверяет, указали ли вы все необходимые значения параметров. Обратите внимание на удобство API рефлексии в данном случае. Если у параметра есть значение по умолчанию, то параметр `param.isOptional` принимает значение `true` и для него можно опустить аргумент — в таком случае будет использоваться значение по умолчанию. Если параметр типа допускает значение `null` (выражение `type.isMarkedNullable` сообщает об этом), то `null` будет использоваться в качестве значения параметра по умолчанию. Для всех остальных параметров необходимо предоставить соответствующие аргументы. В противном случае будет выброшено исключение. Кэш рефлексии обеспечивает поиск аннотаций для единоразовой настройки процесса десериализации, а не для каждого свойства из JSON-данных.

На этом мы завершаем рассмотрение реализации библиотеки `JKid`. В этой главе мы показали вам реализацию библиотеки сериализации и десериализации JSON на базе API рефлексии и использовали аннотации для настройки ее поведения. Разумеется, все приемы и подходы из этой главы можно использовать и для ваших собственных фреймворков.

РЕЗЮМЕ

- Аннотации в Kotlin применяются с помощью синтаксиса `@МояАннотация(параметры)`.
- Kotlin позволяет применять аннотации к широкому кругу целей, в том числе к файлам и выражениям.
- Аргумент аннотации может быть примитивным значением, строкой, перечислением, ссылкой на класс, экземпляром другого класса аннотаций или их массивом.
- Указание целевой точки использования для аннотации, как в выражении `@get:JvmName`, позволяет выбрать, как аннотация будет применяться, если одно объявление Kotlin создает несколько элементов байт-кода.
- `annotation class` объявляется как класс с первичным конструктором. Все параметры в нем отмечены как свойства `val`, и тело класса отсутствует.
- С помощью метааннотаций можно указать цель, режим сохранения во время выполнения и другие атрибуты аннотаций.
- API рефлексии позволяет перечислять и получать доступ к методам и свойствам объекта динамически во время выполнения. В нем есть интерфейсы, представляющие различные виды объявлений, такие как классы (`KClass`), функции (`KFunction`) и т. д.
- Чтобы получить экземпляр класса `KClass`, можно использовать выражение `ClassName::class` для классов или `objName::class` для экземпляров объектов.
- Интерфейсы `KFunction` и `KProperty` расширяют класс `KCallable`, предоставляющий обобщенный метод `call`.
- Метод `KCallable.callBy` можно использовать для вызова методов со значениями параметров по умолчанию.
- `KFunction0`, `KFunction1` и т. д. — функции с различным количеством параметров. Вызвать их можно с помощью метода `invoke`.
- `KProperty0` и `KProperty1` — свойства с различным количеством приемников, и они поддерживают метод `get` для получения значения. `KMutableProperty0` и `KMutableProperty1` расширяют эти интерфейсы, чтобы поддерживать изменение значений свойств с помощью метода `set`.
- Чтобы получить представление времени выполнения для типа `KType`, можно использовать функцию `typeOf<T>()`.

13

Создание DSL

В этой главе

- ✓ Создание предметно-ориентированных языков.
- ✓ Использование лямбда-выражений с приемниками.
- ✓ Применение соглашения `invoke`.
- ✓ Примеры существующих DSL для Kotlin.

В этой главе мы обсудим, как создавать выразительные и идиоматические API для классов Kotlin с помощью *предметно-ориентированных языков* (Domain-Specific Language, DSL). Вдобавок рассмотрим различия между традиционными и API в DSL-стиле. Вы узнаете, как с помощью API в DSL-стиле решать широкий спектр практических задач в таких областях, как доступ к базам данных, генерация HTML, тестирование, написание сценариев сборки, и многих других.

Проектирование DSL в Kotlin зависит от многих особенностей языка, и две из них мы до этого момента описали не полностью. Первая была представлена в главе 5 — лямбды с приемниками. Они позволяют создать DSL-структуру, определяя характерные для кодовых блоков функции, свойства и поведение. Вторая особенность станет для вас новой — это соглашение `invoke`. Используя его, вы сможете более гибко комбинировать лямбды и присваивать свойства в коде DSL. В данной главе мы рассмотрим эти особенности более подробно.

13.1. ОТ API К DSL: СОЗДАНИЕ ВЫРАЗИТЕЛЬНЫХ ПОЛЬЗОВАТЕЛЬСКИХ СТРУКТУР КОДА

Прежде чем погружаться в обсуждение DSL, вам следует лучше понять поставленную задачу. В конечном счете цель разработчика — сделать код максимально читабельным и удобным в сопровождении. Чтобы достичь ее, недостаточно сосредоточиться на отдельных классах. Большая часть кода в классе взаимодействует с другими классами, поэтому необходимо обратить внимание на *интерфейсы* для реализации этого взаимодействия — другими словами, на *API* классов.

Важно помнить, что задача создания хороших API стоит не только перед авторами библиотек — этим должен заниматься каждый разработчик. Подобно тому как библиотека предоставляет программный интерфейс, с помощью которого ее можно использовать, каждый класс в приложении предоставляет другим классам возможности для взаимодействия с ним. Разработчик должен убедиться, что данное взаимодействие легко понять и объяснить, поскольку этот фактор напрямую влияет на поддержание проекта в рабочем состоянии.

На протяжении всей книги вы видели множество примеров возможностей Kotlin для создания *чистых API* классов. Что означает понятие чистого API? Мы подразумеваем под этим два аспекта:

- читателям должно быть понятно, что происходит в коде. Этого можно достичь с помощью хорошего выбора имен и концепций, что важно для любого языка;
- код должен быть лаконичным и не содержать лишнего синтаксиса. Этой задаче и посвящена данная глава. Чистый API может быть даже неотличим от встроенной функциональной возможности языка.

Примерами возможностей Kotlin для создания чистых API могут служить функции-расширения, инфиксные вызовы, сокращения лямбда-синтаксиса и перегрузка операторов. В табл. 13.1 показано, как эти возможности помогают уменьшить количество синтаксического шума в коде.

В этой главе мы выйдем за пределы чистых API и рассмотрим поддержку Kotlin для создания DSL. DSL Kotlin опираются на возможности синтаксиса и расширяют их за счет способности создавать *структуру* из нескольких вызовов методов. В результате DSL могут быть даже более выразительными и приятными в работе, чем API, созданные на основе отдельных вызовов методов.

Как и другие возможности языка, DSL Kotlin *полностью статически типизированы*. Это означает, что при использовании DSL-шаблонов для своих API вы по-прежнему получаете все преимущества статической типизации, такие как обнаружение ошибок во время компиляции и улучшенная поддержка IDE.

Таблица 13.1. Поддержка чистого синтаксиса в Kotlin

Обычный синтаксис	Чистый синтаксис	Используемая возможность
<code>StringUtil.capitalize(s)</code>	<code>s.capitalize()</code>	Функция-расширение
<code>1.to("one")</code>	<code>1 to "one"</code>	Инфиксный вызов
<code>set.add(2)</code>	<code>set += 2</code>	Перегрузка оператора
<code>map.get("key")</code>	<code>map["key"]</code>	Соглашение для метода <code>get</code>
<code>file.use { f -> f.read() })</code>	<code>file.use { it.read() } }</code>	Лямбда за пределами круглых скобок
<code>sb.append("yes") sb.append("no")</code>	<code>with (sb) { append("yes") append("no") }</code>	Лямбда с приемником
<code>val m = mutableListOf<Int>() m.add(1) m.add(2) return m.toList()</code>	<code>return buildList { add(1) add(2) }</code>	Создание функций с помощью лямбд

В качестве краткого обзора приведем пару примеров, показывающих, на что способны DSL Kotlin. Это выражение отправляется в прошлое и возвращает предыдущий день (правда, на самом деле оно не перемещается во времени, а просто выдает предыдущую дату):

```
val yesterday = Clock.System.now() - 1.days
```

Эта функция генерирует таблицу HTML:

```
fun createSimpleTable() = createHTML().
    table {
        tr {
            td { +"cell" }
        }
    }
}
```

На протяжении всей главы мы будем рассказывать, как созданы эти примеры. Но прежде, чем приступить к подробному обсуждению, разберемся, что такое DSL.

13.1.1. Предметно-ориентированные языки

Общая идея DSL существует почти так же давно, как и идея языка программирования. Мы проводим различие между *языком программирования общего назначения*, содержащим набор возможностей для решения компьютером практически любой задачи, и *предметно-ориентированным языком*, сосредоточенным на определенной

задаче или *предметной области*. В таком языке отказываются от функциональности, не имеющей отношения к предметной области.

Самые распространенные DSL, знакомые всем разработчикам, — SQL и регулярные выражения. Они отлично подходят для решения специфических задач по работе с базами данных и текстовыми строками соответственно, но их нельзя использовать для разработки целого приложения. (По крайней мере, мы надеемся, что это так. Мысль о целом приложении, написанном на языке регулярных выражений, заставляет нас содрогнуться.)

Эти языки могут эффективно достичь своей цели, сократив набор предлагаемой ими функциональности. Когда требуется выполнить SQL-запрос, код не начинается с объявления класса или функции. Вместо этого первое ключевое слово в каждом операторе SQL указывает на тип выполняемой операции (SELECT, INSERT...), и у каждого типа операции есть собственный синтаксис и набор ключевых слов, характерных для конкретной задачи. В языке регулярных выражений синтаксиса еще меньше: программа напрямую описывает текст для сопоставления, используя компактный синтаксис пунктуации, чтобы указать, как может меняться этот текст. Благодаря такому компактному синтаксису DSL может выразить специфическую для домена операцию гораздо лаконичнее, чем эквивалентный фрагмент кода на языке общего назначения.

Есть еще один важный момент: DSL, как правило, *декларативные*. А языки общего назначения зачастую *императивные*. В то время как императивный язык описывает точную последовательность шагов, необходимых для выполнения операции, декларативный язык описывает желаемый результат и оставляет детали выполнения на усмотрение интерпретирующего его механизма. Такой подход часто делает выполнение более эффективным, поскольку необходимые оптимизации реализуются только один раз в механизме выполнения. В свою очередь, императивный подход требует, чтобы каждая реализация операции была оптимизирована независимо. Если снова обратиться к SQL, то при выполнении запроса DELETE не нужно перебирать каждую запись в таблице вручную, извлекать отдельные поля и принимать решения о том, какое действие следует выполнить. Вместо этого вы только объявляете условия, которые должны быть выполнены, чтобы запись в таблице была удалена. А механизм выполнения запросов использует эту информацию для формирования оптимального запроса, принимая во внимание индексы, соединения и т. д.

В противовес всем этим преимуществам у DSL такого типа есть один недостаток: их объединение с базовым приложением на языке общего назначения может быть затруднено. У них собственный синтаксис, и его нельзя напрямую внедрить в программы на другом языке. Поэтому вызвать программу на DSL можно, либо сохранив ее в отдельном файле, либо встроив в строковый литерал. Это делает нетривиальной задачу проверки правильности взаимодействия DSL с базовым языком во время компиляции, отладки DSL-программы и помощи IDE при ее написании. Кроме того, отдельный синтаксис требует изучения и часто ухудшает читаемость кода.

Чтобы решить эту проблему, сохранив большинство других преимуществ DSL, Kotlin делает возможным написание *внутренних DSL*.

13.1.2. Внутренние DSL легко интегрируются в остальную часть программы

В отличие от *сторонних DSL* со своим независимым синтаксисом *внутренние DSL* представляют собой часть программ на языке общего назначения и используют точно такой же синтаксис. По сути, внутренний DSL — это не полностью отдельный язык, а, скорее, особый способ использования основного языка, при котором сохраняются ключевые преимущества DSL с независимым синтаксисом.

Чтобы сравнить эти подходы, рассмотрим, как с их помощью решить одну и ту же задачу. Представьте, что у вас есть две таблицы базы данных, `Customer` и `Country`, и каждая запись в таблице `Customer` содержит ссылку на страну проживания клиента. Ваша задача — запросить базу данных и найти страну, в которой проживает большинство клиентов. В качестве стороннего DSL используем SQL. Внутренний предоставляется фреймворком `Exposed` (<https://github.com/JetBrains/Exposed>), который является Kotlin-фреймворком для доступа к базам данных. Решить задачу с помощью SQL можно следующим образом:

```
SELECT Country.name, COUNT(Customer.id)
  FROM Country
 INNER JOIN Customer
    ON Country.id = Customer.country_id
 GROUP BY Country.name
 ORDER BY COUNT(Customer.id) DESC
  LIMIT 1
```

Писать запрос на SQL напрямую будет неудобно: необходимо предоставить средства взаимодействия между основным языком приложения, в данном случае Kotlin, и языком запросов. Обычно лучшим решением станет следующее действие: поместить SQL-код в строковый литерал и надеяться, что ваша IDE поможет написать и проверить его.

Для сравнения приведем тот же запрос, созданный с помощью Kotlin и `Exposed`:

```
(Country innerJoin Customer)
    .slice(Country.name, Count(Customer.id))
    .selectAll()
    .groupBy(Country.name)
    .orderBy(Count(Customer.id), order = SortOrder.DESC)
    .limit(1)
```

Между этими двумя версиями прослеживается сходство. На самом деле при выполнении второй генерируется и выполняется точно такой же SQL-запрос, как и написанный вручную. Но вторая версия — это обычный код Kotlin, а `selectAll`, `groupBy`, `orderBy` и др. — обычные методы Kotlin. Более того, не нужно тратить время на преобразование данных из наборов результатов SQL-запросов в объекты Kotlin — результаты выполнения запросов предоставляются непосредственно в виде нативных объектов Kotlin. Поэтому мы называем это *внутренним DSL* — код, предназначенный для выполнения конкретной задачи (создание SQL-запросов), реализован в виде библиотеки на языке общего назначения (Kotlin).

13.1.3. Структура DSL

Как правило, не существует четко определенной границы между DSL и обычным API. Зачастую различие между ними настолько субъективно, что выражается мыслью: «Я знаю, что это DSL, просто по виду». DSL часто используют возможности языка, которые широко применяются и в других контекстах, например инфиксные вызовы и перегрузка операторов. Но одна черта часто встречается в DSL и обычно не встречается в других API: *структура* или *грамматика*.

Типичный API состоит из множества методов, и клиент использует предлагаемую функциональность, вызывая методы один за другим. В последовательности вызовов нет внутренней структуры, такой как вложения или группы, и между одним вызовом и следующим не сохраняется контекст.

Такой API иногда называют *API команд и запросов*. В отличие от него вызовы методов в DSL существуют в более крупной структуре, определяемой *грамматикой* DSL. В DSL языка Kotlin структура чаще всего создается за счет вложенности лямбд или цепочки вызовов методов. Это хорошо видно на предыдущем примере с SQL-кодом: чтобы выполнить запрос, необходимо комбинировать вызовы методов, служащие для описания различных аспектов требуемого набора результатов. Комбинированный запрос гораздо легче читать, чем один вызов метода, принимающий все аргументы.

Именно эта грамматика позволяет нам называть внутренний DSL *языком*. В естественном языке, таком как английский, предложения создаются из слов, а правила грамматики определяют, как эти слова могут сочетаться друг с другом. Аналогично в DSL одна операция может состоять из нескольких вызовов функций, а механизм проверки типов гарантирует, что вызовы будут объединены осмысленным образом. По сути, имена функций обычно выступают как глаголы (`groupBy`, `orderBy`), а их аргументы играют роль существительных (`Country.name`).

Одно из преимуществ DSL-структуры заключается в том, что она позволяет повторно использовать один и тот же контекст в нескольких вызовах функций, а не повторять его в каждом вызове. Это можно проиллюстрировать на следующем примере. В нем показан DSL Kotlin для описания зависимостей в скриптах сборки Gradle:

```
dependencies {  Структура на основе вложенных лямбд
    testImplementation(kotlin("test"))
    implementation("org.jetbrains.exposed:exposed-core:0.40.1")
    implementation("org.jetbrains.exposed:exposed-dao:0.40.1")
}
```

Для сравнения приведем ту же операцию на базе обычного API для командных запросов. Обратите внимание, что в коде гораздо больше повторений:

```
project.dependencies.add("testImplementation", kotlin("test"))
project.dependencies.add("implementation",
    "org.jetbrains.exposed:exposed-core:0.40.1")
project.dependencies.add("implementation",
    "org.jetbrains.exposed:exposed-dao:0.40.1")
```

Цепочки вызовов методов — еще один способ создать структуру в DSL. Так код становится более удобным для чтения. Например, они часто используются в тестовых фреймворках для разделения утверждения на несколько вызовов методов. С такими утверждениями гораздо проще работать, особенно если вы умеете применять инфиксный синтаксис вызова.

Следующий пример взят из Kotest (<https://github.com/kotest/kotest>), стороннего тестового фреймворка для Kotlin (более подробно мы рассмотрим его в подразделе 13.4.1):

```
str should startWith("kot")
```

← Структура на основе цепочки вызовов методов

Отметим, что тот же пример на базе обычных API в стиле JUnit сильнее зашумлен и не настолько удобочитаем:

```
assertTrue(str.startsWith("kot"))
```

А теперь рассмотрим пример внутреннего DSL более подробно.

13.1.4. Создание HTML-страницы с помощью внутреннего DSL

Одним из анонсов в начале этой главы было применение DSL для создания HTML-страниц. В текущем подразделе мы рассмотрим данную тему более подробно. Здесь используется API из библиотеки `kotlinx.html` (более подробную информацию о ней, в том числе инструкции по ее добавлению в собственный проект, можно найти на сайте <https://github.com/Kotlin/kotlinx.html>). Перед вами небольшой фрагмент для создания таблицы с одной ячейкой:

```
import kotlinx.html.stream.createHTML
import kotlinx.html.*

fun createSimpleTable() = createHTML().
    table {
        tr {
            td { +"cell" }
        }
    }
```

Явно прослеживается, какой HTML-код соответствует предыдущей структуре:

```
<table>
  <tr>
    <td>cell</td>
  </tr>
</table>
```

Функция `createSimpleTable` возвращает строку, содержащую данный HTML-фрагмент.

Почему может потребоваться создать этот фрагмент HTML с помощью кода Kotlin, а не написать его в виде текста? Для начала, версия Kotlin отвечает безопасности типов: тег `td` можно применить только внутри тега `tr`, в противном случае данный код не скомпилируется. Важнее всего то, что это обычный код и в нем можно использовать любые языковые конструкции. Это означает возможность динамически генерировать ячейки таблицы (например, соответствующие элементам карты) в точке определения таблицы:

```
import kotlinx.html.stream.createHTML
import kotlinx.html.*

fun createAnotherTable() = createHTML().table {
    val numbers = mapOf(1 to "one", 2 to "two")
    for ((num, string) in numbers) {
        tr {
            td { +"$num" }
            td { +string }
        }
    }
}
```

Сгенерированный HTML-код содержит нужные данные:

```
<table>
  <tr>
    <td>1</td>
    <td>one</td>
  </tr>
  <tr>
    <td>2</td>
    <td>two</td>
  </tr>
</table>
```

HTML представляет собой канонический пример языка разметки. Он идеален для иллюстрации этой концепции, но данный подход можно применять и для любых языков с похожей структурой, таких как XML. Вскоре мы обсудим, как такой код работает в Kotlin.

Теперь, когда вы знаете, что такое DSL и почему его используют в разработке, посмотрим, как Kotlin помогает в этом. И сначала мы более подробно рассмотрим *лямбды с приемниками* — ключевую функциональную возможность создания грамматики DSL.

13.2. СОЗДАНИЕ СТРУКТУРИРОВАННЫХ API: ЛЯМБДЫ С ПРИЕМНИКАМИ В DSL

Лямбды с приемниками являются мощной функциональной возможностью Kotlin и позволяет создавать API со структурой. А наличие структуры — одна из ключевых черт, отличающих DSL от обычных API. Рассмотрим эту возможность подробнее и обратимся к некоторым использующим ее DSL.

13.2.1. Лямбды с приемниками и типы функций-расширений

Мы кратко познакомили вас с идеей лямбд с приемниками во время обсуждения функций `buildList`, `buildString`, `with` и `apply` из стандартной библиотеки в главе 5. Теперь рассмотрим их реализацию на примере функции `buildString`. Она дает возможность сконструировать строку из нескольких частей содержимого, добавленных в промежуточный класс `StringBuilder`.

Для начала определим функцию `buildString` так, чтобы она принимала в качестве аргумента обычную лямбду (листинг 13.1). В главе 10 показано, как это сделать, так что этот материал должен быть вам знаком.

Листинг 13.1. Определение функции `buildString()`, принимающей лямбду в качестве аргумента

```
fun buildString(
    builderAction: (StringBuilder) -> Unit ←— Объявление параметра типа функции
): String {
    val sb = StringBuilder()
    builderAction(sb) ←— Передача экземпляра StringBuilder
                        в качестве аргумента в лямбду
    return sb.toString()
}

fun main() {
    val s = buildString {
        it.append("Hello, ") ←— Использование функции it для ссылки
                             на экземпляр StringBuilder
        it.append("World!")
    }
    println(s)
    // Hello, World!
}
```

Этот код легко понять, но он выглядит менее удобным в использовании, чем хотелось бы. Обратите внимание, что следует использовать функцию `it` в теле лямбды для ссылки на экземпляр `StringBuilder` (можно определить собственное имя параметра вместо `it`, но оно все равно должно быть указано в явном виде). Основная цель лямбды — заполнить экземпляр `StringBuilder` текстом, поэтому необходимо избавиться от повторяющихся префиксов `it` и вызывать методы класса `StringBuilder` напрямую, заменив выражение `it.append` на `append`.

Для этого необходимо преобразовать просто лямбду в *лямбду с приемником*. По сути, можно придать одному из параметров лямбды особый статус *приемника*, что позволит обращаться к его членам напрямую, без каких-либо уточнений. В листинге 13.2 показано, как это сделать.

Листинг 13.2. Переопределение функции `buildString` для приема лямбды с приемником

```
fun buildString(
    builderAction: StringBuilder.() -> Unit ←— Объявление параметра типа
): String {                                функции с приемником
    val sb = StringBuilder()
    sb.builderAction() ←— Передача экземпляра StringBuilder
                        в качестве приемника в лямбду
    return sb.toString()
}
```

```

fun main() {
    val s = buildString {
        this.append("Hello, ")
        append("World!")
    }
    println(s)
    // Hello, World!
}

```

Ключевое слово `this` указывает на экземпляр `StringBuilder`

Или можно опустить ключевое слово `this` и сослаться на `StringBuilder`

Обратите внимание на различия между листингами 13.1 и 13.2. Для начала подумайте, как улучшился способ использования функции `buildString`. Теперь лямбда с приемником передается в качестве аргумента, чтобы избавиться от `it` в теле лямбды. Вызов функции `it.append()` заменяется на `append()`. Полная форма вызова будет иметь вид `this.append()`, но, как и в случае с обычными членами класса, явное указание ссылки `this` обычно используется только для устранения неоднозначности.

Далее обсудим, как изменилось объявление функции `buildString`. Для объявления типа параметра вместо обычного типа функции используется *тип функции-расширения*. При объявлении типа функции-расширения фактически из круглых скобок извлекается один из типовых параметров функции и перемещается вперед. От остальных типов он будет отделен точкой. В листинге 13.2 выражение `(StringBuilder) -> Unit` заменяется на `StringBuilder.() -> Unit`. Этот специальный тип называется *типом приемника*, а переданное в лямбду значение этого типа становится *объектом-приемником*. На рис. 13.1 показано более сложное объявление типа функции-расширения.

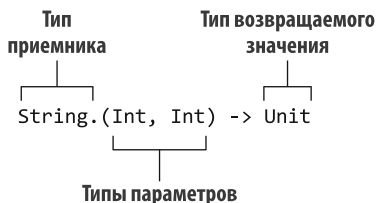


Рис. 13.1. Тип функции-расширения с типом приемника `String` и два параметра типа `Int`, возвращаемое значение — `Unit`

Для чего используется тип функции-расширения? Идея доступа к членам внешнего типа без явного квалификатора может напомнить вам о функциях-расширениях, служащих для определения собственных методов классов, определенных в других частях кода. У функций-расширений и у лямбд с приемниками есть объект-приемник. Его необходимо предоставить при вызове функции, и он должен быть доступен в ее теле. По сути, тип функции-расширения описывает блок кода, который можно вызывать как функцию-расширение.

Способ вызова переменной тоже меняется. Это происходит при преобразовании ее из обычного типа функции в тип функции-расширения. Вместо передачи объекта в качестве аргумента вызывается лямбда-переменная, как если бы она была функцией-расширением. Когда в коде есть обычная лямбда, в качестве аргумента ей передается экземпляр класса `StringBuilder` с помощью следующего синтаксиса: `builderAction(sb)`. Если заменить ее на лямбду с приемником, то код примет вид `sb.builderAction()`. Повторимся: `builderAction` в данном случае не метод из класса `StringBuilder`. Это параметр типа функции, вызываемой с помощью синтаксиса для вызова функций-расширений.

На рис. 13.2 показано соответствие между аргументом и параметром функции `buildString`, а также представлен приемник для вызова тела ламбды.

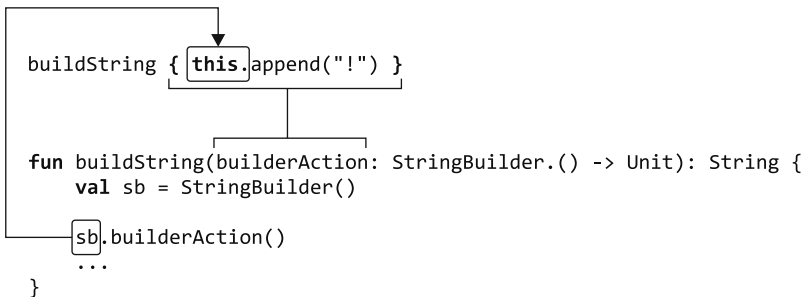


Рис. 13.2. Аргумент функции `buildString` (лямбда с приемником) соответствует параметру типа функции-расширения (`builderAction`). Приемник (`sb`) становится скрытым приемником (`this`) при вызове тела ламбды

Кроме того, можно объявить переменную типа функции-расширения, как показано в листинге 13.3. После этого ее можно вызывать как функцию-расширение или передать в качестве аргумента функции, ожидающей ламбду с приемником.

Листинг 13.3. Хранение ламбды с приемником в переменной

```

val appendExcl: StringBuilder.() -> Unit = { this.append("!") }

fun main() {
    val stringBuilder = StringBuilder("Hi")
    stringBuilder.appendExcl()
    println(stringBuilder)
    // Hi!
    println(buildString(appendExcl))
    // !
}

```

appendExcl — значение типа функции-расширения

Переменную `appendExcl` можно вызвать как функцию-расширение

Также можно передать переменную `appendExcl` в качестве аргумента

Обратите внимание, что ламбда с приемником выглядит точно так же, как и обычная в исходном коде. Чтобы определить наличие приемника у ламбды, нужно посмотреть на функцию, которой передается ламбда. Ее сигнатура подскажет, есть ли у ламбды приемник (рис. 13.3), и если да, то каков его тип. Для примера рассмотрим объявление функции `buildString` (ее документацию можно найти в IDE). Она принимает ламбду типа `StringBuilder.() -> Unit`. Следовательно, можно сделать вывод, что в теле ламбды можно вызывать методы класса `StringBuilder` без квалификатора.

Реализация функции `buildString` в стандартной библиотеке короче, чем в листинге 13.2. Вместо явного вызова метода `builderAction` она передается

```

buildString { this: StringBuilder
    append("Hello!")
}

```

Рис. 13.3. Дополнительные внутритекстовые подсказки в IntelliJ IDEA и Android Studio помогают визуализировать типы приемника ламбды

в качестве аргумента в функцию `apply` (описанную в главе 5). Это позволяет свернуть функцию в одну строку:

```
fun buildString(builderAction: StringBuilder.() -> Unit): String =
    StringBuilder().apply(builderAction).toString()
```

Функция `apply` эффективно принимает объект, для которого была вызвана (в данном случае новый экземпляр `StringBuilder`), и использует его в качестве скрытого приемника для вызова функции или лямбды, указанной в качестве аргумента (`builderAction` в данном примере). Кроме того, вы знакомы с другой полезной библиотечной функцией — `with`. Обсудим их реализации:

```
inline fun <T> T.apply(block: T.() -> Unit): T {
    block()
    return this
}
```

← Эквивалентно выражению `this.block()` — вызывает лямбду с приемником `apply` в качестве объекта-приемника

← Возврат приемника

← Возврат результата вызова лямбды

```
inline fun <T, R> with(receiver: T, block: T.() -> R): R = receiver.block()
```

По сути, вся работа функций `apply` и `with` заключается в вызове аргумента типа функции-расширения для предоставленного приемника. Функция `apply` объявлена как расширение для этого приемника, а `with` принимает его в качестве первого аргумента. Главное различие состоит в том, что функция `apply` возвращает сам приемник, а `with` — результат вызова лямбды.

Так что, если результат неважен, эти функции фактически взаимозаменяемы:

```
fun main() {
    val map = mutableMapOf(1 to "one")
    map.apply { this[2] = "two" }
    with (map) { this[3] = "three" }
    println(map)
    // {1=one, 2=two, 3=three}
}
```

Функции `with` и `apply` часто применяются в Kotlin, и мы надеемся, что вы уже оценили их лаконичность при разработке своего кода.

Мы рассмотрели лямбды с приемниками и поговорили о типах функций-расширений. Теперь перейдем к использованию этих концепций в контексте DSL.

13.2.2. Использование лямбд с приемниками в конструкторах HTML

DSL языка Kotlin для HTML обычно называют *конструктором HTML*, и он представляет собой более общую концепцию *конструкторов с безопасностью типов*. Изначально концепция конструкторов приобрела популярность в сообществе Groovy (https://www.groovy-lang.org/dsls.html#_builders). Она дает возможность создавать иерархию объектов в декларативном виде, что удобно для создания разметки или описания макетов.

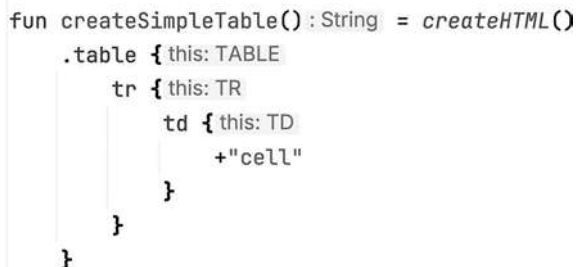
Хотя в них используется та же идея, конструкторы Kotlin отвечают требованиям безопасности типов, что делает их более надежными и удобными в применении. Подробно рассмотрим работу реализации конструктора HTML в среде Kotlin (листинг 13.4).

Листинг 13.4. Создание простой HTML-таблицы с помощью HTML-конструктора Kotlin

```
fun createSimpleTable() = createHTML().
    table {
        tr {
            td { +"cell" }
        }
    }
```

Повторимся, что это обычный код Kotlin, а не специальный язык шаблонов: `table`, `tr` и `td` — просто функции. Каждая из них представляет собой функцию высшего порядка. В качестве аргумента они принимают лямбду с приемником. Если написать `+"cell"`, то просто будет вызвана функция. DSL HTML переопределяет оператор `unaryPlus` (описан в главе 9), чтобы добавить содержимое строки в окружающую ячейку `td`.

Примечательно, что эти лямбды *меняют правила разрешения имен* — вводят дополнительные функции, и их можно вызывать внутри тела лямбд. В лямбде функции `table` можно использовать функцию `tr` для создания HTML-тега `<tr>`. За пределами этой лямбды функция `tr` будет неразрешимой. Точно так же функция `td` доступна только в `tr`, а унарный плюс действует на приемник окружающего тега `td` (рис. 13.4). Обратите внимание, что дизайн API заставляет вас следовать грамматике языка HTML.



```
fun createSimpleTable(): String = createHTML()
    .table { this: TABLE
        tr { this: TR
            td { this: TD
                +"cell"
            }
        }
    }
```

Рис. 13.4. Дополнительные внутритекстовые подсказки в IntelliJ IDEA и Android Studio помогают визуализировать типы приемников для лямбд

Контекст разрешения имен в каждом блоке определяется типом приемника каждой лямбды. Переданная в функцию `table` лямбда содержит приемник особого типа `TABLE`, и он определяет метод `tr`. Аналогично функция `tr` ожидает расширения лямбды до типа `TR`. В листинге 13.5 приведено значительно упрощенное представление объявлений этих классов и методов.

Листинг 13.5. Объявление классов тегов для конструктора HTML

```

open class Tag

class TABLE : Tag {
    fun tr(init: TR.() -> Unit) ← | Функция tr ожидает поступления
                                | ламбды с приемником типа TR
}

class TR : Tag {
    fun td(init: TD.() -> Unit) ← | Функция td ожидает поступления
                                | ламбды с приемником типа TD
}

class TD : Tag

```

TABLE, TR и TD — служебные классы, и они не должны появляться в коде в явном виде. Поэтому их названия написаны заглавными буквами. Все они расширяют суперкласс Tag. Позже мы с его помощью будем добавлять дополнительные полезные ограничения в DSL. Каждый класс определяет методы для создания разрешенных в нем тегов: класс TABLE среди прочих определяет метод tr, а класс TR — метод td.

Следует обратить внимание на типы параметров init функций tr и td — они представляют собой типы функций-расширений TR.() -> Unit и TD.() -> Unit. И определяют типы приемников в ламбдах аргументов: TR и TD соответственно.

Чтобы было лучше прояснить происходящее, перепишем листинг 13.4 и сделаем все приемники явными (листинг 13.6). Напоминаем, что к приемнику ламбды, выступающему в роли аргумента функции foo, можно обратиться как к this@foo.

Листинг 13.6. Явное определение получателей вызовов HTML-конструктора

```

fun createSimpleTable() = createHTML().table {
    this@table.tr { ← | Тип this@table — TABLE
        this@tr.td { ← | Тип this@tr — TR
            +"cell" ← | Здесь доступен скрытый
                     | приемник this@td типа TD
        }
    }
}

```

Если попытаться использовать обычные ламбды вместо ламбд с приемниками для конструкторов, то синтаксис станет таким же нечитаемым, как в этом примере. В таком случае придется использовать ссылку it для вызова методов создания тегов или назначать новое имя параметра для каждой ламбды. Возможность сделать приемник неявным и скрыть ссылку this делает синтаксис конструкторов удобным в работе и похожим на исходный HTML.

Обратите внимание: если одна ламбда с приемником помещается в другую, как в листинге 13.6, то приемник из внешней ламбды остается доступным во вложенной. Например, в ламбде, которая выступает в роли аргумента функции td, доступны все три приемника (this@table, this@tr, this@td).

Если вы работаете с глубоко вложенными структурами, это может привести к путанице, поскольку будет не сразу понятно, на какой приемник ссылается выражение. Например, вы можете случайно обратиться к свойству `href` окружающего тега `a` внутри лямбды `img` (листинг 13.7).

Листинг 13.7. Наличие нескольких приемников в области видимости может привести к запутанности кода.

```
createHTML().body {
    a {
        img {
            href = "https://..." ← Ссылка на приемник лямбды a
        }
    }
}
```

Избежать этого позволяет аннотация `@DslMarker`, предусмотренная в Kotlin. Она ограничивает доступность внешних приемников в лямбдах, как показано в листинге 13.8. Аннотация `@DslMarker` является метааннотацией (см. главу 12) и может применяться к классу аннотации. В файле `kotlinx.html` эта аннотация носит название `HtmlTagMarker`.

Листинг 13.8. Определение класса аннотации маркера для DSL

```
@DslMarker
annotation class HtmlTagMarker
```

Любое объявление с аннотацией `@HtmlTagMarker` будет иметь дополнительные ограничения на скрытые приемники. В частности, в одной области видимости никогда не может быть двух скрытых приемников, если их типы помечены одной и той же аннотацией `@DslMarker`. Все теги представляют собой подклассы класса `Tag`, поэтому можно добавить аннотацию `@HtmlTagMarker`:

```
@HtmlTagMarker
open class Tag
```

Теперь приемник лямбды `a (this: A)` недоступен из лямбды `img (this: IMG)`, поскольку оба типа приемника помечены аннотацией `@HtmlTagMarker`. Код из листинга 13.7 теперь не будет компилироваться. Вместо этого попытка присвоить значение свойству `href` внутри блока `img` приводит к следующей ошибке:

```
var href: String can't be called in this context by implicit receiver.
Use the explicit one if necessary.
```

Вы узнали, что синтаксис HTML-конструкторов основан на концепции лямбд с приемниками. Далее обсудим процесс генерации нужного HTML-кода.

В листинге 13.9 используются функции из библиотеки `kotlinx.html`. Теперь нам предстоит реализовать гораздо более простую версию библиотеки HTML-конструктора,

расширив объявления тегов `TABLE`, `TR` и `TD` и добавив поддержку генерации итогового HTML-кода. В качестве точки входа для этой упрощенной версии функция верхнего уровня `table` создает фрагмент HTML-кода с верхним тегом `<table>`.

Листинг 13.9. Генерация HTML-кода в строку

```
fun createTable() =
    table {
        tr {
            td {
            }
        }
    }

fun main() {
    println(createTable())
    // <table><tr><td></td></tr></table>
}
```

Функция `table` создает новый экземпляр тега `TABLE`, инициализирует (вызывает функцию, переданную в нее в качестве параметра `init`) и возвращает его:

```
fun table(init: TABLE.() -> Unit) = TABLE().apply(init)
```

Лямбда, переданная в качестве аргумента функции `table` в `createTable`, содержит вызов функции `tr`. Вызов можно переписать, чтобы сделать все как можно более явным: `table(init = { this.tr { ... } })`. Функция `tr` будет вызвана для созданного экземпляра тега `TABLE`, как если бы вы написали `TABLE().tr { ... }`.

В этом примере `<table>` — тег верхнего уровня, а другие теги вложены в него. Каждый тег хранит список ссылок на свои дочерние элементы. Поэтому функция `tr` должна не только инициализировать новый экземпляр тега `TR`, но и добавить его в список дочерних элементов, связанных с внешним тегом (листинг 13.10).

Листинг 13.10. Определение функции создания тегов

```
fun tr(init: TR.() -> Unit) {
    val tr = TR()
    tr.init()
    children.add(tr)
}
```

Логика инициализации данного тега и добавления его в дочерние элементы внешнего тега — общая для всех тегов, поэтому ее можно извлечь в виде члена `doInit` суперкласса `Tag`. Функция `doInit` отвечает за хранение ссылки на дочерний тег и вызов лямбды, переданной в качестве аргумента. Затем различные теги просто вызывают ее: например, функция `tr` создает новый экземпляр класса `TR` и передает его функции `doInit` вместе с лямбда-аргументом `init: doInit(TR(), init)`. В листинге 13.11 показан полный код для генерации нужных HTML-структур.

Листинг 13.11. Полная реализация простого HTML-конструктора

```

@DslMarker
annotation class HtmlTagMarker

@HtmlTagMarker
open class Tag(val name: String) {
    private val children = mutableListOf<Tag>() ← Хранение всех вложенных тегов

    protected fun <T : Tag> doInit(child: T, init: T.() -> Unit) {
        child.init() ← Инициализация дочернего тега
        children.add(child) ← Хранение ссылок на дочерние теги
    }

    override fun toString() = "<$name>${children.joinToString("")}</$name>" ←
    fun table(init: TABLE.() -> Unit) = TABLE().apply(init)
    Возврат результирующего HTML-кода в виде значения типа String

    class TABLE : Tag("table") {
        fun tr(init: TR.() -> Unit) = doInit(TR(), init) ← Создание, инициализация и добавление экземпляра тега TR в дочерний элемент тега TABLE

        class TR : Tag("tr") {
            fun td(init: TD.() -> Unit) = doInit(TD(), init) ← Добавление нового экземпляра тега TD в дочерний экземпляр тега TR

            class TD : Tag("td")

            fun createTable() =
                table {
                    tr {
                        td {
                        }
                    }
                }
            }

            fun main() {
                println(createTable())
                // <table><tr><td></td></tr></table>
            }

```

Каждый тег хранит список вложенных тегов и представляет себя соответствующим образом — рекурсивно отображает свое имя и все вложенные теги. В целях сохранения краткости кода в этой реализации не поддерживаются текст внутри тегов или специфические атрибуты тегов. Если вас интересует полная реализация HTML-конструктора для Kotlin, то вы можете просмотреть исходный код вышеупомянутой библиотеки `kotlinx.html`.

Функции создания тегов самостоятельно добавляют соответствующий тег в список дочерних тегов родителя, поэтому теги можно генерировать динамически (листинг 13.12).

Листинг 13.12. Динамическая генерация тегов с помощью HTML-конструктора

```
fun createAnotherTable() = table {
    for (i in 1..2) {
        tr {
            td {
            }
        }
    }
}

fun main() {
    println(createAnotherTable())
    // <table><tr><td></td></tr><tr><td></td></tr></table>
}
```

Каждый вызов функции `tr` создает
новый тег `TR` и добавляет его
в дочерние элементы тега `TABLE`

Вы уже убедились, что лямбды с приемниками — отличный инструмент создания DSL. Контекст разрешения имен можно менять в блоке кода, поэтому они позволяют создавать структуру в вашем API. А это одна из ключевых черт, отличающих DSL от простых последовательностей вызовов методов. Далее обсудим преимущества интеграции этого DSL в статически типизированный язык программирования.

13.2.3. Конструкторы Kotlin: обеспечение абстракции и повторного использования

Существует множество инструментов, позволяющих избежать дублирования и сделать код более удобным для чтения. Помимо прочего, можно извлекать повторяющийся код в новые функции и давать им понятные имена. Это может быть не так просто или даже невозможно, если используется SQL или HTML. А применение внутренних DSL в Kotlin для решения тех же задач дает возможность абстрагировать повторяющиеся куски кода в новые функции и использовать их повторно.

Рассмотрим приложение, отвечающее за генерацию списка аннотаций книг в формате HTML с оглавлением в начале. Написание такого HTML-кода вручную не представляет особой сложности — базовая реализация приложения может выглядеть следующим образом (листинг 13.13).

Листинг 13.13. Создание страницы с оглавлением

```
<body>
<ul>
  <li><a href="#0">The Three-Body Problem</a></li>
  <li><a href="#1">The Dark Forest</a></li>
  <li><a href="#2">Death's End</a></li>
</ul>
<h2 id="0">The Three-Body Problem</h2>
<p>The first book tackles...</p>
<h2 id="1">The Dark Forest</h2>
<p>The second book starts with...</p>
<h2 id="2">Death's End</h2>
<p>The third book contains...</p>
</body>
```


В Kotlin с помощью файла `kotlinx.html` можно использовать функции `u1`, `li`, `h2`, `p` и так далее для повторения той же структуры (листинг 13.14).

Листинг 13.14. Создание страницы с оглавлением с помощью HTML-конструктора Kotlin

```
fun buildBookList() = createHTML().body {
    u1 {
        li { a("#1") { +"The Three-Body Problem" } }
        li { a("#2") { +"The Dark Forest" } }
        li { a("#3") { +"Death's End" } }
    }

    h2 { id = "1"; +"The Three-Body Problem" }
    p { +"The first book tackles..." }

    h2 { id = "2"; +"The Dark Forest" }
    p { +"The second book starts with..." }

    h2 { id = "3"; +"Death's End" }
    p { +"The third book contains..." }
}
```

Но можно сделать еще лучше. Поскольку `u1`, `h2` и т. д. — обычные функции, то можно выделить логику для создания оглавления и краткого содержания в отдельные функции. Результат может выглядеть следующим образом (листинг 13.15).

Листинг 13.15. Создание страницы с оглавлением с помощью вспомогательных функций

```
fun buildBookList() = createHTML().body {
    listWithToc {
        item("The Three-Body Problem", "The first book tackles...")
        item("The Dark Forest", "The second book starts with...")
        item("Death's End", "The third book contains...")
    }
}
```

Теперь ненужные детали скрыты, нет необходимости вручную следить за добавлением элементов в оглавление, а код выглядит гораздо лучше. Но как, собственно, реализовать функции `listWithToc` и `item`?

Как и в листинге 13.11, можно создать собственный класс `LISTWITHTOC`. Он предоставляет функцию `entries` и отслеживает все пары «заголовок — тело», добавленные с помощью функции `item`. Класс аннотирован как `@HtmlTagMarker`, чтобы гарантировать соответствие правилам DSL, описанным ранее в подразделе 13.2.2:

```
@HtmlTagMarker
class LISTWITHTOC {
    val entries = mutableListOf<Pair<String, String>>()
    fun item(headline: String, body: String) {
        entries += headline to body
    }
}
```

После того как класс `LISTWITHTOC` будет определен, необходимо предоставить способ вызвать его, как в листинге 13.5. Для простоты предположим, что требуется получить

возможность добавить список с оглавлением непосредственно в элемент `body` нашего HTML. Поэтому можно сделать `listWithToc` функцией-расширением `BODY`. Ее параметр `block` — лямбда, и в ней можно указать записи в списке. Функцию `item` требуется вызывать внутри этой лямбды, поэтому нужно указать ее тип приемника, `LISTWITHTOC`. Функция `listWithToc` создает новый экземпляр класса `LISTWITHTOC` и вызывает лямбду `block`, переданную в качестве параметра для данного экземпляра. Это вызовет выполнение кода в теле `block`, в том числе всех вызовов функции `item()`, а в список `entries` будут добавляться элементы. Затем вычисленный список `entries` используется для создания оглавления и собственно абзацев с соответствующими заголовками. Наша функция представляет собой расширение класса `BODY`, поэтому можно вызывать упомянутые ранее функции (`ul`, `li`, `h2` и `p`), чтобы добавить список к окружающему тегу:

```
fun BODY.listWithToc(block: LISTWITHTOC.() -> Unit) {
    val listWithToc = LISTWITHTOC()
    listWithToc.block()
    ul {
        for ((index, entry) in listWithToc.entries.withIndex()) {
            li { a("#$index") { +entry.first } }
        }
    }
    for ((index, entry) in listWithToc.entries.withIndex()) {
        h2 { id = "$index"; +entry.first }
        p { +entry.second }
    }
}
```

ПРИМЕЧАНИЕ

Чтобы упростить материал, в этом примере в качестве типа приемника мы использовали класс `BODY`. Однако в файле `kotlinx.html` есть и более обобщенный тип — `HtmlBlockTag`. Он представляет собой любой блочный элемент HTML. Переход от `BODY` к `HtmlBlockTag` позволит вызывать функцию `listWithToc` везде, где предполагается наличие HTML-блока.

Этот пример показывает, как средства абстракции и повторного использования способствуют улучшению кода и делают его более понятным. Теперь рассмотрим еще один инструмент поддержки более гибких структур в ваших DSL — соглашение `invoke`.

13.3. БОЛЕЕ ГИБКОЕ ВЛОЖЕНИЕ БЛОКОВ С ПОМОЩЬЮ СОГЛАШЕНИЯ INVOKE

Соглашение `invoke` позволяет вызывать объекты пользовательских типов как функции. Ранее мы обсуждали, что объекты функциональных типов можно вызывать как функции, а соглашение `invoke` дает вам возможность определить собственные объекты, которые поддерживают тот же синтаксис.

Обратите внимание: эта возможность не предназначена для повседневного использования, поскольку при злоупотреблении ею можно написать трудный для понимания код, например `1()`. Но иногда она очень полезна в DSL. Мы расскажем о причинах, но сначала обсудим само соглашение.

13.3.1. Соглашение `invoke`: объекты, вызываемые как функции

В главе 9 подробно описывалась концепция соглашений в Kotlin: названные особым образом функции вызываются не с помощью обычного синтаксиса вызова методов, а с помощью других, более лаконичных обозначений. Напомним, что одним из обсуждаемых соглашений было `get`. Эта функция позволяет получить доступ к объекту с помощью оператора индексированного доступа. Для переменной `foo` типа `Foo` вызов `foo[bar]` переводится в `foo.get(bar)`. При этом соответствующая функция `get` определяется как член класса `Foo` или функция-расширение класса `Foo`.

По сути, соглашение `invoke` делает то же самое, за исключением того, что квадратные скобки заменяются круглыми. Класс с модификатором `operator`, для которого определен метод `invoke`, можно вызвать как функцию. В листинге 13.16 приведен пример работы этого механизма.

Листинг 13.16. Определение метода `invoke` в классе

```
class Greeter(val greeting: String) {
    operator fun invoke(name: String) { ← Определение метода invoke в классе Greeter
        println("$greeting, $name!")
    }
}

fun main() {
    val bavarianGreeter = Greeter("Servus")
    bavarianGreeter("Dmitry") ← Вызов экземпляра Greeter в качестве функции
    // Servus, Dmitry!
}
```

Этот код определяет метод `invoke` в классе `Greeter`. Он дает возможность вызывать экземпляры `Greeter`, как если бы они были функциями. Внутри системы выражение `bavarianGreeter("Dmitry")` компилируется в вызов метода `bavarianGreeter.invoke("Dmitry")`. Здесь нет никакой тайны. Метод работает подобно регулярному соглашению, предоставляя возможность заменить многословное выражение более кратким и понятным.

Метод `invoke` не ограничен какой-либо конкретной сигнатурой. Можно определить его с любым количеством параметров, а также с любым типом возвращаемого значения или даже определить несколько перегрузок `invoke` с различными типами параметров. При вызове экземпляра класса в качестве функции можно использовать все эти сигнатуры для вызова.

Метод `invoke` уже встречался в десятой главе. Там мы обсуждали, что можно вызвать переменную типа функции, допускающей значение `null`, в форме `lambda?.invoke()`, используя синтаксис безопасного вызова с именем метода `invoke`.

После того как вы изучили соглашение `invoke`, вам должно быть понятно, что обычный способ вызова лямбды (со скобками после нее, как в `lambda()`) — не что иное, как применение этого соглашения. Если лямбды не встроены, то компилируются

в классы, реализующие интерфейсы `FunctionN` (`Function1` и т. д.), а эти интерфейсы определяют метод `invoke` с соответствующим количеством параметров:

```
interface Function2<in P1, in P2, out R> {
    operator fun invoke(p1: P1, p2: P2): R
}
```

← Этот интерфейс обозначает функцию, принимающую ровно два параметра

При вызове лямбды в качестве функции операция преобразуется в вызов метода `invoke`, и это благодаря соглашению.

Рассмотрим практическую ситуацию, когда соглашение `invoke` можно использовать в контексте создания внутренних DSL в Kotlin.

13.3.2. Соглашение `invoke` в DSL: объявление зависимостей в Gradle

Вернемся к примеру DSL в Gradle для настройки зависимостей модуля. Это приведенный ранее код:

```
dependencies {
    testImplementation(kotlin("test"))
    implementation("org.jetbrains.exposed:exposed-core:0.40.1")
    implementation("org.jetbrains.exposed:exposed-dao:0.40.1")
}
```

Часто требуется поддержка показанной здесь вложенной блочной структуры и плоской структуры вызовов в одном и том же API. Другими словами, потребуется разрешить использование следующих вариантов:

```
dependencies.implementation("org.jetbrains.exposed:exposed-core:0.40.1")

dependencies {
    implementation("org.jetbrains.exposed:exposed-core:0.40.1")
}
```

Благодаря такому дизайну пользователи DSL могут применять вложенную структуру блоков при настройке нескольких элементов и плоскую структуру вызовов, чтобы сделать код более лаконичным при настройке только одного элемента.

В первом случае вызывается метод `implementation` для переменной `dependencies`. Вторую нотацию можно выразить, определив метод `invoke` для переменной `dependencies` так, чтобы он принимал в качестве аргумента лямбду. Полный синтаксис этого вызова выглядит следующим образом: `dependencies.invoke {...}`.

Объект `dependencies` представляет собой экземпляр класса `DependencyHandler`, определяющего методы `implementation` и `invoke`. Метод `invoke` принимает в качестве аргумента лямбду с приемником, а тип приемника этого метода снова будет `DependencyHandler`.

Происходящее в теле лямбды уже должно быть вам знакомо: `DependencyHandler` выступает в роли приемника, и такие методы, как `implementation`, можно вызывать для него непосредственно. В следующем коротком примере показано, как реализуется эта часть класса `DependencyHandler` (листинг 13.17).

Листинг 13.17. Использование соглашения `invoke` для поддержки гибкого синтаксиса DSL

```
class DependencyHandler {
    fun implementation(coordinate: String) { ← Определение API обычной команды
        println("Added dependency on $coordinate")
    }

    operator fun invoke(
        body: DependencyHandler.() -> Unit) { ← Определение соглашения invoke
        body() ← Выражение this становится для поддержки API DSL
                приемником тела
                функции — this.body()
    }
}

fun main() {
    val dependencies = DependencyHandler()
    dependencies.implementation("org.jetbrains.kotlin:kotlinx-coroutines-
    ➤ core:1.8.0")
    // Added dependency on org.jetbrains.kotlin:kotlinx-coroutines-core:1.8.0
    dependencies {
        implementation("org.jetbrains.kotlin:kotlinx-datetime:0.5.0")
    }
    // Added dependency on org.jetbrains.kotlin:kotlinx-datetime:0.5.0
}
```

При определении первой зависимости можно вызвать метод `implementation` напрямую. Второй вызов фактически переводится следующим образом:

```
dependencies.invoke({
    this.implementation("org.jetbrains.kotlin:kotlinx-datetime:0.5.0")
})
```

Другими словами, `dependencies` вызывается как функция, и лямбда будет передана в качестве аргумента. Тип параметра лямбды — тип функции с приемником, а тип приемника — тот же тип `DependencyHandler`. Метод `invoke` вызывает лямбда-выражение. Это метод класса `DependencyHandler`, так что экземпляр данного класса доступен в качестве скрытого приемника, поэтому указывать его в явном виде при вызове метода `body()` не нужно.

Один небольшой фрагмент кода, переопределенный метод `invoke`, значительно увеличил гибкость API DSL. Это универсальный паттерн, и вы можете использовать его в собственных DSL, внося минимальные изменения.

Вы познакомились с двумя новыми возможностями Kotlin — лямбдами с приемниками и соглашением `invoke`. Они помогут вам создавать DSL. Теперь посмотрим, как описанные ранее возможности Kotlin проявляются в контексте DSL.

13.4. DSL KOTLIN НА ПРАКТИКЕ

Вы уже знакомы со всеми возможностями Kotlin для создания DSL. Одни из них, такие как расширения и инфиксные вызовы, вы уже должны активно использовать. Другие, такие как лямбды с приемниками, впервые были подробно описаны в текущей главе. Настало время применить все эти знания на практике и рассмотреть ряд практических примеров создания DSL. Мы затронем довольно разнообразные темы: тестирование, поддержка литералов дат и запросы к базе данных.

13.4.1. Цепочка инфиксных вызовов: функция `should` в тестовых фреймворках

Чистый синтаксис — один из ключевых признаков внутреннего DSL, и его можно реализовать, сократив количество знаков препинания в коде. Большинство внутренних DSL сводятся к последовательностям вызовов методов, поэтому в них часто используются любые возможности для уменьшения синтаксического шума в вызовах методов. В Kotlin эти возможности содержат описанный ранее сокращенный синтаксис для вызова лямбд, а также *инфиксные вызовы функций* — их мы обсуждали в главе 3. Здесь же сосредоточимся на их использовании в DSL.

Рассмотрим приведенный ранее в этой главе пример с использованием DSL Kotest (<https://github.com/kotest/kotest>). Тест `testKPrefix` провалится при проверке логики, если значение переменной `s` не начинается с `K`. Обратите внимание, что код читается почти как текст на английском языке: *The s string should start with this letter* (Строка `s` должна начинаться с этой буквы).

В листинге 13.18 показывается, насколько просто применять инфиксные вызовы в контексте DSL и как уменьшить количество шума в коде.

Листинг 13.18. Выражение логического утверждения проверки с помощью Kotest DSL

```
import io.kotest.matchers.should
import io.kotest.matchers.string.startsWith
import org.junit.jupiter.api.Test

class PrefixTest {
    @Test
    fun testKPrefix() {
        val s = "kotlin".uppercase()
        s should startWith("K")
    }
}
```

Чтобы реализовать этот синтаксис в собственном DSL, необходимо объявить функцию `should` с модификатором `infix` (листинг 13.19).

Листинг 13.19. Реализация функции `should`

```
infix fun <T> T.should(matcher: Matcher<T>) = matcher.test(this)
```

В функции `should` ожидается поступление экземпляра `Matcher` — это общий интерфейс выполнения утверждений для переменных. Функция `startsWith` реализует интерфейс `Matcher` и проверяет, начинается ли строка с заданной подстроки (листинг 13.20).

Листинг 13.20. Определение интерфейса сопоставления для Kotest DSL

```
interface Matcher<T> {
    fun test(value: T)
}

fun startWith(prefix: String): Matcher<String> {
    return object : Matcher<String> {
        override fun test(value: String) {
            if(!value.startsWith(prefix)) {
                throw AssertionError("$value does not start with $prefix")
            }
        }
    }
}
```

ПРИМЕЧАНИЕ

Собственно Kotest DSL определяет некоторую дополнительную функциональность в интерфейсе `Matcher` и расширяет функцию `test`, возвращая экземпляр `MatcherResult`, используемый для обработки совпадений. Для краткости мы упростили сигнатуры этих интерфейсов. Однако реализация собственного интерфейса `Matcher`, возвращающего `MatcherResult`, очень похожа на код, представленный в контексте данного подраздела. Поэтому в качестве упражнения вы можете создать собственный настоящий интерфейс сопоставления Kotest.

Это был довольно сложный пример создания DSL, но результат настолько хорош, что стоит разобраться в работе данного паттерна. Сочетание инфиксных вызовов и экземпляров `object` позволяет создавать довольно сложные грамматики для DSL и использовать эти DSL с чистым синтаксисом. И конечно, DSL остается полностью статически типизированным — неправильная комбинация функций и объектов не будет скомпилирована.

13.4.2. Определение расширений для примитивных типов: работа с датами

Теперь рассмотрим последнюю из анонсированных в начале главы тем:

```
val now = Clock.System.now()
val yesterday = now - 1.days
val later = now + 5.hours
```

Библиотека `kotlinx.datetime` (<https://github.com/Kotlin/kotlinx-datetime>) предоставляет такой DSL для работы с датой и временем.

Сам DSL можно описать несколькими строками кода. В листинге 13.21 показана соответствующая часть реализации.

Листинг 13.21. Определение DSL для работы с датой

```
import kotlin.time.DurationUnit

val Int.days: Duration
    get() = this.toDuration(DurationUnit.DAYS)

val Int.hours: Duration
    get() = this.toDuration(DurationUnit.HOURS)
```

← Это выражение указывает на значение числовой константы

← Делегирование преобразования встроенной функции toDuration

В этом коде `days` и `hours` — свойства-расширения типа `Int`. В Kotlin нет ограничений на типы, используемые в качестве приемников для функций-расширений, — можно определять расширения для примитивных типов и вызывать их для констант. Разумеется, с помощью этого подхода можно объявлять собственные пользовательские измерения. Например, две недели (`fortnights`) означают период 14 дней:

```
val Int.fortnights: Duration get() =
    (this * 14).toDuration(DurationUnit.DAYS)
```

Все эти свойства-расширения возвращают значение типа `Duration`, а это тип Kotlin, служащий для представления количества времени между определенными моментами. Теперь, когда вы начали понимать, как работает этот простой DSL, перейдем к более сложной задаче — реализации DSL для запросов к базе данных.

13.4.3. Функции-расширения членов класса: внутренний DSL для SQL

Вы убедились, что функции-расширения играют важную роль в разработке DSL. В этом разделе мы обсудим еще один упоминавшийся ранее прием — объявление функций-расширений и свойств-расширений в классе. Такая функция или свойство будут одновременно и членом содержащего ее класса, и расширением какого-то другого типа. Мы называем такие функции и свойства *расширениями-членами*.

Рассмотрим несколько примеров использования расширений-членов. Они происходят из фреймворка `Exposed` — внутреннего DSL для SQL. Однако прежде необходимо обсудить, как `Exposed` позволяет определять структуру базы данных.

Для работы с таблицами SQL фреймворк `Exposed` требует объявлять их как объекты, расширяющие класс `Table`. Объявление простой таблицы `Country` с двумя столбцами выглядит следующим образом (листинг 13.22).

Листинг 13.22. Объявление таблицы в `Exposed`

```
object Country : Table() {
    val id = integer("id").autoIncrement()
    val name = varchar("name", 50)
    override val primaryKey = PrimaryKey(id)
}
```


Это объявление соответствует таблице в базе данных. Чтобы ее создать, необходимо подключиться к базе данных и вызвать метод `SchemaUtils.create(Country)` в транзакции. В проекте с базой данных H2 (<https://h2database.com/>) в памяти это будет выглядеть следующим образом:

```
fun main() {
    val db = Database.connect("jdbc:h2:mem:test", driver = "org.h2.Driver")
    transaction(db) {
        SchemaUtils.create(Country)
    }
}
```

Этот код генерирует необходимый SQL-оператор на основе объявленной структуры таблицы:

```
CREATE TABLE IF NOT EXISTS Country (
    id INT AUTO_INCREMENT NOT NULL,
    name VARCHAR(50) NOT NULL,
    CONSTRAINT pk_Country PRIMARY KEY (id)
)
```

Объявления в исходном коде Kotlin, как и при генерации HTML, становятся частями сгенерированного SQL-оператора. При изучении типов свойств объекта `Country` можно заметить, что в них есть тип `Column` с необходимым аргументом типа: `id` типа `Column<Int>` и `name` типа `Column<String>`.

Класс `Table` в фреймворке `Exposed` определяет все доступные в таблице типы столбцов, в том числе только что приведенные:

```
class Table {
    fun integer(name: String): Column<Int>
    fun varchar(name: String, length: Int): Column<String>
    // ...
}
```

Методы `integer` и `varchar` создают новые столбцы для хранения целых чисел и строк соответственно.

А теперь установим свойства столбцов. Именно здесь можно использовать расширения-члены:

```
val id = integer("id").autoIncrement()
```

Такие методы, как `autoIncrement`, служат для указания свойств каждого столбца. Каждый метод можно вызвать для экземпляров `Column`, и он вернет экземпляр, для которого был вызван, что позволяет создавать цепочки методов. Упрощенное объявление этой функции будет выглядеть так:

```
class Table {
    fun Column<Int>.autoIncrement(): Column<Int>
    // ...
}
```

← Автоматический инкремент применяется только к целочисленным значениям

Несмотря на то что `autoIncrement` представляет собой функцию-член класса `Table` (она не может быть использована вне области действия этого класса), она остается функцией-расширением типа `Column<Int>`. Теперь понятно, почему имеет смысл объявлять методы как расширения-члены — область видимости будет ограничена. И указать свойства столбца вне контекста таблицы невозможно, так как необходимые методы не будут разрешаться.

Еще одна замечательная особенность функций-расширений — возможность ограничить тип приемника. Любой столбец таблицы может быть ее первичным ключом, но автоматический инкремент применяется только к числовым столбцам. Это можно выразить в API, объявив метод `autoIncrement` как расширение типа `Column<Int>`. Попытка пометить столбец другого типа как автоматически инкрементируемый не будет скомпилирована.

РАСШИРЕНИЯ-ЧЛЕНЫ ОСТАЮТСЯ ЧЛЕНАМИ КЛАССА

У расширений-членов есть и обратная сторона — отсутствие расширяемости. Они принадлежат классу, поэтому определить новые расширения-члены на стороне невозможно.

Для примера представьте, что требуется добавить поддержку новой базы данных в `Exposed` и она поддерживает некоторые новые атрибуты столбцов. Чтобы достичь этой цели, придется изменить определение класса `Table` и добавить в него функции-расширения членов для новых атрибутов. У вас не получится добавить необходимые объявления, не затрагивая исходный класс, что возможно сделать с обычными расширениями (не являющимися членами). Это связано с отсутствием доступа к экземпляру класса `Table`, где они могли бы хранить определения.

На момент написания книги команда Kotlin разрабатывала функциональную возможность языка для решения, помимо прочего, этой проблемы — *контекстные приемники*. Они дадут возможность указать несколько типов приемника для функции. Например, функцию `autoIncrement`, которой требуется доступ к классу `Table` и экземпляру `Column<Int>`, можно будет объявить как функцию-расширение `Column<Int>` с контекстным приемником `Table`:

```
context(Table)
fun Column<Int>.autoIncrement(): Column<Int> {
    // доступ к свойствам и методам `Table` и `Column<Int>`
}
```

Пока что предложение по контекстным приемникам находится на стадии доработки, но возможности прототипа этих сущностей уже можно оценить. Дополнительную информацию, в том числе о том, как опробовать прототип, можно найти в документе *Kotlin Evolution and Enhancement Process (KEEP)*: <http://mng.bz/67Ge>.

Рассмотрим еще одну функцию-расширение из простого запроса `SELECT`. Представьте, что вы объявили две таблицы, `Customer` и `Country`, и каждая запись в `Customer` хранит ссылку на страну принадлежности клиента. Код, приведенный в листинге 13.23, выводит имена всех клиентов, проживающих в США.

Листинг 13.23. Объединение двух таблиц в Exposed

```
val result = (Country innerJoin Customer)
    .select { Country.name eq "USA" }
result.forEach { println(it[Customer.name]) }
```

← Соответствует данному SQL-коду:
WHERE Country.name = "USA"

Функцию `select` можно вызвать для экземпляра `Table` или для объединения двух таблиц. Ее аргументом выступает лямбда, содержащая условие выбора необходимых данных.

Откуда взялся метод `eq`? Теперь мы можем сказать, что это инфиксная функция, принимающая в качестве аргумента "USA". И ваше предположение, что это еще одно расширение-член, будет верным.

Мы снова сталкиваемся с функцией-расширением `Column`, и она также является членом класса. Следовательно, ее можно использовать только в соответствующем контексте: например, для указания условия метода `select`. Упрощенные объявления методов `select` и `eq` выглядят следующим образом:

```
fun Table.select(where: SqlExpressionBuilder.() -> Op<Boolean>) : Query

object SqlExpressionBuilder {
    infix fun<T> Column<T>.eq(t: T) : Op<Boolean>
    // ...
}
```

Объект `SqlExpressionBuilder` определяет множество способов выражения условий: сравнение значений, проверка на отсутствие `null`, выполнение арифметических операций и т. д. Вы не будете ссылаться на него в коде в явном виде, но вам предстоит регулярно вызывать его методы, когда этот объект будет скрытым приемником. Функция `select` принимает в качестве аргумента лямбду с приемником, а объект `SqlExpressionBuilder` будет в ней скрытым приемником. Такая конструкция позволяет использовать в теле лямбды все возможные функции-расширения данного объекта, например `eq`.

Мы обсудили два типа расширений для столбцов: одни служат для объявления класса `Table`, другие — для сравнения значений в условии. Без расширений-членов пришлось бы объявить все эти функции как расширения или члены класса `Column`, и тогда их можно было бы использовать в любом контексте. А подход с расширениями-членами дает возможность контролировать это.

ПРИМЕЧАНИЕ

В главе 9 мы рассмотрели код для работы с Exposed и рассказали об использовании делегированных свойств во фреймворках. Они часто встречаются в DSL, и во фреймворке Exposed хорошо это видно. Мы не будем повторять описание делегированных свойств, поскольку уже подробно рассказывали о них. Но если вы хотите создать DSL для своих нужд или улучшить свой API и сделать его чище, то имейте в виду, что можете использовать эту возможность.

РЕЗЮМЕ

- Внутренние DSL — паттерн проектирования API. С его помощью можно создавать более выразительные API со структурами, состоящими из множества вызовов методов.
- Лямбды с приемниками используют структуру вложенности, чтобы переопределить разрешение методов в теле лямбды.
- Тип параметра, получающего лямбду с приемником, является типом функции-расширения, а вызывающая функция предоставляет экземпляр приемника при вызове лямбды.
- Преимущество использования внутренних DSL Kotlin, а не внешних шаблонов или языков разметки заключается в возможности повторного использования кода и создания абстракций.
- Определение расширений для примитивных типов позволяет создать удобный синтаксис для различных видов литералов, таких как продолжительность.
- С помощью соглашения `invoke` можно вызывать произвольные объекты так, как если бы они были функциями.
- Библиотека `kotlinx.html` предоставляет внутренний DSL для создания HTML-страниц, и ее можно расширить в соответствии с вашими приложениями.
- В библиотеке `Kotest` есть внутренний DSL, который поддерживает читабельные утверждения в модульных тестах.
- Библиотека `Exposed` предоставляет внутренний DSL для работы с базами данных.

Часть III

Конкурентное программирование, корутины и потоки

Вы уже хорошо изучили сам язык Kotlin и поддерживаемые в нем соглашения. В третьей части книги вы узнаете, как осуществить конкурентное программирование в Kotlin.

Конкурентность — важная тема, независимо от того, какие приложения вы пишете на Kotlin: серверные, мобильные или настольные. Большинство современных приложений должны выполнять несколько задач одновременно, будь то обслуживание или выполнение сетевых запросов, поддержание отзывчивого пользовательского интерфейса при выполнении ресурсоемких задач процессора или предоставление пользователям функционала многозадачности другими способами. Будучи разработчиком Kotlin, вам рано или поздно предстоит работать с конкурентным кодом.

В Kotlin реализован уникальный и надежный подход к конкурентному программированию, основанный на концепции *корутин* — легкой абстракции, работающей поверх потоков. Основные примитивы для конкурентного программирования встроены прямо в язык, а его возможности расширяются за счет библиотеки `kotlinx.coroutines`, разработки компании JetBrains. Во множестве библиотек из экосистемы Kotlin внедрена модель конкурентности на основе корутин, и они предоставляют идиоматические и неблокирующие API. Понимая концепции в этих API, вы сможете использовать все возможности таких библиотек.

В главе 14 представлен обзор модели конкурентности Kotlin. Мы рассмотрим функции приостановки, корутины и основные механики, позволяющие писать конкурентный код на Kotlin.

В главе 15 описывается концепция структурированной конкурентности. Она поможет создать структуру и иерархию конкурентных задач и предоставит средства, обеспечивающие работу механизмов отмены и обработки ошибок.

В главах 16 и 17 рассказывается о потоках — способе Kotlin работать с несколькими последовательными значениями во времени. Он создан на основе корутин и библиотеки операторов, которые можно использовать для преобразования этих потоков значений. Вдобавок вы узнаете о разнице между холодными и горячими потоками, о двух больших категориях потоков и поймете, в каких случаях их следует использовать.

В главе 18 вы погрузитесь в тему обработки ошибок и тестирования конкурентного кода.

Таким образом, к концу этой части вы будете хорошо понимать концепции, связанные с конкурентностью в Kotlin, и сможете писать код, используя эту функциональность.

14

Корутины

В этой главе

- ✓ Понятия конкурентности и параллелизма.
- ✓ Функции приостановки как основные строительные блоки конкурентных операций в Kotlin.
- ✓ Подход Kotlin к конкурентному программированию с помощью корутин.

Современные программы редко выполняют только одно действие за раз. Сетевым приложениям может потребоваться сделать или предоставить несколько сетевых запросов, но не нужно ждать завершения каждого сетевого запроса, прежде чем начать следующий. Мобильным и настольным приложениям может потребоваться выполнять любые действия: запрашивать базы данных на устройстве, использовать датчики, взаимодействовать с другими устройствами — и при этом перерисовывать пользовательский интерфейс (user interface, UI) 60 раз в секунду и более.

Чтобы справиться с этим набором задач, современные приложения должны выполнять несколько действий одновременно, асинхронно. Это означает, что разработчикам нужны инструменты, позволяющие различным операциям выполняться независимо, без взаимной блокировки, а также координировать и синхронизировать конкурентные задачи.

В этой главе мы рассмотрим подход Kotlin к выполнению подобных асинхронных вычислений с помощью корутин. Они позволяют писать одновременно конкурентный и параллельный код.

14.1. КОНКУРЕНТНОСТЬ И ПАРАЛЛЕЛИЗМ

Прежде чем углубиться в конкурентное программирование на Kotlin, вкратце определим, что мы имеем в виду, когда говорим о конкурентности, а также о ее связи с концепцией параллелизма.

Конкурентность — общий термин, означающий одновременную работу над несколькими задачами. Но не обязательно, чтобы они физически выполнялись одновременно, — система, в которой объединенно выполняются несколько частей кода, считается конкурентной. Это означает, что даже те приложения, которые работают на одном ядре процессора, могут использовать конкурентность. Они будут переключаться между несколькими конкурентными задачами, и даже одного ядра может быть достаточно для выполнения тяжелых вычислений при сохранении отзывчивости пользовательского интерфейса (рис. 14.1).

Если конкурентность — общая способность кода делиться на части для одновременного выполнения, то *параллелизм* означает физическое выполнение нескольких задач одновременно на нескольких ядрах центрального процессора (ЦП). Параллельные вычисления могут эффективно использовать современное многоядерное оборудование, что зачастую делает их более эффективными. Однако их применение сопряжено и с рядом определенных проблем. Мы кратко поговорим о них в подразделе 14.7.4. На рис. 14.2 показан пример параллелизма.

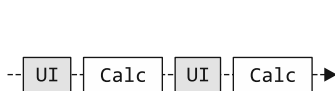


Рис. 14.1. Благодаря чередованию выполнения даже при работе на одном ядре приложение может выполнять несколько задач одновременно. Переключаясь между несколькими задачами по мере необходимости (например перерисовывая пользовательский интерфейс и выполняя части длительного вычисления), оно применяет конкурентность

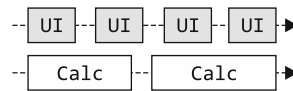


Рис. 14.2. Параллелизм означает физическое выполнение нескольких задач одновременно на нескольких ядрах процессора. В этом примере длительные вычисления выполняются в фоновом режиме, в то время как пользовательский интерфейс находится в процессе рендеринга

Описание различий между темами конкурентности и параллелизма получилось очень кратким, но обе эти области сами по себе достаточно объемные. Прочитав такие книги, как *Grokking Concurrency* Кирилла Боброва¹, вы сможете получить общее представление об их тематике. С помощью корутин Kotlin можно выполнять как конкурентные, так и параллельные вычисления, и в этой главе вы увидите примеры тех и других.

¹ Бобров К. Грокаем конкурентность. — СПб.: Питер, 2025.

14.2. КОНКУРЕНТНОСТЬ KOTLIN: ФУНКЦИИ ПРИОСТАНОВКИ И КОРУТИНЫ

Корутины — мощная функциональная возможность Kotlin. Она предоставляет удобный способ написания неблокирующего конкурентного кода, предназначенного для асинхронного выполнения. По сравнению с традиционными подходами, такими как потоки, корутины гораздо более легковесны. Вдобавок благодаря *структурированной конкурентности* они предоставляют средства для управления конкурентными задачами и их жизненным циклом.

Мы начнем изучение со сравнения классических потоков и корутин Kotlin. Далее рассмотрим базовую абстракцию использования корутин — *функции приостановки*. Они позволяют писать код, который выглядит последовательным без недостатков блокирующих потоков. Кроме того, мы сравним корутины с другими моделями конкурентности, которые подчеркивают простоту абстракций корутин Kotlin. К числу таких моделей можно отнести обратные вызовы, фьючерсы и реактивные потоки.

14.3. СРАВНЕНИЕ ПОТОКОВ И КОРУТИН

Классической абстракцией для конкурентного программирования на JVM стало использование *потоков*. Они позволяют задавать блоки кода для конкурентного и независимого друг от друга выполнения. Читая данную книгу, вы неоднократно убеждались в том, что Kotlin на 100 % совместим с Java, и потоки не станут исключением. Если требуется использовать потоки, как в Java, то можно воспользоваться удобными функциями из стандартной библиотеки Kotlin. В частности, запустить новый поток можно с помощью функции `thread`. В примере ниже она используется для запуска нового потока и отображения его названия:

```
import kotlin.concurrent.thread

fun main() {
    println("I'm on ${Thread.currentThread().name}")
    thread { ← Запуск нового потока, выполняющего заданный блок кода
        println("And I'm on ${Thread.currentThread().name}")
    }
}

// I'm on main
// And I'm on Thread-0
```

Потоки помогут сделать приложение более отзывчивым и лучше использовать современные системы, позволяя распределять работу между отдельными ядрами многоядерного процессора. Однако за использование потоков в приложениях приходится платить. В JVM каждый создаваемый поток обычно соответствует потоку под управлением операционной системы. Создание и управление такими системными потоками может быть дорогостоящим, и даже современные системы могут эффективно управлять лишь несколькими тысячами потоков одновременно.

Каждому системному потоку необходимо выделить несколько мегабайтов памяти, а переключение между потоками — операция уровня ядра операционной системы. Стоимость этих распределений и операций может быстро увеличиться.

Кроме того, когда поток ожидает завершения операции (например, сетевого запроса), он *блокируется*: не может выполнять никакую другую значимую работу в ожидании ответа и просто спит, занимая системные ресурсы. Это означает, что при создании новых потоков следует проявлять осторожность, и, естественно, не рекомендуется использовать их в мелкомодульной или недолговечной структуре.

Вдобавок ко всему потоки по умолчанию существуют как отдельные процессы. Из-за этого при управлении их работой могут возникать проблемы, особенно в отношении таких понятий, как отмена и обработка исключений. Все это накладывает ограничения на применимость потоков: когда создавать новые (поскольку они дороги, а ожидание результатов может их заблокировать) и как ими управлять (поскольку по умолчанию в них нет понятия «иерархия»).

КОРУТИНЫ И PROJECT LOOM

Проект *Project Loom* (<https://wiki.openjdk.org/display/loom>) — это попытка привнести облегченную конкурентность в виде *виртуальных потоков* в JVM, устранив дорогостоящую связь «один к одному» между потоками JVM и операционной системы. Loom и корутины Kotlin решают схожие задачи, поэтому имеет смысл уделить некоторое время изучению их взаимосвязи.

Корутины впервые появились в Kotlin 1.1, в релизе 2016 года. С тех пор они стали зрелой абстракцией для написания конкурентного кода и уже много лет применяются в эксплуатируемых приложениях. Корутины были разработаны как конкурентная абстракция, не зависящая от базовой модели выполнения. Более того, их дизайн отвязан от JVM, что позволяет использовать корутины при выполнении кода Kotlin на других платформах, например iOS.

Основная цель проекта Project Loom — дать возможность перенести существующий устаревший код, перегруженный операциями ввода-вывода, на виртуальные потоки. Именно в этой части проект наиболее силен, но здесь же и его основная слабость. Loom был доработан на основе существующих API Java для потоков и ввода-вывода. Это означает, что на уровне языка нет различий между быстрыми локальными вычислениями и функциями, которые могут ожидать информацию из сети в течение непредсказуемого времени (*функции приостановки* в терминах Kotlin). Это усложняет понимание кода в больших кодовых базах, где локальные операции (такие как UI, кэш и обновление состояния) смешиваются с удаленным доступом к данным.

Устаревшие API потоков Java были разработаны для ситуации, когда потоки стоят дорого и создаются редко. Однако в коде с высокой конкурентностью корутины запускаются постоянно. API корутин Kotlin были разработаны с нуля с учетом эргономики такого кода и оптимизированы для эффективности и минимального потребления памяти в таких ситуациях. Корутины Kotlin требуют настолько малых затрат, что для выполнения тривиальной операции, например инкрементирования счетчика, зачастую можно создать новую корутину. С их помощью можно даже реализовать операции по обработке коллекций для

последовательностей, используя функцию `sequence {}`. Такой уровень эффективности невозможно даже представить при работе с виртуальными потоками Project Loom.

На момент написания книги в Project Loom выполнялись эксперименты по внедрению структурированной конкурентности (тема, подробно описанная в главе 15) в унаследованные API потоков, на которых он основан. Хотя форма этих API еще не окончательна, можно утверждать, что она далека от API корутин Kotlin — в них структурированная конкурентность занимает центральное место и лежит в основе принципов проектирования всего API. В коде, где активно используется конкурентность, очень легко совершить ошибку, если запустить конкурентную операцию и забыть обработать ожидание или отмену. Тем самым создается потенциальная утечка ресурсов. Сама форма API корутин Kotlin изначально направлена на исключение таких ошибок. Чтобы корутина допустила утечку, нужно не пожалеть сил и написать дополнительный код. Кратчайший код на основе корутин всегда будет правильным.

Однако корутины Kotlin смогут использовать и новую функциональность из Loom, которая позволяет получить преимущества этого проекта. А именно, более эффективно интегрировать код, написанный с помощью блокирующих API. Этого можно достичь, предоставив диспетчер виртуальных потоков на базе Loom для корутин. (Подробнее о диспетчерах поговорим в разделе 14.7.)

Изучение корутин научит вас проектировать конкурентный код, учитывая четкое разграничение локальных и удаленных (приостанавливающих) операций и соблюдая практики структурированной конкурентности во избежание утечки ресурсов. Даже если в будущем вы будете программировать в среде, где эти проблемы не проявляются напрямую или незаметны в языке и API, как, например, в корутинах Kotlin, эти практики необходимы для написания чистого конкурентного кода.

В Kotlin появилась альтернативная потокам абстракция под названием «*корутины*». Она представляет собой приостанавливаемые вычисления. Корутины Kotlin можно использовать везде, где обычно используются потоки, но при этом они обладают рядом преимуществ.

- Корутины — очень легкая абстракция. На обычном ноутбуке можно легко запустить 100 000 или более корутин. Их создание и управление обходится недорого, а это означает, что можно использовать их более широко и более точно, чем потоки, даже для очень краткосрочных задач.
- Корутины могут приостанавливать выполнение, не блокируя системные ресурсы, и возобновлять его с точки остановки в более поздний момент. Это делает их эффективными для многих асинхронных задач, таких как ожидание сетевых запросов или задач ввода-вывода, по сравнению с блокирующими потоками.
- Корутины создают структуру и иерархию конкурентных задач с помощью концепции под названием «*структурированная конкурентность*», предоставляя механизмы для отмены и обработки ошибок. Когда часть конкурентных вычислений заканчивается неудачей или больше не требуется, структурированная конкурентность гарантирует, что другие дочерние корутины вычислений будут отменены.

Внутри системы корутины реализуются путем запуска одного или нескольких потоков JVM (рис. 14.3; подробнее об этом поговорим в разделе 14.7). Это означает, что код на основе корутин по-прежнему может использовать параллелизм, заложенный в базовой модели потоков, но при этом вы не ограничены лимитами потоков, наложенными операционной системой.

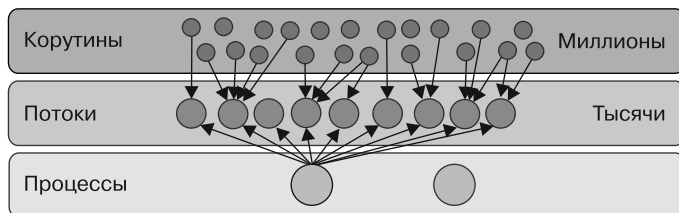


Рис. 14.3. Корутины — облегченная абстракция поверх потоков. Если в одном процессе могут быть тысячи потоков, то с помощью корутин можно запускать миллионы конкурентных задач

Далее мы обсудим модель конкурентного программирования на Kotlin с нуля. И начнем с самого основного строительного блока — функций приостановки.

14.4. ФУНКЦИИ ПРИОСТАНОВКИ

Распространенные подходы к конкурентности подразумевают потоки, реактивные потоки или обратные вызовы. Одно из ключевых отличительных свойств работы с корутинами Kotlin заключается в том, что во многих случаях не требуется сильно менять «форму» вашего кода — он по-прежнему выглядит последовательным. Подробнее рассмотрим, как функции приостановки позволяют это сделать.

14.4.1. Код на основе функций приостановки выглядит последовательным

Чтобы понять, как работают функции приостановки в Kotlin, рассмотрим небольшой фрагмент традиционной логики приложения и подумаем над его улучшением. В листинге 14.1 создается функция `showUserInfo`. Она отвечает за запрос некоторой информации из сети и отображение ее пользователю. Функции `login` и `loadUserData` выполняют сетевые запросы. Пока что в этом коде конкурентность не применяется — он просто вызывает функции одну за другой, а затем возвращает значение, получив ответ на сетевой запрос.

Листинг 14.1. Блокирующий код для вызова нескольких функций

```
fun login(credentials: Credentials): UserID
fun loadUserData(userID: UserID): UserData
fun showData(data: UserData)
```

Будем считать,
что эти функции
блокируются

```
fun showUserInfo(credentials: Credentials) {
    val userID = login(credentials)
    val userData = loadUserData(userID)
    showData(userData)
}
```

С точки зрения вычислений этот код на самом деле выполняет не так уж много работы. Большую часть времени программа ожидает результат сетевой операции, блокируя поток выполнения функции `showUserInfo` (рис. 14.4). Блокировка потоков, как правило, нежелательна, поскольку заблокированный поток тратит ресурсы — количество системных потоков у современных устройств исчисляется тысячами. Например, если этот код используется из сетевого приложения и блокирует один поток на запрос, то блокирующая природа кода может быстро превратиться в верхний предел количества запросов к вашему сервису. Если в приложении есть пользовательский интерфейс, то вызов этой функции без конкурентности приведет к заморозке всего UI до завершения операции. Такое ухудшение пользовательского опыта и производительности системы явно указывает на ошибку в коде.

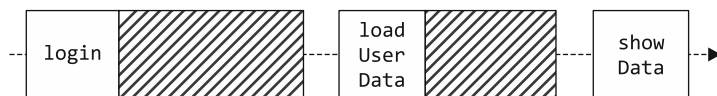


Рис. 14.4. Использование блокирующего кода означает, что приложение тратит вычислительные ресурсы впустую, когда большую часть времени проводит в ожидании (изображено в виде заштрихованного квадрата), а не работая (изображено в виде квадрата с именем функции)

Корутины — точнее, функции приостановки — способны помочь добиться большего. В листинге 14.2 представлен тот же код, но в виде неблокирующей реализации с использованием корутин Kotlin. Обратите внимание, что код по-прежнему выглядит последовательным. Единственное реальное различие заключается в том, что у функций `login`, `loadUserData` и `showUserInfo` есть модификатор `suspend`.

Листинг 14.2. Использование функций приостановки для выполнения той же логики

```
suspend fun login(credentials: Credentials): UserID
suspend fun loadUserData(userID: UserID): UserData
fun showData(data: UserData)

suspend fun showUserInfo(credentials: Credentials) {
    val userID = login(credentials)
    val userData = loadUserData(userID)
    showData(userData)
}
```

Обратите внимание
на модификатор
`suspend`

Так что же означает «пометить функцию приостановкой» (`suspend`)? Это указывает на возможность приостановки данной функции, например, в ожидании ответа сети. Механизм приостановки не блокирует основной поток. Происходит переключение на выполнение другого кода в том же потоке (рис. 14.5).

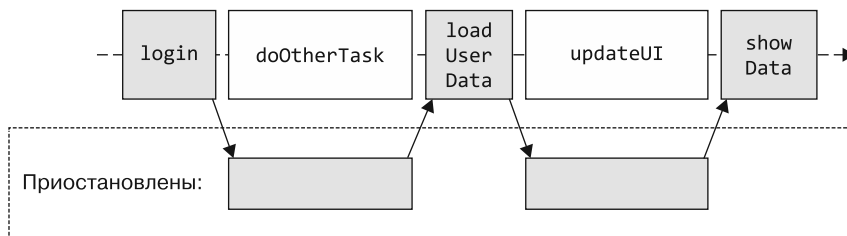


Рис. 14.5. Использование функций приостановки означает, что ожидающие функции не блокируют выполнение. Они приостанавливаются и освобождают место для любых других функций, требующих в это время ресурсов

Обратите внимание, что структура кода осталась без изменений: он по-прежнему выглядит и ведет себя последовательно, но недостатки блокирующего кода удалось исключить. Тело функции `showUserInfo` по-прежнему выполняется сверху вниз, оператор за оператором. Однако, будучи приостановленным, основной поток может выполнять другую работу: рисовать пользовательский интерфейс, обрабатывать запросы пользователей, показывать другие данные и т. д. Описанной ранее проблемы больше не существует — вызов этой функции из потока UI приложения не приведет к зависанию UI. Ожидая ответы от сети, базовый поток может выполнять другую работу.

Конечно, здесь нет никакого волшебства. Программа требует, чтобы базовые библиотеки (например, реализация `login` и `loadUserData`) были реализованы с учетом корутин Kotlin. И действительно, многие библиотеки в экосистеме Kotlin предоставляют такие API. Если речь идет о выполнении сетевых запросов, то это такие библиотеки, как HTTP-клиенты Ktor, Retrofit или OkHttp.

14.5. СРАВНЕНИЕ КОРУТИН С ДРУГИМИ ПОДХОДАМИ

Если вы ранее использовали другие подходы к написанию конкурентного кода — на Java или другом языке программирования, — вам может быть интересно узнать, чем они различаются и как корутины могут улучшить их. Кратко рассмотрим три примера наиболее распространенных подходов: обратные вызовы (callback, или блоки завершения), реактивные потоки (RxJava) и фьючерсы (Future). Не будем углубляться в их подробное описание, но в целях обсуждения рассмотрим несколько примеров реализации. Если вы раньше не использовали эти подходы, можете просто пропустить данный раздел.

При реализации логики, показанной в листинге 14.1, с помощью обратных вызовов необходимо изменить сигнатуру функций `login` и `loadUserData` для предоставления параметра обратного вызова (листинг 14.3).

Листинг 14.3. Использование обратных вызовов для последовательного вызова нескольких функций

```
fun loginAsync(credentials: Credentials, callback: (UserID) -> Unit)
fun loadUserDataAsync(userID: UserID, callback: (UserData) -> Unit)
fun showData(data: UserData)
```

```
fun showUserInfo(credentials: Credentials) {
    loginAsync(credentials) { userID ->
        loadUserDataAsync(userID) { userData ->
            showData(userData)
        }
    }
}
```

Аналогичным образом необходимо переписать функцию `showUserInfo`, используя отдельные обратные вызовы. В итоговом коде теперь есть обратный вызов внутри обратного вызова. Если для двух вызовов функций это еще выглядит терпимо, то по мере роста логики вы быстро окажетесь в ситуации, когда вложенных обратных вызовов много и их трудно читать. Эта проблема настолько известна, что заслужила название «ад обратного вызова» (callback hell).

Со временем появились и другие подходы к борьбе с данной проблемой, но у них свои сложности, требующие изучения и привыкания.

Например, применение класса `CompletableFuture` позволяет избежать вложенности обратных вызовов, но требует изучения семантики новых операторов, таких как `thenCompose` и `thenAccept`. Вдобавок необходимо изменить тип значения, возвращаемого функциями `loginAsync` и `loadUserDataAsync`, — теперь их тип возврата обернут в класс `CompletableFuture` (листинг 14.4).

Листинг 14.4. Использование фьючерсов для последовательного вызова нескольких функций

```
fun loginAsync(credentials: Credentials): CompletableFuture<UserID>
fun loadUserDataAsync(userID: UserID): CompletableFuture<UserData>
fun showData(data: UserData)

fun showUserInfo(credentials: Credentials) {
    loginAsync(credentials)
        .thenCompose { loadUserDataAsync(it) }
        .thenAccept { showData(it) }
}
```

Аналогично реализация с помощью реактивных потоков (например, с помощью `RxJava`) позволяет избежать ада обратных вызовов. Но в таком случае требуется внести изменения в сигнатуры функций: они могут быть и `observable` и использовать такие операторы, как `flatMap`, `doOnSuccess` и `subscribe` (листинг 14.5).

Листинг 14.5. Использование реактивных потоков для реализации той же логики

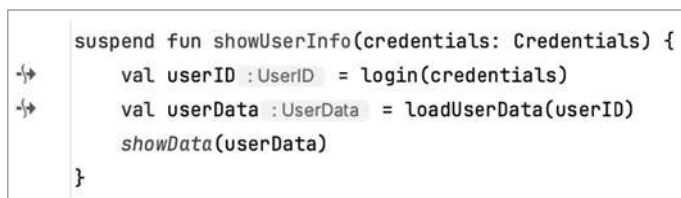
```
fun login(credentials: Credentials): Single<UserID>
fun loadUserData(userID: UserID): Single<UserData>
fun showData(data: UserData)

fun showUserInfo(credentials: Credentials) {
    login(credentials)
        .flatMap { loadUserData(it) }
        .doOnSuccess { showData(it) }
        .subscribe()
}
```

Оба подхода снижают читабельность и увеличивают время на изучение кода. Сравните это с подходом корутин в Kotlin. Если вы применяете его, то вам нужно только пометить функции модификатором `suspend` — остальной код остается прежним, сохраняет свой последовательный вид и позволяет избежать изначальных недостатков блокировки потока. Конечно, существуют разные варианты использования реактивных потоков и фьючерсов. В подразделе 14.6.3 описывается собственная разновидность фьючерсов в Kotlin — *отложенные значения*. А главы 16 и 17 посвящены обсуждению *потоков* — абстракции в стиле реактивного потока, доступной для корутин. Но ни одна из этих абстракций не нужна, когда речь идет о базовом случае применения конкурентности. Если у вас есть код, написанный с использованием других моделей конкурентности, то Kotlin предоставляет функции-расширения, которые позволяют преобразовать многие из их примитивов в версии для работы с корутинами.

14.5.1. Вызов функции приостановки

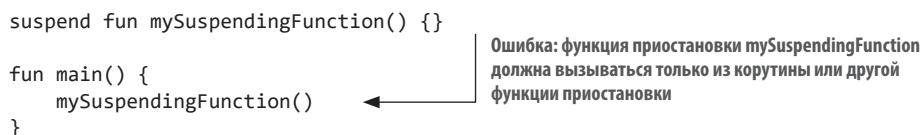
Функция приостановки может на время прекратить свое выполнение, поэтому ее нельзя просто вызвать в любом месте обычного кода — она должна быть вызвана из блока кода, который тоже способен приостановить свое выполнение. Одним из таких типов блоков может быть еще одна функция приостановки. При этом нужно руководствоваться следующей логикой: «Если функция может приостановить свое выполнение, то потенциально может быть приостановлено и выполнение вызывающей ее функции». Как было показано в листинге 14.2, функция приостановки `showUserInfo` может вызывать функции приостановки `login` и `loadUserData`. Конечно, она может вызывать и обычные функции в своем теле, например функцию `showData` (рис. 14.6).



```
suspend fun showUserInfo(credentials: Credentials) {
    val userID : UserID = login(credentials)
    val userData : UserData = loadUserData(userID)
    showData(userData)
}
```

Рис. 14.6. IntelliJ IDEA и Android Studio выделяют вызов функций с модификатором `suspend`, добавляя небольшой значок разрыва потока. Кроме того, в среде IDE можно указать пользовательский цвет для вызовов функций приостановки

Попытка вызвать такую функцию из обычного кода приведет к ошибке:



```
suspend fun mySuspendingFunction() {}

fun main() {
    mySuspendingFunction()
}
```

Ошибка: функция приостановки `mySuspendingFunction` должна вызываться только из корутины или другой функции приостановки

Как же вызвать первую функцию приостановки? Самый простой ответ — пометить модификатором `suspend` функцию `main`. Однако при написании кода в контексте

более крупной кодовой базы, а также в контексте SDK или фреймворка, как в Android, зачастую нельзя просто изменить сигнатуру функции `main`. Именно поэтому такой подход обычно применяется в небольших служебных программах.

Более универсальным и мощным подходом является использование функций *конструкторов корутин*, отвечающих за создание новых *корутин*. Они служат типичными точками входа для вызова функций приостановки.

14.6. ВХОД В МИР КОРУТИН: КОНСТРУКТОРЫ

До настоящего момента функции приостановки были представлены как основной строительный блок конкурентности в Kotlin. Они могут приостанавливать выполнение и вызываются только из другой функции приостановки или из корутины.

Мы приводили логику размышления, с помощью которой можно объяснить, почему одна функция приостановки может вызывать другую. Теперь пора обратить внимание на вторую половину подсказки из сообщения об ошибке — вызов функции приостановки из *корутины*. Данный термин уже использовался в книге, но четкого определения так и не получил. Исправим это.

Корутина — экземпляр приостанавливаемого вычисления. Представьте ее в виде блока кода, который может выполняться конкурентно (или даже параллельно) с другими корутинами, подобно потоку. Эти корутины содержат механизм, необходимый для приостановки выполнения функций, вызванных в их теле. Для создания корутин используется одна из функций конструктора корутин. Доступны несколько функций:

- `runBlocking` предназначена для создания блокирующего контекста, или скоупа, в котором можно вызывать функции приостановки;
- `launch` используется для запуска новых корутин без возвращаемых значений;
- `async` запускает отдельную корутину, которая выполняется параллельно с остальными корутинами, возвращает объект с отложенным выполнением `Deferred`, который ожидает получения результата корутины.

Рассмотрим их более подробно.

ПРИМЕЧАНИЕ

Функциональность базового компилятора Kotlin и стандартной библиотеки, связанная с корутинами, намеренно поддерживается в ограниченном объеме. Большинство описанных далее возможностей не входят в стандартную библиотеку Kotlin, а относятся к сторонней библиотеке `kotlinx.coroutines` (<https://github.com/Kotlin/kotlinx.coroutines>), разработанной компанией JetBrains. Такой подход позволяет функциональности, связанной с конкурентностью, развиваться независимо от циклов выпуска языка. Кроме того, она дает возможность сообществу разработчиков предоставлять альтернативные высокоуровневые библиотеки конкурентности на базе небольшого и стабильного набора основных возможностей. Чтобы проследить код в остальной части книги, необходимо указать артефакт `org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3` или более позднюю версию в качестве зависимости для проекта.

14.6.1. Из обычного кода в область корутин: функция `runBlocking`

Чтобы перекинуть мост между областью «обычного» блокирующего кода и областью функций приостановки, можно использовать функцию `runBlocking` конструктора корутин. Ей требуется передать блок кода, составляющий тело корутины. Она создает и запускает новую корутину и просто блокирует текущий поток до ее завершения. Внутри переданного функции блока кода можно делать вызовы функций приостановки. В примере ниже применяется специальная встроенная функция `delay` для приостановки выполнения корутины на 500 миллисекунд перед выводом текста на экран (листинг 14.6).

Листинг 14.6. Использование функции `runBlocking` для вызова функции приостановки

```
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.milliseconds

suspend fun doSomethingSlowly() {
    delay(500.milliseconds)
    println("I'm done")
}

fun main() = runBlocking {
    doSomethingSlowly()
}
```

← Остановка выполнения функции
на 500 миллисекунд

Но подождите. Разве смысл использования корутин не в том, чтобы избежать блокировки потоков? Так для чего же сейчас применялась функция `runBlocking`? Ведь при ее использовании блокируется один поток. Однако внутри этой корутины можно запустить любое количество дополнительных дочерних корутин, не блокируя потоки. Они будут работать конкурентно, освобождая во время приостановки один занятый поток, чтобы код мог выполняться другими корутинами. Для запуска дополнительных дочерних корутин можно использовать функцию `launch` из конструктора. Обсудим ее более подробно и посмотрим, как будет выглядеть код.

14.6.2. Создание корутин по сценарию «запустить и забыть»: функция `launch`

Функция `launch` используется для запуска новой дочерней корутины. Она обычно применяется в сценариях «запустить и забыть», когда необходимо запустить часть кода и не ждать, пока она вычислит возвращаемое значение. Давайте проверим предыдущее утверждение о том, что функция `runBlocking` блокирует только один поток.

Требуется получить более полное представление о том, когда и где выполняется код, поэтому вместо функции `println` воспользуемся простой функцией `log`. Она добавит дополнительную информацию — временную метку, а также поток вызова функции:

```
private var zeroTime = System.currentTimeMillis()
fun log(message: Any?) =
    println("${System.currentTimeMillis() - zeroTime} " +
        "[${Thread.currentThread().name}] $message")
```

В этом примере конструкторы корутин `runBlocking` и `launch` запускают несколько новых корутин, а функция `log` регистрирует информацию об их выполнении:

```
fun main() = runBlocking {
    log("The first, parent, coroutine starts")
    launch {
        log("The second coroutine starts and is ready to be suspended")
        delay(100.milliseconds)
        log("The second coroutine is resumed")
    }
    launch {
        log("The third coroutine can run in the meantime")
    }
    log("The first coroutine has launched two more coroutines")
}
```

Если запустить этот пример с флагом `-Dkotlin.coroutines.debugJVM` (рис. 14.7) или на экспериментальной площадке Kotlin (где этот флаг добавляется автоматически), то система дополнительно выдаст информацию об имени корутины рядом с именем потока. Эти сведения могут помочь понять работу корутин. В дальнейшем результаты выполнения фрагментов кода будут приводиться с этим активированным флагом.

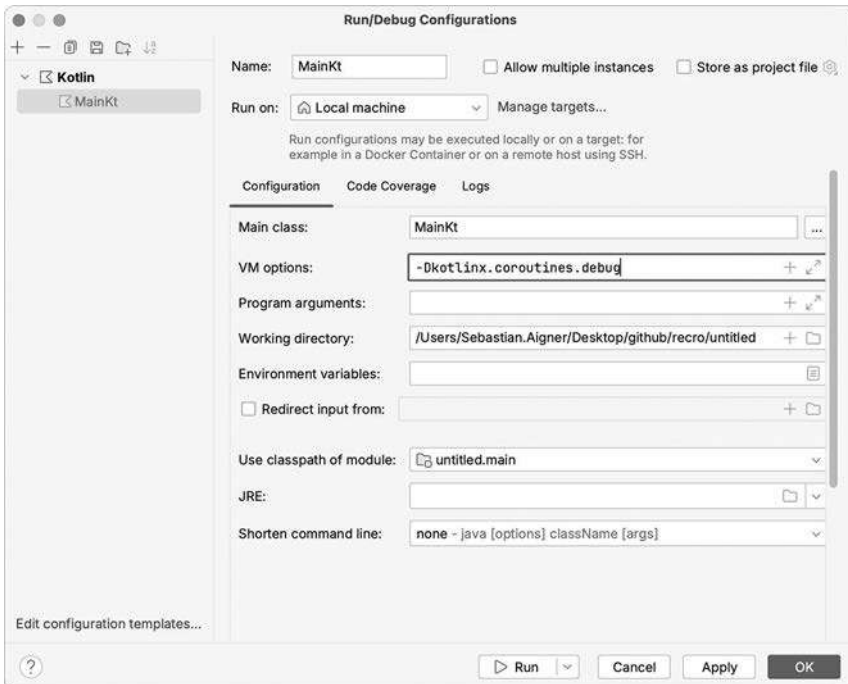


Рис. 14.7. Чтобы задать флаг JVM из IntelliJ IDEA, откройте окно Run Configuration (Конфигурация запуска) в правом верхнем углу и выберите пункт Edit Configurations... (Редактировать конфигурации...). И после этого можно добавить флаг отладки в поле VM Options (Параметры VM)

Когда код будет выполнен, вы получите следующий результат (точное время вывода зависит от вашей машины, но порядок строк остается неизменным):

```
36 [main @coroutine#1] The first, parent, coroutine starts
40 [main @coroutine#1] The first coroutine has launched two more coroutines
42 [main @coroutine#2] The second coroutine starts and is ready to be suspended
47 [main @coroutine#3] The third coroutine can run in the meantime
149 [main @coroutine#2] The second coroutine is resumed
```

В этом примере все корутины выполняются в одном потоке под названием `main`. Выполнение данного кода можно визуализировать (рис. 14.8).

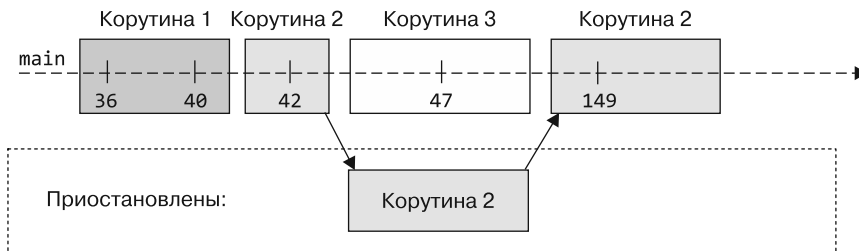


Рис. 14.8. Первая (родительская) корутина начинает и запускает еще две корутины. Корутина 2 выполняется до точки приостановки. Она приостанавливается без блокировки основного потока, освобождая его для выполнения работы корутины 3, а затем возобновляется и завершает свое выполнение

Теперь подробно проследим процесс выполнения, показанный на рис. 14.8. Линия представляет собой временную шкалу основного потока, а прямоугольники на ней — корутины, выполняющиеся в этом потоке в данный момент времени. Ниже показана зона приостановленных в текущий момент корутин.

В этом примере код запускает три корутины: первую (родительскую), запущенную функцией `runBlocking`, и две дочерние, запущенные двойным вызовом функции `launch`. Когда корутина 2 вызывает функцию `delay`, это приводит к приостановке корутины. Это называется *точкой приостановки*. Выполнение корутины 2 теперь приостановлено на указанное время, а основной поток освобожден для работы других корутин. Это означает, что корутина 3 может начать свою работу. Данная корутина содержит только один вызов функции `log`, поэтому быстро завершается. Через указанные 100 миллисекунд корутина 2 возобновляет свою работу, и выполнение всей программы завершается.

ГДЕ ХРАНЯТСЯ ПРИОСТАНОВЛЕННЫЕ КОРУТИНЫ

Вся тяжесть работы с корутинами ложится на компилятор — он генерирует вспомогательный код для приостановки, возобновления и планирования работы корутин. Код приостанавливающих функций преобразуется во время компиляции таким образом, чтобы при приостановке корутины в памяти сохранялась информация о ее состоянии на момент приостановки. На основе этой информации выполнение может быть восстановлено и возобновлено в более поздний момент.

Вспомнив сравнение конкурентности и параллелизма, которое проводилось в разделе 14.1, мы можем определить показанное на рис. 14.8 как пример чередующегося выполнения без параллелизма (все корутины выполняются в одном потоке). Если требуется обеспечить параллельное выполнение в нескольких потоках, то код можно оставить почти без изменений, но надо будет использовать многопоточный диспетчер — об этом мы поговорим в разделе 14.7.

С помощью функции `launch` можно запустить новую, базовую корутину. Но хотя эта функция позволяет выполнять конкурентные вычисления, она не предоставляет возможности легко вернуть значение из кода внутри корутины. Эта функция больше подходит для задач в стиле «запустить и забыть», имеющих некоторые побочные эффекты (например, запись в файл или в базу данных), если не нужно фактически возвращать значение. Функция `launch` возвращает объект типа `Job`, и его можно рассматривать как обработчик для запущенной корутины. Объект `Job` применяется для управления выполнением корутины, например для инициирования отмены (подробнее об отмене поговорим в главе 15). Возврат вычисленного результата выполняется с помощью другой функции-конструктора. Ее название и концепция могут быть вам знакомы из других языков программирования — `async`.

14.6.3. Вычисления с ожиданием: конструктор `async`

Для выполнения асинхронных вычислений можно использовать функцию `async` конструктора. Как и в случае с `launch`, в нее передается блок кода, выполняемый как корутина. Однако возвращаемый тип функции `async` различается — она возвращает экземпляр типа `Deferred<T>`. Применяя тип `Deferred`, вы сможете ожидать результат, используя функцию приостановки `await`.

Рассмотрим листинг 14.7, код в котором предназначен для асинхронного вычисления двух чисел. В этом листинге имитация длительных вычислений осуществляется с помощью функции `delay`. Два вызова к функции `slowlyAddNumbers` завернуты в вызов функции `async` до вызова функции `await` для отложенных значений, возвращаемых функцией-конструктором `async`.

Листинг 14.7. Использование конструктора `async` для запуска новой корутины

```
suspend fun slowlyAddNumbers(a: Int, b: Int): Int {
    log("Waiting a bit before calculating $a + $b")
    delay(100.milliseconds * a)
    return a + b
}

fun main() = runBlocking {
    log("Starting the async computation")
    val myFirstDeferred = async { slowlyAddNumbers(2, 2) }
    val mySecondDeferred = async { slowlyAddNumbers(4, 4) }
    log("Waiting for the deferred value to be available")
    log("The first result: ${myFirstDeferred.await()}")
    log("The second result: ${mySecondDeferred.await()}")
}
```

Запуск новой корутины
для каждого вызова `async`

Ожидание доступности
результатов

В результате выполнения этого кода вы получите следующий результат:

```
0 [main @coroutine#1] Starting the async computation
4 [main @coroutine#1] Waiting for the deferred value to be available
8 [main @coroutine#2] Waiting a bit before calculating 2 + 2
9 [main @coroutine#3] Waiting a bit before calculating 4 + 4
213 [main @coroutine#1] The first result: 4
415 [main @coroutine#1] The second result: 8
```

Изучение временных меток показывает, что вычисление двух значений заняло в общей сложности около 400 миллисекунд. Это соответствует длительности самого длинного вычисления (одно занимает 200 миллисекунд, а другое — 400 миллисекунд). Используя функцию `async`, вы запускаете новую корутину для каждого вызова функции, позволяя обоим вычислениям происходить одновременно (рис. 14.9). Вызов функции `async`, как и в случае с `launch`, не приостанавливает выполнение корутины. При вызове метода `await` корневая корутина приостанавливается до тех пор, пока не будет получено значение.

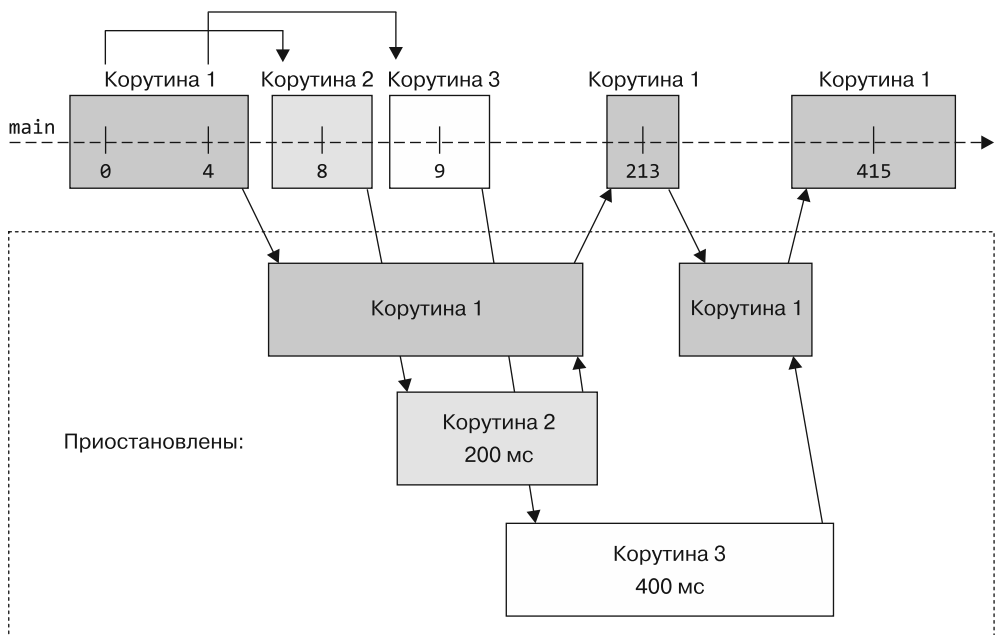


Рис. 14.9. Несмотря на то что все приложение работает в одном потоке, с помощью функции `async` можно одновременно вычислять несколько значений. Корутина 1 запускает два асинхронных вычисления и приостанавливает до тех пор, пока их значения не станут доступны. Каждая новая корутина (2 и 3) приостанавливается внутри своего тела на некоторое время, после чего возвращает полученное значение обратно в корутину 1

Возможно, тип `Deferred` вам знаком из других языков как `Promise`, который используется вместе с `Future`. Концепция во всех случаях одинакова: отложенный объект представляет значение, которое потенциально еще недоступно. Это значение должно быть вычислено или получено. Программа получает обещание, что оно будет известно когда-нибудь в будущем, — другими словами, его вычисление отложено.

Отличие Kotlin от других языков заключается в том, что не нужно использовать методы `async` и `await` в базовом коде, который последовательно вызывает функции приостановки, как это было в листинге 14.2. В Kotlin функция `async` требуется только для параллельного выполнения независимых задач и ожидания их результатов. Если нет необходимости запускать несколько задач одновременно и затем ждать результатов их выполнения, то не обязательно использовать `async` — достаточно простых вызовов функций приостановки.

ПРИМЕЧАНИЕ

Стоит отметить, что функция-конструктор `async{}` и функция приостановки `await` не относятся к ключевым словам Kotlin (как `async/await` во множестве других языков). Это просто функции библиотеки `kotlinx.coroutines`. Используя возможность `Go to Definition` в IDE, можно даже посмотреть, как реализованы все эти функции. Функции приостановки — один из немногих примитивов конкурентности, добавленных непосредственно в ядро языка Kotlin. Они оказались настолько мощными низкоуровневыми абстракциями, что функциональность наподобие `async/await` могла быть просто реализована в отдельной библиотеке. В главе 16 мы расскажем, что механизм приостановки достаточно универсален и его можно использовать еще и для реализации API в стиле реактивного потока.

В табл. 14.1 приведен обзор конструкторов корутин и их использования.

Таблица 14.1. В зависимости от случая использования можно выбрать один из доступных конструкторов корутин

Конструктор	Возвращаемое значение	Применение
<code>runBlocking</code>	Значение, вычисленное в лямбде	Соединяет блокирующий и неблокирующий код
<code>launch</code>	<code>Job</code> (подробнее рассматривается в главе 15)	Задачи по сценарию «запустить и забыть» (с побочными эффектами)
<code>async</code>	<code>Deferred<T></code>	Асинхронное вычисление значения (ожидаемое)

Корутины — абстракция поверх потоков. Но в каких потоках на самом деле выполняется этот код? Мы приводили один особый случай — `runBlocking`. С ним код просто выполняется в потоке, вызывающем функцию. Чтобы получить больше контроля над тем, в каких потоках выполнять код, вы можете использовать *диспетчеры* в корутинах Kotlin.

14.7. РЕШЕНИЕ О ТОЧКЕ ВЫПОЛНЕНИЯ КОДА: ДИСПЕТЧЕРЫ

Диспетчер для корутин определяет, какой поток использует корутина для своего выполнения. Выбрав диспетчер, можно ограничить выполнение корутин определенным потоком или отправить их в пул потоков. Это даст возможность решить, где должна выполняться корутина: в определенном потоке или в нескольких. По своей сути корутина не привязана к какому-либо конкретному потоку. Она может приостановить свое выполнение в одном потоке и возобновить его в другом, как диктует диспетчер.

ЧТО ТАКОЕ ПУЛ ПОТОКОВ

Пул потоков управляет набором потоков и позволяет выполнять в них задачи (или в нашем случае корутины). Он не выделяет новый поток для каждой задачи, поскольку это дорогостоящая операция. Вместо этого пул хранит некоторое количество выделенных потоков и распределяет поступающие задания на основе некоторой внутренней логики, определенной для конкретной реализации.

14.7.1. Выбор диспетчера

В разделе 14.8 говорится, что по умолчанию корутины наследуют диспетчер от своего родителя, поэтому нет нужды указывать его для каждой из них в явном виде. Тем не менее существует множество диспетчеров на выбор. С их помощью можно напрямую запускать корутины в контексте по умолчанию, при работе с UI и основным потоком и длительными задачами и API, которые могут заблокировать основной поток. Рассмотрим каждый из них более подробно.

Многопоточный диспетчер общего назначения — `Dispatchers.Default`

Наиболее универсальный диспетчер для операций общего назначения — `Dispatchers.Default`. Он опирается на пул потоков с количеством потоков, равным числу доступных ядер ЦП. Это означает, что когда вы планируете выполнение корутин в диспетчере по умолчанию, они распределяются для выполнения по нескольким потокам. И следовательно, могут работать параллельно на многоядерных машинах. Обычно при запуске большинства корутин следует использовать диспетчер по умолчанию. Если только вы не столкнулись со сценарием, в котором требуется привязка к конкретному потоку или пулу. Тем не менее следует помнить, что каждая корутина просто приостанавливает, а не блокирует используемый поток. И поэтому даже один поток может обрабатывать тысячи корутин.

Запуск в потоке UI — `Dispatchers.Main`

Фреймворки пользовательского интерфейса (UI), независимо от того, используете ли вы их на настольном компьютере (с JavaFX, AWT или Swing) или в Android, иногда требуют ограничить выполнение конкретных операций определенным

потоком. Этот поток называется *основным* или *потоком UI*. Одним из примеров такой операции может служить перерисовка элементов пользовательского интерфейса. Чтобы безопасно выполнять эти операции из корутины, можно использовать `Dispatchers.Main` при диспетчеризации. (Обратите внимание, что это не означает требования запускать всю корутину на основном диспетчере. О типичном паттерне, в котором используется `Dispatchers.Main`, мы расскажем в подразделе 14.7.3.)

Не существует универсального определения того, что такое UI или основной поток приложения, поэтому фактическое значение `Dispatchers.Main` зависит от используемого фреймворка. Существуют дополнительные артефакты, такие как `org.jetbrains.kotlinx:kotlinx-coroutines-swing` и `org.jetbrains.kotlinx:kotlinx-coroutines-javafx`. Они предоставляют соответствующие реализации для основного диспетчера каждого фреймворка.

В Android реализации диспетчера `Dispatchers.Main` предоставлены общим артефактом `org.jetbrains.kotlinx:kotlinx-coroutines-android`. Он используется для настройки корутин на Android.

Блокировка задач ввода-вывода — `Dispatchers.io`

При использовании сторонних библиотек не всегда есть возможность выбрать API, созданный с учетом корутин. И когда доступен только блокирующий API (например, для взаимодействия с системой баз данных), вы можете столкнуться с проблемами при вызове этой функциональности из диспетчера по умолчанию. Помните, что количество потоков в диспетчере по умолчанию равно количеству доступных ядер процессора. Это означает, что при вызове двух операций с блокировкой потоков на двухъядерной машине стандартный пул потоков будет исчерпан. И никакие другие корутины не смогут работать, пока ваши операции ожидают завершения. Диспетчер ввода-вывода предназначен для решения именно таких задач. Запущенные в этом диспетчере корутины будут выполняться в автоматически масштабируемом пуле потоков. Он выделяется именно для задач, решение которых не требует больших затрат ресурсов ЦП, когда просто ожидается возвращение блокирующего API.

СПЕЦИАЛИЗИРОВАННЫЕ И ИНДИВИДУАЛЬНЫЕ ДИСПЕТЧЕРЫ

Большую часть кода на основе корутин можно отправить в один из описанных выше диспетчеров. Но, конечно, могут возникнуть особые требования к производительности или поведению конкурентных систем на базе корутин Kotlin. Для таких случаев библиотека корутин предоставляет дополнительную функциональность. Например, диспетчер `Unconfined` позволяет выполнять корутины, никак не ограничивая потоки. Если требуется задать собственные ограничения на параллелизм для диспетчера, то можно использовать функцию `limitedParallelism`. Однако они предназначены только для особых случаев. Дополнительную информацию можно найти в документации Kotlin по корутинам и диспетчерам (<http://mng.bz/oeVZ>).

Сравнение доступных диспетчеров приведено в табл. 14.2 и на рис. 14.10.

Таблица 14.2. Встроенные диспетчеры, доступные в корутинах Kotlin

Диспетчер	Количество потоков	Область применения
<code>Dispatchers.Default</code>	Количество ядер ЦП	Операции общего назначения, операции с привязкой к ЦП
<code>Dispatchers.Main</code>	Один	Логика UI («поток UI»), только в контексте фреймворка UI
<code>Dispatchers.IO</code>	До 64 потоков (автомасштабирование) или равно количеству ядер ЦП (в зависимости от того, чего больше)	Разгрузка блокирующих задач ввода-вывода
<code>Dispatchers.Unconfined</code>	«Все, что доступно»	Особые случаи, когда требуется немедленное планирование (не общего назначения)
<code>limitedParallelism(n)</code>	Задается разработчиком (n)	Пользовательские сценарии

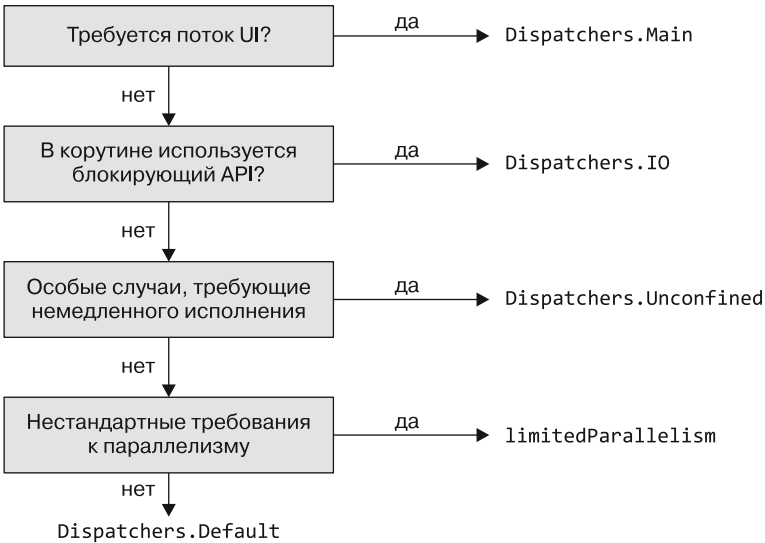


Рис. 14.10. Эта небольшая схема принятия решений поможет вам выбрать диспетчер. Если нет особых причин, например, для работы с потоками UI, блокирующими API, или в особых случаях, то всегда можно выбрать `Dispatchers.Default`

При запуске новой корутины *не требуется* указывать диспетчер. Вопрос в том, где будет выполняться ваш код. Ответ: в диспетчере родительской корутины. Подробнее о том, как наследуются диспетчеры и другие элементы из *контекста* корутин, мы поговорим в главе 15.

14.7.2. Передача диспетчера в конструктор корутин

Чтобы дать корутине указание работать с определенным диспетчером, можно передать диспетчер в качестве аргумента в функцию-конструктор корутины. Такие функции, как `runBlocking`, `launch` и `async`, дают возможность указать диспетчер в явном виде.

В листинге 14.8 приводится конфигурация функции `launch`, с помощью которой вы сможете запустить свою корутину с диспетчером по умолчанию, передавая его в качестве аргумента.

Листинг 14.8. Указание диспетчера в качестве параметра для конструктора корутины

```
fun main() {
    runBlocking {
        log("Doing some work")
        launch(Dispatchers.Default) { ← Установка диспетчера
            log("Doing some background work")    Dispatchers.Default
                                                для корутины
        }
    }
}
```

В результатах работы кода видно, что первый вызов функции `log` выполняется в основном потоке, а второй вызов из запущенной корутины 2 — в одном из потоков из пула потоков диспетчера по умолчанию:

```
26 [main @coroutine#1] Doing some work
33 [DefaultDispatcher-worker-1 @coroutine#2] Doing some background work
```

Вместо того чтобы переключать диспетчер для всей корутины, можно более точно определить, где должны выполняться определенные ее части. Для этого используется функция `withContext`, и ее мы рассмотрим следующей.

14.7.3. Использование функции `withContext` для переключения диспетчера внутри корутины

При работе с фреймворками пользовательского интерфейса может возникнуть острая потребность в том, чтобы код выполнялся в определенном базовом потоке. Классический паттерн при разработке приложений с UI — выполнение длительных вычислений в фоновом режиме. А затем, когда результат становится доступен, осуществляется переключение на поток UI и обновление пользовательского интерфейса.

Чтобы переключить диспетчер для уже существующей корутины, используется функция `withContext` и передается другой диспетчер (рис. 14.11). В этом фрагменте кода запускается новая корутина для выполнения некой фоновой операции, которая может приостановиться. Как только результат будет получен, корутина получает

указание переключиться на поток UI с помощью выражения `Dispatchers.Main` и выполнить (гипотетическую) операцию для UI:

```
launch(Dispatchers.Default) {
    val result = performBackgroundOperation()
    withContext(Dispatchers.Main) {
        updateUI(result)
    }
}
```

← Запуск корутины с диспетчером по умолчанию...

← ...и переключение на основной диспетчер для обновления UI

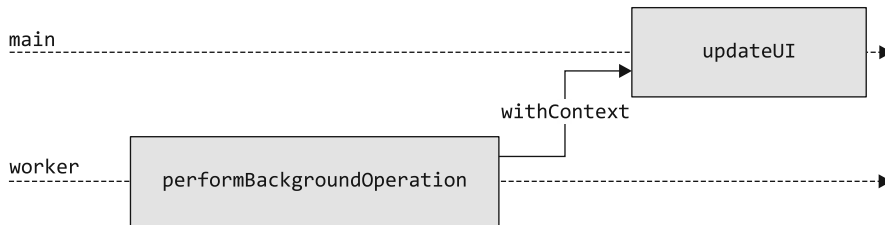


Рис. 14.11. Вызов функции `withContext` дает корутине указание выполняться на указанном диспетчере. А изначально она была запущена на диспетчере по умолчанию и выполнялась на одном из его рабочих потоков. В данном случае это основной поток для обновления пользовательского интерфейса

ПРИМЕЧАНИЕ

В этом примере используется диспетчер `Dispatchers.Main`. В подразделе 14.7.1 рассказывалось о том, что он предоставляется фреймворком пользовательского интерфейса, например JavaFX, Swing, AWT или фреймворком Android. Поэтому данный фрагмент будет работать, только если в проекте настроен один из этих фреймворков. Переключение на другой диспетчер (например, `withContext(Dispatchers.IO)`) работает во всех проектах.

14.7.4. Корутины и диспетчеры не станут волшебным средством для решения проблем безопасности потоков

При рассмотрении `Dispatchers.Default` и `Dispatchers.IO` вы изучили встроенные *многопоточные диспетчеры*. Такой диспетчер распределяет корутины по нескольким потокам. Если вы раньше занимались многопоточным программированием, это может привести вас на мысль, что у нас возникли типичные проблемы безопасности потоков. Это хорошее предчувствие, и оно позволит более детально обсудить семантику корутин.

Одиночная корутина всегда выполняется последовательно — части отдельной корутины не выполняются параллельно. Это означает еще и то, что данные, связанные с одной корутиной, не сталкиваются с типичными проблемами синхронизации. Чтение и изменение данных из нескольких (параллельных) корутин не такая простая задача. Сравним два фрагмента кода, чтобы вы могли увидеть это небольшое,

но важное различие. В листинге 14.9 запускается одна корутина для увеличения счетчика x 10 000 раз.

Листинг 14.9. Запуск одной корутины для увеличения переменной

```
fun main() {
    runBlocking {
        launch(Dispatchers.Default) {
            var x = 0
            repeat(10_000) {
                x++
            }
            println(x)
        }
    }
}

// 10000
```

← Запускает одну корутину на многопоточном диспетчере по умолчанию

Значение x после завершения работы корутины точно соответствует ожидаемому. Это происходит потому, что, хотя запущенная вами единственная корутина работает в произвольном потоке, ее логика выполняется строго последовательно. Проведем сравнение со вторым фрагментом кода (листинг 14.10). В нем запускается 10 000 корутин, и каждая увеличивает счетчик по одному разу.

Листинг 14.10. Запуск нескольких корутин для увеличения переменной

```
fun main() {
    runBlocking {
        var x = 0
        repeat(10_000) {
            launch(Dispatchers.Default) {
                x++
            }
        }
        delay(1.seconds)
        println(x)
    }
}

// 9916
```

← Запуск корутин для многопоточного диспетчера по умолчанию

В данном случае значение счетчика меньше, чем ожидалось, поскольку несколько корутин изменяют одни и те же данные (инкрементируют счетчик). Если некоторые из этих операций выполняются параллельно, из-за того что работа осуществляется с помощью многопоточного диспетчера, то некоторые из операций инкремента могут перезаписывать друг друга.

Как и в любой другой конкурентной системе, где можно обрабатывать данные параллельно, есть несколько подходов для исправления этой ситуации. Как показано в листинге 14.11, корутины предоставляют блокировку `Mutex`. Она позволяет сделать так, что критический участок кода будет выполняться только одной корутиной за один раз.

Листинг 14.11. Использование блокировки Mutex для критического участка

```

fun main() = runBlocking {
    val mutex = Mutex()
    var x = 0
    repeat(10_000) {
        launch(Dispatchers.Default) {
            mutex.withLock {
                x++
            }
        }
    }
    delay(1.seconds)
    println(x)
}

// 10000

```

Вдобавок можно использовать атомарные и безопасные для потоков структуры данных, созданные для конкурентных модификаций, например `AtomicInteger` или `ConcurrentHashMap`. Кроме того, ограничение корутин (или при использовании функции `withContext` их критического участка) однопоточным диспетчером тоже может решить данную проблему. Но в этом случае следует учитывать определенные особенности производительности. Более подробная информация доступна в разделе `Mutable Shared State and Concurrency` (<http://mng.bz/ngx5>) документации Kotlin.

Подводя итог, можно сказать, что при работе с корутинами возникают те же проблемы конкурентности, что и при работе с потоками: пока данные связаны с одной корутиной, код будет вести себя ожидаемо. Когда несколько параллельно работающих корутин изменяют одни и те же данные, необходимо выполнить синхронизацию или блокировку, как в случае с потоками.

14.8. КОРУТИНЫ СОДЕРЖАТ ДОПОЛНИТЕЛЬНУЮ ИНФОРМАЦИЮ В СВОЕМ КОНТЕКСТЕ

В предыдущих разделах различные диспетчеры применялись в качестве аргументов для функций-конструкторов и для функции `withContext`. Но если обратить внимание на имя параметра (и тип) этих функций, то станет понятно, что параметр не представляет собой `CoroutineDispatcher`, а на самом деле является `CoroutineContext`. Разберемся, что это такое.

Каждая корутина содержит дополнительную информацию о контексте в виде `CoroutineContext`. Эту сущность можно представить как набор различных элементов. Одним из них будет диспетчер для определения потока или потоков выполнения данной корутины. Еще `CoroutineContext` обычно содержит связанный с корутиной объект `Job`, отвечающий за ее жизненный цикл и (потенциально с исключением) отмену. Контекст корутины может содержать и дополнительные метаданные, например `CoroutineName` или `CoroutineExceptionHandler`.

Текущий контекст корутины можно проверить. Для этого необходимо обратиться к специальному свойству `coroutineContext` внутри любой функции приостановки. На самом деле это свойство не определено в коде Kotlin — это *intrinsic-функция компилятора*. То есть его фактическая реализация обрабатывается компилятором Kotlin как особый случай:

```
import kotlin.coroutines.coroutineContext

suspend fun introspect() {
    log(coroutineContext)
}

fun main() {
    runBlocking {
        introspect()
    }
}

// 25 [main @coroutine#1] [CoroutineId(1),
//    "coroutine#1":BlockingCoroutine{Active}@610694f1,
//    BlockingEventLoop@43814d18]
```

← Intrinsic-функция `coroutineContext` содержит контекстную информацию о корутине

При передаче параметра в конструктор корутины или в функцию `withContext` этот конкретный элемент переопределяется в контексте дочерней корутины. Чтобы переопределить сразу несколько параметров, можно объединить их с помощью оператора `+`, перегруженного для объектов `CoroutineContext`. Например, можно запустить свою корутину `runBlocking` с диспетчером ввода-вывода и дать ей имя `Coolroutine`:

```
fun main() {
    runBlocking(Dispatchers.IO + CoroutineName("Coolroutine")) {
        introspect()
    }
}

// 27 [DefaultDispatcher-worker-1 @Coolroutine#1]
//    [CoroutineName(Coolroutine), CoroutineId(1),
//    "Coolroutine#1":BlockingCoroutine{Active}@d115c9f, Dispatchers.IO]
```

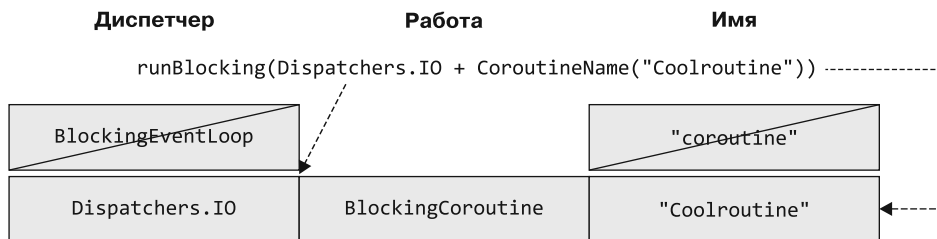


Рис. 14.12. Передаваемые в `runBlocking` параметры переопределяют элементы контекста дочерней корутины: `Dispatchers.IO` заменяет особый диспетчер `BlockingEventLoop` из корутины `runBlocking`, а `Coolroutine` становится именем корутины

Подробнее о значении контекстов корутин поговорим в подразделе 15.1.4. Но сначала необходимо заложить основу, рассказав об одной из самых мощных возможностей корутин Kotlin — структурированной конкурентности. Этой теме посвящена следующая глава.

РЕЗЮМЕ

- Под конкурентностью понимается одновременная работа с несколькими задачами. Она проявляется во взаимосвязанном выполнении. Параллелизм подразумевает одновременное физическое выполнение задач, что позволяет эффективно использовать современные многоядерные системы.
- Корутины (корутины) — легкая абстракция, которая работает поверх потоков и позволяет выполнять задачи конкурентно.
- Основной примитив конкурентности в Kotlin представлен функцией приостановки — ее выполнение может быть остановлено на некоторое время. Функцию приостановки можно вызвать из другой функции приостановки или из корутины.
- В отличие от других подходов, таких как реактивные потоки, обратные вызовы и фьючерсы, приостановка функций не меняет форму кода — он по-прежнему выглядит последовательным.
- Корутина — это экземпляр приостанавливаемого вычисления.
- Блокирующие потоки представляют собой дорогостоящие, ограниченные ресурсы. А корутины позволяют избежать проблем, вызванных их применением.
- Такие конструкторы, как `runBlocking`, `launch` и `async`, позволяют создавать новые корутины.
- Диспетчеры решают, в каком потоке или пуле потоков запускать ваши корутины.
- Различные встроенные диспетчеры служат для разных целей: `Dispatchers.Default` — диспетчер общего назначения, `Dispatchers.Main` помогает запускать операции в потоке UI, а `Dispatchers.IO` применяется для вызова блокирующих задач ввода-вывода.
- Большинство диспетчеров, например `Dispatchers.Default` и `Dispatchers.IO`, многопоточные. Это значит, что вы должны быть особенно внимательными, когда несколько корутин параллельно изменяют одни и те же данные.
- Можно указать диспетчер при создании корутины или переключаться между диспетчерами с помощью функции `withContext`.
- Контекст корутины содержит дополнительную информацию о корутине. Диспетчер корутины представляет собой часть контекста корутины.

15

Структурированная конкурентность

В этой главе

- ✓ Установление иерархии между корутинами с помощью концепции структурированной конкурентности.
- ✓ Как структурированная конкурентность позволяет осуществлять точный контроль над выполнением и отменой кода, автоматически распространяя отмену по иерархии корутин.
- ✓ Связь между контекстом корутин и структурированной конкурентностью.
- ✓ Написание кода с правильным поведением при отмене.

При использовании корутин Kotlin в контексте реальных приложений есть вероятность, что потребуется управлять множеством корутин. При работе с большим количеством конкурентных операций возникает ряд основных проблем: необходимо отслеживать отдельные запущенные задачи, отменять их, когда они перестают быть нужными, и обеспечивать правильную обработку ошибок.

Если за корутинами не следить, то высок риск получить утечку ресурсов и выполнить ненужную работу. Рассмотрим следующий пример: пользователь запрашивает сетевой ресурс и тут же переходит на другой экран. Если нет возможности отслеживать (потенциально десятки) корутин, отвечающих за сетевой запрос и постобработку полученной информации, то остается позволить им работать до завершения, даже если их результат в итоге будет просто отброшен.

К счастью, прямо в ядро Kotlin встроена *структурированная конкурентность*. Это возможность управлять иерархиями корутин и отслеживать их, а также

контролировать время их существования в вашем приложении. Эта функциональная возможность доступна сразу, и отслеживать вручную каждую запущенную корутину не нужно. Если вы будете использовать структурированную конкурентность, то во всем приложении не будет «бесконтрольных» корутин — обычно они выполняются дольше запланированного.

В этой главе мы подробно рассмотрим механизм структурированной конкурентности Kotlin и расскажем, как он позволяет управлять даже большим количеством корутин.

15.1. ОБЛАСТИ ВИДИМОСТИ КОРУТИН УСТАНОВЛИВАЮТ СТРУКТУРУ МЕЖДУ НИМИ

При структурированной конкурентности каждая корутина принадлежит определенной *области видимости*. Они помогают установить отношения между корутинами-родителями и потомками. Функции-конструкторы `launch` и `async` на самом деле представляют собой функции-расширения интерфейса `CoroutineScope`. Это означает, что при создании новой корутины с помощью функции `launch` или `async` в теле другого конструктора новая корутина автоматически становится дочерней по отношению к этой корутине. В следующем фрагменте кода запускается несколько корутин с различным временем выполнения (листинг 15.1).

Листинг 15.1. Запуск нескольких различных корутин

```
fun main() {
    runBlocking { // this: CoroutineScope ← Скрытый приемник
        launch { // this: CoroutineScope ← Корутина, запущенная функцией launch,
            delay(1.seconds)                становится дочерней по отношению
            launch {                        к родительской корутине runBlocking
                delay(250.milliseconds)
                log("Grandchild done")
            }
            log("Child 1 done!")
        }
        launch {
            delay(500.milliseconds)
            log("Child 2 done!")
        }
        log("Parent done!")
    }
}
```

По результатам выполнения кода видно, что, хотя тело функции `runBlocking` завершает свое выполнение почти мгновенно — об этом свидетельствует надпись `Parent done!` — программа на самом деле не завершается, пока не закончатся все дочерние корутины:

```
29 [main @coroutine#1] Parent done!
539 [main @coroutine#3] Child 2 done!
1039 [main @coroutine#2] Child 1 done!
1293 [main @coroutine#4] Grandchild done
```

Это возможно благодаря структурированному параллелизму — между корутинами (строго говоря, объектами `Job`, связанными с вашими корутинами, об этом поговорим в подразделе 15.1.4) существуют отношения «родитель — потомок». Таким образом вызову функции `runBlocking` известно, сколько его дочерних объектов еще работают, и код продолжает ждать, пока они все не завершатся. Эта иерархия представлена на рис. 15.1. Вам не придется вручную отслеживать запущенные корутины и их потомков. Кроме того, не требуется вручную выставить их ожидание — структурированная конкурентность может отследить это автоматически.

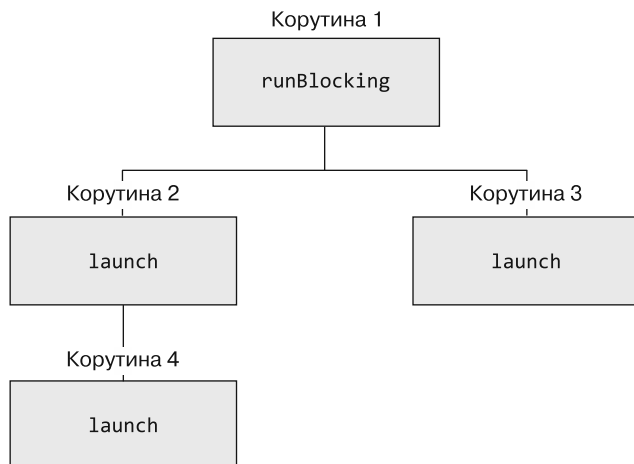


Рис. 15.1. Благодаря структурированной конкурентности корутины имеют иерархию. Каждой корутине известны ее потомки и родители. Это позволяет, например, функции `runBlocking` дожидаться завершения всех своих дочерних функций, прежде чем вернуть результат. Данный подход работает, даже если вы не указали иерархию в явном виде

Корутины хранят информацию о своих потомках и способны отслеживать их поведение. В разделе 15.2 мы рассмотрим, как эта особенность позволяет реализовать такую функциональность, как автоматическая отмена дочерних корутин при отмене родительской. Вдобавок это поможет вам выразить желаемое поведение при обработке исключений — мы обсудим эту тему в главе 18. (Более подробную информацию о том, как устанавливаются отношения между родителями и потомками на техническом уровне, вы получите в подразделе 15.1.4.)

15.1.1. Создание области видимости корутины — функция `coroutineScope`

Когда вы создаете новую корутину с помощью конструкторов, они создают собственные интерфейсы `CoroutineScope`. Но корутины можно группировать на основе собственных областей видимости, не создавая совершенно новую корутину. Для этого можно использовать функцию `coroutineScope`. Это функция приостановки, и она создает новую область видимости корутины и ожидает завершения всех своих дочерних корутин, прежде чем завершится сама.

Типичным случаем использования функции `coroutineScope` будет *конкурентная декомпозиция работы* — совместное использование нескольких корутин для выполнения вычислений. В данном примере этот механизм применяется для вычисления суммы нескольких конкурентно сгенерированных чисел.

Здесь используется возможность функции `coroutineScope` возвращать значение для получения суммы двух значений из блока перед его фиксацией (рис. 15.2). Функция `coroutineScope` приостанавливается, поэтому `computeSum` тоже помечается как приостановленная:

```
import kotlinx.coroutines.*
import kotlin.random.Random
import kotlin.time.Duration.Companion.milliseconds

suspend fun generateValue(): Int {
    delay(500.milliseconds)
    return Random.nextInt(0, 10)
}

suspend fun computeSum() { ← Функция computeSum приостановлена
    log("Computing a sum...")
    val sum = coroutineScope { ← Функция coroutineScope предоставляет
        val a = async { generateValue() } область видимости
        val b = async { generateValue() }
        a.await() + b.await() ← Перед возвратом значения функция
    }                               coroutineScope ожидает завершения
    log("Sum is $sum")           работы дочерней корутины
}

fun main() = runBlocking {
    computeSum()
}

// 0 [main @coroutine#1] Computing a sum...
// 532 [main @coroutine#1] Sum is 10
```

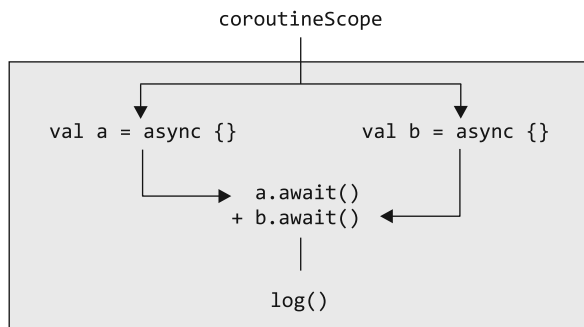


Рис. 15.2. Функция `coroutineScope` используется для конкурентной декомпозиции работы — совместного использования нескольких корутин в целях выполнения вычислений. Две корутины, `a` и `b`, выполняются конкурентно. Их результаты используются для вычисления значения, а после оно фиксируется

15.1.2. Ассоциирование диапазонов корутин с компонентами: CoroutineScope

Функция `coroutineScope` применяется для декомпозиции работы. Но вам также может потребоваться создать класс, который определяет собственный жизненный цикл, управляя запуском и остановкой конкурентных процессов и корутин. Для таких сценариев используется конструктор `CoroutineScope`. Он создает новую автономную область видимости корутины. В отличие от `coroutineScope` эта функция не приостанавливает выполнение — она просто предоставляет новую область видимости корутины, и вы можете использовать ее для запуска новых корутин.

Функция `CoroutineScope` принимает один параметр — контекст, связанный с областью видимости корутины (контексты корутин будут описаны в подразделе 15.1.4). В нем можно указать, например, диспетчер для корутин в этой области видимости.

По умолчанию вызов функции `CoroutineScope` только с диспетчером автоматически создает новый объект `Job`. Однако в большинстве случаев на практике вместо этого лучше использовать тип `SupervisorJob` с функцией `CoroutineScope`. Тип `SupervisorJob` — особый тип работы (`Job`). Он запрещает перехваченным исключениям отменять другие корутины в той же области видимости и распространять исключение дальше (о супервизорах мы подробно поговорим в главе 18, когда будем обсуждать обработку ошибок).

В листинге 15.2 создается класс, который может запускать и управлять корутинами наряду с собственным жизненным циклом. Он принимает диспетчер корутин в качестве аргумента конструктора и использует функцию `CoroutineScope` для создания новой области видимости корутин на основе класса. Функция `start` запускает постоянно выполняющуюся корутину, а также другую, предназначенную для выполнения одноразовой задачи. Функция `stop` отменяет область видимости в классе, а вместе с ней и запущенные ранее корутины (более подробно об отмене мы поговорим в разделе 15.2).

Листинг 15.2. Компонент со связанной областью действия корутины

```
class ComponentWithScope(dispatcher: CoroutineDispatcher = Dispatchers.Default) {

    private val scope = CoroutineScope(dispatcher + SupervisorJob())

    fun start() {
        log("Starting!")
        scope.launch {
            while(true) {
                delay(500.milliseconds)
                log("Component working!")
            }
        }
        scope.launch {
            log("Doing a one-off task...")
            delay(500.milliseconds)
            log("Task done!")
        }
    }
}
```

```

    fun stop() {
        log("Stopping!")
        scope.cancel()
    }
}

```

Можно создать новый экземпляр этого класса `Component` и вызвать функцию `start`, чтобы компонент внутренне запустил свои корутины. Затем для завершения жизненного цикла компонента вызывается функция `stop`:

```

fun main() {
    val c = ComponentWithScope()
    c.start()
    Thread.sleep(2000)
    c.stop()
}

// 22 [main] Starting!
// 37 [DefaultDispatcher-worker-2 @coroutine#2] Doing a one-off task...
// 544 [DefaultDispatcher-worker-1 @coroutine#2] Task done!
// 544 [DefaultDispatcher-worker-2 @coroutine#1] Component working!
// 1050 [DefaultDispatcher-worker-1 @coroutine#1] Component working!
// 1555 [DefaultDispatcher-worker-1 @coroutine#1] Component working!
// 2039 [main] Stopping!

```

Функция `CoroutineScope` часто используется во фреймворках, которым приходится управлять компонентами с жизненным циклом. Один из таких примеров — класс `ViewModel` в Android (будет показан в подразделе 15.2.9).

ФУНКЦИИ `coroutineScope` И `CoroutineScope`

Функции `coroutineScope` и `CoroutineScope` служат разным целям, хотя их названия и очень похожи.

- Функция `coroutineScope` используется для конкурентной декомпозиции работ. Вы запускаете несколько корутин и ожидаете их завершения, потенциально вычисляя какой-то результат. Поскольку `coroutineScope` ожидает завершения работы своих потомков, то представляет собой функцию приостановки.
- Функция `CoroutineScope` используется для создания области видимости, связывающей корутину с жизненным циклом класса. Она создает область видимости, но не ожидает никаких дальнейших операций, поэтому возврат выполняется быстро. Функция возвращает ссылку на область видимости корутины, чтобы ее можно было впоследствии отменить. (Об отмене мы поговорим в разделе 15.2.)

На практике можно встретить гораздо больше случаев использования функции приостановки `coroutineScope`, чем функции-конструктора `CoroutineScope`. Обычно вызовы `coroutineScope` встречаются в теле функции приостановки, а конструктор `CoroutineScope` часто применяется для хранения области видимости корутины в качестве свойства класса.

15.1.3. Опасность GlobalScope

В некоторых примерах или фрагментах кода встречается особый случай области видимости корутины — `GlobalScope`. Как следует из названия, это область видимости корутины на глобальном уровне. Для новичков в корутинах Kotlin она может стать заманчивым вариантом при выборе области видимости для новой корутины, ведь она доступна глобально.

Однако использование компонента `GlobalScope` имеет ряд недостатков. Вкратце можно сказать, что его применение означает отказ от всех преимуществ структурированной конкурентности. Корутины в глобальной области видимости нельзя автоматически отменить, и у них нет жизненного цикла. Это означает, что при использовании глобальной области видимости очень легко создать утечку ресурсов в приложении или просто продолжать выполнять ненужную работу, расходуя вычислительные ресурсы.

Одну из проблем, возникающих при использовании компонента `GlobalScope`, можно наблюдать, если слегка подкорректировать код из листинга 15.1. Код, приведенный в листинге 15.3, завершится немедленно, не дожидаясь ни одной из запущенных корутин.

Листинг 15.3. Компонент `GlobalScope` нарушает структурированную иерархию конкурентности

```
fun main() {
    runBlocking {
        GlobalScope.launch {
            delay(1000.milliseconds)
            launch {
                delay(250.milliseconds)
                log("Grandchild done")
            }
            log("Child 1 done!")
        }
        GlobalScope.launch {
            delay(500.milliseconds)
            log("Child 2 done!")
        }
        log("Parent done!")
    }
}
```

Использование компонента `GlobalScope` в обычном коде приложения не лучшая идея

```
// 28 [main @coroutine#1] Parent done!
```

Этот код немедленно завершается, поскольку использование `GlobalScope` нарушает иерархию, обычно устанавливаемую автоматически при использовании структурированной конкурентности. Корутины со 2-й по 4-ю были запущены таким образом, что связанная с `runBlocking` корутина 1 перестает быть родительской (рис. 15.3). Поэтому у нее нет требующих ожидания потомков, и программа завершается преждевременно.

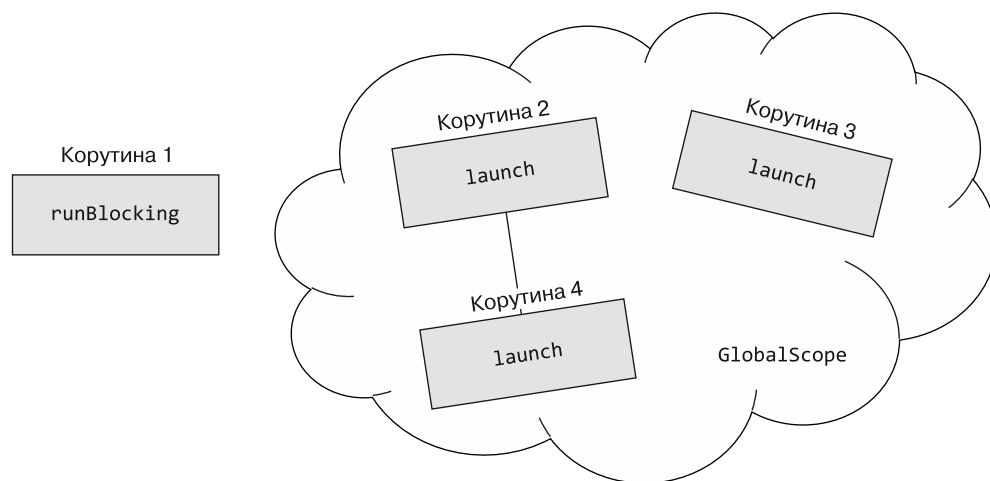


Рис. 15.3. Использование компонента `GlobalScope` нарушает иерархию между корутинами. `runBlocking` перестает быть родителем запущенных корутин, а значит, у нее нет способа автоматически дождаться их завершения

По этим причинам `GlobalScope` объявляется со специальной аннотацией — `@DelicateCoroutinesApi`. Система выдаст предупреждение, призывающее вас к осторожности: *This is a delicate API and its use requires care. Make sure you fully read and understand the documentation of the declaration that is marked as a delicate API* (Это API с пометкой `delicate`, и его использование требует осторожности. Убедитесь, что вы полностью прочитали и поняли документацию к объявлению, помеченному как `delicate API`). В общем коде приложений выбирать компонент `GlobalScope` следует крайне редко (один из таких примеров — фоновые процессы верхнего уровня, которые должны оставаться активными на протяжении *всего* времени существования приложения). Обычно лучше подобрать более подходящую область видимости для запуска корутин — либо с помощью конструкторов, либо с помощью функции `coroutineScope`.

15.1.4. Контексты корутин и структурированная конкурентность

Вы получили общее представление о структурированной конкурентности, и теперь мы можем вернуться к обсуждению контекстов корутин. Они тесно связаны с концепцией структурированной конкурентности и наследуются по иерархии «родитель — потомок», установленной между корутинами.

Что же происходит с контекстом корутины при запуске новой? Сначала дочерняя корутина наследует родительский контекст. Затем создает новый объект `Job` (как в главе 14). Он отвечает за установление отношений «родитель — потомок»: `Job` становится дочерним объектом `Job` в родительской корутине. Наконец, применяются все аргументы контекста корутины. Они могут переопределять все, что было унаследовано ранее.

Во фрагменте кода ниже и на рис. 15.4 это показано наглядно:

```
fun main() {
    runBlocking(Dispatchers.Default) {
        log(coroutineContext)
        launch {
            log(coroutineContext)
            launch(Dispatchers.IO + CoroutineName("mine")) {
                log(coroutineContext)
            }
        }
    }
}

// 0 [DefaultDispatcher-worker-1 @coroutine#1] [CoroutineId(1),
//    "coroutine#1":BlockingCoroutine{Active}@68308697, Dispatchers.Default]
// 1 [DefaultDispatcher-worker-2 @coroutine#2] [CoroutineId(2),
//    "coroutine#2":StandaloneCoroutine{Active}@2b3ce773, Dispatchers.Default]
// 2 [DefaultDispatcher-worker-3 @mine#3] [CoroutineName(mine),
//    CoroutineId(3), "mine#3":StandaloneCoroutine{Active}@7c42841a, Dispatchers.IO]
```

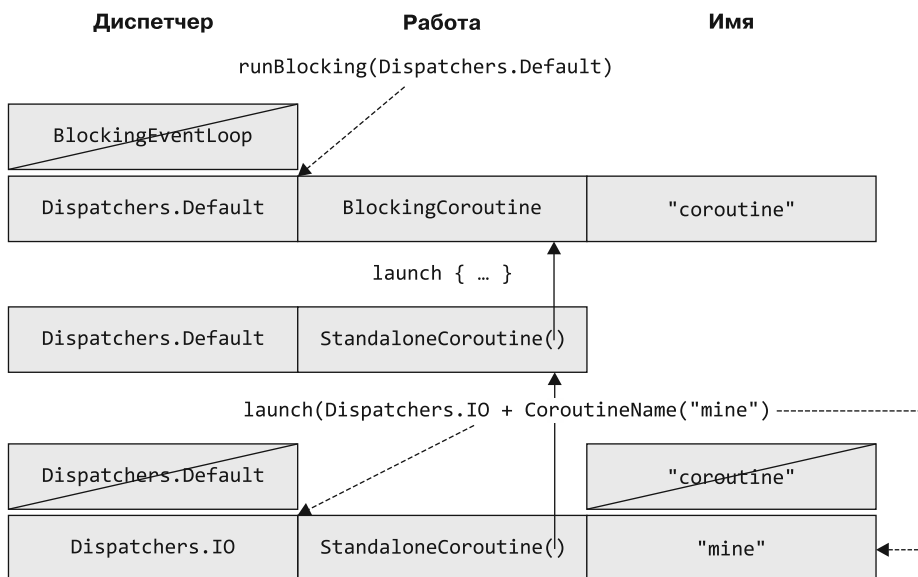


Рис. 15.4. Корутина `runBlocking` изначально содержит специальный диспетчер `BlockingEventLoop`. Он переопределяется заданным параметром и меняется на `Dispatchers.Default`. Он создает объект `Job` для корутины под названием `BlockingCoroutine` и инициализирует имя корутины значением по умолчанию `coroutine`. Функция `launch` наследует диспетчер по умолчанию. Она создает собственный объект `Job` с именем `StandaloneCoroutine` и устанавливает связь с родительским объектом работы — имя корутины остается неизменным. Второй вызов функции `launch` наследует диспетчер, а затем он создает нового потомка объекта `Job`, а также имя корутины. Параметры функции `launch` переопределяют диспетчер на `Dispatchers.IO`, а корутине присваивают имя `mine`.

Теперь вы знаете, как контексты корутин передаются по иерархии, и вам будет проще найти ответ на вопрос «Если запустить новую корутину с помощью функции `launch`, не указав диспетчер, то с каким диспетчером она будет выполняться?». Ответ — *не* `Dispatchers.Default`! Скорее всего, она будет выполняться с родительским диспетчером.

Можно отследить фактические отношения «родитель — потомок» между корутинами в вашем коде — или, более точно, отношения между *работами, связанными с вашими корутинами*. Это можно сделать, проверив свойства `job`, `job.parent` и `job.children` в контексте каждой корутины:

```
import kotlinx.coroutines.job

fun main() = runBlocking(CoroutineName("A")) {
    log("A's job: ${coroutineContext.job}")
    launch(CoroutineName("B")) {
        log("B's job: ${coroutineContext.job}")
        log("B's parent: ${coroutineContext.job.parent}")
    }
    log("A's children: ${coroutineContext.job.children.toList()}")
}

// 0 [main @A#1] A's job: "A#1":BlockingCoroutine{Active}@41
// 10 [main @A#1] A's children: ["B#2":StandaloneCoroutine{Active}@24
// 11 [main @B#2] B's job: "B#2":StandaloneCoroutine{Active}@24
// 11 [main @B#2] B's parent: "A#1":BlockingCoroutine{Completing}@41
```

Подобно процессу запуска с помощью функций-конструкторов `launch` и `async`, функция `coroutineScope` тоже создает свой объект `Job`, участвующий в иерархии «родитель — потомок». В этом можно убедиться, проверив свойство `coroutineContext.job`:

```
fun main() = runBlocking<Unit> { // coroutine#1
    log("A's job: ${coroutineContext.job}")
    coroutineScope {
        log("B's parent: ${coroutineContext.job.parent}") // A
        log("B's job: ${coroutineContext.job}") // C
        launch { //coroutine#2
            log("C's parent: ${coroutineContext.job.parent}") // B
        }
    }
}

// 0 [main @coroutine#1] A's job: "coroutine#1":BlockingCoroutine{Active}@41
// 2 [main @coroutine#1] B's parent:
//    "coroutine#1":BlockingCoroutine{Active}@41
// 2 [main @coroutine#1] B's job: "coroutine#1":ScopeCoroutine{Active}@56
// 4 [main @coroutine#2] C's parent:
//    "coroutine#1":ScopeCoroutine{Completing}@56
```

Кроме того, эти отношения корутин помогают и при отмене, о чем мы поговорим в следующем разделе.

15.2. ОТМЕНА

Под *отменой* понимается остановка выполнения кода до наступления момента его логического завершения. На первый взгляд отмена вычислений может показаться не слишком распространенным явлением. Однако на самом деле почти все современные приложения должны обрабатывать отмену вычислений, так как это позволяет обеспечить надежность и эффективность. Тому есть несколько причин.

Отмена предотвращает выполнение бесполезной работы. В приложениях с пользовательским интерфейсом могут быть запущены вычисления или сетевой запрос, а пользователь просто закроет окно или перейдет в другой раздел. Если бы у вас не было возможности отменить запущенную работу, то вам пришлось бы выполнить вычисления до конца или загрузить весь ответ по сети, а затем отбросить результат. Это может негативно сказаться на пропускной способности серверных приложений, что особенно опасно для мобильных устройств, вычислительные ресурсы и время автономной работы которых ограничены.

Вдобавок отмена помогает избежать утечек памяти или ресурсов. При отсутствии способа отменить неактуальную работу бесконтрольным корутинам будет легко удерживать ресурсы или сохранять ссылки на структуры данных в памяти, не позволяя сборщику мусора освободить место.

Кроме того, отмена играет важную роль при обработке ошибок, о чем мы подробнее поговорим в главе 18. Если коротко, то часто используются несколько совместно работающих корутин для вычисления результата — например, можно запустить несколько асинхронных сетевых запросов, а затем ожидать всех их результатов. Если один из сетевых запросов не сработает, то вычислить значимый результат иногда не получается, поэтому нет смысла ждать завершения оставшихся (или запуска дополнительных) сетевых запросов. Это особый случай исключения ненужной работы.

Отмена — ключевая часть создания конкурентных приложений с хорошим поведением, поэтому корутины Kotlin поставляются со встроенными механизмами для выполнения и обработки отмены. Рассмотрим эту тему подробнее.

15.2.1. Инициирование отмены

Возвращаемое значение различных конструкторов может быть использовано в качестве обработчика для инициирования отмены. Так, конструктор корутин `launch` возвращает объект `Job`, а конструктор `async` — тип `Deferred`. Оба они позволяют вызвать метод `cancel` для отмены соответствующей корутины:

```
fun main() {
    runBlocking {
        val launchedJob = launch { ← Функция launch возвращает объект Job...
            log("I'm launched!")
            delay(1000.milliseconds)
            log("I'm done!")
        }
    }
}
```

```

val asyncDeferred = async { ← ...функция async возвращает тип Deferred...
    log("I'm async")
    delay(1000.milliseconds)
    log("I'm done!")
}
delay(200.milliseconds)
launchedJob.cancel()
asyncDeferred.cancel() | ...и обе можно
                       | отменить
}

// 0 [main @coroutine#2] I'm launched!
// 7 [main @coroutine#3] I'm async

```

В подразделе 15.1.4 мы обсуждали, что контекст корутин для каждой области видимости содержит еще и объект `Job` и его можно использовать для отмены области действия точно таким же образом. Инициирование отмены для корутины выполняется вручную, но можно позволить библиотеке автоматически отменять корутины при определенных условиях.

15.2.2. Автоматическая отмена после превышения лимита времени

В библиотеку корутин Kotlin входят и некоторые удобные функции для автоматической инициации отмены корутин. Функции `withTimeout` и `withTimeoutOrNull` позволяют вычислить значение, ограничивая при этом максимальное количество времени, необходимого для выполнения вычислений.

Как и другие описанные ранее функции стандартной библиотеки, `withTimeout` выбрасывает исключение (точнее, `TimeoutCancellationException`), если тайм-аут был превышен. Для обработки тайм-аута вызов функции `withTimeout` оборачивают в блоки `try` и `catch` выброшенного исключения `TimeoutCancellationException`. Аналогом служит функция `withTimeoutOrNull`. Она возвращает значение `null` при превышении тайм-аута.

ПРИМЕЧАНИЕ

Необходимо улавливать исключение `TimeoutCancellationException`, выброшенное функцией `withTimeout`! В подразделе 15.2.4 вы увидите, что его супертип `CancellationException` используется в качестве специального индикатора для отмены корутин. А значит, что пропуск `TimeoutCancellationException` может вызвать (нежелательную) отмену вызванной корутины (подробнее на странице <https://github.com/Kotlin/kotlinx.coroutines/issues/1374>). Чтобы избежать этой дилеммы, используйте функцию `withTimeoutOrNull`.

В следующем примере кода вызывается функция `calculateSomething`, и на ее выполнение уходит около 3 секунд. Сначала мы вызовем ее с коротким тайм-аутом 500 миллисекунд, что приведет к его истечению. Функция `calculateSomething` будет отменена, а в качестве возврата будет получено значение `null`. А во втором вызове

функции будет предоставлено достаточно времени для завершения, и результатом станет фактическое вычисленное значение (рис. 15.5):

```
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.seconds
import kotlin.time.Duration.Companion.milliseconds

suspend fun calculateSomething(): Int {
    delay(3.seconds)
    return 2 + 2
}

fun main() = runBlocking {
    val quickResult = withTimeoutOrNull(500.milliseconds) {
        calculateSomething()
    }
    println(quickResult)
    // null
    val slowResult = withTimeoutOrNull(5.seconds) {
        calculateSomething()
    }
    println(slowResult)
    // 4
}
```

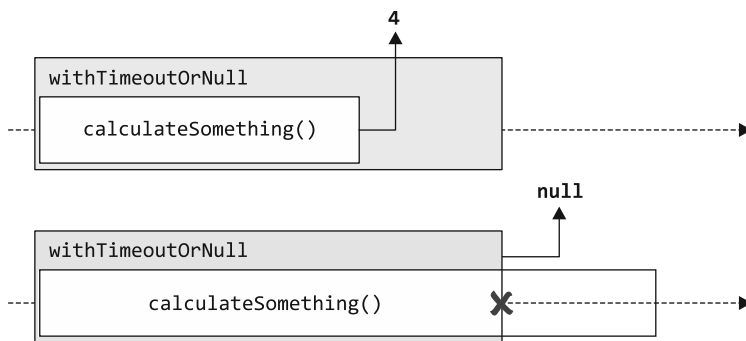


Рис. 15.5. Использование функции `withTimeoutOrNull` ограничивает время выполнения функции приостановки. Если функция возвращает значение в течение заданного тайм-аута, то оно возвращается немедленно. По истечении тайм-аута функция отменяется и возвращается значение `null`

15.2.3. Каскад отмены распространяется на все дочерние объекты

При отмене корутины отменяются и все ее дочерние корутины. Это очень значимая особенность структурированной конкурентности. Каждая корутина знает о запущенных ею корутинах, поэтому всегда может навести порядок после себя. Когда

она завершится, не останется активных фрагментов кода, выполняющих ненужную работу или держащих в памяти лишние данные дольше, чем требуется.

В листинге 15.4 несколько корутин запускают корутины внутри себя. Но даже при таком большом количестве уровней вложенности отмена на самом верхнем уровне корректным образом отменяет даже правнучатую корутину.

Листинг 15.4. Автоматическая отмена всех дочерних корутин

```
fun main() = runBlocking {
    val job = launch {
        launch {
            launch {
                launch {
                    log("I'm started")
                    delay(500.milliseconds)
                    log("I'm done!")
                }
            }
        }
    }
    delay(200.milliseconds)
    job.cancel()
}

// 0 [main @coroutine#5] I'm started ← Но выполнение отменяется корректно!
```

Но как и где можно отменить корутину?

15.2.4. Отмененные корутины выбрасывают `CancellationException` в специальных местах

Общий механизм отмены работы корутин работает путем выброса специального типа исключения, `CancellationException`, в определенных местах — прежде всего в точках приостановки.

В этих точках, как вы узнали в главе 14, выполнение корутины может быть временно прервано. В общем случае можно предположить, что все функции приостановки в библиотеке корутин создают точки, где может возникнуть `CancellationException`. В зависимости от факта отмены области видимости следующий фрагмент кода выведет либо А, либо АВС. Результат АВ получить невозможно, поскольку между В и С нет точки отмены:

```
coroutineScope {
    log("A")
    delay(500.milliseconds)
    log("B")
    log("C")
}
```

Корутины используют исключение для распространения отмены по своей иерархии, поэтому важно следить за тем, чтобы система случайно не проглотила¹ данное исключение или не обработала его самостоятельно. Рассмотрим следующий фрагмент многократного выполнения некоего кода, который может выбросить исключение `UnsupportedOperationException`:

```
suspend fun doWork() {
    delay(500.milliseconds)  ← Здесь выбрасывается исключение CancellationException
    throw UnsupportedOperationException("Didn't work!")
}

fun main() {
    runBlocking {
        withTimeoutOrNull(2.seconds) {
            while (true) {
                try {
                    doWork()
                } catch (e: Exception) { ← ...и проглатывается, предотвращая отмену
                    println("Oops: ${e.message}")
                }
            }
        }
    }
}

// Oops: Didn't work!
// Oops: Didn't work!
// Oops: Didn't work!
// Oops: Timed out waiting for 2000 ms
// ... (does not terminate)
```

Через две секунды функция `withTimeoutOrNull` запрашивает отмену своей дочерней области действия корутины. При этом следующий вызов функции `delay` выбрасывает `CancellationException`. Но поскольку оператор `catch` перехватывает исключения всех типов, то код просто продолжает циклиться до бесконечности. Это можно исправить, либо пробросив исключение (`if (e is CancellationException) throw e`), либо не перехватывая его сразу же (`catch (e: UnsupportedOperationException)`). При любом из этих изменений код отменяется, как и ожидалось.

ПРИМЕЧАНИЕ

Это не просто попытка уловить исключение, которая может вызвать такое нежелательное поведение. Такую же осторожность необходимо проявлять при работе с любым супертипом `CancellationException`: `IllegalStateException`, `RuntimeException`, `Exception` и `Throwable`.

¹ «Проглотить исключение» означает, что система поймала его, но не обработала. — *Примеч. пер.*

15.2.5. Отмена — операция кооперативная

Все функции из корутин Kotlin по умолчанию отменяемы. Аналогично при использовании библиотеки API, например Ktor, можно рассчитывать, что входящие во фреймворк функции приостановки тоже отменяемы. Однако при написании собственного кода необходимо позаботиться об обеспечении возможности отмены. Легко предположить, что ваш код можно отменить. Но взгляните на следующий фрагмент:

```
suspend fun doCpuHeavyWork(): Int {
    log("I'm doing work!")
    var counter = 0
    val startTime = System.currentTimeMillis()
    while (System.currentTimeMillis() < startTime + 500) {
        counter++
    }
    return counter
}

fun main() {
    runBlocking {
        val myJob = launch {
            repeat(5) {
                doCpuHeavyWork()
            }
        }
        delay(600.milliseconds)
        myJob.cancel()
    }
}
```

Имитация вычислений с высокой нагрузкой на ЦП, выполняемая путем увеличения счетчика в течение 500 миллисекунд

Вы можете предположить, что этот код выведет текст `I'm doing work` дважды, прежде чем будет отменен. Однако фактический вывод этого фрагмента кода показывает, что на самом деле все пять итераций функции `doCpuHeavyWork` завершаются до окончания работы программы:

```
30 [main @coroutine#2] I'm doing work!
535 [main @coroutine#2] I'm doing work!
1036 [main @coroutine#2] I'm doing work!
1537 [main @coroutine#2] I'm doing work!
2042 [main @coroutine#2] I'm doing work!
```

Почему так происходит? Напомним, что отмена работает путем выброса исключения `CancellationException` в точке приостановки внутри функции. Однако, несмотря на наличие модификатора `suspend`, тело функции `doCpuHeavyWork` на самом деле не содержит никаких точек приостановки. В нем выполняется вызов функции `log`, а затем некоторые длительные вычисления с высокой нагрузкой на ЦП (здесь они имитированы с помощью непрерывного увеличения счетчика до истечения 500 миллисекунд).

Именно поэтому мы говорим, что отмена в корутинах Kotlin *кооперативная*. Функции приостановки должны сами предоставить логику для выполнения отмены.

Вызов других функций, которые можно отменить, автоматически вводит точку для возможной отмены кода. Например, добавление вызова `delay` в тело функции `doWork` добавляет точку для возможности отмены функции:

```
suspend fun doCpuHeavyWork(): Int {
    log("I'm doing work!")
    var counter = 0
    val startTime = System.currentTimeMillis()
    while (System.currentTimeMillis() < startTime + 500) {
        counter++
        delay(100.milliseconds)
    }
    return counter
}
```

← Этот вызов функции также добавляет точку, где функцию `doCpuHeavyWork` можно отменить

Но, конечно, никто не станет искусственно затягивать вычисления только для поддержки отмены. Поэтому в корутинах Kotlin предусмотрена функциональность, которая позволяет реализовать возможность отмены. В частности, это функции `ensureActive` и `yield`, а также свойство `isActive`. Далее мы подробнее рассмотрим возможности их применения.

15.2.6. Проверка отмены корутины

Чтобы определить, была ли отменена корутина, можно просто проверить логическое свойство `isActive` функции `CoroutineScope`. Значение `false` указывает на то, что корутина больше не активна. Можно завершить текущую работу, закрыть все приобретенные ресурсы, а затем выполнить возврат результата. Например, можно переписать цикл, чтобы проверить, была ли отменена текущая область видимости корутины:

```
val myJob = launch {
    repeat(5) {
        doCpuHeavyWork()
        if(!isActive) return@launch
    }
}
```

Вместо того чтобы проверять свойство `isActive` и выполнять возврат в случае получения значения `false`, корутины Kotlin предоставляют другую удобную функцию — `ensureActive`. Если корутина больше не активна, то данная функция выбрасывает исключение `CancellationException`:

```
val myJob = launch {
    repeat(5) {
        doCpuHeavyWork()
        ensureActive()
    }
}
```

15.2.7. Возможность перехватывать поток другими корутинами: функция `yield`

Библиотека корутин предоставляет другую полезную функцию — `yield`. Помимо добавления точки отмены, она позволяет другим корутинам работать с занятым в данный момент диспетчером. Разовьем тему и рассмотрим код ниже. Он запускает две корутины, и каждая из них выполняет определенную работу:

```
import kotlinx.coroutines.*

fun doCpuHeavyWork(): Int {
    var counter = 0
    val startTime = System.currentTimeMillis()
    while (System.currentTimeMillis() < startTime + 500) {
        counter++
    }
    return counter
}

fun main() {
    runBlocking {
        launch {
            repeat(3) {
                doCpuHeavyWork()
            }
        }
        launch {
            repeat(3) {
                doCpuHeavyWork()
            }
        }
    }
}
```

Если в реализации функции `doCpuHeavyWork` нет точек приостановки, то первая запущенная корутина выполнится до конца еще до того, как начнет выполняться вторая:

```
29 [main @coroutine#2] I'm doing work!
533 [main @coroutine#2] I'm doing work!
1036 [main @coroutine#2] I'm doing work!
1537 [main @coroutine#3] I'm doing work!
2042 [main @coroutine#3] I'm doing work!
2543 [main @coroutine#3] I'm doing work!
```

Почему так происходит? Не имея каких-либо точек приостановки в теле, базовый механизм корутин не сможет приостановить выполнение первой корутины и начать выполнение второй. Проверка свойства `isActive` или вызов функции `ensureActive` ничего не изменит. Эти функции только проверяют отмену, но не приостанавливают выполнение корутины.

В такой ситуации поможет функция приостановки `yield`. Она вводит в код точку, где может быть выброшено исключение `CancellationException`, а также позволяет диспетчеру переключиться на работу с другой корутиной, если та ожидает (корутина *передает* (`yields`) диспетчер). Можно переписать реализацию функции `doCpuHeavyWork`, чтобы она выглядела следующим образом (листинг 15.5).

Листинг 15.5. Использование функции `yield` для переключения на другую корутину

```
suspend fun doCpuHeavyWork(): Int {
    var counter = 0
    val startTime = System.currentTimeMillis()
    while (System.currentTimeMillis() < startTime + 500) {
        counter++
        yield()
    }
    return counter
}
```

При использовании функции `yield` различные корутины могут работать поочередно, причем корутины 2 и 3 будут обрабатывать свою нагрузку поочередно:

```
0 [main @coroutine#2] I'm doing work!
559 [main @coroutine#3] I'm doing work!
1062 [main @coroutine#2] I'm doing work!
1634 [main @coroutine#3] I'm doing work!
2208 [main @coroutine#2] I'm doing work!
2734 [main @coroutine#3] I'm doing work!
```

На рис. 15.6 показана разница между тремя вариантами: отсутствие точек приостановки и отмены; простая проверка с помощью функций `isActive` или `ensureActive`; вызов функции `yield`. А в табл. 15.1 приведен еще один обзор случаев использования каждой из трех функций.

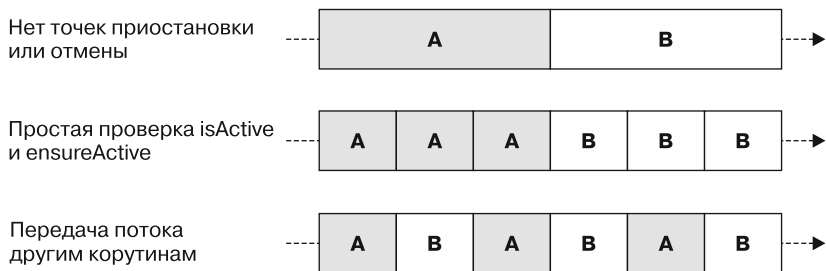


Рис. 15.6. Не имея точек приостановки, несколько корутин всегда будут выполняться до конца и (при однопоточном диспетчере) без чередования. Проверка свойства `isActive` или вызов функции `ensureActive` позволяет корутинам преждевременно завершать свою работу в этих точках отмены. Применение функции `yield` в целях предоставления другим корутинам возможности использовать базовый поток означает, что корутины могут работать поочередно

Таблица 15.1. Механизмы для обеспечения кооперативной отмены

Функция/свойство	Вариант использования
<code>isActive</code>	Проверяет, не было ли запроса на отмену (чтобы сделать некоторые завершающие действия перед остановкой работы)
<code>ensureActive</code>	Вводит «точку отмены» — при отмене выбрасывается исключение <code>CancellationException</code> , мгновенно останавливая работу
<code>yield()</code>	Передаёт вычислительные ресурсы, не позволяя вычислениям с высокой нагрузкой для ЦП исчерпать базовый поток (пул)

15.2.8. Помните об отмене при получении ресурсов

Коду в условиях эксплуатации часто приходится работать с такими ресурсами, как соединения с базами данных, ввод-вывод и т. д. Их необходимо закрывать после использования, чтобы они могли высвободиться надлежащим образом. Отмена, как и любой другой тип исключения, может привести к преждевременному возврату из процесса выполнения кода, поэтому нужно позаботиться о том, чтобы программа случайно не продолжала удерживать ресурсы после того, как корутина будет отменена. В примере ниже для «хранения» строки текста используется объект `DatabaseConnection`, и его необходимо закрыть после использования. Однако в этом фрагменте кода корутина, связанная с выполнением данного запроса, отменяется намеренно, прежде чем сможет закрыть свое соединение с базой данных. В результате функция `close` не вызывается и происходит утечка ресурса:

```
class DatabaseConnection : AutoCloseable {
    fun write(s: String) = println("writing $s!")
    override fun close() {
        println("Closing!")
    }
}

fun main() {
    runBlocking {
        val dbTask = launch {
            val db = DatabaseConnection()
            delay(500.milliseconds)
            db.write("I love coroutines!")
            db.close()
        }
        delay(200.milliseconds)
        dbTask.cancel()
    }
    println("I leaked a resource!")
}
```

Всегда следует разрабатывать код на основе корутин с устойчивостью к отменам. При отмене происходит выброс исключения `CancellationException`. Поэтому в контексте корутин можно использовать тот же механизм, что и для обычной

обработки исключений: блок `finally`. Он выполняется независимо от того, было ли выброшено исключение (листинг 15.6).

Листинг 15.6. Использование блока `finally` для закрытия ресурсов

```
val dbTask = launch {
    val db = DatabaseConnection()
    try {
        delay(500.milliseconds)
        db.write("I love coroutines!")
    } finally {
        db.close()
    }
}
```

Если используемый в корутине ресурс реализует интерфейс `AutoCloseable`, то можно использовать функцию `.use`, описанную в главе 10, как более идиоматическое сокращение для того же поведения (листинг 15.7).

Листинг 15.7. Использование функции `use` для автоматического закрытия ресурсов

```
val dbTask = launch {
    DatabaseConnection().use {
        delay(500.milliseconds)
        it.write("I love coroutines!")
    }
}
```

15.2.9. Фреймворки могут выполнять отмену автоматически

Пока что мы выполняли отмену корутин вручную или, как в случае с функцией `withTimeoutOrNull`, позволяли библиотеке корутин решать, когда инициировать отмену. Во многих приложениях, используемых на практике, фреймворки — например, платформа Android или сетевой фреймворк Ktor — могут обеспечить области видимости корутин, а также инициировать их отмену. В таких случаях ваша задача — выбрать правильную область видимости корутины и убедиться, что написанный код действительно можно отменить.

В контексте Android-приложения класс `ViewModel` предоставляет инструмент `viewModelScope`. Когда класс `ViewModel` очищается — например, когда пользователь выполняет переход с экрана, на котором отображается `ViewModel`, — `viewModelScope` отменяется и все корутины этой области тоже:

```
class MyViewModel: ViewModel() {
    init {
        viewModelScope.launch { ← Запуск корутины в области
            while (true) {          видимости данных модели
                println("Tick!")
                delay(1000.milliseconds)
            }
        }
    }
}
```

Другой пример — серверные приложения на основе Ktor. Здесь каждый обработчик запроса содержит объект типа `PipelineContext` в качестве скрытого приемника, который наследуется от компонента `CoroutineScope`. А значит, есть возможность запускать несколько корутин из обработчика. Эта область действия корутины отменяется, когда клиент отключается. Если клиент закроет соединение с этой конечной точкой менее чем за 5 секунд, то строка `I'm done` не будет выведена. Так произойдет потому, что корутина из области видимости корутины запроса будет отменена:

```
routing {
    get("/") { // this: PipelineContext
        launch {
            println("I'm doing some background work!")
            delay(5000.milliseconds)
            println("I'm done")
        }
    }
}
```

Запуск корутины в области видимости
корутины уровня запроса

Такое поведение не позволяет корутинам уровня запроса выполнять ненужную работу. Когда клиент не ожидает ответа, нет смысла завершать все вычисления, начатые обработчиком запроса. Для асинхронной работы, независимой от присутствия клиента, следует выбрать другую область видимости. В Ktor класс `Application` дублирует область видимости корутины, имеющей то же время жизни, что и приложение Ktor (это означает, что она будет отменена только после остановки приложения). Эта область видимости хорошо подходит для корутин, которые должны выполняться независимо от области видимости запроса. К ним можно получить доступ с помощью переменной `call` (листинг 15.8).

Листинг 15.8. Запуск долгоживущей корутины в Ktor

```
routing {
    get("/") {
        call.application.launch {
            println("I'm doing some background work!")
            delay(5000.milliseconds)
            println("I'm done")
        }
    }
}
```

Запуск корутины в ее области действия
на уровне приложения

Даже когда клиент отменяет HTTP-запрос, запущенная корутина не отменяется. Это связано с тем, что она является дочерней по отношению к области видимости корутины приложения, а не области видимости корутины уровня запроса.

Вы изучили базовые инструменты для написания конкурентного кода Kotlin: корутины, их контексты, области видимости и концепцию структурированной конкурентности. В следующей главе мы перейдем к потокам. Они помогут моделировать последовательные потоки значений поверх изученных ранее абстракций.

РЕЗЮМЕ

- Структурированная конкурентность дает контроль над выполняемой корутинами работой и не позволяет бесконтрольным корутинам избежать отмены.
- Новые области видимости корутин можно создавать с помощью вспомогательной функции приостановки `coroutineScope` и функции-конструктора `CoroutineScope`. Несмотря на схожее название, они служат для разных целей:
 - `coroutineScope` предназначена для конкурентной декомпозиции работы: запуск нескольких корутин, ожидание вычисления результата, его возврат;
 - `CoroutineScope` создает область видимости корутины, используемую для привязки корутин к жизненному циклу класса. Обычно используется с типом `SupervisorJob`.
- `GlobalScope` — специальная область видимости корутины. Она часто показывается в примерах фрагментов кода, но ее не следует применять в коде приложений, поскольку она нарушает структурированную конкурентность.
- Контекст корутин управляет выполнением отдельных корутин. Он наследуется по их иерархии.
- Иерархия «родитель — потомок» между корутинами и их областями видимости устанавливается через ассоциированный объект `Job` в контексте корутины.
- Точки приостановки — это места, в которых можно приостановить выполнение одних корутин и начать работу других.
- Отмена осуществляется путем выброса исключения `CancellationException` в точках приостановки.
- Не допускается проглатывать (перехватить и не обработать) исключения отмены. Их необходимо либо пробрасывать, либо вообще не улавливать.
- Отмена — нормальное явление, и ваш код должен быть рассчитан на его обработку.
- Отмену можно вызвать самостоятельно с помощью функций `cancel` или `withTimeoutOrNull`. Кроме того, многие существующие фреймворки могут отменять корутины автоматически.
- Пометить функцию модификатором `suspend` недостаточно для поддержки отмены. Однако в корутинах Kotlin предусмотрены механизмы, поддерживающие создание отменяемых функций приостановки (например, функции `ensureActive` или `yield`, а также свойство `isActive`).
- Фреймворки используют области видимости корутин, чтобы привязать их к жизненному циклу приложения (например, к времени отображения данных модели на экране или выполнения обработчика запроса).

16

Потоки

В этой главе

- ✓ Работа с потоками как модель для последовательных потоков значений.
- ✓ Примеры использования холодных и горячих потоков, разница между ними.

В предыдущей главе мы рассмотрели корутины и функции приостановки как базовые абстракции, которые применяются для конкурентного программирования в Kotlin. В этой главе мы переключимся на более высокоуровневую абстракцию, созданную поверх корутин, — *потоки*. Они позволяют работать с несколькими последовательными значениями во времени, используя при этом механизм конкурентности Kotlin. Мы обсудим все тонкости и особенности потоков, а также рассмотрим различные типы потоков и то, как их создавать, преобразовывать и потреблять.

16.1. ПОТОКИ МОДЕЛИРУЮТ ПОСЛЕДОВАТЕЛЬНЫЕ ПОТОКИ ЗНАЧЕНИЙ

Функция приостановки, как вы узнали в главе 14, может приостанавливать свое выполнение один или более раз. Однако она может возвращать только одно значение, например примитив, объект или коллекцию объектов. Это показано в листинге 16.1 — функция приостановки `createValues` создает список из трех значений. Чтобы смоделировать более длительное вычисление, можно ввести задержку в одну секунду на элемент. Но при запуске функция возвращает результат только после того, как все значения будут вычислены, — через три секунды на экране появятся три значения.

Листинг 16.1. Функции приостановки не возвращают промежуточных значений

```
import kotlinx.coroutines.delay
import kotlinx.coroutines.runBlocking
import kotlin.time.Duration.Companion.seconds

suspend fun createValues(): List<Int> {
    return buildList {
        add(1)
        delay(1.seconds)
        add(2)
        delay(1.seconds)
        add(3)
        delay(1.seconds)
    }
}

fun main() = runBlocking {
    val list = createValues()
    list.forEach {
        log(it)
    }
}

// 3099 [main @coroutine#1] 1
// 3107 [main @coroutine#1] 2
// 3107 [main @coroutine#1] 3
```

Приостанавливается, пока не будет
заполнен весь список, а после этого
возвращает результат

Все значения выводятся
только спустя три секунды

Однако обзор фактической реализации функции `createValues` показывает, что элемент 1 на самом деле был доступен мгновенно. А элемент 2 мог быть доступен уже через 1 секунду и т. д. В подобных сценариях, когда функция вычисляет несколько значений за определенный промежуток времени, может потребоваться возвращать значения асинхронно, по мере их поступления, а не после завершения выполнения функции. Именно в этом случае применяются потоки.

В Kotlin потоки — это абстракция на основе корутин. Она позволяет работать со значениями, появляющимися со временем. Общий дизайн потоков в значительной степени вдохновлен инициативой Reactive Streams. С этой абстракцией вы, возможно, уже сталкивались: RxJava (<https://reactivex.io/>) и Project Reactor (<https://projectreactor.io/>) — две популярные реализации реактивных потоков на JVM.

Потоки — тоже абстракция общего назначения, она применяется для реализации таких функций, как прогрессивная загрузка, работа с потоками событий или моделирование API в стиле подписки.

16.1.1. Потоки позволяют работать с элементами по мере их создания

Перепишем функцию `createValues`, показанную в листинге 16.1, чтобы использовать потоки. Для этого применим функцию-конструктор `flow` вместо функции `buildList`. Функцию `emit` используем для добавления элементов в поток. После вызова функции можно применить функцию `collect` для итерации элементов

в потоке (листинг 16.2) (функция-конструктор `flow` будет описана в подразделе 16.2.1, а функция `collect` — в подразделе 16.2.2).

Листинг 16.2. Создание и сбор потока

```
import kotlinx.coroutines.delay
import kotlinx.coroutines.runBlocking
import kotlinx.coroutines.flow.*
import kotlin.time.Duration.Companion.milliseconds

fun createValues(): Flow<Int> {
    return flow {
        emit(1)
        delay(1000.milliseconds)
        emit(2)
        delay(1000.milliseconds)
        emit(3)
        delay(1000.milliseconds)
    }
}

fun main() = runBlocking {
    val myFlowOfValues = createValues()
    myFlowOfValues.collect { log(it) }
}

// 29 [main @coroutine#1] 1
// 1100 [main @coroutine#1] 2
// 2156 [main @coroutine#1] 3
```

Каждый произведенный элемент немедленно передается коллектору

Значения выводятся на экран по мере их производства

Изучив временные метки вывода, можно быстро заметить, что элементы из потока отображаются сразу же после их производства. Коду не нужно ждать, пока все значения будут вычислены. Эта базовая абстракция, лежащая в основе потоков в Kotlin, позволяет работать со значениями сразу после их вычисления, а не ждать создания всей партии элементов. Вы будете подробно изучать ее в ходе чтения материала данной главы.

16.1.2. Различные типы потоков в Kotlin

Хотя все потоки в Kotlin предоставляют согласованный набор API для работы со значениями, поступающими с течением времени, в Kotlin различают две категории потоков — холодные и горячие. Короткое их описание приведено ниже.

Холодные потоки представляют собой асинхронные потоки данных. Они начинают производить элементы, только когда их элементы потребляются отдельным коллектором.

Горячие потоки работают в режиме трансляции и производят элементы независимо от того, потребляются ли они на самом деле.

На данный момент это описание может показаться несколько абстрактным, но, когда вы более тщательно изучите эти два типа потоков, их различия и сходства станут для вас очевидными. Мы начнем с обсуждения холодных потоков, а горячие подробно рассмотрим в разделе 16.3.

16.2. ХОЛОДНЫЕ ПОТОКИ

В разделе 16.1.1 мы применили первый холодный поток, чтобы представить асинхронный поток чисел, которые вычисляются с течением времени. Рассмотрим этот код более подробно и обсудим различные аспекты работы с холодными потоками: их создание, условия работы, их отмену и использование в них конкурентности.

16.2.1. Создание холодного потока с помощью функции-конструктора потока

Создать новый холодный поток очень просто: как и в случае с коллекциями, для этого есть функция-конструктор — `flow`. Внутри ее блока можно вызвать специальную функцию `emit`. Она предлагает значение коллектору потока и приостанавливает выполнение функции-конструктора до тех пор, пока значение не будет обработано коллектором. Можно считать, что это асинхронный оператор `return`. В сигнатуре функции `flow` объявляется блок с модификатором `suspend`, поэтому из конструктора можно вызывать и другие функции приостановки, такие как `delay` (листинг 16.3).

Листинг 16.3. Вызов приостанавливающих функций из конструктора `flow`

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*
import kotlin.time.Duration.Companion.milliseconds

fun main() {
    val letters = flow {
        log("Emitting A!")
        emit("A")
        delay(200.milliseconds)
        log("Emitting B!")
        emit("B")
    }
}
```

Функция `emit` предлагает значение коллектору потока

Стоит отметить, что при выполнении данного кода вы не увидите никакого вывода. Это потому, что функция-конструктор возвращает объект типа `Flow<T>`, представляющий собой последовательный поток значений. Как и в случае с последовательностями, этот поток изначально инертен. Он фактически не выполняется до тех пор, пока для него не будет вызван *терминальный оператор*. Он запустит

вычисления, определенные в конструкторе, и любые другие существующие промежуточные операторы (подробнее о промежуточных операторах для потоков расскажем в разделе 17.2, а терминальные операторы рассмотрим в разделе 17.4). Именно поэтому поток называется *холодным*: он инертен по умолчанию, пока его значения не будут собраны.

Вызов функции-конструктора `flow` фактически не запускает никакой работы, поэтому можно создать поток в «обычном» коде Kotlin без приостановки. На практике разработчики сталкиваются с написанием функций с сигнатурой без приостановки, которые возвращают несколько значений с течением времени через холодный поток. И в таком случае можно вызывать функции приостановки внутри конструктора потоков:

```
import kotlinx.coroutines.flow.*

fun getElementFromNetwork(): Flow<String> {
    return flow {
        // здесь приостановка сетевого вызова
    }
}
```

Код внутри функции-конструктора выполняется только после сбора значений, поэтому можно определить и вернуть бесконечный поток, как и в случае с последовательностями, описанном в главе 6. В листинге 16.4 создается поток `counterFlow`. Он производит постоянно увеличивающееся число каждые 200 миллисекунд.

Листинг 16.4. Создание бесконечного потока

```
val counterFlow = flow {
    var x = 0
    while (true) {
        emit(x++)
        delay(200.milliseconds)
    }
}
```

Этот цикл начнет работать только после того, как значения потока будут собираться коллектором. Рассмотрим этот процесс.

16.2.2. Холодные потоки не выполняют никакой работы до тех пор, пока значения не будут собраны

Вызов функции `collect` для потока `Flow` запускает его логику, а код для сбора значений потока обычно называют *коллектором* (или *сборщиком*). При вызове функции `collect` можно предоставить лямбду, и она будет вызываться для каждого производимого элемента потока. Сбор значений потока фактически выполняет приостанавливающий код внутри потока, поэтому `collect` — функция приостановки, ведь она приостанавливает выполнение потока до его завершения. И лямбда,

переданная коллектору, может приостановиться, что позволит вам обращаться к дальнейшим функциям приостановки. Например, коллектор потока может записывать данные в базу данных или выполнять HTTP-запросы на основе полученных от потока значений.

Во фрагменте кода ниже выполняется сборка значений потока `letters`. Поскольку `collect` — это функция приостановки, то ее вызов необходимо обернуть в функцию `runBlocking`. Задержка после сбора каждого элемента позволит проиллюстрировать, что лямбда, переданная в `collect`, представляет собой еще и функцию приостановки.

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*
import kotlin.time.Duration.Companion.milliseconds

val letters = flow {
    log("Emitting A!")
    emit("A")
    delay(200.milliseconds)
    log("Emitting B!")
    emit("B")
}

fun main() = runBlocking {
    letters.collect {
        log("Collecting $it")
        delay(500.milliseconds)
    }
}

// 27 [main @coroutine#1] Emitting A!
// 38 [main @coroutine#1] Collecting A
// 757 [main @coroutine#1] Emitting B!
// 757 [main @coroutine#1] Collecting B
```

Если взглянуть на временные метки результата, то снова становится ясно, что коллектор отвечает за выполнение логики потока: задержка между сбором элементов А и В составляет примерно 700 миллисекунд. Это связано с тем, что происходит такая последовательность событий.

- Коллектор запускает логику из конструктора потока, вызывая производство первого элемента.
- Вызывается лямбда коллектора, которая регистрирует сообщение и задерживает его на 500 миллисекунд.
- Затем поток лямбд продолжается, задерживаясь еще на 200 миллисекунд, прежде чем произойдет производство элемента (связанный с ним сбор).

Данный процесс показан на рис. 16.1, а в подразделе 16.2.4 мы более подробно поговорим об этой взаимосвязанной природе выполнения.

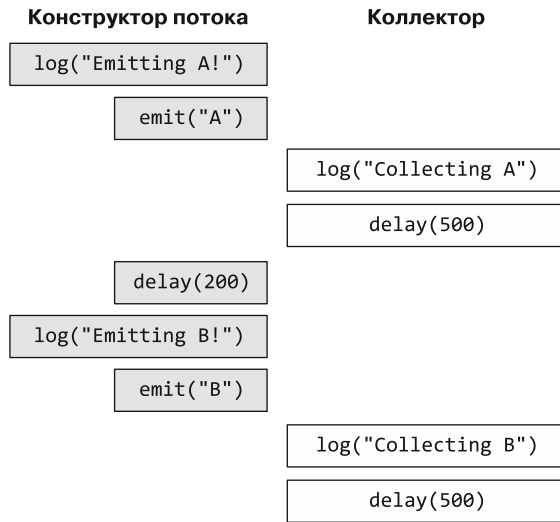


Рис. 16.1. Производство первого элемента в конструкторе потока означает вызов лямбды, связанной с коллектором. Она регистрирует сообщение и задерживает выполнение, прежде чем снова запустить лямбду-конструктор потока. После задержки на 200 миллисекунд и сообщения журнала поток управления снова переходит к коллектору, собирая второй элемент и задерживаясь на последние 500 миллисекунд. Эти операции взаимосвязаны, но выполняются в одной и той же корутине

Стоит отметить, что так же, как последовательности вычисляются повторно при каждом применении терминального оператора, несколько вызовов функции `collect` для холодного потока запускают выполнение его кода несколько раз. На это стоит обратить внимание, если у потока есть побочные эффекты, такие как выполнение сетевых запросов: они будут выполняться несколько раз (листинг 16.5) (о том, как избежать такого поведения, расскажем в разделе 16.3).

Листинг 16.5. Сбор одного и того же потока несколько раз

```

import kotlinx.coroutines.flow.*
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.milliseconds

fun main() = runBlocking {
    letters.collect {
        log("(1) Collecting $it")
        delay(500.milliseconds)
    }
    letters.collect {
        log("(2) Collecting $it")
        delay(500.milliseconds)
    }
}
  
```

```
// 23 [main @coroutine#1] Emitting A!
// 33 [main @coroutine#1] (1) Collecting A
// 761 [main @coroutine#1] Emitting B!
// 762 [main @coroutine#1] (1) Collecting B
// 1335 [main @coroutine#1] Emitting A!
// 1335 [main @coroutine#1] (2) Collecting A
// 2096 [main @coroutine#1] Emitting B!
// 2096 [main @coroutine#1] (2) Collecting B
```

Функция `collect` приостанавливается до тех пор, пока не будут обработаны все элементы потока. Однако, как было показано в подразделе 16.2.1, в потоках потенциально может быть бесконечно много элементов — это означает, что функция `collect` приостановится на неопределенное время. Поток можно отменить, чтобы остановить сбор его элементов до того, как все они будут обработаны.

16.2.3. Отмена сбора потока

В главе 15 мы познакомили вас с механизмами отмены корутин. Они применимы и к коллекторам потоков. Отмена корутины коллектора остановит сбор потока в следующей точке отмены (листинг 16.6).

Листинг 16.6. Отмена сбора потока

```
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.seconds

fun main() = runBlocking {
    val collector = launch {
        counterFlow.collect {
            println(it)
        }
    }
    delay(5.seconds)
    collector.cancel()
}

// 1 2 3 ... 24
```

ПРИМЕЧАНИЕ

Как и другие встроенные функции приостановки, `emit` действует как точка отмены и приостановки для кода.

Более подробно изучив промежуточные и терминальные операторы, вы познакомитесь с некоторыми дополнительными способами отмены выполнения потоков. Одним из таких примеров может быть оператор `take`, о котором мы поговорим в главе 17.

16.2.4. Внутренняя работа холодных потоков

В разделе 16.1 говорилось, что потоки — это абстракция поверх механизма корутин Kotlin, а точнее, функций приостановки. Вы просмотрели нескольких первых потоков, и теперь можно изучить их реализацию. Использование потоков не требует знания их внутреннего устройства, однако, имея о нем представление, вы все же сможете лучше понять холодные потоки и их механику. Они представляют собой умную комбинацию функций приостановки и лямбд с приемниками. Обе эти функциональные возможности Kotlin мы рассматривали ранее. Определение холодных потоков, предоставляемое корутинами, на самом деле состоит всего из нескольких строк и требует только двух интерфейсов: `Flow` и `FlowCollector`. И каждый из них определяет по одной функции — `collect` и `emit`:

```
interface Flow<out T> {
    suspend fun collect(collector: FlowCollector<T>)
}

interface FlowCollector<in T> {
    suspend fun emit(value: T)
}
```

При определении потока с помощью функции-конструктора `flow` у предоставляемой вами лямбды будет тип приемника `FlowCollector`. Именно это позволяет вызывать функцию `emit` из конструктора. Она вызывает лямбду, переданную функции `collect`. По сути, у вас будет две лямбды, вызывающие друг друга:

- вызов функции `collect` инициирует запуск тела функции-конструктора `flow`;
- когда этот код вызывает функцию `emit`, она вызывает лямбду, переданную функции `collect` с параметром, переданным `emit`;
- завершив выполнение лямбда-выражения, функция возвращается обратно в тело функции-конструктора и продолжает выполнение.

На рис. 16.2 показана эта концепция для следующего фрагмента кода:

```
val letters = flow {
    delay(300.milliseconds)
    emit("A")
    delay(300.milliseconds)
    emit("B")
}

letters.collect { letter ->
    println(letter)
    delay(200.milliseconds)
}
```

В обычном холодном потоке не так уж много движущихся частей. Тем не менее они предоставляют чрезвычайно полезную и расширяемую абстракцию, позволяющую писать код обработки потоков значений. Но при этом конструкция остается незатратной в отношении ресурсов.

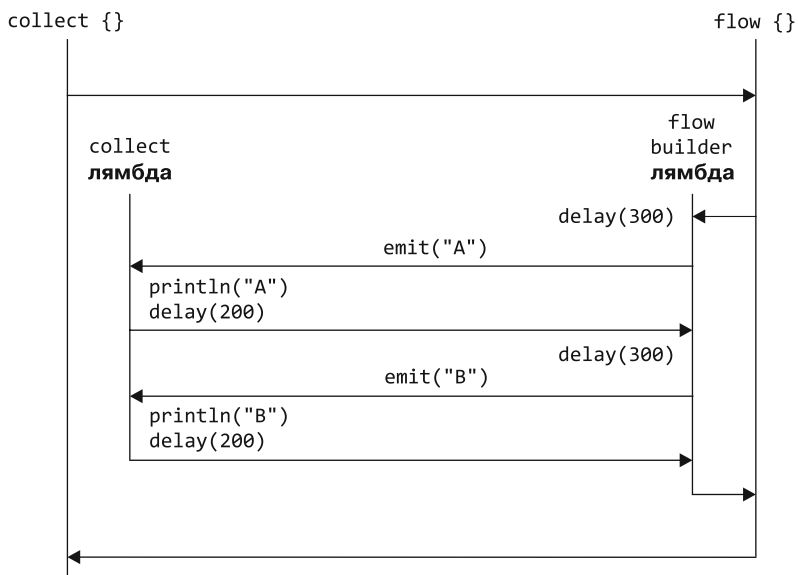


Рис. 16.2. Вызов функции `collect` вызывает лямбду, связанную с функцией-конструктором `flow`. В этом случае лямбда приостанавливается на 300 миллисекунд. Вызов `emit` выполняет лямбду, переданную в `collect`, и доводит ее до завершения. Здесь она выполняет оператор `print` и задерживается на 200 миллисекунд. Затем поток управления возвращается к лямбде конструктора потока, который повторяет этот цикл второй раз, после чего возвращается и завершает первоначальный вызов `collect`. Все это обычные вызовы функций. А весь этот код выполняется в рамках одной корутины

16.2.5. Конкурентные потоки на основе канальных потоков

Все созданные ранее с помощью функции-конструктора `flow` холодные потоки были последовательными. Блок кода выполняется, как и тело функции приостановки, в виде одной корутины. Следовательно, вызовы функции `emit` тоже выполняются последовательно. Для многих случаев этой базовой абстракции более чем достаточно, но, когда поток выполняет ряд независимых вычислений, эта последовательная природа может оказаться узким местом. В листинге 16.7 показано объявление потока `randomNumbers`. Он вычисляет десять чисел с помощью искусственно замедленной функции `getRandomNumber`.

Листинг 16.7. Последовательное выполнение обычного холодного потока

```

import kotlinx.coroutines.flow.*
import kotlinx.coroutines.*
import kotlin.random.Random
import kotlin.time.Duration.Companion.milliseconds

suspend fun getRandomNumber(): Int {
    delay(500.milliseconds)
    return Random.nextInt()
}

```

```

val randomNumbers = flow {
    repeat(10) {
        emit(getRandomNumber())
    }
}

fun main() = runBlocking {
    randomNumbers.collect {
        log(it)
    }
}

// 583 [main @coroutine#1] 1514439879
// 1120 [main @coroutine#1] 1785211458
// 1693 [main @coroutine#1] -996479986
// ...
// 5463 [main @coroutine#1] -2047597449

```

Сбор этого потока занимает чуть более 5 секунд, поскольку вызовы функции `getRandomNumber` выполняются поочередно. Процесс показан на рис. 16.3.

Поток выполняется последовательно, причем все вычисления выполняются в одной и той же корутине. Операции в данном коде (генерация чисел) не зависят друг от друга, поэтому кажутся идеальным кандидатом для конкурентного или, если хотите, параллельного выполнения. Как и в случае с функцией `async`, описанной в главе 14, благодаря этому вы сможете значительно ускорить выполнение, сократив время вычисления всех десяти элементов примерно до 500 миллисекунд.

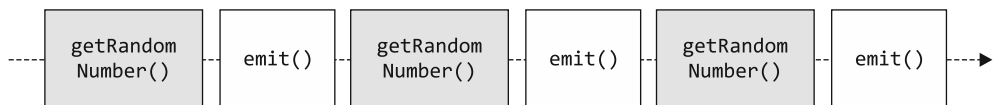


Рис. 16.3. При использовании обычного холодного потока каждое случайное число генерируется и выпускается. Это происходит полностью последовательно

Может возникнуть соблазн применить все накопленные знания, чтобы ввести конкурентность в вызов конструктора потока, запустив несколько фоновых корутин и выдавая значения непосредственно из них. Однако при этом вы получите сообщение об ошибке: `Flow invariant is violated: Emission from another coroutine is detected. FlowCollector is not thread-safe and concurrent emissions are prohibited` (Инвариант потока нарушен: обнаружен выпуск элементов из другой корутины. `FlowCollector` не является потокобезопасным, и конкурентное производство элементов запрещено).

```

val randomNumbers = flow {
    coroutineScope {
        repeat(10) {
            launch { emit(getRandomNumber()) }
        }
    }
}

```

Ошибка: функцию `emit` нельзя вызвать из другой корутины

Это происходит потому, что базовая абстракция холодного потока позволяет вызывать функцию `emit` только из одной и той же корутины. Необходим конструктор потоков, который имеет возможность создавать конкурентный поток и допускает выпуск элементов из нескольких корутин. В Kotlin это называется *канальным потоком*, и его можно создать с помощью функции-конструктора `channelFlow`. Это особый тип холодного потока. В нем нет функции `emit` для последовательного выпуска элементов. Вместо этого несколько корутин могут использовать функцию `send` для предложения значений. Коллектор этого потока по-прежнему получает их последовательно, и лямбда `collect` может выполнять свою работу. В листинге 16.8 канальный поток используется для оптимизации предыдущей реализации. Как и функция `coroutineScope`, лямбда для конструктора `channelFlow` предоставляет область видимости корутин для запуска новых фоновых корутин.

Листинг 16.8. Конкурентная отправка элементов в канальный поток

```
import kotlinx.coroutines.flow.channelFlow
import kotlinx.coroutines.launch

val randomNumbers = channelFlow { ← Создание нового канального потока
    repeat(10) {
        launch {
            send(getRandomNumber()) ← Функцию send можно вызвать
        }                               из других корутин
    }
}
```

Коллектор работает как раньше, но использует канальный поток. Здесь можно заметить, что функция `getRandomNumber` теперь действительно выполняется конкурентно и все выполнение завершается примерно за 500 миллисекунд (рис. 16.4):

```
553 [main] -1927966915
568 [main] 222582016
...
569 [main] 1827898086
```

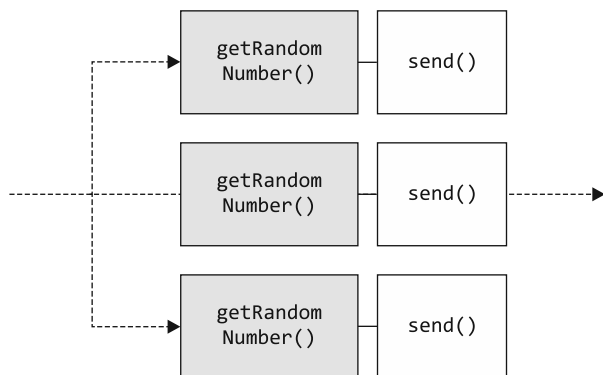


Рис. 16.4. Канальный поток может запускать другие корутины и отправлять элементы одновременно

Функция `channelFlow` знакомит вас с конкурентными вызовами конструктора потоков. На данном этапе вполне естественно задаться вопросом, а есть ли вообще смысл использовать обычный поток, ведь канальный вариант может делать все то же самое, что и обычный, и даже больше! Как же выбрать?

В целом обычные холодные потоки — самая простая и производительная из доступных абстракций. Этот вид потоков строго последователен, и в них нельзя запускать новые корутины, но их очень просто создать, их интерфейс состоит всего из одной функции (`emit`) и у них нет дополнительных движущихся частей или накладных расходов, требующих внимания. Канальные же потоки специально разработаны для использования в ситуациях, когда есть конкурентные операции. Их создание обходится не так дешево, поскольку внутри реализации им приходится управлять еще одним конкурентным примитивом — *каналом*. А это сравнительно низкоуровневая абстракция для межпрограммного взаимодействия. Функция `send` — часть общего более сложного интерфейса, предоставляемого каналами.

Выбирая между обычными холодными и канальными потоками, используйте последние только в том случае, если требуется запускать новые корутины изнутри вашего потока. В противном случае лучше выбрать обычные холодные потоки.

Таким образом, холодные потоки — полезная абстракция для работы со значениями, которые будут вычислены через какое-то время. Позже, в главе 17, вы узнаете об эффективных способах применения дальнейших преобразований в потоках. Однако холодные потоки всегда напрямую связаны со своим коллектором: каждый коллектор выполняет код, заданный для потока, независимо. В следующем разделе вы узнаете о втором типе потоков, позволяющих осуществлять широковещательную передачу значений между корутинами, а также управлять состоянием в конкурентной системе, — горячих потоках.

16.3. ГОРЯЧИЕ ПОТОКИ

Несмотря на общую структуру производства и сбора, у горячих потоков есть ряд отличительных свойств. Коллекторы не запускают выполнение логики потока независимо — горячие потоки распределяют производимые элементы между несколькими коллекторами, они называются *подписчиками*. Это означает, что горячие потоки подходят для случаев передачи события или изменения состояния системы, которые происходят самостоятельно или не связаны с существованием коллектора. Отсюда они и получили свое название: независимо от наличия подписчиков, во всегда активных горячих потоках могут производиться элементы.

Корутины Kotlin сразу поставляются с двумя реализациями горячих потоков:

- *общие (совместно используемые) потоки* используются для передачи значений;
- *потоки состояний* предназначены для особого случая передачи состояний.

Полезно знать, как работают оба типа потоков, но на практике чаще применяются потоки состояний. (В подразделе 16.3.3 мы расскажем, что код, использующий общий поток, обычно можно преобразовать и задействовать вместо него поток состояний.) Сначала рассмотрим общие потоки в действии, а затем перейдем к потокам состояний.

16.3.1. Общие потоки транслируют значения абонентам

Общие потоки работают в *вещательном режиме* — независимо от наличия подписчика (коллектора общего потока), элементы могут создаваться (рис. 16.5). Чтобы увидеть это поведение вещания, вы можете смоделировать реальный передатчик — цифровую станцию (см. статью Соррел-Дежерин «Жуткий мир цифровых станций», 2014 год, <https://www.bbc.com/news/magazine-24910397>). Эти таинственные радиостанции существуют в реальной жизни и передают закодированные сообщения шпионам по всему миру на открытых частотах. Их можно прослушать и попытаться расшифровать. (У нас нет готового шпионского справочника, поэтому придется обойтись генерацией случайных чисел.)

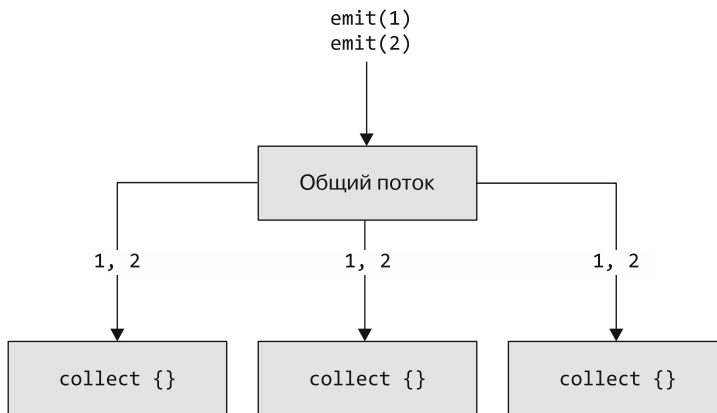


Рис. 16.5. Общие потоки работают в вещательном режиме. Когда элемент появляется (выбрасывается) в общем потоке, все подписчики, собирающие поток в данный момент, получают его

Общие потоки обычно объявляются в классе-контейнере — в нашем случае это класс `RadioStation`. Изменяемую версию общего потока `_messageFlow` можно использовать для выброса (`emit`) значений. Она инкапсулируется как приватное свойство класса. А версия потока только для чтения, `messageFlow`, используется для подписки на выбросы и предоставляется как публичное свойство. Кроме того, определяется функция `beginBroadcasting`, предназначенная для запуска новой корутины в заданной области видимости, которая испускает (и регистрирует) случайные числа в потоке сообщений. Реализация показана в листинге 16.9.

Листинг 16.9. Трансляция значений с помощью функции `SharedFlow`

```

import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*
import kotlin.random.*
import kotlin.time.Duration.Companion.milliseconds

class RadioStation {
    private val _messageFlow = MutableSharedFlow<Int>()
    val messageFlow = _messageFlow.asSharedFlow()

    fun beginBroadcasting(scope: CoroutineScope) {
        scope.launch {
            while(true) {
                delay(500.milliseconds)
                val number = Random.nextInt(0..10)
                log("Emitting $number!")
                _messageFlow.emit(number)
            }
        }
    }
}

```

Определение нового изменяемого потока в качестве приватного свойства

Предоставление общего потока в виде, доступном только для чтения

Выброс значения в изменяемый общий поток из корутины

Как видите, создание горячего потока, например `SharedFlow`, работает иначе, чем создание холодного потока. Вместо того чтобы использовать конструктор потока, вы получаете ссылку на изменяемую версию потока. Производство элементов происходит независимо от наличия подписчиков, поэтому необходимо запустить корутину, фактически выполняющую выбросы. Кроме того, это означает, что в коде может содержаться несколько корутин выброса значений в изменяемый общий поток, не создавая каких-либо дополнительных проблем.

ПОДЧЕРКИВАНИЕ ПРИ ИМЕНОВАНИИ ГОРЯЧИХ ПОТОКОВ

Вы можете задаться вопросом, почему общие потоки (и потоки состояний) следуют паттерну определения приватной переменной с подчеркиванием в начале имени, а также публичной переменной с именем без подчеркивания. На момент написания этой книги Kotlin не поддерживал различные типы для приватных и публичных переменных. Определение изменяемой версии потока как приватной и предоставление свойства типа `SharedFlow<T>` только для чтения исключает доступность изменяемой версии общего потока для классов-потребителей в целях инкапсуляции и сокрытия информации. Это важно, поскольку потребители класса обычно должны только подписываться на поток, а не производить для него элементы. В версии Kotlin 2.x планируется добавить возможность указывать тип свойства в зависимости от точки обращения — внутри или вне класса (см. статью *Support Having a "Public" and a "Private" Type for the Same Property* Светланы Исаковой, <https://youtrack.jetbrains.com/issue/KT-14663>).

При создании экземпляра класса `RadioStation` и вызове функции `beginBroadcasting` вещание начинается немедленно, даже при отсутствии подписчиков:

```
fun main() = runBlocking {
    RadioStation().beginBroadcasting(this)
}

// 575 [main @coroutine#2] Emitting 2!
// 1088 [main @coroutine#2] Emitting 10!
// 1593 [main @coroutine#2] Emitting 4!
// ...
```

← Запуск корутины в области видимости корутины `runBlocking`

Добавление подписчика происходит так же, как и сбор холодного потока: просто вызывается функция `collect`. Введенная вами лямбда будет выполняться при каждом выбросе значения. Однако важно отметить, что подписчики получают производимые значения только с начала работы подписки. Для примера можно начать подписку на общий поток после небольшой задержки (листинг 16.10).

Листинг 16.10. Подписка на общий поток с задержкой

```
fun main(): Unit = runBlocking {
    val radioStation = RadioStation()
    radioStation.beginBroadcasting(this)
    delay(600.milliseconds)
    radioStation.messageFlow.collect {
        log("A collecting $it!")
    }
}
```

В результатах выполнения кода видно, что подписчик не собрал первое значение, произведенное примерно через 500 миллисекунд:

```
611 [main @coroutine#2] Emitting 8!
1129 [main @coroutine#2] Emitting 9!
1131 [main @coroutine#1] A collecting 9!
1647 [main @coroutine#2] Emitting 1!
1647 [main @coroutine#1] A collecting 1!
```

Общие потоки работают в вещательном режиме, поэтому можно добавлять дополнительных подписчиков. Вдобавок ко всему они будут получать выбрасываемые значения существующего потока `messageFlow`. Добавить дополнительного подписчика в свою корутину можно с помощью второго вызова функции `launch` внутри блока `runBlocking`:

```
launch {
    radioStation.messageFlow.collect {
        log("B collecting $it!")
    }
}
```

Он будет получать те же значения, что и все остальные подписчики того же общего потока — ваша цифровая станция выдает свои значения в режиме *вещания*. Обратите внимание: в отличие от холодных потоков, коллекторы не отвечают за фактическое производство элементов в потоке — они являются просто подписчиками и получают уведомление при начале выброса новых элементов.

Повтор значений для подписчиков

При использовании общего потока подписчики вызывают функцию `collect` и получают значения, выпущенные после их подписки. Если требуется, чтобы подписчики получали ранее выданные элементы, то при создании потока `MutableSharedFlow` применяется параметр `replay`. Он создает кэш последних значений для новых подписчиков. В этом примере поток сообщений настраивается для повтора последних пяти значений при добавлении нового подписчика:

```
private val _messageFlow = MutableSharedFlow<Int>(replay = 5)
```

Благодаря этому изменению даже при запуске коллекторов после небольшой задержки 600 миллисекунд они по-прежнему будут получать до пяти элементов, предшествовавших их подписке. В данном случае это означает, что подписчик увидит значение, выданное за 560 миллисекунд, до начала действия подписки:

```
560 [main @coroutine#2] Emitting 6!
635 [main @coroutine#1] A collecting 6!
1080 [main @coroutine#2] Emitting 10!
1081 [main @coroutine#1] A collecting 10!
```

Это удобно, когда требуется, чтобы у подписчиков всегда было несколько последних значений для работы после подписки.

От холодного потока к общему с помощью функции `ShareIn`

Предположим, что в коде есть функция для предоставления потока значений. Например, она выдает значения, поступающие от датчика с интервалом 500 миллисекунд. Эта функция возвращает холодный поток (листинг 16.11).

Листинг 16.11. Простая функция, возвращающая поток значений в заданном интервале

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*
import kotlin.random.*
import kotlin.time.Duration.Companion.milliseconds

fun querySensor(): Int = Random.nextInt(-10..30)

fun getTemperatures(): Flow<Int> {
    return flow {
        while(true) {
            emit(querySensor())
            delay(500.milliseconds)
        }
    }
}
```

Если несколько раз вызвать функцию `collect` для этой функции, например, чтобы вывести температуру один раз в градусах Цельсия, а второй — в градусах Фаренгейта, то каждый коллектор иницирует независимый опрос датчика:


```

fun celsiusToFahrenheit(celsius: Int) =
    celsius * 9.0 / 5.0 + 32.0

fun main() {
    val temps = getTemperatures()
    runBlocking {
        launch {
            temps.collect {
                log("$it Celsius")
            }
        }
        launch {
            temps.collect {
                log("${celsiusToFahrenheit(it)} Fahrenheit")
            }
        }
    }
}

```



При взаимодействии с датчиками, выполнении сетевых запросов или запросов к базе данных желательно избегать излишнего взаимодействия с внешними системами или выполнения длительных вычислений. Поэтому следует использовать поток двумя коллекторами как общий — они должны получать одни и те же элементы.

Этот холодный поток можно преобразовать в горячий общий поток с помощью функции `shareIn` (листинг 16.12). Это преобразование приводит к выполнению кода потока, поэтому функция должна выполняться в корутине. Для этого `shareIn` принимает параметр `scope` типа `CoroutineScope`, для которого запускается эта корутина.

Листинг 16.12. Использование функции `shareIn` для преобразования холодного потока в горячий общий поток

```

fun main() {
    val temps = getTemperatures()
    runBlocking {
        val sharedTemps = temps.shareIn(this, SharingStarted.Lazily)
        launch {
            sharedTemps.collect {
                log("$it Celsius")
            }
        }
        launch {
            sharedTemps.collect {
                log("${celsiusToFahrenheit(it)} Fahrenheit")
            }
        }
    }
}

```

```

// 45 [main @coroutine#3] -10 Celsius
// 52 [main @coroutine#4] 14.0 Fahrenheit
// 599 [main @coroutine#3] 11 Celsius
// 599 [main @coroutine#4] 51.8 Fahrenheit

```

Второй параметр `started` определяет фактический момент запуска потока. Здесь можно задать несколько вариантов поведения:

- `Eagerly` немедленно приступает к сбору потока;
- `Lazily` начинает сбор только при появлении первого подписчика;
- `WhileSubscribed` начинает сбор только при появлении первого подписчика, а после отменяет сбор потока при исчезновении последнего.

В Kotlin принято, что операции для вычисления нескольких значений с течением времени просто предоставляются как холодные потоки. А код приложения при необходимости преобразует их в горячие. Функция `shareIn` участвует в структурированной конкурентности через свою область видимости корутины, поэтому можно гарантировать, что, когда приложению больше не нужна информация из общего потока, его внутренняя логика будет отменена. Это произойдет при отмене окружающей области видимости корутины.

16.3.2. Отслеживание состояния в системе: поток состояний

Особый случай для конкурентных систем — отслеживание некоего изменяющегося со временем значения, или, другими словами, состояния значения. В предыдущем примере это могут быть значения, считываемые датчиком температуры. Они представляют текущее состояние температуры в нашей системе. В корутинах Kotlin есть специализированная абстракция для обработки такого случая — *поток состояний*. Это специальная версия общего потока для легкого отслеживания изменений состояния переменной с течением времени. Существует ряд важных тем, связанных с потоками состояний:

- как создаются и предоставляются подписчикам потоки состояний;
- как безопасно обновлять значение потока состояний даже при параллельном доступе;
- концепция *объединения на основе равенства* дает потокам состояний указание производить элементы только, когда их значения действительно изменяются;
- как преобразовать холодные потоки в потоки состояний.

Создание потока состояний аналогично созданию общего потока: поток создается `MutableStateFlow` как приватное свойство класса и переменная `StateFlow` предоставляется в варианте только для чтения. В листинге 16.13 будет определен счетчик просмотров. Поток состояний представляет собой изменяемое со временем значение, и оно может быть считано в любой момент. Поэтому необходимо предоставить начальное значение в качестве параметра его конструктора (в данном случае ноль просмотров). Для обновления значения потока используется функция `update`, а не `emit`, как было раньше.

Доступ к текущему состоянию из изменяемого потока состояний осуществляется с помощью его свойства `value`. Оно позволяет безопасно прочесть значение, не приостанавливая поток. Обновление значения выполняется с помощью функции `update`, и ее мы рассмотрим более подробно.

Листинг 16.13. Базовая реализация счетчика просмотров с потоком состояний

```
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.*

class ViewCounter {
    private val _counter = MutableStateFlow(0)
    val counter = _counter.asStateFlow()

    fun increment() {
        _counter.update { it + 1 }
    }
}

fun main() {
    val vc = ViewCounter()
    vc.increment()
    println(vc.counter.value)
    // 1
}
```

Создание изменяемого потока состояний с начальным значением, равным 0

Безопасная запись в поток состояний с помощью функции обновления

При более детальном изучении кода становится заметно, что `value` на самом деле представляет собой свойство чтения и записи — ему можно присваивать значения. Поэтому может возникнуть соблазн реализовать функцию `increment` с помощью классического оператора `++`:

```
fun increment() {
    _counter.value++
}
```

Однако в коде есть проблема: эти операции инкремента не *атомарные*. Чтобы проиллюстрировать эту проблему, мы написали короткий фрагмент кода для запуска 10 000 корутин с вызовом функции `increment` (листинг 16.14). Запуск функции `runBlocking` с диспетчером по умолчанию обеспечит их распределение по нескольким потокам.

Листинг 16.14. Увеличение значения переменной `value` потока состояний с помощью 10 000 корутин

```
fun main() {
    val vc = ViewCounter()
    runBlocking(Dispatchers.Default) {
        repeat(10_000) {
            launch { vc.increment() }
        }
    }

    println(vc.counter.value)
    // 4103
}
```

Итоговое число в счетчике гораздо меньше чем 10 000. Это связано с тем, что данные корутины выполняются в нескольких потоках. Операции инкремента выполняются не атомарно, в несколько этапов: считывается текущее значение, вычисляется новое, а затем оно записывается. Если два потока одновременно считывают текущее значение, то одна из операций инкремента будет фактически отменена.

Решить эту проблему в потоках состояний призвана функция `update`. Она помогает выполнять *атомарные* обновления значений потока состояний. Необходимо предоставить лямбда-выражение для вычисления нового значения с учетом предыдущего (в случае со счетчиком просмотров значение должно быть увеличено на единицу). Если два обновления потока состояний происходят параллельно, то функция обновления выполняется снова с обновленным предыдущим значением. Этот механизм гарантирует, что ни одна операция не будет потеряна. Повторный запуск того же тестового кода, представленного в листинге 16.14, с реализацией в листинге 16.13 дает правильный результат.

Потоки состояний производят элементы только в момент фактического изменения значения: объединение на основе равенства

Как и в случае с общим потоком, можно подписаться на поток состояний и получать его значение с течением времени с помощью функции `collect`. В этом примере определяется «селектор направления» с функцией для перевода переключателя в положение «влево» или «вправо» (`LEFT`, `RIGHT`):

```
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.*

enum class Direction { LEFT, RIGHT }

class DirectionSelector {
    private val _direction = MutableStateFlow(Direction.LEFT)
    val direction = _direction.asStateFlow()

    fun turn(d: Direction) {
        _direction.update { d }
    }
}
```

С помощью этого селектора направлений можно выполнить несколько переходов между положениями «влево» и «вправо». Корутина, отвечающая за регистрацию этих переходов, получит уведомление о каждом новом значении из функции `collect`:

```
fun main() = runBlocking {
    val switch = DirectionSelector()
    launch {
        switch.direction.collect {
            log("Direction now $it")
        }
    }
    delay(200.milliseconds)
    switch.turn(Direction.RIGHT)
    delay(200.milliseconds)
}
```

```

switch.turn(Direction.LEFT)
delay(200.milliseconds)
switch.turn(Direction.LEFT)
}

// 37 [main @coroutine#2] Direction now LEFT
// 240 [main @coroutine#2] Direction now RIGHT
// 445 [main @coroutine#2] Direction now LEFT

```

В результатах выполнения этого фрагмента кода видно, что подписчик вызывает-ся только раз, хотя функция `turn` вызывается с параметром `LEFT` два раза подряд. Это происходит из-за того, что потоки состояний выполняют объединение на основе равенства — они передают значения коллекторам только в случае действительного изменения значения. Если предыдущее и обновленное значения совпадают, то выброса не происходит (рис. 16.6). Если при повторном наблюдении элемента он содержит то же значение, то концептуально ничего не изменилось.

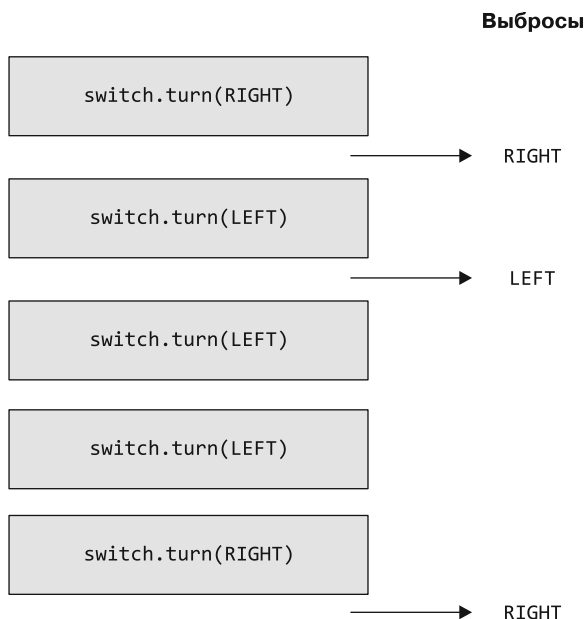


Рис. 16.6. Потоки состояний выполняют объединение на основе равенства. Если значению присваивается уже установленное значение, то новые элементы не создаются

От холодного потока к потоку состояний с помощью функции `StateIn`

При работе с API, предоставляющим холодный поток (как в листинге 16.11), преобразовывать холодный поток в поток состояний можно с помощью функции `stateIn` (листинг 16.15). Это позволит всегда считывать последнее значение, выброшенное исходным потоком. Как и в случае с общими потоками, добавление нескольких коллекторов или обращений к свойству `value` не приведет к выполнению восходящего потока.

Листинг 16.15. Использование функции `stateIn` для преобразования холодного потока в горячий поток состояний

```
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.milliseconds

fun main() {
    val temps = getTemperatures()
    runBlocking {
        val tempState = temps.stateIn(this)
        println(tempState.value)
        delay(800.milliseconds)
        println(tempState.value)
        // 18
        // -1
    }
}
```

В отличие от общих потоков, показанных в подразделе 16.3.1, функция `stateIn` не предоставляет никаких стратегий запуска. Она всегда запускает поток в заданной области видимости и продолжает предоставлять последнее значение своим подписчикам с помощью свойства `value` до момента отмены области видимости.

16.3.3. Сравнение общих потоков и потоков состояний

Вы познакомились с двумя горячими потоками, доступными в Kotlin по умолчанию. Оба они позволяют осуществлять выбросы независимо от присутствия абонентов, но их применение разнится. Общие потоки могут вещать о событиях, а подписчики при этом могут приходить и уходить. Но подписчики получают выбрасываемые элементы только в то время, когда они подписаны. Потоки состояний предназначены для представления любого типа состояния и используют объединение на основе равенства. Это означает, что выбросы происходят только в том случае, если значение в потоке действительно меняется.

В целом потоки состояний предоставляют более простой API, чем общие потоки, — они представляют только одно значение. При использовании общих потоков вы можете столкнуться с дополнительными сложностями, поскольку вам необходимо убедиться в наличии подписчиков, когда они должны принимать производимые элементы. Поэтому стоит потратить немного времени и попытаться переосмыслить решаемые с помощью общего потока задачи. Необходимо разобраться, можно ли вместо него использовать поток состояний.

Например, вы можете столкнуться с разработкой системы, в которой общий поток применяется для передачи сообщений нескольким подписчикам. Однако подобная реализация на самом деле не выводит никаких сообщений (листинг 16.16).

Листинг 16.16. Вещатель выдает все сообщения до появления первого подписчика

```
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.milliseconds
```

```

class Broadcaster {
    private val _messages = MutableSharedFlow<String>()
    val messages = _messages.asSharedFlow()
    fun beginBroadcasting(scope: CoroutineScope) {
        scope.launch {
            _messages.emit("Hello!")
            _messages.emit("Hi!")
            _messages.emit("Hola!")
        }
    }
}

fun main(): Unit = runBlocking {
    val broadcaster = Broadcaster()
    broadcaster.beginBroadcasting(this)
    delay(200.milliseconds)
    broadcaster.messages.collect {
        println("Message: $it")
    }
}

```

// Значения не собираются, ничего не выводится на экран

Подписчик появился только спустя некоторое время после начала вещания сообщений, поэтому не получит ни одного сообщения. Конечно, мы уже показывали точную настройку кэша повторения в общем потоке. Но есть и другое решение: можно применить более простую абстракцию — поток состояний. Вместо того чтобы выдавать отдельные сообщения, поток состояний будет хранить всю историю сообщений в виде списка. И в таком случае подписчики смогут легко получить доступ ко всем предыдущим сообщениям (листинг 16.17).

Листинг 16.17. Использование потока состояний для хранения всей истории сообщений

```

import kotlinx.coroutines.flow.*
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.milliseconds

class Broadcaster {
    private val _messages = MutableStateFlow<List<String>>(emptyList())
    val messages = _messages.asStateFlow()
    fun beginBroadcasting(scope: CoroutineScope) {
        scope.launch {
            _messages.update { it + "Hello!" }
            _messages.update { it + "Hi!" }
            _messages.update { it + "Hola!" }
        }
    }
}

fun main() = runBlocking {
    val broadcaster = Broadcaster()
    broadcaster.beginBroadcasting(this)
}

```

```
        delay(200.milliseconds)
        println(broadcaster.messages.value)
    }

    // [Hello!, Hi!, Hola!]
```

Переформулировав задачу так, чтобы можно было использовать потоки состояний вместо общих потоков, вы сможете задействовать более простой API и позволить появляющимся позже коллекторам видеть всю историю обновлений.

16.3.4. Горячий, холодный, общий потоки, поток состояний: когда использовать тот или иной поток

Мы подробно рассмотрели различные типы потоков из корутин Kotlin. В табл. 16.1 для удобства снова перечислены основные свойства потоков.

Таблица 16.1. У горячих и холодных потоков разные свойства и поведение. Поэтому они служат разным целям

Холодный поток	Горячий поток
По умолчанию инертен (инициируется коллектором)	Активен по умолчанию
Содержит коллектор	Поддерживает множество подписчиков
Коллектор получает все выбросы элементов	Подписчики получают выбросы с момента начала подписки
Потенциально завершается	Не завершается
Выбросы происходят из одной корутины (если не используется конструктор <code>channelFlow</code>)	Выбросы могут происходить из произвольных корутин

Как правило, функции для предоставления сервиса (например, запроса к сети или чтения из базы данных) объявляются с помощью холодных потоков. Другие классы и функции, использующие эти потоки, могут собирать их элементы напрямую или преобразовывать их в потоки состояний и общие потоки, чтобы предоставить эту информацию другим частям системы.

ВЗАИМОДЕЙСТВИЕ С РАЗЛИЧНЫМИ РЕАЛИЗАЦИЯМИ РЕАКТИВНЫХ ПОТОКОВ

Если в проекте уже используется другая абстракция для реактивных потоков, например Project Reactor (<https://projectreactor.io/>), RxJava (<https://reactivex.io/>) или Reactive Streams (<https://www.reactive-streams.org/>), то можно применить встроенные функции преобразования, чтобы конвертировать их представления в потоки Kotlin и наоборот. Это позволяет легко интегрировать потоки в существующий код. Дополнительная информация по этой теме находится в документации (<https://kotlinlang.org/api/kotlinx.coroutines/>).

Сами по себе холодные и горячие потоки уже представляют собой полезную абстракцию для работы со значениями, которые будут вычислены со временем. Однако их истинные возможности раскрываются благодаря *операторам потока*, о которых мы поговорим следующей главе и которые помогают управлять потоками, действуя во многом сродни методам работы с коллекциями и последовательностями.

РЕЗЮМЕ

- В Kotlin потоки — это абстракция на основе корутин, и она дает возможность работать со значениями, которые появятся со временем.
- Различают два типа потоков: горячие и холодные.
- Холодные потоки по умолчанию инертны и связаны с одним коллектором. Создаются с помощью функции-конструктора `flow`. Асинхронное предложение значений можно реализовать с помощью функций `emit`.
- Особый тип холодных потоков — каналные, они позволяют передавать значения из нескольких корутин с помощью функции `send`.
- Горячие потоки всегда активны и связаны с несколькими коллекторами, называемыми подписчиками. Общие потоки и потоки состояния представляют собой экземпляры горячих потоков.
- Общие потоки можно использовать для вещательной передачи значений между корутинами.
- Подписчики общих потоков получают выбросы элементов с момента начала подписки и значения, повторенные общим потоком.
- Потоки состояний можно использовать для управления состояниями в конкурентной системе.
- Потоки состояний выполняют объединение на основе равенства, то есть осуществляют выбросы только при условии, что значение действительно меняется. Если одно значение присваивается несколько раз, выпуска элемента не происходит.
- Холодные потоки можно превратить в горячие с помощью функций `shareIn` и `stateIn`.

17

Операторы потока

В этой главе

- ✓ Операторы для преобразования и работы с потоками.
- ✓ Промежуточные и терминальные операторы.
- ✓ Создание пользовательских операторов потока.

17.1. УПРАВЛЕНИЕ ПОТОКАМИ С ПОМОЩЬЮ ОПЕРАТОРОВ ПОТОКА

В предыдущей главе вы познакомились с потоками как с высокоуровневой абстракцией для работы с несколькими последовательными значениями во времени. Этот процесс реализован с использованием механизма конкурентности Kotlin. В текущей главе мы обсудим, как управлять и трансформировать значения. Kotlin предоставляет огромный выбор операторов для работы с коллекциями (об этом говорилось в главах 5 и 6). Для преобразования потоков тоже можно применять операторы.

Как и в случае с последовательностями, существуют *промежуточные* и *конечные* операторы потока. По аналогии с информацией, представленной в главе 6: промежуточные операторы возвращают другой, измененный поток, не выполняя при этом никакого кода. Терминальные операторы возвращают результат — коллекцию, отдельный элемент из потока, вычисленное значение, — собирая поток и выполняя фактический код (рис. 17.1).

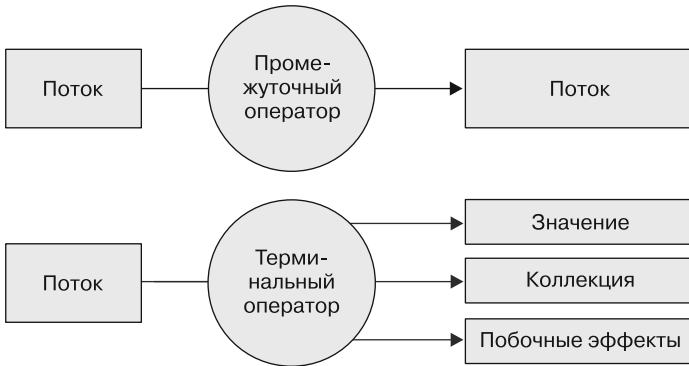


Рис. 17.1. Различия между промежуточными и терминальными операторами потока. Если промежуточные операторы возвращают модифицированный поток, то терминальные возвращают фактический результат. Для этого они выполняют код в потоке

17.2. ПРОМЕЖУТОЧНЫЕ ОПЕРАТОРЫ ПРИМЕНЯЮТСЯ К ВОСХОДЯЩЕМУ ПОТОКУ И ВОЗВРАЩАЮТ НИСХОДЯЩИЙ

Промежуточные операторы применяются к потоку и возвращают сам поток. Чтобы было проще обсуждать эти потоки применительно к операторам, обычно используются названия «*восходящий поток*» и «*нисходящий поток*». Операция производится на восходящем потоке. Нисходящий возвращается промежуточным оператором. В свою очередь, этот нисходящий поток может служить восходящим для следующего оператора и т. д. Схема показана на рис. 17.2. Как и в случае с последовательностями, вызов промежуточного оператора для потока не приводит к выполнению кода — возвращаемый поток остается *холодным*.

ПРИМЕЧАНИЕ

Вы могли задуматься над тем, что происходит в тот момент, когда промежуточный оператор применяется к горячему потоку. Даже в таком случае определенное оператором поведение не будет выполняться до тех пор, пока не будет вызван терминальный оператор типа `collect` для подписки на горячий поток.

Отличная новость заключается в том, что вы уже знаете многие промежуточные операторы для работы с потоками — значительная часть промежуточных операторов из последовательностей применяется к потокам Kotlin. В том числе это самые популярные функции, например `map`, `filter` или `onEach`. Они ведут себя ожидаемо, только оперируют элементами потока, а не последовательности или коллекции. Но есть и операторы потока, с помощью которых можно реализовать особое поведение и функциональность. Их логика выходит за рамки того, что вы уже знаете. Обсудим их более подробно и начнем с оператора `transform`.

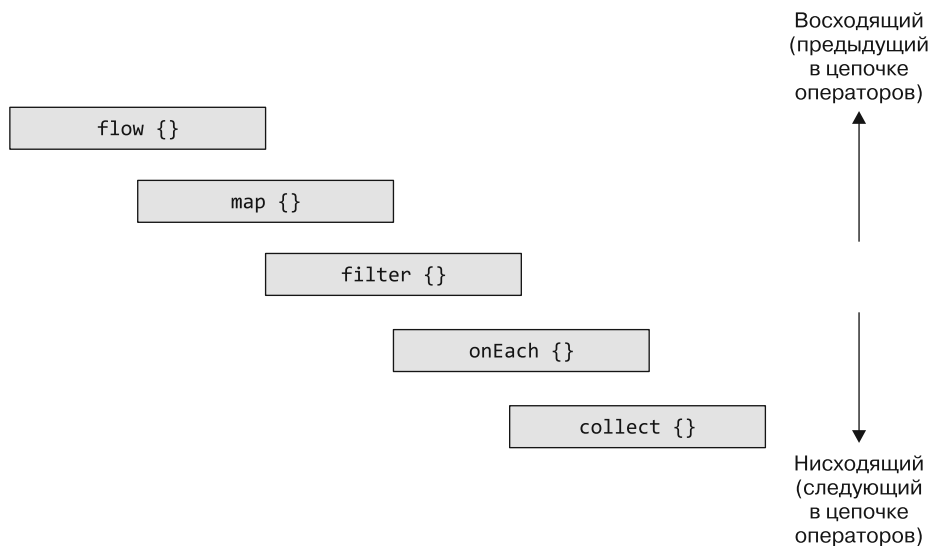


Рис. 17.2. Понятия «восходящий» и «нисходящий» описывают место промежуточного оператора по отношению к другому в их цепочке. Оператор обращается к восходящему потоку, а возвращает нисходящий поток. В данном примере `flow` — восходящий поток оператора `map`, `filter` — нисходящий поток `map`, `onEach` — восходящий поток `collect` и т. д.

17.2.1. Выбросы произвольных значений для каждого элемента восходящего потока: функция `transform`

С помощью функции `map` можно получить восходящий поток, изменить его элементы и вернуть новый нисходящий поток. По сути, для каждого элемента восходящего потока в нисходящем создается новый элемент (листинг 17.1).

Листинг 17.1. Сопоставление потока

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun main() {
    val names = flow {
        emit("Jo")
        emit("May")
        emit("Sue")
    }
    val uppercasedNames = names.map {
        it.uppercase()
    }
    runBlocking {
        uppercasedNames.collect { print("$it ") }
    }
    // JO MAY SUE
}
```

Однако возникают ситуации, когда потребуется производить более одного элемента. Например, выбрасывать не только вариант каждого имени, написанный в верхнем регистре, но и его вариант, полностью состоящий из букв в нижнем регистре. В потоках Kotlin это можно сделать с помощью функции `transform` (листинг 17.2). Она позволяет выбрасывать в нисходящий поток произвольное количество элементов для каждого элемента в восходящем потоке.

Листинг 17.2. Преобразование потока

```
import kotlinx.coroutines.flow.*

fun main() {
    val names = flow {
        emit("Jo")
        emit("May")
        emit("Sue")
    }
    val upperAndLowercasedNames = names.transform {
        emit(it.uppercase())
        emit(it.lowercase())
    }
    runBlocking {
        upperAndLowercasedNames.collect { print("$it ") }
    }
    // JO jo MAY may SUE sue
}
```

В подобных случаях, когда начальный поток просто производит список значений, которые должны быть преобразованы в дальнейшем с помощью операторов потока, можно использовать сокращенную функцию — конструктор потока, которая называется `flowOf`: `val names = flowOf("Jo", "May", "Sue")`.

17.2.2. Семейство операторов `take` может отменить поток

Вы уже познакомились с функциями типа `takewhile` при изучении последовательностей в главе 6. Эти функции можно применять так же, как и к последовательностям. Стоит отметить, что, когда заданные оператором условия больше не действуют, восходящий поток отменяется, то есть больше никаких элементов не производится. Например, при вызове функции `take(5)` в листинге 17.3 для потока, возвращаемого функцией `getTemperatures` (объявлена в главе 16), восходящий поток будет отменен после пяти выбросов.

Листинг 17.3. Функция `take` отменяет восходящий поток после сбора `n` выбросов

```
val temps = getTemperatures()
temps
    .take(5)
    .collect {
        log(it)
    }
}
```

← Отмена потока после пяти выбросов

```
// 37 [main @coroutine#1] 7
// 568 [main @coroutine#1] 9
// 1123 [main @coroutine#1] 2
// 1640 [main @coroutine#1] -6
// 2148 [main @coroutine#1] 7
```

Помимо отмены области видимости корутины коллектора, функция `take` становится еще одним способом контролируемой отмены сбора потока (см. подраздел 16.2.3).

17.2.3. Привязка к фазам потока с помощью функций `onStart`, `onEach`, `onCompletion` и `onEmpty`

Убедиться в том, что поток из листинга 17.3 действительно завершается после сбора пяти элементов, поможет оператор `onCompletion`. С его помощью можно предоставить лямбду, выполняемую после того, как поток завершится, будет отменен или завершится с исключением (об обработке исключений поговорим в главе 18). Доработаем код из листинга 17.3 и уведомим пользователя о нормальном завершении сбора или о завершении с исключением. К последнему состоянию можно обратиться как к параметру для лямбды, что позволит продолжить его обработку:

```
fun main() = runBlocking {
    val temps = getTemperatures()
    temps
        .take(5)
        .onCompletion { cause ->
            if (cause != null) {
                println("An error occurred! $cause")
            } else {
                println("Completed!")
            }
        }
        .collect {
            println(it)
        }
}
```

Оператор `onCompletion` относится к семейству промежуточных. Они нужны для выполнения действий на определенных этапах жизненного цикла потока. Оператор `onStart` выполняется, когда начинается сбор потока, еще до того, как произойдет первый выброс. Оператор `onEach` выполняет действие над каждым выброшенным элементом восходящего потока до того, как он будет передан нисходящему потоку. Особый случай с завершением потока без результата можно обработать с помощью промежуточного оператора `onEmpty`. Он выполняет дополнительную логику или предоставляет элементы по умолчанию.

В листинге 17.4 различные операторы объединяются в функции `process`, чтобы определить поведение начала потока, обработать каждый элемент и завершиться. Вдобавок указывается случай по умолчанию для обработки пустого потока.

Листинг 17.4. Выполнение логики в различных фазах потока

```

flow
    .onEmpty {
        println("Nothing - emitting default value!")
        emit(0)
    }
    .onStart {
        println("Starting!")
    }
    .onEach {
        println("On $it!")
    }
    .onCompletion {
        println("Done!")
    }
    .collect()
}

```

Вызвав эту функцию с пустым и непустым потоком, можно увидеть, что отдельные операторы вызываются в порядке жизненного цикла потока:

```

fun main() {
    runBlocking {
        process(flowOf(1, 2, 3))
        // Starting!
        // On 1!
        // On 2!
        // On 3!
        // Done!
        process(flowOf())
        // Starting!
        // Nothing - emitting default value!
        // On 0!
        // Done!
    }
}

```

Это отличный пример, еще раз показывающий, что эти промежуточные операторы выбрасывают элементы в свой нисходящий поток. Если переместить вызов оператора `onEmpty` дальше по цепочке вызовов (например, поставить прямо перед вызовом функции `collect`), то сообщение `On 0!` не отобразится. Это связано с тем, что `onEach` будет оператором восходящего потока по отношению к `onEmpty` — он не будет получать никаких выбросов оператора нижнего потока.

17.2.4. Буферизация элементов для последующих операторов и коллекторов: оператор `buffer`

Код приложения часто выполняет много тяжелой работы внутри потоков. Элементы потока собираются или обрабатываются с помощью таких операторов, как `onEach`. При этом часто возникает необходимость использовать функции приостановки, а им требуется некоторое время на завершение работы. В примере ниже имитируется

обращение к медленной базе данных, с помощью которого можно получить поток идентификаторов пользователей. Каждый идентификатор пользователя связан с профилем, доступ к которому осуществляется через еще более медленный сетевой ресурс:

```
fun getAllUserIds(): Flow<Int> {
    return flow {
        repeat(3) {
            delay(200.milliseconds) // Задержка базы данных
            log("Emitting!")
            emit(it)
        }
    }
}

suspend fun getProfileFromNetwork(id: Int): String {
    delay(2.seconds) // Сетевая задержка
    return "Profile[$id]"
}
```

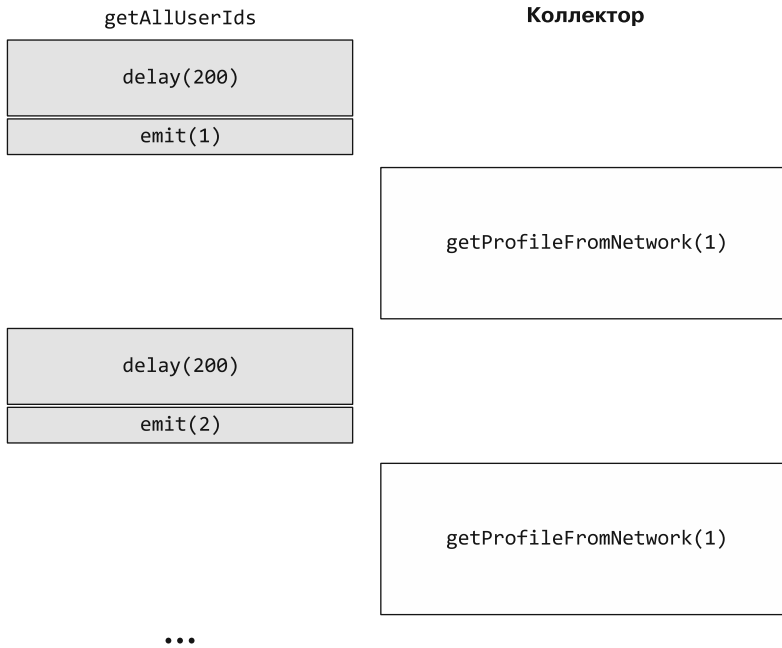
При работе с холодным потоком выполнение производителя значений приостанавливается до тех пор, пока сборщик не закончит обработку предыдущего элемента. Это можно наблюдать, вызвав функцию `getUserIds`, а затем `getProfileFromNetwork` для каждого элемента:

```
fun main() {
    val ids = getAllUserIds()
    runBlocking {
        ids
            .map { getProfileFromNetwork(it) }
            .collect { log("Got $it") }
    }
}

// 310 [main @coroutine#1] Emitting!
// 2402 [main @coroutine#1] Got Profile[0]
// 2661 [main @coroutine#1] Emitting!
// 4732 [main @coroutine#1] Got Profile[1]
// 5007 [main @coroutine#1] Emitting!
// 7048 [main @coroutine#1] Got Profile[2]
```

Как видите, выброс идентификаторов и запрос профилей взаимосвязаны. При выпуске элемента код производителя не продолжает работу до тех пор, пока нисходящий поток не закончит обработку элемента. В данной реализации это означает, что обработка отдельного элемента занимает примерно 1,2 секунды. Процесс показан на рис. 17.3.

Даже не прибегая к более сложным оптимизациям, таким как порождение новых корутин для обработки каждого элемента в потоке, можно сделать еще одно наблюдение: если бы производитель мог выбрасывать элементы, не ожидая, пока они будут обработаны коллектором, выполнение потока можно было бы ускорить. Именно это и позволяет осуществить оператор `buffer`. Он представляет собой буфер для хранения элементов, даже если нисходящий поток все еще занят обработкой



...

Рис. 17.3. Когда поток `getAllUserIds` выбрасывает элемент, код производителя не продолжает работу, пока нисходящий поток не закончит обработку элемента

ранее переданных элементов. Таким образом вы получаете возможность эффективно отделить друг от друга части цепочки операторов обработки потока. Добавив буфер емкостью три элемента, производитель может продолжать создавать новые идентификаторы пользователей и помещать их в буфер до тех пор, пока коллектор не будет готов к дальнейшим сетевым запросам:

```
fun main() {
    val ids = getAllUserIds()
    runBlocking {
        ids
            .buffer(3)
            .map { getProfileFromNetwork(it) }
            .collect { log("Got $it") }
    }
}

// 304 [main @coroutine#2] Emitting!
// 525 [main @coroutine#2] Emitting!
// 796 [main @coroutine#2] Emitting!
// 2373 [main @coroutine#1] Got Profile[0]
// 4388 [main @coroutine#1] Got Profile[1]
// 6461 [main @coroutine#1] Got Profile[2]
```

Сравнивая время выполнения двух реализаций, следует отметить, что добавление буфера сократило время выполнения. Это происходит потому, что поток,

возвращаемый функцией `getAllUserIds`, может продолжать выбрасывать элементы в буфер, пока работает коллектор. Этот процесс показан на рис. 17.4.

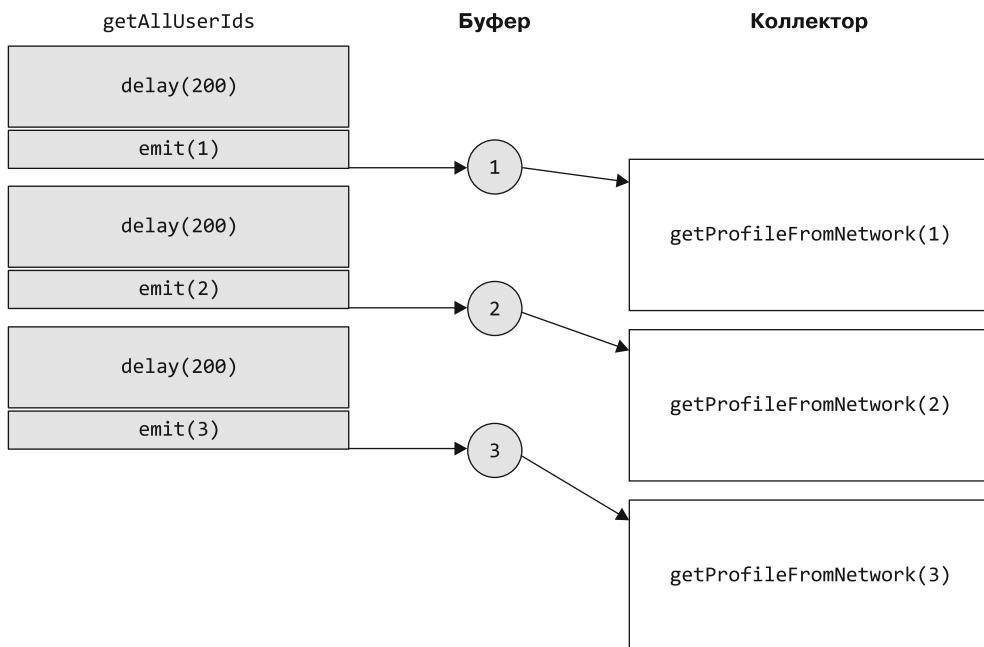


Рис. 17.4. Оператор `buffer` отделяет выполнение восходящего потока от нисходящего: поток, возвращаемый функцией `getAllUserIds`, может продолжать выдавать элементы в буфер. Коллектор использует полученные идентификаторы для вызова `getProfileFromNetwork`. Он может продолжать делать это по мере поступления элементов, работая в собственном темпе

Буферы в цепочке операторов помогут увеличить пропускную способность системы, особенно когда время на производство и обработку элементов потока постоянно меняется. Например, можно считывать и обрабатывать входные данные разного размера и сложности. К тому же оператор `buffer` легко настроить. Кроме параметра размера — `size`, у него есть параметр `onBufferOverflow`. Он определяет, что необходимо предпринять, когда емкость буфера исчерпана: приостановить работу производителя (`SUSPEND`), отбросить самое старое значение в буфере, не приостанавливая работу (`DROP_OLDEST`), или отбросить последнее добавленное значение (`DROP_LATEST`).

17.2.5. Отбрасывание промежуточных значений: оператор `conflate`

Еще один способ обеспечить беспрепятственную работу производителя значений — просто отбрасывать все элементы, производимые до тех пор, пока коллектор занят. В потоках Kotlin это можно сделать с помощью оператора `conflate`. В листинге 17.5 функция `conflate` применяется совместно с функцией `getTemperatures`, предназначенной для считывания температуры с интервалом 500 миллисекунд

(см. главу 16), чтобы собирать элементы потока каждую секунду. В результате выполнения видно, что поток перед функцией `conflate` содержит все элементы из `getTemperatures`, а промежуточные элементы отбрасываются в коллектор нисходящего потока.

Листинг 17.5. Использование функции `conflate` для игнорирования промежуточных значений

```
runBlocking {
    val temps = getTemperatures()
    temps
        .onEach {
            log("Read $it from sensor")
        }
        .conflate()
        .collect {
            log("Collected $it")
            delay(1.seconds)
        }
    }
}
```

```
// 43 [main @coroutine#2] Read 20 from sensor
// 51 [main @coroutine#1] Collected 20
// 558 [main @coroutine#2] Read -10 from sensor
// 1078 [main @coroutine#2] Read 3 from sensor
// 1294 [main @coroutine#1] Collected 3
// 1579 [main @coroutine#2] Read 13 from sensor
// 2153 [main @coroutine#2] Read 26 from sensor
// 2556 [main @coroutine#1] Collected 26
```

Применение функции `conflate` аналогично `buffer` отделяет выполнение восходящего потока от выполнения любых последующих операторов. Бывают ситуации, когда значения потока быстро устаревают и заменяются другими производимыми элементами. В этом случае объединение поможет медленному сборщику не отставать — он будет обрабатывать только последние элементы в потоке.

17.2.6. Фильтрация значений по тайм-ауту: оператор `debounce`

В некоторых сценариях бывает полезно подождать некоторое время, прежде чем обрабатывать значения в потоке. Во фрагменте кода ниже имитируется ввод пользователем поискового запроса в поле. При каждом добавлении нового символа выдается все более длинная строка. Поскольку пользователи не набирают текст мгновенно, а тратят некоторое время на нажатие кнопок, необходимо добавить вызовы функции `delay`, чтобы имитировать ввод текста в течение определенного времени:

```
val searchQuery = flow {
    emit("K")
    delay(100.milliseconds)
}
```

```

    emit("Ko")
    delay(200.milliseconds)
    emit("Kotl")
    delay(500.milliseconds)
    emit("Kotlin")
}

```

Многие приложения стремятся обеспечить «функцию мгновенного поиска», когда пользователь может набрать текст и увидеть результаты, не нажимая дополнительную кнопку «Найти». Если собирать этот поток, никак его не обрабатывая в дальнейшем, то для каждого нажатия клавиши будет запускаться новый поисковый запрос.

Часто используемая стратегия — подождать некоторое время, прежде чем приступить к поиску. Пока пользователь набирает запрос, лишние поисковые запросы не делаются. А как только он перестает набирать текст на некоторое время, результаты отображаются, при этом пользователю не приходится выполнять какие-либо дополнительные действия. Такое поведение легко реализовать в корутинах Kotlin с помощью оператора `debounce`. Он выдает элементы в нисходящий поток только по истечении определенного тайм-аута, когда восходящий поток не выдал ни одного элемента. Благодаря отложенному выполнению с тайм-аутом 250 миллисекунд перед сбором потока `searchQuery` в нисходящий поток попадают и собираются только элементы, которым предшествовала пауза в четверть секунды:

```

fun main() = runBlocking {
    searchQuery
        .debounce(250.milliseconds)
        .collect {
            log("Searching for $it")
        }
}

// 644 [main @coroutine#1] Searching for Kotl
// 876 [main @coroutine#1] Searching for Kotlin

```

17.2.7. Переключение контекста корутины, для которой выполняется поток: оператор `flowOn`

Если операторы потока используют блокирующий ввод-вывод или должны работать с потоком пользовательского интерфейса, то применяется логика работы с обычными корутинами: контекст корутины решает, где будет выполняться логика потока. По умолчанию процесс сбора происходит в контексте вызова функции `collect`. Но с помощью потоков можно создавать довольно сложные конвейеры обработки данных. Может потребоваться сделать так, чтобы некоторые части конвейера обработки выполнялись на другом диспетчере или с другим контекстом корутины. Для этого применяется оператор `flowOn`. Подобно функции `withContext`, описанной в главе 14, он настраивает контекст корутины. В листинге 17.6 он используется для переключения между диспетчером `runBlocking`, диспетчером по умолчанию и диспетчером ввода-вывода.

Листинг 17.6. Переключение диспетчеров с помощью оператора `flowOn`

```

runBlocking {
    flowOf(1)
        .onEach { log("A") }
        .flowOn(Dispatchers.Default)
        .onEach { log("B") }
        .flowOn(Dispatchers.IO)
        .onEach { log("C") }
        .collect()
}

// 36 [DefaultDispatcher-worker-3 @coroutine#3] A
// 44 [DefaultDispatcher-worker-1 @coroutine#2] B
// 44 [main @coroutine#1] C

```

Важно отметить, что оператор `flowOn` влияет только на диспетчер восходящего потока, то есть на поток (и любые промежуточные операторы) до вызова `flowOn`. Нисходящий остается нетронутым, поэтому оператор иначе называют *сохраняющим контекст*. Переключение диспетчера на `Dispatchers.Default` и на `Dispatchers.IO` влияет только на элементы A и B соответственно, а на C предыдущие вызовы оператора `flowOn` не оказывают никакого влияния.

17.3. СОЗДАНИЕ ПОЛЬЗОВАТЕЛЬСКИХ ПРОМЕЖУТОЧНЫХ ОПЕРАТОРОВ

В библиотеку корутин Kotlin входит большой набор операторов для управления потоками. Но как эти операторы работают внутри и как создать собственные промежуточные операторы?

Как правило, промежуточный оператор действует как сборщик и производитель одновременно: он собирает элементы из своего восходящего потока, выполняет преобразования, реализует побочные эффекты или другое пользовательское поведение, а затем выбрасывает новые элементы в нисходящий поток. Мы уже рассмотрели все отдельные части процесса: сбор элементов из восходящего потока осуществляется с помощью функции `collect`, а создать новый нисходящий поток можно благодаря функции-конструктору `flow`. Функция `collect` вызывается для восходящего потока только из конструктора `flow`, поэтому оператор гарантированно остается холодным. Только когда другая функция вызовет `collect` для потока, возвращенного вашим оператором, будет запущен сбор элементов восходящего потока.

Например, можно реализовать оператор для вычисления среднего значения последних `n` элементов в потоке значений типа `double`. Для этого оператор будет внутренне отслеживать список встретившихся чисел, а затем выдавать среднее из найденных значений:

```

fun Flow<Double>.averageOfLast(n: Int): Flow<Double> =
    flow {
        val numbers = mutableListOf<Double>()

```

```

collect {
    if (numbers.size >= n) {
        numbers.removeFirst()
    }
    numbers.add(it)
    emit(numbers.average())
}
}

```

Как и любой другой промежуточный оператор, этот можно вызвать для восходящего потока и собрать элементы его нисходящего потока:

```

fun main() = runBlocking {
    flowOf(1.0, 2.0, 30.0, 121.0)
        .averageOfLast(3)
        .collect {
            print("$it ")
        }
}

// 1.0 1.5 11.0 51.0

```

Большинство стандартных операторов в библиотеке корутин Kotlin работают по схожему принципу, хотя часто содержат дополнительный код, позволяющий оптимизировать производительность. Данный код помогает нескольким цепочкам операторов работать более эффективно. Это всего лишь деталь реализации — эти оптимизации происходят полностью внутри системы и не меняют поведение вашего кода.

17.4. ТЕРМИНАЛЬНЫЕ ОПЕРАТОРЫ ВЫПОЛНЯЮТ ВОСХОДЯЩИЙ ПОТОК И МОГУТ ВЫЧИСЛЯТЬ ЗНАЧЕНИЕ

Промежуточные операторы преобразуют заданный поток в другой, но сами не выполняют никакой код. Это делают *терминальные операторы*: они либо вычисляют одно значение или набор значений, либо просто запускают выполнение потока, выполняя вычисления и побочные эффекты согласно указаниям разработчика. Самый распространенный терминальный оператор — `collect`. Он позволяет указать лямбду для выполнения с каждым элементом потока. Познакомившись с доступными промежуточными операторами, вы поймете, что этот код эквивалентен вызову оператора `onEach` перед вызовом функции `collect` без каких-либо параметров:

```

fun main() = runBlocking {
    getTemperatures()
        .onEach {
            log(it)
        }
        .collect()
}

```

Терминальные операторы отвечают за выполнение восходящих потоков, поэтому постоянно приостанавливают выполнение функций. Вызов функции `collect` всегда приостанавливается до тех пор, пока весь поток не будет собран (или пока не будет отменена область видимости корутины, в которой вызывается `collect`). Другие терминальные операторы, такие как `first` или `firstOrNull`, могут отменить восходящий поток после того, как элемент будет получен. Аналогичные процессы мы наблюдали у промежуточных операторов, описываемых в подразделе 17.2.2. Например, если требуются только отдельные показания датчика температуры, то можно использовать оператор `first` или его родственную функцию `firstOrNull`, с помощью которой можно получить одно значение:

```
fun main() = runBlocking {
    getTemperatures()
        .first()
}
```

17.4.1. Фреймворки предоставляют пользовательские операторы

Некоторые фреймворки в экосистеме Kotlin обеспечивают прямую интеграцию с потоками и предоставляют пользовательские операторы и функции преобразования. Например, Android UI framework Jetpack Compose компании Google и его родственный проект Compose Multiplatform компании JetBrains позволяют преобразовывать потоки Kotlin в объекты `State`. Они используются механизмами UI framework для запуска рекомпозиции пользовательского интерфейса, перерисовывая изменившиеся элементы. Во фрагменте кода ниже функция `collectAsState` преобразования потока целых чисел, переданных в качестве аргумента, используется для создания объекта состояния. Как и в случае с потоками состояний, объекты `State` из проекта Compose ожидают начального значения. Его можно установить в `null`, чтобы указать на отсутствие значения. Затем с помощью переменной `temperature` в DSL Compose описывается компонент пользовательского интерфейса. В этом случае текст просто помещается в рамку:

```
@Composable fun TemperatureDisplay(temps: Flow<Int>) {
    val temperature = temps.collectAsState(null)
    Box {
        temperature.value?.let {
            Text("The current temperature is $it!")
        }
    }
}
```

Потоки — мощное дополнение к вашему набору инструментов на основе корутин. Вы узнали, как с их помощью писать аккуратный код, позволяющий обрабатывать значения с течением времени. В следующей главе мы покажем, как распространяются и обрабатываются ошибки в приложениях на основе корутин, и более подробно обсудим возможности тестирования конкурентного кода.

РЕЗЮМЕ

- Промежуточные операторы используются для преобразования одних потоков в другие. Они работают с *восходящим* потоком и возвращают *нисходящий*. Промежуточные операторы относятся к холодным — не выполняются до тех пор, пока не будет вызван терминальный оператор.
- Значительную часть промежуточных операторов последовательностей можно использовать непосредственно с потоками. Вдобавок потоки предоставляют дополнительные промежуточные операторы, с помощью которых можно выполнять преобразования (`transform`), управлять контекстом выполнения потока (`flowOn`) и запускать код на определенных этапах во время выполнения потока (`onStart`, `onCompletion` и т. д.).
- Терминальные операторы, такие как `collect`, выполняют код потока или, в случае горячих потоков, обеспечивают подписку на поток.
- Можно создавать собственные операторы, собирая поток в другом конструкторе потоков и выбрасывая из него преобразованные элементы.
- Несколько сторонних фреймворков, например Jetpack Compose и Compose Multiplatform, обеспечивают прямую интеграцию с потоками Kotlin.

18

Обработка ошибок и тестирование

В этой главе

- ✓ Контроль поведения вашего кода при возникновении ошибок и исключений.
- ✓ Как обработка ошибок связана с концепцией структурированной конкурентности.
- ✓ Написание кода, имеющего корректное поведение при сбоях в работе частей системы.
- ✓ Написание модульных тестов для конкурентного кода Kotlin.
- ✓ Использование специализированного тестового диспетчера для ускорения выполнения тестов и тестирования ограничений конкурентности.
- ✓ Тестирование потоков с помощью библиотеки Turbine.

Прочитав последние несколько глав, вы получили представление о различных аспектах, связанных с написанием конкурентного кода с помощью корутин Kotlin. Чтобы обеспечить надежность приложений, необходимо учесть еще один вопрос: «Как ведет себя код, когда что-то идет не так?»

Работа с конкурентными приложениями — сложная по своей сути задача. Весьма вероятно, что в вашей системе будет много взаимодействующих друг с другом подвижных частей. Кроме того, приложение, скорее всего, взаимодействует и с другими системами, находящимися за пределами вашего контроля. Эти системы могут

выйти из строя, или соединение с ними может быть ненадежным. Однако даже в таких ситуациях приложение должно работать хорошо. Правильная обработка ошибок — один из ключевых аспектов, позволяющих обеспечивать корректную работу приложения. Необходимо внедрять в него соответствующие механизмы эффективного решения проблем.

При работе с корутинами Kotlin тема обработки ошибок тесно переплетается с концепцией структурированной конкурентности (см. главу 15). В первой половине текущей главы вы узнаете, как перехваченные исключения обрабатываются и распространяются по иерархии корутин, как обрабатываются ошибки в потоках и какие инструменты для управления этим поведением вам доступны.

Еще один аспект создания надежного программного обеспечения — тестирование. Существует множество книг, в которых описываются стратегии и подходы к тестированию кода общего применения. Но во второй половине этой главы мы предлагаем изучить, как именно тестировать код, написанный на основе корутин Kotlin. Вы узнаете о том, как запускать тесты с виртуальным временем, и о способах, облегчающих написание тестов для потоков с помощью библиотеки Turbine. К концу главы вы будете лучше понимать, как работает распространение ошибок, как бороться с ошибками в потоках и корутинах, а также какова механика написания тестов для конкурентного кода.

18.1. ОБРАБОТКА ОШИБОК, ВОЗНИКАЮЩИХ ВНУТРИ КОРУТИН

Код внутри приостанавливающих функций или конструктора корутин, как и любой другой, может выбросить исключение. Чтобы справиться с такими исключениями, вы можете захотеть окружить вызов функции `launch` или `async` блоком `try-catch`. Однако это не сработает. Помните, что это *функции-конструкторы корутин* — они создают новые корутины в коде. Исключения в этих новых корутинах не будут перехвачены блоком `catch` (точно так же, как, например, исключения в только что созданном потоке не будут перехвачены кодом для создания потока). Поэтому такой код, как этот, где исключение `UnsupportedOperationException` выбрасывается из конструктора `launch`, не будет перехватывать исключение:

```
import kotlinx.coroutines.*

fun main(): Unit = runBlocking {
    try {
        launch {
            throw UnsupportedOperationException("Ouch!")
        }
    } catch (u: UnsupportedOperationException) {
        println("Handled $u")  ← Не вызывается
    }
}

// Exception in thread "main" java.lang.UnsupportedOperationException: Ouch!
// at MyExampleKt$main$1$1.invokeSuspend(MyExample.kt:6)
// ...
```

Один из способов правильно перехватить это исключение — переместить блок `try-catch` внутрь блока, связанного с конструктором `launch`. В таком случае исключение не пересекает границы корутины, и его можно было бы обработать, как если бы корутин не было вовсе (в подразделе 18.2.3 и разделе 18.3 мы обсудим, как неперехваченные исключения из корутин можно обработать автоматически, не вызывая аварийного завершения работы всего приложения):

```
import kotlinx.coroutines.*

fun main(): Unit = runBlocking {
    launch {
        try {
            throw UnsupportedOperationException("Ouch!")
        } catch (u: UnsupportedOperationException) {
            println("Handled $u")
        }
    }
}

// Handled java.lang.UnsupportedOperationException: Ouch!
```

Если корутина, созданная с помощью функции `async`, выбрасывает исключение, то вызов `await` для ее результата выбросит это же исключение. Так происходит потому, что функция `await` не может вернуть информативное значение ожидаемого типа и, следовательно, должна выбросить исключение. Этот процесс показан в листинге 18.1. Изначально ожидалось, что функция `myDeferredInt.await()` вернет целочисленное значение. Но поскольку внутри корутины `async` для вычисления этого значения было выброшено исключение, то функция `await()` пробрасывает это исключение. Чтобы его перехватить, окружите функцию `await()` блоком `try-catch`.

Листинг 18.1. Асинхронная корутина, которая выбрасывает исключение

```
import kotlinx.coroutines.*

fun main(): Unit = runBlocking {
    val myDeferredInt: Deferred<Int> = async {
        throw UnsupportedOperationException("Ouch!")
    }
    try {
        val i: Int = myDeferredInt.await()
        println(i)
    } catch (u: UnsupportedOperationException) {
        println("Handled: $u")
    }
}
```

При выполнении этого примера можно заметить, что исключение было перехвачено с помощью блока `try-catch`, окружающего функцию `await()`. Но любопытно, что одновременно с этим оно было выведено в консоль ошибок:

```
Handled: java.lang.UnsupportedOperationException: Ouch!
Exception in thread "main" java.lang.UnsupportedOperationException: Ouch!
    at MyExampleKt$main$1$myDeferred$1.invokeSuspend(MyExample.kt:6)
    ...
```

Это происходит потому, что `await` *пробрасывает* исключение, но можно наблюдать и исходное исключение. В данном примере функция `async` передает исключение своей родительской корутине, созданной с помощью конструктора `runBlocking`. А та выводит исключение в консоль ошибок и завершает работу программы.

Дочерняя корутина всегда *передает* неперехваченное исключение своему родителю. Это означает, что ответственность за обработку данного исключения ложится на родителя. Как и в реальной жизни, когда дети передают свои проблемы родителям, если не знают, как справиться с ними самостоятельно (или если так просто удобнее). В следующих разделах распространение ошибок описано более подробно, а также рассказывается о различных способах обработки родительскими корутинами неперехваченных ошибок, переданных им потомками.

18.2. РАСПРОСТРАНЕНИЕ ОШИБОК В КОРУТИНАХ KOTLIN

Впервые знакомя вас со структурированной конкурентностью в главе 15, мы вскользь упомянули, что, помимо отмены, одной из основных обязанностей этой концепции стала обработка ошибок. Парадигма структурированной конкурентности влияет на процессы в родительской корутине, происходящие в тот момент, когда дочерняя выбрасывает неперехваченное исключение. Существует два концептуальных способа разделения работы между потомками и, соответственно, два способа обработки их ошибок. Ключевое различие заключается в том, что сбой одного потомка должен повлечь за собой сбой родителя.

- Если корутины используются для *конкурентной декомпозиции работы*, то отказ одного из дочерних процессов означает, что получить конечный результат уже невозможно. Родительская корутина тоже должна завершиться с исключением. А остальные запущенные дочерние программы будут отменены, чтобы избежать получения результатов, которые больше не нужны. Неудача одного потомка приводит к неудаче родителей.
- Второй случай — когда сбой одного потомка не приводит к общему сбою. Родитель *контролирует* процесс выполнения своих потомков, устраняя сбой одного из них, и это не приводит к краху всей системы. Как правило, такие контролирующие корутины находятся на вершине иерархии. Например, серверный процесс может запустить несколько дочерних работ и должен контролировать их выполнение. Или компонент пользовательского интерфейса должен оставаться активным, даже если получение последних данных может завершиться неудачей. В этом сценарии сбой одного потомка не приведет к сбою родителя.

В корутинах Kotlin способ обработки дочерних корутин родительской зависит от того, есть ли в контексте корутины последней обычный объект `Job` (отказ дочерней приводит к общему отказу) или объект `SupervisorJob` (родитель контролирует потомков). Далее мы рассмотрим эту разницу более подробно.

18.2.1. Корутины отменяют всех своих потомков, когда в одном из них возникает сбой

Обсуждая контекст корутин в разделе 14.8, мы выяснили, что внутри системы иерархия «родитель — ребенок» между корутинами создается с помощью объектов `Job`. Таким образом, если корутина не была создана с помощью типа `SupervisorJob`, чтобы стать супервизором, то по умолчанию перехваченное исключение из дочерних корутин будет обрабатываться как завершение родительской с исключением.

Дочерняя корутина передает свой сбой родительской, а та выполняет следующие действия:

- отменяет всех своих других потомков, чтобы избежать лишней работы;
- завершает собственное выполнение с тем же исключением;
- распространяет исключение дальше по иерархии.

Этот процесс показан на рис. 18.1.

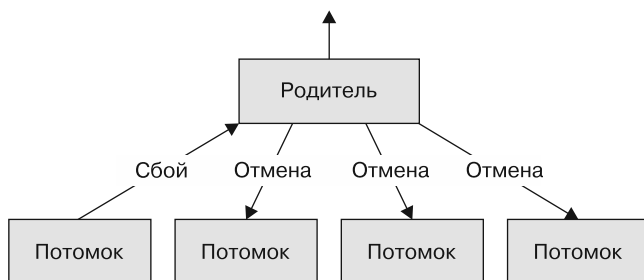


Рис. 18.1. Когда дочерние корутины терпят неудачу из-за перехваченного исключения, они уведомляют об этом своего родителя. Родитель, в свою очередь, отменяет все дочерние корутин и распространяет исключение дальше по иерархии корутин

Это поведение отличается от аналогичных примитивов в других языках (например, его нет у корутин в `Go`). Обычно такая удобная отмена других родственных задач при отказе одной не предусмотрена по умолчанию, и ее часто реализуют вручную. Но это одно из главных преимуществ корутин `Kotlin`.

Такое поведение очень полезно для корутин в одной области видимости, которые совместно выполняют конкурентные вычисления и возвращают общий результат. Когда одна из корутин в области видимости терпит неудачу из-за перехваченного исключения — то есть сталкивается с нерешаемой самостоятельно проблемой, — предполагается, что общий результат больше нельзя адекватно вычислить. Дочерние корутин отменяются, чтобы предотвратить продолжение бесполезных вычислений или удержание ресурсов в такой ситуации. Это позволяет избежать ненужной работы и высвободить ресурсы.

В листинге 18.2 создаются две корутины. Первая действует как корутина `heartbeat` — просто закидывается и выводит на экран сообщение с определенным интервалом. Если возникает исключение, то она завершается, выводя исключение и выбрасывая его повторно. Вторая корутина выбрасывает необрабатываемое исключение через 1 секунду.

Листинг 18.2. Запуск корутины `heartbeat` и корутины для выброса исключения

```
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.milliseconds
import kotlin.time.Duration.Companion.seconds

fun main(): Unit = runBlocking {
    launch {
        try {
            while (true) {
                println("Heartbeat!")
                delay(500.milliseconds)
            }
        } catch (e: Exception) {
            println("Heartbeat terminated: $e")
            throw e
        }
    }
    launch {
        delay(1.seconds)
        throw UnsupportedOperationException("Ow!")
    }
}
```

В результате выполнения видно, что в тот момент, когда родственная корутина выбрасывает исключение, отменяется и корутина `heartbeat`:

```
Heartbeat!
Heartbeat!
Heartbeat terminated: kotlinx.coroutines.JobCancellationException: Parent job
    is Cancelling; job=BlockingCoroutine{Cancelling}@1517365b
Exception in thread "main" java.lang.UnsupportedOperationException: Ow!
```

По умолчанию все создатели корутин, в том числе конструктор `runBlocking` в этом примере, создают обычную, неконтролирующую корутину. Вот почему при завершении одной корутины с неперехваченным исключением отменяются и другие дочерние корутины.

Такое поведение распространения ошибок влияет на все корутины, а не только на запущенные с помощью функции `launch`. Новая дочерняя корутина, запущенная с помощью функции `async`, ведет себя так же — передает свое неперехваченное исключение родителю. Замена функции `launch` на `async` в этом примере приводит к отмене и родственной корутины.

18.2.2. Структурированная конкурентность влияет только на исключения, выброшенные через границы корутин

Поведение с отменой родственных корутин и распространением исключения вверх по иерархии корутин влияет только на nepřехваченные исключения, пересекающие границы корутин. Поэтому самый простой способ не сталкиваться с таким поведением — вообще не выбрасывать исключения через эти границы. Блок `try-catch` в рамках одной корутины (например, внутри тела функции приостановки) будет вести себя прогнозируемо. Можно переписать листинг 18.2, чтобы перехватить само исключение `UnsupportedOperationException`. Переписанный вариант представлен в листинге 18.3.

Листинг 18.3. Перехват исключения внутри корутины

```
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.milliseconds
import kotlin.time.Duration.Companion.seconds

fun main(): Unit = runBlocking {
    launch {
        try {
            while (true) {
                println("Heartbeat!")
                delay(500.milliseconds)
            }
        } catch (e: Exception) {
            println("Heartbeat terminated: $e")
            throw e
        }
    }
    launch {
        try {
            delay(1.seconds)
            throw UnsupportedOperationException("Ow!")
        } catch (u: UnsupportedOperationException) {
            println("Caught $u")
        }
    }
}
```

Благодаря этому изменению стало заметно, что корутина `heartbeat` продолжает выводить на экран свой текст даже после того, как было выброшено исключение:

```
Heartbeat!
Heartbeat!
Caught java.lang.UnsupportedOperationException: Ow!
Heartbeat!
Heartbeat!
...
```

ПРИМЕЧАНИЕ

Стоит еще раз напомнить о том, чему вы научились в подразделе 15.2.4: помните об исключениях отмены и их супертипах, когда перехватываете исключения в корутинах. Отмены относятся к естественной части жизненного цикла корутин, так что код не должен проглотить эти исключения. Вместо этого их нужно либо вообще не ловить, либо пробрасывать.

Распространение не перехваченных исключений вверх по иерархии и сопутствующая этому отмена дочерних корутин помогает внедрить парадигму структурированной конкурентности в приложении. Но, конечно, одно перехваченное исключение не должно разрушить все ваше приложение. Вы должны иметь возможность определять границы распространения ошибок. В корутинах Kotlin это можно сделать с помощью супервизоров.

18.2.3. Супервизоры не допускают отмены родительских и родственных корутин

В отличие от областей видимости с обычными объектами работ супервизоры не выходят из строя, если некоторые из их потомков сообщают о сбое (рис. 18.2). Они не отменяют другие дочерние корутины и не распространяют исключение дальше по структурированной иерархии конкурентности. По этой причине супервизоры часто выступают в роли корневой корутины — самой верхней в иерархии.

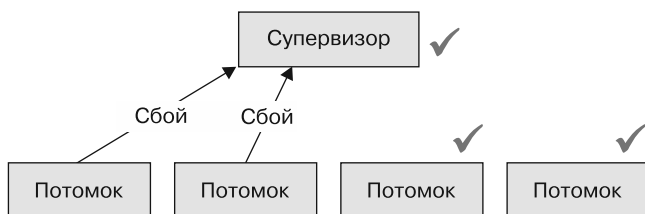


Рис. 18.2. Когда одна или несколько дочерних корутин супервизора терпят неудачу, все дочерние и родительские корутины продолжают работать. Исключение не будет распространяться дальше

Чтобы корутина выступала в роли супервизора для своих дочерних элементов, ее работа должна быть типа `SupervisorJob`, а не обычным объектом `Job`. Она выполняет ту же роль, но предотвращает передачу исключений родителям и в качестве ответной реакции не отменяет другие дочерние работы. Конечно, супервизоры все еще участвуют в структурированной конкурентности — их можно отменить, и исключения отмены будут корректно распространяться.

Чтобы увидеть поведение супервизора, можно создать область видимости с помощью функции `supervisorScope`. Она аналогична функции `coroutineScope`, описанной в подразделе 15.1.1, но имеет ключевое отличие: когда одна из ее дочерних корутин терпит неудачу, ни одна из других дочерних корутин не завершается и перехваченное исключение не распространяется дальше. А родительская и дочерние корутины

продолжают работать. Чтобы исправить листинг 18.2 и сохранить работу корутины `heartbeat`, можно обернуть вызовы функции `launch` в вызов `supervisorScope` (листинг 18.4).

Листинг 18.4. Использование области видимости супервизора для предотвращения дальнейшего распространения исключения

```
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.milliseconds
import kotlin.time.Duration.Companion.seconds

fun main(): Unit = runBlocking {
    supervisorScope {
        launch {
            try {
                while (true) {
                    println("Heartbeat!")
                    delay(500.milliseconds)
                }
            } catch (e: Exception) {
                println("Heartbeat terminated: $e")
                throw e
            }
        }
        launch {
            delay(1.seconds)
            throw UnsupportedOperationException("Ow!")
        }
    }
}
```

При выполнении этого кода видно, что корутина `heartbeat` продолжает работать даже после того, как было выброшено исключение. Супервизор эффективно предотвратил отмену родительской корутины дочерней (помимо кода ниже, показано и на рис. 18.2, см. выше):

```
Heartbeat!
Heartbeat!
Exception in thread "main" java.lang.UnsupportedOperationException: Ow!
...
Heartbeat!
Heartbeat!
...
```

Почему же отображается исключение, хотя приложение продолжает работать? Это происходит потому, что `SupervisorJob` вызывает обработчик `CoroutineExceptionHandler` для дочерних корутин, запущенных с помощью конструктора `launch` (эта тема более подробно описана в разделе 18.3).

Фреймворки с поддержкой корутин часто сразу предоставляют области видимости корутин, выступающие в роли супервизоров. Например, область видимости `Application` фреймворка `Ktor` позволяет использовать объект `Application` для запуска корутин, работающих дольше, чем длится время жизни отдельного обработчика

запроса. Такие корутины могут существовать столько же, сколько и все приложение Ktor. Кроме того, область видимости объекта `Application` выступает в роли супервизора, поскольку одно неперехваченное исключение в корутине не должно разрушить все приложение. Сравните этот подход с классом `PipelineContext` в Ktor, который отвечает за согласование времени существования корутин с обработчиком запросов. Предполагается, что несколько корутин в `PipelineContext` работают совместно для вычисления результата (ответа на полученный запрос), поэтому отказ корутины с неперехваченным исключением означает, что никакой разумный результат больше нельзя вычислить — другие корутины, связанные с этим конкретным запросом, отменяются.

Супервизоры, как правило, располагаются выше в иерархии корутин приложения. С их помощью корутины часто связывают с долгоживущими частями программного обеспечения. Например, с общим временем жизни приложения или временем, в течение которого окно или представление видно пользователю. Более узконаправленные функции обычно не используют супервизоры — все-таки отмену ненужной работы можно считать желательным свойством распространения ошибок в корутинах.

18.3. ПОСЛЕДНЕЕ СРЕДСТВО ДЛЯ ОБРАБОТКИ ИСКЛЮЧЕНИЙ — `COROUTINEEXCEPTIONHANDLER`

Дочерняя корутина передает неперехваченное исключение своему родителю до тех пор, пока исключение не встретит супервизор. Или пока оно не достигнет вершины иерархии — корневой корутины без родителя. В этот момент неперехваченное исключение попадает в специальный обработчик `CoroutineExceptionHandler`. Он представляет собой часть контекста корутины. Если в контексте нет обработчика исключений корутины, то неперехваченное исключение переходит к общесистемному обработчику исключений.

ПРИМЕЧАНИЕ

Общесистемные обработчики исключений в чистых JVM-проектах и Android-проектах различаются. В чистом JVM-проекте обработчик просто выводит трассировку стека исключений в консоль ошибок (в соответствии с интерфейсом `UncaughtExceptionHandler` JVM). В Android общесистемный обработчик исключений выводит ваше приложение из строя.

Поведение обработки неперехваченных исключений можно настроить, предоставив обработчик `CoroutineExceptionHandler` в контексте корутины. Фреймворки Kotlin могут самостоятельно предоставлять обработчик исключений корутин. Например, серверный фреймворк Ktor использует обработчик `CoroutineExceptionHandler` для отправки строкового представления неперехваченных исключений поставщику журналирования. С другой стороны, класс `ViewModel` в Android не определяет `CoroutineExceptionHandler` в контексте типа `SupervisorJob`, связанного с областью

видимости `viewModelScope`. Это означает, что пропущенные исключения из корутин, запущенных с помощью функции `launch` в области `viewModelScope`, приведут к общему сбою вашего приложения (подробнее об этом рассказывается в подразделе 18.3.1).

Определение этого обработчика довольно простое: он получает контекст корутины, а также перехваченное исключение в качестве параметров лямбды. Например, следующий обработчик исключений корутины регистрирует пропущенное исключение с пользовательским префиксом в консоли:

```
val exceptionHandler = CoroutineExceptionHandler { context, exception ->
    println("[ERROR] $exception")
}
```

Обработчик `CoroutineExceptionHandler` — это элемент `CoroutineContext`, который можно добавить в контекст корутины. В листинге 18.5 создается компонент `ComponentWithScope` для определения области видимости с объектом `SupervisorJob`, чтобы сделать компонент супервизором (такой способ создания области видимости корутин приводился в подразделе 15.1.2). Вам необходимо указать собственный обработчик исключений в качестве элемента контекста корутины. Для проверки поведения следует запустить корутину, которая выбрасывает исключение.

Листинг 18.5. Компонент с пользовательским обработчиком исключений корутины

```
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.seconds

class ComponentWithScope(dispatcher: CoroutineDispatcher = Dispatchers.Default) {
    private val exceptionHandler = CoroutineExceptionHandler { _, e ->
        println("[ERROR] ${e.message}")
    }

    private val scope = CoroutineScope(
        SupervisorJob() + dispatcher + exceptionHandler
    )

    fun action() = scope.launch {
        throw UnsupportedOperationException("Ouch!")
    }
}

fun main() = runBlocking {
    val supervisor = ComponentWithScope()
    supervisor.action()
    delay(1.seconds)
}
```

Результат работы кода показывает, что исключение обработано пользовательским обработчиком исключений:

```
[ERROR] Ouch!
```

ПРИМЕЧАНИЕ

Внутри системы прямые дочерние элементы супервизора технически сами обрабатывают исключения, передавая их в пользовательский обработчик исключений корутин в контексте, если он доступен, или в обработчик по умолчанию. Для простоты можно считать, что это супервизор для обработки исключений дочерних корутин. Обработчики исключений корутин вызываются только в том случае, если самая верхняя корутин в иерархии была запущена с помощью конструктора `launch` (подробнее об этом поговорим в подразделе 18.3.1).

Стоит еще раз повторить, что дочерние корутины, а также корутины в области видимости или запущенные из другой корутины делегируют обработку пропущенных исключений своему родителю. Тот, в свою очередь, делегирует обработку дальше, вплоть до корневого элемента иерархии. Соответственно, не существует такого понятия, как «промежуточный» обработчик `CoroutineExceptionHandler`. Некорневые обработчики, установленные в контекстах корутин, никогда не используются.

Эта логика показана в листинге 18.6. Корневая корутина создается с помощью API `GlobalScope.launch` с пометкой `delicate`, как уже было в подразделе 15.1.3, и в качестве части ее контекста предоставляется пользовательский обработчик исключений корутины. Кроме того, необходимо предоставить промежуточный обработчик исключений корутине `launch`. Когда этот будет выполнен, можно заметить, что единственный обработчик исключений корутин вызывается на вершине иерархии — промежуточный не используется.

Листинг 18.6. Вызывается только обработчик исключений на вершине иерархии корутин

```
import kotlin.coroutines.*

private val topLevelHandler = CoroutineExceptionHandler { _, e ->
    println("[TOP] ${e.message}")
}

private val intermediateHandler = CoroutineExceptionHandler { _, e ->
    println("[INTERMEDIATE] ${e.message}")
}

@OptIn(DelicateCoroutinesApi::class)
fun main() {
    GlobalScope.launch(topLevelHandler) {
        launch(intermediateHandler) {
            throw UnsupportedOperationException("Ouch!")
        }
    }
    Thread.sleep(1000)
}

// [TOP] Ouch!
```

Явное использование API
с пометкой `delicate`

Это происходит потому, что исключение все еще может быть передано дальше, в родительскую корутину (рис. 18.3).

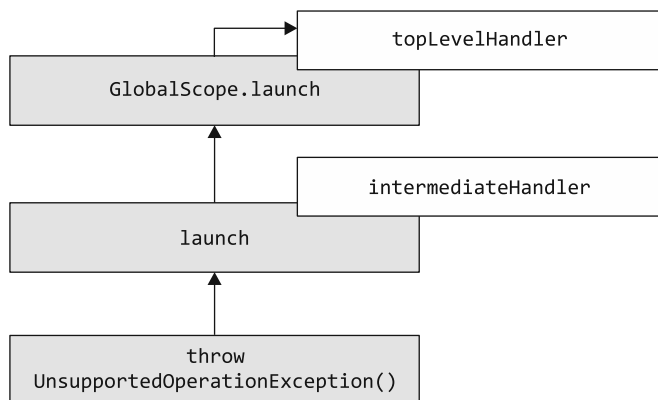


Рис. 18.3. В промежуточном вызове функции `launch` есть обработчик исключений в контексте корутины, но это не корневая корутина. Поэтому исключение продолжает распространяться по иерархии. И обработчик исключений вызывается только в корневой корутине (здесь `GlobalScope.launch`)

18.3.1. Различия при использовании `CoroutineExceptionHandler` с функциями `launch` и `async`

Рассматривая обработчик исключений `CoroutineExceptionHandler` (по умолчанию или пользовательский), важно отметить, что он вызывается только в том случае, если самая верхняя корутина в иерархии была создана с помощью функции `launch`. Если же самой верхней будет корутина `async`, то `CoroutineExceptionHandler` не вызывается.

Эти случаи показаны в следующих двух примерах. В первом корутина запускается с помощью `launch`, которая внутри запускает дополнительную корутину, используя функцию `async`. Повторим, что, когда самая верхняя корутина под супервизором запускается с помощью `launch`, вызывается обработчик `CoroutineExceptionHandler`:

```

import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.seconds

class ComponentWithScope(dispatcher: CoroutineDispatcher = Dispatchers.Default) {
    private val exceptionHandler = CoroutineExceptionHandler { _, e ->
        println("[ERROR] ${e.message}")
    }

    private val scope = CoroutineScope(SupervisorJob() + dispatcher +
        ➤ exceptionHandler)

    fun action() = scope.launch {
        async {
            throw UnsupportedOperationException("Ouch!")
        }
    }
}
  
```

```
fun main() = runBlocking {
    val supervisor = ComponentWithScope()
    supervisor.action()
    delay(1.seconds)
}
```

```
// [ERROR] Ouch!
```

Если изменить реализацию функции `action` так, чтобы внешняя корутина запускалась с помощью функции `async`, то обработчик исключений вызываться не будет (листинг 18.7).

Листинг 18.7. Корутина `async` выбрасывает исключение как непосредственный потомок супервизора

```
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.seconds

class ComponentWithScope(dispatcher: CoroutineDispatcher = Dispatchers.Default) {
    private val exceptionHandler = CoroutineExceptionHandler { _, e ->
        println("[ERROR] ${e.message}")
    }

    private val scope = CoroutineScope(SupervisorJob() + dispatcher + exceptionHandler)

    fun action() = scope.async {
        launch {
            throw UnsupportedOperationException("Ouch!")
        }
    }
}

fun main() = runBlocking {
    val supervisor = ComponentWithScope()
    supervisor.action()
    delay(1.seconds)
}

// Результат на экран не выводится
```

Теперь `async` — внешняя корутина

Причина в следующем: когда самая верхняя корутина запускается с помощью `async`, ответственность за обработку этого исключения путем вызова функции `await()` несет потребитель `Deferred`: обработчик исключений корутины может игнорировать это исключение, а вызывающий код, работающий с `Deferred`, может обернуть свой вызов `await` в блок `try-catch`. Важно отметить, что это не влияет на *отмену* корутин: если бы у переменной `scope` в предыдущем примере не была задана функция `SupervisorJob` в контексте, то это неперехваченное исключение все равно вызвало бы отмену всех других корутин, дочерних по отношению к `scope`.

Мы рассмотрели обработку ошибок в области корутин. Но как быть с ошибками при использовании потоков? Обсудим эту тему подробнее.

18.4. ОБРАБОТКА ОШИБОК В ПОТОКАХ

Как и обычные функции в Kotlin или функции приостановки, потоки тоже выбрасывают исключения. Пример показан в листинге 18.8. После сбора этот поток выдаст пять элементов (числа от 0 до 4), а затем выбросит пользовательское исключение `UnhappyFlowException`.

Листинг 18.8. Поток выбрасывает исключение после производства пяти чисел

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

class UnhappyFlowException: Exception()

val exceptionalFlow = flow {
    repeat(5) { number ->
        emit(number)
    }
    throw UnhappyFlowException()
}
```

Как правило, исключение выбрасывается из функции `collect`, если возникает в любой части потока: во время его создания, преобразования или сбора. Это означает, что можно окружить вызов `collect` блоком `try-catch` с ожидаемым поведением (конечно, не забывая при этом о специальных правилах, касающихся исключения `CancellationException`). Это не зависит от применения к потоку каких-либо промежуточных операторов. Пример показан во фрагменте кода ниже, где сначала поток `exceptionalFlow` преобразуется в `transformedFlow` с помощью функции `map`, а затем вызов `collect` оборачивается в блок `try-catch`:

```
fun main() = runBlocking {
    val transformedFlow = exceptionalFlow.map {
        it * 2
    }
    try {
        transformedFlow.collect {
            print("$it ")
        }
    } catch (u: UnhappyFlowException) {
        println("\nHandled: $u")
    }
    // 0 2 4 6 8
    // Handled: UnhappyFlowException
}
```

Префикс `Handled:` в имени исключения говорит о том, что исключение действительно было перехвачено. Однако есть и другой способ, особенно удобный, если вы создали длинные и более сложные конвейеры потоков, — оператор `catch`.

18.4.1. Обработка исключений восходящего потока с помощью оператора catch

Оператор `catch` является промежуточным и служит для обработки исключений в потоке. В лямбде, связанной с функцией, можно получить доступ к выброшенному исключению через параметр (как правило, со скрытым именем `it` по умолчанию).

Данный оператор представляет собой часть встроенного набора операторов, поэтому уже содержит информацию об исключениях отмены. Это значит, что блок кода, переданный в `catch`, не вызывается в случае отмены. Кроме того, оператор `catch` может самостоятельно выдавать значения. Это очень полезно, когда требуется превратить исключения в значения ошибок для передачи нисходящему потоку. В листинге 18.9 все исключения потока `exceptionalFlow` перехватываются и регистрируются, и в случае их возникновения коллектору передается значение ошибки `-1`.

Листинг 18.9. Использование оператора `catch` для выдачи значения по умолчанию в случае исключения

```
fun main() = runBlocking {
    exceptionalFlow
        .catch { cause ->
            println("\nHandled: $cause")
            emit(-1)
        }
        .collect {
            print("$it ")
        }
}
// 0 1 2 3 4
// Handled: UnhappyFlowException
// -1
```

Важно еще раз подчеркнуть, что оператор `catch` работает только в восходящем потоке — перехватывает исключения, возникшие до него в конвейере обработки потока. Этот процесс можно наблюдать во фрагменте кода ниже, где исключение, выброшенное в лямбде `onEach` после вызова оператора `catch`, не перехватывается:

```
fun main() = runBlocking {
    exceptionalFlow
        .map {
            it + 1
        }
        .catch { cause ->
            println("\nHandled $cause")
        }
        .onEach {
            throw UnhappyFlowException()
        }
        .collect()
}
// Exception in thread "main" UnhappyFlowException
```


ПРИМЕЧАНИЕ

Чтобы перехватить исключение из лямбды `collect`, можно обернуть ее вызов в блок `try-catch`. В качестве альтернативы можно переписать логику с помощью цепочки операторов `onEach`, `catch` и `collect` без лямбды. В таком случае нужно следить, чтобы оператор `catch` находился ниже по потоку от места, где может быть выброшено исключение (рис. 18.4). Данный оператор обрабатывает только исключения в восходящем потоке, поэтому вполне допустимо пробросить исключение из блока `catch` (или как-то преобразовать его), чтобы его обработал другой оператор `catch` в нисходящем потоке.

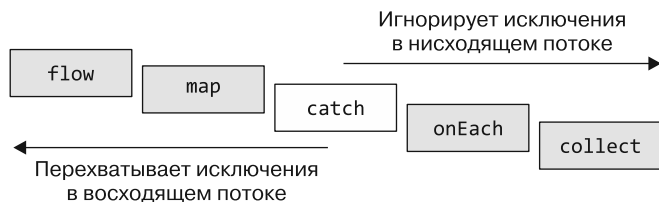


Рис. 18.4. Оператор `catch` перехватывает только возникающие в восходящем потоке исключения. Исключения ниже по потоку он не обрабатывает

18.4.2. Повтор сбора потока, если предикат является истинным: оператор `retry`

Если при обработке потока возникает исключение, то вместо того, чтобы завершать его с сообщением об ошибке, можно повторить операцию. Для этого используется удобный встроенный оператор `retry` — он, как и `catch`, ловит исключения в восходящем потоке. Затем можно с помощью связанной лямбды обработать исключение и вернуть логическое значение. Если лямбда возвращает значение `true`, то будет инициирована повторная попытка (пока количество повторных попыток не достигнет указанного максимума). При повторной попытке элементы восходящего потока собираются заново с самого начала и снова выполняются все промежуточные операции. Работа этого механизма показана в листинге 18.10. В нем смоделирован ненадежный поток, пытающийся выдать десять элементов. Каждый выброс может закончиться неудачей с вероятностью 10 %, поэтому для повторного сбора элементов потока применяется оператор `retry`.

Листинг 18.10. Повторная попытка сбора ненадежного потока

```
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.*
import kotlin.random.Random

class CommunicationException : Exception("Communication failed!")

val unreliableFlow = flow {
    println("Starting the flow!")
    repeat(10) { number ->
        if (Random.nextDouble() < 0.1) throw CommunicationException()
        emit(number)
    }
}
```

```
fun main() = runBlocking {
    unreliableFlow
        .retry(5) { cause ->
            println("\nHandled: $cause")
            cause is CommunicationException
        }
        .collect { number ->
            print("$number ")
        }
}
```

В данном примере используется элемент случайности, поэтому полученные вами результаты могут отличаться. Но после того, как фрагмент кода будет выполнен несколько раз, вы получите похожий результат. В нем будет собрано некоторое количество элементов, а затем будет выброшено исключение. Поэтому операция будет повторена еще несколько раз, прежде чем все десять элементов из потока будут собраны успешно:

```
Starting the flow!
0 1 2 3 4
Handled: CommunicationException: Communication failed!
Starting the flow!
0 1 2 3
Handled: CommunicationException: Communication failed!
Starting the flow!
0 1 2 3 4 5 6 7 8 9
```

Важно отметить, что повторная попытка сбора запускает и все вышестоящие операторы. И если какие-либо шаги выше по потоку выдают побочные эффекты, то они будут реализованы несколько раз. В этом случае следует убедиться, что либо они *идемпотентны* (то есть их многократное применение приводит к тому же результату, что и однократное), либо повторное выполнение обрабатывается корректно.

Вы получили представление об обработке ошибок в коде, созданном на основе корутин и потоков. Перейдем к тестированию — еще одной важной теме, позволяющей писать надежное программное обеспечение.

18.5. ТЕСТИРОВАНИЕ КОРУТИН И ПОТОКОВ

По сути, написание тестов для кода, созданного на основе корутин Kotlin, не отличается от написания обычных тестов. Для перехода в мир корутин из метода тестирования можно использовать специальный конструктор корутин `runTest`.

Сначала обсудим, почему функции `runBlocking` недостаточно и потребовался этот отдельный конструктор. Функция-конструктор `runBlocking`, описанная в подразделе 14.6.1, используется для установки взаимодействия между обычным и конкурентным кодом Kotlin. Ее применяют и для написания тестов функций приостановки и кода, созданного на основе корутин и потоков. Однако в этом подходе есть нюанс. При использовании `runBlocking` тесты выполняются в реальном времени. Это означает, что если в коде указаны задержки, то сначала они будут полностью выполнены, а после произойдет вычисление результатов.

В качестве примера рассмотрим тестирование системы объединения показаний датчиков. Чтобы не перегружать физическое устройство, код в системе запрашивает датчик только каждые 500 миллисекунд. Если написать модульный тест для этой части системы, используя функцию `runBlocking`, то этот тест должен будет ожидать завершения всех задержек. Даже если заменить базовую связь с датчиком тестовой заглушкой! Аналогично вы можете разрабатывать приложение для поисковых запросов, в котором перед отправкой каждого запроса система ждет несколько сотен миллисекунд, чтобы пользователь не отправил слишком много запросов за один раз. В модульном тесте этой функции поиска каждый симулированный запрос будет занимать несколько сотен миллисекунд, и в результате тестирование станет медленным.

По мере увеличения набора тестов это ненужное время выполнения может быстро увеличиться и повлиять на скорость тестирования всего приложения. А запуск медленных тестов станет обременительным для разработчиков. В итоге тестирование будет выполняться реже, а значит, и пользы от тестирования будет меньше. Корутины Kotlin предлагают решение этой проблемы — запуск тестов в *виртуальном времени*.

18.5.1. Быстрое выполнение тестов с помощью корутин: виртуальное время и диспетчер тестов

Вместо того чтобы выполнять тесты в реальном времени, ожидая все задержки, что замедляет их работу, корутины Kotlin дают возможность ускорить выполнение тестов за счет использования виртуального времени. При использовании этой концепции задержки автоматически ускоряются, что устраняет отмеченные ранее проблемы.

В листинге 18.11 функция `runTest` используется для запуска теста с виртуальным временем. Несмотря на объявленную задержку 20 секунд, этот тест выполняется практически мгновенно, всего за несколько миллисекунд (измеряется с помощью функции `System.currentTimeMillis()`).

Листинг 18.11. Запуск теста с виртуальным временем

```
import kotlinx.coroutines.*
import kotlinx.coroutines.test.*
import kotlin.test.*
import kotlin.time.Duration.Companion.seconds

class PlaygroundTest {
    @Test
    fun testDelay() = runTest {
        val startTime = System.currentTimeMillis()
        delay(20.seconds)
        println(System.currentTimeMillis() - startTime)
    }
}
```

Функция `runTest` использует специальный диспетчер и планировщик тестов, чтобы сделать это ускорение возможным. Вместо того чтобы ждать задержки в корутинах, их перематывают вперед.

ПРИМЕЧАНИЕ

Искусственные задержки автоматически ускоряются, поэтому функция `runTest` задает тайм-аут по умолчанию 60 (в реальном времени) секунд. При правильном использовании механизма виртуального времени этого должно быть достаточно. Однако иногда (например, при написании интеграционных тестов) вам может потребоваться больше времени. Для таких случаев можно указать параметр `timeout` при вызове функции `runTest`.

Диспетчер `runTest` однопоточный, как и `runBlocking`. Таким образом, все дочерние корутины по умолчанию выполняются конкурентно, но не параллельно с тестовым кодом (исключением служит дочерняя корутина, явно указывающая многопоточный диспетчер). Чтобы другие корутины могли выполнять свой код при совместном использовании однопоточного диспетчера, код должен предоставлять точки приостановки, и функция `runTest` не станет исключением (мы говорили об этом в подразделе 15.2.4). Этот нюанс особенно важно помнить при написании утверждений для тестов. Утверждение в данном тесте провалится, поскольку в теле функции `runTest` нет точек приостановки, а значит, инициированные корутины не смогут запуститься до утверждения:

```
@Test
fun testDelay() = runTest {
    var x = 0
    launch {
        x++
    }
    launch {
        x++
    }
    assertEquals(2, x)
}
```

Различные способы введения точек приостановки в код мы рассматривали в подразделе 14.6.2. Добавление функций `delay(50.milliseconds)`, `yield()` или другого вызова функции для приостановки позволяет пройти этот тест.

Кроме того, диспетчер тестов предоставляет возможность осуществлять более точный контроль над виртуальным временем с помощью специального планировщика `TestCoroutineScheduler`. Он относится к контексту корутины (о контекстах мы говорили в главе 15). Внутри блока функции-конструктора `runTest` вы получаете доступ к специализированной области видимости `TestScope`. Она предоставляет функциональность `TestCoroutineScheduler` (либо напрямую, с помощью функций-расширений, либо используя свойство `testScheduler`). Его основные функции:

- `runCurrent` — для запуска всех запланированных на данный момент корутин;
- `advanceUntilIdle` — для запуска всех корутин, запланированных к выполнению в любой момент.

Функция `delay` используется для простого перемещения виртуальных часов тестового диспетчера вперед (листинг 18.12). Часы виртуальные, так что задержка проходит мгновенно и код, выполнение которого запланировано как отложенное, будет выполнен. Получить представление о текущем времени виртуального диспетчера можно с помощью свойства `currentTime`.

Листинг 18.12. Переход по виртуальным часам вперед с помощью функции `delay`

```
@OptIn(ExperimentalCoroutinesApi::class)
@Test
fun testDelay() = runTest {
    var x = 0
    launch {
        delay(500.milliseconds)
        x++
    }
    launch {
        delay(1.second)
        x++
    }
    println(currentTime) // 0

    delay(600.milliseconds)
    assertEquals(1, x)
    println(currentTime) // 600

    delay(500.milliseconds)
    assertEquals(2, x)
    println(currentTime) // 1100
}
```

Запустить все запланированные на данный момент корутины можно с помощью функции `runCurrent`. Если в процессе запланировано немедленное выполнение новых корутин, то они выполняются напрямую. Чтобы запустить даже те корутины, выполнение которых запланировано как отложенное, можно применить функцию `advanceUntilIdle` (листинг 18.13).

Листинг 18.13. Запуск всех запланированных корутин с помощью `advanceUntilIdle`

```
@OptIn(ExperimentalCoroutinesApi::class)
@Test
fun testDelay() = runTest {
    var x = 0
    launch {
        x++
        launch {
            x++
        }
    }
    launch {
        delay(200.milliseconds)
        x++
    }
}
```

```

runCurrent()
assertEquals(2, x)
advanceUntilIdle()
assertEquals(3, x)
}

```

ПРИМЕЧАНИЕ

Другие диспетчеры, например `Dispatchers.Default`, не содержат информации о планировщике `TestCoroutineScheduler`. Влияние механизма виртуального времени не распространяется на диспетчеры — корутина, запущенная на нетестовом диспетчере, всегда будет ожидать задержек в полном объеме. Поэтому для того, чтобы обеспечить быстрое тестирование кода, предпочтительнее при его создании закладывать в него возможность изменения диспетчера (например, позволяя указывать его в качестве параметра). В подразделе 15.2.2 и разделе 18.3 мы уже приводили подобные примеры. Там мы создавали компоненты, имеющие собственный жизненный цикл, и в них указывался параметр конструктора `dispatcher`.

18.5.2. Тестирование потоков с помощью библиотеки *Turbine*

Тестирование кода с потоками по своей сути не отличается от тестирования другого приостанавливающегося кода, которое выполняется с помощью функции `runTest`. Например, вызвав функцию `toList`, можно сначала собрать все элементы конечного потока в коллекцию, а затем проверить, все ли ожидаемые элементы действительно находятся в ней (листинг 18.14).

Листинг 18.14. Сбор потока в список

```

val myFlow = flow {
    emit(1)
    emit(2)
    emit(3)
}

@Test
fun doTest() = runTest {
    val results = myFlow.toList()
    assertEquals(3, results.size)
}

```

В проектах код на основе потоков часто бывает более сложным, может включать бесконечные потоки или требовать более сложных инвариантов. Эти случаи поддерживаются библиотекой *Turbine* — она помогает писать тесты для потоков. *Turbine* — сторонняя библиотека, но многие разработчики Kotlin считают, что она необходима для написания тестов для API на основе потоков. Чтобы добавить библиотеку в свой проект, обратитесь к ее документации (<https://github.com/cashapp/turbine>). Когда библиотека будет добавлена, она введет новую функциональность и функции-расширения для потоков, которые помогут вам при написании тестов.

Основу функциональности *Turbine* составляет функция-расширение для потоков — `test`. Она запускает новую корутину и внутри нее собирает поток. В лямбде `test` можно использовать функции `awaitItem`, `awaitComplete` и `awaitError` совместно

с обычными утверждениями тестового фреймворка для определения и проверки инвариантов потоков (листинг 18.15). Вдобавок это гарантирует, что все выпущенные в поток элементы были правильно использованы вашим тестом.

Листинг 18.15. Тестирование потока с помощью библиотеки Turbine

```
@Test
fun doTest() = runTest {
    val results = myFlow.test {
        assertEquals(1, awaitItem())
        assertEquals(2, awaitItem())
        assertEquals(3, awaitItem())
        awaitComplete()
    }
}
```

Библиотека Turbine предоставляет дополнительную функциональность, которая позволяет тестировать несколько потоков в сочетании друг с другом, а также создавать отдельные объекты Turbine. Эти объекты применяются в ситуациях, когда имитации частей системы создаются для целей тестирования. Дополнительную информацию по этим более сложным темам можно найти в документации библиотеки.

На этом мы завершаем обсуждение обработки ошибок и тестирования кода, созданного с использованием корутин Kotlin. Теперь вы понимаете, как в них работает распространение и обработка ошибок. Вы узнали, как с помощью тестовой функциональности корутин обеспечить быстрое выполнение тестов кода и как библиотека Turbine упрощает написание тестов для потоков.

РЕЗЮМЕ

- Исключения, ограниченные одной корутиной, можно обрабатывать так же, как и в обычном коде. Необходимо проявлять особую осторожность, когда исключения выбрасываются через границы корутин.
- По умолчанию при возникновении перехваченного исключения в корутине родительская и все дочерние корутины отменяются. Это позволяет реализовать концепцию структурированной конкурентности.
- Если супервизор используется через `supervisorScope` или другую область видимости корутины и содержит объект `SupervisorJob`, то не отменяет другие дочерние корутины при сбое одной из них. Кроме того, он не распространяет пропущенные исключения дальше по иерархии корутин.
- Функция `await` пробрасывает исключение из корутины `async`.
- Супервизоры часто используются для долго выполняющихся частей приложений. Вдобавок они часто представляют собой встроенные части фреймворков, такие как `Application` в Ktor.
- Пропущенное исключение распространяется до тех пор, пока не встретится с супервизором или не достигнет вершины иерархии. В этот момент перехваченное исключение переходит в обработчик `CoroutineExceptionHandler`, который относится к контексту корутины. Если в контексте нет обработчика

исключений корутины, то перехваченное исключение попадает в общесистемный обработчик исключений.

- Общесистемный обработчик исключений по умолчанию для JVM и Android различается: в JVM он записывает трассировку стека в консоль ошибок, а в Android приводит к сбою приложения.
- Обработчик `CoroutineExceptionHandler` выступает в качестве последнего средства обработки исключений. Он не может перехватывать их, но позволяет настроить способ их регистрации. `CoroutineExceptionHandler` находится в контексте корневой корутины, на самом верху иерархии.
- `CoroutineExceptionHandler` вызывается только в том случае, если самая верхняя корутина была запущена с помощью конструктора `launch`. Если же ее запуск выполнялся с использованием конструктора `async`, то обработчик не вызывается. В таком случае за обработку исключения отвечает код, который ожидает тип `Deferred`.
- Обработка ошибок в потоках может быть выполнена путем обертывания функции `collect` в блок `try-catch` или с помощью специального оператора `catch`.
- Оператор `catch` обрабатывает только исключения восходящего потока, а исключения нисходящего игнорирует. Его даже можно использовать для того, чтобы пробросить исключения, которые будут обрабатываться позже в нисходящем потоке.
- Оператор `retry` в случае возникновения исключения позволяет запустить сбор потока с самого начала, и код получит шанс на восстановление.
- Использование виртуального времени с помощью функции `runTest` позволяет ускорить тестирование кода корутин. Любые задержки автоматически ускоряются.
- Планировщик `TestCoroutineScheduler` является частью области видимости `TestScope`, предоставленной функцией `runTest`, и отслеживает текущее виртуальное время. А для более точного управления тестами можно использовать функции `runCurrent` и `advanceUntilIdle`.
- Диспетчер тестов является однопоточным, а это значит, что необходимо вручную следить за тем, чтобы вновь запущенные корутины успевали выполняться до того, как будут вызваны утверждения.
- Библиотека `Turbine` дает возможность писать удобные тесты для кода на основе потоков. Основной ее API — функция-расширение `test`. Она собирает элементы из потока и позволяет использовать такие функции, как `awaitItem`, для проверки выбросов тестируемого потока.

Приложение А

Создание проектов Kotlin

В этом приложении рассказывается, как выполнять сборку кода Kotlin с помощью Gradle и Maven — двух наиболее популярных систем сборки для проектов Kotlin.

СБОРКА КОДА KOTLIN С ПОМОЩЬЮ GRADLE

Для сборки проектов Kotlin рекомендуется использовать систему Gradle. Де-факто она стала стандартной системой сборки для таких проектов. Gradle обладает гибкой моделью проектов и обеспечивает высокую производительность сборки благодаря поддержке инкрементных сборок, долговременных процессов сборки (Gradle Daemon) и других передовых технологий.

Gradle позволяет задавать сценарии сборки на языке Kotlin или Groovy. В данной системе в качестве стандартного и рекомендуемого подхода для новых проектов сценариев сборки выбран синтаксис Kotlin, и мы использовали его в нашей книге. Побочный эффект выражается в том, что и конфигурация сборки, и само приложение написаны на одном языке. Самый простой способ создать проект Gradle с поддержкой Kotlin — использовать встроенный мастер проектов (рис. А.1) в IntelliJ IDEA.

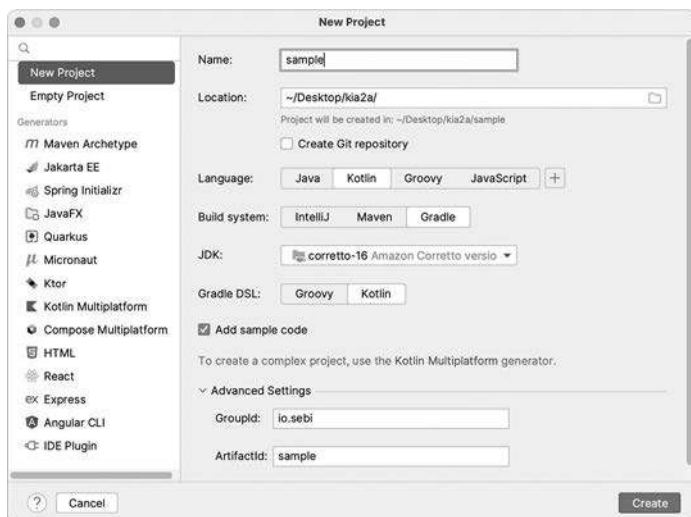


Рис. А.1. Мастер создания нового проекта в IntelliJ IDEA упрощает настройку проекта Kotlin

Он расположен в меню File ► New ► Project (Файл ► Создать ► Проект), или его можно вызвать, нажав кнопку New Project (Новый проект) на экране приветствия.

Стандартный сценарий сборки Gradle для создания проекта Kotlin выглядит следующим образом:

```
plugins {
    kotlin("jvm") version "YOUR_KOTLIN_VERSION"
    application
}

group = "org.example"
version = "1.0-SNAPSHOT"

repositories {
    mavenCentral()
}

dependencies {
    testImplementation(kotlin("test"))
}

tasks.test {
    useJUnitPlatform()
}

kotlin {
    jvmToolchain(20)
}

application {
    mainClass.set("MainKt")
}
```

Применение плагина Kotlin в Gradle и указание версии Kotlin

Стандартная библиотека Kotlin добавляется по умолчанию, указывать ее здесь не нужно

Библиотека тестирования Kotlin добавляется в качестве зависимости

По умолчанию скрипт ищет исходные файлы Kotlin в следующих местах: * `src/main/kotlin` для исходных файлов Kotlin, `src/main/java` для исходных файлов Kotlin и Java, * `src/test/kotlin` и `src/test/java` для соответствующих тестовых исходных файлов. При внедрении Kotlin в существующий проект особенно важно использовать единый исходный каталог, поскольку это позволяет снизить количество затруднений, возникающих при преобразовании Java-файлов в Kotlin.

Если используется рефлексия Kotlin (см. главу 12), то нужно добавить еще одну зависимость — библиотеку рефлексии Kotlin. Для этого добавьте в раздел `dependencies` вашего скрипта сборки Gradle следующий код:

```
implementation(kotlin("reflect"))
```

Кроме того, при использовании корутин Kotlin (их изучению мы посвятили третью часть книги) необходимо добавить соответствующую зависимость:

```
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")
// если планируется работа под ОС Android, добавьте:
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
```

Последнюю версию библиотеки корутин необходимо проверить в документации (<https://github.com/Kotlin/kotlinx.coroutines>).

СОЗДАНИЕ ПРОЕКТОВ С ОБРАБОТКОЙ АННОТАЦИЙ

Некоторые фреймворки Kotlin используют *обработку символов* для интроспекции и генерации кода во время компиляции в проекте. Чтобы это осуществить, такие проекты, как `koin` (<https://insert-koin.io/>), `kotlin-inject` (<https://github.com/evant/kotlin-inject>) и `Ktorfit` (<https://foso.github.io/Ktorfit/>), используют API *Kotlin Symbol Processing* (KSP). Чтобы добавить поддержку KSP в свой проект, следуйте инструкциям по установке со страницы проекта KSP (<https://github.com/google/ksp>).

Если работа в проекте выполняется с Java-фреймворками при обработке аннотаций для генерации кода во время компиляции, то можно использовать `kapt`. Это инструмент обработки аннотаций Kotlin, обратно совместимый с Java-библиотеками. Его необходимо добавить в скрипт сборки, используя следующую строку в блоке `plugins`:

```
kotlin("kapt") version "YOUR_KOTLIN_VERSION"
```

Если у вас есть существующий Java-проект с обработкой аннотаций и вы внедряете в него Kotlin, то необходимо удалить существующую конфигурацию инструмента `apt`. Механизм `kapt` обрабатывает классы Java и Kotlin, и наличие двух отдельных инструментов для обработки аннотаций было бы излишним. Чтобы настроить зависимости для обработки аннотаций, используйте конфигурацию зависимостей `kapt`:

```
dependencies {
    implementation("com.google.dagger:dagger:2.46.1")
    kapt("com.google.dagger:dagger-compiler:2.46.1")
}
```

При использовании обработчиков аннотаций для источника `androidTest` или `test` соответствующие конфигурации `kapt` будут называться `kaptAndroidTest` и `kaptTest`.

СОЗДАНИЕ ПРОЕКТОВ KOTLIN С ПОМОЩЬЮ MAVEN

Если вы предпочитаете выполнять сборку проектов с помощью Maven, то Kotlin поддерживает такую возможность. Самый простой способ создать Kotlin-проект в Maven — использовать архетип `org.jetbrains.kotlin:kotlin-archetype-jvm`. Можно добавить поддержку Kotlin в существующие проекты Maven. Для этого необходимо перейти в меню **Tools** ▶ **Kotlin** ▶ **Configure Kotlin in Project** (Инструменты ▶ Kotlin ▶ Конфигурация Kotlin в проекте) в плагине Kotlin IntelliJ IDEA.

Чтобы добавить поддержку Maven в проект Kotlin вручную, выполните следующие действия.

1. Добавьте зависимость от стандартной библиотеки Kotlin (идентификатор группы `org.jetbrains.kotlin`, идентификатор артефакта `kotlin-stdlib`).

2. Добавьте плагин Kotlin Maven (идентификатор группы `org.jetbrains.kotlin`, идентификатор артефакта `kotlin-maven-plugin`) и настройте его выполнение на этапах `compile` и `testcompile`.
3. Настройте каталоги исходного кода, если предпочитаете хранить код Kotlin в корневом каталоге отдельно от исходного кода Java.

Чтобы сэкономить место в книге, мы не будем приводить здесь полные примеры файлов `pom.xml`, но вы можете найти их в онлайн-документации по адресу <https://kotlinlang.org/docs/maven.html>.

В смешанном проекте Java/Kotlin необходимо настроить плагин Kotlin таким образом, чтобы он запускался раньше плагина Java. Это необходимо, поскольку плагин Kotlin может анализировать исходные файлы Java, в то время как плагин Java может читать только файлы с расширением `.class`. Поэтому файлы Kotlin необходимо скомпилировать в файл `.class` до того, как плагин Java будет запущен. Пример процесса настройки можно найти на сайте <http://mng.bz/v8Op>.

Приложение Б

Документирование кода Kotlin

В этом приложении рассказывается о написании комментариев к документации для кода Kotlin и генерации документации API для модулей Kotlin.

НАПИСАНИЕ КОММЕНТАРИЕВ К ДОКУМЕНТАЦИИ KOTLIN

Формат для написания комментариев к объявлениям Kotlin называется *KDoc*. Комментарии KDoc начинаются с набора символов `/**`, в них применяются теги, начинающиеся с символа `@` и предназначенные для документирования определенных частей объявления (возможно, вы привыкли к этому в Javadoc). KDoc использует диалект Markdown (<https://daringfireball.net/projects/markdown>) в качестве синтаксиса для написания комментариев. Чтобы облегчить написание комментариев к документации, KDoc поддерживает несколько дополнительных соглашений для ссылок на элементы документации, такие как параметры функций.

В листинге Б.1 приведен простой пример комментария KDoc для функции.

Листинг Б.1. Использование комментария KDoc

```
/**  
 * Вычисление суммы двух чисел, [a] и [b]  
 */  
fun sum(a: Int, b: Int) = a + b
```

Чтобы сослаться на объявления из комментария KDoc, их имена необходимо заключить в квадратные скобки. В примере этот синтаксис применяется для ссылки на параметры документируемой функции, но с его помощью вы можете сослаться и на другие объявления. Если объявление, на которое нужно сослаться, импортировано в код с комментариями KDoc, то можно использовать имя объявления напрямую. В противном случае требуется привести полное имя. Если необходимо указать пользовательскую метку для ссылки, то применяются две пары квадратных скобок и метка помещается в первую пару, а имя объявления — во вторую: `[an example]` `[com.mycompany.SomethingTest.simple]`.

В листинге Б.2 приведен несколько более сложный пример, демонстрирующий использование тегов в комментариях.

Листинг Б.2. Использование тегов в комментариях

```
/**
 * Выполнение сложных операций.
 *
 * @param remote, если значение true, операция выполняется удаленно
 * @return Результат выполнения операции
 * @throws IOException, если удаленное соединение нарушено
 */
fun somethingComplicated(remote: Boolean): ComplicatedResult { /* ... */ }
```

Документирует параметр

Документирует возвращаемое значение

Документирует возможное исключение

Помимо прочего, КДос поддерживает и другие теги (табл. Б.1).

Таблица Б.1. Теги и их назначение

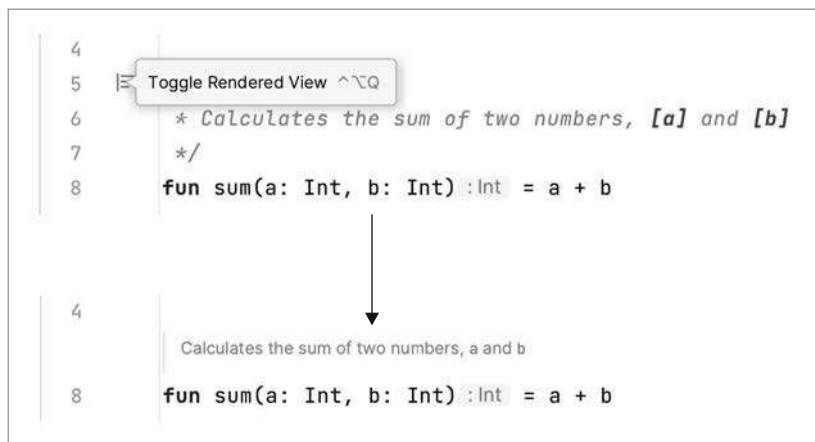
Название тега	Назначение
@param <i>parameterName</i> , @param[<i>parameterName</i>]	Документирование параметра значения функции или параметра типа обобщенной конструкции
@return	Документирование возвращаемого значения функции
@constructor	Документирование первичного конструктора класса
@receiver	Документирование приемника функции-расширения или свойства-расширения
@property <i>propertyName</i>	Документирование свойства класса, если оно объявлено в первичном конструкторе
@throws <i>ClassName</i> , @exception <i>ClassName</i>	Документирование исключений, которые могут быть выброшены функцией. Значение тега — это полное имя включаемого метода
@see <i>otherSymbol</i>	Добавление ссылки на другой класс или функции в блок документации «См. также»
@author	Указание автора
@since	Указание версии представления документированного элемента
@suppress	Удаление этого объявления из создаваемой документации

Полный список поддерживаемых тегов можно найти по адресу <https://kotlinlang.org/docs/kotlin-doc.html>.

Стоит отметить, что команда Kotlin предпочитает свой стиль оформления. Он заключается в документировании параметров и возвращаемого значения функции непосредственно в тексте комментария документации, что было показано в листинге Б.1. Использование тегов, приведенное в листинге Б.2, рекомендуется только в тех случаях, когда параметр или возвращаемое значение имеют сложную семантику и должны быть четко отделены от основного текста документации.

ВИЗУАЛИЗАЦИЯ ДОКУМЕНТАЦИИ В INTELLIJ IDEA И ANDROID STUDIO

Помимо подсветки синтаксиса и навигации по символам в комментариях KDoc, IntelliJ IDEA и Android Studio предоставляют возможность визуализации. Включить ее можно, наведя указатель мыши на номера строк рядом с комментарием KDoc и выбрав пункт **Toggle Rendered View** (Переключить визуализацию). Внешний вид комментариев изменится на вариативный шрифт, и будут отображены ссылки и гиперссылки. Эта возможность особенно удобна в тех ситуациях, когда необходимо просмотреть исходные файлы библиотек и другого кода, доступного только для чтения, поскольку различия между документацией и реализацией становятся еще более очевидными.



ИЗ JAVADOC В KDOC

Помимо разницы в синтаксисе — Markdown в KDoc и HTML в Javadoc, — есть еще несколько особенностей, на которые стоит обратить внимание, если вы привыкли писать Javadoc. Это поможет облегчить переход с одного языка на другой.

Некоторые теги Javadoc не поддерживаются в KDoc:

- `@deprecated` заменен на аннотацию `@Deprecated`;
- `@inheritdoc` не поддерживается, поскольку в Kotlin комментарии к документации всегда автоматически наследуются при переопределении объявлений;
- `@code`, `@literal` и `@link` заменены соответствующим форматированием в Markdown.

СОЗДАНИЕ ДОКУМЕНТАЦИИ ПО API

Инструмент по созданию документации для Kotlin называется Dokka (<https://github.com/kotlin/dokka>). Как и Kotlin, Dokka полностью поддерживает кросс-языковые проекты на Java/Kotlin. Он может читать комментарии Javadoc в коде Java и KDoc в коде Kotlin и генерировать документацию, касающуюся всего API модуля, независимо от того, на каком языке в нем написан каждый класс. Dokka поддерживает

несколько форматов вывода, в том числе обычный HTML и HTML в стиле Javadoc (с использованием синтаксиса Java для всех объявлений и демонстрацией того, как можно получить доступ к API из Java).

Dokka можно запустить из командной строки или как часть скрипта сборки Gradle либо Maven. Рекомендуемый способ запуска — добавить его плагин в скрипт сборки Gradle вашего модуля. Схемы версионирования Dokka и Kotlin совпадают. Минимально необходимая конфигурация Dokka в скрипте сборки Gradle выглядит следующим образом:

```
plugins {  
    id("org.jetbrains.dokka") version "YOUR_KOTLIN_VERSION"  
}
```

Информацию о том, как генерировать документацию для модуля в поддерживаемых форматах, а также об установке дополнительных параметров генерации можно найти в документации Dokka (<https://kotlinlang.org/docs/dokka-introduction.html>). В ней же показано, как запустить Dokka в качестве самостоятельного инструмента или интегрировать в скрипты сборки Maven.

Приложение В

Экосистема Kotlin

За годы существования Kotlin сформировалась обширная экосистема библиотек, фреймворков и инструментов. В текущем приложении мы дадим вам несколько советов по изучению данной экосистемы. Конечно, она продолжает развиваться и расширяться, поэтому наша книга не может служить идеальным средством получения актуальной информации о доступных библиотеках — для этого лучше подходят онлайн-ресурсы. Благодаря тематической странице Kotlin на GitHub (<https://github.com/topics/kotlin>) и спискам, составленным сообществом, например Kotlin is Awesome (<https://kotlin.link/>), вы можете получить более полное представление о вселенной Kotlin.

К тому же стоит еще раз напомнить, что Kotlin полностью совместим со всей экосистемой библиотек Java. Если не получается найти подходящую библиотеку Kotlin, то вполне можно выбрать библиотеку Java. Некоторые из них даже предлагают расширения для Kotlin, обеспечивающие более чистые и идиоматичные API.

Kotlin — язык программирования общего назначения, поэтому его экосистема охватывает множество тем и областей применения. К числу основных категорий можно отнести:

- тестирование;
- покрытие кода;
- бенчмаркинг;
- документирование;
- внедрение зависимостей;
- сетевые приложения;
- сериализацию;
- доступ к базам данных;
- конкурентное программирование;
- разработку пользовательского интерфейса.

Далее мы дадим краткое описание каждой из этих категорий.

ТЕСТИРОВАНИЕ

В дополнение к фреймворкам JUnit и TestNG, хорошо зарекомендовавшим себя при работе с Kotlin, существует Kotest (<https://github.com/kotest/kotest>) — гибкий тестовый фреймворк с поддержкой нескольких различных макетов для написания тестов. Вы уже сталкивались с ним в подразделе 13.4.1, когда мы обсуждали DSL.

Библиотека MockK (<https://github.com/mockk/mockk>) предоставляет DSL, который можно использовать для описания упрощенных реализаций частей вашей системы в целях тестирования, — этот прием называется *mocking* (иногда его называют *мокированием*). Если вы уже знакомы с mocking-фреймворком Mockito, то можете использовать Mockito-Kotlin (<https://github.com/mockito/mockito-kotlin>). Этот фреймворк предоставляет вспомогательные функции, упрощающие работу с ним из Kotlin.

ПОКРЫТИЕ КОДА

Когда требуется понять, какая часть вашей кодовой базы на самом деле тестируется, обычно применяется подход *покрытия кода*. Он измеряет, сколько строк кода выполняется при запуске автоматизированных тестов по отношению к общему количеству строк в вашей программе. Библиотека Kover (<https://github.com/Kotlin/kotlinx-kover>) позволяет использовать в проекте Kotlin агенты покрытия кода, такие как JaCoCo. Она генерирует подробные отчеты, которые дают возможность понять, каково количество строк, покрытых автоматизированными тестами, какие подсистемы в кодовой базе можно подвергнуть дополнительным тестам, а какие части уже получили достаточное покрытие.

БЕНЧМАРКИНГ

Для детальной проверки производительности кода разработан набор инструментов `kotlinx.benchmark` (<https://github.com/Kotlin/kotlinx-benchmark>). Он предоставляет удобный способ многократного запуска одного и того же кода. Библиотека автоматически учитывает факторы, способные исказить результаты измерений, такие как время прогрева и возможные оптимизации во время работы. Вдобавок она представляет подробные отчеты о производительности кода.

ДОКУМЕНТИРОВАНИЕ

Как вы уже знаете, инструмент Dokka (<https://github.com/Kotlin/dokka>) де-факто стал стандартным механизмом документирования для проектов Kotlin. Вы познакомились с ним в приложении Б, где мы рассказывали о документировании кода Kotlin. Dokka может генерировать документацию проектов на чистом Kotlin, смеси Kotlin и Java или Kotlin Multiplatform и поддерживает различные форматы вывода.

ВНЕДРЕНИЕ ЗАВИСИМОСТЕЙ

Внедрение зависимостей (dependency injection) и инверсия управления (inversion of control) — два основных паттерна программирования, используемых в объектно-ориентированных системах. Koin (<https://insert-koin.io/>) и Kodein (<https://github.com/kosi-libs/Kodein>) — фреймворки для внедрения зависимостей, которые предоставляют DSL Kotlin для настройки зависимостей. Такие фреймворки внедрения зависимостей, как Spring, Guice и Dagger Hilt, тоже хорошо работают с Kotlin.

СЕТЕВЫЕ ПРИЛОЖЕНИЯ — СЕРВЕРЫ И КЛИЕНТЫ

Многие современные приложения должны взаимодействовать с другими системами через сеть. Компания JetBrains разработала *Ktor* (<https://ktor.io/>) — современный и простой фреймворк для создания асинхронных клиентских и серверных приложений. В нем полностью реализованы идиомы языка Kotlin, а в качестве базового механизма конкурентности были использованы корутины. Ktor расширяется за счет системы плагинов, позволяющей легко интегрировать его с другими библиотеками — например, для шаблонизации, сериализации, аутентификации и авторизации или мониторинга. Такие проекты, как Ktorfit (<https://github.com/Foso/Ktorfit>), расширяют Ktor парадигмами, особенно популярными в среде разработки Android, например API в стиле Retrofit (<http://square.github.io/retrofit>).

Набор инструментов http4k (<https://http4k.org/>) предоставляет функциональные абстракции для работы с HTTP-сервисами как в качестве сервера, так и в качестве клиента. Кроме того, можно использовать уже существующие фреймворки, такие как Spring, Quarkus или Vert.x и др. В Spring по умолчанию есть поддержка Kotlin и расширения (<https://spring.io/guides/tutorials/spring-boot-kotlin>). Quarkus предоставляет поддержку первоклассных элементов с использованием Kotlin (<https://quarkus.io/guides/kotlin>), как и Vert.x (<https://vertx.io/docs/vertx-core/kotlin/>).

СЕРИАЛИЗАЦИЯ

В главе 12 вы узнали, как создать собственную библиотеку сериализации JSON. Но для приложений в среде эксплуатации обычно применяют уже готовую и оптимизированную библиотеку. Kotlin Serialization (<https://github.com/Kotlin/kotlinx.serialization>) разработана компанией JetBrains и предоставляет плагин компилятора для генерации высокопроизводительного кода сериализации и десериализации. Кроме того, она поддерживает и другие форматы сериализации, помимо JSON, такие как Protobuf, CBOR и др., распространенные в сообществе разработчиков. В качестве альтернативы предлагается Moshi (<https://github.com/square/moshi/>) — поддерживаемая сообществом библиотека, которая предоставляет функции синтаксического анализа и кодирования JSON с чистыми API Kotlin.

ДОСТУП К БАЗАМ ДАННЫХ

Несколько специфических для Kotlin вариантов помогут решить проблемы с доступом к базам данных. В компании JetBrains создается Exposed (<https://github.com/jetbrains/Exposed>) — фреймворк для генерации SQL (мы несколько раз упоминали его в этой книге). Библиотека SQLDelight (<https://github.com/cashapp/sqldelight>) преобразует типичный подход объектно-реляционного отображения (Object-Relational Mapping, ORM), генерируя безопасные API Kotlin из SQL-запросов. Кроме того, решать задачи доступа к базе данных можно с помощью традиционных вариантов Java, таких как Hibernate (<https://hibernate.org/>) или jOOQ (<https://www.jooq.org/>).

КОНКУРЕНТНОЕ ПРОГРАММИРОВАНИЕ

Ранее мы представили `kotlinx.coroutines` (<https://github.com/Kotlin/kotlinx.coroutines>) — основную библиотеку поддержки конкурентности в Kotlin. Существует еще несколько библиотек с функциональностью, удобной для конкурентного программирования.

Библиотека `AtomicFU` (<https://github.com/Kotlin/kotlinx-atomicfu>) обеспечивает атомарные ссылки и операции в Kotlin. Ее API разработан как идиоматический Kotlin, и библиотека вдобавок работает для целевых сред Kotlin, не связанных с JVM, где примитивы Java могут быть недоступны.

Библиотека `kotlinx.collections.immutable` (<https://github.com/Kotlin/kotlinx.collections.immutable>) предоставляет *неизменяемые постоянные коллекции* для Kotlin. Это другой тип реализации интерфейса `Collection`, позволяющий создавать модифицированные копии неизменяемых коллекций без ущерба для производительности, связанного с выделением полной новой коллекции. Это коллекции общего назначения, но они особенно важны в контексте конкурентного программирования.

`Turbine` (<https://github.com/cashapp/turbine>) — библиотека тестирования для потоков. Она предоставляет удобные функции, позволяющие тестировать поведение API на основе потоков, автоматически вызывая код проверки для отдельных или нескольких потоков.

РАЗРАБОТКА ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

Для разработки пользовательских интерфейсов JetBrains создает `Compose Multiplatform` (<https://jb.gg/compose>) — современный декларативный фреймворк пользовательского интерфейса на основе инструмента Jetpack Compose компании Google. Данный фреймворк позволяет разрабатывать красивые настольные приложения на JVM с помощью DSL Kotlin. И особое внимание в нем уделяется многократно используемым компонентам пользовательского интерфейса. Кроме того, фреймворк дает возможность создавать общие мобильные пользовательские интерфейсы, работающие на Android, iOS и других платформах.

Алфавитный указатель

А

Анонимная функция 326
Анонимный объект 160
Аргумент типа 329

Б

Библиотека среды выполнения Kotlin 49
Блок инициализатора 133

В

Вариативность в точке использования 356
Вариативность в точке объявления 356
Вариативность точки
 использования 329
Вариативность точки
 объявления 328
Виртуальное время 539
Виртуальный поток 434
Внутренний DSL 403
Восходящий поток 507
Встроенный класс 161
Выведение типов 32
Выражение 55

Г

Горячий поток 482

Д

Делегат 289
Делегирование 289
Десериализация 373
Дженерики 328
Диапазон 80
Диспетчер 448
Диспетчер тестов 540

Ж

Жесткое ключевое слово 69

З

Запечатанный класс 131
Записи 149
Захват переменных лямбдой 175

И

Идемпотентность 538
Идиоматичный Kotlin 53
Инвариантный класс 349
Инструкция 55
Интерфейс SAM 180
Инфиксный вызов 108
Исчерпывающая конструкция 72
Итерации 79

К

Канальный поток 491
Карта (map) 91
Ковариантный класс 349
Коллектор потока 484
Коллекции, доступные только
 для чтения 243
Коммутативность операторов 271
Компаратор 354
Конечный оператор потока 506
Конкурентная декомпозиция работы 460
Конкурентность 432
Константа времени компиляции 370
Конструктор корутин 441
Конструктор HTML 410
Конструктор SAM 181
Контекстные приемники 426
Контрвариантность 353
Корутины 433

Л

Лексемы значений 390
Лексер 390

Лексический анализатор 390
Локальная функция 115
Локальный return 324
Лямбда 165
Лямбда-выражение 165
Лямбда с приемником 185

М

Массив 263
Массив с переменным количеством аргументов 107
Метааннотации 376
Метаданные 368
Метка для операторов 80
Метод 66
Методы доступа 63
Многопоточные диспетчеры 452
Множество 73
Модификаторы видимости 126
Модуль 126
Мягкое ключевое слово 69

Н

Наблюдаемость 292
Неизменяемость 33, 166
Нелокальный return 324
Нисходящий поток 507

О

Обобщенная функция 330
Общий поток 492
Объединение на основе равенства 501
Объект-компаньон 154
Объектные выражения 159
Объект-приемник 100, 408
Объявление деструктуризации 108, 284
Объявление с отложенной инициализацией 234
Объявления объектов 152
Ограничения типовых параметров 333
Оператор безопасного вызова 223
Оператор индексированного доступа 278
Оператор объединения с null 225
Оператор тождества или равенства ссылок 276
Оператор spread 107

Открытый диапазон 281
Отложенная инициализация 234, 290
Отмена выполнения 467

П

Параллелизм 432
Параметры вещественного типа 328
Парсер 390
Первичный конструктор 133
Перегрузка бинарных операторов 269
Перегрузка оператора сравнения 275
Перегрузка унарного оператора 274
Платформенный тип 239
Подкласс 349
Подписчик потока 492
Подтип 348
Позиционность деструктуризации 288
Покрытие кода 554
Последовательность 211
Поток 433, 480
Поток состояний 492
Поток UI 449
Предикат 199
Предметно-ориентированный язык 401
Примитивный класс 163
Примитивный тип 244
Проблема хрупкого базового класса 123
Проверяемое исключение 86
Прогрессия 80
Проекция типа 358
Промежуточная операция 212
Промежуточный оператор потока 506
Псевдоним типа 364
Пул потоков 448
Пустая истина 200

Р

Работа с коллекциями 192
Расширения-члены 424
Рефлексия 379

С

Свойство-расширение 105
Свойство члена класса 383
Сериализация 373

Синтаксис «звездной» проекции 360
Синтаксический анализатор 390
Синтетический тип 307
Словарь 91
Соглашения 268
Составной оператор присваивания 273
Ссылка на конструктор 177
Ссылка на член 176
Ссылка с возможностью
переназначения 59
Ссылка только для чтения 58
Ссылочный тип 244
Стандартная библиотека Kotlin 50
Стирание типов 337
Строковый шаблон 60
Структура/грамматика DSL 404
Структурированная
конкурентность 433, 457
Супертип 348
Сырой тип 330

Т

Тело выражения 56
Теневое поле 140
Теневое свойство 291
Терминальная операция 212
Типовые параметры 329
Тип приемника 99, 408
Типы функций 302
Точка приостановки 444

У

Умное преобразование 76
Утверждение «не null» 228

Ф

Фабричный метод 155
Функции первого класса 33

Функции приостановки 433
Функциональный интерфейс 180
Функция высшего порядка 302
Функция контекста 232
Функция области видимости 232
Функция первого класса 166
Функция-расширение 99
Функция с блочным телом 56
Функция-член 66

Х

Холодный поток 482

Ц

Целевая точка использования 370

Ч

Чистая функция 166
Чистый API 400

Я

Явный режим API 126

А

API команд и запросов 404

І

Intrinsic-функция компилятора 455

М

Mocking-тестирование 554

Н

Null-безопасность 218

*Себастьян Айгнер, Роман Елизаров,
Светлана Исакова, Дмитрий Жемеров*

Kotlin в действии

2-е издание

Перевел с английского Яков Голуб

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Руководитель проекта	<i>А. Питиримов</i>
Ведущий редактор	<i>Н. Гринчик</i>
Литературный редактор	<i>Н. Хлебина</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>О. Андриевич, Е. Павлович</i>
Верстка	<i>Г. Блинов</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга». Место нахождения и фактический адрес:
194044, Россия, г. Санкт-Петербург, Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 03.2025. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные
профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 26.02.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 45,150. Тираж 1000. Заказ 0000.